

U.N.IN.A



Algoritmi e Strutture Dati I

Pasquale Miranda
Vincenzo Mennillo
Giuseppe Di Martino

1st Edition
2024

Versione 01.00 - 13 luglio 2024

Il seguente testo è liberamente disponibile per il download nella community U.N.IN.A. telegram ufficiale della facoltà di Informatica dell'Università Federico II di Napoli.

[Clicca qui per entrare nella community U.N.IN.A.!](#)

Oppure usa il seguente QR code!



Il seguente testo è una sintesi del corso di Algoritmi e Strutture Dati I, tenuto dal Professore Fabio Mogavero, nell'anno accademico 2023-2024, nella facoltà di Informatica dell'Università degli studi di Napoli Federico II.

Si tratta di un testo che ha avuto una genesi alquanto particolare.

È nato inizialmente come un comune file di appunti schematico; dopo poco, però, ci siamo resi conto che data la vastità e la complessità degli argomenti trattati, e soprattutto considerata l'importanza della materia, bisognava tentare di scrivere un testo quanto più dettagliato e approfondito possibile.

Speriamo di esserci riusciti.

Pertanto, vista la dimensione del testo, potrebbero esserci errori grammaticali, di punteggiatura, typo vari, e (si spera di no) errori logici o concettuali. Preghiamo, quindi, i lettori di segnalare qualsiasi problema agli autori mediante la community U.N.IN.A.

Sono doverosi alcuni ringraziamenti.

Ringraziamo i colleghi (in ordine alfabetico), che hanno contribuito donando i loro appunti personali dell'intero corso, per lo spiccato spirito di condivisione.

- Alfredo Volpe
- Angelo Paoletta
- Vittorio Emanuele Testa

Senza il loro contributo tutto ciò sarebbe risultato più difficile, se non impossibile.

In ultimo, non per importanza chiaramente ma nell'ottica del *dulcis in fundo*, ringraziamo il Professore Fabio Mogavero che con le sue dettagliate ed appassionanti spiegazioni ha contribuito indirettamente, ma in modo fondamentale, alla stesura del corrente testo. Se non ci avesse appassionato alla materia non avremmo mai pensato di scrivere un testo di tale portata.

Concludiamo nella speranza che la lettura del testo possa contribuire a stimolare la curiosità degli studenti e a farli appassionare alla materia.

In fede, gli autori.

Pasquale Miranda

Vincello Mennillo

Giuseppe Di Martino

"Remember... all I'm offering is the truth. Nothing more"

– Morpheus (The Matrix)

Indice

I	Introduzione al Corso	1
I.1	Testi Consigliati	2
I.2	Descrizione del Corso	3
I.3	Prerequisiti	4
I.4	Esame	4
I.5	Argomenti del Corso	5
II	Registro delle Lezioni 2023/2024	7
1	Fondamenti di Analisi degli Algoritmi	14
1.1	Definizione Informale di Algoritmo	14
1.2	Definizione di Struttura Dati	17
1.3	Definizione Formale di Algoritmo	18
1.4	Modello Computazionale	19
1.4.1	Macchina di Turing	20
1.4.2	Modello RAM	27
1.5	Accenno alle Classi di Complessità	28
1.5.1	Definizione di O-grande	28
1.5.2	Definizione di Omega	28
1.5.3	Definizione di Theta	28
1.5.4	Proprietà	28
1.6	Pseudo-Codice	29

2	Esempi di Algoritmi e Relativa Analisi	32
2.1	Sequenza di Fibonacci	32
2.1.1	Problema	32
2.1.2	Algoritmo: Fibonacci Ricorsivo	33
2.1.3	Algoritmo: Fibonacci Iterativo v.01	36
2.1.4	Excursus sugli Algoritmi	38
2.1.5	Algoritmo: Fibonacci Iterativo v.02	40
2.2	Conteggio Coppie	42
2.2.1	Problema	42
2.2.2	Algoritmo: Count v.01	43
2.2.3	Algoritmo: Count v.02	44
2.2.4	Algoritmo: Count v.03	46
2.2.5	Algoritmo: Count v.04	48
2.2.6	Conclusioni	48
2.3	Max Subsequence Sum (MSS)	49
2.3.1	Problema	49
2.3.2	Algoritmo: MSS v.01	50
2.3.3	Algoritmo: MSS v.02	52
2.3.4	Algoritmo: MSS v.03	54
2.3.5	Conclusioni	60
3	Strutture Dati Lineari	61
3.1	Introduzione	61
3.2	Caratterizzazione generale di una Struttura Dati	62
3.2.1	Definizione di Struttura Dati Statica	63
3.2.2	Definizione di Struttura Dati Dinamica	63
3.2.3	Definizione alternativa di Struttura Dati Statica	63
3.3	Funzioni generali relative alle Strutture Dati	64
3.4	Caratterizzazione di Strutture Dati Lineari	67
3.4.1	Array	67

3.4.2	Array Ordinati	68
3.4.3	Liste Concatenate	69
3.4.4	Liste Concatenate Ordinate	70
3.4.5	Stack	71
3.4.6	Queue	72
3.5	Algoritmi sugli Array	73
3.5.1	Introduzione	73
3.5.2	Algoritmo: Ricerca in un Array con Ripetizioni	74
3.5.3	Algoritmo: Ricerca in un Array senza Ripetizioni	75
3.5.4	Algoritmo: Ricerca Binaria Ricorsiva in un Array Ordinato	76
3.5.5	Algoritmo: Ricerca Binaria Iterativa in un Array Ordinato	79
3.5.6	Algoritmo: Ricerca del Minimo in un Array non ordinato	85
3.5.7	Algoritmo: Ricerca del Minimo in un Array Ordinato	87
3.5.8	Algoritmo: Ricerca del Successore in un Array non ordinato	88
3.5.9	Algoritmo: Ricerca del Successore Ricorsiva in un Array Ordinato	91
3.5.10	Ricerca del Successore Iterativa in un Array Ordinato	94
3.6	Algoritmi sulle Liste Concatenate	97
3.6.1	Algoritmo: Ricerca in una Lista Concatenata non ordinata	98
3.6.2	Algoritmo: Ricerca ricorsiva in una Lista Concatenata Ordinata	99
3.6.3	Algoritmo: Ricerca iterativa in una Lista Concatenata Ordinata	102
3.7	Operazioni di Inserimento negli Array	104
3.7.1	Esempio di Inserimento in un Array	105
3.8	Operazioni di Rimozione negli Array	106
3.9	Algoritmi sugli Stack e Queue	107
4	Alberi	110
4.1	Introduzione	110
4.2	Definizione di Albero	112
4.3	Visite su un albero binario	114
4.3.1	Algoritmo: Visita in preorder di una Lista	115

4.3.2	Algoritmo: Visita in postorder di una Lista	115
4.3.3	Visite su un albero binario	116
4.3.4	Algoritmo: visita in preorder su un albero binario	117
4.3.5	Algoritmo: visita in postorder su un albero binario	118
4.3.6	Algoritmo: visita in inorder su un albero binario	119
4.3.7	Algoritmo: visita in ampiezza su un albero binario	120
4.3.8	Algoritmo: DFS in un Albero Binario	123
4.4	Alberi binari di ricerca - BST	126
4.5	Algoritmi su BST	127
4.5.1	Algoritmo: ricerca di un dato in un BST	127
4.5.2	Algoritmo: ricerca ricorsiva del minimo in un BST	128
4.5.3	Algoritmo: ricerca iterativa del minimo in un BST	129
4.5.4	Algoritmo: ricerca del successore in un BST versione non tail recursive	130
4.5.5	Algoritmo: ricerca del successore in un BST versione tail recursive	131
4.5.6	Algoritmo: inserimento in un BST	133
4.5.7	Algoritmo: cancellazione in un BST	135
4.6	Bilanciamento negli alberi binari	143
4.7	Alberi pieni	144
4.8	Alberi perfettamente bilanciati	145
4.8.1	Proprietà alberi perfettamente bilanciati	147
4.9	Alberi bilanciati	149
4.10	Alberi AVL	151
4.10.1	Alberi AVL minimi - MAVL	154
4.10.2	Proprietà degli alberi AVL	157
4.10.3	Inserimento in un albero AVL	162
4.10.4	Rotazione in un albero binario	164
4.10.5	Inserimento in un albero AVL: caso 1	166
4.10.6	Inserimento in un albero AVL: caso 2	167
4.10.7	Rappresentazione in memoria di un nodo di un albero AVL	170

4.10.8	Algoritmo: gestione altezza in un albero AVL	171
4.10.9	Algoritmo: rotazione in un albero AVL	173
4.10.10	Algoritmo: inserimento in un albero AVL	175
4.10.11	Cancellazione in un albero AVL: caso 1A	178
4.10.12	Cancellazione in un albero AVL: caso 1B	179
4.10.13	Cancellazione in un albero AVL: caso 2	180
4.10.14	Algoritmo: cancellazione in un albero AVL	182
4.10.15	Considerazioni finali sugli alberi AVL	186
4.11	Alberi Red Black	187
4.11.1	Caratteristiche degli alberi RB	188
4.11.2	Colorazione degli Alberi RB	190
4.11.3	Proprietà degli alberi RB	191
4.11.4	Inserimento in un albero RB	194
4.11.5	Inserimento in un albero RB: caso 1A	197
4.11.6	Inserimento in un albero RB: caso 1B	198
4.11.7	Inserimento in un albero RB: caso 2	200
4.11.8	Inserimento in un albero RB: caso 3	201
4.11.9	Algoritmo: inserimento in un albero RB	203
4.11.10	Cancellazione in un albero RB	207
4.11.11	Cancellazione in un albero RB: caso 1	209
4.11.12	Cancellazione in un albero RB: caso 2	211
4.11.13	Cancellazione in un albero RB: caso 3	214
4.11.14	Cancellazione in un albero RB: caso 4	217
4.11.15	Algoritmo: cancellazione in un albero RB	220
5	Traduzione di algoritmi ricorsivi in iterativi	226
5.1	Introduzione	226
5.2	Esempio 1	227
5.3	Esempio 2	230
5.4	Esempio 3	233

5.5	Esempio 2 - Continuo	236
5.6	Schema generale	240
5.6.1	Passi da effettuare per la traduzione dell'algoritmo ricorsivo generale in iterativo	243
5.7	Esempio 4	250
5.8	Esempio 5	254
6	Grafi	258
6.1	Introduzione	258
6.2	Rappresentazione di un grafo tramite matrice di adiacenza	260
6.2.1	Vantaggi e svantaggi della rappresentazione matriciale	262
6.2.2	Grado di un grafo	262
6.3	Considerazioni sulla rappresentazione matriciale di un grafo	263
6.4	Grafo Trasposto	265
6.5	Rappresentazione di un grafo tramite lista di adiacenti	267
6.5.1	Vantaggi e svantaggi della rappresentazione con lista di adiacenti	268
6.6	Percorso in un grafo	270
6.6.1	Esempio famiglia di grafi con numero di percorsi semplici esponenziale	271
6.7	Raggiungibilità in un grafo	273
6.8	Relazione di connessione in un grafo	277
6.9	Distanza in un grafo	280
6.10	Visita in ampiezza su un grafo	282
6.10.1	Algoritmo: Breadth First Search in un grafo	285
6.10.2	Algoritmo: Percorso minimo in un grafo	288
6.10.3	Correttezza dell'algoritmo BFS	290
6.10.4	Complessità dell'algoritmo BFS	297
6.11	Visite in profondità sui grafi	299
6.11.1	Algoritmo: Depth First Search su un grafo	300
6.11.2	Complessità dell'algoritmo DFS	303
6.12	Analisi algoritmo DFS e relativa foresta di visita	304
6.12.1	Proprietà algoritmo DFS	311

6.13	Partizionamento archi in un grafo mediante algoritmo DFS	318
6.14	Cicli in un grafo	320
6.14.1	Algoritmo: ricerca cicli in un grafo	321
6.14.2	Correttezza algoritmo ricerca cicli in un grafo	323
6.14.3	Complessità algoritmo ricerca cicli in un grafo	325
6.15	Ordinamento topologico per un grafo	326
6.15.1	Algoritmo: ordinamento topologico in un grafo basato algoritmo DFS	329
6.15.2	Correttezza algoritmo topological_ordering (DFS)	332
6.15.3	Algoritmo: ordinamento topologico in un grafo basato grado entrante	333
6.16	Componenti connesse e fortemente connesse in un grafo	345
6.16.1	Algoritmo: calcolo componenti fortemente connesse in un grafo orientato	356
6.16.2	Correttezza algoritmo calcolo componenti fortemente connesse in un grafo orientato	360
6.17	Grafi pesati	364
6.17.1	Definizione di grafo pesato	366
6.17.2	Minimo percorso in un grafo pesato	367
6.17.3	Peso di un percorso in un grafo pesato	369
6.17.4	Calcolo del percorso minimo in un grafo pesato	374
6.17.5	Algoritmo di Dijkstra	383
6.17.6	Complessità algoritmo di Dijkstra	386
6.17.7	Correttezza algoritmo di Dijkstra	388
6.17.8	Algoritmo di Bellman-Ford	393
7	Algoritmi di Ordinamento	399
7.1	Introduzione	399
7.2	Complessità asintotica	400
7.2.1	Esempio analisi asintotica data una funzione	403
7.3	Algoritmi di ordinamento basati su confronti	406
7.4	Insertion sort	407
7.4.1	Algoritmo: insertion sort ricorsivo	408
7.4.2	Analisi dell'algoritmo insertion sort	410

7.4.3	Algoritmo: insertion sort iterativo	413
7.4.4	Complessità dell'algoritmo insertion sort versione iterativa	414
7.4.5	Complessità dell'algoritmo insertion sort versione ricorsiva	417
7.4.6	Correttezza algoritmo insertion sort	420
7.5	Selection sort	422
7.5.1	Algoritmo: selection sort	423
7.6	Alberi Completi	426
7.7	Alberi Heap	430
7.8	Heap sort	436
7.8.1	Algoritmo: heap sort	443
7.8.2	Complessità dell'algoritmo heap sort	446
7.9	Heap nell'algoritmo di Dijkstra	449
7.10	Merge sort	452
7.10.1	Algoritmo: merge sort	453
7.10.2	Correttezza dell'algoritmo merge sort	456
7.10.3	Complessità dell'algoritmo merge sort	457
7.11	Equazioni di ricorrenza	461
7.11.1	Esempio 1: equazione di ricorrenza dell'algoritmo merge sort	462
7.11.2	Esempio 2	464
7.11.3	Esempio 3	466
7.11.4	Esempio 4	471
7.11.5	Esempio 5	473
7.12	Quick sort	475
7.12.1	Algoritmo: quick sort	481
7.12.2	Proprietà della funzione partition	485
7.12.3	Proprietà dell'algoritmo quick sort	489
7.12.4	Complessità dell'algoritmo quick sort	491
7.13	Teorema sul limite asintotico dell'ordinamento basato su confronti	499

Elenco degli algoritmi

1	Calcolo della sequenza di Fibonacci versione ricorsiva.	33
2	Calcolo della sequenza di Fibonacci prima versione iterativa.	36
3	Calcolo della sequenza di Fibonacci seconda versione iterativa.	40
4	Conteggio delle coppie prima versione.	43
5	Conteggio delle coppie seconda versione.	44
6	Conteggio delle coppie terza versione.	46
7	Conteggio delle coppie quarta versione.	48
8	Max subsequence sum prima versione.	50
9	Max subsequence sum seconda versione.	52
10	Max subsequence sum terza versione.	56
11	Ricerca in un array con ripetizioni	74
12	Ricerca in un array senza ripetizioni	75
13	Ricerca binaria ricorsiva in un array ordinato (funzione di interfaccia)	77
14	Ricerca binaria ricorsiva in un array ordinato (funzione accessoria)	78
15	Ricerca binaria iterativa in un array ordinato	80
16	Ricerca del minimo in un array.	85
17	Ricerca del minimo in un array ordinato.	87
18	Ricerca del successore in un array non ordinato.	89
19	Ricerca del successore ricorsiva in un array ordinato (funzione di interfaccia)	91
20	Ricerca del successore ricorsiva in un array ordinato (funzione accessoria)	92
21	Ricerca del successore iterativa in un array ordinato	95

22	Ricerca in una lista concatenata non ordinata	98
23	Ricerca ricorsiva in una lista concatenata ordinata	100
24	Ricerca iterativa in una lista concatenata ordinata	102
25	Visita in preorder in una lista concatenata	115
26	Visita in postorder in una lista concatenata	115
27	Visita in preorder in un albero binario	117
28	Visita in postorder in un albero binario	118
29	Visita in inorder in un albero binario	119
30	Visita in ampiezza in un albero binario	121
31	DFS preorder in un albero binario	123
32	DFS postorder in un albero binario	124
33	DFS inorder in un albero binario	125
34	Ricerca di un dato in un BST	127
35	Ricerca ricorsiva del minimo in un BST non vuoto	128
36	Ricerca iterativa del minimo in un BST non vuoto	129
37	Ricerca del successore in un BST versione non tail recursive	130
38	Ricerca del successore in un BST versione tail recursive funzione di interfaccia	131
39	Ricerca del successore in un BST versione tail recursive funzione accessoria	131
40	Inserimento di un dato in un BST	134
41	Cancellazione in un BST: funzione <code>delete</code>	137
42	Cancellazione in un BST: funzione <code>delete_data</code>	138
43	Cancellazione in un BST: funzione <code>skip_right</code>	139
44	Cancellazione in un BST: funzione <code>skip_left</code>	140
45	Cancellazione in un BST: funzione <code>get_&delete_min</code>	141
46	Cancellazione in un BST: funzione <code>swap_child</code>	142
47	Rotazione da sinistra a destra	165
48	Altezza di un nodo	171
49	Aggiornamento dell'altezza di un nodo	171
50	Rotazione da sinistra a destra in un albero AVL	173

51	Rotazione doppia in un albero AVL	173
52	Inserimento in un albero AVL	175
53	Bilanciamento da sinistra	176
54	Cancellazione in un albero AVL	183
55	Cancellazione di un nodo in un albero AVL	184
56	Cancellazione del minimo dal sottoalbero di destra AVL	185
57	Inserimento in un albero RB	204
58	Bilanciamento dell'inserimento a sinistra in un albero RB	205
59	Funzione per le violazioni possibili a seguito dell'inserimento a sinistra in un albero RB	206
60	Cancellazione in un albero RB	220
61	Cancellazione di un nodo in un albero RB	221
62	Cancellazione del minimo del sottoalbero destro in un albero RB	222
63	Funzione di propagazione verso l'alto del colore nero	223
64	Funzione di skip left in un albero RB	223
65	Funzione di bilanciamento di una cancellazione a sinistra in un albero RB	224
66	Funzione delle violazioni a seguito di una cancellazione a sinistra in un albero RB	225
67	Esempio 1 (ricorsivo)	227
68	Traduzione Esempio 1 (iterativo)	229
69	Esempio 2 (ricorsivo)	230
70	Esempio 3 (ricorsivo)	233
71	Traduzione Esempio 3 (iterativo)	234
72	Traduzione Esempio 2 (iterativo)	237
73	Algoritmo ricorsivo generale	241
74	Traduzione in algoritmo iterativo generale	247
75	Esempio 4 (ricorsivo)	250
76	Traduzione Esempio 4 (iterativo)	252
77	Esempio 5 (ricorsivo)	254
78	Esempio 5 (ricorsivo) v. 2	255
79	Traduzione Esempio 5 (iterativo)	256

80	Breadth First Search in un grafo: BFS	286
81	Funzione di inizializzazione (BFS)	287
82	Percorso minimo in un grafo	288
83	Funzione per la costruzione del percorso minimo in un grafo	289
84	Visita in profondità di un grafo: DFS	300
85	Funzione di inizializzazione (DFS)	301
86	Funzione accessoria ricorsiva per la DFS in un grafo	302
87	Funzione che verifica l'aciclicità di un grafo	321
88	Funzione accessoria per la verifica dell'aciclicità di un grafo, parte ricorsiva	322
89	Funzione per l'ordinamento topologico in un grafo (DFS)	329
90	Funzione accessoria per l'ordinamento topologico in un grafo, parte ricorsiva (DFS)	330
91	Funzione ausiliaria per il calcolo del grado entrante sul grafo	341
92	Funzione ausiliaria per trovare e restituire l'insieme dei vertici con grado entrante zero	342
93	Funzione per l'ordinamento topologico in un grafo aciclico basato su grado entrante	343
94	Funzione accessoria per fare il trasposto di un grafo	356
95	Funzione per trovare le componenti fortemente connesse di un grafo	357
96	Funzione accessoria per la visita in profondità di un grafo, dato un determinato ordine	358
97	Funzione accessoria per la visita in profondità di un grafo, dato un determinato ordine (ricorsiva)	359
98	Funzione accessoria per l'inizializzazione di un grafo pesato	383
99	Funzione accessoria per il rilassamento di un arco in un grafo pesato	384
100	Algoritmo di Dijkstra	385
101	Algoritmo di Bellman-Ford su grafi pesati	397
102	Algoritmo di Insertion sort versione ricorsiva	408
103	Funzione accessoria per l'insertion sort versione ricorsiva	409
104	Algoritmo di Insertion sort versione iterativa	413
105	Funzione accessoria per il selection sort	423
106	Algoritmo di Selection sort	424
107	Funzione accessoria che costruisce un max heap partendo da un array	443
108	Funzione accessoria che corregge un albero quasi heap in albero heap	444

109	Algoritmo di heap sort	445
110	Funzione accessoria per l'algoritmo di Dijkstra utilizzando uno heap	451
111	Algoritmo di merge sort: funzione di interfaccia	453
112	Algoritmo di merge sort: parte ricorsiva	453
113	Algoritmo di merge sort: funzione merge	455
114	Algoritmo di Quick sort: funzione di interfaccia	481
115	Algoritmo di Quick sort: funzione ricorsiva	482
116	Algoritmo di Quick sort: funzione accessoria partition	483

Elenco delle figure

I.1	Slides-Introduzione	2
1.1	Macchina di Turing che incrementa di 1 un numero binario in input	24
2.1	Esempio di esecuzione dell'algoritmo MSS versione ottimale	59
3.1	Esempio di Inserimento negli Array	105
4.1	Esempio di disaccoppiamento della memorizzazione dei dati dal loro ordine	111
4.2	Nodo di un albero	112
4.3	Albero	113
4.4	Esempio di Visita in Ampiezza	122
4.5	Esempio di esecuzione della <code>DFS_pre_order</code>	124
4.7	Esempio di BST e albero non di ricerca	126
4.8	Esempio di ricerca del successore in un BST	132
4.9	Inserimento in un BST	133
4.10	Esempio di cancellazione in BST: nodo con un solo figlio	136
4.11	Esempio di cancellazione in BST: nodo con un solo figlio	136
4.12	Esempio di cancellazione in BST: nodo con due figli	137
4.13	Esempi di cancellazione in BST applicando la funzione <code>skip_right</code>	139
4.14	Esempi di cancellazione in BST: applicando la funzione <code>skip_left</code>	140
4.15	Esempio di cancellazione in BST: nodo con due figli	142
4.16	Esempio di costruzione di un albero degenero usando l'inserimento per i BST	143
4.18	Esempio di albero perfettamente bilanciato e non perfettamente bilanciato	146

4.19	Bilanciamento di alberi perfettamente bilanciati	148
4.20	Alberi AVL minimi di altezza 0, 1, 2, 3, 4 e 5	154
4.21	Generalizzazione della struttura di un albero AVL minimo di altezza h	155
4.22	Esempio di inserimento in un AVL	162
4.23	Esempio di inserimento in un AVL	163
4.24	Esempio di rotazione da sinistra a destra	165
4.25	Inserimento in AVL (caso 1)	166
4.26	Inserimento in AVL (caso 2 con risoluzione sbagliata)	167
4.27	Inserimento in AVL (caso 2)	168
4.28	Nodo di un albero AVL	170
4.29	Rotazione doppia in AVL	174
4.30	Cancellazione in AVL (caso 1)	179
4.31	Cancellazione in AVL (caso 2)	181
4.32	Colorazione di un albero RB	190
4.33	Inserimento in albero RB (Caso 1A)	197
4.34	Inserimento in albero RB (Caso 1B)	198
4.35	Inserimento in albero RB (Caso 2)	200
4.36	Inserimento in albero RB (Caso 3)	201
4.37	Nodo di un albero RB	203
4.38	Cancellazione in albero RB (Caso 1 con radice del sottoalbero rossa)	209
4.39	Cancellazione in albero RB (Caso 1 con radice del sottoalbero nera)	210
4.40	Cancellazione in albero RB (Caso 2)	211
4.41	Cancellazione in albero RB (Caso 3)	214
4.42	Cancellazione in albero RB (Caso 4)	217
5.1	Esempio di esecuzione dell'algoritmo sopra descritto	231
5.2	Esempio di albero delle chiamate ricorsive del merge sort	251
6.1	Esempio di grafo rappresentato mediante nodi e archi	261
6.2	Esempio di grafi completi	263

6.3	Esempio di grafo trasposto	265
6.4	Esempio di grafo rappresentato con lista di adiacenti	267
6.5	Esempio di famiglia di grafi con numero di simple path esponenziale	271
6.6	Esempio 2 di grafo	274
6.7	Esempio di calcolo della raggiungibilità con prodotto tra matrici	276
6.8	Esempi di grafi connessi e fortemente connessi	278
6.9	Esempi di sottografi e sottografi indotti	279
6.10	Esempio di visita in ampiezza su un grafo	283
6.11	Esempio grafico del lemma delle distanze dei grafi semplici	291
6.12	Esempio 1 di visita in profondità del grafo	307
6.13	Esempio 2 di visita in profondità del grafo	309
6.14	Foreste di visita degli esempi 1 e 2 della DFS	310
6.15	Esempio di struttura a parentesi della DFS	311
6.16	Esempio della dimostrazione sul percorso bianco	317
6.17	Esempio di grafo aciclico con grado entrante	333
6.18	Esempio di componenti connesse e componenti connesse massimali	345
6.19	Esempio di foreste di visita partendo da grafi orientati	347
6.20	Esempio di componenti connesse nelle foreste di visita	348
6.21	Ulteriore esempio di grafo	349
6.22	Esempio di grafo delle componenti	351
6.23	Esempio di foresta di visita per componenti massimali, con tempi di visita	352
6.24	Grafo trasposto dell'esempio 3 di grafo	353
6.25	Ordine per fine visita delle componenti connesse dell'esempio 3 di grafo	353
6.26	Stack alla fine della DFS sull'esempio 3 di grafo	354
6.27	Esempio di grafo pesato	367
6.28	Esempio di rappresentazione di un grafo pesato tramite matrice di adiacenza	367
6.29	Esempio di rappresentazione di un grafo pesato tramite lista di adiacenti	368
6.30	Esempio di grafo pesato con pesi non negativi	370
6.31	Esempio 2 di grafo pesato con pesi non negativi	375

6.32	Esempio di applicazione dell'idea intuitiva naïve per il calcolo del percorso minimo in un grafo pesato	376
6.33	Esempio di applicazione dell'idea intuitiva smart per il calcolo del percorso minimo in un grafo pesato	380
6.34	Esempio 3 di grafo pesato con pesi non negativi	381
6.35	Esempio complesso per il calcolo del percorso minimo in un grafo pesato	382
6.36	Esempio di grafo pesato senza cicli a pesi negativi	394
6.37	Esempio di esecuzione dell'algoritmo di Bellman-Ford su un grafo pesato senza cicli negativi . . .	395
6.38	Esempio di grafo pesato con cicli a pesi negativi	396
6.39	Esempio complesso per il calcolo del percorso minimo in un grafo pesato	396
7.1	Esempio di vettore random per l'insertion sort	416
7.2	Esempio di individuazione di alberi completi	427
7.3	Forma caratteristica di un albero completo	428
7.4	Visualizzazione degli alberi pieni nell'albero completo	428
7.6	Esempi di alberi heap	430
7.7	Esempio di rappresentazione di un albero completo in un array	431
7.9	Esempi di alberi heap	432
7.10	Esempio di rappresentazione di un albero completo in un array	432
7.11	Simulazione di correzione di un albero quasi heap in max heap	435
7.12	Esempio di linearizzazione dell'albero visitato in ampiezza in un array	437
7.13	Esempio di funzionamento dell'algoritmo heap sort	442
7.14	Categorizzazione del numero di foglie negli alberi completi	446
7.15	Esempio di cambio di priorità in un albero min heap	449
7.16	Simulazione di correzione di un albero quasi heap in min heap	451
7.17	Albero di ricorsione del merge sort	462
7.18	Albero di ricorsione dell'esempio 2	464
7.19	Albero di ricorsione dell'esempio 3	466
7.20	Forma generica dell'albero di ricorsione dell'esempio 3	467
7.21	Esempio 3 limite inferiore	468
7.22	Esempio 3 limite superiore	469

7.23 Esempio 3 limite inferiore e superiore	469
7.24 Albero di ricorsione dell'esempio 4	471
7.25 Albero di ricorsione dell'esempio 5	473
7.26 Esempio astratto della proprietà richiesta	476
7.27 Esempio di esecuzione del quick sort	480
7.28 Albero di ricorsione del quick sort	493
7.29 Albero di ricorsione del quick sort	494
7.30 Albero di ricorsione del quick sort	496
7.31 Forma generica dell'albero di ricorsione del quick sort con k proporzionale a n	497
7.32 Esempio di array per l'albero di decisione	500
7.33 Esempio di albero di decisione su un array di dimensione 3	501
7.34 Esempio di albero di decisione con una foglia rimossa	502
7.35 Esempio di forma generale di un albero di decisione per l'insertion sort	504
7.36 Esempio della forma del migliore albero di decisione	505
7.37 Esempio di spostamento di nodi foglie nell'albero completo	509
7.38 Miglior albero di decisione per il caso medio	510

Elenco dei teoremi

2.1.1	Proposizione (Tempo algoritmo ricorsivo <code>fib</code>)	34
2.3.1	Teorema (Teorema <code>MSS</code>)	54
4.8.1	Teorema (Altezza albero pieno)	147
4.8.2	Teorema (Altezza albero perfettamente bilanciato)	147
4.10.1	Teorema (Struttura albero AVL minimo)	155
4.10.2	Teorema (Correlazione tra NN_{MAVL} e <i>Fib</i>)	157
4.10.3	Teorema (Altezza albero AVL minimo)	159
4.10.1	Proposizione (Altezza albero AVL)	160
4.11.1	Proposizione (Numero nodi interni albero RB)	191
4.11.1	Teorema (Altezza albero RB)	192
6.10.1	Lemma (Distanze grafi semplici)	291
6.10.2	Lemma (Invarianti <code>BFS</code>)	292
6.10.1	Teorema (Correttezza <code>BFS(G, v)</code>)	293
6.12.1	Teorema (Struttura a parentesi della DFS)	312
6.12.1	Lemma (Percorsi nella foresta di visita della DFS)	314
6.12.2	Teorema (Percorso Bianco)	316
6.14.1	Teorema (Dimostrazione correttezza algoritmo di aciclicità)	323
6.15.1	Lemma (Non esistenza ordinamento topologico in grafo aciclico)	328
6.15.1	Proposizione (Correttezza <code>topological_ordering</code> (DFS))	332
6.15.2	Lemma (Grado entrante grafo aciclico finito)	339
6.15.3	Lemma (Sottografo di un grafo aciclico)	340

6.16.1	Lemma (Percorsi di una componente connessa)	360
6.16.2	Lemma (Componenti connesse massimali nella foresta di visita della DFS)	361
6.16.1	Teorema (Dimostrazione correttezza algoritmo SCC)	361
6.17.1	Lemma (Sottopercorso di un percorso pesato minimo)	371
6.17.1	Corollario (Scomposizione sottopercorso di un percorso pesato minimo)	372
6.17.2	Lemma (Distanze grafi pesati)	372
6.17.3	Lemma (Invariante relax)	388
6.17.4	Lemma (Limite stima distanza pesata)	389
6.17.2	Corollario (Corollario limite stima distanza pesata)	390
6.17.5	Lemma (Stima definitiva peso sequenza relax)	390
6.17.1	Teorema (Dimostrazione correttezza algoritmo di Dijkstra)	391
7.2.1	Teorema (Definizione complessità asintotiche)	401
7.4.1	Teorema (Dimostrazione di correttezza dell'insertion sort ricorsivo)	420
7.4.2	Teorema (Dimostrazione di correttezza dell'insertion sort iterativo)	421
7.6.1	Teorema (Altezza albero completo)	429
7.10.1	Teorema (Dimostrazione di correttezza del merge sort)	456
7.12.1	Teorema (Dimostrazione proprietà di partition)	485
7.12.2	Teorema (Dimostrazione terminazione del quick sort)	489
7.12.3	Teorema (Dimostrazione correttezza del quick sort)	489

Capitolo I

Introduzione al Corso

"You take the blue pill... the story ends, you wake up in your bed and believe whatever you want to believe. You take the red pill... you stay in Wonderland, and I show you how deep the rabbit hole goes"

– Morpheus (The Matrix)

Questo è un corso in cui dobbiamo imparare a ragionare, non è un corso dove focalizzarci sulle nozioni fini a se stesse perché non servirà a molto, è un corso che ci serve per capire come impostare un ragionamento per poi poter affrontare problemi non necessariamente esattamente identici ma che si possono vedere come nella stessa sfera di quelli che analizzeremo, quindi è un corso di metodologia di risoluzione di problemi, non è, diciamo, l'ennesimo corso di programmazione.

È un corso in cui dovremmo imparare a progettare soluzioni, soluzioni a problemi ovviamente di natura informatica, quindi un corso di approcci alla soluzione di problemi.

Questo dovrebbe essere chiaro sin dall'inizio.

Algoritmi e Strutture Dati I

Testi di Riferimento

- [CLRS'09] T.H. CORMEN, C.E. LEISEN, R.L. RIVEST, C. STEIN - INTRODUCTION TO ALGORITHMS / 3rd Ed.
- [Eri'19] J. ERICKSON - ALGORITHMS / On-line version
- [GKP'94] R.L. GRAHAM, D.E. KNUTH, O. PATASHNIK - CONCRETE MATHEMATICS / 2nd Ed.

F. MOGAVERO
A.A. 2023/2024

Argomenti del Corso

- Elementi di Analisi degli Algoritmi
- Strutture Dati per la gestione di Collezioni di Dati
- Grafi
- Algoritmi di Ordinamento e relativa analisi di complessità

Prerequisiti (corsi del 1° anno)

- Analisi I: Somme, Serie, Limiti, Derivate e Studi di funzione.
- Algebra: Insiemi, Funzioni, Relazioni (e.g., ordine, equivalenze) / Induzione, Ricorsione / Strutture
- Programmazione: Programmazione / Strutture di Controllo / Computazionale / Storiche & Ricorsione

Lezioni: 11 Settembre - 15 Dicembre

- Lunedì (16-18), Martedì (11-13) e Giovedì (11-13)
- Aula G3 • Intervizione 6-10 Novembre

Ritorno

- Giovedì ore 14:30-16:30 (Via Claudio, Edificio 3)

Accesso all'Esame

- Cognomi H-Z (Gruppo/Canale 2)

Esame

- Scritto + Orale (OPZIONALE)

Figura I.1: Slides-Introduzione

I.1 Testi Consigliati

- [CLRS'09] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein - Introduction to Algorithms / 3rd Ed.

Il primo testo consigliato è il Cormen, in realtà sono quattro autori ma è noto come Cormen che è il nome del primo autore, ed è, tra virgolette, una delle bibbie riguardanti gli algoritmi.

- [Eri'19] Jeff Erickson - Algorithms / On-line version

Il secondo testo, che invece è possibile trovare gratuitamente online, è l'Erikson, chiamato semplicemente Algorithm. Sulla pagina web di questo autore è scaricabile in chiaro senza dover pagare alcunché.

- [GKP'94] Ronald L. Graham, Donald E. Knuth, Oren Patashnik - Concrete Mathematics / 2nd Ed.

Il terzo testo è un testo più matematico, che potrebbe essere utile nel momento in cui andiamo a fare qualche analisi leggermente più matematica e vogliamo un riscontro di un libro di testo a cui fare riferimento.

Il seguente manuale non ha la pretesa di sostituire i libri di testo. Per avere una preparazione adeguata bisogna far sempre riferimento a dei libri di testo.

I.2 Descrizione del Corso

Il corso è composto da 36 lezioni da due ore, quindi 72 ore.

Nelle prime lezioni quello che andremo a fare è capire che cosa significa analizzare e studiare un algoritmo. Partiremo dalle basi. Vedremo cosa si intende per algoritmo e vedremo, basandoci puramente su esempi, cioè senza impostare una teoria generale, quindi più che altro impara tramite esempi, cosa significa studiare degli algoritmi.

Vedremo degli algoritmi estremamente semplici all'inizio.

Dopodiché lavoreremo direttamente sulle strutture dati.

Quindi partiremo con strutture dati elementari, come gli array e le liste concatenate, e poi ci muoveremo verso strutture dati più elaborate.

Vedremo determinate strutture dati, in particolare gli alberi binari di ricerca, e diverse versioni di queste strutture dati.

Fatto ciò andremo a studiare i grafi. Andremo a motivare perché sono importanti queste strutture dati e ne studieremo gli algoritmi associati.

Solo alla fine ci muoveremo sull'analisi degli algoritmi di ordinamento che non sarà fine a se stessa; non utilizziamo gli algoritmi di ordinamento come strumento, cioè per insegnare quali sono gli algoritmi di ordinamento, quello che faremo è utilizzare gli algoritmi di ordinamento come strumento esemplificativo delle tecniche, questa parte sarà leggermente più impostata su un approccio formale matematico.

I.3 Prerequisiti

Allora, quali sono i prerequisiti?

Attenzione che prerequisiti non significa che il corso o l'esame debba essere per forza stato fatto precedentemente, ma quali sono le conoscenze che ci si aspetta come prerequisito nel momento in cui si affronta questo corso.

Bisogna avere delle nozioni di Analisi I necessarie come: sommatorie, serie, limiti, derivate e che cosa significa fare uno studio di funzioni.

Dal corso di algebra, per connettersi al concetto di grafo, bisogna sapere cos'è un insieme, una funzione, le relazioni, ad esempio cos'è una relazione di equivalenza o una relazione d'ordine, avere il concetto di induzione, di prova per induzione, di costruzione ricorsiva, e strutture dati ad esempio grafi.

Di programmazione ovviamente bisogna conoscere cosa sia un programma, strutture di controllo, ovviamente, cosa significa computare, cos'è la computazione, e avere un'idea di iterazione e ricorsione.

I.4 Esame

L'esame è strutturato in una prova scritta ed una prova orale.

Lo scritto è obbligatorio e a seguito della prova scritta verrà dato un voto; se tale voto è sufficiente darà la possibilità di registrare l'esame così come è.

L'orale è opzionale, tuttavia su richiesta ci sarà la possibilità di sostenere un esame orale per potenzialmente migliorare (o peggiorare) il voto.

I.5 Argomenti del Corso

Nella prima parte del corso cercheremo di capire cosa significa davvero un algoritmo.

Dopodiché cercheremo di capire cosa significa davvero struttura dati.

Successivamente cercheremo di capire che differenza fa il modello, detto modello computazionale, cioè il modello di calcolo che noi utilizziamo rispetto a potenziali altri modelli.

E quindi andremo a capire molto sommariamente che differenza c'è tra i vari modelli computazionali e perché si predilige utilizzare un certo modello piuttosto che un altro.

Dopodiché capiremo che cosa significa l'analisi di un algoritmo, che cosa significa analizzare un certo algoritmo, e in particolare andare a capire cosa significa mostrare che un algoritmo sia corretto, cioè che faccia quello per cui è stato pensato.

Quindi dimostrare in generale quando un algoritmo è corretto e quando invece non lo è.

Ma questo è solo uno degli aspetti dell'analisi degli algoritmi. Perché?

Perché per risolvere un certo problema non è detto che ci sia un unico approccio alla sua risoluzione.

Così come non è detto che ci sia un unico algoritmo.

Supponendo ora che tutti gli algoritmi che abbiamo a disposizione sono corretti, perché dobbiamo sceglierne uno al posto di un altro?

Abbiamo sempre una scelta univoca?

Abbiamo piuttosto una scelta che dipende dal contesto applicativo?

Quali sono le informazioni del contesto applicativo che ci permettono di scegliere tra più algoritmi quale sia quello più adatto?

In alcuni casi potrebbe esserci più di una scelta ottimale, ed allora la scelta la facciamo piuttosto basandoci su quello che meglio conosciamo, oppure quello che ci è più disponibile perché è già presente in libreria.

Ma in altri casi la scelta è univoca ed è guidata dal contesto perché un algoritmo è efficiente.

Cosa significa efficiente?

Andremo a capire cosa significa efficienza, in particolare la differenza tra efficienza asintotica e efficienza media. Efficienza non va immaginata solo a livello temporale, ci sono diversi tipi di efficienza, ma anche efficienza su diversi tipi di risorse.

Il tempo è una risorsa, lo spazio è un'altra risorsa, in generale però, soprattutto quando lavoriamo con sistemi complessi, esistono anche altre risorse.

Purtroppo il corso è molto limitato, di conseguenza ci soffermeremo piuttosto sulla risorsa tempo.

Qualche volta accenneremo alla risorsa a spazio, a nessun'altra, ma maggiormente ci soffermeremo a livello temporale.

Per dare una base leggermente formale a questo concetto di efficienza introdurremo la notazione asintotica. La introdurremo formalmente, ma la utilizzeremo in gran parte ad un livello più intuitivo.

In seconda battuta cominceremo ad analizzare le strutture dati. Partiremo da come si distinguono alcune strutture dati.

Due strutture dati che possono contenere lo stesso tipo di dati non hanno le stesse proprietà. E allora come facciamo a scegliere?

Dipende dal contesto applicativo.

Anche solo la scelta di una struttura dati richiede conoscenze. Quindi ci serve capire perché sono fatte in un certo modo e quali proprietà hanno.

Analizzeremo i concetti quindi di array e di lista, di stack e queue, infine lavoreremo sugli alberi.

Dopodiché cercheremo di andare a capire come un algoritmo che è ricorsivo, cioè impostato con le operazioni ricorsive, quindi con chiamate a funzioni dello stesso tipo, possa essere trasformato in un algoritmo iterativo.

Terminata questa parte ci muoveremo sui grafi.

Il concetto di grafo è uno dei concetti più importanti in computer science, in informatica, perché tantissimi problemi sono problemi su grafi.

Infine, utilizzeremo la rimanente parte del corso per cercare di dare un maggiore insight, una maggiore profondità, sulla parte di calcolo della complessità e utilizzeremo gli algoritmi di ordinamento per mostrare le diverse tecniche e anche come strumento per capire ancora meglio perché sono state fatte certe scelte e in particolare come, partendo da un'idea molto semplice, si possa poi ottenere qualcosa di più raffinato solo cambiando una parte all'interno di un algoritmo, quindi identificando qual è il collo di bottiglia di un approccio.

Capitolo II

Registro delle Lezioni 2023/2024

"There is no spoon"

– *Spoon Boy (The Matrix)*

Per i codici dei testi si faccia riferimento a I.1

Lezione n.01 del 11/09/23

Argomenti:

- Introduzione al corso (Testi di riferimento; Argomenti trattati; Strutturazione delle lezioni; Modalità d'esame).
- Fondamenti di Analisi degli Algoritmi Parte I (Definizione informale di Algoritmo e Struttura Dati e relative proprietà [CLRS09(1.1)] [Eri09(0.1)]).
- Modelli computazionali: Macchine di Turing vs Modelli ad Accesso Casuale.

Lezione n.02 del 12/09/23

Argomenti:

- Fondamenti di Analisi degli Algoritmi Parte II (Modelli computazionali: Macchine di Turing vs Modelli ad Accesso Casuale; Linguaggi di descrizione degli algoritmi: Pseudo codice [Eri19(0.5)]).
- Strutture di controllo e relative stime dei tempi di esecuzione; Analisi degli algoritmi: Correttezza, Tempo di esecuzione, Occupazione di memoria [CLRS09(2.2)] [Eri19(0.6)]).

Lezione n.03 del 21/09/23

Argomenti:

- Fondamenti di Analisi degli Algoritmi Parte III (Esempi di algoritmi e relativa analisi: Sequenza di Fibonacci e Conteggio coppie).
- Analisi asintotica: Motivazioni e definizione informale).

Lezione n.04 del 25/09/23

Argomenti:

- Fondamenti di Analisi degli Algoritmi Parte IV (Esempi di algoritmi: MaxSubsequenceSum [CLRS'09(4.1)] [Eri'19(2.6, 2.7)]).

Lezione n.05 del 26/09/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte I (Strutture dati elementari: vettori, liste, stack, queue [CLRS'09(10)]).

Lezione n.06 del 28/09/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte II (Collezioni statiche con rappresentazione concreta a funzione caratteristica [CLRS'09(10)]).

Lezione n.07 del 02/10/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte III (Collezioni dinamiche con rappresentazione concreta a lista concatenata [CLRS'09(10)]).

Lezione n.08 del 03/10/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte IV (Algoritmi su collezioni statiche e dinamiche di tipo lineare [CLRS'09(10)]).

Lezione n.09 del 05/10/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte V (Alberi binari; Visite su alberi [CLRS'09(12)]).

Lezione n.10 del 09/10/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte VI (Alberi binari di ricerca e relativi algoritmi [CLRS'09(12)]).

Lezione n.11 del 10/10/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte VII (Alberi binari bilanciati e perfettamente bilanciati; Introduzione agli alberi AVL [CLRS'09(12)]).

Lezione n.12 del 12/10/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte VIII (Alberi AVL [CLRS'09(12)]).

Lezione n.13 del 16/10/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte IX (Alberi AVL [CLRS'09(12)]).

Lezione n.14 del 17/10/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte X (Alberi Red-Black [CLRS'09(12)]).

Lezione n.15 del 19/10/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte XI (Alberi Red-Black [CLRS'09(12)]).

Lezione n.16 del 23/10/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte XII (Alberi Red-Black [CLRS'09(12)]).
- Trasformazioni da algoritmi ricorsivi a algoritmi iterativi.

Lezione n.17 del 24/10/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte XIII (Trasformazioni da algoritmi ricorsivi a algoritmi iterativi).

Lezione n.18 del 26/10/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte XIV (Trasformazioni da algoritmi ricorsivi a algoritmi iterativi).

Lezione n.19 del 30/10/23

Argomenti:

- Strutture Dati per la Gestione delle Collezioni/Insiemi di Dati Parte XV (Trasformazioni da algoritmi ricorsivi a algoritmi iterativi).
- Grafi Parte I (Rappresentazioni concrete e nozioni elementari [CLRS'09(22)]).

Lezione n.20 del 31/10/23

Argomenti:

- Grafi Parte II (Rappresentazioni concrete e nozioni elementari [CLRS'09(22)]).

Lezione n.21 del 02/11/23

Argomenti:

- Grafi Parte III (Visite in ampiezza su grafi [CLRS'09(22)]).

Lezione n.22 del 14/11/23

Argomenti:

- Grafi Parte IV (Visite in ampiezza su grafi [CLRS'09(22)]).

Lezione n.23 del 16/11/23

Argomenti:

- Grafi Parte V (Visite in profondità su grafi [CLRS'09(22)]).

Lezione n.24 del 20/11/23

Argomenti:

- Grafi Parte VI (Visite in profondità su grafi [CLRS'09(22)]).

Lezione n.25 del 21/11/23

Argomenti:

- Grafi Parte VII (Ordinamento topologico [CLRS'09(22)]).

Lezione n.26 del 23/11/23

Argomenti:

- Grafi Parte VIII (Componenti fortemente connesse [CLRS'09(22)]).

Lezione n.27 del 27/11/23

Argomenti:

- Grafi Parte IX (Componenti fortemente connesse [CLRS'09(22)], Grafi pesati [CLRS'09(24)]).

Lezione n.28 del 28/11/23

Argomenti:

- Grafi Parte X (Grafi pesati e Algoritmo di Dijkstra [CLRS'09(24)]).

Lezione n.29 del 30/11/23

Argomenti:

- Grafi Parte XI (Algoritmi di Dijkstra e Bellman-Ford [CLRS'09(24)]).

Lezione n.30 del 04/12/23

Argomenti:

- Algoritmi di Ordinamento Parte I (Richiami di Analisi asintotica [CLRS'09(3)] [GKP'94(9)], Insertion sort [CLRS'09(2.1, 2.2)]).

Lezione n.31 del 05/12/23

Argomenti:

- Algoritmi di Ordinamento Parte II (Insertion sort e Selection sort [CLRS'09(2.1, 2.2)], Alberi completi e relativa rappresentazione vettoriale; Heap [CLRS'09(6.1)]).

Lezione n.32 del 07/12/23

Argomenti:

- Algoritmi di Ordinamento Parte III (Heap e Heap Sort [CLRS'09(6)]).

Lezione n.33 del 11/12/23

Argomenti:

- Algoritmi di Ordinamento Parte IV (Merge sort [CLRS'09(2.3)]; Risoluzione delle equazioni di ricorrenza).

Lezione n.34 del 12/12/23

Argomenti:

- Algoritmi di Ordinamento Parte V (Quick sort [CLRS'09(7)]).

Lezione n.35 del 14/12/23

Argomenti:

- Algoritmi di Ordinamento Parte VI (Limite inferiore ordinamento via confronti [CLRS'09(8.1)]).

Lezione n.36 del 18/12/23

Argomenti:

- Approcci e metodologie di risoluzione dei problemi algoritmici.

Capitolo 1

Fondamenti di Analisi degli Algoritmi

"Ignorance is bliss"

– *Cypher (The Matrix)*

1.1 Definizione Informale di Algoritmo

In prima battuta, che cos'è un algoritmo?

Questa è la prima cosa che bisogna domandarsi.

Una prima approssimazione è la seguente: una sequenza, quindi un insieme ordinato di operazioni, atte alla risoluzione di un certo problema.

Questa è una prima approssimazione che ad un certo livello di astrazione è corretta ma se poi andiamo ad analizzarla ci rendiamo conto che ci sono diverse ambiguità.

Che cosa significa per esempio operazione?

Un'operazione dovrebbe essere qualcosa che trasformi una certa informazione in un'altra informazione.

La seconda informazione dovrebbe essere più vicina, in un senso lato, alla risoluzione rispetto a quella di partenza.

Inoltre immaginiamo debba essere che se l'input è lo stesso anche il risultato dovrebbe essere lo stesso, quindi un'operazione dovrebbe essere qualcosa di funzionale, qualcosa che trasformi un'informazione sempre nello stesso modo.

In effetti esistono varianti che utilizzano una versione randomizzata, cioè in cui c'è qualcosa di non deterministico; ma sono applicazioni con un motivo ben preciso, ad esempio quando si parla di trasporto dell'informazione sicura spesso ci si basa su un approccio non deterministico al problema.

Noi non analizzeremo quella parte, quindi per noi gli algoritmi saranno sempre deterministici, quindi ogni operazione deve prendere dell'informazione e trasformarla in modo deterministico funzionale in un'altra informazione che, ripetiamo, potenzialmente deve portarci verso la soluzione e non allontanarci.

Inoltre l'algoritmo deve essere qualcosa che non dipenda dalla particolare istanza del problema.

L'algoritmo deve essere in grado, più o meno efficientemente, di risolvere una qualunque istanza di un problema.

Quindi deve essere generale: la sequenza di passi da eseguire dipende esclusivamente dal problema generale da risolvere, non dai dati che ne definiscono un'istanza specifica.

Inoltre l'algoritmo deve essere non ambiguo: tutti i passi che definiscono l'algoritmo devono essere non ambigui e chiaramente comprensibili all'esecutore.

Poi che cosa deve essere?

Deve essere corretto.

Un algoritmo è corretto se produce il risultato corretto a fronte di qualsiasi istanza del problema ricevuta in ingresso. La correttezza di un algoritmo può essere stabilita, ad esempio, tramite:

- dimostrazione formale (matematica)
- ispezione informale

Poi deve essere efficiente.

Efficiente cosa significa?

L'efficienza si può solo calcolare con qualcosa di quantitativo non di qualitativo, cioè dobbiamo andare a misurare l'efficienza, cioè la misura delle risorse computazionali che esso impiega per risolvere un problema. Alcuni esempi sono:

- tempo di esecuzione
- memoria impiegata
- altre risorse: banda di comunicazione

Spesso, quando si dice che un algoritmo è efficiente si intende che possa risolvere il problema in tempo polinomiale.

Giusto per dare un'idea ad altissimo livello, tempo polinomiale cosa significa?

Che deve esistere un polinomio in cui il parametro è la dimensione del problema ed entro quel tempo l'algoritmo in questione deve risolvere il suddetto problema.

Noi spesso non lavoreremo in questione di tempo ma la nostra unità di misura sarà l'operazione elementare, cioè la più semplice operazione che il nostro modello computazionale sia grado di eseguire.

Noi conteremo quante operazioni di quel tipo dobbiamo fare per risolvere il problema.

Un algoritmo viene detto polinomiale se esiste un polinomio che ci dà un numero e dice: l'algoritmo entro quel numero risolve il problema.

Purtroppo non tutti i problemi sono risolvibili in tempo polinomiale, ci sono problemi che per essere risolti impiegano più di un qualunque polinomio.

Sono problemi che vengono detti esponenziali e per alcuni di questi è stato dimostrato che non si può fare meglio.

Per un'altra classe di problemi, che vengono detti NP, al momento non è noto se si possa fare meglio di esponenziale ma ciò non significa che gli unici algoritmi per risolvere tali problemi siano esponenziali; al momento matematicamente non sappiamo far di meglio ma non c'è una dimostrazione che non si possa fare meglio.

In questo corso noi affronteremo solo algoritmi polinomiali.

1.2 Definizione di Struttura Dati

Che cos'è invece una struttura dati?

Allora sicuramente una struttura dati presuppone una collezione di dati, non un insieme di dati perché un insieme a livello matematico non precisa l'ordine tra gli elementi, né ammette ripetizioni.

Una struttura dati potrebbe invece voler preservare un certo ordine per qualche motivo.

Quindi di solito si parla di collezione di dati per astrarre.

Alle volte si parla dei cosiddetti multi insieme se vogliamo tenere traccia esclusivamente delle ripetizioni di elementi.

Quindi una collezione di dati sicuramente è una caratteristica fondamentale di una struttura dati, ma una struttura dati non è solo questo.

Sicuramente c'è un'informazione a livello di come i dati verranno poi rappresentati in memoria, e questa informazione viene detta rappresentazione concreta dei dati.

Quindi c'è un livello astratto in cui studiamo la struttura dati che è la collezione di dati; tali dati devono essere memorizzati in qualche modo in un calcolatore, e pertanto c'è un livello concreto di rappresentazione dei dati in memoria e non è detto che il modo di rappresentare tali dati sia unico.

Quindi una struttura dati è una collezione di dati che presuppone una rappresentazione concreta in memoria, ma non solo questo.

Manca ancora un concetto fondamentale riguardo le strutture dati.

Sono le operazioni di accesso ai dati.

Le modalità di accesso e il costo delle operazioni di accesso fanno la differenza tra una struttura dati ed un'altra.

Quindi le strutture dati si differenziano in base alle operazioni di lettura e di accesso e non solo, ma anche in base alle operazioni, nel caso di strutture dati dinamiche, cioè che possono cambiare il loro contenuto, che si possono effettuare per cambiare i dati contenuti all'interno.

Questo in senso astratto. Una struttura dati quindi è una collezione di dati con delle proprietà astratte, con una rappresentazione concreta in memoria e con certe operazioni che devono ovviamente preservare quelli che vengono detti invarianti, cioè le proprietà che noi vogliamo vengano rispettate sui dati.

1.3 Definizione Formale di Algoritmo

Allora, cosa è un algoritmo?

Una sequenza finita di istruzioni elementari eseguibili da un interprete meccanico.

Attenzione che meccanico non significa un ente che l'uomo costruisce basato o su una struttura meccanica vera oppure su un calcolatore.

È una qualunque cosa, compreso l'uomo, che è in grado di eseguire correttamente passo dopo passo, queste istruzioni elementari.

Quindi interprete meccanico qui significa in qualche modo qualunque cosa che sia rappresentabile come un sistema meccanico di esecuzione.

Ciò non toglie che può essere benissimo un sistema biologico, addirittura è stato dimostrato che determinati sistemi biologici sono potenti quanto la macchina di Turing, o meglio l'approssimazione finita di una macchina di Turing.

Quindi abbiamo una sequenza finita di istruzioni, questo verrà poi successivamente anche chiamato programma, eseguita da un interprete meccanico.

Un interprete meccanico è un qualche modello computazionale.

Ma per far cosa?

Per trasformare, oppure atto a trasformare, valori in ingresso, qualunque questo voglia dire, cioè una qualunque informazione, spesso detta informazione in input, in valori in uscita, spesso detta informazione in output.

Quindi un qualunque sistema che tenga in considerazione una sequenza finita di istruzioni, cioè un programma, che possono essere eseguiti da un sistema meccanico, cioè formalizzabile, matematicamente rappresentabile, che viene detto modello computazionale, e che trasforma informazioni in input, in informazioni in output, è detto algoritmo.

Questa formalizzazione è comunque una semiformalizzazione, perché è ancora in linguaggio naturale.

Noi non andremo in maggior dettaglio per quanto riguarda l'algoritmo.

1.4 Modello Computazionale

Allora, qual è un esempio di modello computazionale?

Il primo esempio è stato la macchina di Turing.

Alan Turing è stato il primo a dare un concetto formale di metodi di macchina calcolatrice, ossia di modello computazionale.

Il suo modello non era efficiente, ma indipendentemente dal fatto che potesse essere poco efficiente, era potente. Era potente a sufficienza per poter descrivere un qualunque algoritmo.

Il nostro calcolatore è solo qualcosa di più veloce ma meno potente di una macchina di Turing. Perché?

Perché ha memoria finita, ovviamente.

Allora, una macchina di Turing è un modello computazionale astratto.

Poi ci sposteremo su un altro modello computazionale che rappresenta meglio il nostro calcolatore, che è il modello RAM.

RAM qui non sta ad indicare Random Access Memory, ma sta ad indicare Random Access Machine.

Macchina ad accesso casuale, nel senso con accesso in qualunque possibile posizione della memoria.

E vedremo perché questo si differenzia rispetto al modello di Turing.

1.4.1 Macchina di Turing

Allora, cos'è una macchina di Turing?

Formalmente è una struttura algebrica. Cioè una serie, una sequenza, di insiemi e operazioni matematiche.

$$M = (Q, q_i \in Q, q_f \in Q, \Sigma, \sqcup \in \Sigma, \delta : Q \setminus \{q_f\} \times \Sigma \rightarrow Q \times \Sigma \times \{-1, 0, +1\}) \quad (1.1)$$

Di cosa è fatto?

Abbiamo un insieme Q di stati.

Cosa sono gli stati?

Sono semplicemente le situazioni in cui la macchina si potrà trovare.

Ora, questo insieme è l'insieme degli stati di tutto il sistema. Ed è un insieme finito.

Questo insieme di stati, in effetti, corrisponde nel prossimo modello alle righe del codice del programma, non al programma (vedremo il programma a cosa corrisponderà nella macchina di Turing), ma in un certo senso indicherà quale istruzione stiamo eseguendo.

Attenzione, ripetiamo, gli stati qui stanno a rappresentare in un programma le righe, quante righe di codice, quante operazioni, per essere più precisi, non le operazioni.

Ossia, in un certo senso, rappresentano in un calcolatore gli indirizzi di memoria del nostro programma e muovendosi tra questi indirizzi ci saranno evoluzioni.

In effetti, questa è un'approssimazione non proprio corretta, successivamente daremo un miglioramento di questa approssimazione.

Ciò che sarà scritto negli indirizzi, cioè il programma, viene memorizzato in un'altra parte, è contenuto in un'altra parte di questa struttura.

Per quello che ci interessa, cioè effettuare trasformazioni di un certo insieme di informazioni in input in informazioni in output, ci sarà uno stato speciale, lo stato iniziale, quello in cui l'evoluzione comincia ad essere eseguita, a svilupparsi.

Quindi avremo un elemento q_i di Q , che è lo stato iniziale.

Allo stesso tempo, poiché noi vogliamo che questa macchina trasformi le informazioni in input in informazioni in output, dobbiamo immaginare che da un certo punto ci dica che il lavoro è terminato.

Come fa a comunicarcelo?

Bene, semplicemente si porrà la macchina in uno stato finale q_f di Q , che indica che da questo punto in poi la macchina non farà più nulla perché ha compiuto il suo dovere.

Ovviamente questi sono stati, stato iniziale e stato finale, e pertanto sono elementi dell'insieme degli stati.

A questo punto però, cosa ci manca?

Noi abbiamo detto cosa sono gli stati, ma dobbiamo dire questa macchina cosa fa.

Questo viene spiegato tramite una funzione, chiamata di solito delta (δ), che viene detta Funzione di Transizione. Cioè praticamente è il vero e proprio programma.

La Funzione di Transizione viene detta così perché definisce il transito tra gli stati. Ecco perché viene detta Funzione di Transizione.

Quindi sarà quella parte che conterrà, in una qualche forma, il programma che vogliamo eseguire e che permetterà a questi stati di evolvere.

Ma se questa funzione serve appunto a far evolvere gli stati allo scopo di fare del calcolo, di trasformare dell'informazione in input in informazione in output, deve in qualche modo sapere qual è l'informazione in input. Deve sapere in particolare di cosa è fatta l'informazione in input.

Ogni informazione verrà scritta in un modo che non è detto sia uniforme per tutti.

Attualmente l'informazione nelle nostre reti di comunicazione è sempre, per il momento, trattata tramite un'unità fondamentale di informazione che è il bit.

Per il momento perché quando e se ci saranno nuovi calcolatori di tipo quantistico l'informazione cambierà e molto probabilmente si tramuterà almeno in parte in quello che viene detto il qubit, quanti bit. Ma per il momento uniformemente l'informazione più elementare è il concetto di bit.

Ma a questo livello di astrazione, siamo molto più elevati rispetto al bit, noi dobbiamo dire alla macchina qual è l'alfabeto su cui l'informazione in input viene scritta.

Questo alfabeto è semplicemente un insieme finito di simboli che denoteremo con sigma (Σ).

In questo insieme di simboli c'è un simbolo speciale che è l'assenza di simboli ($_$), cioè lo spazio bianco per capirci, che servirà più che altro a dire dove l'informazione termina, perché l'informazione in input dovrà essere un'informazione finita ovviamente.

Di conseguenza, come facciamo a capire quando finisce l'informazione?

Semplicemente ci sarà un carattere speciale, lo spazio vuoto per capirci. A questo punto abbiamo la possibilità di scrivere come è fatto, in un certo senso, il nostro programma, o meglio, la versione astratta del nostro programma che è formalizzata dalla funzione di transizione.

Questa funzione di transizione è una funzione che prende uno stato in input e prende un simbolo in input, quindi è una funzione che va dal prodotto cartesiano degli stati per l'alfabeto, e ci dice dove spostarci negli stati, cioè qual è il prossimo stato, e trasforma potenzialmente il simbolo in input, quindi ci darà un nuovo simbolo, non necessariamente per forza diverso e poi fa anche un'altra cosa.

Questa macchina da dove legge l'informazione?

Dove scrive l'informazione?

Questo modello assume, quello che viene detto, un nastro di memoria infinita. Ed è questo uno dei motivi per cui tale è un modello astratto: perché non ha limiti di memoria, cosa che ovviamente concretamente non potrebbe avvenire.

Questo nastro si muove sotto una testina di lettura, che può leggere qual è il simbolo attuale e può scrivere il nuovo simbolo.

Questa bobina ovviamente si sposterà, perché altrimenti scrive sempre e legge sempre la stessa cella. E andrà o a destra o a sinistra.

Quindi dobbiamo immaginare come se questa macchina avesse effettivamente un nastro infinito, al cui interno è possibile avere solo elementi che appartengono all'alfabeto scelto.

E in particolare, la maggior parte dei simboli deve essere quello vuoto, perché come detto l'input deve essere finito.

Quindi, nonostante il nastro sia infinito, la parte in cui ci sono simboli diversi dal vuoto, dallo spazio vuoto, è finita, deve essere finita. Non abbiamo un limite su quanta informazione ci sia, ma sicuramente questa informazione, per quanto grande sia, dovrà essere necessariamente finita.

Quindi la maggior parte è vuota.

Quindi avremo questo nastro e dobbiamo immaginare come l'esistenza di una testina che punta al nastro e legge quel simbolo. A seconda dello stato capisce cosa fare, potenzialmente lo riscrive, e poi si sposta di una sola posizione, o a destra o a sinistra.

Formalmente significa che qui decide il prossimo stato, decide se cambiare il simbolo e cosa scrivere, e se spostarsi o meno a destra o a sinistra, o anche rimanere ferma.

I simboli che definiscono lo spostamento della testina sono: $-1, 0, +1$.

- 0 significa nessuno spostamento
- $+1$ significa che andiamo verso destra
- -1 significa che andiamo verso sinistra

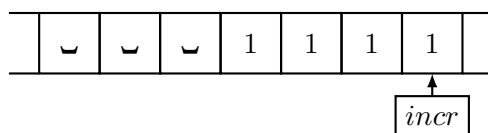
Esempio di una Macchina di Turing

Vediamo ora un esempio banale di una Macchina di Turing, M1, che incrementi di 1 un numero binario in input a partire dal bit meno significativo.

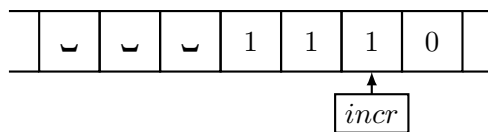
$$M1 = (Q, q_i \in Q, q_f \in Q, \Sigma, \sqcup \in \Sigma, \delta : Q \setminus \{q_f\} \times \Sigma \rightarrow Q \times \Sigma \times \{-1, 0, +1\})$$

- $Q = \{incr, halt\}$
- $q_i = incr$
- $q_f = halt$

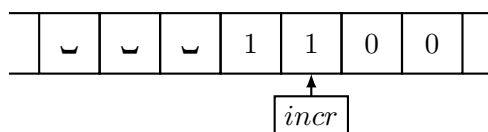
- $\Sigma = \{0, 1, \sqcup\}$
- $\delta(incr, 0) \mapsto (halt, 1, 0)$
- $\delta(incr, 1) \mapsto (incr, 0, -1)$
- $\delta(incr, \sqcup) \mapsto (halt, 1, 0)$



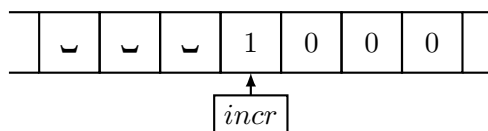
(a) Inizio



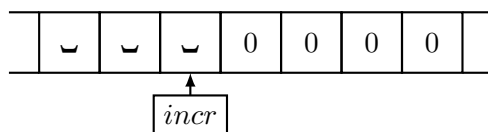
(b) Primo step



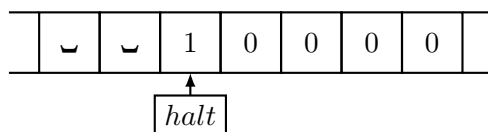
(c) Secondo step



(d) Terzo step



(e) Quarto step



(f) Quinto step

Figura 1.1: Macchina di Turing che incrementa di 1 un numero binario in input

Limiti della Macchina di Turing

Quali sono i limiti di questa macchina?

Cerchiamo di analizzare la struttura algebrica che descrive la macchina di Turing.
Analizziamo un po' questa struttura e vediamo di capire dove possono essere i limiti.

Può essere mai un limite avere un insieme finito di stati?

No, perché un algoritmo è una sequenza finita di passi, come abbiamo detto, e non potrebbe essere infinita.

Quindi sicuramente la limitazione non sta qui.

Ma non sta tanto meno nel fatto di avere uno stato iniziale e uno stato finale, perché i nostri calcolatori sicuramente hanno uno stato iniziale ed uno stato finale; è necessario.

Quindi anche qua non abbiamo una limitazione.

Può essere l'alfabeto dei simboli che è finito?

No, perché addirittura tutte le informazioni che attualmente gestiamo vengono rappresentate con bit, quindi due soli simboli, 0, 1.

Quindi la limitazione non sta qui.

Di certo non sta nel fatto di avere un simbolo che limita quanti simboli significativi abbiamo, perché come al solito l'informazione è finita.

È rimasta la funzione di transizione.

Vediamo però dove abbiamo la limitazione.

Di certo non abbiamo la limitazione nel fatto di analizzare lo stato attuale e decidere cosa fare basandoci sullo stato attuale, perché è quello che farebbe un programma normale in un calcolatore.

Basandosi sugli input?

No, non è quello ovviamente.

Il problema è l'accesso ai dati.

Perché?

Perché l'accesso ai dati in questa macchina può avvenire solo ed esclusivamente in modo sequenziale.

Ci troviamo in una cella, possiamo fare quello che vogliamo basandoci sullo stato in cui siamo e sull'informazione che abbiamo in quella cella.

Ma poi ci possiamo spostare di una cella soltanto, a destra o a sinistra.

E come se noi nel programma fossimo obbligati a leggere nella memoria un certo indirizzo, e poi per forza o quello di prima o quello di dopo. Non come facciamo normalmente, cioè allocando memoria dinamica e utilizzando i puntatori per andare in una zona della memoria che non ha nulla a che fare con quella in cui stavamo prima.

Sembra una sciocchezza, ma è una limitazione terribile di questa macchina a livello di efficienza, che invece le macchine che noi utilizziamo non hanno.

Quindi il problema è di accesso all'informazione troppo locale.

Significa che non abbiamo la possibilità di accedere a una qualunque cella tempo costante conoscendo l'indirizzo della cella a cui vogliamo accedere, a differenza di quello che accade nei calcolatori che usiamo quotidianamente, nei quali queste operazioni di accesso sono indipendenti da quale cella vogliamo leggere.

Ora, si dimostra formalmente, noi non lo faremo, che, assumendo memoria infinita, non c'è macchina più espressiva della macchina di Turing.

Potere espressivo significa cosa e quali sono le possibili trasformazioni che si possono implementare nella macchina.

1.4.2 Modello RAM

L'altro modello, è quello che viene detto modello RAM, (RAM sta per Random Access Machine).

Cioè una macchina che può accedere alla memoria in qualunque cella, indipendentemente da quella letta precedentemente, sempre allo stesso costo.

Non formalizzeremo cos'è una macchina RAM.

Potrebbe essere vista come un insieme di simboli, un nastro di memoria, che però non deve essere visto come nastro, ma come un array infinito di memoria, dove però la funzione di transizione ha una forma diversa in quanto definisce in quale cella leggere ed in quale scrivere.

Questa sarebbe una semi-formalizzazione, ma a noi interessa lavorare con un modello mentale più intuitivo basato sulla macchina RAM.

Ci basterà sapere che possiamo scrivere un programma finito che possiamo formalmente rappresentarlo in notazione matematica, che abbiamo una memoria che noi astrarremo essere infinita, ma concretamente è finita, e che possiamo accedere a qualunque cella di memoria a tempo costante.

Quello che faremo è assumere di avere un modello computazionale di questo tipo, una memoria infinita ma di cui useremo una parte finita e a cui possiamo accedere ad ogni cella con lo stesso tempo, quindi con un costo fissato e in cui noi faremo anche un'altra cosa: qualunque operazione andremo ad eseguire, che possa essere una somma, una moltiplicazione, un test condizionale, per noi avranno tutte costo fissato e costante; quindi la nostra unità di misura sarà il numero di istruzioni.

1.5 Accenno alle Classi di Complessità

Siano $f: \mathbb{N} \rightarrow \mathbb{N}$ e $g: \mathbb{N} \rightarrow \mathbb{N}$ funzioni.

Per noi le funzioni sono tutte funzioni discrete.

1.5.1 Definizione di O-grande

Diremo che una funzione $f(n)$ è O-grande di una funzione $g(n)$ intuitivamente se g è un limite superiore ad f .

Tuttavia non è che deve essere punto per punto un limite superiore, lo sarà da un certo punto in poi; e non solo questo.

Non è detto neanche che debba veramente essere un limite superiore, ma che una sua scalatura, nel senso una sua versione moltiplicata, lo sia.

Formalmente.

$$f(n) = O(g(n)) : \iff \exists c \in \mathbb{R}^+, \exists n_0 \in \mathbb{N} (\forall n \in \mathbb{N}_{\geq n_0} (f(n) \leq c \cdot g(n))) \quad (1.2)$$

1.5.2 Definizione di Omega

Identica cosa per Omega (Ω), ma considerando il limite inferiore

Formalmente.

$$f(n) = \Omega(g(n)) : \iff \exists c \in \mathbb{R}^+, \exists n_0 \in \mathbb{N} (\forall n \in \mathbb{N}_{\geq n_0} (c \cdot g(n) \leq f(n))) \quad (1.3)$$

1.5.3 Definizione di Theta

Che cosa significa theta (Θ)?

Significa semplicemente che la funzione che vogliamo utilizzare può essere sia un limite inferiore sia un limite superiore, ovviamente riscalato in modo diverso.

Formalmente.

$$f(n) = \Theta(g(n)) : \iff \exists c_1, c_2 \in \mathbb{R}^+, \exists n_0 \in \mathbb{N} (\forall n \in \mathbb{N}_{\geq n_0} (c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n))) \quad (1.4)$$

1.5.4 Proprietà

Risulta quindi ovvio che

$$f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \wedge f(n) = \Omega(g(n)) \quad (1.5)$$

1.6 Pseudo-Codice

Da ora in poi useremo uno pseudo-codice; cioè un codice di programmazione che non è proprio di un particolare linguaggio, tipo C, Java, ecc. . . , ma che ovviamente saremo in grado di riconoscere e di capire in modo intuitivo.

Come di consueto nello pseudo-codice troveremo i comuni costrutti condizionali.

```
if (Condition) then
```

```
  ...
```

```
  Instruction
```

```
  ...
```

```
if (Condition) then
```

```
  ...
```

```
  Instruction
```

```
  ...
```

```
else
```

```
  ...
```

```
  Instruction
```

```
  ...
```

```
if (Condition1) then
```

```
  ...
```

```
  Instruction
```

```
  ...
```

```
else if (Condition2) then
```

```
  ...
```

```
  Instruction
```

```
  ...
```

```
else
```

```
  ...
```

```
  Instruction
```

```
  ...
```

```
switch (Condition) do
| case (Case1) do
|   ...
|   Instruction
|   ...
| ...
| case (CaseN) do
|   ...
|   Instruction
|   ...
| otherwise do
|   ...
|   Instruction
|   ...
|
```

Ovviamente, troveremo anche i comuni costrutti iterativi.

```
for (Condition) do
| ...
| Instruction
| ...
|
```

```
foreach (Condition) do
| ...
| Instruction
| ...
|
```

```
loop (Condition) times
| ...
| Instruction
| ...
|
```

```
while (Condition) do
| ...
| Instruction
| ...
|
```

```
repeat
|  ...
|  Instruction
|  ...
until (NOT Condition)
```

Eventuali altri dettagli dello pseudo-codice saranno esplicitati se necessario nel testo dove serve.

Capitolo 2

Esempi di Algoritmi e Relativa Analisi

"Do you think that's air you're breathing now?"

– Morpheus (The Matrix)

2.1 Sequenza di Fibonacci

Vediamo ora che anche all'interno dello stesso modello e anche per fare la stessa identica computazione possiamo scrivere programmi diversi che hanno efficienza a livello di tempo, cioè il numero di istruzioni, diverse.

Un primo esempio molto semplice di calcolo è calcolare il numero di Fibonacci.

2.1.1 Problema

Il problema è il seguente: dato un numero naturale n , calcolare l' n -esimo numero di Fibonacci.

La sequenza di Fibonacci, $\text{Fib}(n)$, è definita con:

$F_0 = 1$ e $F_1 = 1$ (questa è una delle due definizioni, c'è una definizione che parte con $F_0 = 0$ e $F_1 = 1$)
e poi $F_n = F_{n-1} + F_{n-2}$ con $n \geq 2$

Allora vogliamo scrivere un programma che calcoli questo e che sia ovviamente quanto più efficiente possibile, cioè che impieghi il numero minore di istruzioni.

2.1.2 Algoritmo: Fibonacci Ricorsivo

Vediamo un primo pseudo-codice per un programma che risolva questo problema.

Ovviamente l'informazione in input che stiamo trasformando, seguendo la definizione di algoritmo, è un numero naturale e lo vogliamo trasformare in un altro numero naturale, che in questo caso è l' n -esimo numero di Fibonacci.

La prima cosa che andremo a fare è la signature, la firma, cioè il prototipo della funzione.

In questo caso il nome della funzione sarà `fib`. `fib` è la funzione che prende in input un numero naturale e restituisce un altro numero naturale.

A questo punto, una volta che abbiamo la signature, andremo a scrivere il programma vero e proprio.

Il primo programma in particolare sarà ricorsivo, poi vedremo altri diversi programmi che risolvono lo stesso problema non ricorsivi.

La prima istruzione dovrà capire se siamo in un caso base o se siamo nel caso ricorsivo.

Quindi con un banale test: se $n \leq 1$ (che significa $n = 0$ o $n = 1$ visto che siamo nel mondo dei numeri naturali) restituiamo 1, seguendo la definizione della sequenza di Fibonacci.

Altrimenti dobbiamo calcolare il valore, facendoci sempre guidare dalla definizione matematica.

Andremo a calcolare il valore che inseriremo in una cella di memoria chiamata `a`, che è il valore del nostro programma su un input inferiore, $n - 1$.

Andremo a calcolare il valore che inseriremo in una cella di memoria chiamata `b`, che è il valore del nostro programma su un input inferiore, $n - 2$.

Infine restituiremo `a + b`.

Algoritmo 1: Calcolo della sequenza di Fibonacci versione ricorsiva.

```

Signature fib:  $\mathbb{N} \rightarrow \mathbb{N}$ 
function fib(n)
    // Se sono nel caso base
1   if (n <= 1) then
2       return 1
3   else
4       // Altrimenti calcolo il valore su input n-1 ed n-2
5       a ← fib(n-1)
6       b ← fib(n-2)
       // ... e ne restituisco la somma
       return a + b

```

I programmi ricorsivi in prima battuta non manifestano direttamente il loro costo perché il costo dell'intero algoritmo dipende dal costo dello stesso algoritmo su qualcosa di più piccolo. Vedremo più avanti come si farà

a calcolare il costo di un programma ricorsivo, ora daremo un'intuizione per capire quanto possa costare un algoritmo del genere.

Noi abbiamo delle istruzioni che consideriamo a tempo costante, per semplicità facciamo che ogni istruzione costi tempo 1.

Il costo del tempo di Fibonacci, $T_{\text{fib}(n)}$, cambia a seconda dell'input, quindi non possiamo avere un'unica espressione per il tempo ma dovremo fare un'analisi per casi:

- Se $n \leq 1$ eseguiamo 2 istruzioni, quindi diciamo costa 2.
- Se $n > 1$ eseguiamo 4 istruzioni atomiche più il costo di `fib` su input $n - 1$ più il costo di `fib` su input $n - 2$.

Se uno volesse essere preciso dovrebbe calcolare il costo esatto di tutte le operazioni.

In realtà non cambia nulla, se è un numero fissato per noi sarà una costante, che potremmo, per essere più formali, indicare con c .

C'è un approccio per calcolare queste equazioni, dette equazioni di ricorrenza.

Si può osservare che questa funzione cresce praticamente come i numeri di Fibonacci, cioè che:

$$\text{fib}(n) \leq T_{\text{fib}(n)}$$

Come lo si può osservare?

Banalmente facendo una semplicissima prova per induzione.

Il numero di Fibonacci cresce esponenzialmente ed è uno dei pochi casi in cui c'è una formula chiusa, chiamata formula di Binet, definita come:

$$\text{fib}(n) = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}} \quad (2.1)$$

Dimostriamo la seguente proposizione per induzione

Proposizione 2.1.1 (Tempo algoritmo ricorsivo `fib`).

La funzione di tempo dell'algoritmo ricorsivo di Fibonacci cresce come i numeri di Fibonacci.

Formalmente: $\text{fib}(n) \leq T_{\text{fib}(n)}$

Dimostrazione.

I casi base sono banali.

$$1 = \text{fib}(0) = \text{fib}(1) < T_{\text{fib}}(0) = T_{\text{fib}}(1) = 2 \quad (2.2)$$

Consideriamo vero l'assunto fino ad un generico n e valutiamo per $n+1$.

$$\text{fib}(n+1) = \text{fib}(n) + \text{fib}(n-1) \leq T_{\text{fib}}(n) + T_{\text{fib}}(n-1) \leq T_{\text{fib}}(n+1) \quad (2.3)$$

□

Quindi questo algoritmo per calcolare il numero di Fibonacci è particolarmente dispendioso, in quanto cresce esponenzialmente al crescere di n ($n \rightarrow +\infty$).

Complessità dell'Algoritmo

In altri termini, questo algoritmo appartiene alla classe di complessità di $\Theta(a^n)$, ovviamente con $a > 1$.

2.1.3 Algoritmo: Fibonacci Iterativo v.01

Si può fare di meglio per eseguire esattamente lo stesso compito?

In questo caso sì.

Vediamo quest'altro programma. La signature è esattamente la stessa.

Algoritmo 2: Calcolo della sequenza di Fibonacci prima versione iterativa.

Signature fib: $\mathbb{N} \rightarrow \mathbb{N}$

function fib(n)

```

    // assegno i valori iniziali della sequenza
    // a conterrà il valore corrente
    // b il valore precedente al corrente
1   a, b ← 1
2   while (n >= 2) do
    // calcolo il valore corrente della sequenza
3       c ← a + b
    // assegno a b il vecchio valore corrente
4       b ← a
    // assegno ad a il valore corrente
5       a ← c
    // decremento n
6       n ← n - 1
    // restituisco il valore corrente della sequenza
7   return a

```

Questo programma, a differenza di quello di prima, che implementava l'equazione di ricorrenza dei numeri di Fibonacci praticamente tale e quale, risolve esattamente lo stesso problema ma sfrutta informazioni che non sono esplicite nella definizione ma che con un minimo di ragionamento si possono osservare e sfruttare per ottenere algoritmi più efficienti.

Potremmo scrivere una dimostrazione matematica che dimostri che questo programma fa esattamente quello che serve, ma per i nostri scopi è superfluo; ma andiamo a cercare di capire quanto costa questo nuovo algoritmo.

Capire quante volte i cicli **while** vengano eseguiti in generale non è immediatamente evidente, a volte è estremamente difficile capire quante volte vengono eseguiti; ma in questo caso specifico è molto semplice perché il test lo facciamo rispetto a n e ad ogni esecuzione lo decrementiamo, quindi lo eseguiamo $n - 1$ volte.

Quindi complessivamente il tempo di questa versione di **fib** per un certo input, $T_{\text{fib}(n)}$ sarà:

$$T_{\text{fib}(n)} = \begin{cases} 4 & \text{se } n \leq 1 \\ 3 + 5(n - 1) & \text{se } n > 1 \end{cases}$$

Ne consegue che l'algoritmo iterativo di Fibonacci (v.1) è molto più efficiente ¹ di quello ricorsivo, poiché la sua funzione di tempo cresce linearmente al crescere di n ($n \rightarrow +\infty$).

Complessità dell'Algoritmo

Questo algoritmo appartiene alla classe di complessità di $\Theta(n)$.

Quindi questo algoritmo risulta estremamente più efficiente della versione ricorsiva.

¹*Analisi I.* Le funzioni esponenziali con base > 1 crescono asintoticamente più velocemente di qualsiasi polinomio

2.1.4 Excursus sugli Algoritmi

Sottolineiamo che tutti i problemi, anche di natura diversa, si possono formalizzare in un linguaggio più o meno formale, e pertanto entrano a far parte di una formalizzazione matematica.

Abbiamo fatto vedere due diversi algoritmi, che risolvono esattamente lo stesso problema, questa è dimostrazione del fatto che per un certo problema non è detto che esista un'unica soluzione, o un unico modo di calcolare una certa soluzione particolare.

Abbiamo fatto osservare che per calcolare il numero di Fibonacci, seguendo l'algoritmo ricorsivo, era necessario effettuare un numero di operazioni esponenziale.

Abbiamo detto che a noi interessa contare il numero di operazioni come stima del tempo, perché assumiamo tutte le operazioni atomiche, come operazioni a tempo costante, tutte con lo stesso tempo.

Abbiamo anche spiegato il perché abbiamo fatto questa assunzione che è ovviamente una assunzione semplificativa, ma tuttavia sufficiente per quello che ci interessa.

L'unica cosa che noi trattiamo in questo corso sono algoritmi corretti, non algoritmi che risolvono il problema in modo approssimativo, o approssimato per essere più precisi.

Nel nostro corso noi andremo a guardare soltanto problemi semplici di cui è nota ovviamente una soluzione esatta. Quindi i nostri algoritmi devono essere tutti quanti corretti.

Una volta che abbiamo quindi messo da parte la correttezza o la precisione, perché gli algoritmi che affronteremo devono essere tutti corretti, l'unico modo per discriminare tra di essi è quante risorse usino gli algoritmi per risolvere il problema.

E nel particolare caso quello che si è dovuto andare a stimare è il tempo, cioè il numero di operazioni che dobbiamo effettuare per poter risolvere il problema.

Non abbiamo fatto caso a quanta memoria utilizzino gli algoritmi proposti. Andando ad analizzare, l'algoritmo iterativo ha bisogno di tre variabili: a , b e c .

Quindi, memoria costante, non dipende dall'input.

Il tempo ovviamente crescerà linearmente, ma la memoria occupata sarà sempre la stessa.

Una cosa diversa, invece, è la memoria occupata dall'algoritmo ricorsivo che impiega non solo un tempo esponenziale ma anche memoria che non è costante.

Perché?

Perché è una funzione ricorsiva, e le funzioni ricorsive hanno bisogno di uno stack, il cosiddetto stack d'attivazione. Più chiamate innestate vengono effettuate più profondo sarà lo stack.

Quindi algoritmi diversi possono essere diversi, a parità di soluzione del problema, per tempo, per spazio, in generale per le risorse di cui questi hanno bisogno.

Ci sono addirittura algoritmi che per migliorare una risorsa peggiorano in un'altra, e allora lì la scelta di quale algoritmo usare non è ovvia.

Starà poi a noi, nel momento in cui andiamo a risolvere un problema, capire qual è la scelta giusta.

Ed è per questo che questo corso è importante, perché ci deve far capire che non dobbiamo andare alla cieca,

dobbiamo studiare un certo problema, affrontarlo con conoscenza, con competenza e fare scelte oculate a seconda dell'ambito.

2.1.5 Algoritmo: Fibonacci Iterativo v.02

Consideriamo l'algoritmo iterativo.

Si può fare meglio?

E se sì, di quanto si può fare meglio?

Un altro algoritmo equivalente è il seguente.

Algoritmo 3: Calcolo della sequenza di Fibonacci seconda versione iterativa.

Signature `fib`: $\mathbb{N} \rightarrow \mathbb{N}$

function `fib(n)`

```

    // assegno i valori iniziali della sequenza
    // a conterrà il valore corrente
    // b il valore precedente al corrente
1   a, b ← 1
2   while (n >= 2) do
    // calcolo il valore corrente della sequenza
3     a ← a + b
    // assegno a b il vecchio valore corrente
4     b ← a - b
    // decremento n
5     n ← n - 1
    // restituisco il valore corrente della sequenza
6   return a

```

Quello che cambia è il corpo del ciclo `while`.

Allora, la prima cosa che dobbiamo osservare è se risolve effettivamente il problema che vogliamo affrontare.

Se $n < 2$ è ovvio.

L'invariante qui è che in a e b siano presenti due valori della sequenza di Fibonacci e sono esattamente i due valori che precedono quello che vogliamo calcolare, dove a è il più grande dei due.

Se andiamo a porre in a la somma di a e b , ovviamente stiamo sommando i precedenti due valori, quindi il valore di a è esattamente il valore che ci interessa.

Se riusciamo a osservare che anche b è il valore che precede, dimostriamo la correttezza dell'algoritmo.

Ma se a è il vecchio a più b , e gli sottraiamo b , in b andiamo a mettere il vecchio a , cioè:

$$a = a + b \implies b = a - b = a + b - b = a$$

Quindi l'algoritmo è corretto.

Tuttavia è evidente che non utilizza una variabile rispetto all'algoritmo iterativo precedente, c .

Quindi in memoria è migliore, ma di quanto?

Di una cella di memoria.

E a livello di tempo è migliore?

È vero che sarà migliore se andiamo a contare esattamente il numero di operazioni.

Tuttavia hanno lo stesso tipo di andamento lineare dal punto di vista asintotico, e pertanto dal punto di vista della complessità possono considerarsi equivalenti.

2.2 Conteggio Coppie

Per il prossimo esempio, vedremo quattro algoritmi diversi che risolvono esattamente lo stesso problema e che hanno tutti andamenti diversi di ordine asintotico.

Quello che vogliamo mostrare è che per uno stesso problema si possono avere algoritmi che coprono un'intera scala di andamenti.

2.2.1 Problema

Il problema è il seguente: dato un numero naturale n , vogliamo calcolare quante sono le coppie di numeri naturali (i, j) con $i \leq j \leq n$. Quindi calcolare la cardinalità dell'insieme $A = \{(i, j) \in \mathbb{N} \times \mathbb{N} \mid i \leq j \leq n\}$.

Questo è un problema puramente matematico.

Esempio

Per $n = 3$ le coppie che appartengono all'insieme A sono:

$$A = \left\{ \begin{array}{l} (0, 0), (0, 1), (0, 2), (0, 3) \\ (1, 1), (1, 2), (1, 3) \\ (2, 2), (2, 3) \\ (3, 3) \end{array} \right\}$$

In questo caso un esempio è sufficiente in generale per capire l'andamento. Si tratta dei numeri triangolari. Complessivamente:

$$|A| = \sum_{i=0}^n (i+1) = \sum_{j=1}^{n+1} j = \frac{(n+1) \cdot (n+2)}{2} \quad (2.4)$$

Il passaggio dalla prima serie alla seconda è stato effettuato per sostituzione. Abbiamo sostituito: $j = i + 1$.

Gli estremi di j poi li abbiamo calcolati semplicemente sostituendo ad i i corrispettivi valori estremi:

$$\begin{cases} \text{se } i = 0, \text{ allora } j = 1; \\ \text{se } i = n, \text{ allora } j = n + 1; \end{cases}$$

La serie trovata è nota, infatti basta utilizzare la formula di Gauss² (facendo attenzione però che si sta sommando i primi $n + 1$ numeri).

Vogliamo far vedere come approcciare, diciamo in modo algoritmico, la soluzione di questo problema.

²Analisi I. La somma dei primi n -numeri naturali è uguale a: $\frac{n \cdot (n+1)}{2}$

2.2.2 Algoritmo: Count v.01

La prima cosa che possiamo immaginare è di andare ad enumerare tutte le coppie di nostro interesse e per ognuna di queste semplicemente tenere traccia in un contatore di quante ne abbiamo create.

Algoritmo 4: Conteggio delle coppie prima versione.

Signature count: $\mathbb{N} \rightarrow \mathbb{N}$

```

function count(n)
    // inizializzo il contatore
1   c ← 0
    // costruisco tutte le possibili coppie
2   for (i from 0 to n) do
3       for (j from 0 to n) do
4           // se la coppia verifica la condizione richiesta
           if (i <= j) then
5               // incremento il contatore
               c ← c + 1
        // restituisco il contatore
6   return c

```

Quello che fa questo algoritmo è quindi, mediante due cicli **for** innestati, generare tutte le coppie di numeri naturali (i, j) con $i, j \leq n$ e ad ogni generazione tramite un **if** controlliamo se la coppia generata verifica la proprietà $i \leq j$ e nel caso incrementiamo un contatore, c (settato inizialmente a 0).

Infine, ovviamente, restituiamo il numero di coppie.

È chiaro che l'algoritmo sia corretto in quanto traduzione uno a uno della formalizzazione matematica del problema posto.

Cerchiamo di calcolare quanto costi questo algoritmo.

Sintetizzando il calcolo del tempo, osserviamo:

$$T_{\text{Count}_{v.1}}(n) = 2 + (n+1)(1 + 4(n+1)) = 2 + (n+1)(5 + 4n) = 2 + 5n + 4n^2 + 5 + 4n = 4n^2 + 9n + 7 \quad (2.5)$$

Dunque, la funzione del tempo dell'algoritmo è espressa da un polinomio di secondo grado.

Complessità dell'Algoritmo

Risulta pertanto ovvio che tale algoritmo appartiene alla classe di complessità di $\Theta(n^2)$.

2.2.3 Algoritmo: Count v.02

Attenzione. Quando abbiamo un problema da affrontare non bisogna porsi immediatamente nell'ottica di pensare di ottimizzare subito.

È errata e deleteria e ci porta a creare codice di basso livello che poi appena dovremo modificare sono dolori. Bisogna approssciare il problema con semplicità migliorandolo man mano.

È possibile fare di meglio?

Chiaramente sì.

La proprietà da rispettare $i \leq j \leq n$ è facile da codificare in altro modo all'interno dei cicli `for`.

Come?

Se sappiamo che j non può essere più piccolo di i tanto vale far partire j direttamente da i .

Algoritmo 5: Conteggio delle coppie seconda versione.

Signature count: $\mathbb{N} \rightarrow \mathbb{N}$

```

function count(n)
    // inizializzo il contatore
1   c ← 0
    // costruisco tutte le possibili coppie che
    // verificano la proprietà richiesta
    // facendo partire l'indice j direttamente da i
2   for (i from 0 to n) do
3       for (j from i to n) do
4           // incremento il contatore
           c ← c + 1
    // restituisco il contatore
5   return c

```

È evidente che si tratti di un algoritmo corretto.

Questa ottimizzazione fa davvero qualcosa di tangibile? Nel senso: cambia davvero l'essenza del costo dell'algoritmo?

Vedremo che la risposta è no.

Cerchiamo di calcolare quanto costi questo algoritmo.

Sintetizzando il calcolo del tempo, osserviamo:

$$\begin{aligned}
 T_{\text{Count}_{v.2}}(n) &= 2 + \sum_{i=0}^n (2(n-i+1) + 1) = 2 + \sum_{i=0}^n (2n - 2i + 3) = 2 + \sum_{i=0}^n (2n + 3) - 2 \sum_{i=0}^n i = \\
 &= 2 + (n+1)(2n+3) - n(n+1)
 \end{aligned} \tag{2.6}$$

Dai cui segue:

$$T_{Count_{v.2}} = n^2 + 4n + 5 \quad (2.7)$$

Confrontando la funzione di tempo appena calcolata con quella del precedente algoritmo (2.5) possiamo notare che, come volevamo, la versione 2 risulta essere più efficiente, nonostante ciò asintoticamente non vi sono differenze tra i due algoritmi proposti.

Complessità dell'Algoritmo

Risulta pertanto ovvio che tale algoritmo appartiene alla classe di complessità di $\Theta(n^2)$.

2.2.4 Algoritmo: Count v.03

Questa è l'unica ottimizzazione che si può fare dell'algoritmo?

In verità, no. Si può far di meglio.

Se andiamo a guardare il **for** più interno del precedente algoritmo osserviamo che l'incremento della variabile c viene eseguito per ogni iterazione del suddetto ciclo **for**; cioè $n - i + 1$ volte.

Ma se questo **for** viene eseguito $n + 1$ volte e l'unica cosa che fa è incrementare una variabile, significa che il valore dopo l'intero **for** della variabile c rispetto al valore che aveva prima sarà maggiore esattamente di $n - i + 1$.

Di conseguenza possiamo utilizzare direttamente l'operazione atomica di somma per fare una cosa che nel precedente algoritmo viene fatta con un ciclo *for*.

Quindi applichiamo direttamente la definizione di somma invece di definire la somma come incrementi.

Di conseguenza un algoritmo equivalente sarà il seguente.

Algoritmo 6: Conteggio delle coppie terza versione.

Signature count: $\mathbb{N} \rightarrow \mathbb{N}$

function count(n)

```

1  // inizializzo il contatore
   c ← 0
2  for (i from 0 to n) do
   // incremento il contatore
   // del fattore riferito all'indice i
3  c ← c + n - i + 1
   // restituisco il contatore
4  return c
```

È evidente che si tratti di un algoritmo corretto.

Ora, però, c'è una differenza non nel comportamento ma nel costo dell'algoritmo. Poiché abbiamo eliminato un ciclo **for**, abbiamo eliminato un numero dipendente da i di iterazioni, quindi di istruzioni, e l'abbiamo sostituito con un'unica istruzione che consideriamo a tempo costante.

Cerchiamo di calcolare quanto costi questo algoritmo.

Sintetizzando il calcolo del tempo, osserviamo:

$$T_{Count_{v.3}} = 2 + 2(n + 1) = 2n + 4 \quad (2.8)$$

Come si può facilmente osservare la funzione di tempo della versione 3 è nettamente migliore delle precedenti.

Complessità dell'Algoritmo

Risulta pertanto ovvio che tale algoritmo appartiene alla classe di complessità di $\Theta(n)$.

2.2.5 Algoritmo: Count v.04

Si può fare di meglio?

In questo caso ancora sì.

Perché così come nel caso precedente ci siamo resi conto che il ciclo interno era un incremento della variabile un certo numero di volte, qui non è un incremento costante, ma è comunque un incremento dipendente fatto un certo numero di volte.

Abbiamo già visto che basta applicare la formula di Gauss per calcolare il valore cercato.

Una quarta versione equivalente dell'algoritmo è la seguente.

Algoritmo 7: Conteggio delle coppie quarta versione.

Signature count: $\mathbb{N} \rightarrow \mathbb{N}$

function count(n)

```

1  | // formula di Gauss
   | return (n+1)*(n+2)/2

```

È evidente che si tratti di un algoritmo corretto.

È chiaro che la funzione tempo della versione 4 è indipendente dall'input, infatti abbiamo:

$$T_{Count_{v.4}} = 1 \quad (2.9)$$

Complessità dell'Algoritmo

Risulta pertanto ovvio che tale algoritmo appartiene alla classe di complessità di $\Theta(1)$.

2.2.6 Conclusioni

Quindi da un algoritmo quadratico, siamo passati ad un algoritmo ancora quadratico, un po' meglio come costante moltiplicativa, ma sinteticamente identico, ad un algoritmo lineare, fino ad un algoritmo addirittura a tempo costante.

2.3 Max Subsequence Sum (MSS)

Completeremo questa prima introduzione all'analisi degli algoritmi con un problema che, sebbene molto semplice, ha un suo motivo d'essere in quanto viene anche utilizzato in determinati ambiti.

Scriveremo tre versioni dell'algoritmo, con complessità man mano migliori.

2.3.1 Problema

Max subsequence sum, questo è il nome del problema.

Come input abbiamo un array di interi.

Spesso l'array lo chiameremo A .

Ovviamente stiamo assumendo che l'informazione relativa all'array includa anche quanti elementi ci sono nell'array.

Lo dobbiamo vedere come una struttura dati che contiene i dati e l'informazione su quanti dati ci sono.

Immaginiamo che A sia la coppia puntatore all'array, dimensione; $A = (A, n)$.

L'output è, come dice il nome del problema, la massima somma degli elementi di una sottosequenza contigua.

Quindi l'output è $\left(\max_{0 \leq i \leq j < n} \sum_{k=i}^j A[k] \right)$.

Immaginiamo per semplicità di avere almeno un numero non negativo nell'array.

Esempio

Per rendere più chiaro il problema facciamo un esempio con $n = 9$:

pos	0	1	2	3	4	5	6	7	8
$A[\text{pos}]$	-2	1	-3	4	-1	2	1	-5	4

$$(i, j) = (0, 1) \mapsto -2 + 1 = -1$$

...

$$(i, j) = (3, 6) \mapsto 4 - 1 + 2 + 1 = 6;$$

...

$$(i, j) = (5, 6) \mapsto 2 + 1 = 3$$

...

2.3.2 Algoritmo: MSS v.01

Il problema matematico ci dà già quasi completamente un'idea di un potenziale algoritmo.

È chiaro che se dobbiamo prendere tutte le possibili sottosequenze dovremo andare ad iterare sulle sottosequenze; quindi iteriamo sugli indici che ci vanno ad identificare la sotto sequenza. Visto che j deve essere maggiore di i si tratta due indici semi-indipendenti.

Abbiamo già visto qualcosa di simile affrontando il problema precedente.

Osserviamo, infatti, che il numero di sottosequenze contigue è identico al numero di coppie del problema precedente con input n , dove n è in questo caso la lunghezza dell'array A .

Andremo a generare quindi tutte le possibili sottosequenze, per ognuna di loro andiamo a calcolare la somma, e terremo traccia del valore massimo.

L'algoritmo è il seguente.

Algoritmo 8: Max subsequence sum prima versione.

Signature $mss: \mathbb{Z}^* \times \mathbb{N} \rightarrow \mathbb{N}$

```

function mss((A,n))
    // setto il massimo
1   m ← −∞
    // genero tutte le possibili sottosequenze
2   for (i from 0 to n-1) do
3       for (j from i to n-1) do
4           // per ciascuna sottosequenza setto la somma a 0
           s ← 0
           // scorro la sequenza
5           for (k from i to j) do
6               // incremento la somma
               s ← s + A[k]
           // setto il massimo attuale
7       m ← max(m,s)
    // restituisco il massimo
8   return m

```

Un array di solito matematicamente si indica con la potenza star-esima dell'alfabeto degli elementi, dove star indica tutti i possibili numeri naturali; quindi una sequenza finita ma di cui non è nota a priori la lunghezza. Se è di una dimensione prefissata, d , con la potenza d -esima.

Osserviamo che non stiamo ammettendo per semplicità le sottosequenze vuote.

Poiché i valori dell'array sono interi settiamo a $-\infty$ la variabile che conterrà il valore massimo all'inizio del

programma.

Non abbiamo fatto altro che prendere la definizione e tradurla in un codice che fa praticamente esattamente la stessa cosa.

La genesi di tutte le sottosequenze contigue è stata fatta per mezzo di due `for`, con un terzo `for` viene effettuata la somma.

Una volta che c'è un valore di somma vediamo se è migliorativo rispetto al valore attuale del massimo, e nel caso lo sostituiamo.

Per valutare il massimo tra due interi è stata utilizzata una funzione `max`.

Ovviamente prima di partire con l'analisi della complessità bisognerebbe dimostrare la correttezza di quest'algoritmo.

La dimostrazione è una dimostrazione per induzione, che non faremo. Assumiamo pertanto l'algoritmo corretto.

Il calcolo della complessità dei cicli, a differenza del modo classico di andare dall'esterno verso l'interno, è quello di andare dal centro verso l'esterno.

Un altro modo per vedere la complessità di un algoritmo in modo leggermente più impreciso è vedere quanti cicli sono innestati. Il caso peggiore è che i cicli siano tra di loro tutti indipendenti, cioè ognuno faccia un numero di iterazioni indipendente dagli altri.

Quindi in prima battuta, una prima approssimazione per capire il caso peggiore di un algoritmo è vedere quanti sono i cicli `for` innestati e fare il prodotto di quante volte facciamo iterazioni; la complessità dell'algoritmo potrebbe essere migliore, anche asintoticamente, ma quello è sicuramente un primo limite superiore.

Complessità dell'Algoritmo

Essendoci in questo caso specifico tre `for` innestati possiamo dire che l'algoritmo appartiene alla classe di complessità $\Theta(n^3)$.

2.3.3 Algoritmo: MSS v.02

Possiamo far meglio?

Vediamo come poter sfruttare le informazioni sul problema e trovare una versione migliore.

Prima osservazione.

Andiamo a guardare il calcolo della somma interna. La somma interna è una somma consecutiva di elementi.

Cosa hanno in comune la somma di una sottosequenza di inizio i e fine j e la somma di una sottosequenza di inizio i e fine $j + 1$?

Chiaramente tutti gli elementi che vanno da i fino a j .

Formalmente, $Sum[i, j + 1] = Sum[i, j] + A[j + 1]$.

L'algoritmo precedente ogni volta che cambia j ricomincia la somma da 0, non sfruttando tale osservazione.

Utilizzando questa osservazione, l'algoritmo è il seguente.

Algoritmo 9: Max subsequence sum seconda versione.

Signature $mss: \mathbb{Z}^* \times \mathbb{N} \rightarrow \mathbb{N}$

```

function  $mss((A, n))$ 
    // setto il massimo
1    $m \leftarrow -\infty$ 
    // per ogni sottosequenza con un certo inizio
2   for ( $i$  from 0 to  $n-1$ ) do
        // resetto la somma a 0
3        $s \leftarrow 0$ 
        // scorro l'array a partire dall'inizio relativo
4       for ( $j$  from  $i$  to  $n-1$ ) do
            // incremento la somma
5            $s \leftarrow s + A[j]$ 
            // setto il massimo attuale
6            $m \leftarrow \max(m, s)$ 
    // restituisco il massimo
7   return  $m$ 

```

Una volta assunto che l'algoritmo precedente è corretto diventa evidente che si tratti di un algoritmo corretto.

Siamo riusciti a recuperare un ordine di grandezza dalla complessità con questa semplice osservazione.

Un altro modo per vederlo più velocemente senza fare calcoli è semplicemente osservare che l'algoritmo in questione ha solo due cicli **for**.

Complessità dell'Algoritmo

Per quanto detto prima l'algoritmo potrà essere al più quadratico, ovvero appartiene alla classe di complessità $\Theta(n^2)$.

2.3.4 Algoritmo: MSS v.03

Si può far meglio?

Decisamente sì.

In effetti c'è una terza versione dell'algoritmo. In questo caso il ragionamento, è comunque molto semplice, però non è così ovvio.

Permette di eliminare un ulteriore ciclo `for`.

Uno potrebbe obiettare "come faccio ad eliminarlo visto che le sequenze sono sempre comunque in totale quadratiche rispetto alla lunghezza dell'array?".

In effetti non serve andare ad esplorare tutte le sequenze.

Se siamo in grado di scartare le sottosequenze che sicuramente non ci possono dare il valore migliore e ci focalizziamo solo su quelle dove potenzialmente il valore migliore c'è, troviamo che il numero di sottosequenze importanti da quadratiche diventano lineari nella dimensione dell'array.

Qual è effettivamente l'idea?

L'idea è semplice: immaginiamo di sommare un po' di valori e ad un certo punto nella somma ci capita, a causa ovviamente dei numeri negativi, di andare in negativo.

Ha senso a quel punto continuare a vagliare estensioni di quel pezzo di sottosequenza che è già andato in negativo? No, ovviamente. Se continuassimo a sommare avremmo sicuramente un valore minore rispetto al valore che otterremmo partendo dall'elemento successivo.

Questo ovviamente è il ragionamento ad alto livello.

Per capire veramente se questa idea è effettivamente poi concretizzabile bisogna dimostrare effettivamente delle proprietà.

Teorema 2.3.1 (Teorema MSS).

Ipotesi.

Supponiamo di avere un array tale per cui:

$$a) \forall j \in [i, r) (Sum[i, j] \geq 0)$$

Cioè se andiamo a sommare il valore di tutte le sottosequenze che partono da i e arrivano ad r escluso non abbiamo trovato fino a quel punto qualcosa che va negativo.

$$b) Sum[i, r] < 0$$

Se però andiamo a sommare anche r la somma diventa negativa.

Tesi.

$$c) \forall p \in [i, j], \forall j \in [i, r] (Sum[p, j] \leq Sum[i, j])$$

Quindi se andiamo a prendere un qualunque suffisso della sottosequenza che va da i a j , prendere solo quel suffisso è peggiore di prendere l'intera sequenza.

In pratica: fino a quel punto abbiamo fatto bene ad estendere la sequenza perché ci potevamo solo guadagnare ma non perdere.

Prendere pezzi più piccoli è deleterio ed inutile perché possiamo solo perdere qualcosa.

$$d) \forall p \in [i, r], \forall k (Sum[p, r+k] < Sum[r+1, r+k])$$

Cioè andiamo a prendere un qualunque punto all'interno della nostra sequenza che va da i a r e sommiamo anche elementi che superano r , che è l'indice del valore che ha portato la somma in negativo, peggioriamo.

In pratica: se andiamo a cavallo del punto il cui valore ci ha portato la somma in negativo tanto vale saltare alla cella successiva e ricominciare a sommare da lì.

Dimostrazione.

Dimostriamo innanzitutto il punto c) della tesi.

Osserviamo che:

$$Sum[i, j] = \sum_{h=i}^j A[h] = \sum_{h=i}^{p-1} A[h] + \sum_{h=p}^j A[h] = Sum[i, p-1] + Sum[p, j]$$

Chiaramente $p-1 < j$ e quindi applicando il punto a) dell'ipotesi segue banalmente che

$$Sum[i, p-1] \geq 0$$

Dal quale segue chiaramente:

$$0 \leq Sum[i, p-1] = Sum[i, j] - Sum[p, j]$$

Quindi la tesi c):

$$Sum[p, j] \leq Sum[i, j]$$

Dimostriamo ora il punto d) della tesi.

Osserviamo che:

$$Sum[p, r+k] = Sum[p, r] + Sum[r+1, r+k]$$

Per ottenere la tesi ci basta dimostrare che $Sum[p, r] < 0$, ma questo è immediato.

Consideriamo quanto appena dimostrato (punto c) della tesi) ed il punto b) dell'ipotesi. Segue direttamente che:

$$Sum[p, r] \leq Sum[i, r] < 0$$

Da cui segue:

$$Sum[p, r+k] - Sum[r+1, r+k] = Sum[p, r] \leq Sum[i, r] < 0$$

Quindi la tesi d)

$$Sum[p, r+k] < Sum[r+1, r+k]$$

c.v.d

□

A questo punto possiamo scrivere un algoritmo che sfrutta questa ipotesi.

L'algoritmo è il seguente.

Algoritmo 10: Max subsequence sum terza versione.

Signature $\text{mss}: \mathbb{Z}^* \times \mathbb{N} \rightarrow \mathbb{N}$

```

function  $\text{mss}((A,n))$ 
  // setto il massimo e la somma
1   $m \leftarrow -\infty, s \leftarrow 0$ 
  // scorro l'array
2  for ( $i$  from 0 to  $n-1$ ) do
    // aggiungo alla somma
3     $s \leftarrow s + A[i]$ 
    // se la somma è negativa
4    if ( $s < 0$ ) then
      // ... la resetto a 0
5       $s \leftarrow 0$ 
    // setto il massimo attuale
6     $m \leftarrow \max(m,s)$ 
  // restituisco il massimo
7  return  $m$ 

```

Abbiamo un unico ciclo **for** che scorre l'array man mano sommando le somme parziali, controllando se ci sia un miglioramento rispetto al massimo attuale, fin quando non si rende conto di aver trovato una somma negativa, a quel punto ci dimentichiamo della somma parziale fatta fino a quel punto e proseguiamo da quel punto in poi.

L'algoritmo quindi segue l'ipotesi dimostrata e risulta chiaramente corretto.

Vediamo quanto costa.

Come possiamo vedere l'algoritmo è composto da un unico ciclo **for**, a parte ovviamente due operazioni esterne al ciclo, il cui corpo è composto da una somma, un assegnamento, un **if** con un azzeramento e un altro calcolo di **max**; che significa che il numero di operazioni è costante.

Complessità dell'Algoritmo

Poiché vi è soltanto un **for** l'algoritmo potrà essere al più lineare, ovvero appartiene alla classe di complessità $\Theta(n)$.

Esempio

Forniamo adesso un esempio del funzionamento dell'algoritmo appena visto con un array di interi di lunghezza 10.

Passo 0:

Settiamo la stima del massimo che indicheremo con m al valore più piccolo possibile, per i nostri scopi lo setteremo a $-\infty$ e la somma parziale che indicheremo con s a 0. L'array considerato è quello di fianco.

10	-5	2	-8	3	1	-2	5	4	-3
----	----	---	----	---	---	----	---	---	----

$s = 0 \qquad m = -\infty$

Passo 1:

Iniziamo con indice $i = 0$, sommiamo s con il valore dell'array all'indice i . Controlliamo che s sia non negativa, altrimenti settiamo s a 0. Settiamo m al valore che è più grande tra la somma parziale e la vecchia stima del massimo.

Nel nostro caso, il valore di s sarà 10, essa non verrà settata a 0 poiché $s \geq 0$ e, il valore di m poiché s è più grande di $-\infty$ sarà proprio s , ovvero 10.

i									
10	-5	2	-8	3	1	-2	5	4	-3

$s = 10 \qquad m = 10$

Passo 2:

Incrementiamo l'indice i di uno. Faremo ciò ad ogni passo fino a quando, i non diventerà uguale a 10, la dimensione dell'array, in quel caso l'algoritmo terminerà restituendo la stima del massimo.

In questo caso, il valore di s sarà 5, essa non verrà settata a 0 poiché $s \geq 0$ e, il valore di m poiché s è più piccolo di 10 continuerà ad essere 10.

i									
10	-5	2	-8	3	1	-2	5	4	-3

$s = 5 \qquad m = 10$

Passo 3:

Incrementiamo l'indice i di uno.

In questo caso, il valore di s sarà 7, essa non verrà settata a 0 poiché $s \geq 0$ e, il valore di m poiché s è più piccolo di 10 continuerà ad essere 10.

i									
10	-5	2	-8	3	1	-2	5	4	-3

$s = 7 \qquad m = 10$

Passo 4:

Incrementiamo l'indice i di uno.

In questo caso, il valore di s sarà -1, essa verrà settata a 0 poiché $s < 0$ e, il valore di m poiché s è più piccolo di 10 continuerà ad essere 10.

i									
10	-5	2	-8	3	1	-2	5	4	-3

$s = -1 \rightarrow 0 \qquad m = 10$

Passo 5:

Incrementiamo l'indice i di uno.

In questo caso, il valore di s sarà 3, essa non verrà settata a 0 poiché $s \geq 0$ e, il valore di m poiché s è più piccolo di 10 continuerà ad essere 10.

				i					
10	-5	2	-8	3	1	-2	5	4	-3
$s = 3$					$m = 10$				

Passo 6:

Incrementiamo l'indice i di uno.

In questo caso, il valore di s sarà 4, essa non verrà settata a 0 poiché $s \geq 0$ e, il valore di m poiché s è più piccolo di 10 continuerà ad essere 10.

					i				
10	-5	2	-8	3	1	-2	5	4	-3
$s = 4$					$m = 10$				

Passo 7:

Incrementiamo l'indice i di uno.

In questo caso, il valore di s sarà 2, essa non verrà settata a 0 poiché $s \geq 0$ e, il valore di m poiché s è più piccolo di 10 continuerà ad essere 10.

						i			
10	-5	2	-8	3	1	-2	5	4	-3
$s = 2$					$m = 10$				

Passo 8:

Incrementiamo l'indice i di uno.

In questo caso, il valore di s sarà 7, essa non verrà settata a 0 poiché $s \geq 0$ e, il valore di m poiché s è più piccolo di 10 continuerà ad essere 10.

							i		
10	-5	2	-8	3	1	-2	5	4	-3
$s = 7$					$m = 10$				

Passo 9:

Incrementiamo l'indice i di uno.

In questo caso, il valore di s sarà 11, essa non verrà settata a 0 poiché $s \geq 0$ e, il valore di m poiché s è più grande di 10 diventerà proprio quello di s , ovvero 11.

								i	
10	-5	2	-8	3	1	-2	5	4	-3
$s = 11$					$m = 11$				

Passo 10:

Incrementiamo l'indice i di uno.

In questo caso, il valore di s sarà 8, essa non verrà settata a 0 poiché $s \geq 0$ e, il valore di m poiché s è più piccolo di 11 continuerà ad essere 11.

									i
10	-5	2	-8	3	1	-2	5	4	-3
$s = 8$					$m = 11$				

Passo 11:

Incrementiamo l'indice i di uno. Poiché $i = 10$ l'algoritmo termina restituendo il massimo m (11).

Figura 2.1: Esempio di esecuzione dell'algoritmo MSS versione ottimale

2.3.5 Conclusioni

Osserviamo che tutti gli algoritmi proposti per il problema del max subsequence sum possono essere generalizzati al caso in cui i valori dell'array non contengano necessariamente almeno un numero non negativo, ma avrebbe solo aggiunto della complessità inutile per quello che ci interessa far vedere, cioè che in generale partendo da un problema la cui scrittura è puramente matematica possiamo prima cercare di implementarlo traducendo quasi uno a uno le operazioni matematiche e poi man mano, ragionando sul problema, osservandone le caratteristiche e capendo quali siano le proprietà del problema e sfruttando chiaramente queste proprietà, cercare di semplificarlo e quindi ottenere algoritmi per quanto possibile ottimali, ottimali nel senso di ottenere algoritmi che meglio di quello non si può fare.

Nel caso specifico è dimostrabile che meno di lineare non si può fare.

Perché meno che lineare non possiamo sperare di fare?

Perché siamo costretti a scorrere comunque l'intera sequenza per poter essere certi di non aver dimenticato qualche pezzo.

Capitolo 3

Strutture Dati Lineari

"I can only show you the door, you're the one that has to walk through it"

– Morpheus (The Matrix)

3.1 Introduzione

Abbiamo affrontato un'introduzione molto snella all'idea di algoritmo e di calcolo di progettazione di un algoritmo partendo da un problema. Ci rivolgeremo, adesso, per una certa parte del nostro corso, ad un'analisi di strutture dati e degli algoritmi associati a tali strutture dati che sono stati già progettati.

Spiegheremo perché tali algoritmi sono stati fatti in un certo modo, perché hanno una certa complessità e perché a volte va utilizzata una struttura dati piuttosto che un'altra e quali sono i criteri che dobbiamo imparare a saper utilizzare per scegliere quali sono le strutture dati che sono necessarie a risolvere un certo problema che dobbiamo affrontare.

Vediamo come queste strutture dati si possano incastonare all'interno di un ragionamento più ampio che è quello della gestione dei dati a livello di collezione dei dati.

Una collezione di dati è un insieme, nel senso non matematico del termine, ma nel senso del linguaggio parlato del termine che è molto più ampio e molto più vago; cioè un insieme di dati in cui non necessariamente ogni dato occorre una sola volta, che ha una certa struttura, nel senso che questi dati potrebbero avere relazioni tra di loro, e che ha una certa rappresentazione concreta in memoria. Inoltre, ogni struttura, ogni collezione di dati, ha determinate operazioni per poter accedere ai dati e per poter modificare i dati, dove modifica si intende in senso lato sia l'inserimento di nuovi dati sia la cancellazione dei dati.

Noi vedremo strutture dati per la gestione degli insiemi matematici senza ripetizioni nella maggior parte dei casi.

All'inizio partiremo con collezioni che sono dette multi-insieme, dove ci sono le duplicazioni, ma poi successivamente ci soffermeremo solo su insiemi nel senso matematico, per questioni di semplicità.

3.2 Caratterizzazione generale di una Struttura Dati

Queste strutture dati quindi per poter essere identificate hanno bisogno di:

1. Tipo dei dati da gestire.

Il tipo dei dati qui non significa necessariamente soltanto interi, stringhe o altri oggetti; potremmo avere anche collezioni eterogenee dei dati.

Quindi una collezione è sicuramente caratterizzata da quali sono i dati che possiamo inserire nella struttura dati.

2. Dalle relazioni tra i dati.

Ad esempio se i dati sono ordinati, ...

Quindi le relazioni tra i dati sono informazioni relative a come questi dati sono tra di loro collegati.

3. La rappresentazione concreta in memoria.

Una struttura dati potrebbe cambiare al variare della rappresentazione.

4. Un'altra caratteristica fondamentale è la staticità e/o dinamicità dei dati.

Possiamo avere una struttura dati che costruiamo all'inizio di un processo di calcolo per poi non toccarla più e accedere solo in lettura alla struttura.

Una struttura del genere, a meno di non considerare il cambio di un dato all'interno della struttura, viene vista come statica.

Diversa è la situazione in cui invece possiamo andare a modificare, ad aggiungere nuovi dati, in particolare aggiungere o rimuovere dati.

Queste sono strutture che vengono dette dinamiche.

Avere una chiara distinzione è importante perché a seconda dell'applicazione ci sono strutture dati efficienti statiche che non possono essere così efficienti se sono dinamiche, e viceversa.

5. Le operazioni di creazione e di distruzione della struttura.

Perché ovviamente una struttura dati va costruita, va creata, vanno impostati alcuni parametri della struttura dati e poi una volta che non ci serve più dovrà essere distrutta.

Non ci soffermeremo molto su questo punto.

6. Operazioni di inserimento e di rimozione dei dati, se ovviamente è dinamica.

7. In generale, operazioni di accesso e modifica dei dati.

Una modifica del dato non modifica la struttura di per sé, modifica solo il contenuto, invece un inserimento e una rimozione modificano l'intera struttura, per questo sono state considerate separatamente.

3.2.1 Definizione di Struttura Dati Statica

Una struttura dati viene detta statica se nel momento in cui andiamo a modificare la struttura inserendo o rimuovendo un dato siamo costretti a fare tanto lavoro quanti dati abbiamo all'interno della struttura.

3.2.2 Definizione di Struttura Dati Dinamica

Una struttura dati viene detta dinamica se la modifica della struttura non dipende da quanto la struttura sia grande, da ciò che la struttura contiene.

3.2.3 Definizione alternativa di Struttura Dati Statica

Diamo una definizione un po' più assertiva di struttura dati statica.

Una struttura dati è statica se abbiamo necessariamente bisogno in input di conoscerne la dimensione.

Questa definizione ha senso se vogliamo prendere come parametro in input quanto occupa in memoria, ha meno senso se ci interessa tenere come parametro in input il numero di dati.

3.3 Funzioni generali relative alle Strutture Dati

In generale, quello che ci interessa quando abbiamo una collezione di dati è ovviamente avere possibilità di leggere i dati che sono presenti già nella collezione di dati. Se non avessimo modo di accedere anche solo in lettura ai dati la collezione non avrebbe motivo di esistere.

Quindi le prime funzioni su cui noi ci andremo a focalizzare sono le funzioni di ricerca, di accesso ai dati, che denoteremo in modo generale come *search*.

La funzione *search* prende una collezione, qualunque questa sia (può essere poi un array, una lista o in generale un qualcosa di più complesso come gli alberi binari), prende un certo dato, e cosa restituisce?

Ci sono diverse versioni che si possono fare della ricerca.

Si può fare una ricerca che semplicemente vada a restituire un valore booleano per indicare se sia presente o meno quel particolare dato oppure, quello che si fa, è restituire non semplicemente un valore booleano, ma piuttosto un puntatore, un riferimento ai dati.

Quindi noi potremo andare ad indicare come valore di output l'insieme dei valori booleani,

$\mathbb{B} = \{F, T\} \iff \{0, 1\} \iff \{\perp, \top\}$ oppure un riferimento al dato, in questo modo sarà possibile accedere a quel dato specifico della collezione, potenzialmente poi andandolo a modificare, in caso di assenza del dato il riferimento avrà valore NULL o $\perp$.

$$\text{search} : \text{Collection}(D) \times D \rightarrow \mathbb{B} \quad (3.1)$$

$$\text{search} : \text{Collection}(D) \times D \rightarrow \text{Ref}(D) \quad (3.2)$$

Attenzione. Per questioni di semplicità quando andremo ad implementare la funzione *search* scriveremo la versione che restituisce un valore booleano.

Un'altra possibilità non è semplicemente cercare la presenza di un dato specifico, piuttosto quella di andare all'interno di una collezione di dati, i cui dati però devono essere tra di loro confrontabili secondo una relazione d'ordine totale (non stiamo dicendo che la collezione di dati sia ordinata, ma che i dati ammettano una relazione d'ordine totale), ed andare a fare una ricerca del minimo elemento o del massimo elemento, secondo la suddetta relazione d'ordine totale.

Quindi quando andremo ad indicare funzioni *min* e *max*, quello che staremo automaticamente assumendo è che l'insieme dei dati D sia ordinabile secondo una relazione d'ordine totale, altrimenti il concetto di minimo e massimo non ha proprio senso di esistere; tali funzioni avranno come output un riferimento all'elemento minimo e massimo rispettivamente.

$$\text{min} : \text{Collection}(D) \rightarrow \text{Ref}(D) \quad (3.3)$$

$$\text{max} : \text{Collection}(D) \rightarrow \text{Ref}(D) \quad (3.4)$$

Altre funzioni sono quelle dette di predecessor e successor.

Queste funzioni prendono in input una collezione di dati, sui cui dati è possibile definire una relazione d'ordine

totale, ed un dato, confrontabile con i dati della collezione, e restituiscono in output il riferimento all'interno della struttura del predecessore o successore del dato in input, rispettivamente.

$$\text{predecessor} : \text{Collection}(D) \times D \rightarrow \text{Ref}(D) \quad (3.5)$$

$$\text{successor} : \text{Collection}(D) \times D \rightarrow \text{Ref}(D) \quad (3.6)$$

Attenzione. Non è importante, a differenza della ricerca, che l'elemento in input all'algoritmo sia o meno presente nella struttura, perché quello che ci interessa non è il dato per sé, ma è il dato che precede o succede l'elemento in input all'interno della struttura.

Le funzioni predecessor e successor si possono formalizzare algebricamente come:

Sia S un insieme non vuoto, $<$ una relazione di ordine stretto definita su S , e d un elemento confrontabile con tutti gli elementi di S rispetto a $<$.

$$\text{predecessor}(S, d) = \max \{a \in S \mid a < d\} \quad (3.7)$$

$$\text{successor}(S, d) = \min \{a \in S \mid d < a\} \quad (3.8)$$

Ovviamente quello che abbiamo visto è a livello matematico puramente astratto, non ci stiamo interessando su come la struttura dati sia rappresentata concretamente.

Questi saranno i metodi che andremo ad analizzare quando parleremo semplicemente di operazioni di lettura dei dati.

Attenzione che quando diciamo di lettura non necessariamente significa veramente solo lettura perché nei casi in cui abbiamo un valore che è passato per riferimento, cioè abbiamo un riferimento al dato, teoricamente con questo riferimento poi possiamo anche modificare il dato, avendo accesso al dato potenzialmente in scrittura.

Quello che però differisce tra le funzioni di lettura dei dati e le funzioni ad esempio di inserimento e rimozione, che andremo a vedere a breve, è il fatto che le funzioni di lettura dei dati, sono funzioni che non vanno ad intaccare la struttura.

Non devono essere, come si dice, funzioni distruttive, devono essere funzioni che ci lasciano la struttura dati su cui vengono applicate identica e precisa a prima della chiamata a funzione.

Differente invece è il caso di funzioni tipo l'inserimento o la rimozione.

La funzione insert prende in input una certa collezione ed un dato e va ad inserire il dato nella collezione, cioè a costruire una nuova collezione contenente il dato in input più quelli di prima. Questa funzione in un certo senso è distruttiva nell'accezione che la vecchia collezione ora non è più presente.

Quindi abbiamo apportato una modifica alla struttura che ci ha reso in un certo senso la vecchia struttura non più presente, a meno di non averne fatto una copia.

$$\text{insert} : \text{Collection}(D) \times D \rightarrow \text{Collection}(D) \quad (3.9)$$

In maniera simile, forse anche in modo più evidente, la funzione *remove* prende in input una certa collezione ed un dato e va a rimuovere il dato dalla collezione, se presente, cioè a costruire una nuova collezione, a partire da quella di prima, senza il dato in input.

$$\text{remove} : \text{Collection}(D) \times D \rightarrow \text{Collection}(D) \quad (3.10)$$

Attenzione che la specifica semantica di inserimento e rimozione dipende in effetti dalla struttura dati su cui andiamo a applicare.

Ad esempio, in caso di liste o array dove non ci interessa un ordine e non ci interessa neanche se siano presenti o meno duplicati dei dati, inserimento e cancellazione sono quello che immaginiamo.

Inseriamo il dato nella struttura per l'inserimento; per quanto riguarda la cancellazione se troviamo il dato all'interno lo cancelliamo, se non lo troviamo non lo cancelliamo.

Nel caso, ad esempio, di array o liste in cui i dati non devono essere duplicati potremmo provare a inserire un dato nella collezione, ma quel dato non verrà inserito se è già presente.

Di conseguenza le funzioni di inserimento e rimozione non è detto che vadano sempre a modificare la struttura.

Inoltre, c'è sempre una funzione chiamata *emptyX*, dove *X* è una meta variabile che sta ad indicare il nome della struttura dati che ci interessa, che va a restituire una struttura dati di tipo *X* vuota.

$$\text{emptyX} : X(D) \quad (3.11)$$

3.4 Caratterizzazione di Strutture Dati Lineari

Le prime strutture dati che andremo ad analizzare sono gli array, le liste, le queue (code) e gli stack (pile).

3.4.1 Array

Vediamo un po' di capire ad esempio come vengono classificati gli array utilizzando questa separazione delle informazioni.

1. Come dati da gestire in generale un array potrebbe gestire un qualunque tipo di dato. L'array non è altro che una sequenza di dati, quindi se sappiamo come memorizzare il dato semplicemente prendiamo il dato e lo memorizziamo.

Non c'è particolare richiesta di proprietà sui dati.

2. Come relazione tra i dati, essendo la struttura lineare, a meno di non avere ulteriori informazioni esterne all'array, cosa che ovviamente non stiamo considerando, possiamo o non avere proprio relazioni tra i dati, oppure avere un ordine totale tra i dati.

3. La rappresentazione in memoria è contigua, viene appunto detta rappresentazione contigua.

Si tratta infatti di una sequenza di dati senza soluzione di continuità.

4. Si tratta di una struttura dati statica.

Se vogliamo andare ad inserire un nuovo dato va ricostruita da capo.

5. Se vogliamo creare un array partendo da un insieme di n dati, una volta creato l'array di lunghezza n , banalmente inseriamo i dati con un ciclo `for`; ha pertanto costo $\Theta(n)$.

Costo analogo ovviamente per la distruzione della struttura.

6. Le operazioni di inserimento e rimozione sono tutte operazioni che devono ricostruire l'array e quindi hanno costo $\Theta(n)$, dove n è quanti dati già abbiamo nell'array.

Per essere precisi, per inserire un nuovo dato in generale allochiamo un nuovo array con una nuova cella, inseriamo i dati vecchi e il dato nuovo nella cella a disposizione, qualunque essa sia.

7. Le operazioni di accesso e modifica hanno costo $\Theta(1)$, conoscendo l'indice del dato.

Questo è dovuto alla rappresentazione contigua dei dati in memoria, grazie alla quale indirizzare un array costa tempo costante.

Se invece dobbiamo cercare un certo dato, considerando assenza di relazioni tra i dati, essendo la ricerca sequenziale, avremo ovviamente $O(n)$ e $\Omega(1)$.

Abbiamo, quindi, incastonato una particolare struttura dati all'interno di un'organizzazione più strutturata e generale.

3.4.2 Array Ordinati

Vediamo gli Array Ordinati.

1. Come dati da gestire in generale dobbiamo pretendere che i dati siano ordinabili totalmente.
2. La relazione tra i dati deve essere un ordine totale tra i dati.
3. La rappresentazione in memoria è contigua.
4. Si tratta di una struttura dati statica.
5. Se vogliamo creare un array ordinato partendo da un insieme di n dati, una volta creato l'array di lunghezza n , banalmente inseriamo i dati con un ciclo `for`, dopodiché dobbiamo fare dell'ulteriore lavoro per ordinarlo (che potrebbe essere, una volta che inseriamo i dati applichiamo un algoritmo di ordinamento e solo dopo aver applicato l'algoritmo di ordinamento avremo davvero creato un array ordinato), l'ordinamento nel caso ottimale ha costo $\Theta(n \log(n))$.

Il costo per la distruzione della struttura è invece $\Theta(n)$ ovviamente.

6. Le operazioni di inserimento e rimozione sono tutte operazioni che devono ricostruire l'array e quindi hanno costo $\Theta(n)$, dove n è quanti dati già ho nell'array.

Per essere precisi, per inserire un nuovo dato dobbiamo allocare un nuovo array con una nuova cella e mediante confronti inserire i dati secondo l'ordinamento totale che si vuole mantenere; è importante notare che sebbene tali operazioni abbiano la stessa complessità che si ha in un array, gli algoritmi sono differenti.

7. Le operazioni di accesso e modifica hanno costo $\Theta(1)$, conoscendo l'indice del dato.

Questo è dovuto alla rappresentazione contigua dei dati in memoria, grazie alla quale indirizzare un array costa tempo costante.

Se invece dobbiamo cercare un certo dato, visto che i dati sono totalmente ordinati, possiamo utilizzare la ricerca binaria, detto anche algoritmo di bisezione, e avremo quindi un costo di $O(\log(n))$ e $\Omega(1)$.

Stiamo osservando, quindi, che Array ed Array Ordinati sono due strutture dati diverse; il fatto che concretamente la rappresentazione in memoria o il fatto che quando andiamo a programmare utilizziamo le stesse modalità non significa che la struttura dati sia la stessa.

La rappresentazione in memoria è la stessa ma una struttura dati non è solo la sua rappresentazione in memoria.

Array ed Array Ordinati sono due strutture dati diverse che hanno operazioni diverse il cui costo vedremo sarà diverso, l'unica cosa che coincide è la rappresentazione in memoria. Si tratta di una caratteristica su tante.

3.4.3 Liste Concatenate

Vediamo le liste concatenate.

1. Come dati da gestire in generale una lista concatenata, potrebbe gestire un qualunque tipo di dato.
2. Come relazione tra i dati possiamo o non avere proprio relazioni tra i dati, oppure avere un ordine totale tra i dati.
3. La rappresentazione in memoria non è contigua.
Siamo costretti sempre a utilizzare la memoria dinamica.
4. Pertanto la struttura è dinamica.
Per essere precisi è a metà tra l'essere dinamica e statica visto che la cancellazione di un dato costringe ad effettuare una ricerca che può essere lineare e quindi dipendente da quanti dati abbiamo nella struttura.
5. Se vogliamo creare una lista concatenata partendo da n dati, per ciascun dato andremo a creare un nodo che poi inseriremo in testa; quindi la creazione ha costo $\Theta(n)$.
Analogo costo per la distruzione della struttura.
6. Se non ci interessa avere informazione tra i dati l'inserimento, in testa, è a tempo costante, quindi $\Theta(1)$; la cancellazione non lo è perché è necessario trovare il dato e pertanto fare una ricerca che può essere lineare, possiamo solo dire che è $O(n)$ e $\Omega(1)$.
7. Le operazioni di accesso non sono a tempo costante; quindi avremo che le operazioni di lettura e modifica hanno un costo che va da $O(n)$ ad $\Omega(1)$.
Costo analogo per la ricerca di un dato.

A differenza degli array, quindi, nelle liste concatenate manca totalmente un accesso diretto.

Se ci interessa solo la ricerca queste due strutture hanno lo stesso identico costo; l'unica cosa che gli array hanno in più quindi è l'accesso diretto.

Quindi se non ci interessa l'accesso diretto, forse vale la pena utilizzare la lista concatenata visto che ha una costruzione più semplice.

Questo fin tanto che non ci interessa tenere traccia delle informazioni sui dati.

Se volessimo, invece, avere array ordinati o liste concatenate ordinate la situazione cambia.

3.4.4 Liste Concatenate Ordinate

Vediamo se i vantaggi che abbiamo in un array ordinato possano essere in qualche modo esportati su una lista concatenata ordinata.

1. Come dati da gestire in generale dobbiamo pretendere che i dati siano ordinabili totalmente.
2. La relazione tra i dati deve essere un ordine totale tra i dati.
3. La rappresentazione in memoria non è contigua.
4. Si tratta di una struttura dati dinamica.

Per essere precisi è a metà tra l'essere dinamica e statica, come per le liste concatenate.

5. Se vogliamo creare una lista concatenata ordinata partendo da n dati, per ciascun dato andremo a creare un nodo che poi inseriremo in ordine confrontando i nodi già inseriti; quindi la creazione ha costo $\Theta(n^2)$.

È possibile migliorare il costo della creazione di una lista concatenata ordinata sfruttando un array ordinato con i dati da inserire e poi travasare i dati nella lista concatenata ordinata.

La distruzione della struttura costa $\Theta(n)$.

6. Visto l'ordine totale tra i dati l'inserimento di un dato, visto che non possiamo sapere a priori dove andare ad inserire il suddetto dato, avrà costo $O(n)$ e $\Omega(1)$; analogo discorso per la cancellazione di un dato.
7. Le operazioni di accesso non sono a tempo costante; quindi avremo che le operazioni di lettura e modifica hanno un costo che va da $O(n)$ ad $\Omega(1)$.

Costo analogo per la ricerca di un dato.

Significa che con le liste concatenate ordinate rispetto alle liste concatenate abbiamo praticamente peggiorato sempre; quindi se ci serve una struttura dati in cui i dati devono essere ordinati, a meno di non avere altri motivi per farlo, non è una buona idea scegliere una lista concatenata ordinata.

3.4.5 Stack

Vediamo gli Stack (pile).

1. Può gestire un qualunque tipo di dato.
2. La relazione tra i dati deve essere un ordine totale, l'ordine di arrivo inverso; lo facciamo sempre e possiamo fare solo questo (LIFO).
3. A livello di rappresentazione la situazione non è così chiara perché gli stack sono strutture dati in un certo senso di livello superiore rispetto agli array e le liste.

Gli array e le liste sono strutture dati in qualche modo native della macchina, del modello computazionale, che abbiamo a disposizione (l'array è proprio il modo in cui è organizzata la memoria, le liste sono una piccola sovrastruttura ottenuta grazie ai puntatori).

Gli stack non sono strutture dati che la macchina immediatamente permette di avere, ma possono essere implementate sia con gli array sia con le liste.

4. È considerata una struttura dati dinamica.
5. Non ci soffermeremo molto su questo punto.
6. Le operazioni di inserimento e rimozione, dette rispettivamente push e pop, sono sempre in testa e hanno quindi costo costante: $\Theta(1)$.
7. Visto che l'accesso ai dati è ristretto solo alla testa dello stack, le operazioni di lettura e modifica hanno costo costante, quindi $\Theta(1)$.

La ricerca non è consentita.

3.4.6 Queue

Vediamo le Queue (code).

1. Può gestire un qualunque tipo di dato.
2. La relazione tra i dati deve essere un ordine totale, l'ordine di arrivo diretto; lo facciamo sempre e possiamo fare solo questo (FIFO).
3. A livello di rappresentazione la situazione non è così chiara perché le queue sono strutture dati in un certo senso di livello superiore rispetto agli array e le liste.

Gli array e le liste sono strutture dati in qualche modo native della macchina, del modello computazionale, che abbiamo a disposizione (l'array è proprio il modo in cui è organizzata la memoria, le liste sono una piccola sovrastruttura ottenuta grazie ai puntatori).

Le queue non sono strutture dati che la macchina immediatamente permette di avere, ma possono essere implementate sia con gli array sia con le liste.

4. È considerata una struttura dati dinamica.
5. Non ci soffermeremo molto su questo punto.
6. Le operazioni di inserimento e rimozione, dette rispettivamente enqueue e dequeue, sono sempre in coda e in testa, rispettivamente, e hanno quindi costo costante: $\Theta(1)$.
7. Visto che l'accesso ai dati è ristretto solo alla testa della queue, le operazioni di lettura e modifica hanno costo costante, quindi $\Theta(1)$.

La ricerca non è consentita.

3.5 Algoritmi sugli Array

3.5.1 Introduzione

Focalizziamo adesso la nostra attenzione sulle tecniche degli algoritmi; vedremo ancora algoritmi molto semplici su array, in particolare sia array ordinati che non, per poi passare ad algoritmi su liste concatenate.

A questo punto passiamo agli array.

Andiamo a soffermarci, assumendo che la struttura dati sia stata già costruita e popolata, sull'andare a fare una ricerca di un dato nella struttura, oppure di restituire il minimo, o il massimo, o di andare a calcolare quale sia, se presente, il predecessore o il successore di un certo dato.

Andiamo a vedere come fare una ricerca negli array. Ne abbiamo ampiamente parlato a livello astratto; ora ci soffermeremo sugli array con ripetizioni, array senza ripetizioni, e array ordinati.

Ovviamente quando abbiamo il concetto di array ordinato stiamo assumendo, che sui dati esista un ordinamento totale.

Quando abbiamo il concetto di array senza ripetizione, quello che stiamo automaticamente assumendo è l'esistenza dell'operatore di uguaglianza e ovviamente del suo negato, l'operazione di differenza, perché per poter dire che esista o meno una ripetizione all'interno di una certa sequenza dobbiamo poter fare confronti di uguaglianza o in generale di equivalenza.

Quando invece non siamo interessati né all'ordine dei dati né alle ripetizioni non ci interessa né avere una relazione d'ordine totale né avere il confronto, visto che tanto semplicemente popoliamo la struttura senza dover fare confronti.

Ovviamente nel caso in cui stiamo facendo ricerche, che è il motivo per cui queste strutture sono state pensate, serve per forza il confronto.

Questa è una cosa che diamo spesso per scontato, ed è ragionevole, perché in generale per tutti i dati, anche quelli più astrusi, c'è un modo canonico di definire un concetto di uguaglianza: prendiamo il dato, osserviamo come è rappresentato in memoria e assumiamo che due dati siano uguali se la rappresentazione in memoria è identica. Ma in generale se pensiamo a strutture dati complesse, a parte la versione canonica dell'uguaglianza, un concetto di uguaglianza tra dati di solito va definito.

In generale va sempre definito un concetto di uguaglianza, altrimenti nella maggior parte delle strutture dati non è possibile fare praticamente nulla.

Diverso il concetto invece dell'ordinamento; è molto più frequente avere dati non ordinati. Non tutte le strutture hanno necessariamente bisogno dell'ordinamento.

3.5.2 Algoritmo: Ricerca in un Array con Ripetizioni

Implementiamo una search in un array.

Ricordiamo che per noi un array è una coppia che è formata dal puntatore al primo elemento e da un numero che rappresenta il numero di celle presenti.

Sia d il dato in input da cercare. Cosa possiamo immaginare di fare per verificare se effettivamente quel dato sia o meno presente all'interno dell'array?

Scorriamo tutto il vettore per la sua lunghezza e confrontiamo ogni elemento col dato da cercare.

Se troviamo l'elemento da cercare restituiamo True, altrimenti False.

L'algoritmo è il seguente.

Algoritmo 11: Ricerca in un array con ripetizioni

Signature search: $Array(D) \times D \rightarrow \mathbb{B}$

```
function search((A,n), d)
    // scorro l'array
1   for (i from 0 to n-1) do
        // se l'elemento corrente è uguale al dato da cercare
2       if (A[i] = d) then
            // ... restituisco true
3           return True
        // ... restituisco false
4   return False
```

Questo algoritmo ha complessità lineare visto che vengono effettuate un numero di operazioni costanti per un numero di volte al più pari ad n .

Complessità dell'Algoritmo

Avremo quindi una complessità di $O(n)$ e $\Omega(1)$, in quanto se siamo fortunati potremmo trovare l'elemento da cercare in posizione 0.

3.5.3 Algoritmo: Ricerca in un Array senza Ripetizioni

Vediamo se c'è qualcosa di diverso nell'algoritmo di ricerca nel caso degli array senza ripetizione.

Si può creare un algoritmo migliore di quello che abbiamo scritto ora, avendo la conoscenza che l'array non contiene elementi duplicati?

No. Ovviamente adesso abbiamo un'informazione aggiuntiva; tuttavia questa informazione aggiuntiva non è sufficiente per poter ottenere un algoritmo più efficiente riguardo alla ricerca

L'algoritmo è il seguente.

Algoritmo 12: Ricerca in un array senza ripetizioni

Signature search: $\text{Array}(D) \times D \rightarrow \mathbb{B}$

```
function search((A,n), d)
    // scorro l'array
1   for (i from 0 to n-1) do
    // se l'elemento corrente è uguale al dato da cercare
2       if (A[i] = d) then
3           // ... restituisco true
           return True
    // ... restituisco false
4   return False
```

Essendo lo stesso identico algoritmo della ricerca in un array, ovviamente anche la complessità sarà la stessa.

Complessità dell'Algoritmo

pertanto $O(n)$ e $\Omega(1)$.

3.5.4 Algoritmo: Ricerca Binaria Ricorsiva in un Array Ordinato

A questo punto possiamo invece agli array ordinati.

Sappiamo che in questo caso possiamo effettuare una ricerca binaria, detta anche algoritmo di bisezione.

La prima versione che analizzeremo è quella forse più naturale, cioè la versione ricorsiva.

Più naturale perché è propria di un approccio alla risoluzione dei problemi, che è uno degli approcci più potenti che presenteremo in questo corso, che è il concetto di approccio divide et impera.

Immaginiamo di avere un problema di una certa complessità.

Cerchiamo di risolvere questo problema facendo del lavoro locale, locale inteso come lavoro sulla struttura dati attuale, ma questo lavoro deve essere un lavoro di livello "piccolo"; poi andremo a rendere più proprio questo termine piccolo.

Dopodiché, il nostro intento sarà di andare su una sottoparte dell'intero problema e cercare di trovare una soluzione di una sottoparte del problema.

Potenzialmente lo facciamo per tutte le sottoparti, o almeno se dividiamo il problema in n pezzi cerchiamo di trovare una soluzione per tutti i pezzi; a volte non serve farlo per tutti i pezzi, ma solo per una parte o addirittura solo per un pezzo.

Dopodiché, una volta che abbiamo una soluzione su un sottoproblema, cerchiamo di ottenere una soluzione del problema originale.

Descriviamo in modo estremamente semplificato tale approccio al problema specifico.

Abbiamo un array di una certa lunghezza e dobbiamo cercare un elemento al suo interno.

Qual è il lavoro locale che vogliamo fare?

Il lavoro locale che vogliamo fare è identificare l'elemento mediano, questo è un pezzo del lavoro che facciamo a livello locale.

Sicuramente dovremo calcolare l'elemento mediano e sicuramente dovremo fare un confronto tra l'elemento mediano e il dato da cercare per capire in primo luogo se abbiamo già trovato il dato, e, nel caso invece non siamo stati così fortunati, capire su che sottoparte dell'array, quindi in particolare su che sottoparte del problema, vogliamo andare a lavorare.

Questo approccio è possibile visto che sappiamo che l'array è ordinato.

A quel punto, una volta che abbiamo identificato che è necessario andare in una parte dell'array, cosa andremo a fare?

Andremo a ripetere il processo su un sottoproblema di dimensione più "piccola". E così via fino a quando non avremo risolto l'intero problema.

Si tratta di un approccio estremamente potente che verrà utilizzato in tanti altri ambiti dove le cose sono un po' più complesse e più interessanti in generale.

Ovviamente la funzione deve prendere in input un array ordinato e il dato da cercare, tuttavia come abbiamo detto il primo algoritmo che vogliamo mostrare è quello ricorsivo.

Avere la solita firma del metodo di ricerca ci permette di applicare un approccio ricorsivo?

No, non ce lo permette perché non ci permette di identificare qual è la sottoparte del problema su cui noi vogliamo andare a lavorare.

Pertanto quello che si fa è creare una funzione di interfaccia e poi questa funzione darà il compito di risolvere il vero problema ad una funzione accessoria che prende in input un array ordinato, due indici che stanno ad indicare in quale ambito, in quale range, in quale intervallo degli elementi dell'array ci interessa andare a fare la ricerca, e ovviamente il dato da cercare.

Cosa fa quindi la funzione di interfaccia?

La prima funzione non deve fare altro che delegare il compito della risoluzione alla funzione accessoria di riserva vera e propria.

Quindi quello che farà sarà banalmente una chiamata alla funzione `search`, passando in input l'array ordinato, il primo indice dell'array, l'ultimo indice dell'array e il dato da cercare.

Attenzione che come indice di fine intervallo andremo a indicare l'indice successivo a quello dell'intervallo, ecco perché partiamo da n .

L'algoritmo è il seguente.

Algoritmo 13: Ricerca binaria ricorsiva in un array ordinato (funzione di interfaccia)

Signature search: $\text{Array}(D) \times D \rightarrow \mathbb{B}$

function `search((A,n), d)`

```

1  | // chiamo la funzione ricorsiva
  | return search((A,n), 0, n, d)

```

Fatto ciò possiamo soffermarci sull'algoritmo vero e proprio, che è il seguente, che è la vera e propria ricerca. Attenzione. Per sottolineare che la funzione opera su array ordinati useremo `SortedArray`.

Algoritmo 14: Ricerca binaria ricorsiva in un array ordinato (funzione accessoria)**Signature search:** $SortedArray(D) \times \mathbb{N} \times \mathbb{N} \times D \rightarrow \mathbb{B}$

```

function search((A,n), i, j, d)
    // supponiamo che l'array sia ordinato in modo crescente
    // se il sottoarray considerato non è vuoto
1   if (i < j) then
        // prendo l'indice mediano
2       k ← floor((i+j)/2)
        // se il dato da cercare è minore del valore attuale
3       if (d < A[k]) then
            // ... valuta la sottoparte dell'array precedente al valore corrente
4           return search((A,n), i, k, d)
        // se il dato da cercare è maggiore del valore attuale
5       else if (A[k] < d) then
            // ... valuta la sottoparte dell'array successiva al valore corrente
6           return search((A,n), k+1, j, d)
7       else
            // se il dato da cercare è stato trovato restituisco true
8           return True
    // se il sottoarray considerato è vuoto restituisco false
9   return False

```

In un algoritmo ricorsivo come prima cosa si individuano i casi base.

Intuitivamente il caso base è quando la porzione di array che ci interessa analizzare è vuota, quindi quando $i \leq j$: in questo caso restituiamo False. Altrimenti bisogna trovare l'elemento mediano dell'intervallo che va da i a j , quindi andare a calcolare k che è $\text{floor}((i+j)/2)$.

Fatto ciò possiamo confrontare il valore mediano della porzione di array che stiamo considerando con il dato da cercare. Ecco dove, ovviamente, è fondamentale avere l'uguaglianza.

Nel caso il valore mediano sia uguale a quello da cercare ovviamente restituiamo True. Altrimenti che cosa dobbiamo fare?

Dobbiamo capire ovviamente se $A[k]$ è inferiore o superiore all'elemento da cercare, d .

Quindi se $A[k] < d$, e qui stiamo sfruttando finalmente l'ordinamento sui dati, l'elemento che stiamo cercando potrà essere presente nell'array solo nella porzione che si trova ad essere successiva ad $A[k]$ (supponendo un array di dati ordinati in senso crescente). Quindi andremo a lavorare sulla sottoparte di interesse.

Discorso esattamente analogo nel caso in cui $d < A[k]$, andando a lavorare nella sottoparte opposta.

3.5.5 Algoritmo: Ricerca Binaria Iterativa in un Array Ordinato

Vediamo ora la versione iterativa dell'algoritmo di ricerca binaria in un array ordinato.

Essenzialmente è identico alla versione ricorsiva se non fosse che la ricorsione ovviamente deve essere in qualche modo simulata da un ciclo; in effetti quello che andremo a fare è simulare la ricorsione.

In futuro daremo anche degli strumenti per capire proprio come tradurre sempre un algoritmo ricorsivo in un algoritmo iterativo.

Nella versione iterativa, poiché appunto non ci serve suddividere in modo esplicito l'array, sebbene poi implicitamente viene fatto tale e quale, non è necessaria una funzione che lavora sul sottoproblema, possiamo fare tutto all'interno della funzione search.

L' algoritmo è il seguente.

Algoritmo 15: Ricerca binaria iterativa in un array ordinato

Signature search: $SortedArray(D) \times D \rightarrow \mathbb{B}$

```

function search((A,n),d)
    // supponiamo che l'array sia ordinato in modo crescente
    // i indice iniziale del sottoarray
    // j indice finale del sottoarray
    // setto gli indici e il flag
1   (i, j, f) ← (0, n, False)
    // fino a quando il sottoarray considerato è non vuoto
2   while (i < j) do
        // prendo l'indice mediano
3       k ← floor((i+j)/2)
        // se il dato da cercare è minore del valore attuale
4       if (d < A[k]) then
            // ... sposto l'indice finale all'indice mediano
            // per andare a valutare la sottoparte dell'array precedente al valore corrente
5           j ← k
        // se il dato da cercare è maggiore del valore attuale
6       else if (d > A[k]) then
            // ... sposto l'indice iniziale al successivo dell'indice mediano
            // per andare a valutare la sotto-parte dell'array successivo al valore corrente
7           i ← k + 1
8       else
            // se il dato da cercare è stato trovato, setto il flag a true
9           f ← True
            // ed interrompo il ciclo while
10          break
    // restituisco il flag
11  return f

```

Dobbiamo simulare in pratica l'algoritmo ricorsivo, quindi anche qui avremo degli indici i, j che ci andranno ad identificare qual è il sottopezzo di array su cui ci interessa porre l'attenzione e sicuramente all'inizio quando cominciamo a lavorare dobbiamo impostare $i = 0$ e $j = n$.

Quindi con questa istruzione stiamo simulando in un certo senso la chiamata alla funzione di ricerca dell'algoritmo ricorsivo perché abbiamo impostato, come nel caso dell'algoritmo ricorsivo l'indice $i = 0$ e $j = n$.

Per semplicità diamo un nome al valore di ritorno, cioè al valore che vogliamo restituire, che sarà un flag che indica se abbiamo trovato o meno il dato da cercare (f di found).

All'inizio ovviamente poiché non abbiamo ancora analizzato l'array poniamo questo flag a false, in modo tale che se poi all'interno del ciclo il dato da cercare non viene mai trovato l'algoritmo restituirà false, il valore corretto. Se invece all'interno del ciclo **while** troviamo il valore che ci interessa, semplicemente andremo a cambiare il flag a true e poi lo restituiamo.

Intuitivamente stiamo dando a questo flag, **f**, la semantica: diventerà true solo e soltanto se avremo trovato il dato da cercare.

Ovviamente è possibile inserire un'istruzione di return all'interno del ciclo **while**, in modo anche da evitare di avere un flag accessorio.

Lo scopo però è far capire il modo nel quale vanno scritti gli algoritmi, che devono essere quanto più lineari possibile.

Solo quando siamo davvero pratici con la progettazione avrà senso abbreviare il codice con delle operazioni che magari non sono sempre da fare perché rendono il codice ovviamente meno comprensibile.

Se non troviamo il dato da cercare è perché semplicemente ci riduciamo all'array vuoto e non abbiamo trovato precedentemente il dato.

Questo caso semplicemente si tradurrà in un ciclo **while**, con una condizione del ciclo che è opposta a quella del caso base dell'algoritmo ricorsivo, cioè **while (i < j)**, che significa che l'intervallo su cui stiamo lavorando non è vuoto perché per essere vuoto deve essere $j \leq i$ (caso base dell'algoritmo ricorsivo).

Il corpo del ciclo **while** è praticamente identico al corpo dell'algoritmo ricorsivo; invece di andare a effettuare una chiamata ricorsiva andremo solo e soltanto a cambiare i parametri, i valori delle variabili di stato locali **i** e **j** in funzione del valore di **A[k]** rispetto al valore di **d**, ma tale settaggio sarà praticamente corrispondente alla chiamata ricorsiva.

Praticamente facciamo le stesse identiche istruzioni di confronto dell'algoritmo ricorsivo.

La restituzione di True nella chiamata ricorsiva corrisponde al settaggio del flag a true, perché è stato trovato il valore, e all'interruzione del ciclo **while** con l'istruzione **break**.

Attenzione. È sempre possibile evitare l'istruzione **break**.

Nel caso specifico basterebbe cambiare la condizione del ciclo while in: **while (i < j AND f = False) do {...}**.

Attenzione. È stato scelto volutamente di partire dall'algoritmo ricorsivo e andare all'iterativo perché molto spesso si è tentati di scrivere direttamente un algoritmo iterativo.

Questo approccio è quasi sempre è un approccio sbagliato che porta ad avere del codice più complesso del necessario e soprattutto molto più pronò agli errori.

Quando un problema è facilmente modellabile con un approccio divide et impera, allora la prima implementazione che bisogna fare dell'algoritmo è un approccio ricorsivo; l'iterazione va fatta solo in una seconda battuta.

Analizzare un algoritmo iterativo e in generale capire che cosa sta facendo, è molto più complesso di capire un algoritmo ricorsivo.

Questo perché un algoritmo ricorsivo assume di essere in grado di trovare la soluzione al problema in un sottoproblema, quindi è necessario concentrarsi solo su ciò che si fa localmente.

Un algoritmo iterativo, in generale, è più complesso, pertanto buttarsi direttamente ad implementare qualcosa iterativo di solito, quasi sempre, è sbagliato, soprattutto perché gli algoritmi diventano spesso più complessi e più prone ad errori.

Ovviamente l'algoritmo iterativo simula esattamente quello ricorsivo e quindi la complessità asintotica di entrambi sarà la stessa.

Ma come fare a capire quanto costano?

Sappiamo che l'algoritmo di ricerca binaria su array ordinati ha un andamento del tipo $O(\log(n))$ e $\Omega(1)$. Ma perché è logaritmico di n ?

Semplicemente perché ogni volta che effettuiamo un confronto, dopo lavoreremo su una parte dell'array che ha una dimensione pari alla metà, approssimata ovviamente all'intero inferiore, della dimensione dell'array al confronto precedente; quindi ad ogni confronto l'array si dimezza.

Poiché possiamo dimezzare un array fino ad arrivare a dimensione 1 soltanto un numero logaritmico di volte, perché significa che stiamo andando a dividere l'array originale per una potenza di 2, ovviamente basta fare un po' di conti per osservare effettivamente che allora, se per dimezzare un array possiamo dimezzare al più un numero logaritmico di volte ed ogni volta che lo dimezziamo stiamo facendo un numero di confronti costante, globalmente ovviamente l'algoritmo non potrà costare più che logaritmico.

Complessità dell'Algoritmo di Ricerca Binaria in un Array ordinato

Vedremo, con un approccio più metodologico, che cerca di costruire dall'algoritmo una equazione di ricorrenza e da questa equazione di ricorrenza poi tirare fuori quale sia la funzione che è la soluzione dell'equazione di ricorrenza, di confermare se l'algoritmo non costa più che logaritmico.

Lavoreremo considerando l'algoritmo ricorsivo.

A noi interessa la complessità della funzione accessoria visto che la funzione di interfaccia fa semplicemente una chiamata e quindi il suo costo è il costo della chiamata a funzione, che consideriamo costante, più quanto costa la funzione accessoria ricorsiva.

Questo algoritmo è di complessità asintotica inferiore $\Omega(1)$, visto che potremmo essere fortunati e trovare l'elemento da cercare al primo tentativo.

Inoltre, l'algoritmo è di complessità asintotica superiore $O(\log(n))$, con n la lunghezza dell'array ordinato, questo perché potremmo essere sfortunati ed il valore da cercare non è proprio presente e di conseguenza andremo avanti ricorsivamente fino a quando l'intervallo dell'array che stiamo considerando non diventa vuoto; in questo caso faremo un numero di operazioni che è $O(\log(n))$.

Che sia $\Omega(1)$ è ovvio.

Che sia $O(\log(n))$ non è invece immediato ed è necessario dimostrarlo.

Chiaramente ci poniamo nel caso in cui l'elemento da cercare non sia presente nell'array, visto che vogliamo analizzare l'O-grande, quindi il caso peggiore.

Se guardiamo l'algoritmo, non è così immediato capire quali sono i parametri da valutare e come impostare una dimostrazione. Perché?

Perché si può osservare che come caratteristica critica, come parametro critico per capire quante chiamate ricorsive l'algoritmo faccia, non possiamo prendere uno dei parametri da solo.

Non c'è alcun parametro sul quale si possa fare una dimostrazione induttiva perché i e j sono dei parametri che non riusciamo a controllare.

Quello che è importante è la loro differenza, è quella che decresce.

All'inizio la differenza tra i e j quando viene effettuata la chiamata esterna è esattamente n .

Successivamente, dopo la prima chiamata, visto che ci stiamo ponendo nel caso peggiore, verrà effettuata una nuova chiamata ricorsiva nel sottoarray posto alla destra o alla sinistra dell'elemento mediano. Se la distanza tra i e j è dispari entrambi gli intervalli avranno esattamente la stessa identica dimensione, se è pari avranno una dimensione che differisce di una sola unità; chiaramente non cambia nulla a livello asintotico.

Quindi andiamo ad applicare la funzione `search` su un certo intervallo, di una certa dimensione, in particolare all'inizio di dimensione n (per dimensione intendiamo l'intervallo di estremi i , j). Nel caso peggiore, andremo a richiamare la funzione `search` su dimensione $\frac{n}{2}$, e così via fino a quando l'intervallo considerato non è vuoto; visto che il nuovo intervallo diventa l'input dell'algoritmo chiamato ricorsivamente.

Quindi una chiamata ricorsiva su input di dimensione n andrà a chiamare la stessa funzione su dimensione $\frac{n}{2}$, che a sua volta chiamerà la stessa funzione ricorsivamente su $\frac{n}{4}$, e così via.

Quindi, a seconda di quanti passi dobbiamo fare, bisogna capire quanto costa l'algoritmo; ma l'algoritmo nel corpo ha solo operazioni costanti, escludendo la chiamata ricorsiva ovviamente.

Di conseguenza, il costo delle istruzioni che non dipendono dalla chiamata ricorsiva lo possiamo calcolare.

Quindi possiamo scrivere la funzione del tempo nel seguente modo:

$$T_n = \begin{cases} 2 & \text{se } n \leq 1 \\ 5 + T_{\lfloor \frac{n}{2} \rfloor} & \text{altrimenti} \end{cases}$$

Possiamo tranquillamente trascurare la parte inferiore di $\frac{n}{2}$ visto che non cambia nulla a livello asintotico, di conseguenza la funzione del tempo sarà:

$$T_n = \begin{cases} 2 & \text{se } n \leq 1 \\ 5 + T_{\frac{n}{2}} & \text{altrimenti} \end{cases}$$

Questo tipo di equazioni, che abbiamo già affrontato quando abbiamo parlato dell'algoritmo di Fibonacci, vengono dette equazioni di ricorrenza.

In particolare questa è un'equazione semplice perché c'è un unico termine su cui effettua la ricorrenza.

Trovare una soluzione alle equazioni di ricorrenza è un problema complesso, ma in questo caso particolare non è tanto complesso. Poiché qui c'è un unico termine sul cui ricorriamo, possiamo rappresentare graficamente questa equazione di ricorrenza in un modo molto semplice, come una catena di valori.

All'inizio abbiamo che l'algoritmo viene applicato su un input di dimensione n . L'algoritmo poi richiama se stesso su un input di dimensione $\frac{n}{2}$, poi su un input di dimensione $\frac{n}{4}$, $\frac{n}{8}$, e così via.

Per semplicità di calcoli non consideriamo il floor visto che si può dimostrare che non cambia assolutamente il valore asintotico.

Se andiamo a vedere i livelli, intesi come numero di chiamate ricorsive effettuate, osserviamo.

Livello	Dimensione
0	n
1	$\frac{n}{2}$
2	$\frac{n}{4}$
3	$\frac{n}{8}$
$\dots$	$\dots$
i	$\frac{n}{2^i}$

Dobbiamo quindi capire qual è l'indice i che ci permette di raggiungere il caso base, cioè $n \leq 1$, visto che quando viene raggiunto il caso base, come è immediato osservare dall'equazione del tempo, l'algoritmo non richiama più se stesso e il calcolo del tempo è immediato.

Equivale a risolvere la seguente disequazione:

$$\frac{n}{2^i} \leq 1 \iff n \leq 2^i \iff \log_2(n) \leq i$$

Da cui segue:

$$i = \lceil \log_2(n) \rceil$$

Sappiamo quindi esattamente quanti livelli verranno raggiunti dall'algoritmo in base alla dimensione iniziale dell'input.

Ora, se sappiamo per ogni livello quanto lavoro locale deve effettuare l'algoritmo, ci basterà sommare tutti i lavori per ottenere il costo totale.

Nel caso specifico abbiamo osservato che il costo localmente era di 5, cioè localmente l'algoritmo per chiamare se stesso su una dimensione di un input dimezzata deve fare 5 istruzioni.

Quindi il costo totale sarà: $2 + 5 \cdot \log_2(n)$.

Quindi, come volevamo dimostrare, l'algoritmo ha un costo asintotico superiore di $O(\log(n))$.

Attenzione. La base del logaritmo è irrilevante. È sempre possibile convertire un logaritmo in una base b_1 in un logaritmo in una base b_2 con l'algoritmo del cambio di base:

$$\log_{b_2}(x) = \frac{\log_{b_1}(x)}{\log_{b_1}(b_2)}$$

Tale algoritmo non fa altro che cambiare il coefficiente moltiplicativo che già scappiamo essere trascurabile nell'ottica della complessità asintotica.

3.5.6 Algoritmo: Ricerca del Minimo in un Array non ordinato

Ovviamente tutti i ragionamenti che effettueremo per la ricerca del minimo valgono in modo equivalente per la ricerca del massimo.

Immaginiamo di avere un array non ordinato e di voler calcolare l'indice nell'array dell'elemento minimo.

Possiamo sperare di avere una ricerca del minimo che sia meno di lineare?

Chiaramente no visto che l'array non è ordinato siamo costretti a percorrere l'array per tutta la sua lunghezza.

Questa è una differenza forte rispetto invece ad avere un algoritmo di ricerca del minimo in un array ordinato.

Chiaramente in questo caso il minimo o si trova nella prima cella dell'array o nell'ultima, quindi sarà un algoritmo a tempo costante.

L'algoritmo di ricerca del minimo in un array è il seguente.

Algoritmo 16: Ricerca del minimo in un array.

Signature min: $Array(D) \rightarrow \mathbb{N} \cup \{\perp\}$

function min((A,n))

```

1  // suppongo che non esista il minimo
   m ← ⊥
   // se l'array non è vuoto
2  if (n > 0) then
   // la prima stima è che l'indice dell'elemento minimo sia il primo
3   m ← 0
   // scorro l'array
4   for (i from 1 to (n-1)) do
   // se l'elemento corrente è minore del minimo attuale
5   if (A[i] < A[m]) then
   // aggiorniamo l'indice dell'elemento minimo
6   m ← i
   // restituisco l'indice dell'elemento minimo
7  return m

```

Attenzione. $\mathbb{N} \cup \{\perp\}$ è l'insieme dei numeri naturali unito il simbolo bottom ($\perp$) che in questo caso indica assenza di informazione.

Se l'array è vuoto non esiste minimo e quindi restituiamo $\perp$, altrimenti poniamo l'indice 0 come prima stima del minimo e poi scorrendo l'array per tutta la sua lunghezza valutiamo se è presente un elemento strettamente

minore di quello nella posizione attuale del minimo e nel caso aggiorniamo il valore della posizione del minimo; che alla fine restituiamo.

Attenzione. L'algoritmo in questione restituisce la prima posizione del minimo di un array non vuoto.

Complessità dell'Algoritmo

È ovvio osservare che questo algoritmo ha come complessità esattamente quella che ci si aspetta, cioè lineare, visto che è composto da un ciclo **for** che esegue per $n-1$ volte un numero di operazioni costanti.

È chiaramente $\Theta(n)$.

3.5.7 Algoritmo: Ricerca del Minimo in un Array Ordinato

Immaginiamo di avere un array ordinato e di voler calcolare l'indice nell'array dell'elemento minimo.

L'algoritmo di ricerca del minimo in un array è il seguente.

Algoritmo 17: Ricerca del minimo in un array ordinato.

Signature min: $SortedArray(D) \rightarrow \mathbb{N} \cup \{\perp\}$

function min((A,n))

```

1  // suppongo che non esista il minimo
   m ← ⊥
   // se l'array non è vuoto
2  if (n > 0) then
   // la prima stima è che l'indice dell'elemento minimo sia il primo
3  m ← 0
   // confronto il primo elemento dell'array con l'ultimo
   // se l'ultimo elemento è minore del primo
   // visto che l'array è ordinato
4  if (A[n-1] < A[m]) then
   // la posizione dell'elemento minimo è l'ultima
5  m ← n - 1
   // restituisco l'indice dell'elemento minimo
6  return m

```

Stiamo sfruttando il fatto che l'array sia ordinato andando a confrontare, nel caso di array non vuoto, solo i valori estremi dell'array.

Complessità dell'Algoritmo

Risulta immediato che l'algoritmo abbia complessità costante, quindi $\Theta(1)$.

3.5.8 Algoritmo: Ricerca del Successore in un Array non ordinato

Ovviamente tutti i ragionamenti che effettueremo per la ricerca del successivo di un elemento valgono in modo equivalente per la ricerca del precedente.

Immaginiamo invece di voler calcolare la posizione del successore di un elemento dato in un array non ordinato. Vedremo poi se c'è un vantaggio invece nel calcolare il successore in un array ordinato.

Abbiamo già visto che nel caso del minimo il vantaggio c'è: si passa, infatti, da lineare a costante. Possiamo sperare lo stesso per il successore?

Putroppo no.

Però, sebbene non possiamo sperare di arrivare a tempo costante, se l'array è ordinato, utilizzando una variante della ricerca binaria, si può raggiungere un tempo logaritmico.

Quindi c'è una importante differenza tra un array non ordinato, dove in generale le ricerche costano il peggio possibile, e un array ordinato, dove si viene a creare una scaletta di complessità in base a cosa si deve cercare.

Ricordando la definizione successore ci interessa trovare il minimo elemento nell'array strettamente superiore al dato, d , in input.

Attenzione. A differenza del minimo, dove nel caso l'array non sia vuoto il minimo esiste sempre, il successore non è detto che ci sia anche se l'array non è vuoto.

L'algoritmo di ricerca del successore di un dato d in un array è il seguente.

Algoritmo 18: Ricerca del successore in un array non ordinato.

Signature successor: $Array(D) \times D \rightarrow \mathbb{N} \cup \{\perp\}$

function successor((A,n), d)

```

1  // suppongo che non esista il successore
   s ← ⊥
   // se l'array non è vuoto
2  if (n > 0) then
   // scorro l'array
3  for (i from 0 to n-1) do
   // se l'elemento corrente è maggiore del dato in input
4  if (A[i] > d) then
   // se il successore non è stato assegnato
   // o il valore corrente è minore del successore attuale
   // NOTA: OR cortocircuitato
5  if ((s = ⊥) OR (A[i] < A[s])) then
   // aggiorna l'indice del successore all'indice corrente
6  s ← i
   // restituisco l'indice del successore del dato in input
7  return s

```

Se l'array è vuoto chiaramente non c'è il successore. Altrimenti, visto che l'array non ha alcuna proprietà, siamo costretti a scorrere tutto l'array; dovremo effettuare una stima del successore nell'intervallo dell'array che abbiamo già analizzato, e solo alla fine saremo sicuri che la stima sarà un valore definitivo.

Agiamo quindi in modo simile alla ricerca del minimo in un array non ordinato; c'è però un'importante differenza, ora non possiamo prendere, nel caso di array non vuoto, il primo elemento come possibile candidato.

Il primo candidato per il successore sarà il primo elemento lungo l'array che è maggiore del dato in input, d; potremmo non trovare tale elemento.

Quindi la stima iniziale sarà considerare l'assenza di un successore. Dopodiché scorriamo l'array.

Chiaramente se c'è un candidato per il successore di d, questo deve essere maggiore di d.

A questo punto bisogna solo valutare se la stima precedente sia ancora corretta, quindi valutare se il nuovo candidato sia minore della stima precedente, e nel caso aggiornare il valore della posizione del successore.

L'unica accortenza che bisogna avere è che all'inizio quando non abbiamo una prima stima " $s = \perp$ " non possiamo effettuare i confronti, quindi la prima volta che troviamo un candidato sarà per forza corretto.

Dopo aver percorso tutto l'array restituiamo il valore della posizione del successore di d.

Attenzione. Stiamo utilizzando l'OR cortocircuitato; se è vero il primo argomento non va a valutare il secondo.

L'**if** interno non potrebbe funzionare altrimenti.

Complessità dell'Algoritmo

Anche quest'algoritmo ha complessità lineare. Siamo costretti necessariamente a scorrere l'array con il ciclo **for**, che effettua delle operazioni costanti.

Quindi $\Theta(n)$.

3.5.9 Algoritmo: Ricerca del Successore Ricorsiva in un Array Ordinato

Vediamo se è possibile ottenere qualcosa di migliore per la ricerca della posizione del successore di un elemento in un array ordinato.

Abbiamo già detto che si può ottenere, ma non si può sperare di ottenere un algoritmo a tempo costante proprio perché non sapendo dov'è posizionata la prima stima, non sapremo neanche quanto l'elemento mediano sia distante dal valore di interesse; pertanto anche solo per identificare la prima potenziale stima potremmo essere costretti a fare tante ricorsioni quante sono quelle necessarie per raggiungere il caso base dell'algoritmo di ricerca binaria.

A differenza della ricerca del successore in un array non ordinato che è iterativo perché bisogna scorrere un array su cui non abbiamo alcuna informazione su come decomporre il problema, e quindi siamo naturalmente tentati a lavorare in modo iterativo, quando abbiamo struttura ricorsiva sul problema, come quella di un array ordinato, il primo approccio da tentare è quello ricorsivo, non quello iterativo.

Un po' come per la ricerca binaria c'è una funzione esterna di interfaccia e poi la versione che invece ha i due indici che vanno ad identificare l'intervallo dell'array su cui andremo a concentrarci.

Anche in questo caso, come nella ricerca binaria, i è l'indice della prima cella dell'intervallo di interesse e j è quella immediatamente superiore.

Stiamo supponendo un array di valori ordinati in senso crescente secondo una relazione d'ordine totale.

L'algoritmo per la funzione di interfaccia è il seguente.

Algoritmo 19: Ricerca del successore ricorsiva in un array ordinato (funzione di interfaccia)

Signature successor: $\text{Array}(D) \times D \rightarrow \mathbb{N} \cup \{\perp\}$

function `successor((A,n), d)`

```

1  | // chiamo la funzione ricorsiva
  | return successor((A,n), 0, n, d)

```

L'algoritmo per la funzione accessoria è il seguente.

Algoritmo 20: Ricerca del successore ricorsiva in un array ordinato (funzione accessoria)

Signature successor: $SortedArray(D) \times \mathbb{N} \times \mathbb{N} \times D \rightarrow \mathbb{N} \cup \{\perp\}$

```

function successor((A,n), i, j, d)
    // suppongo un array ordinato in modo crescente
    // se il sottoarray considerato non è vuoto
1   if (i < j) then
        // prendo l'indice mediano
2       k ← floor((i+j)/2)
        // se il dato da cercare è minore del valore attuale
3       if (A[k] > d) then
            // setta l'indice del successore al valore restituito
            // dalla funzione chiamata sulla sottoparte dell'array
            // precedente al valore corrente
4           s ← successor((A,n), i, k, d)
            // se nella sottoparte dell'array precedente al valore corrente
            // non è stato trovato un successore
5           if (s = ⊥) then
                // assegna all'indice del successore l'indice corrente
6               s ← k
            // restituisco l'indice del successore del dato in input
7           return s
8       else
            // se il dato da cercare non è minore del valore attuale
            // restituisci l'output della funzione chiamata
            // sulla sottoparte dell'array successiva al valore corrente
9           return successor((A,n), k+1, j, d)
10  return ⊥

```

Come nella ricerca binaria la prima cosa che facciamo è controllare se l'intervallo di interesse è vuoto, nel caso non ci sarà un successore.

Se l'intervallo non è vuoto, come nella ricerca binaria, dividiamo l'array in due parti perché il successore di un elemento di certo non può stare in entrambe le parti e in base al confronto del dato in input con il valore mediano sapremo dove andare a concentrare la nostra ricerca.

Quindi calcoliamo k come l'elemento mediano e separiamo la casistica; a differenza della ricerca binaria il caso minore e uguale vengono fusi perché ci interessa prendere il minimo degli elementi strettamente maggiori.

Se $A[k] > d$ abbiamo già identificato un candidato e visto che cerchiamo il minimo tra gli elementi strettamente maggiori di d non avrà senso andare ulteriormente a destra di k , dove ci sono solo valori maggiori, pertanto andremo a concentrarci sui valori posti a sinistra del candidato trovato perché k potrebbe essere la posizione di

un valore maggiore ma non il più piccolo tra i valori maggiori.

Richiamiamo quindi l'algoritmo ricorsivamente su un intervallo posto alla sinistra di k (k escluso), in questo modo se la chiamata ricorsiva ci restituirà un candidato, visto che l'array è ordinato sapremo che tale candidato è minore del valore all'indice k ed è quindi una stima migliore, se invece non restituisce un candidato, quindi restituisce $\perp$, il valore all'indice k sarà proprio il successivo di d .

Altrimenti andremo a concentrarci sui valori posti alla destra dell'elemento mediano calcolato.

Calcolo della Complessità

Come ci si aspettava l'algoritmo ha complessità: $\Theta(\log(n))$

3.5.10 Ricerca del Successore Iterativa in un Array Ordinato

Vediamo ora la versione iterativa dello stesso algoritmo, che poi vedremo si può derivare con una opportuna trasformazione.

In questo momento non ci interessa come si ottiene tramite trasformazione.

Ragioniamo in modo simile al caso della ricerca binaria visto precedentemente; se prendiamo il corpo dell'algoritmo ricorsivo e opportunamente lo embeddiamo, lo andiamo ad inserire all'interno di un ciclo, possiamo trasformarlo in una versione iterativa semplicemente in cui le chiamate ricorsive vanno a sostituire il cambio dei parametri e il ciclo sta lì ad essere eseguito intanto che non raggiungiamo il caso base.

Questo ragionamento non si applica esattamente in questo caso perché l'approccio che abbiamo appena spiegato si applica quando le funzioni ricorsive vengono dette ricorsive in coda.

Definizione funzioni ricorsive in coda

Una funzione ricorsiva è detta ricorsiva in coda quando termina solo e soltanto in due modi: o con un **return** di un valore calcolato localmente oppure con il valore restituito da una chiamata ricorsiva direttamente.

Se abbiamo una chiamata ricorsiva dopo la quale effettuiamo del lavoro prima di restituire qualcosa, questa funzione non viene detta tail recursive, cioè ricorsiva in coda, perché c'è una ricorsione che non sta in coda.

Per trasformare questo algoritmo, in generale, l'approccio è un po' più complesso. L'algoritmo sarà molto molto simile ma c'è una differenza che andremo a sottolineare.

L'algoritmo è il seguente.

Algoritmo 21: Ricerca del successore iterativa in un array ordinato

Signature successor: $Array(D) \times D \rightarrow \mathbb{N} \cup \{\perp\}$

```

function successor((A,n), d)
    // suppongo un array ordinato in modo crescente
    // setto gli indici
    // i indice iniziale del sottoarray
    // j indice finale del sottoarray
1  (i, j, s) ← (0, n, ⊥)
    // fino a quando il sotto-array considerato è non vuoto
2  while (i < j) do
    // prendo l'indice mediano
3    k ← floor((i+j)/2)
    // se il dato da cercare è minore del valore attuale
    // ho trovato un candidato per il successore
4    if (A[k] > d) then
        // pongo l'indice finale del sottoarray da considerare a quello attuale
        // in questo modo andremo a valutare la sottoparte dell'array
        // precedente al valore corrente
        // e setto l'indice del successore al dato in input a quello attuale
5        (j, s) ← (k, k)
6    else
        // se il dato da cercare non è minore del valore attuale
        // pongo l'indice iniziale del sottoarray da considerare
        // al successivo di quello attuale
        // in questo modo andremo a valutare la sottoparte dell'array
        // successivo al valore corrente
7        i ← k + 1
    // restituisco l'indice di posizione del successore del dato in input
8  return s

```

Come prima istruzione facciamo un assegnamento, ponendo come nel caso della versione ricorsiva i a 0 e j ad n ; inoltre nella versione iterativa partiamo con una stima che è l'assenza del successore. È come se stessimo vagliando l'array vuoto, in un array vuoto non c'è successore.

Quando vediamo una versione iterativa bisogna pensare sempre che le variabili stanno dando un'informazione corretta e completa su quella parte di array che abbiamo già analizzato; rispetto al valore finale potranno essere completamente sballate ma sul pezzo che abbiamo analizzato dovranno essere corrette; poi aumentando le informazioni tali valori verranno aggiornati.

Ovviamente per aumentare l'informazione quello che andremo a fare sarà un ciclo **while**.

Ovviamente un algoritmo iterativo equivalente ad un algoritmo ricorsivo necessariamente ha bisogno di un ciclo; non c'è altro modo.

Per essere precisi, potrebbe non essere sufficiente un ciclo. Al momento tutti gli algoritmi che abbiamo analizzato non hanno bisogno di uno stack esplicito ma questo perché erano algoritmi particolari e soprattutto sono algoritmi con chiamate ricorsive uniche, nel senso che tutte le chiamate ricorsive sono indipendenti.

Calcoliamo ovviamente il nostro k attuale e a questo punto andiamo a confrontare.

Qui i due algoritmi si separano nel senso che non potremo semplicemente prendere il codice della versione ricorsiva e copiarlo andando a sostituire soltanto le chiamate ricorsive con opportuni cambi di parametri così come abbiamo visto nell'esempio della ricerca binaria.

In questo caso se abbiamo una stima precedente essa avrà analizzato una parte dell'array dove quella stima è sicuramente o la migliore globalmente o qualcosa di peggiore rispetto a quella che troveremo.

Di conseguenza se ora troviamo un valore che è esso stesso una stima, questa sarà migliorativa rispetto a quella già trovata e semplicemente la rimpiazziamo; a questo punto dobbiamo andare ad analizzare quella sottoparte di array che ci interessa in base al valore del confronto.

Complessità dell'Algoritmo

Chiaramente l'algoritmo in questione ha la stessa complessità della versione ricorsiva che come ricordiamo ha complessità $\Theta(\log(n))$.

3.6 Algoritmi sulle Liste Concatenate

A questo punto passiamo alle liste.

Andiamo a vedere come fare una ricerca nelle liste. Ne abbiamo ampiamente parlato a livello astratto; ora ci soffermeremo sulle Liste Concatenate e Liste Concatenate Ordinate.

Utilizzeremo la notazione punto indipendentemente dal fatto che sia un puntatore o meno, risulta indifferente al nostro livello di astrazione.

3.6.1 Algoritmo: Ricerca in una Lista Concatenata non ordinata

Se vogliamo fare una ricerca in una lista non ordinata siamo costretti a scorrere tutta la lista, con la differenza che non dobbiamo conoscere quanto sia lunga la lista.

Stiamo assumendo la lista come un puntatore a nodi, quindi L è il puntatore al primo nodo della lista.

L'algoritmo è il seguente.

Algoritmo 22: Ricerca in una lista concatenata non ordinata

Signature search: $List(D) \times D \rightarrow \mathbb{B}$

```
function search(L, d)
    // fino a quando la sottolista considerata è non vuota
1   while (L  $\neq$   $\perp$ ) do
        // se il dato contenuto nel nodo corrente è uguale al dato da cercare
2       if (L.dat = d) then
            // ... restituisco true
3           return True
4       else
            // altrimenti avanzo nella lista
5           L  $\leftarrow$  L.next
        // se la lista è terminata e non ho trovato il dato restituisco false
6   return False
```

Quindi l'uscita dal ciclo **while** starà ad indicare il non aver trovato il dato.

Complessità dell'Algoritmo

Come ci si aspettava la complessità dell'algoritmo è: $O(n)$ e $\Omega(1)$, dove n è il numero di nodi della lista L .

3.6.2 Algoritmo: Ricerca ricorsiva in una Lista Concatenata Ordinata

Supponiamo invece di avere una lista di dati ordinati.

È possibile ottenere una complessità asintotica migliore di lineare?

No, perché non abbiamo l'accesso diretto.

Tuttavia l'algoritmo migliore è comunque quello della ricerca di un dato in una lista o c'è modo di migliorare l'algoritmo? E se sì in che senso si può migliorare l'algoritmo?

Quello che possiamo fare è andare a distinguere effettivamente due differenti complessità.

Abbiamo sempre detto che le operazioni di confronto, come quella tra `L.dat` e `d`, sono a tempo costante.

Se facciamo tale assunzione allora l'algoritmo di ricerca di un dato in una lista risulta essere il migliore anche per la ricerca di un dato in una lista ordinata.

Tuttavia, se consideriamo il fatto che i confronti non siano a tempo costante, ma che abbiano una complessità non trascurabile (ad esempio nel caso di confronti tra stringhe che hanno una complessità lineare) allora cercare di minimizzare i confronti tra dati ci permette di ottenere un algoritmo migliore.

Alla fine dovremo comunque scorrere l'intera lista, ma se possiamo evitare i confronti, per quanto possibile ovviamente, è meglio.

Allora, nel caso delle liste ordinate si può ottenere un miglioramento di questo algoritmo, assumendo appunto che i confronti siano pesanti, dove il numero di operazioni da fare è lineare ma il numero di confronti non è più lineare ma logaritmico.

Sfruttiamo ancora una volta l'idea della ricerca binaria, solo che, poiché non abbiamo indirizzamento diretto, dobbiamo usare un surrogato.

In pratica calcoliamo un indice, poi ci dobbiamo spostare fino a quell'indice, però senza confrontare. A quel punto confrontiamo.

E allora confrontiamo praticamente come confrontavamo con l'algoritmo di ricerca binaria anche se per arrivare nella posizione voluta dobbiamo spostarci un passo alla volta.

In questo caso però è necessario conoscere la dimensione della lista ordinata.

Daremo prima l'algoritmo ricorsivo.

L'algoritmo ricorsivo è quasi identico a quello su array, c'è la differenza che invece di avere due indici uno di essi viene sostituito dal puntatore al nodo della sottolista che ci interessa.

Nello specifico l'indice di posizione della fine della sottolista considerata rimane; quello che indica l'inizio viene sostituito da un puntatore.

Questo algoritmo assume che `n` sia minore o uguale della lunghezza della lista.

L'algoritmo è il seguente.

Algoritmo 23: Ricerca ricorsiva in una lista concatenata ordinata

Signature search: $List(D) \times \mathbb{N} \times D \rightarrow \mathbb{B}$

```

function search(L, n, d)
    // suppongo una lista ordinata in modo crescente
    // se la sottolista considerata è non vuota
1  if (n > 0) then
    // copio il puntatore che indica l'inizio della sottolista
2      Z ← L
    // calcolo l'indice mediano
3      k ← floor(n/2)
    // mi sposto al nodo di mediano nella sottolista
4      for (i from 1 to k) do
5          Z ← Z.next
    // se il dato contenuto nel nodo corrente è maggiore al dato da cercare
6      if (Z.dat > d) then
        // ... aggiorno la lunghezza della sottolista da considerare
        // valuto la sottoparte della lista precedente al valore corrente
7          return search(L, k, d)
    // se il dato contenuto nel nodo corrente è minore del dato da cercare
8      else if (Z.dat < d) then
        // ... aggiorno la lunghezza della sottolista da considerare
        // valuto la sottoparte della lista successiva al valore corrente
9          return search(Z.next, n-k-1, d)
10     else
        // se il dato contenuto nel nodo corrente è uguale al dato da cercare
        // restituisco true
11         return True
    // se la sottolista considerata è vuota e non ho trovato il dato in input
    // restituisco false
12 return False

```

La differenza con l'algoritmo della ricerca binaria è che dobbiamo utilizzare un surrogato dell'indirizzamento, in questo caso un ciclo **for**, per raggiungere il nodo di posizione mediana.

A questo punto l'algoritmo è come quello della ricerca binaria con la differenza che quando andiamo a considerare la parte della lista successiva al nodo corrente non dobbiamo cambiare un indice ma il puntatore che indica l'inizio della lista da considerare, mentre quando ci spostiamo nella sottolista precedente al nodo corrente cambiamo solo la lunghezza della lista da considerare.

Ovviamente il numero di operazioni da effettuare, intese come operazioni di spostamento, è lineare.

Perché?

Il ciclo **for** viene effettuato una volta per $\frac{n}{2}$, la seconda volta per $\frac{n}{4}$, la terza per $\frac{n}{8}$ e così via.

Sappiamo che $\sum_{i=1}^{+\infty} \frac{1}{2^i}$ converge ad 1.

Di conseguenza, nel caso peggiore, nel complesso le operazioni effettuate nel ciclo **for** vengono effettuate n volte.

Ma le operazioni di confronto con il dato da cercare non vengono eseguite un numero lineare di volte, ma un numero logaritmico di volte rispetto alla lunghezza della lista in input ovviamente, per le stesse motivazioni che portano l'algoritmo di ricerca binaria ad avere una complessità logaritmica nel caso peggiore.

Complessità dell'Algoritmo

L'algoritmo appena mostrato ha complessità a livello di tempo pari a: $\Theta(n)$; invece la sua complessità a livello di confronti è pari a: $O(\log(n))$ e $\Omega(1)$.

3.6.3 Algoritmo: Ricerca iterativa in una Lista Concatenata Ordinata

A questo punto vediamo la visione iterativa.

Si tratta di una semplice traduzione come nel caso dell'algoritmo della ricerca binaria.

L'algoritmo è il seguente.

Algoritmo 24: Ricerca iterativa in una lista concatenata ordinata

Signature search: $List(D) \times \mathbb{N} \times D \rightarrow \mathbb{B}$

```

function search(L, n, d)
    // suppongo una lista ordinata in modo crescente
    // se la sottolista considerata è non vuota
1  while (n > 0) do
    // copio il puntatore che indica l'inizio della sottolista
    // e calcolo l'indice mediano
2    (Z, k) ← (L, floor(n/2))
    // mi sposto al nodo di mediano nella sottolista
3    for (i from 1 to k) do
4      Z ← Z.next
    // se il dato contenuto nel nodo corrente è maggiore al dato da cercare
5    if (Z.dat > d) then
        // ... aggiorno la lunghezza della sottolista da considerare
        // per valutare la sottoparte della lista precedente al valore corrente
6      n ← k
    // se il dato contenuto nel nodo corrente è minore del dato da cercare
7    else if (Z.dat < d) then
        // ... aggiorno l'inizio della sottolista da considerare
        // e aggiorno la lunghezza della sottolista da considerare
        // per valutare la sottoparte della lista successiva al valore corrente
8      (L, n) ← (Z.next, n-k-1)
9    else
        // se il dato contenuto nel nodo corrente è uguale al dato da cercare
        // restituisco true
10     return True
    // se la sottolista considerata è vuota e non ho trovato il dato in input
11 return False

```

Complessità dell'Algoritmo

L'algoritmo appena mostrato essendo una traduzione del suo corrispettivo ricorsivo ha le stesse complessità, ovvero: ha complessità a livello di tempo pari a: $\Theta(n)$; invece la sua complessità a livello di confronti è pari a: $O(\log(n))$ e $\Omega(1)$.

Ovviamente si possono fare anche gli algoritmi di successor per le liste e per le liste ordinate, ma non li vedremo. Sono algoritmi essenzialmente identici a quelli visti per gli array.

3.7 Operazioni di Inserimento negli Array

Vediamo ora brevemente l'inserimento in un array non ordinato.

Come si effettua?

Semplicemente si deve allocare un nuovo spazio in memoria, dopodiché copiamo i dati dell'array e inseriamo il nuovo dato.

Ora, ci sono i due approcci sul ridimensionamento di un array.

Possiamo creare esattamente un array di dimensione $n + 1$, in questo modo si utilizza il minimo indispensabile di memoria però ciò forza che ogni inserimento significa fare un nuovo spostamento di dati e significa avere un inserimento che costa $\Theta(n)$.

Un altro approccio è quello di andare ad allocare il doppio della memoria utilizzata dall'array o comunque una memoria proporzionale a quella già allocata.

Cioè dobbiamo tener traccia di due informazioni all'interno della nostra struttura dati.

Oltre al puntatore all'array e alla sua lunghezza se vogliamo rendere l'inserimento più efficiente dobbiamo disaccoppiare la lunghezza intesa come numero di dati dalla lunghezza intesa come effettiva occupazione della struttura.

Quando abbiamo lavorato sugli array abbiamo sempre visto l'array come una coppia (A, l) perché assumevamo l'essere sia il footprint in memoria, cioè quanto spazio occupa in memoria, sia quanti dati abbiamo. Se vogliamo però avere un inserimento efficiente allora dobbiamo disaccoppiare l'occupazione di memoria dai dati presenti.

Un array per noi sarà quindi una terna (A, n, m) dove n sono dati ed m è quante celle di memoria stiamo occupando.

Ovviamente deve essere verificato un invariante, una proprietà che dice che necessariamente $n \leq m$; possiamo quindi avere meno dati delle celle allocate.

Con questa idea quello che si fa è che quando inseriamo un dato se n corrisponde ad m , cioè se l'array è saturo, allora per forza siamo costretti a riallocare un nuovo array e inserire un nuovo dato, ma nel momento in cui lo andiamo a riallocare, cioè quando ci troviamo nella situazione (A, n, n) ci sposteremo in un array A' con $n + 1$ dati e con una dimensione in memoria proporzionale alla precedente secondo un fattore moltiplicativo, per i nostri ragionamenti considereremo il doppio della memoria precedentemente allocata; quindi ci sposteremo in una situazione $(A', n + 1, 2n)$.

Cosa accade ora?

Al prossimo inserimento non dovremo più allocare memoria, abbiamo ancora $n - 1$ celle a disposizione, possiamo fare $n - 1$ inserimenti a tempo costante perché il valore che indica la dimensione di quanti dati abbiamo è automaticamente anche l'indice alla prima cella a disposizione.

Quando avremo saturato l'array e vogliamo effettuare un nuovo inserimento, saremo costretti a raddoppiare nuovamente la memoria.

Ma supponendo la dimensione iniziale dell'array n , dopo il primo ridimensionamento abbiamo aspettato come tempo $n - 1$, al secondo ridimensionamento aspetteremo $2n - 1$, al successivo ridimensionamento aspetteremo $4n - 1$ e così via.

È facile rendersi conto che stiamo dilazionando le allocazioni e distanziando i punti nei quali necessitiamo di

riallocare memoria.

Andando a sommare tutti i contributi osserviamo che il contributo totale è lineare, non più quadratico e questo significa che se facciamo n operazioni di inserimento mediamente facciamo $O(n)$ operazioni totalmente; quando andiamo a fare la media, cioè se facciamo n operazioni di inserimento e totalmente abbiamo $O(n)$ operazioni, cioè un certo fattore moltiplicativo per n , significa che mediamente (non la vera singola operazione) ogni operazione di inserimento è costante, mentre nell'approccio precedente ogni operazione di inserimento era lineare, ecco perché complessivamente diventava quadratico mentre ora complessivamente è solo lineare.

Il coefficiente moltiplicativo 2 non è fondamentale, possiamo usare un qualunque fattore moltiplicativo ovviamente strettamente maggiore di 1.

3.7.1 Esempio di Inserimento in un Array

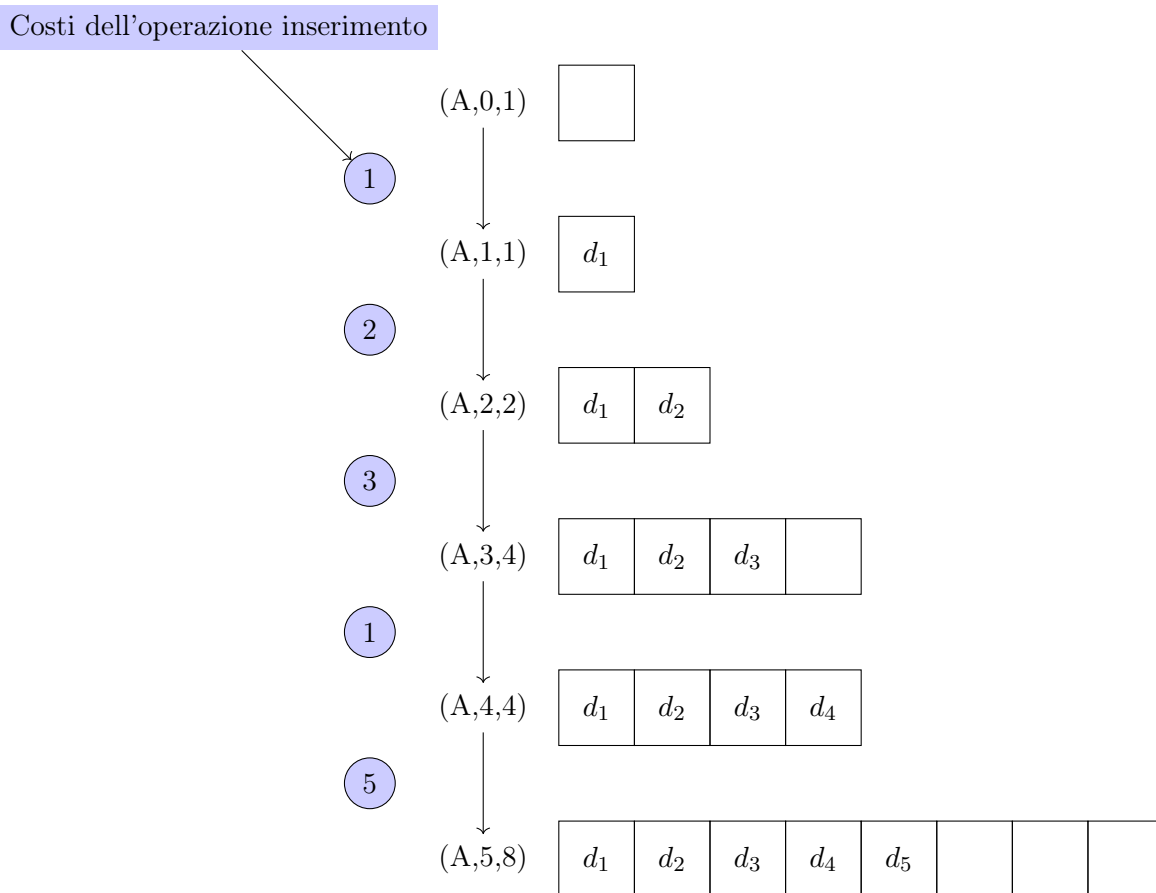


Figura 3.1: Esempio di Inserimento negli Array

3.8 Operazioni di Rimozione negli Array

Abbiamo visto la gestione dell'inserimento di un nuovo dato in un array, in rimozione si fa qualcosa di molto simile.

Se vogliamo andare a rimuovere dal nostro array un dato ovviamente dobbiamo andare a cercarlo, una volta che l'abbiamo identificato si potrebbe pensare di riallocare nuova memoria per una cella in meno, ma si avrebbero gli stessi problemi visti per l'inserimento; ogni operazione costerebbe lineare e in particolare bisognerebbe andare a copiare i dati ogni volta, senza nemmeno considerare il costo della ricerca.

Quello che si fa invece di solito è deallocare memoria solo quando andiamo a utilizzarne una frazione della memoria di partenza.

Per esempio, se il fattore moltiplicativo è 2, di solito quello che si fa è che si va a deallocare memoria solo se l'occupazione di memoria è $\frac{1}{4}$.

Perché?

Perché in quel modo si dimezza il footprint di memoria e ci troveremo con metà dati e metà celle vuote in caso di possibili inserimenti.

Il fattore moltiplicativo di riduzione di solito è correlato a quello moltiplicativo di incremento ma non è necessario che siano strettamente correlati; potremmo avere fattori moltiplicativi di riduzione e incremento separati.

Attenzione che in alcuni casi potrebbe essere necessario shiftare i dati, aggiungendo costo computazionale. Lo shift dei dati è necessario se vogliamo mantenere informazioni sui dati, come nel caso di array senza ripetizioni o array ordinati.

3.9 Algoritmi sugli Stack e Queue

Abbiamo parlato di stack e queue.

Queste strutture dati sono molto importanti in particolare in funzione del loro utilizzo all'interno di algoritmi un po' più avanzati, quali possono essere gli algoritmi su alberi, su grafi e così via.

E pertanto è importante avere chiara quale sia l'interfaccia di gestione di queste strutture dati.

Noi non andremo tanto nel dettaglio su come implementare queste strutture dati; ma è importante avere un'interfaccia comune in modo tale che nel momento in cui poi andremo ad utilizzarle sia chiaro quale sia la funzione, l'utilizzo, la semantica delle funzioni su stack e queue. Daremo le signature delle funzioni.

In particolare uno stack, ha bisogno di essere costruito, quindi ci sarà un metodo chiamato `EmptyStack`, il cui tipo è il tipo stack su un certo insieme di dati D , non necessariamente noto a priori.

$$\text{empty_stack} : \text{Stack}(D) \quad (3.12)$$

Questo è un esempio molto semplice di generics, cioè di una struttura dati che non ha bisogno a priori di conoscere qual è il tipo di dato, perché tutte le funzioni sono agnostiche sul tipo di dato specifico.

Ovviamente poi quando va implementato, quando va diciamo fatta una chiamata vera e propria, il tipo di dato dovrà essere noto, ma l'implementatore della struttura può farne a meno, cioè non ha bisogno di conoscere questo tipo di dato, perché tutte le operazioni che verranno eseguite sono assolutamente generiche, per questo infatti vengono chiamate anche generics rispetto al tipo di dato.

Quindi c'è una funzione sicuramente di costruzione di uno stack arbitrario, che creerà uno stack vuoto.

Similmente ci sarà una funzione di `EmptyQueue`, il cui tipo è ovviamente una queue su un tipo di data astratto.

$$\text{empty_queue} : \text{Queue}(D) \quad (3.13)$$

Questo viene detto in letteratura un costruttore, a noi non interessa come sia fatto questo costruttore, l'importante è che sia nota l'esistenza di un metodo, appunto, che costruisce uno stack e una queue vuoti.

Fatto ciò vediamo invece le funzioni che ci permettono di gestire stack e queue.

Ovviamente partendo da uno stack, o una queue, vuoto, l'idea è di poterlo popolare con i dati che ci interessano, di conseguenza dovremmo avere una funzione di inserimento in stack e in queue. Queste funzioni normalmente vengono dette di push per gli stack e di enqueue per le code.

Quindi avremo una funzione push che in input prende uno stack su un certo tipo di dati, il dato che noi vogliamo inserire e restituisce un nuovo stack sempre sullo stesso tipo di dati.

$$\text{push} : \text{Stack}(D) \times D \rightarrow \text{Stack}(D) \quad (3.14)$$

Ovviamente la stessa identica cosa per le queue.

$$\text{enqueue} : \text{Queue}(D) \times D \rightarrow \text{Queue}(D) \quad (3.15)$$

Facciamo un esempio di utilizzo.

La funzione push prende in input uno stack e un dato e sostituisce il vecchio stack in input con un nuovo stack che ora conterrà, in testa ovviamente, il dato in input.

La funzione enqueue prende in input una queue e un dato e sostituisce la vecchia queue in input con una nuova queue che ora conterrà, in coda ovviamente, il dato in input.

Una volta che abbiamo un certo stack, o una queue, popolato potremmo voler leggere senza modificarlo l'unico elemento accessibile in stack o queue, in generale senza modificare lo stack o la queue.

In questo caso ovviamente possiamo avere un'operazione di top, che prende uno stack e restituisce un dato, cioè il dato che è in testa allo stack; similmente possiamo avere la funzione di head, che prende una queue e restituisce un certo dato d che è in testa alla queue.

$$\text{top} : \text{Stack}(D) \rightarrow D \quad (3.16)$$

$$\text{head} : \text{Queue}(D) \rightarrow D \quad (3.17)$$

Facciamo anche qui l'esempio di utilizzo.

La funzione top semplicemente va a restituire il valore dell'elemento in testa allo stack in input.

La funzione dequeue semplicemente va a restituire il valore dell'elemento in testa alla queue in input.

Ovviamente queste sono funzioni di sola lettura, cioè semplicemente restituiscono qual è il dato in testa, ma la struttura stack e queue rimane del tutto invariata.

Potremmo invece voler modificare la struttura andando ad eliminare i dati.

Le funzioni che di solito vengono utilizzate per eliminare i dati da stack e queue sono chiamate pop per lo stack e dequeue per la queue.

La funzione pop prende uno stack e restituisce un altro stack.

$$\text{pop} : \text{Stack}(D) \rightarrow \text{Stack}(D) \quad (3.18)$$

La funzione dequeue prende una queue e restituisce un'altra queue.

$$\text{dequeue} : \text{Queue}(D) \rightarrow \text{Queue}(D) \quad (3.19)$$

Facciamo anche qui l'esempio di utilizzo.

La funzione pop semplicemente va a sostituire lo stack in input con un nuovo stack, che è quello in input da cui abbiamo eliminato l'elemento in testa.

La funzione *dequeue* semplicemente va a sostituire la *queue* in input con una nuova *queue*, che è quella in input da cui abbiamo eliminato l'elemento in testa.

Poi infine, a volte è utile avere un'operazione che vada a fondere queste due, cioè che non solo ci permetta di leggere il dato, ma ci permetta anche di rimuoverlo automaticamente.

Queste due funzioni di solito si chiamano *top_&_pop* per lo *stack* e *head_&_dequeue* per la *queue*.

$$\text{top_}\&_\text{pop} : \text{Stack}(D) \rightarrow \text{Stack}(D) \times D \quad (3.20)$$

$$\text{head_}\&_\text{dequeue} : \text{Queue}(D) \rightarrow \text{Queue}(D) \times D \quad (3.21)$$

Facciamo anche qui l'esempio di utilizzo.

La funzione *top_&_pop* semplicemente va a sostituire lo *stack* in input con un nuovo *stack*, che è quello in input da cui abbiamo eliminato l'elemento in testa, elemento che viene anche restituito.

La funzione *head_&_dequeue* semplicemente va a sostituire la *queue* in input con una nuova *queue*, che è quella in input da cui abbiamo eliminato l'elemento in testa, elemento che viene anche restituito.

Ovviamente si potrebbe fare a meno di queste ultime funzioni perché ottenibili da quelle descritte precedentemente.

Noi non andremo ad analizzare direttamente come implementare queste funzioni; però è importante avere chiara in mente l'idea di qual è l'interfaccia di accesso, cioè praticamente quali sono le funzioni che permettono di modificare una struttura dati, in particolare per *stack* e *queue*, perché poi le utilizzeremo molto spesso.

Capitolo 4

Alberi

"What's really going to bake your noodle later on is, would you still have broken it if I hadn't said anything?"

– The Oracle (The Matrix)

4.1 Introduzione

Abbiamo visto che quando l'array è ordinato abbiamo un'efficientissima funzione di ricerca, però il problema è gestire l'array.

Gestire un array significa che possiamo pure allocare la memoria in modo efficiente ma se è ordinato dobbiamo tenerlo ordinato, il che significa che non possiamo sperare di avere operazioni costanti anche avendo spazio a disposizione perché se dobbiamo inserire un dato nell'array siamo costretti ad inserirlo in una posizione ben precisa, dovendo spostare quindi i dati presenti di conseguenza.

In effetti il problema è che l'ordinamento strutturale lineare dell'array che coincide con l'ordinamento dei dati è un vincolo troppo stretto.

Ora, dal punto di vista strutturale abbiamo un problema perché l'ordinamento lineare della memorizzazione corrisponde strettamente all'ordinamento dei dati e questa corrispondenza completa tra i due ci forza che nel momento in cui vogliamo inserire un dato in una certa posizione siamo costretti a spostare i dati fisicamente in memoria.

Allora possiamo pensare in un qualche modo disaccoppiare la memorizzazione dei dati dall'ordine dei dati e trarre vantaggio da questo disaccoppiamento.

La struttura che ora andremo a vedere grazie a tale disaccoppiamento riesce ad avere una ricerca logaritmica ed anche un inserimento logaritmico, si tratta degli alberi binari di ricerca bilanciati. Cioè generalizziamo la struttura dell'array a non essere più lineare ma ad essere ad albero.

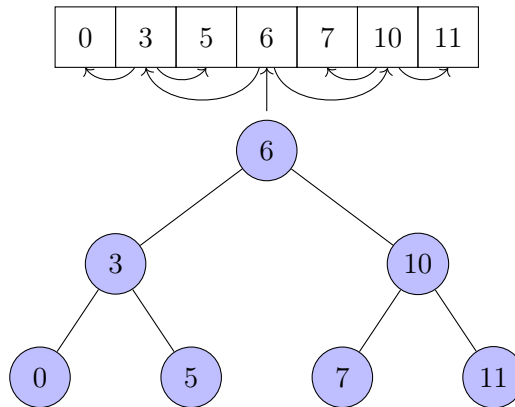
Esempio

Figura 4.1: Esempio di disaccoppiamento della memorizzazione dei dati dal loro ordine

In particolare noi lavoreremo su alberi binari.

In effetti quello che diciamo si potrebbe generalizzare su alberi di arietà arbitraria, ma per semplificarci la vita lavoreremo solo su alberi binari dove binario significa che ogni nodo ha al più 2 figli.

4.2 Definizione di Albero

Matematicamente che cos'è un albero?

Un albero è un grafo senza cicli e connesso.

La definizione che andremo a dare noi è equivalente a questa ma è un pò più vicina a quella che poi sarà la rappresentazione in memoria dell'albero.

Per noi un albero sarà un insieme di nodi.

La definizione di albero è la seguente:

$$T \text{ è un albero} \iff T = \emptyset \vee \exists n \in T (\exists T_l, T_r \subseteq T \setminus \{n\} (T_l \cap T_r = \emptyset)) \quad (4.1)$$

Dove: T_l e T_r sono alberi e sono rispettivamente, l'albero di sinistra e l'albero di destra e n è un nodo di T , i cui figli sono gli alberi sopra citati.

Questa è una definizione molto astratta a livello matematico. Dobbiamo però riversarla in una rappresentazione concreta in memoria.

Sicuramente l'albero vuoto può essere facilmente rappresentato con un puntatore a `NULL`; se l'albero non è vuoto deve esistere un nodo almeno, che sarà la radice, e faremo in modo che il puntatore punti a una struttura dati chiamata nodo.

Dobbiamo, però, avere delle proprietà su questo nodo perché il nodo sicuramente avrà un dato e poi avrà un campo chiamato `left` (l) e un altro campo chiamato `right` (r).

Il campo l sarà un puntatore alla radice del sottoalbero sinistro T_l , allo stesso modo il campo r sarà un puntatore alla radice del sottoalbero destro, T_r .

Se il sottoalbero sinistro (o destro) dovesse essere vuoto semplicemente utilizzeremmo la stessa identica idea dell'albero vuoto, cioè un puntatore a `NULL`, che indicheremo con $\perp$.

Un albero si dice pieno se non è possibile aggiungere un nodo all'albero senza aumentarne i livelli; cioè data l'altezza è l'albero più grande possibile.

Esempio di nodo

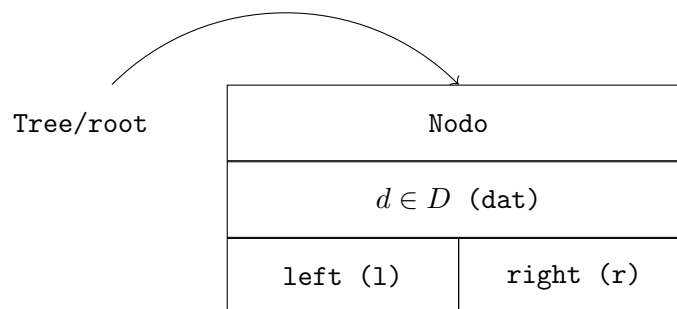


Figura 4.2: Nodo di un albero

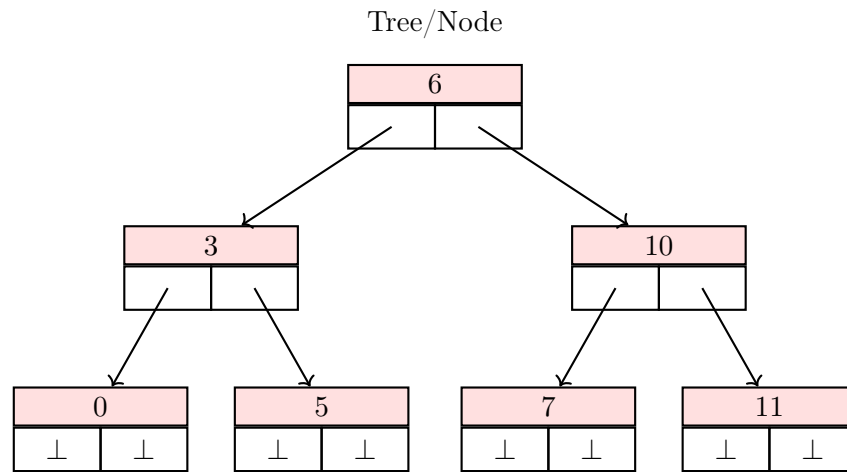
Esempio di albero

Figura 4.3: Albero

4.3 Visite su un albero binario

Vediamo ora l'analisi dei dati nell'albero e in particolare come questa analisi possa avvenire.

Ad esempio immaginiamo di voler semplicemente stampare il contenuto di una lista. Ci sono due modi per farlo: nell'ordine dalla testa alla coda e nell'ordine inverso.

Qualunque cosa venga effettuato sui dati viene detta una visita di quel dato.

Ci sono, quindi, due ordinamenti di visita che spesso vengono detti in preorder, cioè prima lavoriamo sul nodo corrente e poi andiamo sui prossimi, questo è l'ordine che va dalla testa alla coda; o l'altro che viene detto in postorder, cioè prima di lavorare sul nodo lavoriamo su ciò che viene dopo e solo al ritorno, quando avremo finito di lavorare sul resto, lavoreremo sul nodo; essenzialmente le operazioni di scoperta e visita del nodo avvengono in momenti separati.

Scoprire un nodo significa accedervi mediante il puntatore, visitarlo invece denota operazioni sul dato contenuto nel nodo.

In questo momento, ci interessa far capire che su una lista ci sono due visite canoniche: in preorder e in postorder. Perché solo due?

Perché la struttura è lineare.

Scriviamo l'algoritmo per entrambi i metodi.

4.3.1 Algoritmo: Visita in preorder di una Lista

Stampa di una lista in preorder.

Algoritmo 25: Visita in preorder in una lista concatenata

Signature print: $List(D)$

```

procedure print(L)
  // se la sottolista non è terminata
1  if (L  $\neq \perp$ ) then
    // stampo il dato
2    print_data(L.dat)
    // chiamo la funzione print sul prossimo nodo
3    print(L.next)
  
```

Una funzione che non restituisce nulla viene anche chiamata procedura o procedure. Da questo momento useremo il termine function anche per le procedure.

Sottolineiamo che questo algoritmo è tail recursive, quindi la traduzione nella versione iterativa risulta immediata.

4.3.2 Algoritmo: Visita in postorder di una Lista

Stampa di una lista in postorder.

Algoritmo 26: Visita in postorder in una lista concatenata

Signature print: $List(D)$

```

function print(L)
  // se la sottolista non è terminata
1  if (L  $\neq \perp$ ) then
    // chiamo la funzione print sul prossimo nodo
2    print(L.next)
    // stampo il dato
3    print_data(L.dat)
  
```

Sottolineiamo che questo algoritmo non è tail recursive, vedremo che per essere tradotto nella corrispondente versione iterativa sarà necessario uno stack.

4.3.3 Visite su un albero binario

Andiamo ora a guardare quante visite canoniche sono possibili su un albero binario.

Preannunciamo che le visite canoniche su un albero binario sono 4.

La prima visita, che dovrebbe essere quella in un certo senso più intuitiva, è quella di scorrere l'albero per livelli; è detta visita in ampiezza perché appunto visitiamo l'albero andando a guardare la sua ampiezza prima (per convenzione da sinistra a destra), non andiamo in profondità.

Questa visita però non ha nulla a che vedere con il collegamento immediato tra i nodi, non segue infatti i puntatori. L'unico collegamento è il fatto di essere sul medesimo livello.

Come dicevamo esistono 3 viste in profondità: una chiamata preorder, e una postorder e una chiamata inorder.

Così come nelle liste abbiamo detto che la funzione che prima lavora sul nodo e poi va a lavorare sul resto era detta preorder possiamo assumere di fare esattamente la stessa identica cosa sull'albero; si parte in radice, perché la radice è l'accesso all'albero (non abbiamo altri modi di accedere all'albero), e prima stampiamo il valore del nodo radice; poi cosa facciamo?

Qui non possiamo immaginare di avere una sola chiamata ricorsiva perché l'albero ha due direzioni sinistra e destra (assumiamo per convenzione che si vada prima a sinistra e poi a destra), quindi andremo prima nel sottoalbero sinistro e solo dopo averlo visitato andremo nel sottoalbero destro.

4.3.4 Algoritmo: visita in preorder su un albero binario

Vediamo la versione ricorsiva dell'algoritmo di visita in preorder in un albero binario.

Algoritmo 27: Visita in preorder in un albero binario

Signature print: $Tree(D)$

```
function print(T)
  // se il sottoalbero è non vuoto
1  if (T !=  $\perp$ ) then
    // stampo il dato
2    print_data(T.dat)
    // esploro il sottoalbero sinistro
3    print(T.l)
    // esploro il sottoalbero destro
4    print(T.r)
```

Osservando questa funzione è abbastanza evidente che ci sono altre due funzioni immediate.

Basandosi sull'albero in figura 4.3, un esempio di esecuzione della visita in preorder è il seguente:

Visita in profondità (preorder): 6 - 3 - 0 - 5 - 10 - 7 - 11

4.3.5 Algoritmo: visita in postorder su un albero binario

Ricordiamo che il postorder è quando scopriamo un nodo però prima di effettuare il lavoro sul nodo lavoriamo sul resto della struttura, che nel caso di un albero binario significa andare prima a sinistra, poi a destra, e solo dopo tornare al nodo ed effettuare del lavoro.

Ecco l'algoritmo della visita in postorder in un albero binario.

Algoritmo 28: Visita in postorder in un albero binario

Signature print: $Tree(D)$

```
function print(T)
    // se il sottoalbero è non vuoto
1   if (T !=  $\perp$ ) then
        // esploro il sottoalbero sinistro
2       print(T.l)
        // esploro il sottoalbero destro
3       print(T.r)
        // stampo il dato
4       print_data(T.dat)
```

Basandosi sull'albero in figura 4.3, un esempio di esecuzione della visita in postorder è il seguente:

Visita in profondità (postorder): 0 - 5 - 3 - 7 - 11 - 10 - 6

4.3.6 Algoritmo: visita in inorder su un albero binario

Nulla vieta di stampare a metà, cioè effettuiamo prima il lavoro a sinistra, poi il lavoro sul nodo corrente, e poi andiamo a destra.

Questa invece è la cosiddetta visita inorder.

Questa visita ha senso però solo per gli alberi binari.

Ecco l'algoritmo della visita inorder in un albero binario.

Algoritmo 29: Visita in inorder in un albero binario

Signature print: $Tree(D)$

```

function print(T)
  // se il sottoalbero è non vuoto
1  if (T !=  $\perp$ ) then
    // esploro il sottoalbero sinistro
2    print(T.l)
    // stampo il dato
3    print_data(T.dat)
    // esploro il sottoalbero destro
4    print(T.r)
  
```

Si può osservare facilmente che se i dati sono ordinati, nel senso che per ogni nodo alla sinistra abbiamo tutti i dati strettamente inferiori del valore del nodo e alla destra tutto ciò che è strettamente superiore, (concetto di albero binario ordinato, che viene detto albero binario di ricerca) una visita in ordine percorrerà i dati nell'ordine dei dati.

Basandosi sull'albero in figura 4.3, un esempio di esecuzione della visita in inorder è il seguente:

Visita in profondità (inorder): 0 - 3 - 5 - 6 - 7 - 10 - 11

La visita in post ordine è quella più potente, in particolare con il post ordine è possibile simulare le altre due visite in profondità (non spiegheremo come farlo).

Il post ordine, è quello che si ricorda delle informazioni più a lungo, perché le utilizzerà solo dopo aver fatto tutte e due le chiamate, ciò permette di simulare opportunamente con il postorder le altre due, inorder dove ci serve ricordarla a metà, o il preorder dove non ci serve proprio ricordarla, ma non è possibile fare il viceversa, perché una volta che perdiamo l'informazione non è possibile recuperarla.

Successivamente vedremo come generalizzare questa funzione di stampa a qualcosa che non è la semplice stampa, che viene detta funzione di fold.

4.3.7 Algoritmo: visita in ampiezza su un albero binario

Vediamo adesso la visita in ampiezza.

In ampiezza purtroppo a differenza della profondità non possiamo basarci banalmente sull'adiacenza tra figli.

C'è un'informazione in comune che è appunto un'informazione relativa ai livelli ma in qualche modo va sintetizzata in un'informazione più elementare che è semplicemente l'informazione dell'adiacenza.

Come facciamo a farlo?

L'idea è molto semplice: se bisogna andare per livelli dobbiamo cercare di tenere traccia di quest'ordine tenendo da parte in memoria una lista di quei nodi che sono sullo stesso livello ma che ancora non hanno potuto dar vita alla propria elaborazione, alla propria stampa.

Vediamo ora gli algoritmi, in particolare non semplicemente la versione di stampa, ma la versione che crea il cosiddetto accumulatore, che ci dà un'informazione sintetica dell'albero; che può essere per esempio la somma dei valori contenuti nell'albero, se contiene numeri ovviamente; oppure una stringa ottenuta mediante concatenazione.

Ovviamente, se volessimo effettuare una ricerca mediante queste funzioni in un albero binario, la ricerca è una delle operazioni in cui l'ordine di visita non importa (stiamo assumendo alberi binari non ordinati).

Vediamo ora l'algoritmo di visita in ampiezza, anche detto di ricerca in ampiezza, o breadth-first search (BFS).

Tale algoritmo prende in input un albero binario su dei dati D , una funzione che legge i dati e prende un accumulatore che va ad aggiornare e restituisce, ed infine prende l'accumulatore con un certo valore iniziale nel quale inserire il valore in output.

Algoritmo 30: Visita in ampiezza in un albero binario

Signature BFS: $BT(D) \times (D \times A \rightarrow A) \times A \rightarrow A$

function BFS(T, F, a)

```

1  // se l'albero non è vuoto
  if (T  $\neq \perp$ ) then
    // costruisco una coda inserendo la radice dell'albero
2    Q  $\leftarrow$  singleton_queue(T)
    // fino a quando la coda non è vuota
3    repeat
      // prendo il valore in testa alla coda
      // e modifico la coda
4      (Q, T)  $\leftarrow$  head_&_dequeue(Q)
      // se il sottoalbero sinistro è non vuoto
5      if (T.l  $\neq \perp$ ) then
        // ... lo inserisco in coda
6        Q  $\leftarrow$  enqueue(Q, T.l)
      // se il sottoalbero destro è non vuoto
7      if (T.r  $\neq \perp$ ) then
        // ... lo inserisco in coda
8        Q  $\leftarrow$  enqueue(Q, T.r)
      // aggiorno l'accumulatore mediante la funzione in input
9      a  $\leftarrow$  F(T.dat, a)
10   until (is_empty(Q))
    // restituisco l'accumulatore
11  return a

```

`singleton_queue` è una funzione che chiama prima `empty_queue` e poi inserisce un elemento nella coda appena creata.

Esempio di Visita in Ampiezza

Di seguito è mostrato un esempio di esecuzione dell'algoritmo di visita in Ampiezza, basato sull'albero in figura 4.3. Supponiamo che la funzione F sia la funzione che concatena di valori dei nodi dell'albero.

Step	T.dat	Q	a
1	6	6	" "
2	6	3, 10	"6"
3	3	10, 0, 5	"63"
4	10	0, 5, 7, 11	"6310"
5	0	5, 7, 11	"63100"
6	5	7, 11	"631005"
7	7	11	"6310057"
8	11		"631005711"

Figura 4.4: Esempio di Visita in Ampiezza

4.3.8 Algoritmo: DFS in un Albero Binario

Riportiamo ora gli algoritmi della visita in profondità su alberi, che in un certo senso abbiamo già visto in una versione molto semplice, che era quella in cui andavamo semplicemente a riportare a video, a stampare a video, il valore contenuto nei nodi.

Abbiamo invece già cominciato con quello in ampiezza a fare una funzione di visita un po' più generale che permettesse in qualche modo di estrarre un'informazione cumulativa dall'albero.

Ecco il codice generale per le visite in profondità, in versione ricorsiva.

Come nel caso dell'algoritmo di BFS gli algoritmi di depth-first search, DFS, prendono in input il nodo dell'albero, una funzione F per aggiornare un accumulatore e restituirlo, e l'accumulatore.

Algoritmo 31: DFS preorder in un albero binario

Signature DFS_pre_order: $BT(D) \times (D \times A \rightarrow A) \times A \rightarrow A$

```

function DFS_pre_order(T, F, a)
    // se il sottoalbero è non vuoto
1   if (T != ⊥) then
        // aggiorni l'accumulatore mediante la funzione in input
2       a ← F(T.dat, a)
        // esploro il sottoalbero sinistro aggiornando l'accumulatore
3       a ← DFS_pre_order(T.l, F, a)
        // esploro il sottoalbero destro aggiornando l'accumulatore
4       a ← DFS_pre_order(T.r, F, a)
    // restituisco l'accumulatore
5   return a

```

Di seguito vediamo un esempio di esecuzione dell'algoritmo.

Stiamo supponendo che la funzione F sia una funzione che concatena i valori contenuti nei nodi.

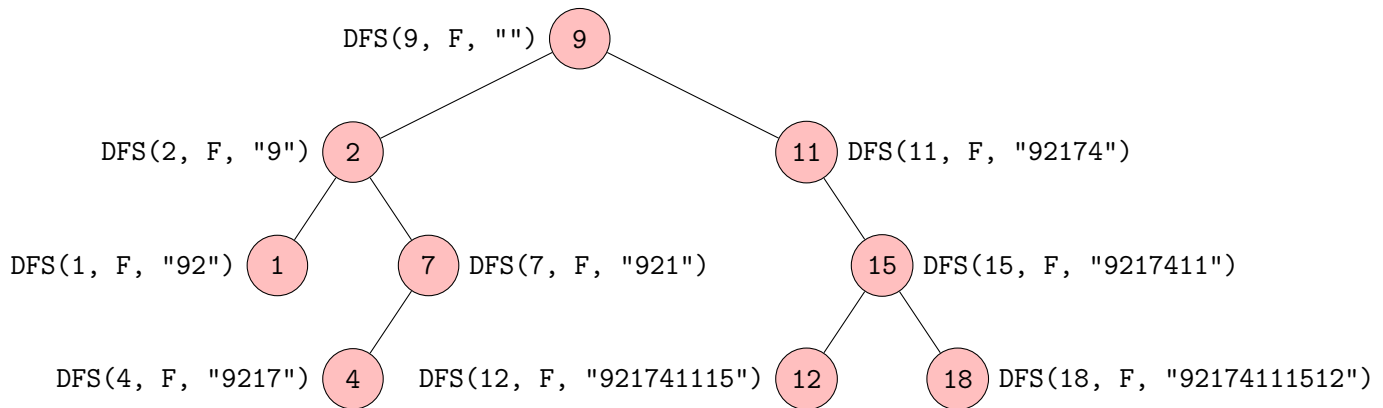


Figura 4.5: Esempio di esecuzione della DFS_pre_order

Algoritmo 32: DFS postorder in un albero binarioSignature DFS_post_order: $BT(D) \times (D \times A \rightarrow A) \times A \rightarrow A$

```

function DFS_post_order(T, F, a)
  // se il sottoalbero è non vuoto
1  if (T != ⊥) then
    // esploro il sottoalbero sinistro aggiornando l'accumulatore
2    a ← DFS_post_order(T.l, F, a)
    // esploro il sottoalbero destro aggiornando l'accumulatore
3    a ← DFS_post_order(T.r, F, a)
    // aggiorno l'accumulatore mediante la funzione in input
4    a ← F(T.dat, a)
    // restituisco l'accumulatore
5  return a

```

Algoritmo 33: DFS inorder in un albero binario

Signature DFS_in_order: $BT(D) \times (D \times A \rightarrow A) \times A \rightarrow A$ **function** DFS_in_order(T, F, a)

```
1  // se il sottoalbero è non vuoto
   if ( $T \neq \perp$ ) then
2      // esploro il sottoalbero sinistro aggiornando l'accumulatore
        $a \leftarrow \text{DFS\_in\_order}(T.l, F, a)$ 
3      // aggiorno l'accumulatore mediante la funzione in input
        $a \leftarrow F(T.dat, a)$ 
4      // esploro il sottoalbero destro aggiornando l'accumulatore
        $a \leftarrow \text{DFS\_in\_order}(T.r, F, a)$ 
   // restituisco l'accumulatore
5  return  $a$ 
```

Questo conclude le visite degli alberi, dove in generale però non è importante avere alcuna proprietà sull'ordinamento dei dati all'interno di un albero.

4.4 Alberi binari di ricerca - BST

Vediamo ora gli alberi binari di ricerca, i Binary Search Tree, detti anche BST.

Qual è la definizione di un BST?

Un BST è un albero binario tale che per ogni nodo nell'albero, se andiamo ad analizzare tutti i dati del sottoalbero sinistro, questi dovranno essere inferiori al dato presente nel nodo; dualmente se andiamo ad analizzare tutti i dati del sottoalbero destro, questi dovranno essere maggiori al dato presente nel nodo.

Formalmente.

Un albero è detto albero binario di ricerca se:

$$\forall x \in T (\forall y \in x.l (y.dat < x.dat) \wedge \forall z \in x.r (x.dat < z.dat)) \quad (4.2)$$

Facciamo un esempio di albero binario di ricerca e di un albero che non è un albero binario di ricerca.

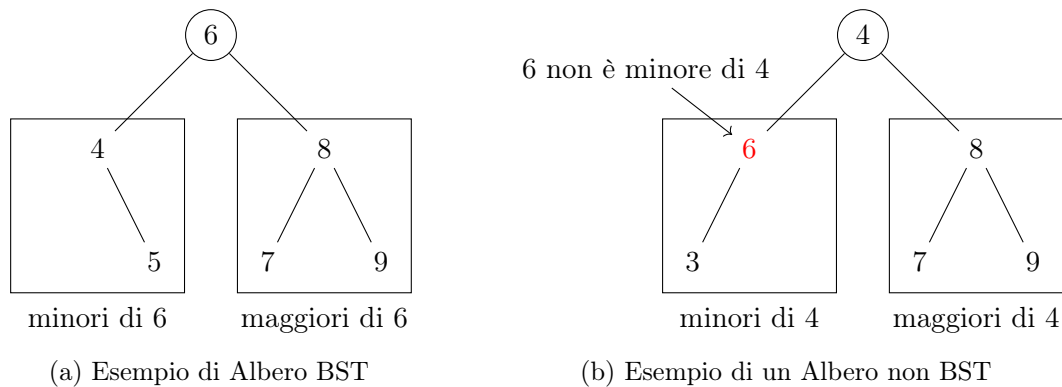


Figura 4.7: Esempio di BST e albero non di ricerca

Tutti i dati nell'albero sono necessariamente diversi, per due motivi in particolare.

1. Spesso gli alberi binari di ricerca sono pensati per modellare insiemi di dati, e come noto, gli insiemi ammettono l'occorrenza di un elemento una sola volta;
2. Nel caso fossimo davvero interessati alla molteplicità degli elementi, cioè a tener traccia di copie dei dati, non si va a cambiare la struttura dell'albero, si aggiunge solo un campo al nodo che va ad indicare la molteplicità del dato.

Tuttavia noi per semplicità non andremo a considerare nodi con ripetizione, quindi per noi i dati saranno sempre unici.

4.5 Algoritmi su BST

Vediamo quali sono gli algoritmi sugli alberi binari di ricerca.

4.5.1 Algoritmo: ricerca di un dato in un BST

Ovviamente la prima operazione che affronteremo, dato il nome d'altra parte, è effettuare una ricerca.

Se l'albero è un albero binario di ricerca, quello che ci interessa fare durante una ricerca è verificare che il dato da cercare sia nel nodo presente.

Nel caso non sia nel nodo presente, allora cosa accade? Che o è maggiore o è minore, perché l'ordinamento tra i dati è un ordinamento totale. E a seconda del fatto che sia maggiore o minore, andremo a destra o a manca.

Ecco l'algoritmo di ricerca di un dato in un BST.

Algoritmo 34: Ricerca di un dato in un BST

Signature search: $BST(D) \times D \rightarrow \mathbb{B}$

```

function search(T, d)
    // se il sottoalbero è non vuoto
1   if (T != ⊥) then
        // se il dato nel nodo corrente è maggiore del dato in input
2       if (T.dat > d) then
            // richiamo la funzione sul sottoalbero sinistro
3         return search(T.l, d)
        // se il dato nel nodo corrente è minore del dato in input
4       else if (T.dat < d) then
            // richiamo la funzione sul sottoalbero destro
5         return search(T.r, d)
6       else
            // se ho trovato il dato da cercare
7         return True
    // se il sottoalbero è vuoto e non ho trovato il dato da cercare
8   return False
  
```

Questo algoritmo fa esattamente quello che faceva la ricerca binaria nell'Array, con la differenza che invece di lavorare con gli indici, lavora sui nodi, ma è esattamente la stessa tipologia di algoritmo.

4.5.2 Algoritmo: ricerca ricorsiva del minimo in un BST

Quando parliamo degli Array, dicemmo che nel caso di Array ordinati, la ricerca del minimo è banale.

Si fa addirittura a tempo costante perché semplicemente se i dati sono ordinati in modo crescente, il minimo si troverà in indice zero, se i dati sono ordinati in modo decrescente, il minimo sarà nell'indice dimensione dell'Array meno uno.

In un albero non è così ovvio, perché nella struttura d'albero, che è una struttura puntata, cioè tramite puntatori, ovviamente non abbiamo indirizzamento diretto.

Ma comunque risulta estremamente semplice scrivere un algoritmo per la ricerca del minimo (e dualmente del massimo) in un BST.

Ecco la versione ricorsiva dell'algoritmo per la ricerca del minimo in un BST.

Supponiamo che l'albero binario di ricerca sia non vuoto.

Algoritmo 35: Ricerca ricorsiva del minimo in un BST non vuoto

Signature min: $BST(D) \setminus \{\perp\} \rightarrow BST(D) \setminus \{\perp\}$

function min(T)

```

1  // se non esiste il sottoalbero sinistro
   if (T.l =  $\perp$ ) then
2      // ... restituisco il puntatore al nodo
       return T
3  else
4      // richiamo la funzione sul sottoalbero sinistro
       return min(T.l)

```

Questa funzione ricorsiva (tail recursive) si implementa facilmente nella corrispondente versione iterativa.

4.5.3 Algoritmo: ricerca iterativa del minimo in un BST

Ecco la versione iterativa dell'algoritmo per la ricerca del minimo in un BST. Supponiamo che l'albero binario di ricerca sia non vuoto.

Algoritmo 36: Ricerca iterativa del minimo in un BST non vuoto

Signature min: $BST(D) \setminus \{\perp\} \rightarrow BST(D) \setminus \{\perp\}$

```
function min(T)
    // fino a quando esiste un sottoalbero sinistro
1   while (T.l !=  $\perp$ ) do
    // scendo nel sottoalbero sinistro
2   |   T  $\leftarrow$  T.l
    // restituisco il puntatore al nodo
3   |   return T
```

4.5.4 Algoritmo: ricerca del successore in un BST versione non tail recursive

Vediamo adesso la funzione di ricerca successore in un BST.

Vedremo due varianti, una che è molto simile alla versione che abbiamo dato per la ricerca del successore su un array ordinato, ha praticamente la stessa identica struttura.

Poi faremo vedere una versione che a differenza della precedente è tail recursive, ricorsiva in coda.

Ecco l'algoritmo per la prima variante della ricerca del successore in un BST.

Algoritmo 37: Ricerca del successore in un BST versione non tail recursive

Signature successor: $BST(D) \times D \rightarrow BST(D)$

```

function successor(T, d)
    // se il sottoalbero è non vuoto
1   if (T != ⊥) then
        // se il dato nel nodo corrente è maggiore del dato in input
2       if (T.dat > d) then
            // richiamo la funzione sul sottoalbero sinistro
3           S ← successor(T.l, d)
            // se non è stato trovato un successore nel sottoalbero sinistro
4           if (S = ⊥) then
                // ... il successore è il nodo corrente
5               S ← T
            // ... restituisco il successore
6       return S
7   else
        // se il dato nel nodo corrente non è maggiore del dato in input
        // richiamo la funzione sul sottoalbero destro
8       return successor(T.r, d)
    // se il sottoalbero è vuoto e non ho trovato un successore
9   return ⊥
  
```

Ora questa è la prima versione di **successor** che però non è tail recursive a livello sintattico perché c'è del lavoro che bisogna compiere a valle della chiamata ricorsiva.

Tuttavia c'è una versione per questa funzione specifica che è tail recursive, quindi dove le chiamate ricorsive sono sempre le ultime operazioni che andiamo ad eseguire.

4.5.5 Algoritmo: ricerca del successore in un BST versione tail recursive

Si tratta di una versione che ha una funzione iniziale di accesso di interfaccia che non è ricorsiva, poi una funzione ricorsiva con tre parametri: l'albero, il dato in input, e la stima del successore.

Quindi nella prima chiamata il terzo parametro sarà $\perp$.

Ecco la versione tail recursive della ricerca del successore in un BST.

Algoritmo 38: Ricerca del successore in un BST versione tail recursive funzione di interfaccia

Signature successor: $BST(D) \times D \rightarrow BST(D)$

```
function successor(T, d)
  // chiamo la funzione ricorsiva
1  return successor(T, d,  $\perp$ )
```

Algoritmo 39: Ricerca del successore in un BST versione tail recursive funzione accessoria

Signature successor: $BST(D) \times D \times BST(D) \rightarrow BST(D)$

```
function successor(T, d, S)
  // se il sottoalbero è non vuoto
1  if (T !=  $\perp$ ) then
    // se il dato nel nodo corrente è maggiore del dato in input
    // ho trovato un candidato
2    if (T.dat > d) then
      // richiamo la funzione sul sottoalbero sinistro
      // con la stima del successore aggiornata
3      return successor(T.l, d, T)
4    else
      // se il dato nel nodo corrente non è maggiore del dato in input
      // richiamo la funzione sul sotto-albero destro
      // con la stima del successore corrente
5      return successor(T.r, d, S)
  // restituisco il successore
6  return S
```

Questi sono gli algoritmi che analizzano l'albero senza modificarlo.

Tutti questi algoritmi di ricerca ovviamente hanno una complessità nel caso peggiore pari all'altezza dell'albero,

cioè qual è il percorso più lungo al più presente nell'albero partendo dalla radice.

Facciamo un esempio di esecuzione dell'algoritmo.

Supponiamo che si voglia cercare il successore di 11.

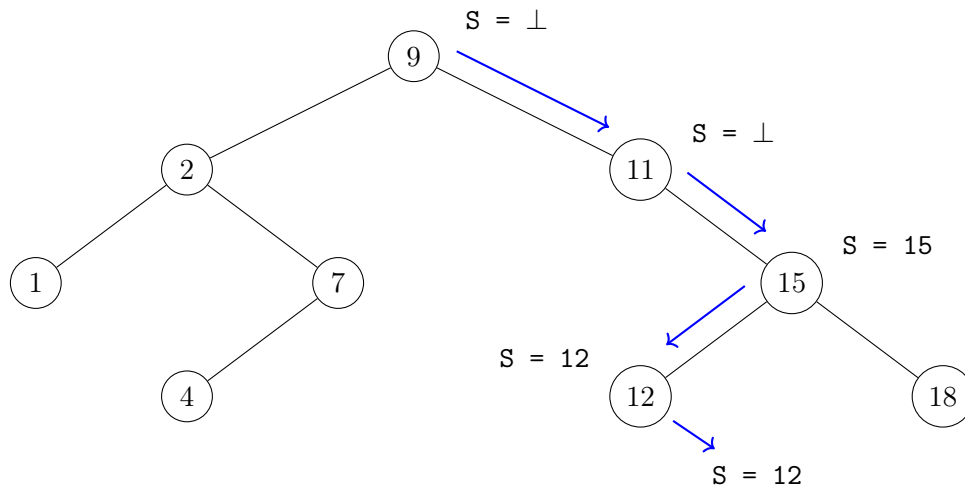


Figura 4.8: Esempio di ricerca del successore in un BST

4.5.6 Algoritmo: inserimento in un BST

A questo punto vediamo come costruire alberi binari di ricerca, in particolare come effettuare l'inserimento e poi la cancellazione in BST.

Analizziamo prima l'inserimento di un dato in un BST.

L'idea intuitiva è di cercare nell'albero quale possa essere la posizione dove ipoteticamente può essere presente il dato e nel caso in cui arriviamo ad un punto in cui il dato non lo troviamo, la posizione ultima in cui ci troviamo nell'albero è praticamente la posizione dove quel dato potrebbe stare senza violare la proprietà dell'albero binario di ricerca.

Facciamo un esempio di inserimento in un BST.

Supponiamo di voler inserire nel seguente BST il nodo con dato 10.

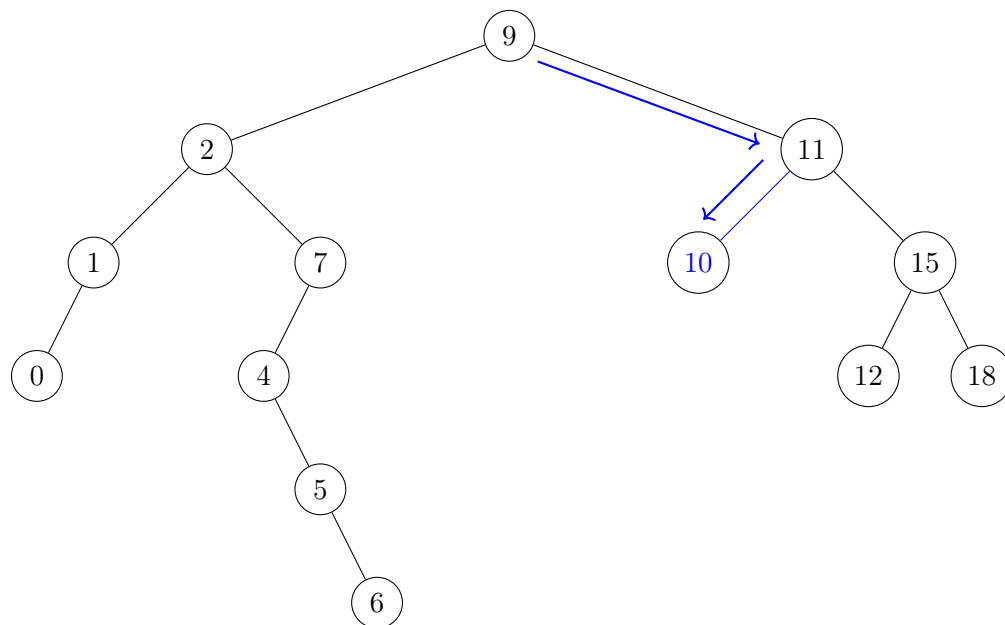


Figura 4.9: Inserimento in un BST

Supponiamo adesso di voler inserire sempre nello stesso albero un nodo con dato 11.

Ricordiamo che ciò non è possibile perché nel BST non ci possono essere dati duplicati, essendo un nodo con dato 11 già presente al suo interno.

L'algoritmo che ora vedremo è un banale riadattamento della funzione di ricerca dove quando troviamo il caso base, cioè quando l'albero è vuoto, invece di non far nulla creiamo un nuovo nodo.

L'algoritmo è il seguente.

Algoritmo 40: Inserimento di un dato in un BST

Signature insert: $BST(D) \times D \rightarrow BST(D)$

```

function insert(T, d)
    // se il sottoalbero è vuoto
1   if (T =  $\perp$ ) then
        // costruisco un nuovo nodo con il dato in input
        // e lo inserisco nella posizione corrente
2       T  $\leftarrow$  build_node(D)
3   else
        // se il sottoalbero non è vuoto
        // se il dato nel nodo corrente è maggiore del dato in input
4       if (T.dat > d) then
            // richiamo la funzione sul sottoalbero sinistro
            // nel caso modificandone la radice
5           T.l  $\leftarrow$  insert(T.l, d)
6       else if (T.dat < d) then
            // richiamo la funzione sul sottoalbero destro
            // nel caso modificandone la radice
7           T.r  $\leftarrow$  insert(T.r, d)
        // restituisco il nodo corrente
8   return T

```

La funzione `build_node` costruisce un nodo contenente il dato d con i puntatori destro e sinistro a $\perp$, e ne restituisce l'indirizzo.

La funzione `insert` restituirà il puntatore alla radice del nuovo albero in cui abbiamo inserito il dato; attenzione, il dato potrebbe anche non essere stato inserito davvero perché già era presente.

4.5.7 Algoritmo: cancellazione in un BST

Soffermiamoci ora sulla cancellazione di un nodo in un BST.

La cancellazione non è così ovvia come l'inserimento, perché non avviene ovviamente sempre in fondo, cioè in foglia, in quanto il dato che vogliamo cancellare potrebbe stare ovunque. E, a seconda di dove troviamo quel dato, in particolare a seconda della struttura dell'albero, non è sempre immediato capire come fare a cancellare.

Quello che si fa è un'analisi per casi. Come prima cosa ovviamente si cerca il dato da cancellare o meglio dove è presente il dato, sempre che questo ci sia, perché se il dato non c'è semplicemente nulla deve essere fatto.

Una volta identificato il nodo contenente il dato da cancellare si analizzano le possibilità: ha al più un figlio o ha entrambi i figli?

- Se ha un solo figlio, ovviamente si collega il figlio del nodo da cancellare al padre del nodo da cancellare (Figura 4.10 e Figura 4.11);
- Se invece ci dovessimo trovare nella situazione in cui vogliamo cancellare il dato contenuto in un nodo che ha entrambi i figli (Figura 4.12 e Figura 4.15), allora non andiamo a cancellare il nodo che contiene il dato perché altrimenti la struttura verrebbe trasformata in una forma che poi non sapremmo trattare; quello che faremo è cancellare solo il dato e rimpiazzare nel nodo un altro dato che sappiamo che può essere messo in quella posizione.

Il dato che può essere inserito nel nodo in questione deve essere ancora sempre maggiore di tutto ciò che sta a sinistra e inferiore di tutto ciò che sta a destra; altrimenti andremmo a violare la proprietà invariante del BST.

Un dato che è sempre maggiore di tutto ciò che sta a sinistra e minore di tutto ciò che sta a destra effettivamente c'è ed è o il massimo del sottoalbero sinistro o il minimo del sottoalbero destro.

Perché?

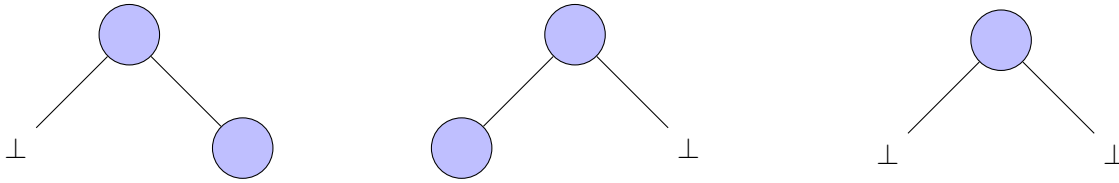
Ad esempio prendiamo il caso del minimo del sottoalbero destro, essendo il minimo del sottoalbero destro, lui è più piccolo di tutti gli altri, e quindi non viola alcuna proprietà rispetto ai nodi alla sua destra.

Inoltre essendo lui un nodo che stava alla destra di quello che abbiamo cancellato di sicuro sarà maggiore di tutto ciò che stava a sinistra di quello che abbiamo cancellato e di conseguenza non viola la proprietà del BST.

Dualmente potremmo scegliere invece il massimo del sottoalbero sinistro.

In particolare i nodi corrispondenti al minimo del sottoalbero a destra e al massimo del sottoalbero a sinistra essendo minimi e massimi hanno per forza al più un figlio, quindi la loro cancellazione è facile, non richiede un caso ricorsivo perché è uno dei casi che sappiamo trattare facilmente semplicemente connettendo il figlio al nonno; perché appunto il minimo non può avere figli a sinistra altrimenti non sarebbe minimo, il massimo non può avere figli a destra perché altrimenti non sarebbe il massimo, di conseguenza o il sinistro del minimo o il destro del massimo sono per forza NULL.

Dobbiamo trovare un nodo che ha al più un figlio:



Il nodo che ha al più un figlio è

- il minimo nel caso del sottoalbero sinistro
- il massimo nel caso del sottoalbero destro

Facciamo un esempio di cancellazione in BST di un nodo con un solo figlio.

Supponiamo di voler cancellare dal seguente BST il nodo con dato 7.

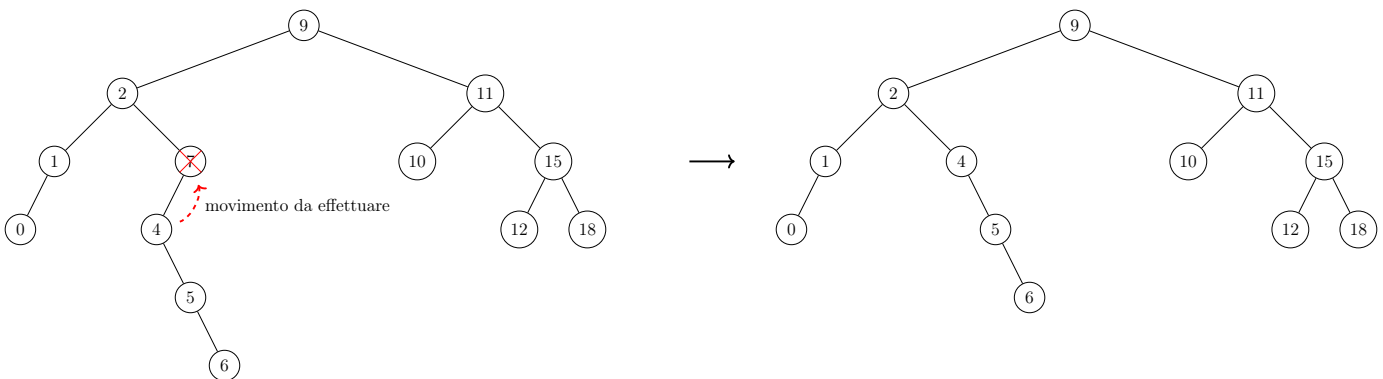


Figura 4.10: Esempio di cancellazione in BST: nodo con un solo figlio

Supponiamo di voler cancellare dal seguente BST il nodo con dato 1.

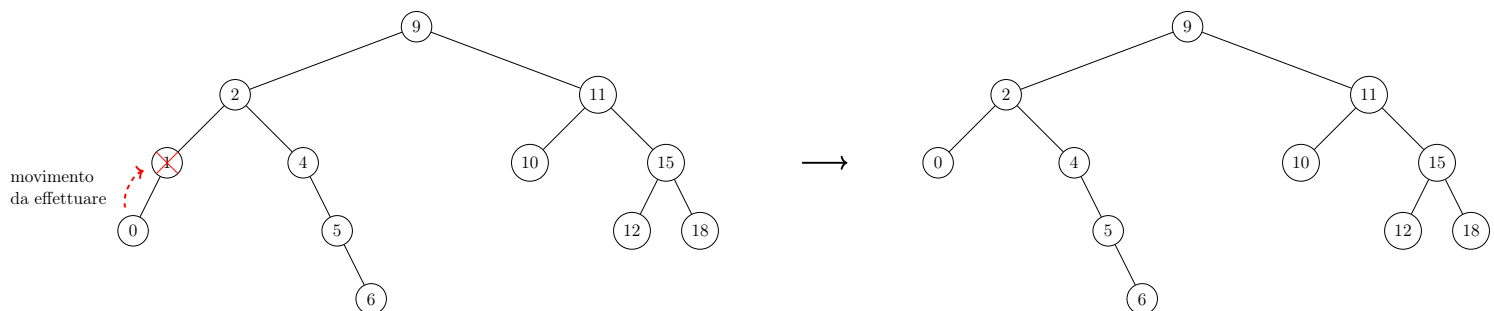


Figura 4.11: Esempio di cancellazione in BST: nodo con un solo figlio

Facciamo un esempio di cancellazione in BST di un nodo con entrambi i figli.

Supponiamo di voler cancellare dal seguente BST il nodo con dato 2.

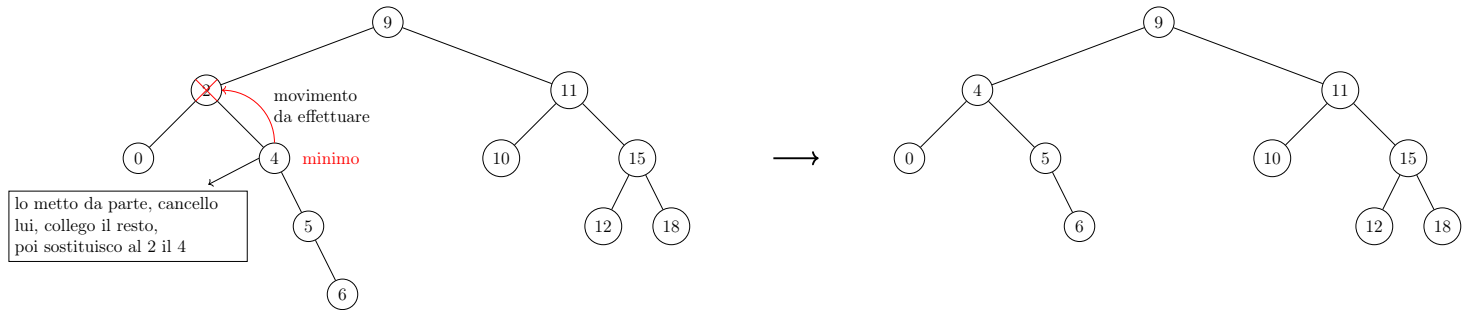


Figura 4.12: Esempio di cancellazione in BST: nodo con due figli

Passiamo all'algoritmo.

L'algoritmo richiede, a differenza dell'inserimento, un insieme di sotto procedure.

Cominciamo per prima cosa a formalizzare l'algoritmo di cancellazione esterno.

Algoritmo 41: Cancellazione in un BST: funzione `delete`

Signature delete: $BST(D) \times D \rightarrow Ref(D)$

```

function delete(T, d)
    // se il sottoalbero non è vuoto
1   if (T != ⊥) then
        // se il dato nel nodo corrente è maggiore del dato in input
2       if (T.dat > d) then
            // richiamo la funzione sul sottoalbero sinistro
            // nel caso modificandone la radice
3           T.l ← delete(T.l, d)
        // se il dato nel nodo corrente è minore del dato in input
4       else if (T.dat < d) then
            // richiamo la funzione sul sottoalbero destro
            // nel caso modificandone la radice
5           T.r ← delete(T.r, d)
6       else
            // se ho trovato il dato da cancellare
            // elimino il dato chiamando una funzione accessoria
7           T ← delete_data(T)
    // restituisco il nodo corrente
8   return T

```

Come nel caso dell'inserimento la funzione `delete` restituirà il puntatore alla radice del nuovo albero in cui abbiamo eliminato il dato.

Vediamo ora la funzione `delete_data`.

Assumeremo che questa funzione venga chiamata su un nodo che non è `NULL`. Che è chiaramente verificato.

Non verrà mai chiamata esternamente, non è una di quelle funzioni che vengono rese pubbliche all'utente in quanto non è una funzione `safe` visto che se venisse chiamata su `NULL` andrebbe in eccezione.

Algoritmo 42: Cancellazione in un BST: funzione `delete_data`

Signature `delete_data`: $BST(D) \rightarrow Ref(D)$

```

function delete_data(T)
    // se il sottoalbero sinistro è vuoto
1   if (T.l =  $\perp$ ) then
        // elimino il nodo ed al suo posto inserisco il figlio destro
        // mediante una funzione accessoria
2       T  $\leftarrow$  skip_right(T)
    // se il sottoalbero destro è vuoto
3   else if (T.r =  $\perp$ ) then
        // elimino il nodo ed al suo posto inserisco il figlio sinistro
        // mediante una funzione accessoria
4       T  $\leftarrow$  skip_left(T)
5   else
        // se il nodo ha entrambi i figli
        // elimino il minimo del sottoalbero destro
        // sostituendo al dato corrente, da eliminare,
        // il dato del minimo del sottoalbero destro
6       T.dat  $\leftarrow$  get_&_delete_min(T.r, T)
7   return T

```

Vediamo ora l'algoritmo della funzione `skip_right`.

Algoritmo 43: Cancellazione in un BST: funzione `skip_right`**Signature** `skip_right`: $BST(D) \rightarrow Ref(D)$ **function** `skip_right`(T)

```

    // salvo la radice del sottoalbero destro
1  Y ← T.r
    // elimino il nodo in input
2  deallocate_node(T)
    // restituisco la radice salvata
3  return Y

```

Facciamo un esempio del comportamento della funzione `skip_right`.

Di seguito mostriamo un esempio di sottoalbero a cui è applicata la `skip_right`.



Figura 4.13: Esempi di cancellazione in BST applicando la funzione `skip_right`

Vediamo ora l'algoritmo della funzione `skip_left`.

Algoritmo 44: Cancellazione in un BST: funzione `skip_left`**Signature** `skip_left`: $BST(D) \rightarrow Ref(D)$ **function** `skip_left`(T)

```

    // salvo la radice del sottoalbero sinistro
1   Y ← T.l
    // elimino il nodo in input
2   deallocate_node(T)
    // restituisco la radice salvata
3   return Y

```

Facciamo un esempio del comportamento della funzione `skip_left`. Di seguito mostriamo un esempio di sottoalbero a cui è applicata la `skip_left`.

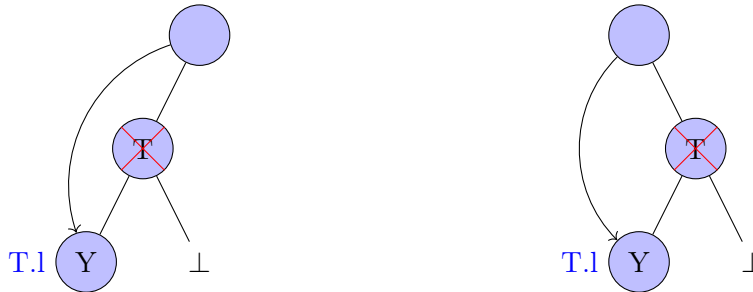


Figura 4.14: Esempi di cancellazione in BST: applicando la funzione `skip_left`

Passiamo adesso alla funzione `get_&_delete_min`.

Il primo parametro è la radice dell'albero da cui bisogna effettuare la cancellazione del minimo, l'altro parametro è il padre di questo nodo.

Dobbiamo quindi andare a cancellare il minimo ma allo stesso tempo dobbiamo restituire un valore, tuttavia allo stesso tempo dobbiamo ricollegare i nodi ai genitori, visto che c'è stata una cancellazione.

Quindi ci sono due approcci: o ammettiamo di restituire due valori di ritorno, la radice del nuovo albero e il dato; oppure restituire uno dei due valori, in particolare il dato, ed effettuare il collegamento tra nodi all'interno della funzione, delegando quindi alla funzione che cancella anche tale compito.

Per poter utilizzare il secondo approccio è necessario però passare come parametro anche il nodo padre.

Vedremo il secondo approccio in quanto più semplice da implementare nei comuni linguaggi di programmazione.

L'algoritmo è il seguente.

Algoritmo 45: Cancellazione in un BST: funzione `get_&_delete_min`Signature `get_&_delete_min`: $BST(D) \times BST(D) \rightarrow D$

```

function get_&_delete_min(T, P)
    // se il sottoalbero sinistro è vuoto
    // e quindi ho raggiunto il minimo
1   if (T.l = ⊥) then
        // salvo il dato contenuto nel nodo
2       d ← T.dat
        // elimino il nodo
        // mediante una funzione accessoria
        // e salvo il puntatore
3       Y ← skip_right(T)
        // ricollego in maniera appropriata
        // mediante una funzione accessoria
4       swap_child(P, T, Y)
5   else
        // se il sottoalbero sinistro non è vuoto
        // richiamo la funzione sul sottoalbero sinistro
        // e salvo il minimo trovato
6       d ← get_&_delete_min(T.l, T)
        // restituisco il valore del minimo
7   return d

```

Vediamo ora la funzione `swap_child`. Questa funzione prende in input il nodo padre, a cui verrà sostituito uno dei due campi (figlio destro o sinistro). Per capire se vada sostituito il figlio destro o sinistro prende in input anche il nodo da sostituire e chiaramente il nodo col quale il nodo da sostituire va rimpiazzato.

Algoritmo 46: Cancellazione in un BST: funzione `swap_child`**Signature** `swap_child`: $BST(D) \times BST(D) \times BST(D) \rightarrow \emptyset$ **function** `swap_child`(P, T, Y)

```

    // se il nodo da sostituire è il figlio sinistro
1  if (P.l = T) then
    | // inserisco il nodo da posizionare come figlio sinistro
2  | P.l  $\leftarrow$  Y
3  else
    | // se il nodo da sostituire è il figlio destro
    | // inserisco il nodo da posizionare come figlio destro
4  | P.r  $\leftarrow$  Y

```

Sottolineiamo che tale funzione valuta l'indirizzo di memoria del nodo sebbene esso sia stato deallocato.

La funzione `get_&_delete_min` è tail recursive. Non c'è nessun lavoro vero che viene effettuato dopo la chiamata ricorsiva.

A livello sintattico si dice tail recursive, una funzione che, dopo l'ultima chiamata ricorsiva al più effettua delle operazioni sui dati che vengono prodotti dalla chiamata ricorsiva.

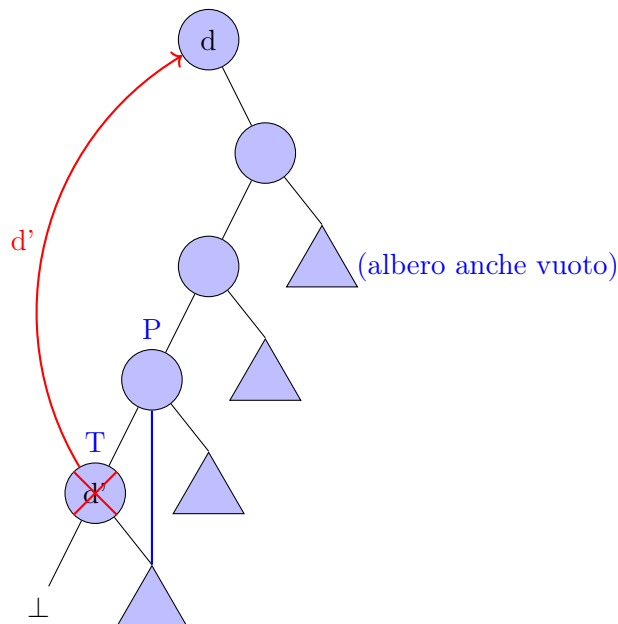


Figura 4.15: Esempio di cancellazione in BST: nodo con due figli

4.6 Bilanciamento negli alberi binari

A questo punto cerchiamo di capire come risolvere il problema dello sbilanciamento.

Se volessimo creare un albero partendo dall'albero vuoto solo per inserimento con le funzioni che abbiamo visto con dei dati che sono in un ordine (crescente o decrescente), quello che andremo ad ottenere è sicuramente un albero, ma un albero degenero. Che non permette una ricerca efficiente.

Significa che gli algoritmi che abbiamo visto non permettono di avere alberi ben strutturati, o bilanciati.

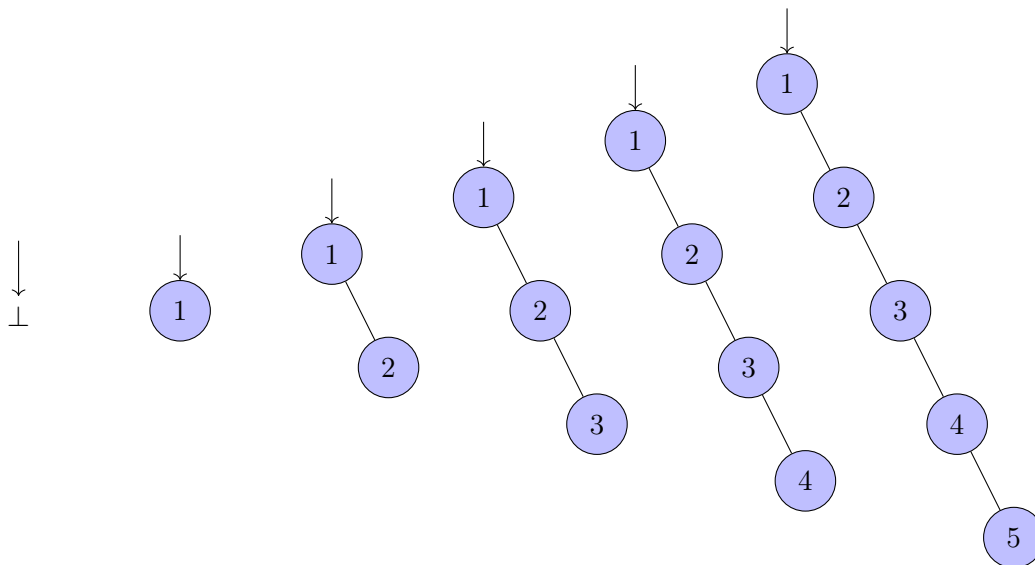


Figura 4.16: Esempio di costruzione di un albero degenero usando l'inserimento per i BST

Come fare ad ovviare a questo problema?

Dobbiamo, in qualche modo, definire per prima cosa la classe di alberi che ci interessa, cioè la classe degli alberi su cui un algoritmo funziona bene.

L'albero migliore dovrebbe essere quello che a parità di dati, a parità di numeri di nodi, è il più basso possibile.

Significa che i nodi devono essere distribuiti in un certo senso in ampiezza; l'albero migliore da questo punto di vista è l'albero pieno.

4.7 Alberi pieni

L'albero pieno è l'albero che ha il massimo numero di nodi possibili per una certa altezza.

Andiamo a considerare il numero di nodi di un albero pieno in base all'altezza.

Per convenzione l'altezza dell'albero vuoto viene definita a -1 ; l'altezza del singolo dato a 0 .

L'albero pieno di altezza -1 (albero vuoto) contiene 0 nodi.

L'albero pieno di altezza 0 contiene 1 nodo.

L'albero pieno di altezza 1 contiene 3 nodi.

L'albero pieno di altezza 2 contiene 7 nodi.

Riassumiamo queste informazioni nella seguente tabella.

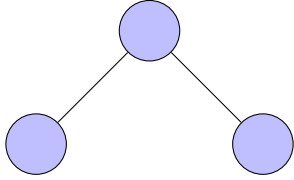
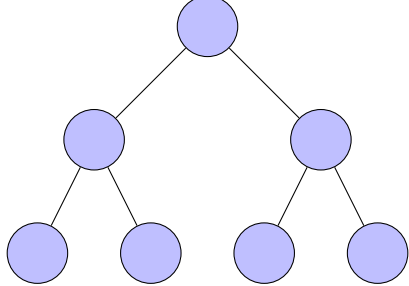
Altezza	Numero di nodi	Rappresentazione
-1	0	$\emptyset$
0	1	
1	3	
2	7	

Tabella 4.1: Alberi pieni al variare dell'altezza

È ovvio quindi che il numero di nodi in un albero pieno è proporzionale all'altezza secondo la formula $2^{h+1} - 1 = n$, dove h è l'altezza dell'albero e n è il numero dei nodi.

Questi sono gli alberi più bassi possibili che hanno il massimo numero di nodi. Se questi fossero tutti e soli gli alberi di cui potremmo avere una rappresentazione concreta avremmo un problema a livello di quantità di dati perché il numero di nodi non copre tutto l'intervallo di numeri naturali; non sono pertanto sufficientemente flessibili.

4.8 Alberi perfettamente bilanciati

Una variante degli alberi pieni, che appunto non saranno pieni ma hanno le stesse proprietà di essere i più bassi possibili, sono gli alberi detti perfettamente bilanciati.

Un albero è perfettamente bilanciato se praticamente rappresenta a livello strutturale di albero un array nel modo classico ottenuto dalla bisezione.

Se prendiamo un array di dimensioni dispari avremo il mediano e un vettore a destra e un vettore a sinistra esattamente uguali; se l'array invece è di dimensione pari a parte l'elemento centrale, uno dei due vettori laterali è di dimensione differente dall'altro di un'unità in meno dell'altro.

Questa idea la si può applicare sugli alberi dicendo che un albero è perfettamente bilanciato se, per ogni nodo, quando andiamo a prendere la cardinalità, cioè quanti nodi abbiamo, nel sottoalbero a sinistra e la cardinalità nel sottoalbero a destra, al più lo scarto è di 1; che è esattamente quello che succede con i vettori quando andiamo a ripartirli con l'algoritmo di bisezione.

Inoltre visto che un vettore può contenere qualunque insieme di dati, qualunque insieme di dati può essere rappresentato in un albero perfettamente bilanciato.

$$\text{Un albero } T \text{ è perfettamente bilanciato} \iff \forall x \in T (||x.l| - |x.r|| \leq 1) \quad (4.3)$$

Dove per $|x.l|$ o $|x.r|$ si intende rispettivamente la cardinalità del sottoalbero sinistro e destro di x .
Mentre per $||x.l| - |x.r||$ si intende il valore assoluto della sottrazione tra le due cardinalità.

Attenzione che l'essere perfettamente bilanciato è indipendente come proprietà rispetto all'essere albero binario di ricerca; è una proprietà puramente strutturale, non fa riferimento ai dati.

Facciamo un esempio mostrando un albero perfettamente bilanciato e un albero non perfettamente bilanciato.

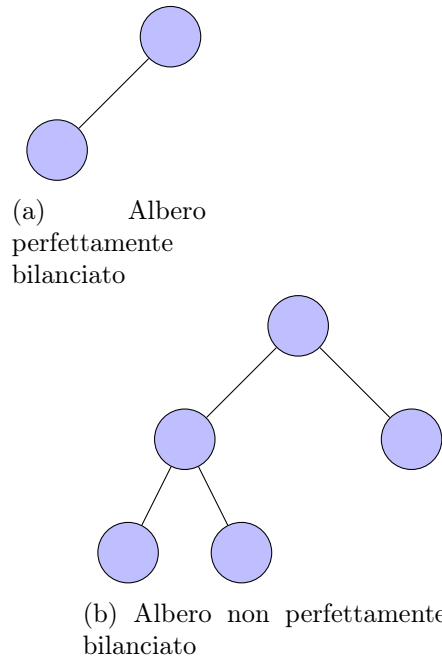


Figura 4.18: Esempio di albero perfettamente bilanciato e non perfettamente bilanciato

4.8.1 Proprietà alberi perfettamente bilanciati

Vedremo ora che negli alberi perfettamente bilanciati, l'altezza è logaritmica nel numero di nodi.

Nell'albero pieno è ovvio.

Teorema 4.8.1 (Altezza albero pieno).

Sia h l'altezza di un albero pieno ed n è il numero dei suoi nodi.

Dimostrazione.

Sappiamo che se T è un albero pieno allora $n = 2^{h+1} - 1$, dove n è il numero di nodi dell'albero e h è l'altezza dell'albero.

Segue, chiaramente

$$\begin{aligned} 2^{h+1} - 1 = n &\iff 2^{h+1} = n + 1 \iff \log_2(2^{h+1}) = \log_2(n + 1) \iff \\ h + 1 = \log_2(n + 1) &\iff h = \log_2(n + 1) - 1 \end{aligned} \quad (4.4)$$

□

Dimostriamo ora che passando dalla classe degli alberi pieni, a quella degli alberi perfettamente bilanciati non si perda tale proprietà.

Teorema 4.8.2 (Altezza albero perfettamente bilanciato).

Sia h l'altezza di un albero perfettamente bilanciato, n è il numero dei suoi nodi e $MaxN$ il numero massimo di nodi abbiamo che l'altezza è logaritmica nel numero di nodi.

Dimostrazione.

Dimostriamo il teorema per induzione su h .

Il caso base, $h = 0$, risulta banalmente vero.

Passimo al caso induttivo, considerando il teorema vero per un generico ma fissato h , e consideriamo il caso $h + 1$.

Data l'ipotesi e la definizione di albero perfettamente bilanciato, segue

$$MaxN(h + 1) \leq 1 + MaxN(h) + MaxN(h) \iff MaxN(h + 1) = 1 + 2 \cdot MaxN(h) \quad (4.5)$$

Dall'ipotesi induttiva segue la tesi.

□

Purtroppo anche gli alberi binari di ricerca perfettamente bilanciati per quanto siano i migliori alberi possibili da utilizzare a livello di ricerca, visto che sono sufficientemente flessibili da permettere di rappresentare un insieme di arbitraria numerosità e sono i più bassi possibili avendo l'altezza logaritmica nel numero di nodi, hanno un problema: sono ancora troppo rigidi nel caso di modifiche a seguito di inserimenti o cancellazioni.

A seguito di inserimenti può essere necessario trasformare l'albero nuovamente in un albero perfettamente bilanciato mediante delle rotazioni.

Può accadere su un albero profondo qualsivoglia che all'improvviso per poterlo bilanciare ci si trovi costretti a prendere una foglia e a portarla in radice e a spostare una marea di altri dati perché non è detto che sia solo questo il problema, il problema lo potremmo trovare allo stesso tempo in tanti punti diversi.

In generale, esistono in letteratura algoritmi che prendono un albero perfettamente bilanciato e dopo un inserimento che va a distruggere la proprietà di essere perfettamente bilanciato la ristabiliscono; peccato che per far questo è necessario, sebbene non in tutti i casi, dover fare un lavoro lineare sull'intera struttura dell'albero. Si perde quindi tutto il vantaggio di una struttura in cui il tempo di accesso, l'inserimento e la cancellazione debba essere basso, debba essere al più lineare nella lunghezza di un percorso; invece con questo diventerebbe lineare nella dimensione dell'albero.

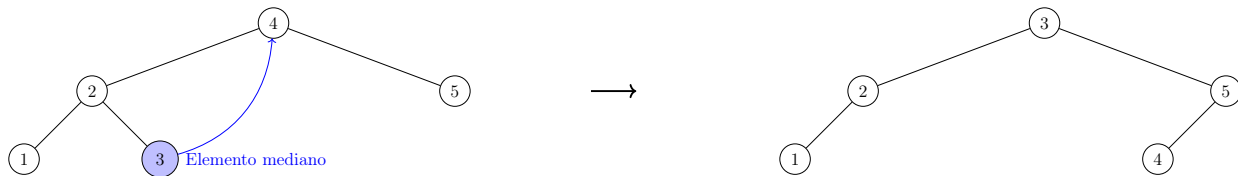


Figura 4.19: Bilanciamento di alberi perfettamente bilanciati

Una struttura del genere può essere utilizzata quando gli inserimenti e le cancellazioni sono rare, mentre tutte le operazioni di lettura sono molto frequenti; ma se la struttura è particolarmente dinamica, cioè è molto frequente che avvengano inserimenti e cancellazioni, allora non è questa la famiglia di alberi che bisogna adottare perché è ancora troppo rigida.

4.9 Alberi bilanciati

Ma qual è veramente la proprietà fondamentale della famiglia degli alberi perfettamente bilanciati che a noi interessa? È che sia davvero così ben bilanciata, cioè che da una parte abbiamo un certo numero di nodi e dall'altra lo stesso numero al più con uno scarto di 1?

In verità no.

Quello che a noi interessa di questa famiglia è che l'altezza dell'albero sia logaritmica nel numero di nodi. Se dovessimo anche distanziarci da questa famiglia pur preservando la proprietà di avere l'altezza dell'albero logaritmica nel numero di nodi a livello asintotico non cambierebbe nulla.

Quindi nonostante andiamo a pagare un incremento dell'altezza, che ovviamente costa un po' di più nelle letture, anche se asintoticamente non cambia nulla, ci conviene perché poi gli inserimenti li possiamo fare allo stesso tempo.

Quello che faremo è identificare una nuova famiglia di alberi che continui ad avere l'altezza logaritmica e allo stesso tempo abbia una struttura sufficientemente flessibile da far sì che nel momento in cui andiamo a danneggiarla la si possa correggere in un modo molto più locale, facendo pochi aggiustamenti, possibilmente nelle vicinanze strette del punto dove si è creato il problema e non modifiche che richiedono di andare dalla radice alla foglia per aggiustare la situazione.

Quindi la classe di alberi che ci interessa (di cui ovviamente la classe degli alberi perfettamente bilanciati fa parte e quindi anche gli alberi pieni che appartengono a quest'ultima) viene detta la classe degli alberi bilanciati.

L'altezza di un albero è la lunghezza del percorso più lungo massimale dalla radice.

La classe di alberi bilanciati chiede qualcosa di molto più semplice che è la seguente cosa: l'altezza dell'albero deve essere $\Theta(\log(n))$, dove n è il numero di nodi.

Avremmo anche potuto dire $O(\log(n))$, perché a parità di nodi un albero più basso di $\lceil \log_2(n) \rceil$ non può esistere. Quindi sappiamo sicuramente che l'altezza è $\Omega(\log(n))$ e quindi per definizione anche $\Theta(\log(n))$.

Quando l'albero è bilanciato tutte le funzioni di ricerca che abbiamo visto, e potenzialmente altre che non abbiamo avuto il tempo di discutere ma che esistono in letteratura, che sono lineari nell'altezza dell'albero (cioè il numero di operazioni è lineare nella massima lunghezza di un percorso dell'albero) sono automaticamente logaritmiche nel numero di nodi.

Un singolo inserimento e/o una singola cancellazione sono a loro volta logaritmiche.

Il problema è che in generale le cancellazioni e gli inserimenti possono portare un albero fuori da una specifica classe, di conseguenza poi le funzioni si comporteranno potenzialmente in modo diverso.

Dobbiamo quindi identificare una classe di alberi in cui le funzioni di inserimento e cancellazione, a valle di alcune correzioni che andremo a fare, rimangano chiuse.

Vogliamo identificare una classe che sia bilanciata e allo stesso tempo in cui si abbia un modo di modificare le funzioni di inserimento e cancellazione in modo tale che ogni volta che applichiamo una di queste dalla classe di partenza si rimanga nella stessa classe.

Vedremo due classi di alberi con tali proprietà.

La prima classe sono gli alberi AVL. A, V ed L sono le iniziali degli autori che lo definirono per primi negli anni '60.

Successivamente vedremo gli alberi red black, RB.

4.10 Alberi AVL

Gli alberi AVL hanno come definizione qualcosa di molto simile a quella di perfettamente bilanciato con una differenza.

La definizione degli alberi perfettamente bilanciati diceva che per ogni nodo dell'albero se prendiamo la cardinalità del sottoalbero a sinistra e del sottoalbero a destra le due differiscono al più di 1; se invece di andare a guardare il numero di nodi andassimo a considerare l'altezza dei sottoalberi otterremmo la classe degli alberi AVL.

Formalmente.

$$T \text{ è un albero AVL } \iff \forall x \in T (|h(x.l) - h(x.r)| \leq 1) \quad (4.6)$$

In effetti una delle correlazioni è vera.

$$T \text{ è perfettamente bilanciato } \implies T \text{ è AVL} \quad (4.7)$$

Un albero perfettamente bilanciato è AVL perché, avendo solo lo scarto di uno tra il numero di nodi dei sottoalberi, quello che può accadere al più è che un sottoalbero abbia un nodo in più che sarà una foglia e l'altro manca proprio di tale nodo.

Non vale il viceversa perché i due sottoalberi pur avendo la stessa altezza possono avere numeri di nodi particolarmente diversi con scarto maggiore di 1.

Ovviamente gli alberi AVL essendo una classe più ampia degli alberi perfettamente bilanciati possono rappresentare insiemi arbitrari di dati.

Non è però evidente immediatamente come dalla definizione di albero AVL si possa concludere che l'altezza di un albero AVL è $\Theta(\log(n))$, con n numero di nodi.

Per capire come fare cominciamo prima ad analizzare una casistica, vediamo un po' di costruire alcuni alberi.

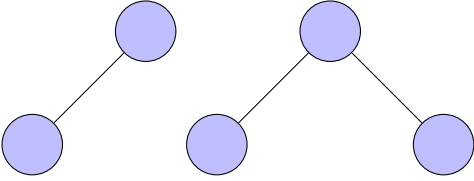
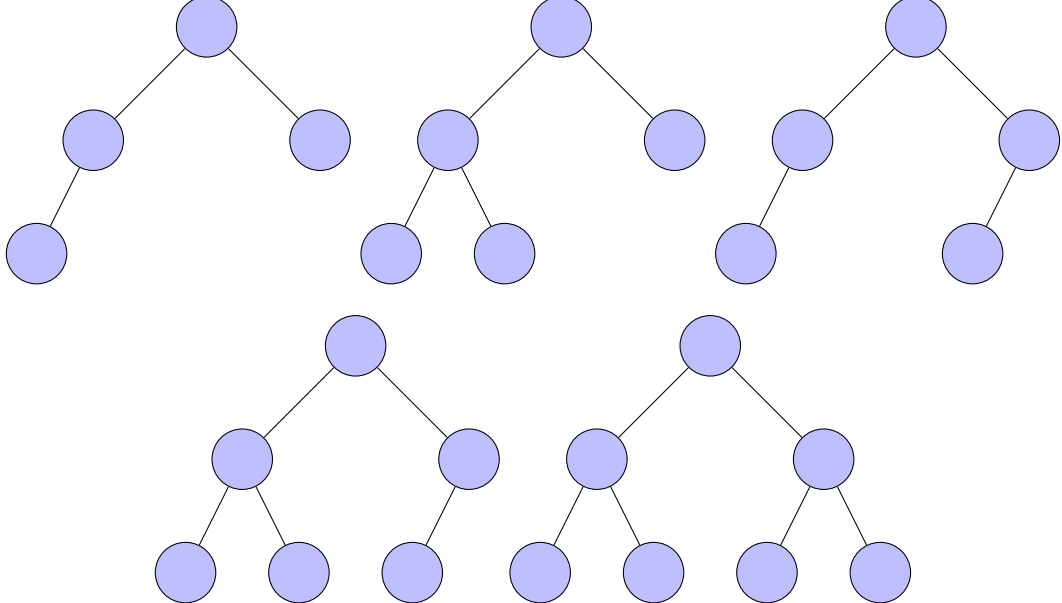
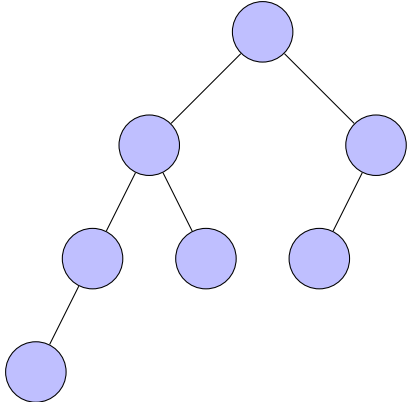
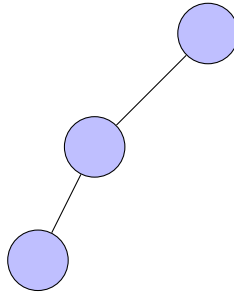
Altezza	Rappresentazione
-1	$\emptyset$
0	
1	
2	
3	

Tabella 4.2: Esempi di alberi AVL

Facciamo un esempio di un Albero non AVL.



Non è un albero AVL

Come facciamo a dimostrare in generale che qualunque albero andiamo a prendere che soddisfa la proprietà degli alberi AVL effettivamente non superi un andamento logaritmico?

Allora l'idea è di prendere, tra virgolette, gli alberi AVL peggiori, cioè quelli che sono più vicini al limite della violazione della proprietà.

4.10.1 Alberi AVL minimi - MAVL

La definizione di albero minimo AVL, che sono appunto quelli più sbilanciati possibile, è la seguente: un albero AVL di altezza h è minimo se è quello che contiene il minimo numero di nodi a parità di altezza.

Facciamo alcuni esempi di AVL minimi.

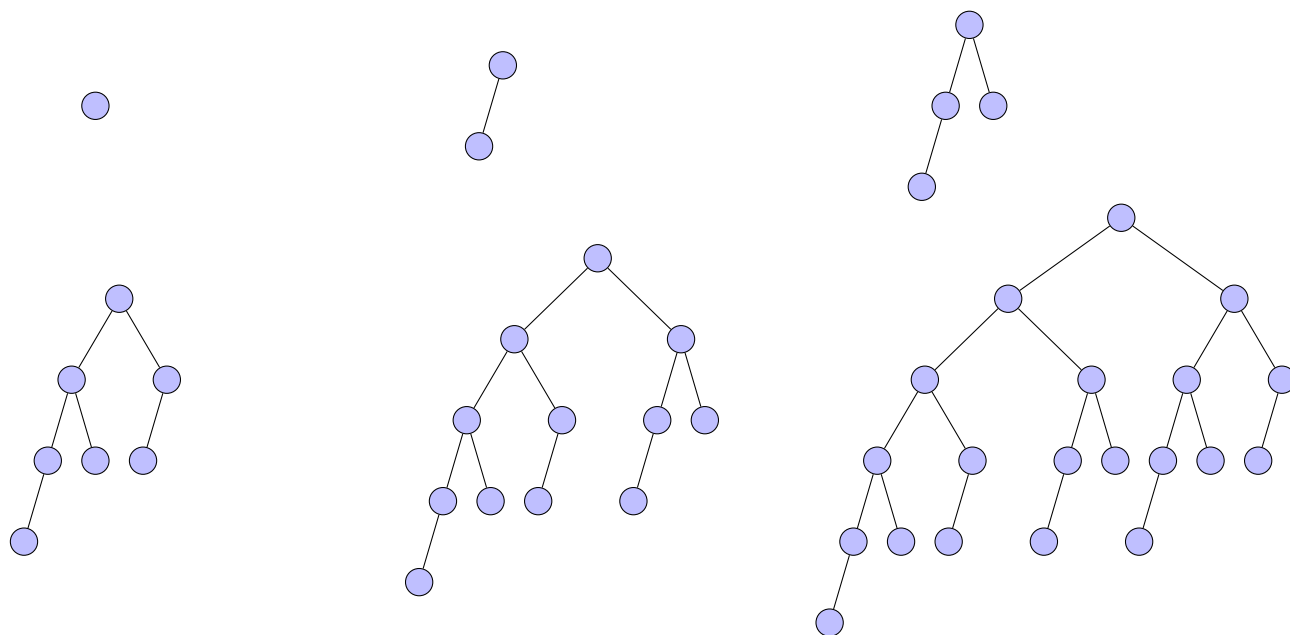


Figura 4.20: Alberi AVL minimi di altezza 0, 1, 2, 3, 4 e 5

Osserviamo a questo punto una proprietà particolare degli AVL minimi.

Perché ci interessano gli AVL minimi?

Intuitivamente sono quelli più sbilanciati, quindi se riusciamo a controllare l'altezza di quelli più sbilanciati potremo facilmente fare lo stesso con gli altri migliori.

Quindi, se riusciamo a far vedere che gli AVL minimi hanno un'altezza limitata dal logaritmo del numero di nodi, gli altri AVL, che sono solo meglio, non possono avere un'altezza peggiore. Di conseguenza, invece di prendere in considerazione tutti i possibili alberi AVL ci soffermeremo sui minimali.

Vedremo ora che effettivamente sappiamo controllarli e non solo, possiamo descriverli in un modo molto semplice che ci permette di analizzarli.

Mostriamo adesso un esempio di generalizzazione degli AVL minimi.

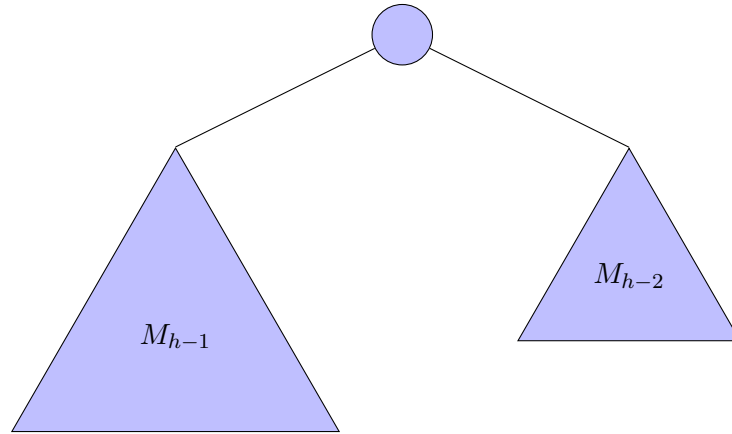


Figura 4.21: Generalizzazione della struttura di un albero AVL minimo di altezza h

Dove con h si intende l'altezza dell'albero AVL e con M_h l'albero AVL minimo con altezza h .

Sembrerebbe che ogni AVL minimo con una certa altezza h sia fatto, modulo permutazioni, da un nodo più un AVL minimo di altezza $h - 1$ e da un AVL minimo di altezza $h - 2$.

Questa è un'osservazione empirica, va dimostrata. Lo faremo per induzione.

Abbiamo detto che entrambi i sottoalberi connessi alla radice dell'AVL minimo devono essere a loro volta AVL minimi. Questo perché se così non fosse significherebbe che c'è un nodo che possiamo togliere lasciando invariata l'altezza.

Avremo quindi un nuovo albero con la stessa altezza ma con un nodo in meno, quindi l'albero di partenza non sarebbe minimo, che è assurdo.

Dobbiamo stare attenti e dimostrare che rimuovendo un nodo non si perda la proprietà di essere AVL. Ma questo è immediato visto che la proprietà in questione deve essere vera in tutto l'albero.

Teorema 4.10.1 (Struttura albero AVL minimo).

Dobbiamo quindi dimostrare che effettivamente un AVL minimo debba avere per forza la struttura descritta.

Dimostrazione.

I casi base sono banali, come mostrato con esempi.

Se per ipotesi l'albero è un AVL minimo di altezza h , come abbiamo osservato, i due sottoalberi destro e sinistro devono essere necessariamente AVL minimi di altezza $h - 1$ e $h - 2$.

Supponiamo quindi l'assunto vero per h e consideriamo la proprietà per $h + 1$.

Risulta immediato che per ottenere un albero AVL minimo di altezza $h + 1$ necessariamente uno dei due sottoalberi della radice dovrà avere altezza h , ma non entrambi, in quanto chiaramente non sarebbe minimo. Per ottenere un albero AVL minimo di altezza $h + 1$ dobbiamo quindi necessariamente avere come figli del nodo radice un sottoalbero AVL minimo di altezza h ed uno di altezza $h - 1$.

Vista l'ipotesi induttiva questo termina la dimostrazione.

□

Come si nota la dimostrazione è banale viste le proprietà dell'albero AVL.

Attenzione. Stiamo usando il termine minimo in modo leggermente improprio. Non stiamo considerando tutte le possibili permutazioni.

Sarebbe più corretto formalmente utilizzare il termine minimale.

4.10.2 Proprietà degli alberi AVL

Stiamo impropriamente, ma per semplicità di trattazione, usando il termine AVL per indicare gli alberi binari di ricerca AVL. Useremo l'acronimo MAVL per indicare un albero binario di ricerca AVL minimale.

Questa struttura ci dà un controllo enorme sul numero di nodi perché possiamo scrivere un'equazione di ricorrenza per il numero di nodi di un albero AVL minimo di una certa altezza.

$$NN_{MAVL} = \begin{cases} 0 & \text{se } h = -1 \\ 1 & \text{se } h = 0 \\ 1 + NN_{MAVL}(h-1) + NN_{MAVL}(h-2) & \text{altrimenti} \end{cases} \quad (4.8)$$

Osserviamo chiaramente una similitudine tra l'equazione di ricorrenza appena scritta e l'equazione di ricorrenza dei numeri di Fibonacci.

$$Fib(n) = \begin{cases} 0 & \text{se } n = 0 \\ 1 & \text{se } n = 1 \vee n = 2 \\ Fib(n-1) + Fib(n-2) & \text{altrimenti} \end{cases} \quad (4.9)$$

Ci si può aspettare quindi che in qualche modo questa funzione si comporti grosso modo come la funzione dei numeri di Fibonacci.

Per capire cosa significa grosso modo, quello che si può fare in prima battuta è cercare di analizzare un prefisso della sequenza, quindi vedere per esempio per i primi tre, quattro, cinque valori come si comporta la funzione, cercare di vedere se effettivamente la nostra intuizione sia supportata dall'evidenza sperimentale e solo dopo aver capito che esiste la relazione tra le due provarlo per induzione.

h	0	1	2	3	4	5	6	7	8	9	10
$NN_{MAVL}(h)$	1	2	4	7	12	20	33	54	88	143	232
Fib(h)	0	1	1	2	3	5	8	13	21	34	55

Tabella 4.3: Valori di $NN_{MAVL}(h)$ e $Fib(h)$

Sembrerebbe che la sequenza del numero di nodi di un albero AVL sia esattamente quella dei numeri di Fibonacci riscalata in avanti al quale dobbiamo sottrarre 1.

Nello specifico:

$$NN_{MAVL}(h) = Fib(h+3) - 1$$

Teorema 4.10.2 (Correlazione tra NN_{MAVL} e Fib).

$$NN_{MAVL}(h) = Fib(h+3) - 1$$

Dimostrazione.

I casi base sono banali, basta osservare i valori della tabella per $h = 0$ e $h = 1$.

Assumiamo, quindi, la proprietà vera per un generico ma fissato $h - 1$ e consideriamola per h .

Segue

$$\begin{aligned}
 NN_{MAVL}(h) &= 1 + NN_{MAVL}(h - 1) + NN_{MAVL}(h - 2) \\
 &= 1 + [Fib((h - 1) + 3) - 1] + [Fib((h - 2) + 3) - 1] \\
 &= Fib(h + 2) + Fib(h + 1) - 1 \\
 &= Fib(h + 3) - 1
 \end{aligned}$$

□

Quindi l'andamento del numero di nodi in funzione dell'altezza negli alberi AVL minimi segue l'andamento dei numeri di Fibonacci.

Ricordiamo, ora, lo sviluppo intero in forma chiusa di Fibonacci.

$$Fib(n) = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}} \quad (4.10)$$

Per le osservazioni fatte segue chiaramente

$$NN_{MAVL}(h) = Fib(h + 3) - 1 = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^{h+3} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+3}}{\sqrt{5}}$$

Se noi togliessimo dalla formula di Binet la parte minore otterremmo ovviamente qualcosa che sovrasta Fibonacci, e visto che a noi interessa trovare un limite superiore per la funzione, è più che sufficiente.

Per semplificare i calcoli, effettuiamo, inoltre, la seguente sostituzione.

$$\phi = \frac{1 + \sqrt{5}}{2}$$

Ma allora, è ovvio, che il numero di nodi in un albero AVL minimo non può essere più grande del limite superiore appena trovato.

$$NN_{MAVL}(h) \leq \frac{\phi^{h+3}}{\sqrt{5}} - 1$$

Quello che possiamo immediatamente ottenere è quindi, dato il numero di nodi, qual è l'altezza dell'albero AVL minimo.

Teorema 4.10.3 (Altezza albero AVL minimo).

L'altezza di un albero AVL minimo è logaritmica nel numero di nodi.

Dimostrazione.

Dalle osservazioni fatte precedentemente segue immediatamente

$$NN_{MAVL}(h) \leq \frac{\phi^{h+3}}{\sqrt{5}} - 1 \iff \sqrt{5}(NN_{MAVL}(h) + 1) \leq \phi^{h+3} \iff \log_{\phi}(\sqrt{5}(NN_{MAVL}(h) + 1)) \leq h + 3$$

Abbiamo, infine

$$h = O(\log(NN_{MAVL}(h))) \quad (4.11)$$

□

Quindi gli alberi AVL minimi non possono avere, dato il numero di nodi, un'altezza troppo alta perché con una certa altezza abbiamo un numero esponenziale di nodi; se invertiamo le cose, otteniamo chiaramente, come abbiamo provato, che l'altezza di un albero AVL minimo è logaritmica nel numero di nodi.

Possiamo scrivere in generale che l'altezza di un certo albero AVL minimo è $O(\log(n))$, con n il numero di nodi. Equivalentemente $O(\log(NN_{MAVL}(h)))$.

Una volta che abbiamo limitato superiormente il numero di nodi dell'albero quello che dovremmo ottenere è un limite sull'altezza di un albero AVL.

Ricordiamo che ci siamo focalizzati sugli alberi AVL minimi perché sono più facili da descrivere e sarebbe stato più semplice trovare una relazione, cosa che abbiamo fatto, e perché sono i peggiori possibili, cioè sono quelli che a parità di numero di nodi sono i più alti visto che sono i più piccoli (inteso in riferimento al numero di nodi) a parità di altezza.

Di conseguenza se sono i più piccoli a parità di altezza qualunque altro albero con la stessa altezza dovrebbe essere più grande, il che significa che quindi abbiamo ancora spazio per mettere altri dati senza forzare l'altezza a crescere.

Quindi a parità di nodi un altro albero AVL non minimo deve essere più basso o di stessa altezza, non può essere di certo più alto, e quindi si ottiene la proprietà voluta, cioè che ogni albero AVL ha un'altezza che è logaritmica nel numero di nodi.

L'idea è di dimostrare questa proprietà per assurdo. Per semplicità ci mettiamo nel caso in cui consideriamo alberi aventi una dimensione pari alla dimensione di un possibile albero AVL minimo.

Per il ragionamento appena fatto non cambia assolutamente nulla ai fini della dimostrazione.

Formalmente.

Proposizione 4.10.1 (Altezza albero AVL).

$\forall T \in AVL$ con n Nodi, $\exists T^* \in MAVL$ con $n^* \leq n$ Nodi, $(h(T) \leq h(T^*))$

Dimostrazione.

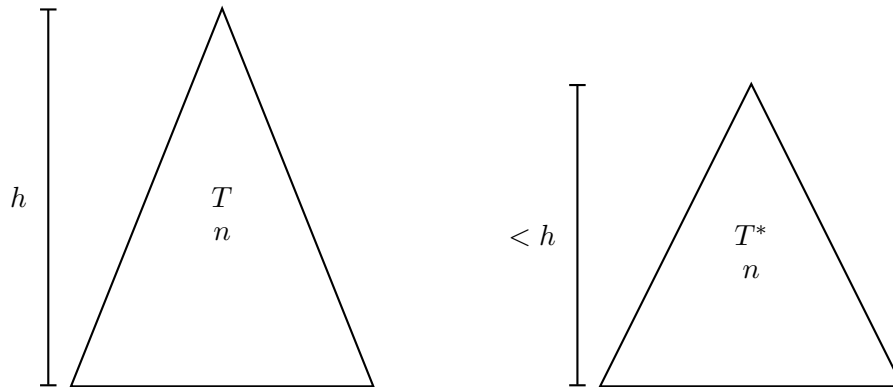
Dimostriamo la proposizione per assurdo, negando la tesi e mettendoci nel caso peggiore. Quindi, supponiamo vera la seguente proposizione

$\exists T \in AVL$ con n Nodi ($\forall T^* \in MAVL$ con n Nodi $(h(T) > h(T^*))$)

Facciamo una prima osservazione.

Abbiamo dimostrato che, a meno di permutazioni, non esistono due alberi AVL minimi avente lo stesso numero di nodi o la stessa altezza; pertanto fissata una dimensione n , l'albero T^* risulta unico.

Quindi, fissato un albero AVL T di altezza h , possiamo rappresentare la situazione della proposizione che nega la tesi nel seguente modo.



Immaginiamo, ora, di prunare via tutte le foglie poste ad altezza h dell'albero T . Siamo sicuri di eliminare qualche nodo in quanto l'albero T ha altezza h .



A seguito di questa operazione i due alberi avranno necessariamente identica altezza. Ora, l'albero T ha quindi un numero di nodi strettamente minore del numero di nodi dell'albero T^* , che ricordiamo essere un albero AVL minimo.

Se ora mostriamo che rimuovendo le foglie ad altezza h di un albero AVL di altezza h otteniamo ancora un albero AVL avremo chiaramente ottenuto un assurdo, in quanto abbiamo già dimostrato che gli alberi AVL minimi sono gli alberi AVL che per una certa altezza hanno il minor numero di nodi.

Supponiamo, quindi, per assurdo che un albero AVL $T1$ di altezza $h1$, a seguito della rimozione delle foglie ad altezza $h1$, passi ad essere un albero che non appartiene alla classe degli alberi AVL, che chiameremo $T2$, con altezza $h2 = h1 - 1$.

Visto che l'albero $T1$ per ipotesi era un albero AVL, mentre stiamo assumendo per assurdo che l'albero $T2$ non lo sia, deve esistere necessariamente almeno un nodo dell'albero $T2$ che viola la proprietà degli alberi AVL e che quindi ha come figli sottoalberi che hanno scarto almeno di 2 dal punto di vista delle altezze relative. Questo risulta essere banalmente un assurdo in quanto partivamo da un albero AVL ed è chiaramente impossibile che partendo da una condizione nella quale ogni nodo rispettava la proprietà di AVL, rimuovendo dei nodi, questa possa essere violata. $\square$

Abbiamo, quindi, dimostrato che gli alberi AVL hanno un'altezza logaritmica nel numero di nodi.

Nella semiformalizzazione precedente siamo stati leggermente imprecisi. Gli alberi AVL minimi hanno dimensioni che sono descritte dall'equazione di ricorrenza precedentemente analizzata e quindi non coprono l'intero arco dei potenziali valori.

Pertanto quello che ci interessa non è trovare un albero AVL minimo con lo stesso identico numero di nodi ma prendere un albero AVL minimo che può avere al più quel numero di nodi, in quanto comunque il vincolo dimostrato riguardo l'altezza risulta essere valido. Si tratta di un dettaglio minimo per i nostri scopi.

Quindi pur avendo guardato solo ai minimi siamo stati in grado di avere un limite su tutti gli altri. Abbiamo anche inoltre dimostrato che gli alberi AVL minimi sono i peggiori tra tutti gli alberi AVL.

Questo comporta che se applichiamo gli algoritmi di lettura ad un albero AVL siamo sicuri che la complessità degli algoritmi sarà logaritmica nel numero di nodi perché è lineare nell'altezza dell'albero che abbiamo appena dimostrato essere al più logaritmica nel numero di nodi.

Il problema ora è che le funzioni di modifica, cioè inserimento e cancellazione, che abbiamo visto, pur partendo da un AVL se le lasciassimo così come le abbiamo viste purtroppo ci possono portare fuori dalla classe degli AVL.

Di conseguenza ogni volta che ci accorgiamo della presenza di una violazione dobbiamo subito correggerla. Capiremo quindi quali possono essere le violazioni e come correggerle.

Quindi abbiamo una classe di alberi che ci permette di rappresentare insiemi di cardinalità arbitraria e con un'altezza logaritmica, dove le ricerche sono ottimali.

4.10.3 Inserimento in un albero AVL

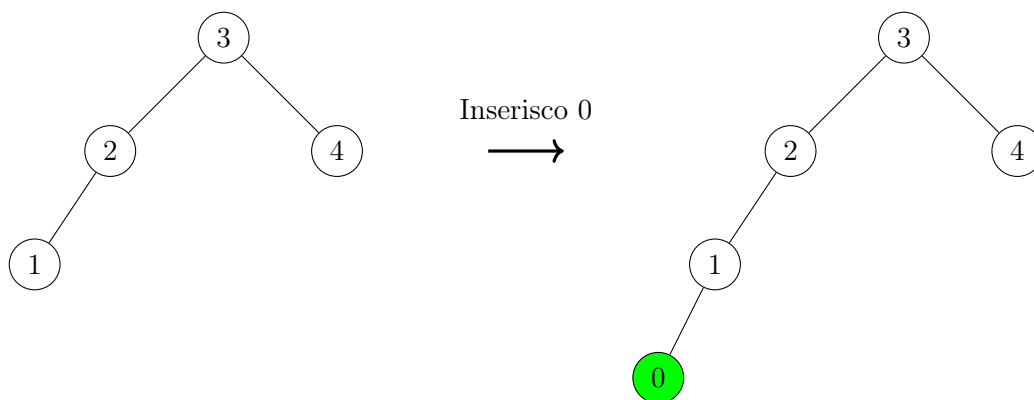
Parliamo prima dell'inserimento, la cancellazione è molto simile ma la vedremo in un secondo momento.

Supponiamo di andare ad inserire un dato in un AVL e che tale inserimento porti ad una violazione.

Non è detto che ci sia un'unica violazione, l'inserimento di un nodo potrebbe aver creato più violazioni. Andiamo a prendere quella più profonda, cioè quella che troveremo prima andando a ritroso nelle chiamate ricorsive (ricordiamo che l'algoritmo di inserimento è ricorsivo).

Quello che faremo per correggere tale violazione sarà implementare un'operazione di ricorsione per ristabilire la proprietà di albero AVL.

Esempi di Inserimento con violazione.



Dopo l'inserimento si è riscontrata una violazione ovvero, l'albero non appartiene più alla famiglia degli AVL. Risolviamo tale violazione applicando una rotazione da sinistra a destra sul nodo 2 (il primo nodo a partire dal basso sul quale viene rilevata una violazione)

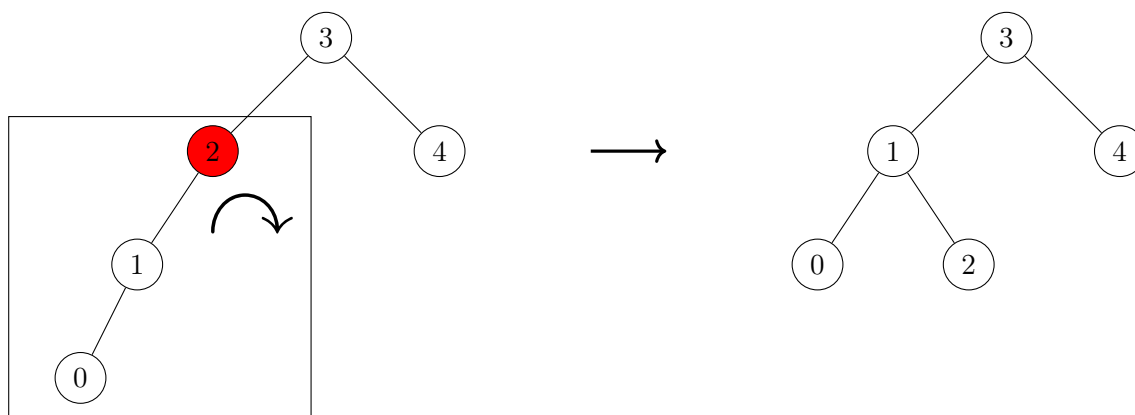
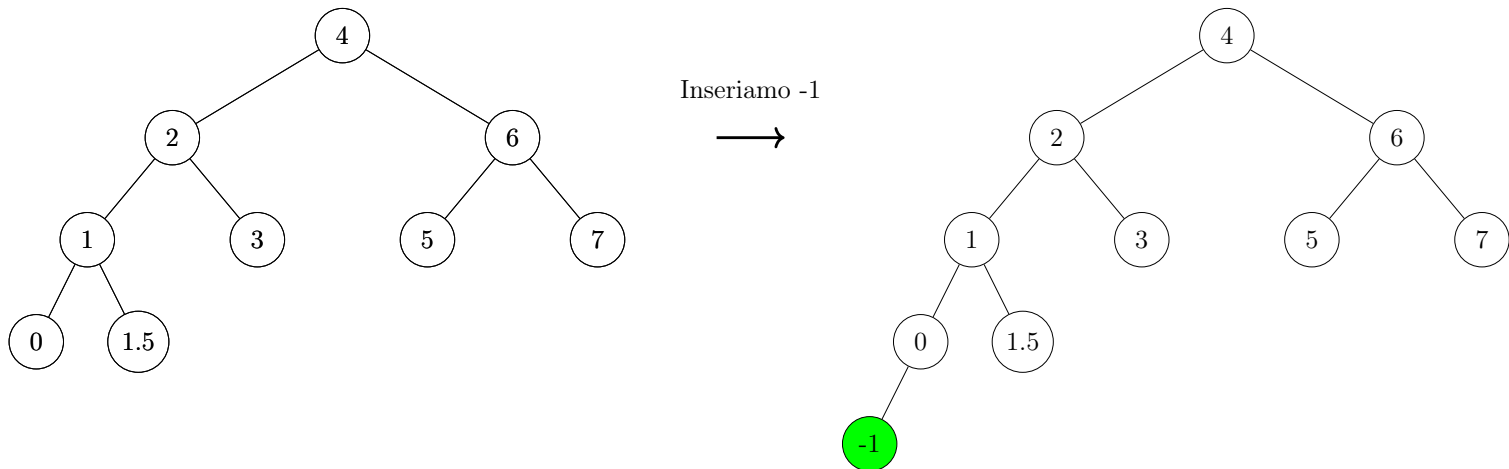


Figura 4.22: Esempio di inserimento in un AVL



Risolviamo applicando una rotazione da sinistra a destra sul nodo 2 (il primo nodo a partire dal basso sul quale viene rilevata una violazione)

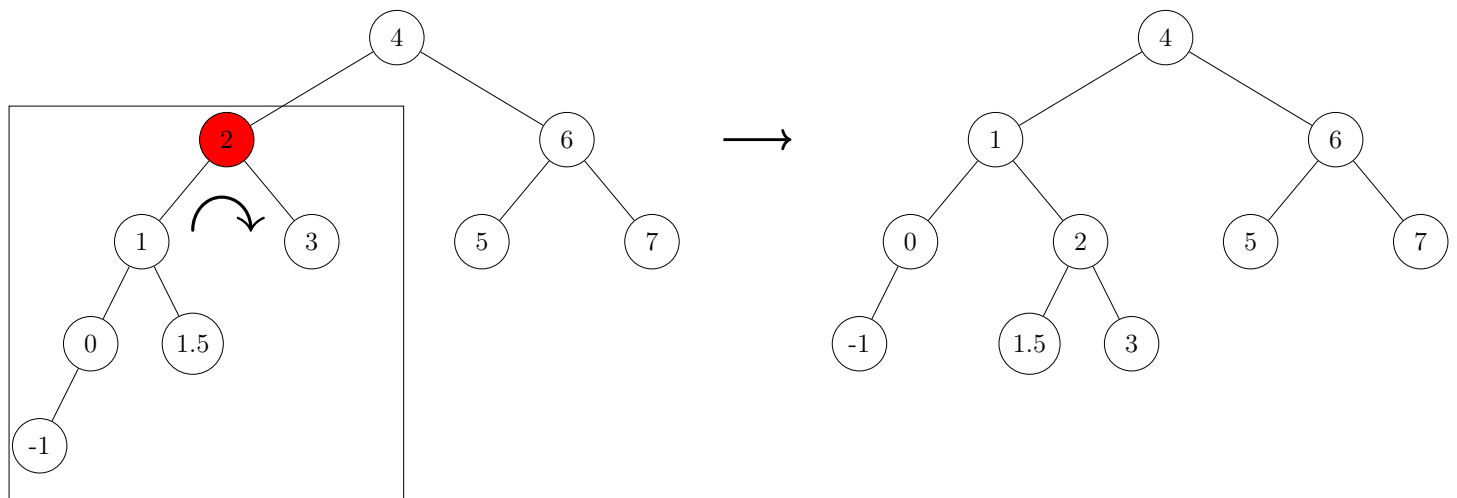
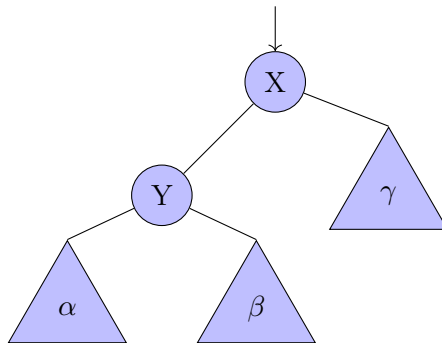


Figura 4.23: Esempio di inserimento in un AVL

4.10.4 Rotazione in un albero binario

Vediamo in modo astratto le operazioni di rotazione in un albero.

Consideriamo un albero fatto nel seguente modo: c'è un nodo a cui daremo il nome X , un nodo figlio Y e a questi poi sono collegati gli alberi α , β e γ . Nello specifico gli alberi α e β saranno figli del nodo Y e γ figlio del nodo X .



Non è importante la struttura degli alberi α , β e γ ; in particolare non è importante che nel complesso l'albero considerato sia un AVL.

L'operazione di rotazione si applica su alberi di struttura arbitraria. In effetti vedremo tale operazione per gli alberi AVL e per gli alberi red black, ma nulla vieta di applicare tale operazione ad un albero generico.

Supponiamo di voler applicare la rotazione che chiameremo da sinistra a destra, `left_to_right`, sul nodo X .

Osserviamo che quella che analizzeremo è solo una delle due rotazioni, c'è una rotazione speculare da destra verso sinistra, `right_to_left`. Chiaramente gli algoritmi che vedremo devono correggere entrambi i problemi. Quindi ci sarà sempre una parte di aggiustamento a destra e una parte di aggiustamento a sinistra, noi vedremo solo quella di sinistra.

Lo scopo di fondo è accorciare il ramo lungo dell'albero ed allungare quello corto, per questo è necessaria la rotazione da sinistra verso destra.

Il nodo Y deve diventare la nuova radice.

Ciò che sta alla sinistra di Y non viene modificato, in quanto non dava problemi.

Ciò che sta alla destra di X non viene modificato, in quanto non dava problemi.

Ciò che sta alla destra di Y va modificato e posizionato alla sinistra di X .

X va posizionato alla destra di Y , che è come detto la nuova radice.

Sottolineiamo che le rotazioni non violano mai la proprietà di albero binario di ricerca. Possono violare quelle di albero AVL.

Ecco l'algoritmo per la rotazione da sinistra a destra in un albero binario non vuoto.

Algoritmo 47: Rotazione da sinistra a destra**Signature** L2R_rotation: $BT(D) \setminus \{\perp\} \rightarrow BT(D) \setminus \{\perp\}$ **function** L2R_rotation(X)

```

    // salvo il puntatore alla nuova radice
    // senza questa istruzione avrei perso il puntatore al nodo Y
1  Y ← X.l
    // sposto il sottoalbero alla destra della nuova radice
    // alla sinistra della vecchia radice
2  X.l ← Y.r
    // pongo la vecchia radice alla destra della nuova
3  Y.r ← X
    // restituisco la nuova radice
4  return Y

```

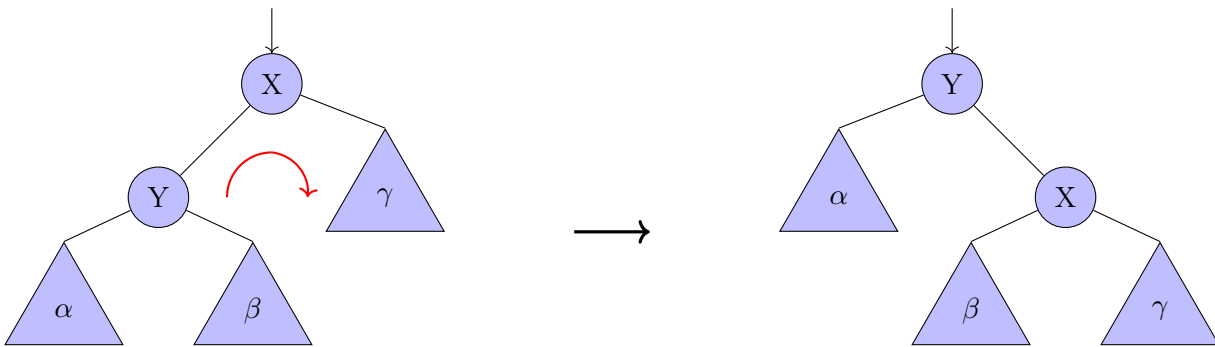


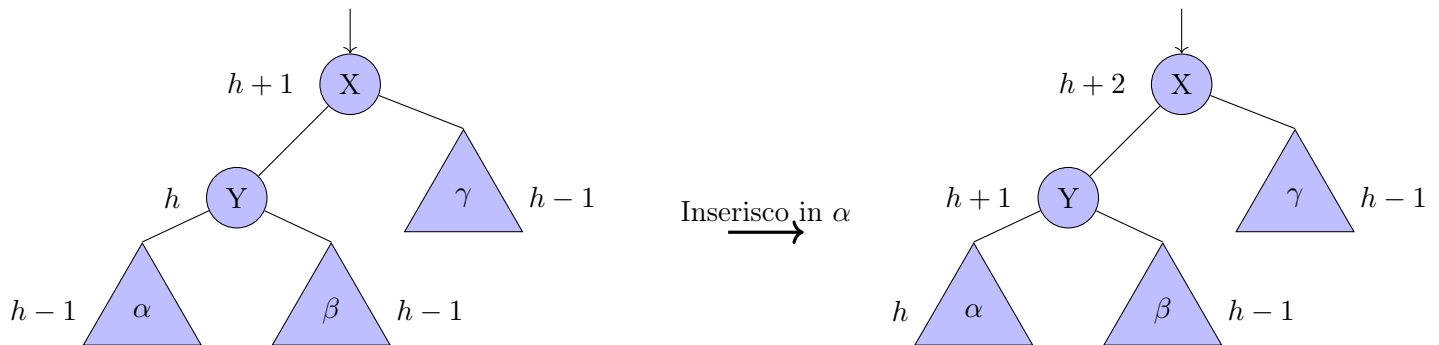
Figura 4.24: Esempio di rotazione da sinistra a destra

Una volta che sappiamo come effettuare le rotazioni concretamente vediamo qual è la casistica degli inserimenti.

4.10.5 Inserimento in un albero AVL: caso 1

Sottolineiamo che affronteremo i casi in cui a valle dell'inserimento nasce un errore. Se le altezze fossero diverse l'inserimento non è detto che andrebbe a causare l'errore e in particolare non è detto che andrebbe a causare l'errore che ci interessa.

Quindi quando diamo lo schema delle altezze stiamo dando un esempio astratto del caso in cui a valle di un inserimento si viene a creare uno sbilanciamento di quel particolare tipo.



Risolviamo applicando una rotazione da sinistra a destra sul nodo X (il primo nodo a partire dal basso sul quale viene rilevata una violazione)

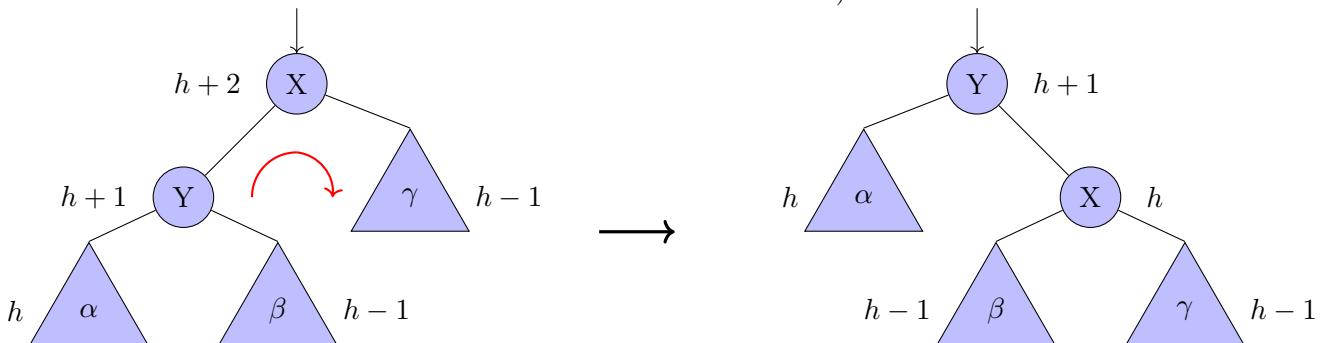


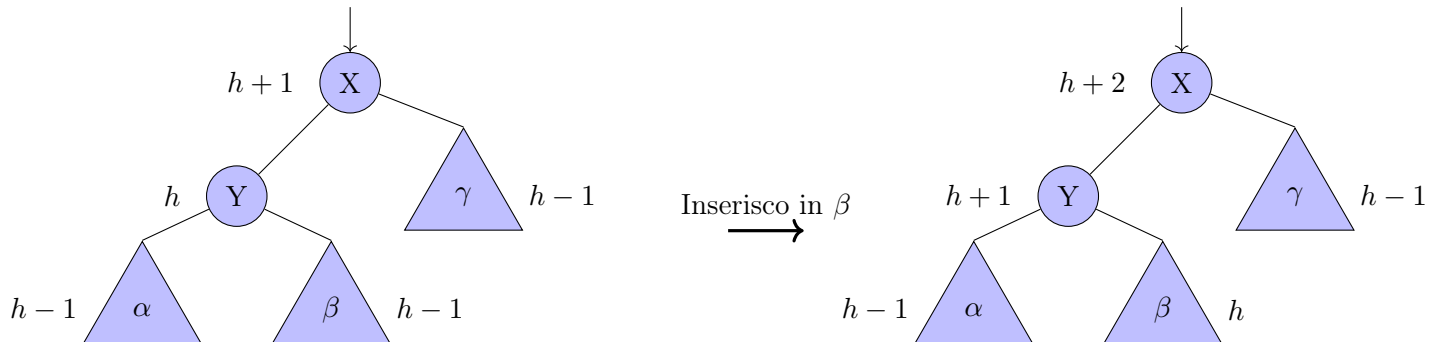
Figura 4.25: Inserimento in AVL (caso 1)

Abbiamo visto il primo caso dell'inserimento in AVL e abbiamo osservato come risolvere il problema nel caso in cui l'inserimento va completamente a sinistra (e specularmente a destra) rispetto al nodo in cui è presente l'errore.

4.10.6 Inserimento in un albero AVL: caso 2

Il problema nasce nel momento in cui l'inserimento a sinistra non va completamente a sinistra rispetto al nodo in cui è presente l'errore.

Abbiamo accennato al fatto che purtroppo se andassimo ad applicare esattamente lo stesso identico algoritmo, la stessa identica correzione del primo caso non risolveremmo il problema, perché semplicemente stiamo spostando l'albero molto profondo, lasciando però il problema esattamente tal quale, perché semplicemente andiamo a spostare il problema ma non a risolverlo.



Proviamo a risolvere applicando una rotazione da sinistra a destra sul nodo X (il primo nodo a partire dal basso sul quale viene rilevata una violazione), ma notiamo che **così non risolviamo nulla!**

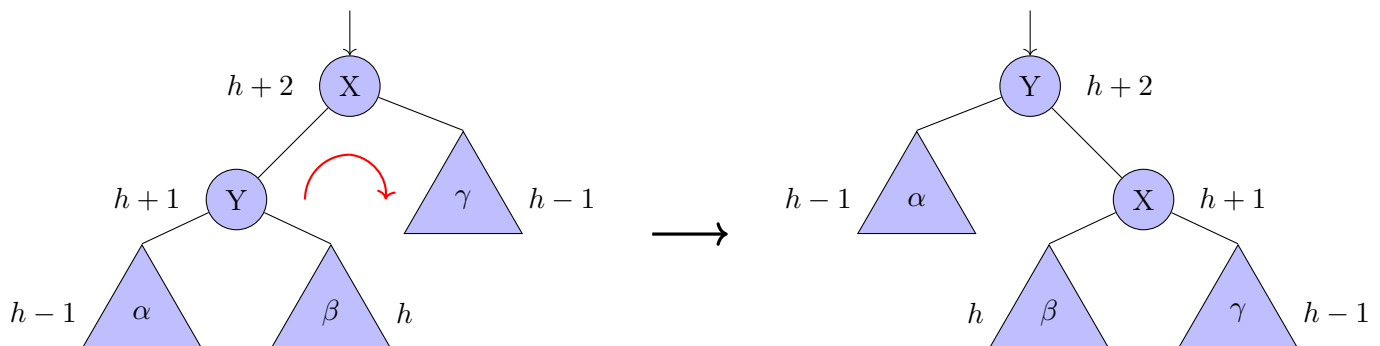
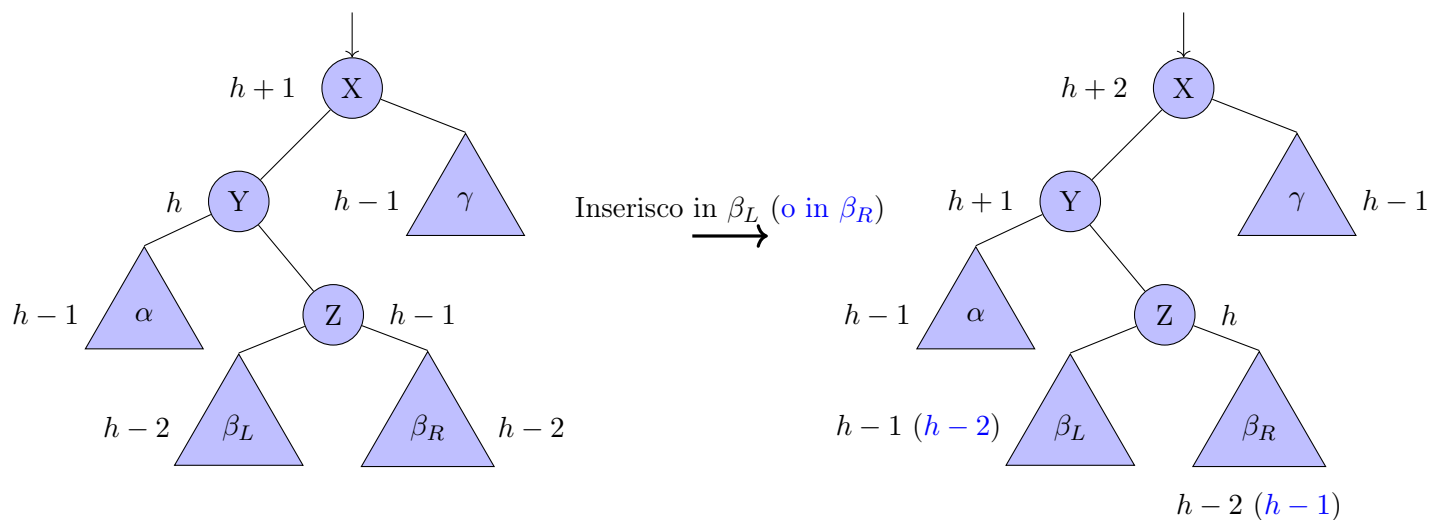


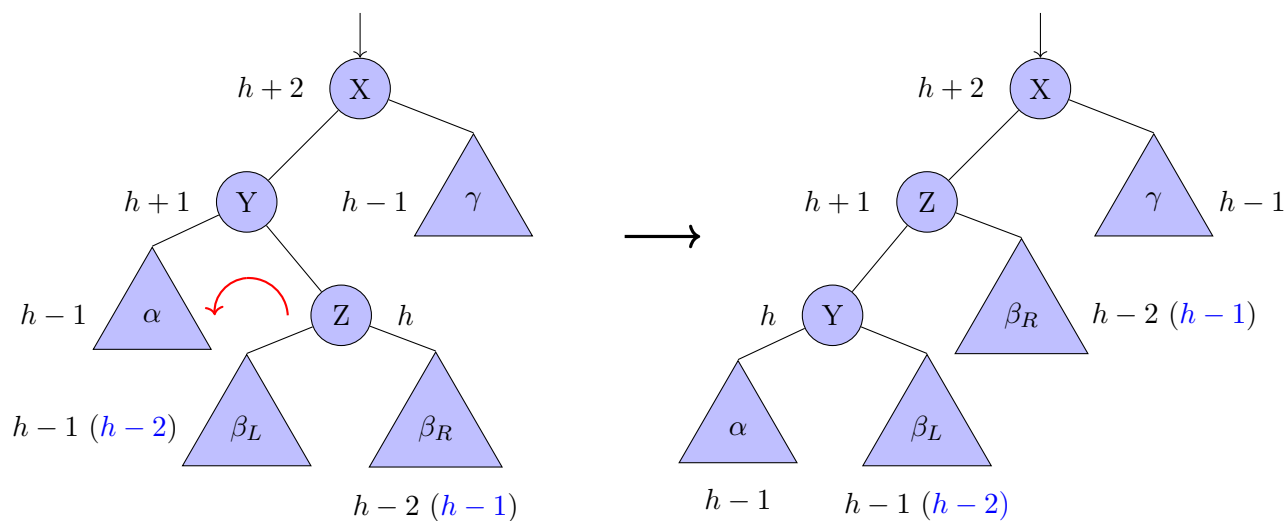
Figura 4.26: Inserimento in AVL (caso 2 con risoluzione sbagliata)

Per risolvere questo problema quello che dobbiamo fare è spezzare l'albero centrale, quello più pesante, nelle sue due componenti e trattarle separatamente. Cioè prima separarle tramite una rotazione inversa da destra a sinistra sul nodo figlio di quello dove è presente l'errore e poi una volta che si saranno separate le due parti possiamo bilanciarle con la stessa identica operazione di rotazione del primo caso perché una volta separate sarà possibile spostare solo una delle due parti pesanti in modo tale da non riportare l'intero peso da sinistra a destra ma solo una parte, e questo risolve il problema.

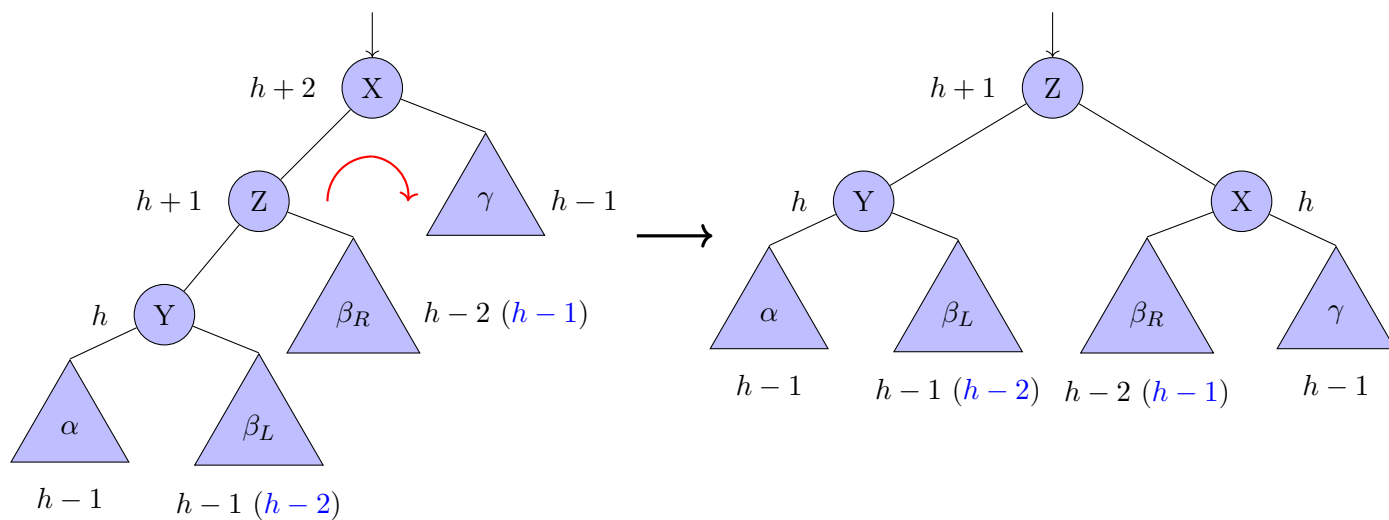
Ora vedremo la versione astratta e il ragionamento. Dopodiché passeremo all'algoritmo.



Risolviamo applicando prima una rotazione da destra a sinistra sul nodo Y ...



... e poi una rotazione da sinistra a destra sul nodo X



Poi vedremo la cancellazione.

Anticipiamo che i problemi di cancellazione sono esattamente speculari a quelli dell'inserimento. Quando si va a cancellare un nodo uno dei due alberi diventerà più basso e ovviamente si genereranno problemi di sbilanciamento che saranno esattamente l'opposto di quelli di inserimento.

Quindi, per ricapitolare.

Andiamo ad inserire come se fosse un albero binario di ricerca arbitraria.

A ritorno dalle chiamate ricorsive controlliamo lo sbilanciamento.

Per fare questo dobbiamo controllare le altezze.

Se troviamo un problema dobbiamo capire dove è avvenuto l'inserimento e conseguentemente correggerlo con la procedura adatta.

4.10.7 Rappresentazione in memoria di un nodo di un albero AVL

Prima di guardare l'algoritmo, dobbiamo soffermarci sul fatto che l'altezza non possiamo ricalcolarla ogni volta perché valutare l'altezza non è a tempo costante. In particolare, visto che non sappiamo dove si pone il ramo più profondo, diventerebbe lineare nella dimensione dell'albero calcolare l'altezza.

È necessario quindi memorizzare l'altezza del nodo. Ogni nodo deve avere all'interno un campo accessorio oltre alla chiave e ai puntatori figlio destro, figlio sinistro. Un campo accessorio per l'altezza. Altrimenti tutti questi ragionamenti non li possiamo fare. Dovremmo ogni volta ricalcolare l'altezza e come detto, calcolare l'altezza di un albero significa percorrere tutti i percorsi.

Quindi abbiamo bisogno di un campo altezza. Quando viene creato un nodo, poiché è una foglia, l'altezza sarà zero. E poi man mano, salendo verso l'alto, andando a percorrere l'albero a ritroso, aggiusteremo le altezze. Quindi è un'operazione a tempo costante.

Questo fatto di avere un'informazione accessoria non è qualcosa che sarà legata solo all'AVL, vedremo che caratterizza anche gli alberi red black.

Un nodo di un albero AVL non è più una terna, chiave e poi puntatore figlio sinistro e puntatore figlio destro, ma sarà una quaterna formata da: chiave, altezza e poi i due campi sinistro-destro.

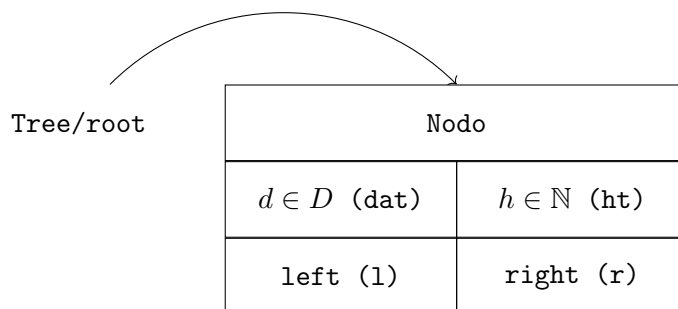


Figura 4.28: Nodo di un albero AVL

Quando si viene a creare un nuovo nodo con una `build_node`, si inserirà il dato, si settano i puntatori figli a `NULL` e l'altezza a 0.

4.10.8 Algoritmo: gestione altezza in un albero AVL

Passiamo agli algoritmi.

Il primo algoritmo serve ad ottenere l'altezza di un nodo.
Serve una funzione per gestire il puntatore a NULL.

Ecco l'algoritmo.

Algoritmo 48: Altezza di un nodo

Signature height: $BTAVL(D) \rightarrow \mathbb{N} \cup \{-1\}$

```

function height(X)
  // se il nodo è NULL
1  if (X =  $\perp$ ) then
    // ... restituisco -1
2    return -1
3  else
    // altrimenti restituisco l'altezza
4    return X.ht
  
```

Serve ora un algoritmo per aggiornare l'altezza.

Algoritmo 49: Aggiornamento dell'altezza di un nodo

Signature update_height: $BTAVL(D)$

```

function update_height(X)
  // prendo l'altezza del figlio sinistro
1  hl  $\leftarrow$  height(X.l)
  // prendo l'altezza del figlio destro
2  hr  $\leftarrow$  height(X.r)
  // aggiorniamo l'altezza con il successivo
  // del massimo delle due altezze
3  X.ht  $\leftarrow$  1 + max(hl, hr)
  
```

Tale algoritmo assume che dato un nodo le altezze dei figli siano corrette.

In generale, visto che andiamo a ritroso, le altezze vengono aggiornate man mano che risaliamo e pertanto è

un'assunzione valida.

4.10.9 Algoritmo: rotazione in un albero AVL

Andiamo ora a definire la funzione che effettua la rotazione in un albero AVL.

Algoritmo 50: Rotazione da sinistra a destra in un albero AVL

Signature L2R_rotation_AVL: $BTAVL(D)$

function L2R_rotation_AVL(X)

```

1  // chiama la funzione di rotazione generica
  Y ← L2R_rotation(X)
  // aggiorna le altezze
2  update_height(X)
3  update_height(Y)
  // restituisce la nuova radice
4  return Y

```

Attenzione.

Visto che dopo la rotazione il nodo X diventa figlio del nodo Y è necessario aggiornare prima l'altezza di X e successivamente quella di Y.

Questa funzione esegue la rotazione semplice.

Vediamo ora la funzione che esegue la doppia rotazione.

Algoritmo 51: Rotazione doppia in un albero AVL

Signature LD_rotation_AVL: $BTAVL(D)$

function LD_rotation_AVL(X)

```

  // chiama la funzione di rotazione singola in AVL
  // ovviamente da destra a sinistra
  // sul figlio sinistro
1  X.l ← R2L_rotation_AVL(X.l)
  // chiama la funzione di rotazione singola in AVL
  // ovviamente da sinistra a destra
  // sul nodo in input
2  return L2R_rotation_AVL(X)

```

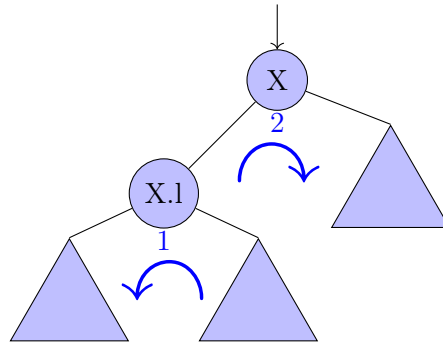


Figura 4.29: Rotazione doppia in AVL

4.10.10 Algoritmo: inserimento in un albero AVL

A questo punto abbiamo le funzionalità base per vedere l'intero algoritmo.

Algoritmo 52: Inserimento in un albero AVL

```

Signature insert_AVL:  $BTAVL(D) \times D \rightarrow BTAVL(D)$ 
function insert_AVL(X, d)
  // se sono arrivato in foglia
  // e non ho trovato nodi con il campo dato uguale
1  if (X =  $\perp$ ) then
    // procedo con l'inserimento
2    X  $\leftarrow$  build_node_AVL(d)
3  else
    // se il dato contenuto nel nodo corrente
    // è maggiore del dato da inserire
4    if (X.dat > d) then
      // richiamo la funzione sul sottoalbero sinistro
5      X.l  $\leftarrow$  insert_AVL(X.l, d)
      // bilancio in caso di sbilanciamento
      // mediante funzione accessoria
6      X  $\leftarrow$  left_balance(X)
    // se il dato contenuto nel nodo corrente
    // è minore del dato da inserire
7    else if (X.dat < d) then
      // richiamo la funzione sul sottoalbero destro
8      X.r  $\leftarrow$  insert_AVL(X.r, d)
      // bilancio in caso di sbilanciamento
      // mediante funzione accessoria
9      X  $\leftarrow$  right_balance(X)
    // restituisco il nodo corrente
10 return X
  
```

Vediamo ora la funzione che bilancia in caso di sbilanciamento. Vedremo solo la funzione `left_balance`.

Algoritmo 53: Bilanciamento da sinistra

```

Signature left_balance:  $BTAVL(D) \rightarrow BTAVL(D)$ 
function left_balance(X)
    // se sul nodo in input c'è uno sbilanciamento
    // a seguito di un inserimento nel sottoalbero sinistro
1   if (height(X.l) - height(X.r) > 1) then
        // bisogna capire dove sia avvenuto l'inserimento
        // se è avvenuto nel sottosottoalbero sinistro
2       if (height(X.l.l) > height(X.l.r)) then
            // applico la rotazione semplice
3         X ← L2R_rotation_AVL(X)
        else
4         // se è avvenuto nel sottosottoalbero destro
            // applico la doppia rotazione
5         X ← LD_rotation_AVL(X)
6   else
        // se non c'è sbilanciamento
        // aggiorniamo l'altezza
7       update_height(X)
    // restituisco il nodo corrente
8   return X

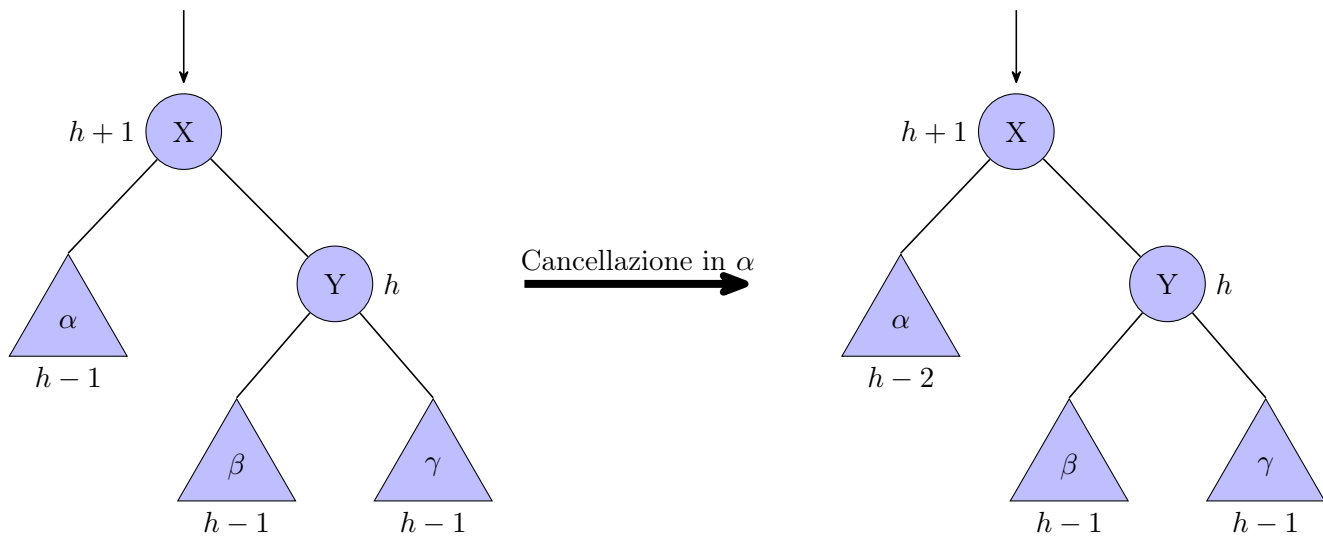
```

Come possiamo vedere questo algoritmo di inserimento è praticamente identico a quello classico con l'unica differenza che chiamiamo il bilanciamento. Vedremo che anche con gli alberi red black sarà essenzialmente la stessa cosa.

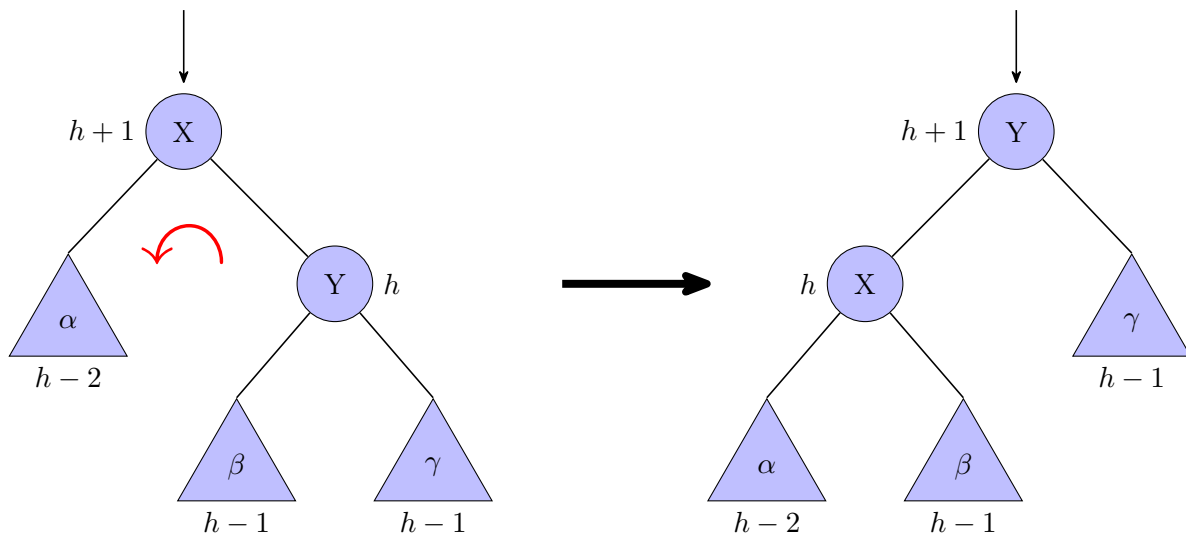
Questo algoritmo è agnostico rispetto al tipo di albero; l'unica cosa che deve sapere è quale procedura di correzione chiamare.

A questo punto guardiamo ciò che accade con la cancellazione e vediamo se effettivamente qualcosa di diverso oppure tutto ciò che abbiamo detto per l'inserimento vale tal quale per la cancellazione.

4.10.11 Cancellazione in un albero AVL: caso 1A

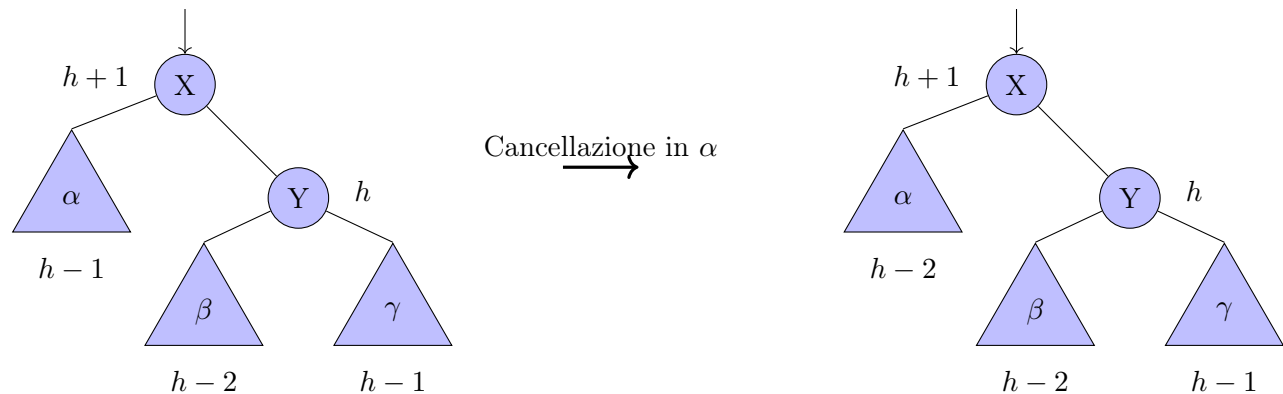


Risolviamo applicando una rotazione da destra a sinistra sul nodo X (il primo nodo a partire dal basso sul quale viene rilevata una violazione).



Osservando l'altezza del nodo radice del sottoalbero considerato nell'esempio a monte dell'inserimento e a valle della rotazione si conclude chiaramente che si tratti di un caso definitivamente risolutivo.

4.10.12 Cancellazione in un albero AVL: caso 1B



Risolviamo applicando una rotazione da destra a sinistra sul nodo X (il primo nodo a partire dal basso sul quale viene rilevata una violazione)

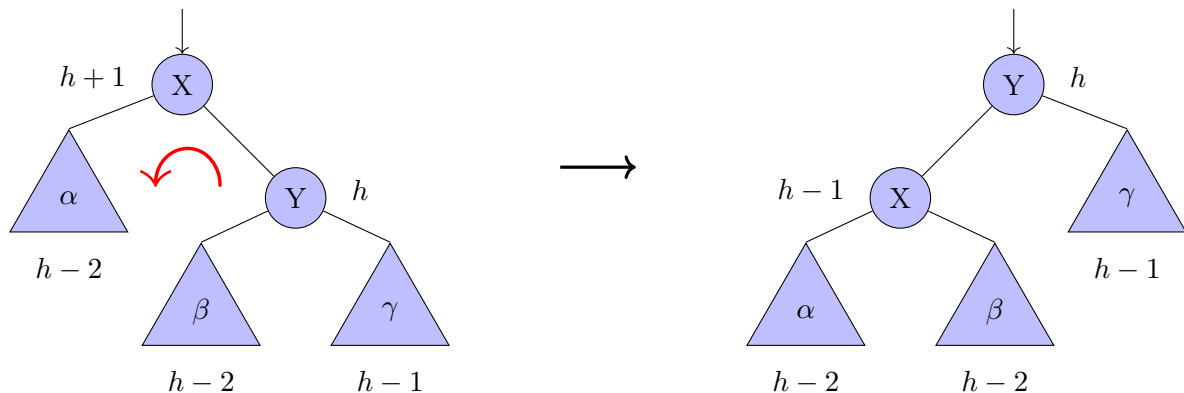
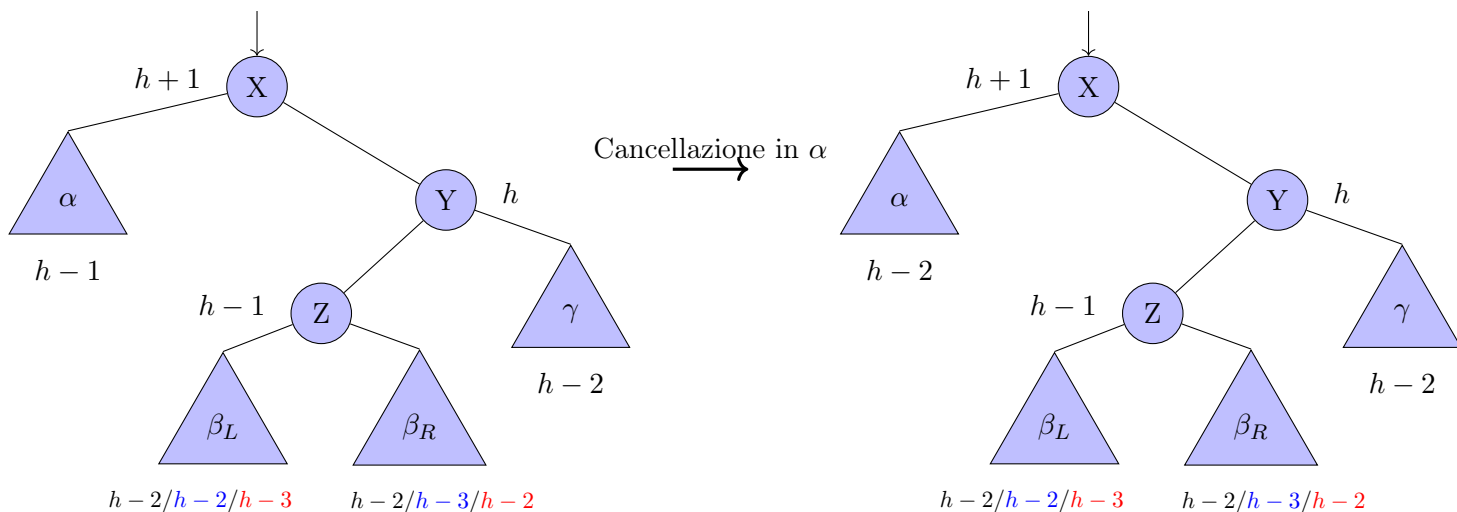


Figura 4.30: Cancellazione in AVL (caso 1)

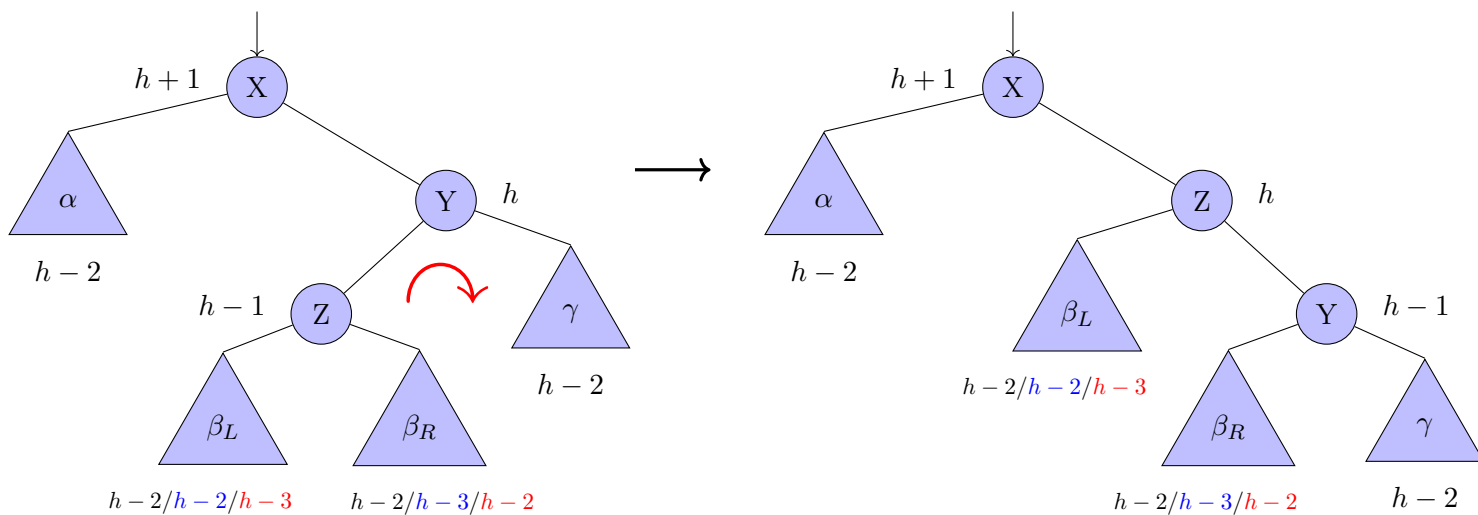
Da quanto si è potuto osservare, l'altezza del sottoalbero iniziale era $h+1$, ed al termine del bilanciamento l'altezza è diventata pari ad h , questa variazione di altezza ci fa intuire che il problema è stato risolto localmente, ma potrebbe aver propagato l'errore verso l'alto.

Riassumendo quindi, il caso 1 si applica quando il figlio destro del problema, ha come figlio più pesante quello alla sua destra (γ). Ciò si applica specularmente nel caso in cui la cancellazione avvenga a destra.

4.10.13 Cancellazione in un albero AVL: caso 2



Risolviamo applicando prima una rotazione da sinistra a destra sul nodo Y ...



... e poi una rotazione da destra a sinistra sul nodo X

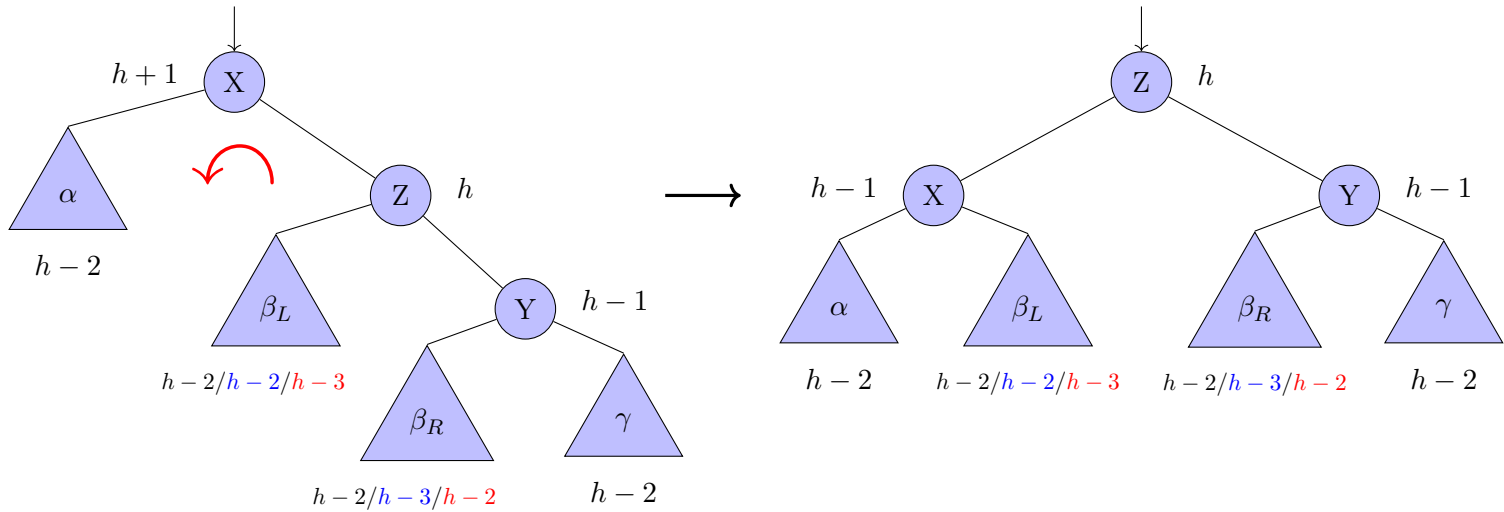


Figura 4.31: Cancellazione in AVL (caso 2)

Come per il caso 1, l'altezza del sottoalbero al termine del bilanciamento è variata e dunque, potrebbe aver propagato l'errore verso l'alto.

Riassumendo quindi, il caso 2 si applica quando il figlio destro del problema, ha come figlio più pesante quello alla sua sinistra (Z). Ciò si applica specularmente nel caso in cui la cancellazione avvenga a destra.

4.10.14 Algoritmo: cancellazione in un albero AVL

Purtroppo a differenza del caso dell'inserimento in cancellazione l'errore si può propagare fino in radice, quindi potremmo essere costretti ad effettuare più correzioni per una cancellazione perché a seguito delle correzioni l'altezza del sottoalbero considerato potrebbe cambiare e pertanto comportare la propagazione dell'errore verso l'alto.

Significa che a differenza dell'inserimento dove la funzione di correzione doveva agire esclusivamente localmente, in questo caso la correzione potrebbe essere necessaria fino in radice per correggere effettivamente il problema.

Se per correggere uno sbilanciamento dovuto all'eliminazione di un nodo sono necessarie due rotazioni l'altezza diminuisce e ciò potrebbe determinare la propagazione dell'errore verso l'alto.

In particolare il caso peggiore della cancellazione è la cancellazione in un albero AVL minimo perché un albero AVL minimo è l'albero con meno nodi possibili di quell'altezza. Se andiamo a cancellare la foglia meno profonda ci troveremo nel caso peggiore e la propagazione dell'errore raggiungerà la radice.

In generale la cancellazione può essere vista come un inserimento dal lato opposto; quindi l'algoritmo di bilanciamento verrà chiamato semplicemente al contrario rispetto all'inserimento.

Andremo a bilanciare dal lato opposto della cancellazione.

Vediamo l'algoritmo.

Algoritmo 54: Cancellazione in un albero AVL

Signature delete_AVL: $BTAVL(D) \times D \rightarrow BTAVL(D)$
function delete_AVL(X, d)
 // se il sottoalbero è non vuoto
 1 **if** (X $\neq \perp$) **then**
 // se il dato del nodo corrente è maggiore del dato in input
 2 **if** (X.dat > d) **then**
 // richiamo la funzione sul sottoalbero sinistro
 3 X.l $\leftarrow$ delete_AVL(X.l, d)
 // bilancio il lato opposto
 // mediante una funzione accessoria
 4 X $\leftarrow$ right_balance(X)
 // se il dato del nodo corrente è minore del dato in input
 5 **else if** (X.dat < d) **then**
 // richiamo la funzione sul sottoalbero destro
 6 X.r $\leftarrow$ delete_AVL(X.r, d)
 // bilancio il lato opposto
 // mediante una funzione accessoria
 7 X $\leftarrow$ left_balance(X)
 8 **else**
 // se ho trovato il dato da eliminare
 9 X $\leftarrow$ delete_data_AVL(X)
 // restituisco il nodo corrente
 10 **return** X

Vediamo ora la funzione che cancella il nodo in un albero AVL.

Algoritmo 55: Cancellazione di un nodo in un albero AVL

```

Signature delete_data_AVL:  $BTAVL(D) \rightarrow BTAVL(D)$ 
function delete_data_AVL(X)
    // se il figlio sinistro è NULL
1   if (X.l =  $\perp$ ) then
        // elimino il nodo
        // e restituisco il nodo da collegare
        // mediante una funzione accessoria
2       X  $\leftarrow$  skip_right(X)
3   else if (X.r =  $\perp$ ) then
        // se il figlio destro è NULL
        // elimino il nodo
        // e restituisco il nodo da collegare
        // mediante una funzione accessoria
4       X  $\leftarrow$  skip_left(X)
5   else
        // se il nodo da eliminare ha entrambi i figli
        // elimino il minimo del sottoalbero destro
        // sostituendo al dato corrente, da eliminare,
        // il valore del minimo del sottoalbero destro
        // mediante una funzione accessoria
6       X.dat  $\leftarrow$  get_&_delete_min_AVL(X.r, X)
        // bilancio il lato opposto
        // mediante una funzione accessoria
7       X  $\leftarrow$  left_balance(X)
    // restituisco il nodo corrente
8   return X

```

Sottolineiamo che nel caso di cancellazione di nodi con al più un figlio non è possibile creare sbilanciamenti visto che stiamo lavorando con alberi AVL e pertanto al più il sottoalbero figlio avrà altezza 1 nel caso specifico. Pertanto si usano le stesse funzioni di skip viste in precedenza senza curarsi di dover bilanciare.

Vediamo ora la cancellazione del minimo dal sottoalbero di destra; avviene fondamentalmente nello stesso identico modo, già visto in precedenza.

Ovviamente, poiché andiamo ad effettuare una cancellazione quello che può succedere è che si viene a creare uno sbilanciamento, quindi anche all'interno della funzione si dovrà bilanciare.

Ecco l'algoritmo.

Algoritmo 56: Cancellazione del minimo dal sottoalbero di destra AVL

```

Signature get_&_delete_min_AVL:  $AVL(D) \times AVL(D) \rightarrow D$ 
function get_&_delete_min_AVL(X, P)
    // se il sottoalbero sinistro è NULL
    // il minimo è stato trovato
1   if (X.l =  $\perp$ ) then
        // salvo il valore del minimo
2       d  $\leftarrow$  X.dat
        // elimino il nodo e lo sostituisco con il figlio destro
        // mediante una funzione accessoria
        // salvo il valore del puntatore al nodo
3       Y  $\leftarrow$  skip_right(X)
4   else
        // se non ho ancora raggiunto il minimo
        // richiamo la funzione sul figlio sinistro
5       d  $\leftarrow$  get_&_delete_min_AVL(X.l, X)
        // bilancio il lato opposto
        // salvo la nuova radice
6       Y  $\leftarrow$  right_balance(X)
        // collego la nuova radice
        // usando una funzione accessoria
7       swap_child(P, X, Y)
        // restituisco il valore del minimo
8   return d

```

Sottolineiamo che la nuova radice può essere generata in entrambi i branch del costrutto `if...else` e pertanto la funzione `swap_child` viene chiamata sempre.

Con questo abbiamo terminato gli algoritmi su AVL.

Tutti gli algoritmi di lettura fanno solo uso del fatto che l'albero è un albero binario di ricerca, poi l'efficienza di tali algoritmi è dovuta a fatto che si tratti di AVL.

L'unica cosa che abbiamo dovuto fare è solo aggiustare l'inserimento e la cancellazione in modo tale che la proprietà di AVL venga preservata.

4.10.15 Considerazioni finali sugli alberi AVL

Si può dimostrare (non lo faremo), andando a sfruttare pochi raffinamenti rispetto a quello che abbiamo visto essere il bound sull'altezza di un AVL, che l'altezza di un albero AVL, T , è approssimativamente:

$$h(T) \approx 1.44 \log_2(n + 2) - 0.328 \quad (4.12)$$

Ricordiamo, invece, che la formula per l'altezza di un albero pieno, T , è:

$$h(T) = \lceil \log_2(n + 1) - 1 \rceil \quad (4.13)$$

Quello che è importante è che nel caso di alberi pieni $h(T)$ è praticamente il logaritmo di base di 2 di n mentre nel caso di alberi AVL il valore del logaritmo va moltiplicato per un fattore moltiplicativo di 1.44, cosa che sottolinea come negli alberi AVL l'altezza per numero di nodi sia leggermente maggiore, il che rende gli algoritmi di lettura leggermente meno efficienti, visto che sono lineari rispetto all'altezza dell'albero.

Paghiamo però questa perdita di efficienza, che per altro a livello asintotico è irrilevante, guadagnando una maggiore dinamicità della struttura, in quanto come abbiamo visto sia gli inserimenti che le cancellazioni in un albero AVL costano logaritmico.

Lo stesso identico ragionamento è applicabile anche considerando gli alberi perfettamente bilanciati.

Quindi gli algoritmi in un albero AVL sono un po' più costosi, a livello di costante moltiplicativa, rispetto agli alberi pieni o perfettamente bilanciati, ma questo costo è il meglio che possiamo fare visto che gli alberi perfettamente bilanciati non sono gestibili (inserimenti e cancellazioni) in tempo logaritmico.

Questo ragionamento serve a sottolineare che a volte vale la pena anche pagare, dal punto di vista della costante moltiplicativa, un po' di più rispetto agli alberi AVL perché per avere un valore, tra virgolette, così basso siamo costretti a bilanciare spesso.

Facilmente ci troveremo nella situazione in cui viene violato il vincolo sulle altezze e se facilmente ci si trova in tale situazione, ovviamente, sarà necessario bilanciare.

Quindi per avere alberi così bassi siamo costretti a pagare molto frequentemente il costo di bilanciamenti.

A volte, soprattutto se l'insieme di dati da gestire è estremamente dinamico, quindi avvengono tante cancellazioni e tanti inserimenti, potrebbe convenire pagare una costante moltiplicativa maggiore, avendo quindi alberi non così bassi e permettersi di avere funzioni di inserimento e cancellazione un po' più efficienti.

Ripetiamo che il ragionamento che stiamo effettuando è sempre a livello di costante moltiplicativa. Dal punto di vista asintotico non c'è una effettiva differenza.

4.11 Alberi Red Black

Vedremo ora una famiglia di alberi, detti alberi red black, RB, che fanno esattamente questo.

Tale famiglia di alberi rilassa il vincolo che caratterizza gli alberi AVL, in modo tale da renderlo più facilmente soddisfacibile, al costo di avere un 2 come costante moltiplicativa, ma le funzioni di inserimento e cancellazione sono mediamente più efficienti.

Quindi quando l'insieme di dati da gestire è particolarmente dinamico può convenire usare questa struttura dati.

Vediamo adesso qual è l'idea degli alberi red black.

Abbiamo già visto più volte che gli alberi pieni per quanto buoni non siano flessibili, tanto da non coprire neanche tutti i numeri.

Una prima possibilità è quella degli alberi perfettamente bilanciati che però ha i suoi contro e quindi non è una strada percorribile; un'altra possibilità invece è quella di dire: "io so che gli alberi pieni sono bassi e allo stesso tempo sono rigidi; come faccio a rilassare la rigidità senza far esplodere l'altezza?"

L'idea in un certo senso è di prendere l'albero pieno, sezionarlo livello per livello, ed interporre tra due livelli consecutivi dell'albero pieno un qualcosa che pieno non è detto che sia.

Quello che andremo a fare quando andremo a definire gli alberi red black è praticamente vedere un albero pieno sezionato nelle sue componenti e questo albero pieno sarà visibile all'interno dell'albero red black come strati dell'albero red black.

Tutti i livelli neri, cioè i nodi neri all'interno di un albero red black (l'albero red black, lo vedremo, ha nodi colorati rosso e nero) saranno corrispondenti ai nodi di un albero pieno. Se tra nodi neri possiamo interporre al più un nodo rosso quello che succede è che possiamo al più distanziare due livelli neri di uno, il che significa che un albero RB non può esplodere in altezza più del doppio dell'albero pieno. Ecco spiegato il motivo del perché 2 come costante moltiplicativa.

Quindi, abbiamo un albero il doppio più alto di quello pieno, che però è logaritmico, e allo stesso tempo abbiamo la flessibilità di aver disaccoppiato l'albero pieno, che è estremamente rigido, in qualcosa che ci permette di avere della flessibilità tanto da poter rappresentare insiemi di dimensioni arbitraria.

Quindi l'idea è semplicemente di vedere all'interno di un albero un po' più alto un albero pieno; l'altezza è vincolata dal fatto che non stiamo separando arbitrariamente i livelli dell'albero pieno, al più li separiamo di uno e quindi conseguentemente l'altezza rimane comunque logaritmica nel numero di nodi ma con una costante moltiplicativa maggiore.

Quindi le operazioni di lettura, vista una maggiore altezza, saranno meno efficienti; ma visto che i vincoli di struttura saranno più facilmente soddisfacenti andremo ad incappare meno spesso nel caso in cui si deve bilanciare e quindi mediamente le funzioni di inserimento e cancellazione pesano meno.

Nella pratica gli alberi RB sono quelli più utilizzati.

4.11.1 Caratteristiche degli alberi RB

Vediamo cos'è un albero red black.

1. Come per gli AVL la proprietà richiede che l'albero sia un BST, cioè un albero binario di ricerca.

In effetti, così come per gli AVL, le proprietà accessorie sulla struttura non fanno utilizzo del fatto che i dati abbiano una certa organizzazione. Però sono stati definiti solo su AVL perché le operazioni di inserimento e cancellazione sono quelle di un BST e così anche per il red black. Nonostante i concetti strutturali non faranno mai utilizzo delle informazioni sui dati, il concetto di red black lo si definisce solo nel caso di alberi binari di ricerca.

- 2a. Gli alberi RB avranno i dati solo sui nodi interni.

Quindi le foglie, a differenza degli AVL o in generale degli alberi binari di ricerca, saranno parte dell'albero ma non conterranno dati. In effetti noi chiameremo le foglie nodi nil; sono in pratica dei nodi reali che rappresentano però un unico nodo pozzo che è come se fosse il puntatore a NULL.

Perché non avere semplicemente il puntatore a NULL?

Si può fare ma complica gli algoritmi. Le foglie verranno accorpate in un unico nodo nil. In pratica anche se l'albero ha diverse foglie nil (chiaramente è un inutile spreco rappresentare in memoria N foglie nil che non hanno nessuna informazione) queste saranno tutti quanti un puntatore allo stesso nodo.

Qual è il vantaggio di questa scelta?

Le foglie dovranno essere per forza nodi neri, conseguentemente avere in foglia un nodo nero impone certi vincoli; non avere un nodo fisico forzerebbe gli algoritmi ogni volta ad effettuare diversi controlli che andrebbero solo a complicare il tutto.

- 2b. Le foglie identiche e nil.

Per noi, visto che ragioniamo in maniera astratta, potranno anche essere considerate come diversi nodi nil (o NULL), ma come accennato, in pratica, si sceglie per economia di spazio di avere un unico nodo nil verso il quale tutte le foglie puntano. Questo nodo viene anche detto NULL RB. Non ci sarà modo (né motivo) di distinguere tra loro le varie foglie.

Sottolineiamo che si tratta solo di una soluzione tecnica con lo scopo di semplificare. Non è quindi una proprietà imprescindibile.

Come proprietà 3 avremo effettivamente 4 diverse proprietà che andranno a descrivere i vincoli veri e propri sulla colorazione dei nodi.

- 3a. Tutti i nodi sono colorati di rosso (red) o di nero (black).

Ripetiamo che i nodi neri rappresenteranno la corrispondente parte dell'albero pieno e i nodi rossi sono quelli che rendono la struttura flessibile.

- 3b. Le foglie sono nere.

Negli alberi RB le foglie vengono considerate esplicitamente come nodi nil. Per il punto 2b sono tutte nere, identiche e nil.

Vediamo ora le ultime due proprietà.

Questi vincoli sono quelli caratterizzanti la proprietà red black. Sono fondamentali nel senso che sono esattamente quelli che rendono concreta l'idea dell'albero pieno e embeddato (cioè visto all'interno) come una parte dell'albero red black. Se non avessimo questi due vincoli non avremmo questa idea formalizzata correttamente.

3c. Nodi rossi (che ovviamente sono solo nodi interni, cioè non foglia) non possono avere figli rossi.

Perché questo vincolo è importante?

Prima abbiamo detto che quando andavamo a vedere l'albero pieno all'interno dell'albero RB potevamo al più distanziare i nodi dell'albero pieno interno di uno. Visto che il pieno è la parte nera di un albero RB se ammettessimo di avere un nodo rosso con un figlio rosso staremmo distanziando due livelli neri più di una unità.

Salta completamente il concetto di albero RB.

La parte nera di un albero RB deve essere equivalente ad un albero pieno. Come sappiamo un albero pieno ha la caratteristica che ogni percorso che parte dalla radice e raggiunge le foglie ha esattamente la stessa lunghezza. Vogliamo esattamente questo. Ecco l'ultima proprietà.

3d. Partendo da un qualsiasi nodo interno ogni percorso massimale verso le foglie ha lo stesso numero di nodi neri.

Il numero di nodi neri lungo un percorso massimale a partire da un qualsiasi nodo interno viene anche detto lunghezza nera.

Quindi se consideriamo solo la parte nera di tutti i percorsi massimali che originano da un nodo interno stiamo praticamente identificando un albero pieno.

Sottolineiamo che in effetti dentro un albero RB non c'è un unico albero pieno, ce ne sono tanti. Approfondiremo a breve tale aspetto.

Le ultime due proprietà assicurano che l'iniezione dell'idea dell'albero pieno all'interno di un albero red black sia ben formalizzata.

Poiché tutti i percorsi partendo da qualunque nodo e quindi in particolare partendo dalla radice devono avere la stessa lunghezza nera possiamo definire il concetto di altezza nera dell'albero come la massima lunghezza nera di un percorso massimale.

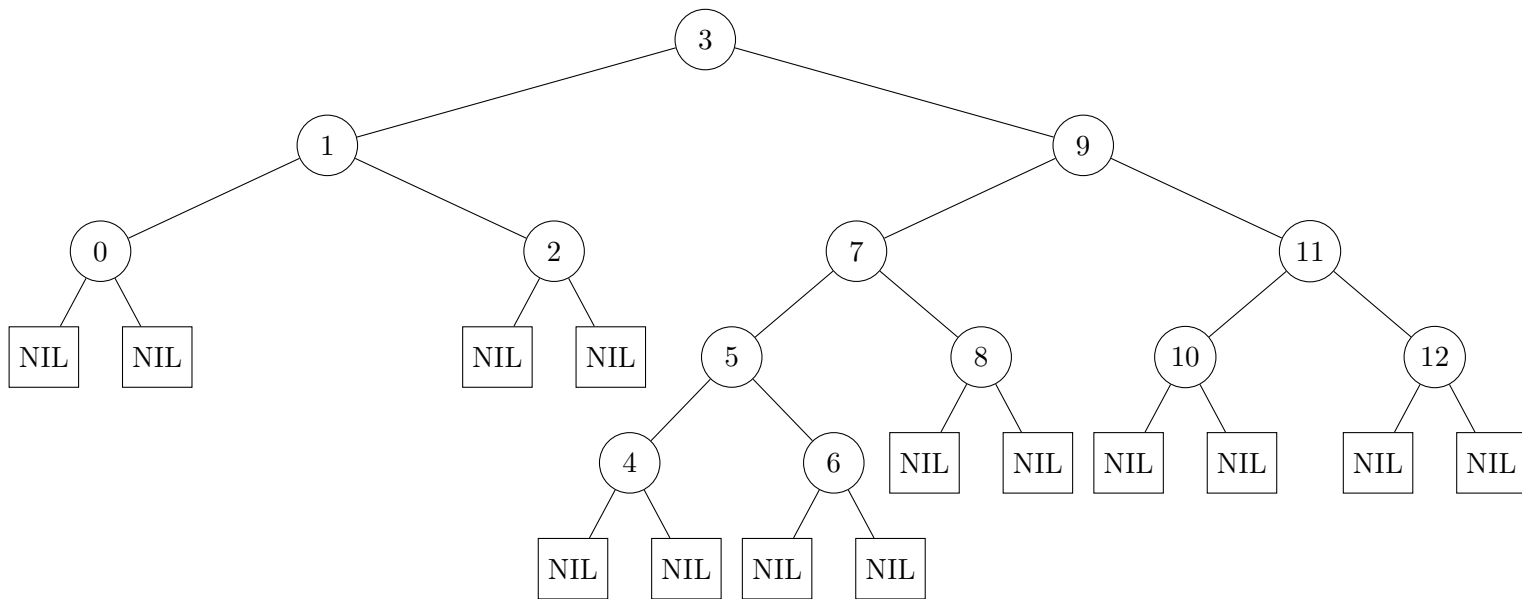
Nella lunghezza nera di un percorso non si conta mai l'origine del percorso. Pertanto il colore della radice è indifferente.

Per questioni di semplicità tecnica ed implementativa spesso si utilizza la sottoclasse degli alberi red black con radice nera.

4.11.2 Colorazione degli Alberi RB

Sottolineiamo che esistono procedure per capire se un albero è colorabile come RB. Non è detto che per un dato albero esista un'unica configurazione RB.

Facciamo un esempio di colorazione.



Coloriamo (se possibile) questo albero binario così da renderlo un albero RB.

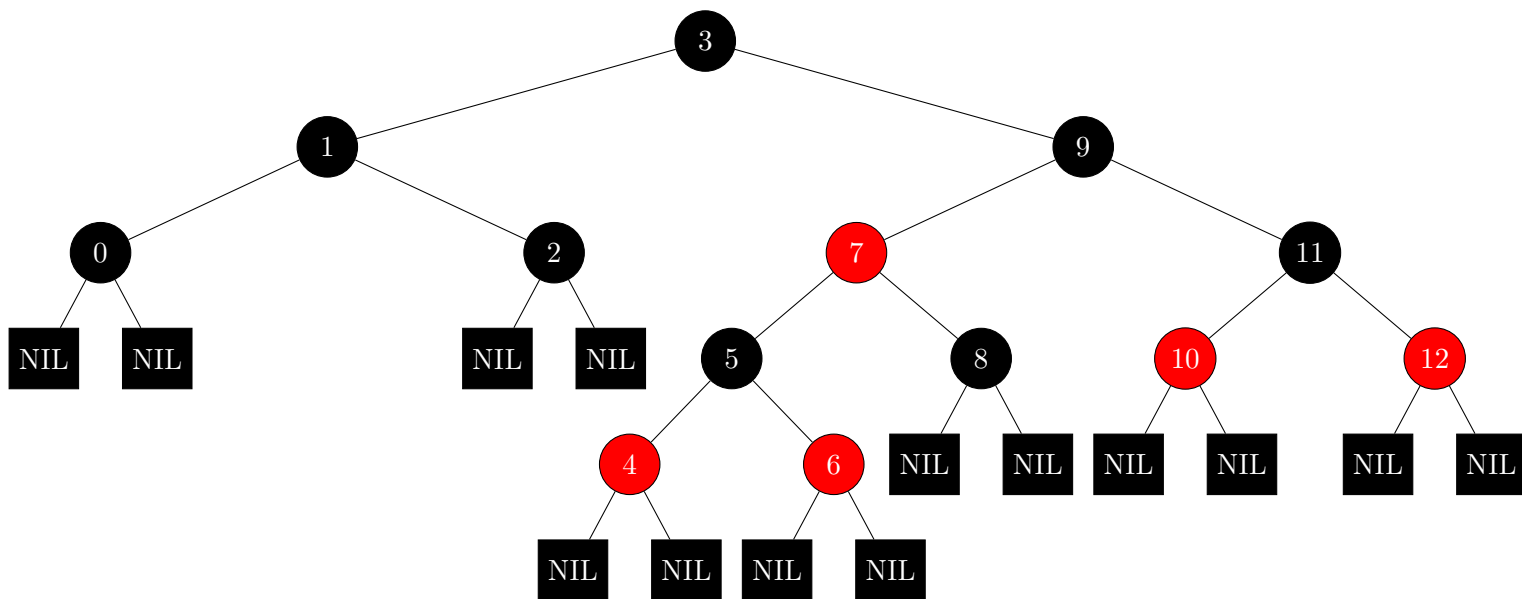


Figura 4.32: Colorazione di un albero RB

4.11.3 Proprietà degli alberi RB

Si può dimostrare che ogni albero AVL è red black colorable, cioè c'è sempre un modo di colorare un AVL in RB; ma non viceversa.

La classe dei RB è quindi più ampia degli AVL. Quindi sarà più difficile uscire dalla classe stessa, cosa che comporta in media meno bilanciamenti.

In generale in modo intuitivo possiamo dire che la parte bassa di un albero RB deve avere una densità di nodi neri maggiore rispetto alla parte alta dell'albero.

In ogni percorso, in particolare, non possiamo avere un numero di rossi che sia maggiore della metà della lunghezza del percorso stesso, in quanto altrimenti sarebbe necessario avere 2 nodi rossi consecutivi.

Il che significa che la parte più bassa di un albero RB è al più alta la metà della controparte alta dell'albero. Questo crea un vincolo sulle altezze molto stringente che non permette all'albero di degenerare.

A livello intuitivo sono questi i punti di forza di un albero RB, cioè grazie ai nodi neri di essere sufficientemente vicino ad un albero pieno ma, grazie ai nodi rossi di essere abbastanza flessibile da poter gestire insiemi di dati di cardinalità arbitraria e di necessitare in media di pochi bilanciamenti.

Dimostriamo ora che in generale qualunque albero red black non può avere un'altezza superiore al doppio dell'altezza dell'albero pieno che contiene logicamente. Lo faremo per induzione.

Sottolineiamo che sebbene abbiamo più modi per estrarre da un albero red black un albero pieno, qualunque modo andiamo a scegliere l'albero pieno estratto avrà comunque la stessa altezza, pari ad $h_N(T)$.

La notazione $\#Int(T)$ indicherà il numero di nodi interni dell'albero RB T ; $h_N(T)$ invece indicherà l'altezza nera dell'albero RB, T .

Formalmente.

Proposizione 4.11.1 (Numero nodi interni albero RB).

$$\forall T \in RB \left(\#Int(T) \geq 2^{h_N(T)} - 1 \right)$$

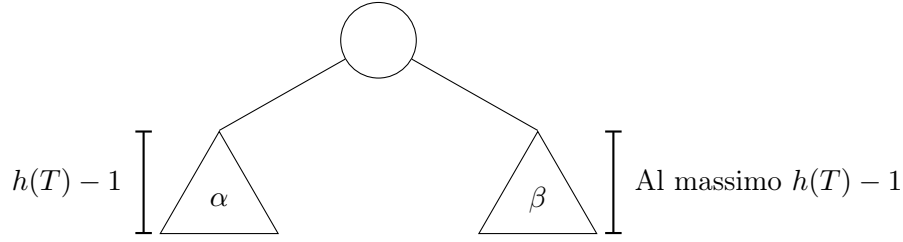
Dimostrazione.

Dimostreremo la proposizione per induzione su $h(T)$.

Il caso base risulta banale. Un albero RB avente altezza 0 chiaramente ha un solo nodo interno, la radice (nera), e le due foglie nere.

$$h(T) = 0 \implies \#Int(T) = 1 = 2^1 - 1 = 2^{h_N(T)} - 1$$

Passiamo al caso induttivo. Consideriamo il seguente albero RB, T , di altezza $h(T) > 0$.



Chiaramente, uno dei due sottoalberi della radice dell'albero T (supponiamo senza ledere generalità α) avrà necessariamente altezza $h(T) - 1$; l'altro sottoalbero può avere anche altezza minore ($\leq h(T) - 1$).

Segue, chiaramente

$$\#Int(T) = 1 + \#Int(\alpha) + \#Int(\beta) \geq 1 + (2^{h_N(\alpha)} - 1) + (2^{h_N(\beta)} - 1) = 2^{h_N(\alpha)} + 2^{h_N(\beta)} - 1$$

Dalle proprietà degli alberi RB abbiamo necessariamente

$$h_N(\alpha) = h_N(\beta) = h_N(T) - 1$$

Concludendo

$$\#Int(T) \geq 2 \cdot 2^{h_N(T) - 1} - 1 = 2^{h_N(T)} - 1$$

□

Una volta dimostrata la proposizione abbiamo effettivamente un limite sull'altezza di un albero RB in funzione del numero di nodi.

Teorema 4.11.1 (Altezza albero RB).

L'altezza di un albero RB è logaritmica nel numero di nodi.

Dimostrazione.

Dalla proposizione appena dimostrata, segue logicamente

$$\#Int(T) \geq 2^{h_N(T)} - 1 \iff \#Int(T) + 1 \geq 2^{h_N(T)}$$

Passando ai logaritmi

$$\log_2(\#Int(T) + 1) \geq h_N(T)$$

Come sappiamo, tra i nodi due neri può esserci al più un nodo rosso. Di conseguenza, considerando anche l'altezza dell'albero segue

$$2 \cdot \log_2 (\#Int(T) + 1) \geq 2 \cdot h_N(T) \leq h(T) \quad (4.14)$$

□

Abbiamo, pertanto, dimostrato che l'altezza di un albero RB è logaritmica nel numero di nodi; come anticipato. Quindi, gli alberi red black sono alberi bilanciati.

4.11.4 Inserimento in un albero RB

Ora vediamo come si può effettivamente progettare una serie di algoritmi di inserimento e cancellazione e in particolare vediamo quali sono i problemi che ovviamente possono essere incontrati a valle dell'inserimento e come risolverli.

Anche per i red black sarà fondamentale l'idea delle rotazioni perché il problema è lo stesso. A valle di un inserimento si creerà una parte più profonda di un'altra, così come succedeva negli AVL, dove l'essere più profondo in un RB è legato al numero di rossi.

Ricordiamo che abbiamo 4 criteri fondamentali per un albero RB:

1. Ogni nodo deve essere colorato rosso o nero.
2. Le foglie sono nere, ma le foglie sono quelle nil che non contengono dati.
Quindi il nodo inserito è una foglia dell'albero dei dati non dell'albero red e black vero e proprio.
3. Ogni rosso non può avere figli se non neri.
4. La lunghezza nera di ogni percorso che si diparte da un qualunque nodo dell'albero e raggiunge le foglie deve essere la stessa.

Dobbiamo quindi avere un concetto di altezza nera definita uniformemente rispetto a percorsi.

Quello che vedremo è che tra questi vincoli, in inserimento, quello che andremo a violare sarà quello relativo al rosso (n. 3).

Vediamo il motivo.

Un'analisi dei criteri semplicemente fa evincere il seguente ragionamento.

Andare per esempio a violare la lunghezza nera di un percorso è estremamente deleterio, rende la struttura praticamente ingestibile. Quindi andare ad inserire un nodo nuovo colorandolo di nero automaticamente sempre viola la proprietà sulla lunghezza nera in quanto quel percorso avendo, a seguito dell'inserimento, un altro nero, avrà un nero in più di tutti gli altri.

Ciò comporterebbe che ogni volta che inseriamo un nodo nero c'è una violazione e questo fa vanificare completamente l'idea di avere una struttura più flessibile per minimizzare il numero di correzioni. Quindi non ha alcun senso inserire un nodo nero.

Ma se non si vuole violare la proprietà che ogni nodo è colorato, necessariamente il nodo da inserire deve essere rosso.

In prima battuta si potrebbe pensare di inserire un nodo con un colore diverso sia da rosso che da nero. In effetti questa tecnica è utilizzata nella cancellazione. Nel caso dell'inserimento però non porta alcun vantaggio.

Di conseguenza andremo ad inserire un nodo rosso.

Questo ci porta automaticamente ad avere potenzialmente, non sempre, una violazione della proprietà che dice che un nodo rosso non può avere figli rossi.

Perché?

Perché nel momento in cui andiamo a creare una foglia di dati, (quando parliamo di dati intendiamo come foglia sempre un nodo dell'albero dei dati che è foglia, ma nell'albero red black non è mai foglia perché ci sono i nodi neri nil), quando andiamo a creare questo nuovo nodo e lo coloriamo di rosso, tale nodo non ha problemi rispetto ai figli perché essendo una foglia nell'albero dei dati sarà genitore delle foglie nere nil. Avendo inserito il rosso, la lunghezza nera del percorso in cui quel nodo fa parte è rimasta invariata (notiamo che essendo una foglia c'è un unico percorso che si diparte da un qualunque nodo dell'albero e che raggiunge tale foglia).

Quello che può succedere è che il nodo inserito sia figlio di un nodo rosso. Questo ovviamente può capitare perché avendo inserito un nodo in foglia nell'albero dei dati e sapendo che le foglie di un RB sono sempre nere non abbiamo alcun vincolo sul colore del precedente genitore delle foglie nil di un RB e pertanto potrebbe essere un nodo rosso, cosa che comporterebbe una violazione del vincolo relativo al rosso (n. 3).

Quello che faremo è analizzare una casistica in cui cercheremo di capire dov'è il problema, che tipo di problema è, e come affrontarlo.

In questo caso la casistica sarà tripartita, (non è come negli AVL in cui ci sono 2 casi). Vedremo che addirittura in cancellazione ci saranno ben 4 casi.

In soldoni quello che osserveremo è sempre lo stesso identico criterio che abbiamo già sfruttato negli AVL. Noteremo che a valle dell'inserimento c'è una parte troppo più alta di quello necessario e quello che andremo a fare è effettuare delle rotazioni per uniformare l'altezza, fino a un certo punto, rendendo l'albero quanto più bilanciato possibile, fino a soddisfare i criteri. E anche qui vedremo che con una o due rotazioni, più potenziali ricolorazioni dei nodi, riusciremo sempre a risolvere il problema.

A differenza degli AVL, nei quali un problema sorto a seguito di un inserimento una volta risolto localmente lo era anche definitivamente, quello che accadrà con i RB è che un problema a valle di un inserimento potrebbe semplicemente essere spostato più in alto.

Il problema che sposteremo più in alto sarà sempre e soltanto la colorazione rossa di un nodo figlio di rosso. Questo è l'unico vincolo che potremo violare e gli altri rimarranno sempre soddisfatti per ogni trasformazione che faremo.

Sottolineiamo che spostare in alto il problema è un modo per semplificare la situazione perché quando arriva in radice il problema dei due rossi scompare visto che la radice non ha padri. Quindi spostare il problema in alto è effettuare un lavoro nella direzione della soluzione.

Ricordiamo che, come al solito, il ragionamento di correzione si fa in post ordine; cioè prima si inserisce e al ritorno dall'inserimento, quindi man mano che torniamo su e svuotiamo lo stack delle chiamate ricorsive, andiamo a controllare se c'è o meno un problema. Ci fermiamo al primo problema che incontriamo, correggiamo quel problema e poi proseguiamo, perché, come detto, potremmo trovarci nella situazione di correggere ancora ulteriori problemi più in alto.

Quindi, quello che faremo è sempre assumere che nel momento in cui abbiamo un problema da risolvere, la struttura sottostante sia fatta bene.

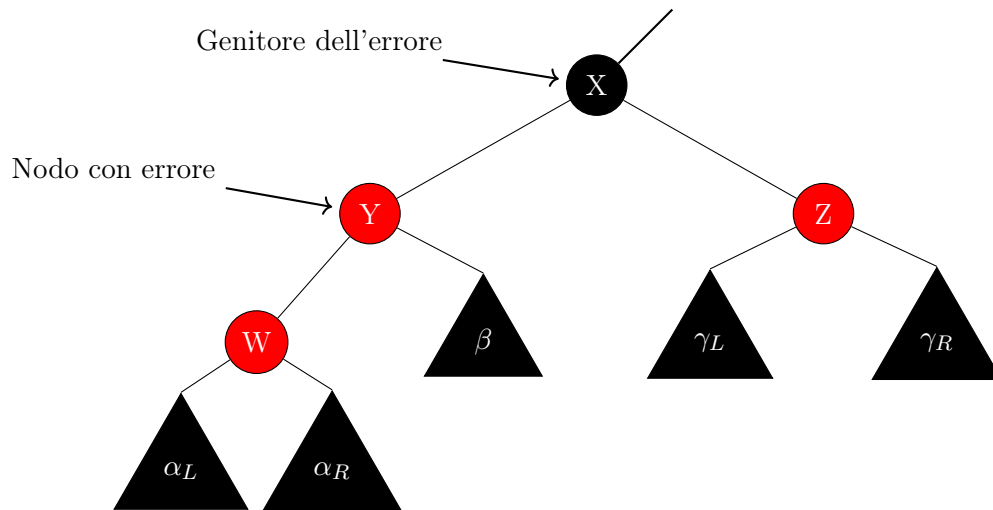
Se cominciasse a correggere dall'alto potremmo trovare diversi nodi che violano il vincolo, cosa che renderebbe tutto molto più complesso, soprattutto a livello di complessità asintotica, perché nel momento in cui risolveremo qualcosa, poi potremmo trovarci a dover risalire e a dover risolvere, e così via. Quando invece facendolo semplicemente a ritroso lo facciamo una sola volta.

Ripetiamo. L'unica possibile violazione è avere due rossi consecutivi.

Tutto il ragionamento lo faremo centrandonoci sul genitore del nodo rosso che ha un problema di rosso con figlio. Non lavoreremo come con gli AVL andando a puntare sul nodo che aveva le violazioni tra figli. Andiamo sul genitore di quello che ha la violazione della proprietà.

Vediamo qual è la casistica. Ovviamente, come per gli AVL, ogni caso ha uno speculare. Ne faremo vedere solo uno dei due.

4.11.5 Inserimento in un albero RB: caso 1A



Correggiamo il problema facendo una ricolorazione del genitore dell'errore e dei suoi figli così da propagare il problema verso l'alto.

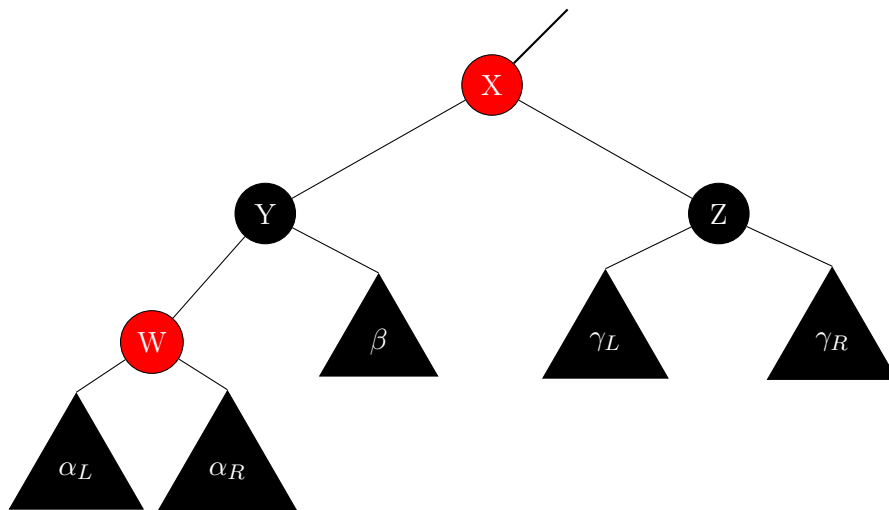
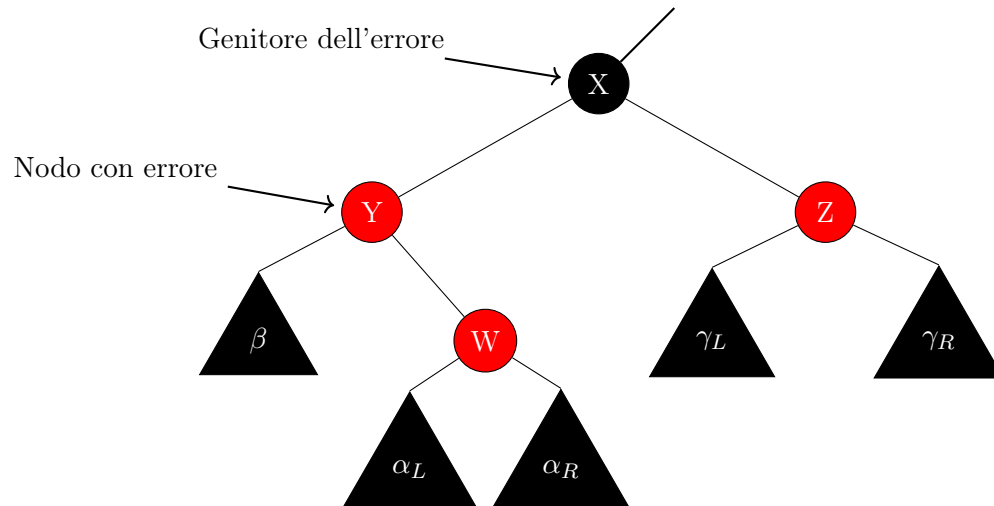


Figura 4.33: Inserimento in albero RB (Caso 1A)

4.11.6 Inserimento in un albero RB: caso 1B

Vediamo adesso un caso d'inserimento analogo al Caso 1A, lo chiameremo 1B. L'unica variazione saranno i figli di Y, che si scambieranno di posto, W sarà figlio destro e β sarà figlio sinistro.



Correggiamo il problema facendo una ricolorazione del genitore dell'errore e dei suoi figli così da propagare il problema verso l'alto.

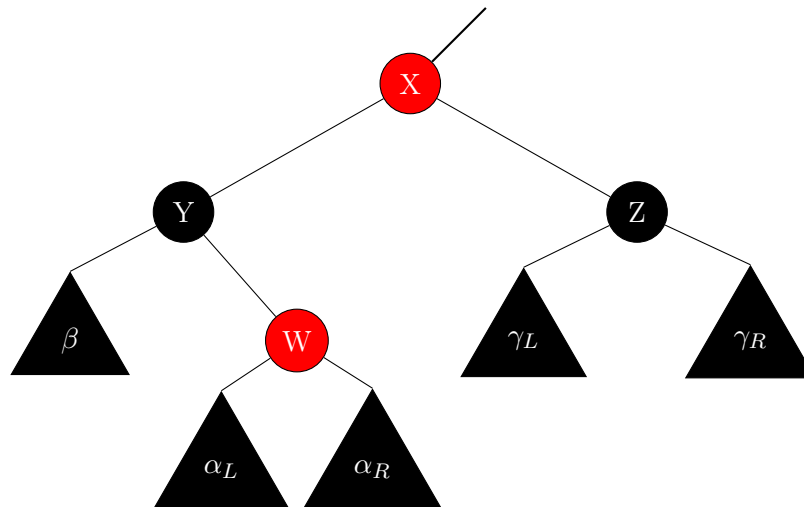
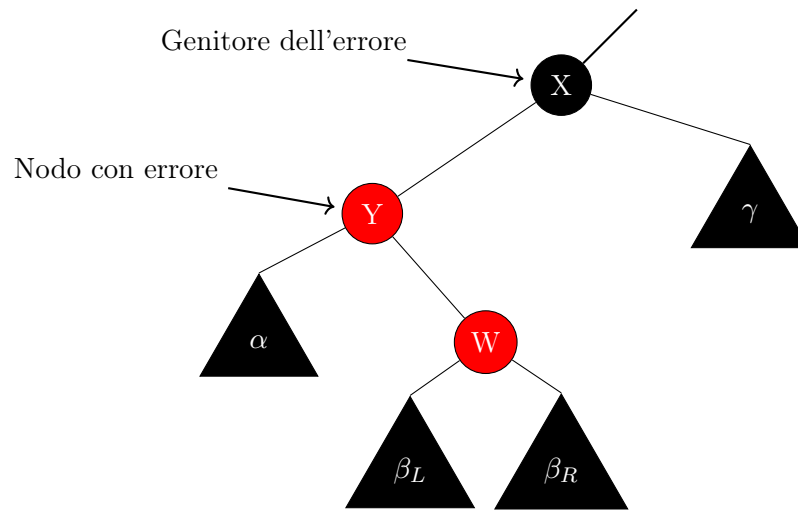


Figura 4.34: Inserimento in albero RB (Caso 1B)

Si è deciso di ricolorare per entrambi i casi (Caso 1A e 1B) il genitore del problema di rosso e i suoi figli di nero, così da mantenere invariata la lunghezza nera, in quanto se avessimo deciso di non ricolorare il colore del genitore del problema ma di ricolorarne i figli, ci saremmo ritrovati con una lunghezza nera pari alla precedente più uno.

Se invece avessimo ricolorato di nero soltanto uno dei due figli ci saremmo trovati nel problema che la lunghezza nera non sarebbe stata uguale in tutti i percorsi, poiché se per esempio avessimo colorato solo il figlio sinistro, avremmo avuto una lunghezza a sinistra più grande di uno rispetto a quella di destra.

4.11.7 Inserimento in un albero RB: caso 2



Proviamo a correggere il problema facendo una rotazione da destra verso sinistra sul nodo con errore Y.

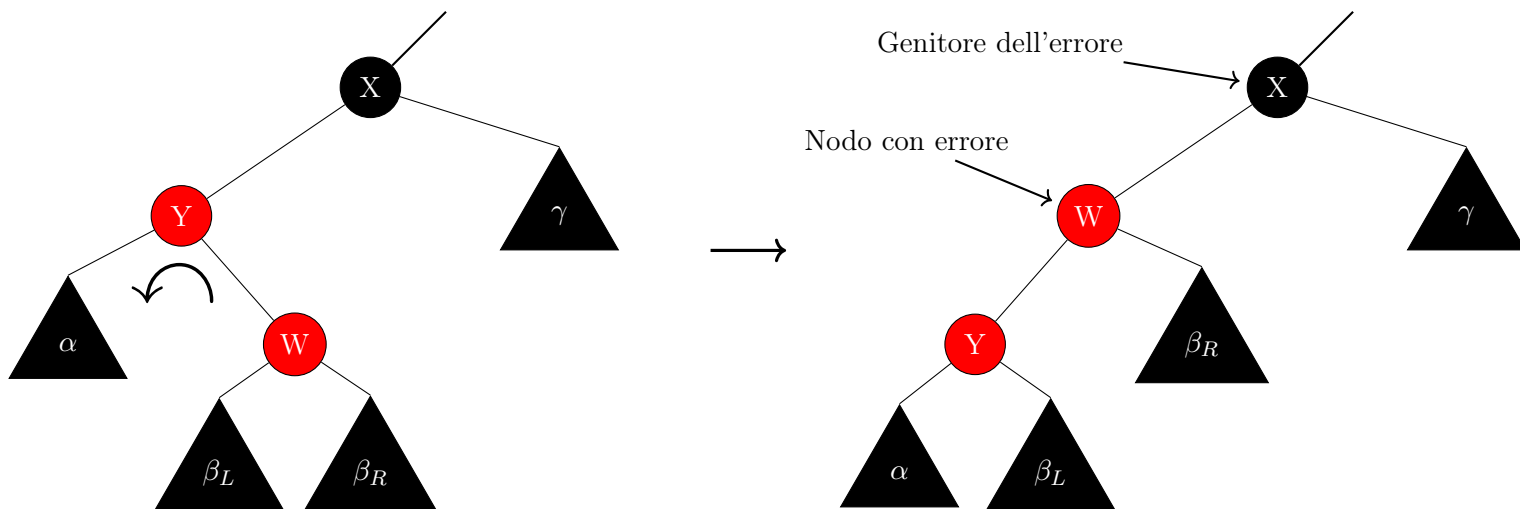


Figura 4.35: Inserimento in albero RB (Caso 2)

A seguito di questa rotazione, ci siamo trovati nella disposizione dei nodi dell'albero del caso 3.

4.11.8 Inserimento in un albero RB: caso 3

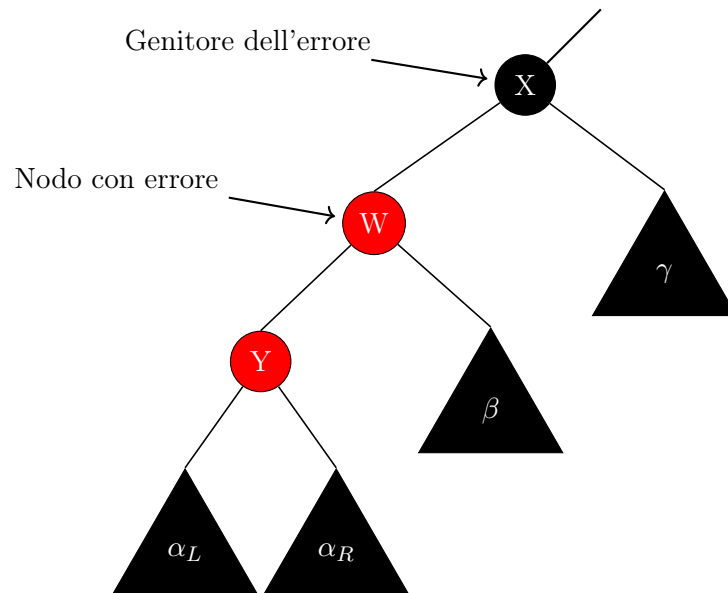
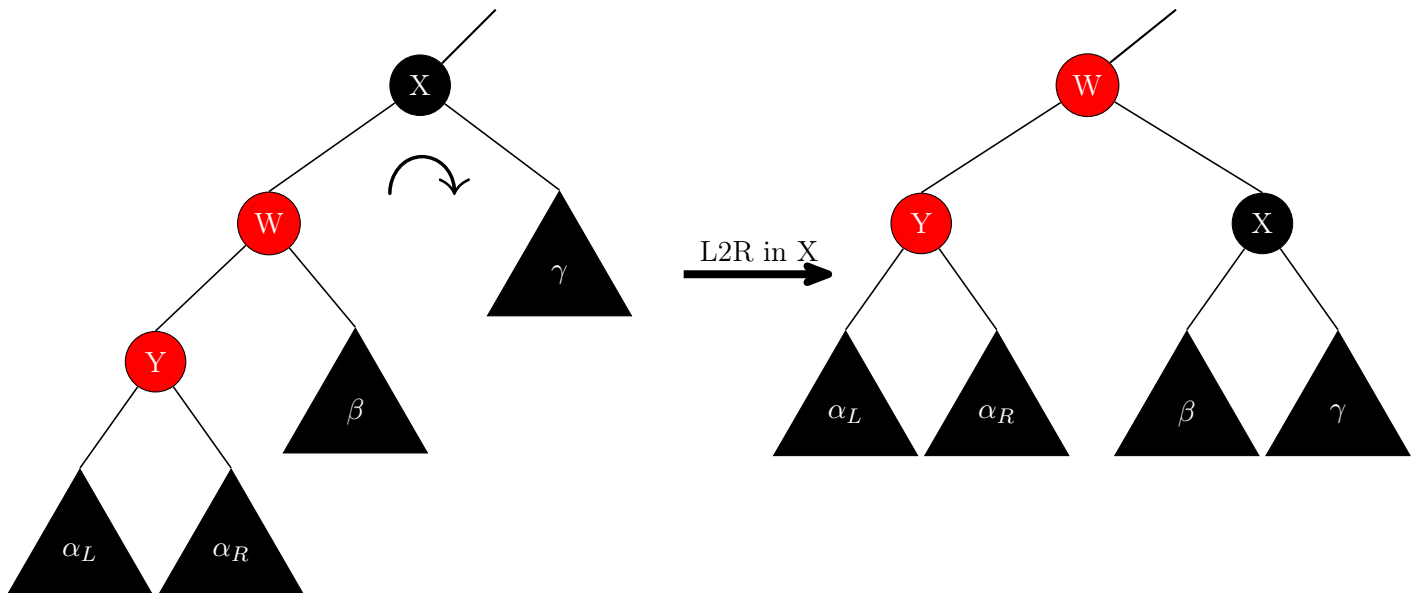
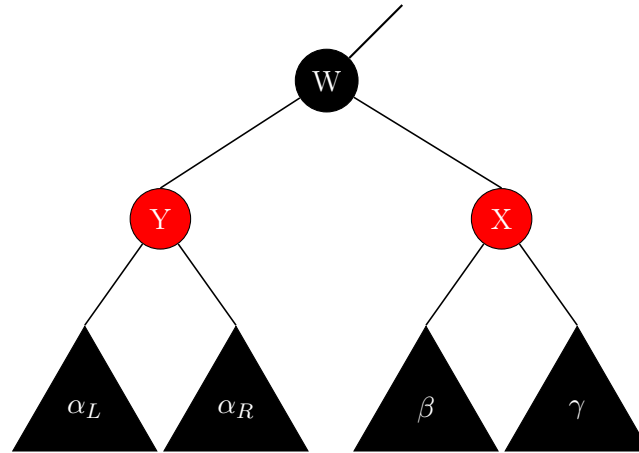


Figura 4.36: Inserimento in albero RB (Caso 3)

Correggiamo il problema facendo una rotazione da sinistra a destra sul genitore dell'errore X e applicando l'adeguata ricolorazione.



Applichiamo la colorazione.



La ricolorazione è stata applicata secondo il seguente criterio: la nuova radice del sottoalbero W , è stata ricolorata dello stesso colore della vecchia radice X , e poiché X era nero, per rimanere invariata la lunghezza nera, il nodo X è stato ricolorato rosso. Perché altrimenti la lunghezza nera a destra sarebbe stata maggiore della lunghezza di sinistra.

4.11.9 Algoritmo: inserimento in un albero RB

Vediamo adesso gli algoritmi di inserimento.

Gli algoritmi sono simili a quelli visti per gli AVL con una sola vera differenza, che poiché la casistica è un po' più ampia, un po' più complessa, per identificare la casistica si utilizza una funzione apposita.

In inserimento facciamo esattamente la stessa cosa che abbiamo fatto in AVL. Inseriamo, al ritorno chiamiamo l'opportuna funzione di bilanciamento.

Attenzione. Poiché in un albero RB abbiamo i nodi nil espliciti, per capire di essere arrivati in foglia valuteremo l'uguaglianza con il nodo foglia unico che indicheremo con `NIL_RB`, non è più con $\perp$, cioè col puntatore a `NULL`. Il significato concettuale è però il medesimo.

Prima di vedere l'algoritmo vediamo com'è rappresentato in memoria un nodo di un albero RB.

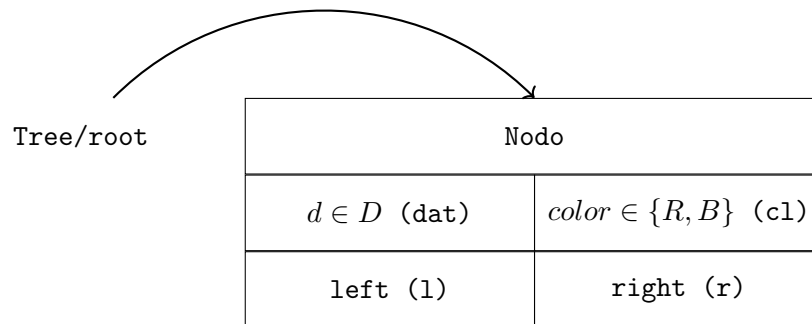


Figura 4.37: Nodo di un albero RB

Ecco l'algoritmo.

Algoritmo 57: Inserimento in un albero RB

```

Signature insert_RB:  $RB(D) \times D \rightarrow RB(D)$ 
function insert_RB(X, d)
    // se siamo arrivati in foglia
    // dove va inserito il dato in input
1  if (X = NIL_RB) then
    // costruisco ed inserisco il nuovo nodo
    // non è necessario indicare il colore
    // visto che è sempre rosso
2  X  $\leftarrow$  build_node_RB(d)
3  else
    // se non abbiamo ancora raggiunto il punto dove inserire
    // se il dato del nodo corrente è maggiore del dato da inserire
4  if (X.dat > d) then
    // richiamo la funzione nel sottoalbero sinistro
5  X.l  $\leftarrow$  insert_RB(X.l, d)
    // bilancio se necessario
    // mediante una funzione accessoria
6  X  $\leftarrow$  L_ins_balance(X)
    // se il dato del nodo corrente è minore del dato da inserire
7  else if (X.dat < d) then
    // richiamo la funzione nel sottoalbero destro
8  X.r  $\leftarrow$  insert_RB(X.r, d)
    // bilancio se necessario
    // mediante una funzione accessoria
9  X  $\leftarrow$  R_ins_balance(X)
    // restituisco il nodo corrente
10 return X

```

Vediamo ora la funzione che serve a bilanciare l'albero red black a seguito di un inserimento a sinistra. La funzione per il caso speculare destro è essenzialmente identica ovviamente.

Algoritmo 58: Bilanciamento dell'inserimento a sinistra in un albero RB

```

Signature L_ins_balance:  $RB(D) \rightarrow RB(D)$ 
function L_ins_balance(X)
    // se il figlio sinistro è diverso dal nodo foglia nero
1   if (X.l != NIL_RB) then
        // valuto le casistiche
        // mediante una funzione accessoria
2       switch (L_ins_violation(X)) do
            // in base alla casistica
            // bilancio con opportune funzioni accessorie
3           case (1) do
4               X ← L_ins_balance_RB_1(X)
5           case (2) do
6               X ← L_ins_balance_RB_2(X)
7           case (3) do
8               X ← L_ins_balance_RB_3(X)
        // restituisco il nodo corrente
9   return X

```

Sottolineiamo che nel caso in cui il nodo in input ha il figlio sinistro NIL_RB non è necessario bilanciare in quanto per le proprietà degli alberi RB il nodo in questione avrà entrambi i figli NIL_RB e non potrà essere quindi radice di un albero RB sbilanciato.

Inoltre per capire in che casistica ci si trovi è necessario conoscere il colore dei figli del suddetto nodo, cosa non possibile se un nodo ha come figlio un nodo nil, visto che non è possibile dereferenziare un puntatore a NULL.

Vediamo ora la funzione che serve a riconoscere in quale particolare casistica ci si trovi.

Algoritmo 59: Funzione per le violazioni possibili a seguito dell'inserimento a sinistra in un albero RB

Signature $L_ins_violation: RB(D) \rightarrow \{0, 1, 2, 3\}$
function $L_ins_violation(X)$

```

  // setto i valori iniziali
  // v denota il tipo di violazione, 0 assenza di violazione
  // l è il figlio sinistro del nodo in input, r il destro
1  (v, l, r) ← (0, X.l, X.r)
  // se il figlio sinistro è rosso
2  if (l.cl = R) then
    // se il figlio destro è rosso
3    if (r.cl = R) then
      // possiamo trovarci nel caso 1
      // se uno dei figli del figlio sinistro è rosso
4      if ((l.l.cl = R) OR (l.r.cl = R)) then
        // ci troviamo nella casistica di violazione n. 1
5        v ← 1
      else
6        // se il figlio destro è nero possiamo trovarci nel caso 2 o 3
        // se il figlio destro del figlio sinistro è rosso
7        if (l.r.cl = R) then
          // ci troviamo nella casistica di violazione n. 2
8          v ← 2
        // se il figlio sinistro del figlio sinistro è rosso
9        else if (l.l.cl = R) then
          // ci troviamo nella casistica di violazione n. 3
10         v ← 3
  // restituisco il valore che denota la violazione
11 return v

```

Ricordiamo che in questo caso il nodo X in input è il padre di un nodo nel quale possibilmente è presente una violazione.

4.11.10 Cancellazione in un albero RB

Vediamo adesso la cancellazione di un nodo da un albero red black.

Con la cancellazione l'idea è leggermente diversa visto che il dato da cancellare potrebbe trovarsi indifferentemente in un nodo rosso o nero.

Immaginiamo di andare a cancellare un dato contenuto in un nodo rosso.

Quello che possiamo facilmente osservare è che ciò non comporta alcun problema.

Perché?

Perché aver cancellato un rosso al più può aver fatto avvicinare due nodi neri visto che si può solo trovare tra due nodi neri. Inoltre, avendo cancellato un rosso non abbiamo cambiato la lunghezza nera di qualsiasi percorso, quindi la proprietà relativa all'altezza nera è ancora soddisfatta.

La proprietà della colorazione dei nodi è chiaramente ancora valida.

Le foglie nil sono rimaste invariate.

Quindi se bisogna cancellare un dato che si trovi in un nodo rosso non abbiamo nulla da fare.

Immaginiamo adesso di andare a cancellare un nodo nero. In questo caso invece nascono dei problemi.

Cancellando un nodo nero potremmo avvicinare due rossi. Ma soprattutto andremo a ridurre l'altezza nera.

Quello che faremo è la seguente cosa.

Nel momento in cui cancelliamo un nodo nero portiamo il colore del nodo al padre.

Se il genitore era rosso abbiamo risolto il problema perché essendo rosso non entrava in gioco a livello di altezza nera. Per giunta, visto che era rosso, potenzialmente poteva creare problemi.

Inoltre, i percorsi che passavano dal nodo eliminato ora sono collegati ad esso, di conseguenza abbiamo semplicemente spostato il colore nero da una parte all'altra e risolto il problema.

Se però il padre era nero, la situazione diventa più complicata.

Quello che si fa è di scegliere di violare la proprietà di bicolorazione dei nodi piuttosto che le altre, cioè del doppio rosso o della lunghezza nera.

Introduciamo temporaneamente un nodo chiamato double black, doppio nero, con la caratteristica che a livello di conteggio del percorso nero vale 2.

Ovviamente è una condizione temporanea durante la correzione che serve a tener traccia del fatto che necessariamente va introdotto un nuovo livello.

Quindi temporaneamente il nodo double black, DB, si prende carico di tenere memoria del fatto che prima c'erano due livelli; prima o poi questa informazione va tradotta un'altra volta in due livelli.

Un livello è stato perso in quanto abbiamo cancellato un nodo nero e non possiamo dimenticarne in quanto altrimenti si avrebbe una violazione a livello di altezza nera. Dobbiamo però vedere come correggere tale situazione.

A differenza dell'inserimento dove il doppio rosso caratterizzava una parte dell'albero RB troppo profonda rispetto all'altra, ora c'è una parte troppo bassa dove c'è un nodo doppio nero. Quindi, intuitivamente, andremo a caricare il lato basso spostando informazione dal lato alto, per distanziare i livelli ed eliminare la situazione temporanea del nodo double black.

Come nel caso dell'inserimento sarà possibile risolvere il problema localmente oppure spostare il problema, cioè

il nodo double black, in alto.

Anche in questo caso spostare il problema in alto è lavorare nel verso della soluzione in quanto se il nodo DB arriva in radice il problema viene risolto.

Ricordiamo infatti che nel caso del calcolo dell'altezza nera il nodo radice dell'albero non viene considerato, pertanto sarà possibile colorare il nodo radice double black in nodo nero semplice e risolvere quindi il problema.

Questa è l'idea ad altissimo livello per la correzione a valle della cancellazione in un albero red black.

Anche qui ci sarà una casistica, questa volta divisa in 4 casi.

Come sempre, assumiamo che la cancellazione venga effettuata a sinistra.

Come sempre la correzione avviene in post ordine.

Attenzione che il nodo che andremo a cancellare non è detto sia quello che contiene il dato che vogliamo cancellare; quello è uno dei tre casi, quando il nodo contenente il dato che vogliamo cancellare ha al più un figlio. Nel caso in cui quel nodo abbia entrambi i figli quello che andremo a cancellare è il minimo del sottoalbero a destra (potremmo scegliere dualmente ed in modo equivalente il massimo del sottoalbero sinistro) che non è il nodo che contiene il dato, come già visto per le cancellazioni in un albero binario di ricerca.

Quindi il colore del nodo che andiamo a cancellare non è detto neanche sia il colore del nodo contenente il dato.

Supponiamo quindi che la cancellazione sia avvenuta a sinistra, di aver già applicato l'algoritmo che stiamo andando a vedere, e che in qualche modo l'errore sia stato portato a ritroso nel nodo evidenziato. Quindi gli alberi sottostanti il nodo saranno corretti, come anche ovviamente tutta la parte di albero che non è interessata dalla cancellazione.

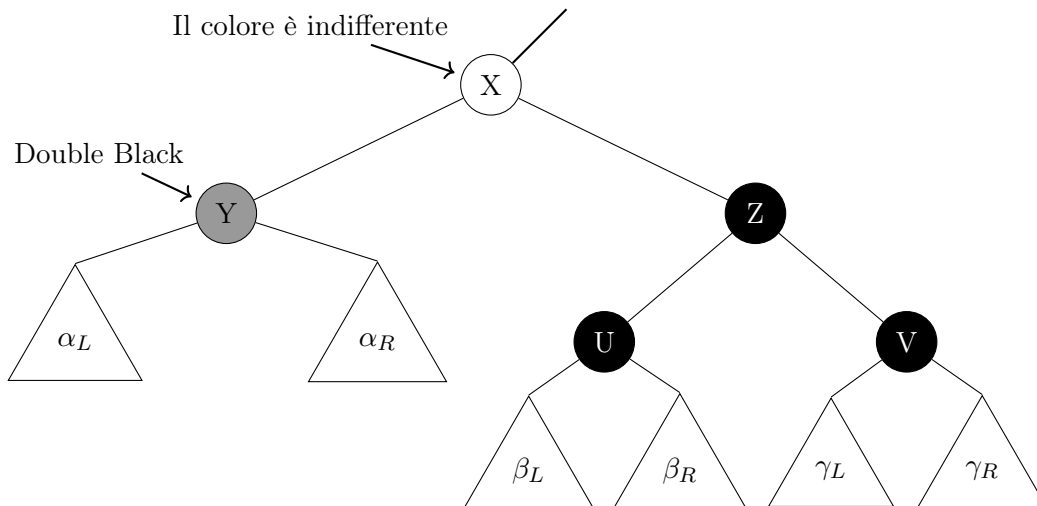
Vediamo che in linea generale le correzioni fanno capo agli stessi identici ragionamenti visti per gli alberi AVL e si basano quindi sullo spostamento delle parti di maggior peso dell'albero verso parti meno pesanti.

Quindi, a parte casi che si risolvono semplicemente ricolorando i nodi, come al solito se il peso maggiore è esterno basterà una singola rotazione, se il peso maggiore è interno ne serviranno due.

Oltre a questo, inoltre, potrà essere necessario ricolorare.

4.11.11 Cancellazione in un albero RB: caso 1

Caso 1: Il nodo Y è double black, e suo fratello il nodo Z è nero ed entrambi i suoi figli sono neri.



Correggiamo il problema facendo una ricolorazione del nodo X e dei suoi figli. Distinguiamo i casi in cui X è rosso o nero.

Se X è rosso la correzione sarà:

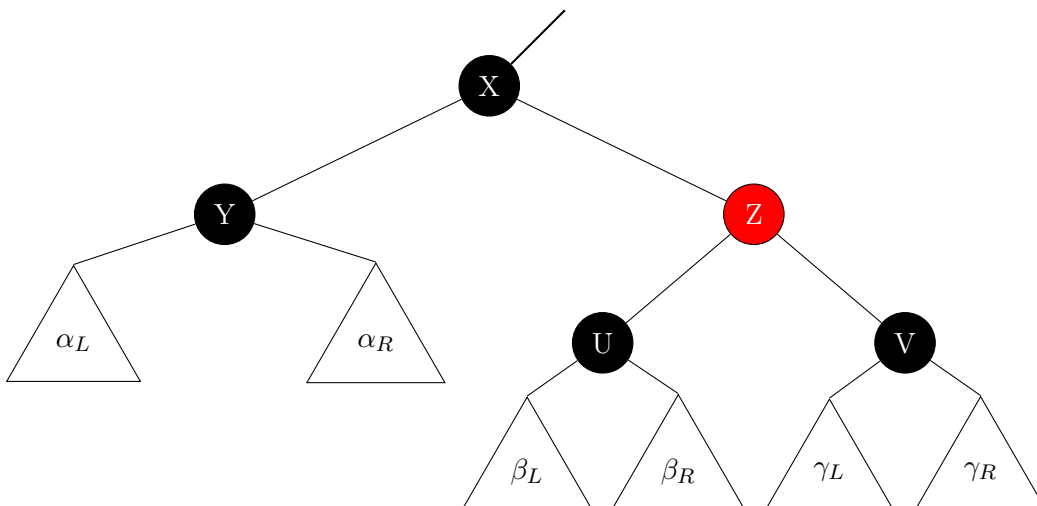
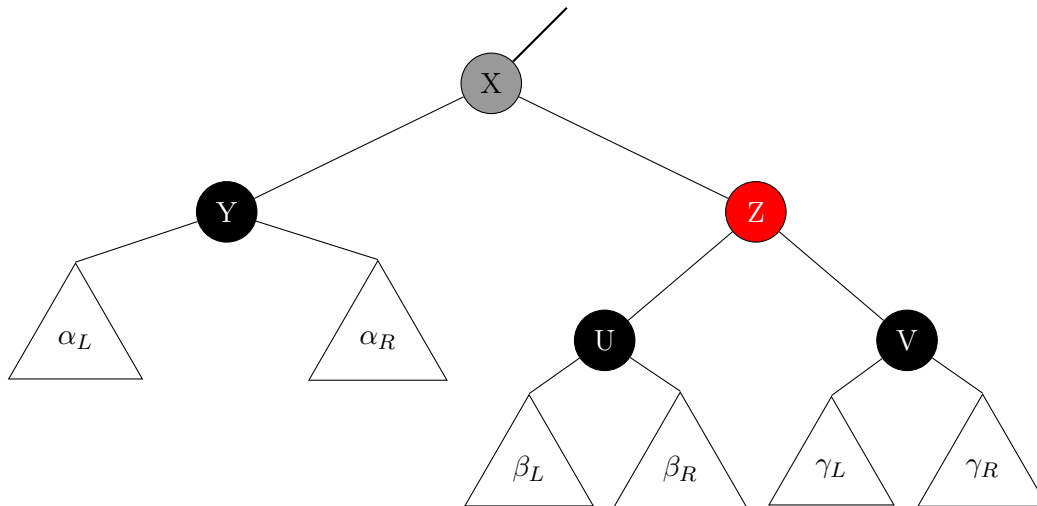


Figura 4.38: Cancellazione in albero RB (Caso 1 con radice del sottoalbero rossa)

Notiamo come in questo caso la ricolorazione effettuata risolva definitivamente il problema.

Vediamo adesso il caso in X è nero:



In questo caso invece la ricolorazione effettuata non risolve definitivamente il problema, ma lo propaga verso l'alto.

Figura 4.39: Cancellazione in albero RB (Caso 1 con radice del sottoalbero nera)

La ricolorazione per entrambi i casi è stata effettuata secondo il seguente criterio:

- Il nodo Y diventa nero.
- Il nodo X varia il suo colore a seconda del suo colore originario, infatti in seguito al passaggio di colore di Y da doppio nero a nero, il genitore X deve iniziare a contribuire (passaggio da rosso a nero) o contribuire doppiamente (passaggio da nero a doppio nero) alla lunghezza nera, questo perché la lunghezza del percorso nero deve rimanere invariata.
- Il nodo Z diventa rosso, sempre per far rimanere invariata la lunghezza nera, poiché in seguito alla variazione di colore del nodo X, il percorso nero da X a Z è più lungo di uno rispetto a prima della colorazione, dunque è necessario diminuire il percorso nero, ovvero far diventare Z rosso.

4.11.12 Cancellazione in un albero RB: caso 2

Caso 2: Il nodo Y è double black, e suo fratello il nodo Z è nero, con almeno un figlio rosso e peso maggiore esterno.

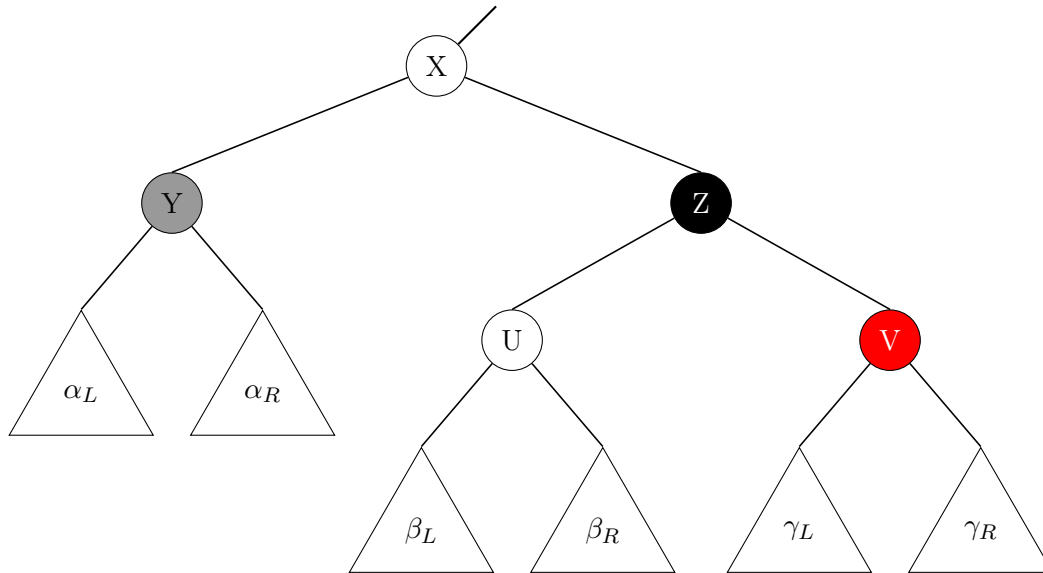
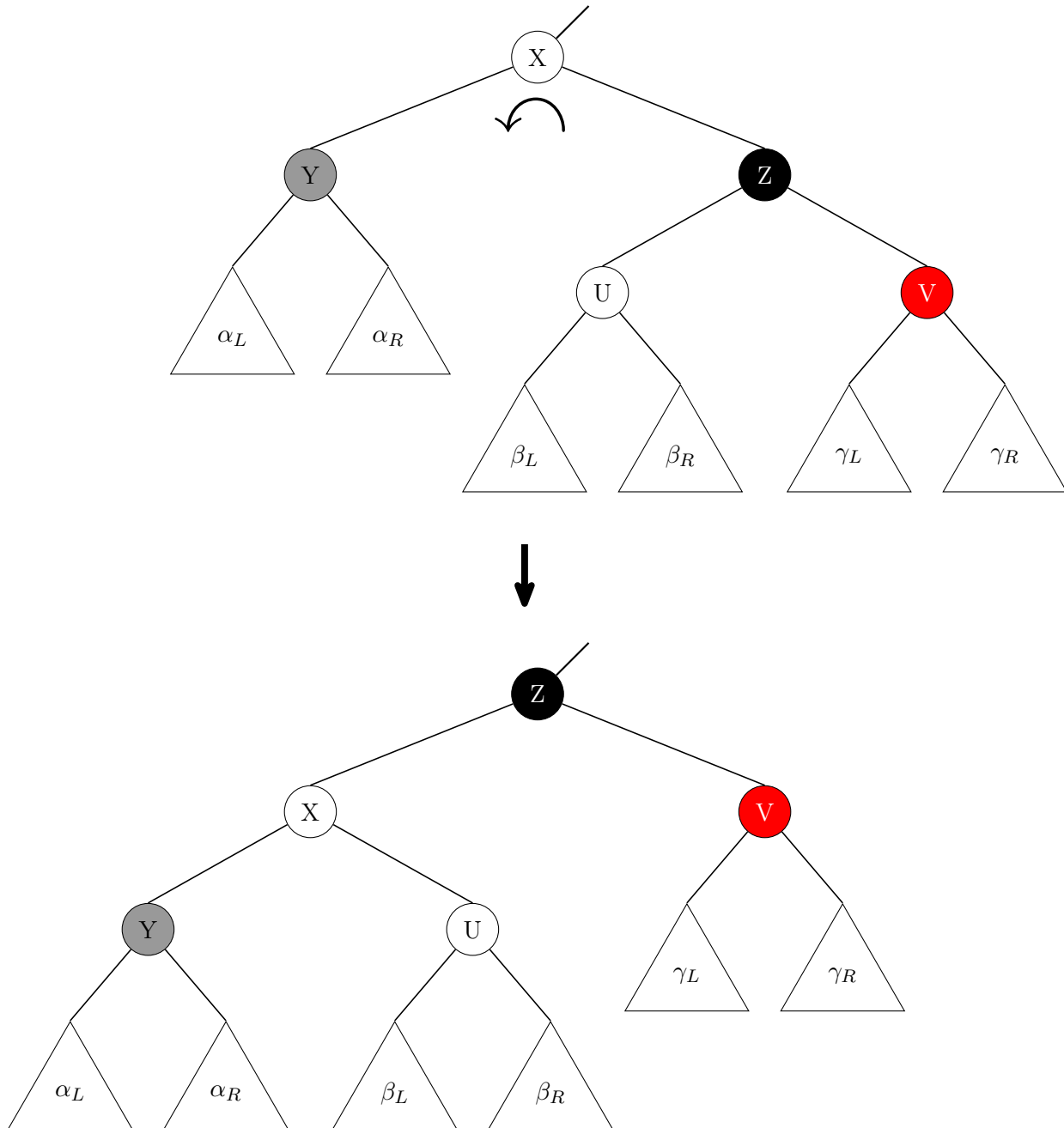


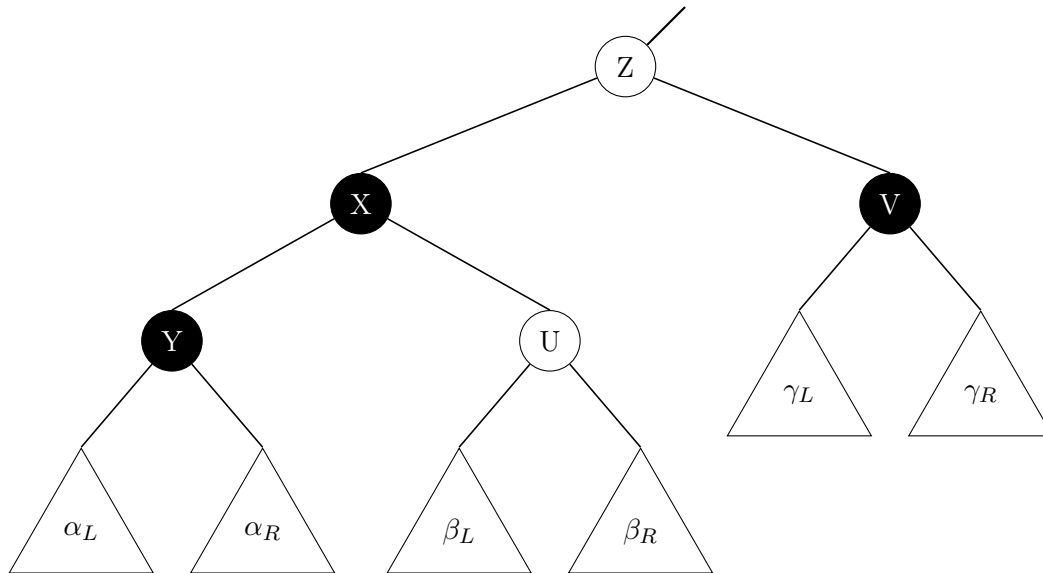
Figura 4.40: Cancellazione in albero RB (Caso 2)

Il peso maggiore è esterno poiché il figlio destro di Z, V, è rosso e quindi è più profondo. Essendo il colore di U indifferente, se U fosse nero, il suo peso sarebbe più leggero rispetto a V, mentre se U fosse rosso, si avrebbe stesso peso di V.

È per questo motivo dunque che il peso maggiore è esterno. Sappiamo, inoltre, che per risolvere il problema basta effettuare una rotazione da destra verso sinistra sul nodo X ed un eventuale ricolorazione.



Applichiamo la ricolorazione.



La ricolorazione è stata effettuata secondo il seguente criterio:

- Il colore di U è stato lasciato invariato.
- Il colore di X diventa nero, indipendentemente dal suo colore precedente, poiché il percorso nero (sull'albero prima della rotazione) da X a U e da X a Y, era rispettivamente lungo 1 e 2, e quindi per mantenere invariata questa lunghezza dopo la rotazione è necessario far diventare X nero, poiché essendo X genitore di Y e U, nel percorso nero partendo da Z, in entrambi i percorsi si passa per il nodo X.
- Il colore di Y diventa nero, per i motivi spiegati nel punto precedente.
- Il nodo Z prende il colore della vecchia radice X.
- Il nodo V diventa nero, poiché prima della rotazione, il percorso da X a V attraversava il nodo nero Z, per rimanere invariata questa lunghezza è necessario far diventare nero V, poiché Z è stato spostato, a seguito della rotazione, a sinistra.

Osserviamo che in questo caso, la ricolorazione effettuata risolve definitivamente il problema.

4.11.13 Cancellazione in un albero RB: caso 3

Caso 3: Il nodo Y è double black, e suo fratello il nodo Z è nero, con un figlio rosso e peso maggiore interno.

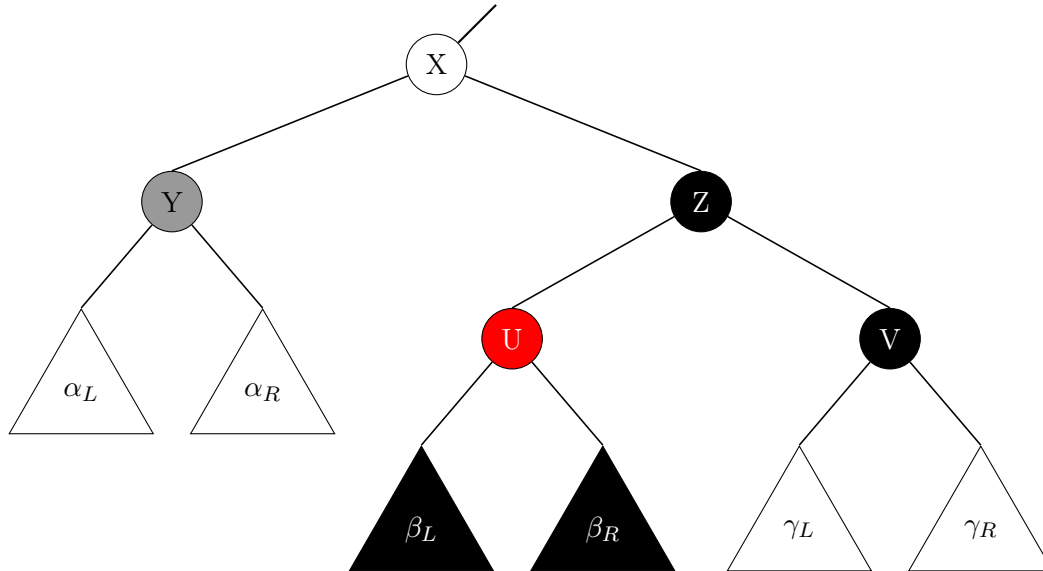
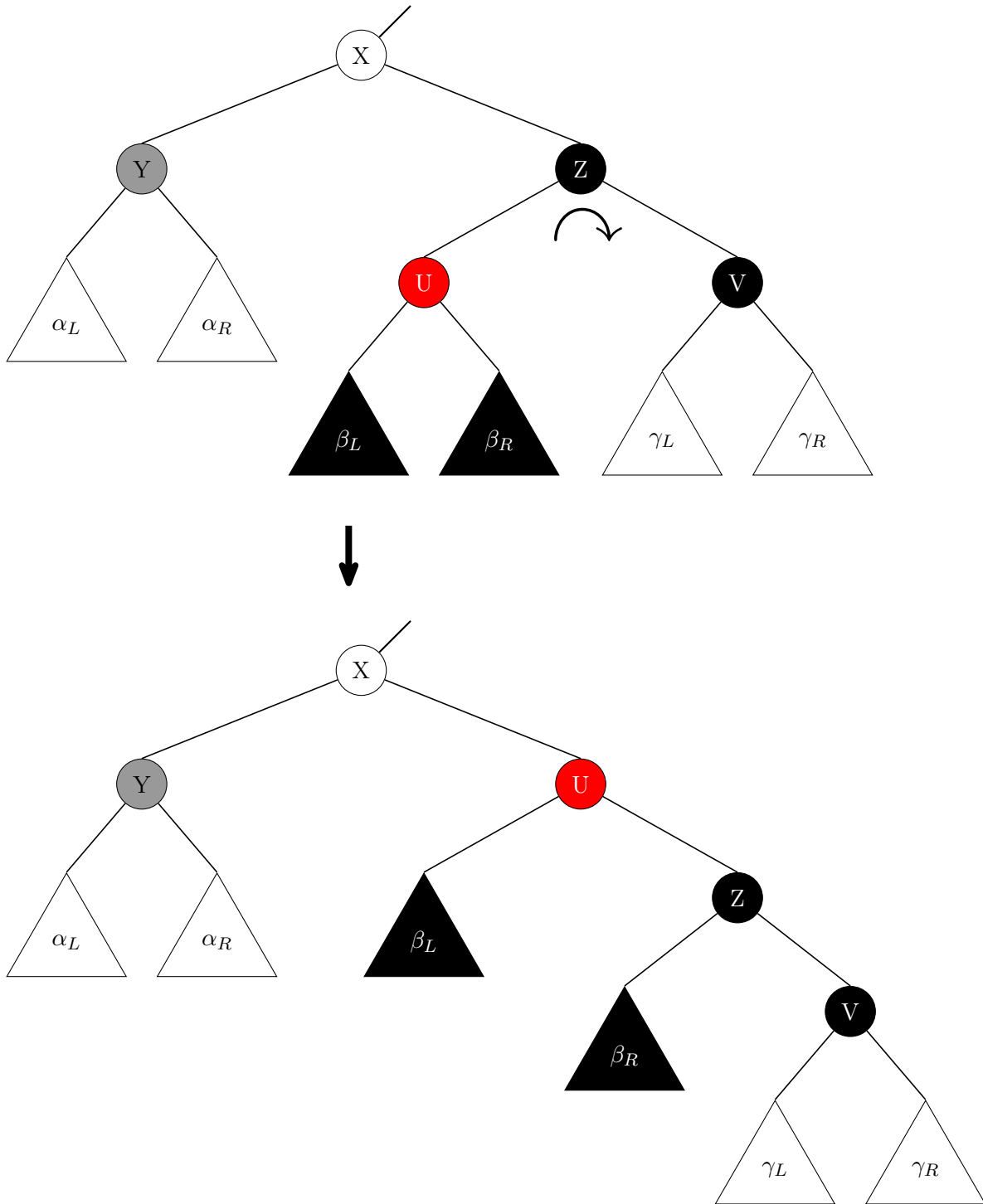


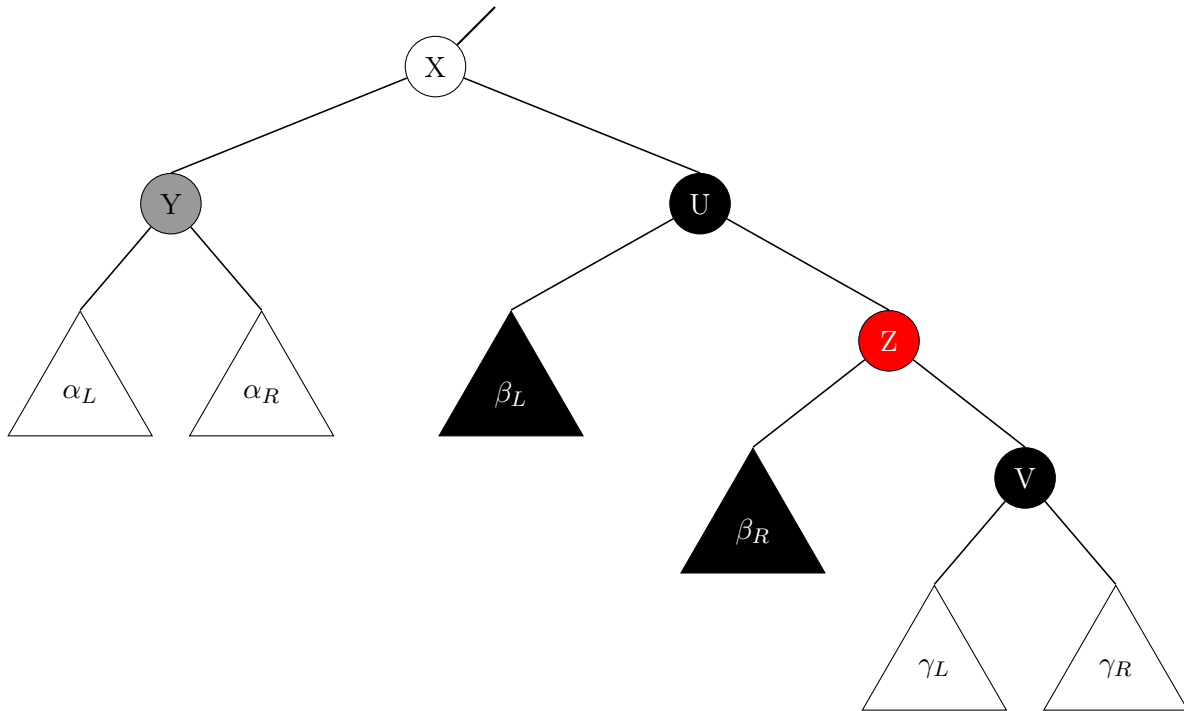
Figura 4.41: Cancellazione in albero RB (Caso 3)

Il peso maggiore è interno per lo stesso ragionamento del caso precedente, ovvero poiché il figlio sinistro di Z, U è rosso. Inoltre, poiché U è rosso, necessariamente i sottoalberi β_L e β_R hanno come radice un figlio nero, poiché altrimenti ci sarebbe la violazione di un padre rosso, con nodo figlio rosso.

Proviamo a risolvere il problema facendo una rotazione da sinistra verso destra sul nodo Z e applicando l'adeguata ricolorazione.



Applico la ricolazione.



Con questa rotazione, ci siamo ricondotti al caso precedente (Caso 2), quindi applicando la stessa procedura mostrata per quel caso, si risolve definitivamente il problema.

La ricolorazione è stata effettuata secondo il seguente criterio:

- Il colore di U diventa nero per essere sicuri di evitare di violare la proprietà dell'albero RB, del nodo padre e figlio rosso.
- Il colore di Z diventa rosso per rimanere invariata la lunghezza nera, poiché se il colore non fosse stato modificato, avremmo che prima della rotazione il percorso da X a Z attraversava un nero (Z stesso), mentre dopo la rotazione lo stesso percorso attraversa due neri (U e Z).

4.11.14 Cancellazione in un albero RB: caso 4

Caso 4: Il nodo Y è double black, e suo fratello il nodo Z è rosso, con figli neri.

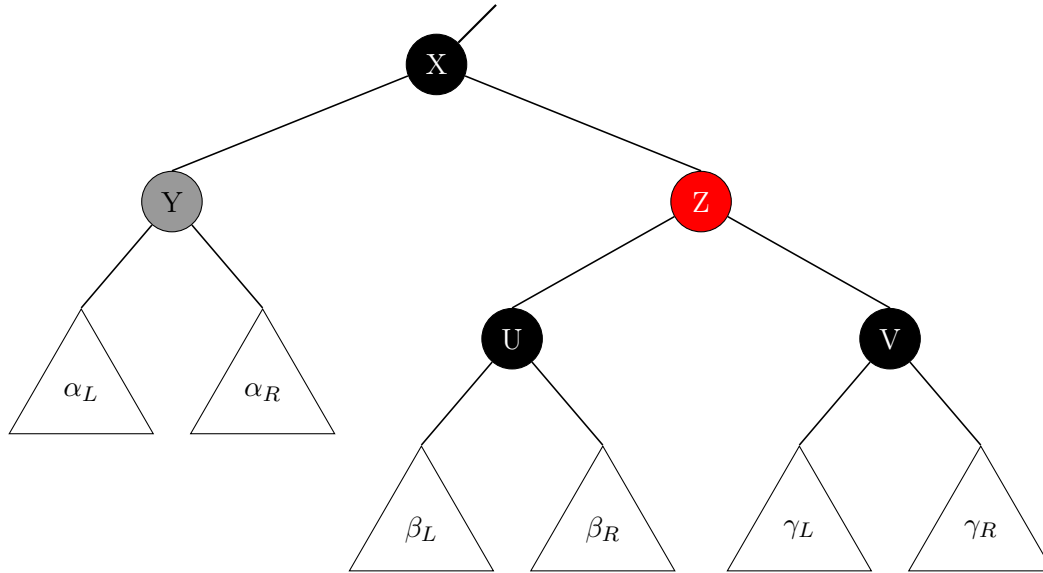
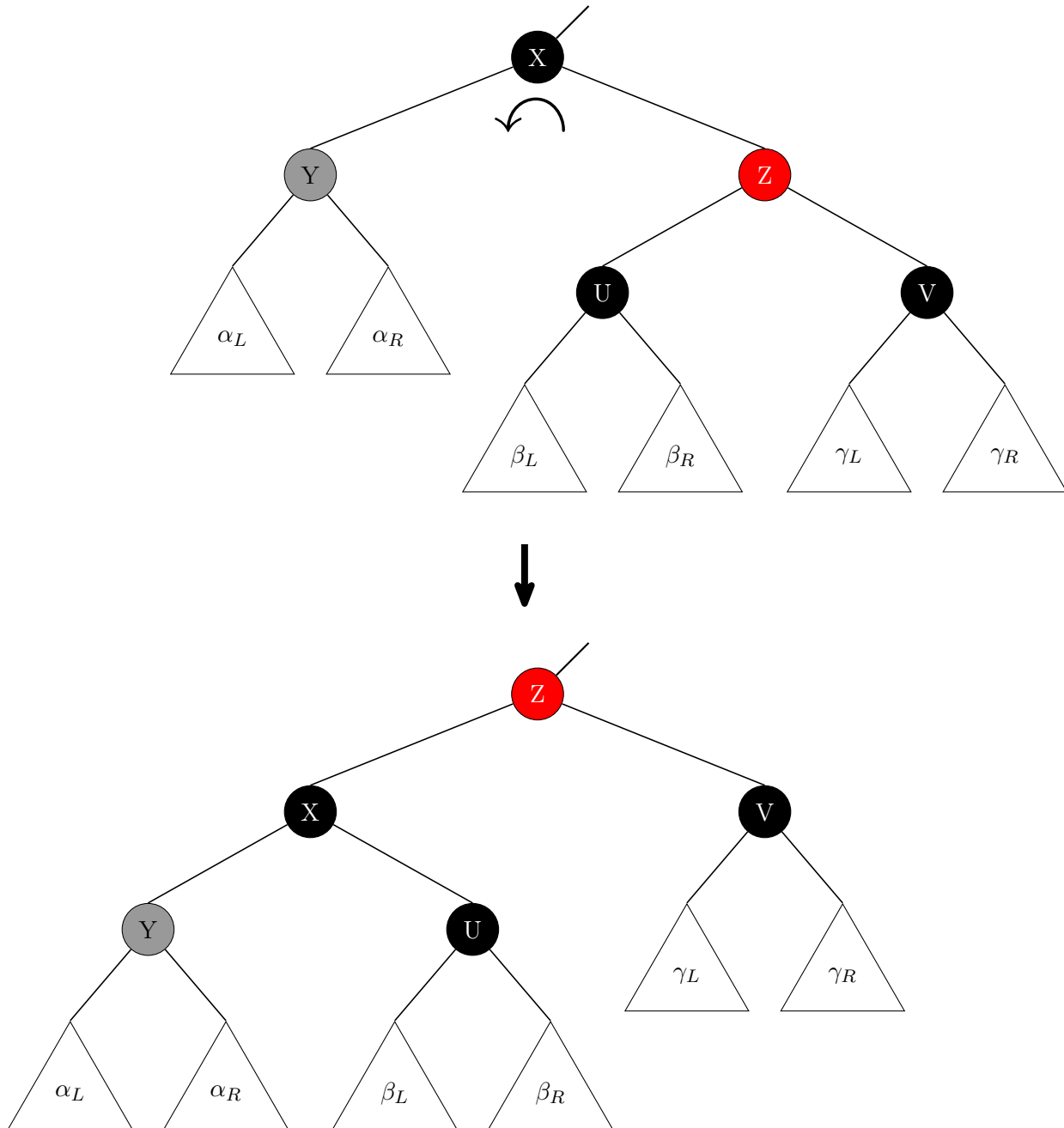


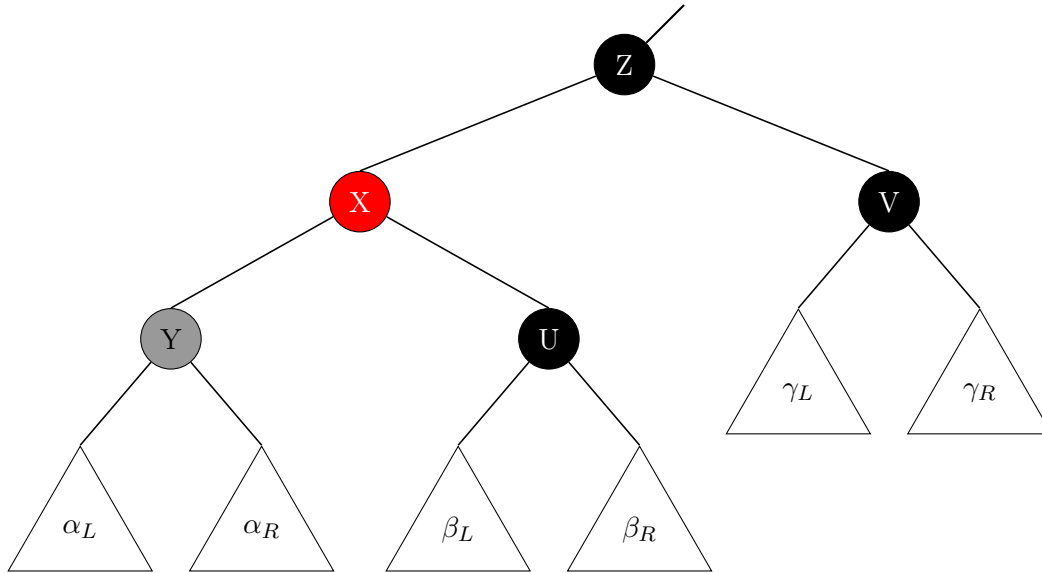
Figura 4.42: Cancellazione in albero RB (Caso 4)

La radice del sottoalbero X deve essere necessariamente nera, altrimenti ci sarebbe una violazione dell'albero RB.

Il peso maggiore si trova nel sottoalbero destro con radice Z, sempre perché Z è rosso, e quindi bisogna creare dello spazio a sinistra, ciò implica una rotazione da destra verso sinistra.



Applichiamo la ricolorazione.



Con questa rotazione, considerando il sottoalbero di radice X, ci siamo ricondotti a uno dei tre casi precedenti, quindi applicando la stessa procedura mostrata per quei casi, sul sottoalbero di radice X, si risolve definitivamente il problema.

La ricolorazione è stata effettuata secondo il seguente criterio:

- Il colore di Z diventa nero per far rimanere invariata la lunghezza del percorso nero dalla radice del sottoalbero a V dopo la rotazione.
- Il colore di X diventa rosso per far rimanere invariata la lunghezza del percorso nero dalla radice del sottoalbero a U dopo la rotazione.

4.11.15 Algoritmo: cancellazione in un albero RB

A questo punto possiamo agli algoritmi.

Algoritmo 60: Cancellazione in un albero RB

```

Signature delete_RB:  $RB(D) \times D \rightarrow RB(D)$ 
function delete_RB(X, d)
  // se il nodo è non foglia
1  if (X  $\neq$  NIL_RB) then
    // se il dato del nodo corrente è maggiore del dato da eliminare
2    if (X.dat > d) then
3      X.l  $\leftarrow$  delete_RB(X.l, d)
      // bilancio mediante una funzione accessoria
4      X  $\leftarrow$  L_del_balance_RB(X)
    // se il dato del nodo corrente è minore del dato da eliminare
5    else if (X.dat < d) then
      // richiamo la funzione sul sottoalbero destro
6      X.r  $\leftarrow$  delete_RB(X.r, d)
      // bilancio mediante una funzione accessoria
7      X  $\leftarrow$  R_del_balance_RB(X)
8    else
      // ho trovato il dato da cancellare
9      X  $\leftarrow$  delete_rode_RB(X)
  // restituisco il nodo corrente
10 return X
  
```

Si nota immediatamente che l'algoritmo è concettualmente analogo a quello visto per gli alberi AVL, con ovviamente una differente implementazione per le funzioni coinvolte visto che si tratta di strutture dati differenti.

Vediamo adesso la funzione di cancellazione di un nodo in un albero red black.

Algoritmo 61: Cancellazione di un nodo in un albero RB

```

Signature delete_node_RB:  $RB(D) \rightarrow RB(D)$ 
function delete_node_RB(X)
    // se il nodo da eliminare non ha figlio sinistro
1   if (X.l = NIL_RB) then
    |   // cancello il nodo mediante una funzione accessoria
2   |   X  $\leftarrow$  skip_right_RB(X)
    // se il nodo da eliminare non ha figlio destro
3   else if (X.r = NIL_RB) then
    |   // cancello il nodo mediante una funzione accessoria
4   |   X  $\leftarrow$  skip_left_RB(X)
5   else
    |   // se il nodo contenente il dato da eliminare ha entrambi i figli
    |   // sostituisco il dato mediante una funzione accessoria
    |   // eliminando il nodo contenente il minimo del sotto-albero destro
6   |   X.dat  $\leftarrow$  get_&_delete_min_RB(X.r, X)
    |   // bilancio mediante una funzione accessoria
7   |   X  $\leftarrow$  R_del_balance_RB(X)
    // restituisco il nodo corrente
8   return X

```

Vediamo adesso la funzione di cancellazione del minimo del sottoalbero destro in un albero red black.

Algoritmo 62: Cancellazione del minimo del sottoalbero destro in un albero RB

```

Signature get_&_delete_min_RB:  $RB(D) \times RB(D) \rightarrow D$ 
function get_&_delete_min_RB(X, P)
    // se ho raggiunto il minimo
1   if (X.l = NIL_RB) then
        // salvo il valore del minimo
2       d  $\leftarrow$  X.dat
        // elimino il nodo e ne salvo la nuova radice del sotto-albero
        // mediante una funzione accessoria
3       Y  $\leftarrow$  skip_right_RB(X)
4   else
        // se non ho raggiunto il minimo
        // richiamo la funzione sul sottoalbero sinistro
5       d  $\leftarrow$  get_&_delete_min_RB(X.l, X)
        // bilancio mediante una funzione accessoria
        // e salvo la nuova radice del sottoalbero
6       Y  $\leftarrow$  L_del_balance_RB(X)
        // collego la nuova radice in modo corretto
        // mediante una funzione accessoria
7   swap_child(P, X, Y)
        // restituisco il dato corrente
8   return d

```

Notiamo che a differenza degli alberi AVL, dove chiamavamo le funzioni `skip_right/skip_left` dei BST, in questo caso abbiamo necessità, a volte, di propagare il colore nero al nodo padre del nodo cancellato e pertanto sono necessarie versioni specializzate delle funzioni skip negli alberi RB.

Sottolineiamo che le uniche vere note di differenza rispetto ai medesimi algoritmi visti per gli AVL sono le seguenti funzioni; senza considerare ovviamente la creazione del nodo.

In effetti, sarebbe possibile implementare una classe in cui le funzioni di cancellazione ed inserimento sono agnostiche sulle funzioni precedentemente viste.

Vediamo ora la funzione che essenzialmente porta il colore nero verso l'alto.

Algoritmo 63: Funzione di propagazione verso l'alto del colore nero

```

Signature propagate_black:  $RB(D)$ 
function propagate_black(X)
    // se il nodo corrente è rosso
1   if (X.cl = R) then
    // cambia il colore a nero
2   X.cl  $\leftarrow$  B
3   else
    // se il nodo corrente è nero
    // cambia il colore a doppio nero
4   X.cl  $\leftarrow$  DB

```

Ecco l'algoritmo della funzione di skip per gli alberi RB.

Algoritmo 64: Funzione di skip left in un albero RB

```

Signature skip_left_RB:  $RB(D) \rightarrow RB(D)$ 
function skip_left_RB(X)
    // se il nodo corrente è nero
1   if (X.cl = B) then
    // propaga il colore nero su figlio sinistro
    // mediante una funzione accessoria
2   propagate_black(X.l)
    // elimina il nodo mediante una funzione accessoria
3   return skip_left(X)

```

Notiamo che per semplicità di algoritmo nel caso in cui il nodo da cancellare fosse nero viene propagato il colore nero sul figlio sinistro del nodo da eliminare in quanto altrimenti sarebbe necessario un puntatore al nodo padre. Il risultato è chiaramente equivalente.

Ecco la funzione che bilancia un albero RB a seguito di una cancellazione a sinistra.

Algoritmo 65: Funzione di bilanciamento di una cancellazione a sinistra in un albero RB

```

Signature L_del_balance_RB:  $RB(D) \rightarrow RB(D)$ 
function L_del_balance_RB(X)
    // se il nodo ha entrambi i figli
1   if ((X.l != NIL_RB) AND (X.r != NIL_RB)) then
        // identifico la casistica mediante una funzione accessoria
        // e chiamo la funzione specifica
2       switch (L_del_violation_RB(X)) do
3           case (1) do
4               X ← L_del_balance_RB1(X)
5           case (2) do
6               X ← L_del_balance_RB2(X)
7           case (3) do
8               X ← L_del_balance_RB3(X)
9           case (4) do
10              X ← L_del_balance_RB4(X)
        // restituisco il nodo corrente
11  return X

```

Ecco l'algoritmo che identifica la casistica della cancellazione nella quale ci si trova.

Anche in questo caso, come negli alberi AVL, ci troviamo nel nodo padre del nodo contenente una violazione (colore doppio nero).

Algoritmo 66: Funzione delle violazioni a seguito di una cancellazione a sinistra in un albero RB

```

Signature L_del_violation_RB:  $RB(D) \rightarrow \{0, 1, 2, 3, 4\}$ 
function L_del_violation_RB(X)
    // setto i valori iniziali, assumo che non ci sia violazione
1   (v, l, r)  $\leftarrow$  (0, X.l, X.r)
    // se il figlio sinistro del nodo corrente è doppio nero
2   if (l.cl = DB) then
        // ... ho una violazione e bisogna identificare il caso
        // se il figlio destro del nodo corrente è rosso
3       if (r.cl = R) then
            // siamo nella casistica n.4
4         v  $\leftarrow$  4
5       else
            // se il figlio destro del nodo corrente è nero
            // bisogna controllarne i figli
            // se il figlio destro è rosso
6         if (r.r.cl = R) then
            // siamo nella casistica n.2
7           v  $\leftarrow$  2
            // se il figlio sinistro è rosso e il destro nero
8         else if (r.l.cl = R) then
            // siamo nella casistica n.3
9           v  $\leftarrow$  3
10        else
            // se sono entrambi neri
            // siamo nella casistica n.1
11          v  $\leftarrow$  1
    // restituisco il valore della violazione
12  return v

```

Con questo abbiamo terminato la trattazione riguardante gli alberi red black.

Capitolo 5

Traduzione di algoritmi ricorsivi in iterativi

"Never send a human to do a machine's job"

– Agent Smith (The Matrix)

5.1 Introduzione

Vediamo adesso alcune tecniche per tradurre un algoritmo dalla versione ricorsiva all'equivalente versione iterativa.

Cominceremo con alcuni esempi e poi vedremo lo schema generale, che al massimo con piccole modifiche, ci permetterà di tradurre un generico algoritmo ricorsivo nella versione iterativa.

5.2 Esempio 1

Prendiamo la funzione di ricerca di un dato in input in un albero binario di ricerca; per semplicità considereremo la funzione che restituisce vero o falso.

Ecco l'algoritmo nella versione ricorsiva.

Algoritmo 67: Esempio 1 (ricorsivo)

Signature `rec_search`: $BST(D) \times D \rightarrow \mathbb{B}$

```

function rec_search(X, d)
    // se il nodo corrente è null
1  if (X =  $\perp$ ) then
    // sono arrivato alla fine e non ho trovato il dato
2      return false
3  else
    // posso dereferenziare il campo dato del nodo
    // se il dato del nodo corrente è maggiore del dato da cercare
4      if (X.dat > d) then
        // richiamo la funzione sul sottoalbero sinistro
5          return rec_search(X.l, d)
    // se il dato del nodo corrente è minore del dato da cercare
6      else if (X.dat < d) then
        // richiamo la funzione sul sottoalbero destro
7          return rec_search(X.r, d)
8      else
        // se ho trovato il dato da cercare
9          return true

```

Questo è un algoritmo ovviamente ricorsivo.

La prima cosa che dobbiamo osservare è che si tratta di un algoritmo ricorsivo in cui nonostante ci siano due chiamate queste due chiamate sono mutuamente esclusive. O ne facciamo una o facciamo l'altra, potremmo perfino non farne nessuna. Non ci sono quindi due chiamate consecutive.

Inoltre a livello sintattico la chiamata ricorsiva è l'ultima istruzione che effettuiamo.

Per essere proprio precisi non è proprio l'ultima perché c'è un ritorno di quel valore ma quello che facciamo è semplicemente chiamare la funzione ricorsiva e poi restituire il valore.

Quindi questa è una funzione tail recursive, e come abbiamo anticipato più volte, una funzione tail recursive è una funzione che può essere simulata a livello iterativo senza aver bisogno di uno stack.

Qual è l'idea di sviluppo della versione iterativa?

Ovviamente potremmo intuitivamente immaginare un algoritmo iterativo che faccia esattamente le stesse cose; quello che però vogliamo ottenere è un approccio metodologico da poter usare indipendentemente dall'algoritmo ricorsivo che si vuole tradurre.

Dovremmo quindi applicare qualcosa che è agnostico rispetto al comportamento dell'algoritmo di partenza.

In effetti, quello che possiamo osservare analizzando l'algoritmo appena scritto è che ogni volta che incappiamo in un caso base semplicemente dobbiamo restituire il valore di quella funzione. L'unico problema è quando andiamo ad analizzare uno dei flussi di computazione che portano alle chiamate ricorsive.

In quel caso per un algoritmo come questo, cioè *tail recursive* quello che osserviamo è che ci sono dei parametri che vengono cambiati ma una volta cambiati quei parametri non serve mai avere il valore delle variabili precedenti.

Come potrebbe funzionare un algoritmo iterativo che faccia esattamente la stessa cosa?

Ripetiamo che non ci interessa trovare un algoritmo tradotto ad hoc, quello che ci interessa è capire come approcciare il problema ed ottenere una metodologia generale.

Una versione iterativa di questo algoritmo potrebbe essere la seguente.

In prima battuta cerchiamo di eseguire il codice così come proposto fino ad arrivare ad un caso base (in questo caso abbiamo due casi base, uno quando il nodo corrente è `null(⊥)`, l'altro quando abbiamo trovato il dato) oppure simuliamo l'algoritmo fino ad arrivare ad una delle chiamate ricorsive.

Se siamo in un caso base il valore della funzione è il valore da restituire. Perché?

Notiamo che nell'algoritmo ricorsivo il valore restituito nel ritorno dalle chiamate ricorsive non subirà alcuna modifica visto che si tratta di una funzione *tail recursive*.

Quindi il valore ottenuto in uno dei casi base sarà poi semplicemente fatto arrivare in cima per poi essere restituito; non c'è nessuna trasformazione da applicare su quel valore.

Quindi questo algoritmo può essere simulato nel seguente modo: nel momento in cui raggiungiamo un caso base abbiamo finito e restituiamo il valore.

Se invece non siamo in un caso base semplicemente cambieremo lo stato locale simulando la chiamata ricorsiva.

La situazione si può simulare nel seguente modo.

Quello che andremo a fare è inserire questo algoritmo nella sua interezza, ovviamente escluse le due chiamate ricorsive, all'interno di un ciclo `while`.

Nel momento in cui abbiamo un caso base semplicemente interrompiamo il ciclo e restituiamo il valore; altrimenti quello che faremo sarà simulare la chiamata ricorsiva semplicemente andando a cambiare alcuni parametri.

Vediamo ora una possibile traduzione iterativa dell'algoritmo ricorsivo di ricerca di un dato in un BST precedentemente visto.

Algoritmo 68: Traduzione Esempio 1 (iterativo)

Signature `itr_search`: $BST(D) \times D \rightarrow \mathbb{B}$

```

function itr_search(X, d)
  // indefinitivamente
1  while (true) do
2    if (X =  $\perp$ ) then
3      // sono arrivato alla fine e non ho trovato il dato
      return false
4    else
5      // posso dereferenziare il campo dato del nodo
      if (X.dat > d) then
6        // simulo la chiamata ricorsiva
        // cambiando il valore del nodo corrente
        // con il figlio sinistro
        X  $\leftarrow$  X.l
7      else if (X.dat < d) then
8        // simulo la chiamata ricorsiva
        // cambiando il valore del nodo corrente
        // con il figlio destro
        X  $\leftarrow$  X.r
9      else
10     // se ho trovato il dato da cercare
        return true

```

Questa traduzione sfrutta fortemente il fatto che l'algoritmo ricorsivo di partenza fosse tail recursive.

Come vedremo a breve per generalizzare l'approccio avremo bisogno di introdurre degli stack per le informazioni di cui abbiamo bisogno anche dopo le chiamate ricorsive e conseguentemente andremo a cambiare la condizione del ciclo `while` da un banale `true` con una condizione più elaborata che come vedremo andrà a distinguere il caso in cui stiamo effettuando delle chiamate ricorsive dal caso in cui stiamo tornando dalle chiamate ricorsive e bisogna fare ancora qualcosa.

Questa suddivisione non è necessaria in questo caso vista la natura tail recursive dell'algoritmo preso come esempio.

Vedremo che non appena eseguiremo istruzioni a valle della chiamata ricorsiva sarà necessaria tale distinzione in quanto sarà necessario simulare anche il ritorno dalla chiamata ricorsiva.

5.3 Esempio 2

Vediamo ora un ulteriore esempio.

Consideriamo il seguente algoritmo ricorsivo.

Sottolineiamo ancora una volta che il nostro scopo è trovare una metodologia generale per tradurre un algoritmo da una versione ricorsiva a quella iterativa. Non è necessario, quindi, che l'algoritmo abbia senso.

Algoritmo 69: Esempio 2 (ricorsivo)

Signature `ric_f`: $BST(D) \times D \rightarrow \mathbb{N}$

```
function ric_f(X, d)
  if (X =  $\perp$ ) then
    return 0
  else
    if (X.dat > d) then
      return 2 * ric_f(X.l, d)
    else if (X.dat < d) then
      return 3 * ric_f(X.r, d)
    else
      return 1
```

Notiamo che questo algoritmo è ottenuto mediante una serie di piccole variazioni di quello della ricerca ricorsiva in BST visto precedentemente.

Ancora una volta non abbiamo bisogno di ricordare i valori precedenti, in questo caso di X.

A livello sintattico, però, questa funzione non è più tail recursive in quanto vi sono istruzioni che vengono eseguite a valle della chiamata ricorsiva (moltiplicazione per 2 o 3).

Osserviamo però che il valore restituito in output dipende solo da ciò che viene calcolato nella chiamata ricorsiva e non da cose che vengono definite prima della chiamata ricorsiva.

Pertanto come già accennato questa funzione sebbene non sia sintatticamente tail recursive risulta essere semanticamente tail recursive.

Infatti, vedremo che nella traduzione, sebbene un po' più complessa, visto che sarà necessario capire se a seguito della chiamata ricorsiva dobbiamo moltiplicare per 2 o per 3, non sarà comunque necessario avere uno stack. Proprio perché è semanticamente tail recursive.

Tuttavia non possiamo utilizzare lo stesso approccio di prima visto che per poter tradurre adeguatamente questo algoritmo dobbiamo distinguere la fase di chiamata ricorsiva dalla fase di ritorno dalla chiamata ricorsiva.

Guardiamo un esempio di funzionamento dell'algoritmo.

Supponiamo che **X** sia la radice dell'albero e **d** il valore 4.

Inoltre supponiamo di aver già trovato il dato **d** e quindi di essere nella fase di ritorno dalla chiamata ricorsiva.

Il valore sull'arco di ritorno dalla chiamata ricorsiva, indica il valore restituito dalla chiamata ricorsiva.

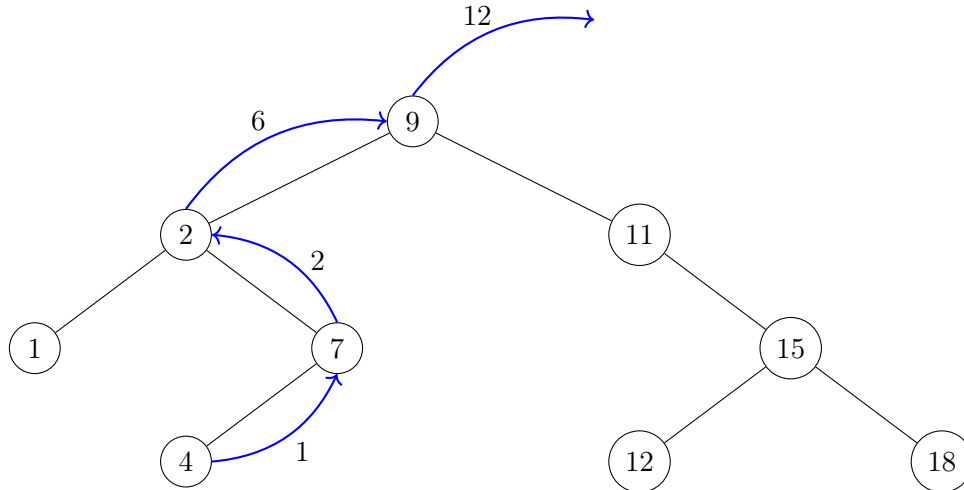


Figura 5.1: Esempio di esecuzione dell'algoritmo sopra descritto

Qual è l'idea?

L'idea è la seguente.

Ovviamente se stiamo iniziando a simulare stiamo effettuando una chiamata ricorsiva; stiamo effettuando la prima chiamata alla funzione. Quindi di certo dobbiamo simulare una chiamata.

Avremo quindi una variabile, che denoteremo con **call**, booleana che setteremo inizialmente a **true**, visto che come abbiamo appena detto la prima cosa che faremo nella simulazione dell'algoritmo è simulare ovviamente una chiamata ricorsiva.

Quindi **call** sarà una variabile booleana che ci permette di distinguere una chiamata da un ritorno. Se **call** è **true** stiamo simulando le chiamate ricorsive, se è **false** stiamo tornando dalle chiamate ricorsive.

Quello che faremo quindi è la seguente cosa.

Intanto che **call** è **true** dobbiamo simulare le chiamate.

Ora, è vero che questo algoritmo è tail recursive però abbiamo un problema nel distinguere da dove stiamo tornando.

Per capire questo, come possiamo fare?

Nel caso specifico dobbiamo sapere se abbiamo chiamato la funzione ricorsiva nel sottoalbero destro o sinistro del nodo corrente. E per avere tale informazione dobbiamo avere conoscenza del nodo corrente, **X**.

Quindi in questo caso, nonostante sia una funzione semanticamente tail recursive, abbiamo bisogno di uno stack per tener traccia di alcune informazioni essenziali per permetterci di distinguere da quale ramo dell'algoritmo stiamo tornando.

Per rendere ancora più chiaro il concetto, andremo prima a tradurre una versione semplificata dell'esempio, nel quale le chiamate ricorsive sono identiche e quindi non abbiamo bisogno di distinguere da quale chiamata stiamo tornando; successivamente mostreremo che senza una struttura di appoggio (stack) sarebbe impossibile effettuare la traduzione dell'esempio con chiamate ricorsive diverse.

5.4 Esempio 3

Vediamo quindi ora, prima un esempio semplificato per poi tornare a discutere quello appena visto.

Algoritmo 70: Esempio 3 (ricorsivo)

Signature `ric_fs`: $BST(D) \times D \rightarrow \mathbb{N}$

```
function ric_fs(X, d)
  if (X =  $\perp$ ) then
    return 0
  else
    if (X.dat > d) then
      return 2 * ric_fs(X.l, d)
    else if (X.dat < d) then
      return 2 * ric_fs(X.r, d)
    else
      return 1
```

Seguendo il ragionamento appena fatto possiamo dare l'algoritmo iterativo ottenuto dalla traduzione dell'algoritmo appena mostrato.

Utilizzeremo un contatore per tenere traccia delle chiamate ricorsive effettuate ed equivalentemente del numero delle chiamate ricorsive dalle quali è necessario tornare.

L'algoritmo è il seguente.

Algoritmo 71: Traduzione Esempio 3 (iterativo)

Signature itr_fs: $BST(D) \times D \rightarrow \mathbb{N}$

```

function itr_fs(X, d)
    // inizializzo il contatore delle chiamate ricorsive a zero e call a true
1   n_call  $\leftarrow$  0
2   call  $\leftarrow$  true
    // fino a quando siamo in fase di chiamata ricorsiva o abbiamo chiamate da cui tornare
3   while (call OR (n_call  $\neq$  0)) do
        // se siamo in fase di chiamata ricorsiva
4       if (call) then
            // se raggiungiamo un caso base
5           if (X =  $\perp$ ) then
                // setto il valore di ritorno come da algoritmo ricorsivo
6                 ret  $\leftarrow$  0
                // setto call a false indicando che comincia la fase di ritorno
7                 call  $\leftarrow$  false
8           else
9               if (X.dat > d) then
                    // simulo la chiamata ricorsiva
                    // cambiando il valore del nodo corrente con il figlio sinistro
10                  X  $\leftarrow$  X.l
                    // incremento il contatore delle chiamate ricorsive
11                  n_call  $\leftarrow$  n_call + 1
12              else if (X.dat < d) then
                    // simulo la chiamata ricorsiva
                    // cambiando il valore del nodo corrente con il figlio destro
13                  X  $\leftarrow$  X.r
                    // incremento il contatore delle chiamate ricorsive
14                  n_call  $\leftarrow$  n_call + 1
15              else
                    // altro caso base
                    // setto il valore di ritorno come da algoritmo ricorsivo
16                  ret  $\leftarrow$  1
                    // setto call a false indicando che comincia la fase di ritorno
17                  call  $\leftarrow$  false

```

```
18 |   else
    |       // se siamo in fase di ritorno dalle chiamate ricorsive
    |       // come da algoritmo ricorsivo multiplico per due il valore di ritorno
19 |       ret ← 2 * ret
    |       // decremento il contatore delle chiamate ricorsive
20 |       n_call ← n_call - 1
    |
    | // restituisco il valore di ritorno
21 | return ret
```

Questa traduzione è possibile proprio perché a seguito di ciascuna chiamata ricorsiva vengono eseguite sempre le medesime istruzioni.

Nel momento in cui vogliamo simulare l'esempio 2 non possiamo più utilizzare questa tecnica perché dobbiamo distinguere i casi in cui a seguito del ritorno da una chiamata ricorsiva moltiplicheremo il valore di ritorno per due o per tre.

Abbiamo quindi necessità di generalizzare ulteriormente questa tecnica.

5.5 Esempio 2 - Continuo

Vediamo quindi ora come possiamo effettuare la traduzione iterativa di un algoritmo ricorsivo semanticamente tail recursive con un approccio più generale che utilizza uno stack per poter salvare informazioni essenziali per permetterci di distinguere da quale chiamata ricorsiva stiamo tornando.

Per poter effettuare questo meccanismo dobbiamo ricordare l'ordine delle chiamate. Dobbiamo quindi capire il criterio con cui abbiamo fatto una chiamata piuttosto che un'altra.

Nel caso specifico possiamo indistintamente utilizzare il valore del nodo corrente, oppure ricordare dove siamo andati mediante un'opportuna variabile *last*.

Entrambe le soluzioni sono applicabili e, come vedremo, sottolineano diverse sfaccettature della questione.

Noi in questo caso mostreremo quella che tiene traccia del valore del nodo.

È quindi immediato che per poter ricordare tali informazioni dobbiamo necessariamente salvarle. Inoltre abbiamo necessità che seguano l'andamento delle chiamate ricorsive e successivi ritorni e pertanto abbiamo necessità di uno stack.

Algoritmo 72: Traduzione Esempio 2 (iterativo)**Signature** itr_f: $BST(D) \times D \rightarrow \mathbb{N}$ **function** itr_f(X, d)

```

// inizializzo lo stack per contenere i nodi
1  Stack S
// setto call a true per indicare che inizia la fase di chiamata
2  call  $\leftarrow$  true
// fino a quando siamo nella fase di chiamate ricorsive
// oppure se nella fase di ritorno dalle chiamate abbiamo ancora chiamate
// dalle quali tornare e quindi lo stack contenente i nodi non è vuoto
3  while (call OR (NOT is_empty(S))) do
    // se siamo nella fase di chiamata
4    if (call) then
        // se siamo in un caso base
5        if (X =  $\perp$ ) then
            // setto il valore di ritorno come da algoritmo ricorsivo
6            ret  $\leftarrow$  0
            // setto call a false indicando che comincia la fase di ritorno
7            call  $\leftarrow$  false
8        else
9            if (X.dat > d) then
                // aggiungo il nodo corrente allo stack
10             S  $\leftarrow$  push(S, X)
                // simulo la chiamata ricorsiva cambiando il valore del nodo corrente
                // con il figlio sinistro
11             X  $\leftarrow$  X.l
12            else if (X.dat < d) then
                // aggiungo il nodo corrente allo stack
13             S  $\leftarrow$  push(S, X)
                // simulo la chiamata ricorsiva cambiando il valore del nodo corrente
                // con il figlio destro
14             X  $\leftarrow$  X.r
15            else
                // altro caso base
                // setto il valore di ritorno come da algoritmo ricorsivo
16             ret  $\leftarrow$  1
                // setto call a false indicando che comincia la fase di ritorno
17             call  $\leftarrow$  false

```

```

18 |   else
    |       // se siamo nella fase di ritorno dalle chiamate ricorsive
    |       // leggo il nodo in testa allo stack
19 |       X ← top(S)
    |       // seguendo il criterio di chiamata ricorsiva capisco quale sia stata
    |       // l'ultima chiamata
    |       // se il dato contenuto nel nodo recuperato dallo stack
    |       // è maggiore del dato in input
20 |       if (X.dat > d) then
    |           // è stata effettuata la chiamata sul sottoalbero sinistro
    |           // e quindi multiplico per 2 il valore di ritorno
21 |           ret ← 2 * ret
22 |       else
    |           // se il dato contenuto nel nodo recuperato dallo stack
    |           // è minore del dato in input
    |           // è stata effettuata la chiamata sul sottoalbero destro
    |           // e quindi multiplico per 3 il valore di ritorno
23 |           ret ← 3 * ret
    |       // elimino la testa dallo stack
24 |       S ← pop(S)
    |
    |       // restituisco il valore di ritorno
25 |   return ret

```

In questo caso potremmo usare direttamente `top_&_pop` per leggere e rimuovere il dato in testa allo stack ma, visto che nel caso generale sarà necessario effettuare tale suddivisione si è preferito cominciare a seguire tale linea.

Quindi ricapitolando.

Se dobbiamo tradurre nella corrispondente versione iterativa una funzione sintatticamente tail recursive abbiamo visto che è sufficiente simulare, addirittura mediante un ciclo `while(true)` le chiamate e nel momento in cui si raggiunge un caso base quello è il valore che viene restituito, indipendentemente dal numero di chiamate che sono state fatte in precedenza.

Abbiamo poi visto la versione nella quale dopo la chiamata ricorsiva effettuiamo del lavoro uniforme e abbiamo visto che non ci serve ricordare tanto se non un'informazione di quante chiamate abbiamo effettuato.

Abbiamo quindi introdotto una variabile booleana `call` per distinguere la fase di simulazione di chiamata ricorsiva dalla fase di simulazione di ritorno e abbiamo anche introdotto un contatore che nella fase di ritorno ci permette di capire quando terminare.

Infine abbiamo visto la versione in cui effettuiamo del lavoro non uniforme al ritorno dalle chiamate ricorsive e quindi è necessario capire da quale chiamata si sta tornando e per far questo l'unico modo è tener traccia di come siamo arrivati alla chiamata ed è quindi necessario lo stack visto che dobbiamo tenere traccia anche dell'ordine di chiamata.

Siamo partiti da un esempio concreto per ottenere uno schema iniziale.

5.6 Schema generale

Vedremo ora uno schema leggermente più generale in cui andremo totalmente ad astrarre dallo specifico esempio ed in cui addirittura l'introduzione degli stacks per le variabili non sarà dettato solo dalla necessità di risalire a quale chiamata ricorsiva stiamo facendo ritorno ma sarà necessario perché l'algoritmo è puramente non tail recursive.

Ossia il lavoro che viene fatto dopo le chiamate ricorsive ha bisogno di conoscere i valori che vengono calcolati prima delle chiamate ricorsive.

Quindi l'unico modo per tenere traccia di questo è mettere tali informazioni in uno stack altrimenti andranno perse.

Vedremo quindi una costruzione che si applica in generale.

Un esempio di funzione non tail recursive che abbiamo visto sono le funzioni di inserimento e cancellazione in alberi bilanciati a causa della necessità di bilanciamento a seguito dell'operazione.

Quello che vedremo ora è uno schema che praticamente astrae dall'esempio concreto.

Ne vedremo una prima versione che si può applicare a funzioni in cui c'è una sola chiamata ricorsiva per ramo di computazione. Successivamente vedremo il caso più generale in cui abbiamo più chiamate ricorsive nello stesso ramo di computazione.

Vedremo come nel caso più generale sarà necessario memorizzare informazione aggiuntiva per capire da quale chiamata ricorsiva stiamo tornando.

Ecco uno schema generale di funzione ricorsiva su albero binario di ricerca non tail recursive in cui c'è una sola chiamata ricorsiva per ramo di computazione.

Algoritmo 73: Algoritmo ricorsivo generale

Signature `rec_gen_F`: $BST(D) \times ? \times ? \rightarrow ?$

function `rec_gen_F`(X, i, j)

```

// inizializzo una variabile locale a con il valore di una funzione
// che utilizza valori in input
// ... indica il lavoro effettuato prima di cominciare
1  a ← Fini(i, j)
// controllo se il nodo corrente è bottom
2  if (X = ⊥) then
    // se il nodo corrente è null siamo nel primo caso base
    // setto il valore di una variabile locale m con il valore di una funzione
    // che prende in input il valore della variabile a precedentemente calcolata
3    m ← F⊥(a)
4  else
    // se l'albero non è vuoto
    // se una certa condizione che utilizza il nodo X e la variabile a è vera
    // ... l sta per left
5    if (Cl(X, a)) then
        // eseguo lavoro preliminare prima della chiamata ricorsiva
6        (i', j', h) ← Flpre(X, a)
        // effettuo la chiamata ricorsiva sul sottoalbero sinistro
        // utilizzando le variabili appena calcolate
7        z ← rec_gen_F(X.l, i', j')
        // eseguo lavoro dopo la chiamata ricorsiva utilizzando
        // variabili calcolate in precedenza alla chiamata ricorsiva
        // e variabili calcolate dalla chiamata ricorsiva
8        m ← Flpost(X, h, z)
    // se una certa condizione che utilizza il nodo X e la variabile a è vera
    // ... r sta per right
9    else if (Cr(X, a)) then
        // eseguo lavoro preliminare prima della chiamata ricorsiva
10       (i', j', h) ← Frpre(X, a)
        // effettuo la chiamata ricorsiva sul sottoalbero destro
        // utilizzando le variabili appena calcolate
11       z ← rec_gen_F(X.r, i', j')
        // eseguo lavoro dopo la chiamata ricorsiva utilizzando
        // variabili calcolate in precedenza alla chiamata ricorsiva
        // e variabili calcolate dalla chiamata ricorsiva
12       m ← Frpost(X, h, z)

```

```
13 |   | else  
   |   |   // se nessuna delle due condizioni è vera siamo nel secondo caso base  
   |   |   // eseguo lavoro per settare la variabile m  
14 |   |   m  $\leftarrow$  Floc(X, a)  
   |   |  
   |   // eseguo lavoro finale e restituisco il valore  
15 | return Ffin(m)
```

Sebbene possa sembrare un algoritmo complesso si tratta fondamentalmente di una versione astratta e quindi più generale degli esempi visti precedentemente.

È ovvio che tale algoritmo non può racchiudere tutti i casi possibili ma è abbastanza generale per permetterci di capire come poi lavorare in generale su qualunque algoritmo di tipo simile.

5.6.1 Passi da effettuare per la traduzione dell'algoritmo ricorsivo generale in iterativo

Per poter effettuare la traduzione dell'algoritmo appena visto nella versione iterativa è in prima battuta necessario capire quali variabili vadano messe sullo stack e quali no.

Abbiamo mostrato uno schema di algoritmo abbastanza generale in cui le chiamate ricorsive avvengono sempre e soltanto in rami disgiunti e vedremo come un algoritmo del genere può essere tradotto in una versione iterativa. Per far ciò, quindi per mettere in atto le intuizioni che abbiamo discusso sugli esempi, quello che bisogna fare in prima battuta è andare a identificare le variabili di cui sarà fondamentale ricordare il valore in uno stack.

Si parte dai parametri in input, non perché ci sia un ordine fissato da seguire ma piuttosto perché ci conviene usare un ordine e sempre quello così da non dimenticare qualcosa.

Quindi partiamo dai valori in input X , i e j .

X essendo un parametro in input proviene dal chiamante e dunque la sua creazione avviene prima di ogni chiamata ricorsiva.

Nell'algoritmo il valore di X viene utilizzato per fare dei test, per fare del lavoro in preordine e viene sicuramente valutato nelle condizioni per andare a sinistra o a destra; tuttavia viene anche utilizzato dopo una chiamata ricorsiva, quindi per lavoro in postordine.

Nello specifico X viene utilizzata sia dopo la chiamata ricorsiva a destra sia dopo quella a sinistra, ma è sufficiente che esista anche soltanto un ramo di computazione dove la variabile è prodotta prima di una chiamata ricorsiva (in questo caso è prodotta in input) e utilizzata dopo una chiamata ricorsiva, senza che venga semplicemente sovrascritta e quindi il valore viene prodotto subito dopo, per far sì che questa variabile abbia bisogno di essere memorizzata in uno stack.

Se noi andassimo semplicemente a simulare una chiamata ricorsiva sostituendo il valore di X non avremmo più modo di recuperarlo e visto che a valle della chiamata ricorsiva X viene utilizzato per effettuare del lavoro questo comporterebbe l'impossibilità di effettuare una traduzione corretta.

Questo ci porta al primo criterio per decidere se una variabile abbia bisogno di uno stack nel processo di traduzione di un algoritmo ricorsivo nella versione iterativa.

In generale se una variabile è prodotta prima di una chiamata ricorsiva ed il suo valore viene utilizzato dopo una chiamata ricorsiva è necessario mettere tale variabile sullo stack.

Quindi X ha bisogno di uno stack.

Vediamo ora i .

i viene prodotto prima di una chiamata ricorsiva, utilizzato prima di una chiamata ricorsiva e mai utilizzato dopo una chiamata ricorsiva.

Non ha quindi bisogno di uno stack.

Attenzione che con dopo una chiamata ricorsiva si intende una qualsiasi istruzione eseguita a seguito di una chiamata ricorsiva e che quindi fa capo allo stesso ramo di computazione contenente la chiamata ricorsiva.

Essere parametro della chiamata ricorsiva denota un utilizzo prima della chiamata ricorsiva visto che i parametri sono settati prima della chiamata, ovviamente.

In generale se una variabile è prodotta prima di una chiamata ricorsiva e il suo valore non viene mai utilizzato dopo una chiamata ricorsiva non è necessario mettere tale variabile sullo stack.

Quindi non aggiungeremo uno stack per *i*.

La variabile *j* ha un comportamento analogo alla variabile *i*. Quindi non è necessario uno stack.

Passiamo ora alle variabili locali.

La variabile *a* viene prodotta prima di una chiamata ricorsiva e sebbene utilizzata in diverse istruzioni non viene mai utilizzata dopo la chiamata ricorsiva, pertanto sembrerebbe non aver bisogno dello stack.

In realtà in questo caso entra in gioco il secondo criterio che definisce la necessità o meno di avere uno stack per una variabile.

In generale se una variabile è prodotta prima di una chiamata ricorsiva e il suo valore non viene mai utilizzato dopo una chiamata ricorsiva ma tale variabile è cruciale per distinguere i casi, per distinguere in quale ramo di computazione l'algoritmo ricorsivo procederà, allora è necessario mettere tale variabile sullo stack.

Attenzione. È bene fare una doverosa precisazione. Se a causa dei criteri appena visti una variabile ha bisogno di uno stack ma il valore della variabile in questione è calcolato deterministicamente all'interno della funzione e il calcolo di tale valore non apporta un effettivo overhead allora è possibile valutare anche la possibilità di calcolare nuovamente il valore della variabile in questione visto che questo approccio comporta anche un minore utilizzo di spazio.

Non è quindi possibile evitare uno stack nel caso in cui la variabile sia calcolata nella funzione ma in modo non deterministico.

Nel caso specifico visto che la variabile *a* viene calcolata mediante una funzione che prende *i*, e *j* in input e visto che tali variabili cambiano nelle varie chiamate ricorsive e non teniamo traccia di tali cambiamenti non è possibile ricalcolare il valore di *a*.

Pertanto servirà uno stack per la variabile *a*.

Andiamo a guardare la variabile *m*.

La variabile *m* viene prodotta in un ramo di computazione non ricorsivo oppure se prodotta in un ramo ricorsivo viene prodotta a seguito della chiamata ricorsiva.

Pertanto non è necessario mettere tale variabile sullo stack.

In generale, se una variabile è prodotta dopo una chiamata ricorsiva non solo non è necessario salvarne il valore su uno stack ma è addirittura impossibile farlo visto che non abbiamo alcun valore da salvare.

Vediamo ora *i'*.

Viene prodotto prima della chiamata ricorsiva ed utilizzato come parametro della chiamata stessa ma non utilizzato dopo la chiamata ricorsiva.

Pertanto non ha bisogno di uno stack.

La variabile j' ha un comportamento analogo alla variabile i' e pertanto non ha bisogno di uno stack.

Passiamo alla variabile h .

La variabile h viene prodotta prima della chiamata ricorsiva ed utilizzata dopo la chiamata.

Ha pertanto bisogno di uno stack.

Attenzione. Nel codice la variabile h prodotta dal ramo ricorsivo destro è distinta dalla variabile h prodotta dal ramo computazionale ricorsivo sinistro visto che i due rami sono disgiunti; è come se avessimo due variabili diverse.

Convienne quindi tener traccia in due stack differenti della variabile h prodotta nel ramo destro e della variabile h prodotta nel ramo sinistro.

Potremmo anche utilizzare un unico stack ma questo porterebbe ad una gestione più complicata che è quindi meglio evitare anche perché la complessità di spazio è identica per le due alternative.

Arriviamo infine alla variabile z .

La variabile viene prodotta dopo la chiamata ricorsiva, visto che è addirittura prodotta dalla chiamata stessa.

Pertanto non ha bisogno di uno stack.

In conclusione avremo quindi bisogno per uno stack per la variabile X , uno per la variabile a , uno per la variabile h prodotta dal ramo di computazione ricorsivo destro (h_r) ed uno per la variabile h prodotta dal ramo di computazione ricorsivo sinistro (h_l).

Per riassumere. Il primo passo da effettuare è capire quali variabili hanno bisogno di stack e quali no.

Per capire ciò seguiamo i seguenti criteri:

- In generale se una variabile è prodotta prima di una chiamata ricorsiva ed il suo valore viene utilizzato dopo una chiamata ricorsiva è necessario mettere tale variabile sullo stack.
- In generale se una variabile è prodotta prima di una chiamata ricorsiva e il suo valore non viene mai utilizzato dopo una chiamata ricorsiva non è necessario mettere tale variabile sullo stack.
- In generale se una variabile è prodotta prima di una chiamata ricorsiva e il suo valore non viene mai utilizzato dopo una chiamata ricorsiva ma tale variabile è cruciale per distinguere i casi, per distinguere in quale ramo di computazione l'algoritmo ricorsivo procederà, allora è necessario mettere tale variabile sullo stack.
- In generale se una variabile è prodotta dopo una chiamata ricorsiva non solo non è necessario salvarne il valore su uno stack ma è addirittura impossibile farlo visto che non abbiamo alcun valore da salvare.

Una volta definiti gli stack necessari dobbiamo identificare lo stack, o potenzialmente gli stack, che sono fondamentali per capire quando l'algoritmo termina.

In generale non è detto che sia sufficiente guardare solo uno stack.

Intuitivamente quando facciamo una chiamata ricorsiva almeno uno dei parametri o gruppi di parametri devono in un certo senso diventare più semplici, cioè procedere verso casi base; infatti se c'è un ramo di computazione che non fa progresso verso il caso base l'algoritmo non termina.

I rispettivi stack delle variabili che procedono verso i casi base sono quelli che dobbiamo controllare all'interno del ciclo `while` in quanto sono gli stack che ci informano della terminazione dell'algoritmo.

Nel caso del nostro algoritmo siamo quindi sicuri che basta considerare solo uno stack e nello specifico quello riferito alla variabile `X`.

Per entrare in un caso base infatti andiamo o a controllare che `X` sia $\perp$ oppure che entrambe le condizioni destra e sinistra non siano verificate.

Possiamo quindi dare la traduzione iterativa dell'algoritmo ricorsivo generale analizzato.

Algoritmo 74: Traduzione in algoritmo iterativo generale

Signature `itr_gen_F`: $BST(D) \times ? \times ? \rightarrow ?$

function `itr_gen_F`(X, i, j)

```

// creo gli stack necessari
1  Stack  $S_X, S_a, S_{h_l}, S_{h_r}$ 
// setto la variabile che indica che siamo in fase di chiamata a true
2  call  $\leftarrow$  true

// fino a quando siamo in fase di chiamata
// o lo stack che contiene la variabile che informa sulla terminazione
// non è vuoto
3  while (call OR (NOT is_empty( $S_X$ ))) do
    // se siamo in fase di chiamate ricorsive
4    if (call) then
        // effettuo lavoro preliminare
        // simulando l'algoritmo ricorsivo
5        a  $\leftarrow$   $F_{ini}(i, j)$ 

        // se il nodo corrente è bottom
        // e quindi sono nel primo caso base
6        if ( $X = \perp$ ) then
            // simulo algoritmo ricorsivo effettuando lavoro
7            m  $\leftarrow$   $F_{\perp}(a)$ 
            // simulo ritorno della chiamata ricorsiva
8            ret  $\leftarrow$   $F_{fin}(m)$ 
            // indico che la inizia la fase di ritorno dalle chiamate
9            call  $\leftarrow$  false
10       else
11       if ( $C_1(X, a)$ ) then
            // effettuo lavoro in preordine
            // simulando l'algoritmo ricorsivo
12            ( $i', j', h$ )  $\leftarrow$   $F_{pre}^1(X, a)$ 
            // salvo il valore delle variabili  $X, a$  e  $h$ 
13             $S_X \leftarrow$  push( $S_X, X$ )
14             $S_a \leftarrow$  push( $S_a, a$ )
15             $S_{h_l} \leftarrow$  push( $S_{h_l}, h$ )
            // simulo chiamata ricorsiva con cambio di parametri
16            ( $X, i, j$ )  $\leftarrow$  ( $X.l, i', j'$ )

```

```

17      else if ( $C_r(X, a)$ ) then
18          // effettuo lavoro in preordine
19          // simulando l'algoritmo ricorsivo
20          ( $i', j', h$ )  $\leftarrow F_{pre}^r(X, a)$ 
21          // salvo il valore delle variabili X, a e h
22           $S_X \leftarrow \text{push}(S_X, X)$ 
23           $S_a \leftarrow \text{push}(S_a, a)$ 
24           $S_{h_r} \leftarrow \text{push}(S_{h_r}, h)$ 
25          // simulo chiamata ricorsiva con cambio di parametri
26          ( $X, i, j$ )  $\leftarrow (X.r, i', j')$ 
27      else
28          // se siamo nell'altro caso base
29          // effettuo lavoro locale simulando l'algoritmo
30           $m \leftarrow F_{loc}(X, a)$ 
31          // simulo ritorno della chiamata ricorsiva
32           $ret \leftarrow F_{fin}(m)$ 
33          // indico che inizia la fase di ritorno
34           $call \leftarrow \text{false}$ 
35      else
36          // se siamo nella fase di ritorno dalle chiamate ricorsive
37          // leggo l'elemento dallo stack di X
38           $X \leftarrow \text{top}(S_X)$ 
39          // simulo valore di ritorno della chiamata ricorsiva
40           $z \leftarrow ret$ 
41          // bisogna capire da quale ramo ricorsivo si proviene
42          // recupero il valore di a
43           $a \leftarrow \text{top}(S_a)$ 
44          if ( $C_l(X, a)$ ) then
45              // ... se provengo da quello sinistro
46              // recupero il valore della variabile h
47              // generata nel ramo ricorsivo sinistro
48               $h \leftarrow \text{top}(S_{h_l})$ 
49              // effettuo lavoro in postordine
50              // simulando l'algoritmo ricorsivo
51               $m \leftarrow F_{post}^l(X, h, z)$ 
52              // simulo ritorno della chiamata ricorsiva
53               $ret \leftarrow F_{fin}(m)$ 
54              // rimuovo valore dallo stack di  $h_l$ 
55               $S_{h_l} \leftarrow \text{pop}(S_{h_l})$ 

```

```

36 |   |   | else
    |   |   | // ... se provengo da quello destro
    |   |   | // recupero il valore della variabile h
    |   |   | // generata nel ramo ricorsivo destro
37 |   |   |  $h \leftarrow \text{top}(S_{h_r})$ 
    |   |   | // effettuo lavoro in postordine
    |   |   | // simulando l'algoritmo ricorsivo
38 |   |   |  $m \leftarrow F_{\text{post}}^r(X, h, z)$ 
    |   |   | // simulo ritorno della chiamata ricorsiva
39 |   |   |  $\text{ret} \leftarrow F_{\text{fin}}(m)$ 
    |   |   | // rimuovo valore dallo stack di  $h_r$ 
40 |   |   |  $S_{h_r} \leftarrow \text{pop}(S_{h_r})$ 
    |   |   |
41 |   |   |  $S_X \leftarrow \text{pop}(S_X)$ 
42 |   |   |  $S_a \leftarrow \text{pop}(S_a)$ 
    |   |   |
    |   |   | // restituisco il valore di ritorno
43 |   |   | return ret

```

5.7 Esempio 4

Guardiamo l'ultima cosa che ci interessa relativamente alla traduzione.

Supponiamo di avere un algoritmo che non faccia una sola chiamata ricorsiva per ramo di computazione. Ne faccia due.

Ad esempio il merge sort ricorsivo è un algoritmo di questo tipo.

Ricordiamo che gli algoritmi che hanno due chiamate ricorsive per ramo di computazione non possono essere tail recursive perché dopo una chiamata ricorsiva c'è un'altra cosa da fare.

Questo significa che uno stack dovrà esserci per forza.

Vediamo proprio come esempio l'algoritmo del merge sort ricorsivo, ci concentreremo direttamente sulla funzione ricorsiva chiamata dalla funzione di interfaccia.

Algoritmo 75: Esempio 4 (ricorsivo)

Signature `rec_merge_sort`: $A(D) \times \mathbb{N} \times \mathbb{N} \rightarrow \emptyset$

function `rec_merge_sort`(A, i, j)

```

1  // se il sottoarray considerato è non vuoto o di una sola cella
   if (i < j) then
2      // calcolo l'indice mediano
       m ← floor((i+j)/2)
3      // richiamo la funzione sul sottoarray sinistro
       rec_merge_sort(A, i, m)
4      // richiamo la funzione sul sottoarray destro
       rec_merge_sort(A, m+1, j)
5      // effettuo il merge mediante una funzione accessoria
       merge(A, i, m, j)

```

Un algoritmo del genere come va simulato?

La prima cosa che dobbiamo fare è identificare quali sono le variabili che vanno messe sullo stack.

L'array A chiaramente non va messo sullo stack visto che può essere visto come un qualcosa di costante che non cambia nell'esecuzione dell'algoritmo.

Non avviene infatti nessun cambiamento del valore di A come punto di accesso per le informazioni contenute nell'array.

Guardiamo la variabile i.

Essa viene prodotta prima delle chiamate ricorsive, utilizzata prima delle chiamate e anche dopo, dalla funzione `merge`.

Avrà quindi bisogno di uno stack.

Discorso analogo per la variabile j . Come la variabile i , anche la variabile j avrà bisogno di uno stack.

Analizziamo m .

Viene prodotta prima delle chiamate ricorsive e poi utilizzata anche dopo le chiamate ricorsive.

È necessario quindi uno stack per la variabile m .

Notiamo però che la variabile m viene calcolata all'interno della funzione in modo deterministico con un calcolo alquanto semplice che fa capo esclusivamente alle variabili i e j che come detto andremo a salvare nei rispettivi stack.

Possiamo pertanto evitare di avere uno stack per la variabile m e quindi andare a calcolarlo direttamente.

Nel caso specifico vista la semplicità delle operazioni necessarie per tale calcolo avremo non solo un guadagno di spazio ma anche di operazioni.

Scegliamo quindi di calcolare m e di non utilizzare uno stack.

Dobbiamo ora determinare quali stack prendere in considerazione per la terminazione dell'algoritmo.

Risulta chiaro che è necessario valutare entrambi gli stack.

Se immaginiamo di avere l'albero delle chiamate ricorsive notiamo facilmente che lungo un ramo di computazione uno degli indici non farà progresso verso il caso base; pertanto è necessario considerarli entrambi.

Con $M(0,10)$ si intende una chiamata ricorsiva del merge sort con indice $i = 0$, e indice $j = 10$. È inutile scrivere ogni volta l'array A , poiché come già osservato nella chiamate ricorsiva è costante.

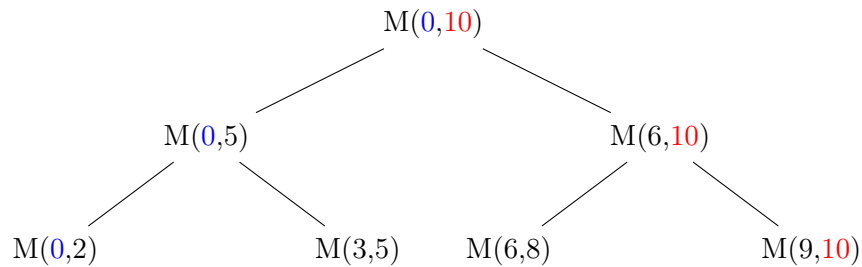


Figura 5.2: Esempio di albero delle chiamate ricorsive del merge sort

Per ora, quindi, la procedura è identica al caso generale nel quale l'algoritmo è caratterizzato da rami di computazione indipendenti con una sola chiamata ricorsiva.

Quello che effettivamente cambia nel caso di un algoritmo con più chiamate ricorsive per uno o più rami di computazione è solo la fase di ritorno dalle chiamate perché nella fase di ritorno dalla prima chiamata andrà simulata la chiamata ricorsiva successiva.

Il problema è quindi ora capire da quale chiamata ricorsiva si sta tornando.

L'unica cosa che possiamo sfruttare è il fatto che le due chiamate ricorsive abbiano differenti parametri in input e mediante tale osservazione possiamo capire da quale chiamata ricorsiva stiamo tornando.

Salveremo l'ultimo parametro in questione utilizzato in una variabile, che chiameremo **last**, e mediante un confronto potremo quindi capire da quale chiamata ricorsiva stiamo effettivamente tornando.

Nel caso specifico ad esempio possiamo osservare che il secondo argomento delle chiamate ricorsive una volta è i , un'altra $m+1$; e visto come viene calcolato m possiamo essere certi che $i \neq m+1$ e pertanto avere un modo per distinguere le due chiamate ricorsive.

Avremmo anche potuto considerare il terzo argomento della funzione ricorsiva in questo caso.

Ecco la traduzione iterativa dell'algoritmo di merge sort ricorsivo.

Algoritmo 76: Traduzione Esempio 4 (iterativo)

```

Signature itr_merge_sort:  $A(D) \times \mathbb{N} \times \mathbb{N} \rightarrow \emptyset$ 
function itr_merge_sort(A, i, j)
    // dichiaro gli stack
1   Stack  $S_i, S_j$ 
    // setto la variabile di call per denotare che siamo nella fase di chiamata
2   call  $\leftarrow \text{true}$ 
    // fino a quando siamo nella fase di chiamata
    // o se nella fase di ritorno ho ancora chiamate dalle quali tornare
3   while (call OR ((NOT is_empty( $S_i$ )) OR (NOT is_empty( $S_j$ )))) do
    // se siamo nella fase delle chiamate ricorsive
4   if (call) then
    // simulo algoritmo prima della chiamata
5   if ( $i < j$ ) then
6    $m \leftarrow \text{floor}((i+j)/2)$ 
    // salvo i valori di i e j
7    $S_i \leftarrow \text{push}(S_i, i)$ 
8    $S_j \leftarrow \text{push}(S_j, j)$ 
    // simulo la prima chiamata ricorsiva cambiando parametri
9    $j \leftarrow m$ 
10  else
    // sono nel caso base e bisogna simulare un ritorno
11  call  $\leftarrow \text{false}$ 
    // ricordo il parametro della chiamata ricorsiva
    // in modo tale da sapere da quale chiamata stiamo tornando
12  last  $\leftarrow i$ 

```

```

13 |   else
    |       // se sono nella fase di ritorno
    |       // bisogna capire da quale chiamata ricorsiva stiamo tornando
    |       // recupero i valori delle variabili negli stack
14 |       i ← top(Si)
15 |       j ← top(Sj)
    |       // calcolo il valore di m
16 |       m ← floor((i+j)/2)
    |       // confronto il valore di last con i per capire
    |       // se stiamo tornando dalla prima chiamata ricorsiva o meno
17 |       if (last = i) then
    |           // se stiamo tornando dalla prima chiamata simulo la seconda
    |           // mediante un cambio di parametri, possiamo evitare di salvare i valori
    |           // delle variabili i e j visto che le abbiamo solo lette
    |           // e non rimosse dagli stack, è come se fossero state già pushate
18 |           i ← m + 1
19 |           call ← true
20 |       else
    |           // se stiamo tornando dalla seconda chiamata simulo l'algoritmo
21 |           merge(A, i, m, j)
    |           // rimuovo gli elementi dallo stack
22 |           Si ← pop(Si)
23 |           Sj ← pop(Sj)
    |           // ricordo il parametro di interesse dell'ultima chiamata
24 |           last ← i

```

Osserviamo che la condizione del **while** può essere semplificata visto che i due stack sono allineati ma per correttezza formale si è deciso di considerarli entrambi.

La condizione poteva essere quindi sostituita con **(call OR (NOT is_empty(S_i)))**.

Sottolineiamo che il valore della variabile **last** va aggiornato ogni volta che si torna da una chiamata ricorsiva per indicare con quale parametro è stata effettuata la chiamata stessa.

5.8 Esempio 5

Vediamo ora un ultimo esempio leggermente più complesso.

Si tratta di un algoritmo basato sulla visita in profondità in un albero binario, la visita in preorder.

Algoritmo 77: Esempio 5 (ricorsivo)

Signature `rec_DFS`: $BT(D) \times (D \times A \rightarrow A) \times A \rightarrow A$

```

function rec_DFS(X, F, a)
    // se il nodo corrente è diverso da null
1  if (X  $\neq \perp$ ) then
    // aggiorniamo l'accumulatore mediante la funzione in input
2    a  $\leftarrow$  F(X.dat, a)
    // chiamo ricorsivamente la funzione sul sottoalbero sinistro
3    a  $\leftarrow$  rec_DFS(X.l, F, a)
    // chiamo ricorsivamente la funzione sul sottoalbero destro
4    a  $\leftarrow$  rec_DFS(X.r, F, a)
    // restituisco il valore aggiornato dell'accumulatore
5    return a
6  else
    // effettuo del lavoro mediante una funzione accessoria
7    a  $\leftarrow$  G(a)
    // restituisco l'accumulatore
8    return a
  
```

Se volessimo tradurre questo algoritmo dovremmo eseguire esattamente la stessa procedura vista in generale.

C'è però un importante dettaglio aggiuntivo da considerare.

Chiaramente il parametro che indica la terminazione dell'algoritmo è il valore del nodo corrente `X`, non possiamo prendere in considerazione l'accumulatore `a` in quanto non è detto che nel corso dell'algoritmo faccia effettivamente progresso.

Quindi l'unico parametro sul quale possiamo basare il criterio di last è il nodo `X`.

Immaginiamo però di essere in un nodo foglia. La chiamata nel sottoalbero sinistro verrà effettuata su un nodo che ha valore $\perp$ così come la chiamata nel sottoalbero destro; non abbiamo quindi alcun modo di distinguere questi due casi.

Dobbiamo quindi gestire il caso limite del nodo foglia in modo differente.

Quello che faremo per gestire questo caso sarà andare sempre ad effettuare la chiamata nel sottoalbero sinistro, mentre a destra andremo solo se il sottoalbero destro non è $\perp$.

In questo modo al ritorno dalla chiamata ricorsiva nel sottoalbero sinistro potremo anche avere come valore di

last $\perp$, mentre dal ritorno da destra non sarà possibile e quindi ci assicuriamo che ci sarà sempre una differenza.

Attenzione. Se l'algoritmo prevede del lavoro anche quando il nodo è $\perp$, allora sebbene a destra non ci si vada il lavoro dovrà comunque essere simulato.

Passiamo quindi ad analizzare le variabili.

La variabile **X** chiaramente va messa sullo stack.

Analizziamo la variabile **a**.

Attenzione. La variabile **a** viene sovrascritta da ogni chiamata ricorsiva, pertanto può essere vista come una variabile che viene prodotta prima della chiamata, utilizzata prima e mai utilizzata dopo. Non ha quindi bisogno di uno stack.

Per rendersi conto velocemente della veridicità di tale affermazione possiamo anche sostituire il nome delle varie occorrenze della variabile **a** nel codice lasciando invariato il comportamento dell'algoritmo.

Di seguito mostriamo l'algoritmo precedente con la modifica proposta.

Algoritmo 78: Esempio 5 (ricorsivo) v. 2

Signature `rec_DFS`: $BT(D) \times (D \times A \rightarrow A) \times A \rightarrow A$

function `rec_DFS(X, F, a)`

```

1  // se il nodo corrente è diverso da null
  if ( $X \neq \perp$ ) then
2      // aggiorni l'accumulatore mediante la funzione in input
       $a' \leftarrow F(X.d, a)$ 
3      // chiamo ricorsivamente la funzione sul sottoalbero sinistro
       $a'' \leftarrow \text{rec\_DFS}(X.l, F, a')$ 
4      // chiamo ricorsivamente la funzione sul sottoalbero destro
       $a''' \leftarrow \text{rec\_DFS}(X.r, F, a'')$ 
5      // restituisco il valore aggiornato dell'accumulatore
      return  $a'''$ 
6  else
7      // effettuo del lavoro mediante una funzione accessoria
       $a \leftarrow G(a)$ 
8      // restituisco l'accumulatore
      return  $a$ 

```

Quindi l'unico valore che ci interessa mettere in uno stack è **X**.

Mostriamo di seguito la traduzione della DFS preorder ricorsiva su un albero binario, nella sua versione iterativa.

Algoritmo 79: Traduzione Esempio 5 (iterativo)**Signature** itr_DFS: $BT(D) \times (D \times A \rightarrow A) \times A \rightarrow A$

```

function itr_DFS(X, F, a)
    // setto lo stack
1   Stack SX
    // indico che inizia la fase di chiamata
2   call  $\leftarrow$  true
    // fino a quando siamo in fase di chiamata
    // oppure in fase di ritorno ma ci sono ancora chiamate da cui ritornare
3   while (call OR (NOT is_empty(SX))) do
    // se sono in fase di chiamata
4   if (call) then
    // se non sono in un caso base
5   if (X  $\neq$   $\perp$ ) then
    // simulo l'algoritmo effettuando lavoro in preorder
6   a  $\leftarrow$  F(X.d, a)
    // salvo il valore di X nello stack
7   SX  $\leftarrow$  push(SX, X)
    // simulo la prima chiamata ricorsiva cambiando il valore di X
8   X  $\leftarrow$  X.l
9   else
    // se sono in un caso base effettuo lavoro simulando l'algoritmo
10  a  $\leftarrow$  G(a)
    // simulo il ritorno
11  ret  $\leftarrow$  a
    // indico che inizia la fase di ritorno
12  call  $\leftarrow$  false
    // ricordo il valore dell'ultimo parametro della chiamata
13  last  $\leftarrow$  X
14  else
    // se sono in fase di ritorno prendo il valore di X dallo stack
15  X  $\leftarrow$  top(SX)
    // simulo il ritorno della chiamata
16  a  $\leftarrow$  ret
    // se sto tornando dalla chiamata sinistra
17  if (last = X.l) then
    // ... e il sottoalbero destro è non vuoto
18  if (X.r  $\neq$   $\perp$ ) then
    // simulo la chiamata destra
19  X  $\leftarrow$  X.r
    // indico che siamo in fase di chiamata
20  call  $\leftarrow$  true

```

```
21 | | | else
    | | | // se il sottoalbero destro è vuoto
    | | | // simulo il lavoro che avrebbe dovuto effettuare
22 | | | a ← G(a)
    | | | // simulo ritorno
23 | | | ret ← a
    | | | // ricordo il valore del parametro della chiamata
24 | | | last ← X
    | | | // rimuovo il valore di X dallo stack
25 | | | SX ← pop(SX)
    | | |
26 | | | else
    | | | // se sto tornando dalla chiamata destra
    | | | // ricordo il valore del parametro della chiamata
27 | | | last ← X
    | | | // rimuovo il valore di X dallo stack
28 | | | SX ← pop(SX)
    | | |
    | | | // restituisco il valore di ritorno
29 | | | return ret
```

Con questo esempio terminiamo la trattazione sulla traduzione degli algoritmi dalla versione ricorsiva in quella iterativa.

Capitolo 6

Grafi

"Neo, sooner or later you're going to realize just as I did that there's a difference between knowing the path and walking the path"

– Morpheus (The Matrix)

6.1 Introduzione

L'ultima classe di strutture dati che analizzeremo sono i grafi.

Cosa sono i grafi?

A livello algebrico un grafo è una coppia formata da un insieme, detto insieme di punti o di nodi o di vertici, che indicheremo con V e da una relazione E che, nel caso di grafi non orientati, è un sottoinsieme dell'insieme delle parti di V avente come elementi solo insiemi di cardinalità 2. Stiamo quindi mettendo in relazione senza preoccuparci dell'ordine (per questo parliamo di grafi non orientati) due elementi dell'insieme di vertici V .

Formalmente.

$$G = \langle V, E \rangle, \text{ dove } V \text{ è l'insieme dei vertici e } E \subseteq V \times V \text{ è l'insieme degli archi o edge} \quad (6.1)$$

Nel caso di grafo non orientato, l'insieme E è uguale a:

$$E \subseteq \{A \subseteq V \mid |A| = 2\} \quad (6.2)$$

Noi saremo un po' più laschi dei matematici per due motivi:

1. Ammetteremo anche relazioni di elementi con se stessi;
2. Addirittura rilasseremo ulteriormente il concetto di grafo permettendo di avere relazioni non necessariamente simmetriche.

Quali sono degli esempi di grafi?

Un esempio di grafo è la struttura di una rete di calcolatori. I calcolatori sono i vertici e le interconnessioni tra due calcolatori sono i cosiddetti archi o edge.

Un altro esempio di grafo non simmetrico è la relazione di parentela tra individui.

Un altro esempio è il concetto di friendship utilizzato in qualunque social network.

Un altro esempio sono le reti stradali.

Tale struttura dati risulta quindi essenziale nell'informatica, possiamo infatti dire che la quasi totalità delle informazioni che è possibile gestire in ambito informatico è descrivibile sotto forma di grafi.

Proprio per questo, quello che vedremo non è che è la punta di un iceberg perché esistono in letteratura decine di migliaia di algoritmi che lavorano su grafi.

I grafi sono una delle strutture più flessibili per rappresentare informazioni perché ci permettono di rappresentare informazione gerarchica, non gerarchica, informazione mista o comunque di complessità arbitraria e tuttavia sempre rappresentata a livello di grafi.

Quindi, nel nostro ambito che cos'è un grafo?

A livello matematico generalizziamo la nozione di grafo nel seguente modo.

Un grafo G è una coppia formata da un insieme dei vertici V (quasi sempre lavoreremo su grafi finiti e quando parliamo di grafo finito intendiamo che l'insieme dei vertici è finito, se parleremo di qualcosa infinito sarà solo per esemplificare qualche caso limite) e da una relazione binaria E definita su V , ossia E è un sottoinsieme di $V \times V$. In particolare nessuno obbliga la simmetria del grafo.

Formalmente.

$$G = \langle V, E \rangle \quad \text{con} \quad |V| = n \in \mathbb{N} \text{ e } E \subseteq V \times V \quad (6.3)$$

Quindi questo è per noi a livello astratto un grafo.

Fondamentalmente un insieme di elementi e delle proprietà tra questi elementi.

Della natura specifica degli elementi a noi non importa, quello che sarà davvero importante sono le relazioni.

L'informazione è codificata nell'esistenza o meno di relazioni tra elementi.

Questo è un concetto di grafo a livello astratto e come abbiamo già visto per altre strutture dati una cosa è la rappresentazione astratta un'altra cosa è rappresentare tali informazioni in memoria all'interno di un calcolatore.

Vedremo che le rappresentazioni dei grafi, come negli alberi (anche se degli alberi ne abbiamo vista solo una), sono diverse.

Le rappresentazioni possibili non sono poche, noi ne vedremo due macrovarianti, ognuna delle quali ha dei pro e contro, e cercheremo di capire in quale contesto è preferibile una rappresentazione piuttosto che un'altra.

6.2 Rappresentazione di un grafo tramite matrice di adiacenza

Facciamo un'importante osservazione.

Come detto la relazione binaria di un grafo è un insieme di coppie dell'insieme dei vertici, quindi se un grafo ha n nodi, quindi un grafo di dimensione n , non possiamo avere più di n^2 archi.

Questa osservazione ci permette di introdurre la prima rappresentazione di un grafo, che avviene mediante una matrice, chiamata matrice di adiacenza.

Facciamo un esempio per rendere più intuitivo il concetto.

Quello che vogliamo mostrare è che è possibile rappresentare un grafo mediante una matrice nella quale viene valutata la presenza o assenza di una relazione tra una specifica riga ed una specifica colonna.

Sia V l'insieme dei numeri naturali da 1 a 6 (estremi compresi), $V = [1, 6]$, e sia $E = \{(a, b) \in V \times V \mid a \text{ divide } b\}$

	1	2	3	4	5	6
1	(1,1)	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)
2		(2,2)		(2,4)		(2,6)
3			(3,3)			(3,6)
4				(4,4)		
5					(5,5)	
6						(6,6)

Possiamo inoltre rappresentare un grafo mediante una matrice quadrata di dimensione $|V| \times |V|$ di soli valori 0 e 1. Mediante tale matrice possiamo infatti rappresentare la medesima informazione che siamo in grado di mostrare elencando le coppie della relazione E .

Questa è una delle due rappresentazioni che analizzeremo, che viene chiamata rappresentazione tramite matrice di adiacenza. Viene chiamata di adiacenza perché la matrice avrà in posizione (i, j) un 1 se il vertice di posizione i ha un arco, quindi è adiacente, al vertice di posizione j nella rappresentazione matriciale.

	1	2	3	4	5	6
1	1	1	1	1	1	1
2	0	1	0	1	0	1
3	0	0	1	0	0	1
4	0	0	0	1	0	0
5	0	0	0	0	1	0
6	0	0	0	0	0	1

Quindi mediante questa rappresentazione se dobbiamo memorizzare un grafo di n vertici dichiariamo una matrice di dimensione $n \times n$, matrice semplicemente di valori booleani 0, 1.

Vedremo poi un'applicazione in cui questi valori booleani diventeranno dei numeri; per il momento per noi sono soltanto 0, 1 che denotano rispettivamente assenza o presenza di un arco.

Attenzione. Il fatto che ci sia un arco che va da i a j non implica l'esistenza di un arco da j ad i ; se avessimo questa restrizione, cioè che ogni volta che c'è un arco da i a j c'è anche un arco da j a i staremmo rappresentando i cosiddetti grafi non orientati.

Noi generalizziamo e quindi rappresentiamo anche grafi orientati; poi con una struttura come questa che permette di memorizzare grafi orientati ovviamente possiamo memorizzare anche grafi non orientati semplicemente avendo una matrice simmetrica.

Vediamo adesso un ulteriore esempio partendo da un grafo rappresentato in modo human readable mediante nodi e archi.

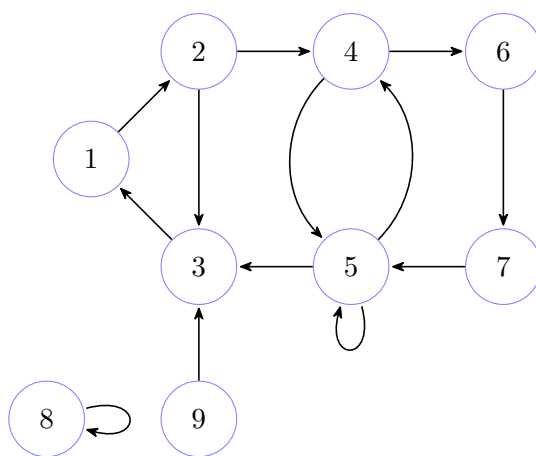


Figura 6.1: Esempio di grafo rappresentato mediante nodi e archi

Di seguito è proposta la sua rappresentazione attraverso la matrice di adiacenza.

	1	2	3	4	5	6	7	8	9
1	0	1	0	0	0	0	0	0	0
2	0	0	1	1	0	0	0	0	0
3	1	0	0	0	0	0	0	0	0
4	0	0	0	0	1	1	0	0	0
5	0	0	1	1	1	0	0	0	0
6	0	0	0	0	0	0	1	0	0
7	0	0	0	0	1	0	0	0	0
8	0	0	0	0	0	0	0	1	0
9	0	0	1	0	0	0	0	0	0

6.2.1 Vantaggi e svantaggi della rappresentazione matriciale

Vediamo quali sono i vantaggi e gli svantaggi della rappresentazione di un grafo mediante matrice di adiacenza.

Il vantaggio di avere una rappresentazione del genere è che possiamo verificare l'esistenza di un arco a tempo costante. Così come anche possiamo modificare il valore di un arco a tempo costante.

Tuttavia modificare un grafo andando ad aggiungere o rimuovere un nodo risulta estremamente dispendioso in quanto comporta il dover riallocare l'intera matrice e quindi ha una complessità quadratica nel numero di vertici, ovvero è $\Theta(|V|^2)$.

Dunque le operazioni di accesso, lettura e scrittura, ad uno specifico arco del grafo hanno complessità $\Theta(1)$; mentre le operazioni di modifica del grafo per inserimento o cancellazione di nodi hanno complessità $\Theta(|V|^2)$.

6.2.2 Grado di un grafo

Faremo alle volte riferimento alla stella uscente di un nodo, cioè i nodi che possono essere raggiunti in un passo da un certo nodo, con $adj(nodo)$.

Inoltre, i nodi appartenenti alla stella uscente di un determinato nodo sono anche detti nodi adiacenti.

Una nomenclatura che spesso useremo è il grado del grafo.

Il grado del grafo è il massimo tra le cardinalità di tutte le stelle uscenti dei nodi del grafo, cioè il massimo numero di archi uscenti da un nodo.

In questa rappresentazione, se volessimo andare a verificare quali sono gli archi uscenti di un certo nodo, come possiamo fare?

Basta indicizzare la riga corrispondente al nodo e scorrerla alla ricerca delle celle con valore 1. Pertanto si tratta di un'operazione lineare nel numero dei nodi, ovvero ha complessità $\Theta(|V|)$.

Se infatti, volessimo sapere quali sono gli archi uscenti del nodo 5 del secondo esempio, ci basta a guardare la riga 5 della matrice e osservare che sono gli archi da 5 a 4 e da 5 verso sé stesso (anche detto self loop).

Quindi esplorare la stella uscente di un determinato nodo costa tempo lineare nel numero di vertici.

Di conseguenza nella rappresentazione matriciale di un grafo, nonostante un grafo possa avere grado basso, siamo costretti, per esplorare una stella uscente, comunque ad impiegare un numero di passi lineare nel numero di vertici, perfino in un grafo completamente sconnesso.

Un grafo sconnesso è un insieme di vertici senza alcun arco.

6.3 Considerazioni sulla rappresentazione matriciale di un grafo

Di conseguenza uno può cominciare a chiedersi se ci sia un modo migliore per rappresentare un grafo.

In verità la rappresentazione mediante la matrice di adiacenza potrebbe essere la migliore, come al solito, dipende.

Se il grafo è particolarmente denso, ossia se il numero di archi è un theta del quadrato del numero di nodi, cioè è proporzionale al quadrato del numero di nodi per un certo fattore non nullo, allora la rappresentazione mediante la matrice di adiacenza è la migliore.

Formalmente.

Un grafo è denso se:

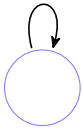
$$|E| = \Theta(|V|^2), \text{ ovvero } \exists c \in \mathbb{R}^+ \left(|E| = c \cdot |V|^2 \right) \quad (6.4)$$

Il caso limite è chiaramente il grafo completo.

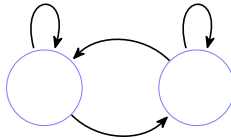
Un grafo si dice completo se ogni vertice ha archi verso tutti i vertici.

Mostriamo qualche esempio.

Grafo Completo con 1 nodo



Grafo Completo con 2 nodi



Grafo Completo con 3 nodi

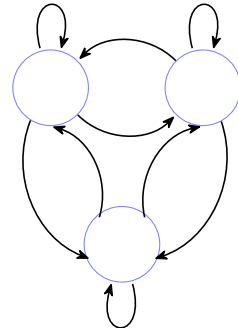


Figura 6.2: Esempio di grafi completi

Sottolineiamo che in verità se il numero di non archi è basso non conviene comunque rappresentare il grafo mediante una matrice. Perché?

Perché possiamo rappresentare la matrice al contrario, invece di avere un 1 quando c'è l'arco avremo un 1 quando non c'è l'arco e 0 quando c'è.

Quindi quando si dice che il grafo è denso, per essere precisi, quello che si intende oltre all'enunciato precedente è anche che la differenza tra il quadrato del numero di nodi e gli archi è theta del quadrato del numero di nodi.

Formalmente.

Un grafo è denso, quindi, se vale:

$$\begin{aligned} |E| = \Theta(|V|^2) &\iff \exists c \in \mathbb{R}^+ \left(|E| = c \cdot |V|^2 \right) \\ \text{AND} \\ |V|^2 - |E| = \Theta(|V|^2) &\iff \exists c \in \mathbb{R}^+ \left(|V|^2 - |E| = c \cdot |V|^2 \right) \end{aligned} \quad (6.5)$$

Cioè non bisogna avere né troppi archi, perché troppi archi significa avere pochi non archi e potremmo memorizzare semplicemente i non archi; né dobbiamo avere invece troppi pochi archi perché a quel punto conviene rappresentare gli archi; ma se siamo in una via di mezzo avremo un grafo che ha in generale $|V|^2/2$ archi e non archi e non possiamo sfruttare nessuno dei due sbilanciamenti.

Quando si dice grafo sparso, cioè il contrario di denso, si intende che il numero di archi è O-grande del numero di nodi.

Formalmente.

Un grafo è sparso se vale:

$$|E| = O(|V|) \quad (6.6)$$

Nel caso di grafi sparsi, la rappresentazione più efficiente non è quella matriciale perché nella matrice andremo a mettere al più un numero lineare rispetto ai nodi di 1 ma una quantità proporzionale al quadrato del numero di nodi di 0; portando a farci occupare molta memoria solo per posizionare un gran numero di 0.

C'è un altro vantaggio della rappresentazione di un grafo mediante una matrice di adiacenza che vale la pena evidenziare prima di passare alla prossima rappresentazione.

Faremo alle volte riferimento alla stella entrante di un nodo, cioè i nodi che raggiungono in un solo passo un certo nodo, con *inc(nodo)*. Inoltre, i nodi appartenenti alla stella entrante di un determinato nodo sono anche detti nodi incidenti.

La rappresentazione matriciale ci dà automaticamente accesso non solo agli adiacenti di un vertice ma anche agli incidenti.

In maniera analoga al caso della ricerca dei archi uscenti, se vogliamo verificare quali sono gli archi entranti in un certo nodo, la cosiddetta stella entrante, in questa rappresentazione ci basterà andare ad indicizzare la colonna corrispondente al nodo in questione e scorrerla alla ricerca di 1.

Anche in questo caso si tratta di un'operazione con complessità di tempo lineare rispetto al numero di vertici, ovvero ha complessità $\Theta(|V|)$.

La seconda rappresentazione che vedremo non avrà questa caratteristica.

6.4 Grafo Trasposto

Prima di passare alla prossima rappresentazione diamo la definizione di grafo trasposto.

Il grafo trasposto è esattamente un grafo con gli stessi vertici ma con gli archi invertiti.

Sottolineiamo, sebbene sia ovvio, che gli adiacenti di un vertice in un grafo sono gli incidenti dello stesso vertice nel grafo trasposto, e viceversa.

Questa proprietà sarà utile per ottenere gli incidenti ad un nodo nella rappresentazione che vedremo a breve.

Nella rappresentazione matriciale per ottenere il grafo trasposto basta fare la trasposta della matrice di adiacenza.

Utilizzando l'esempio di grafo precedente, facciamo il suo trasposto e verifichiamo che prendendo la rappresentazione con matrici di adiacenza del grafo originario, se facciamo la trasposta della matrice, otteniamo la rappresentazione con matrici di adiacenza del grafo trasposto.

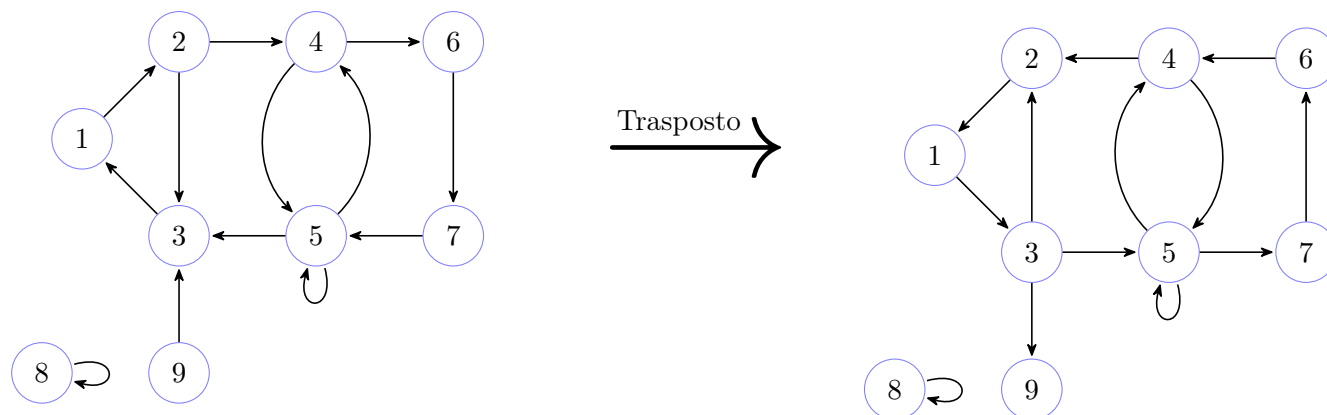


Figura 6.3: Esempio di grafo trasposto

Rappresentazione matriciale del grafo originario

	1	2	3	4	5	6	7	8	9
1	0	1	0	0	0	0	0	0	0
2	0	0	1	1	0	0	0	0	0
3	1	0	0	0	0	0	0	0	0
4	0	0	0	0	1	1	0	0	0
5	0	0	1	1	1	0	0	0	0
6	0	0	0	0	0	0	1	0	0
7	0	0	0	0	1	0	0	0	0
8	0	0	0	0	0	0	0	1	0
9	0	0	1	0	0	0	0	0	0

Trasposta della matrice (matrice di adiacenza del grafo trasposto)

	1	2	3	4	5	6	7	8	9
1	0	0	1	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0
3	0	1	0	0	1	0	0	0	1
4	0	1	0	0	1	0	0	0	0
5	0	0	0	1	1	0	1	0	0
6	0	0	0	1	0	0	0	0	0
7	0	0	0	0	0	1	0	0	0
8	0	0	0	0	0	0	0	1	0
9	0	0	0	0	0	0	0	0	0

6.5 Rappresentazione di un grafo tramite lista di adiacenti

Quale potrebbe essere invece una rappresentazione più efficiente a livello di memoria quando il grafo è sparso?

Pensiamo alla rappresentazione mediante matrice di adiacenza.

Ovviamente una riga in una matrice è un vettore, ma qual è un'altra rappresentazione lineare per le informazioni oltre al vettore? Una lista.

Potremmo quindi immaginare un vettore di vertici e da questi si diparte per ognuno di loro una lista di adiacenti. Questa rappresentazione di un grafo si dice proprio rappresentazione tramite lista di adiacenti.

Sottolineiamo che siamo sempre in un'ottica astratta.

Nell'implementazione concreta le strutture dati per ottenere tale rappresentazione variano in base alle necessità.

Gli algoritmi che vedremo astraggono completamente dalla specifica struttura e si possono quindi applicare in generale.

Mostriamo di seguito, il grafo 6.1 rappresentato con lista di adiacenti.

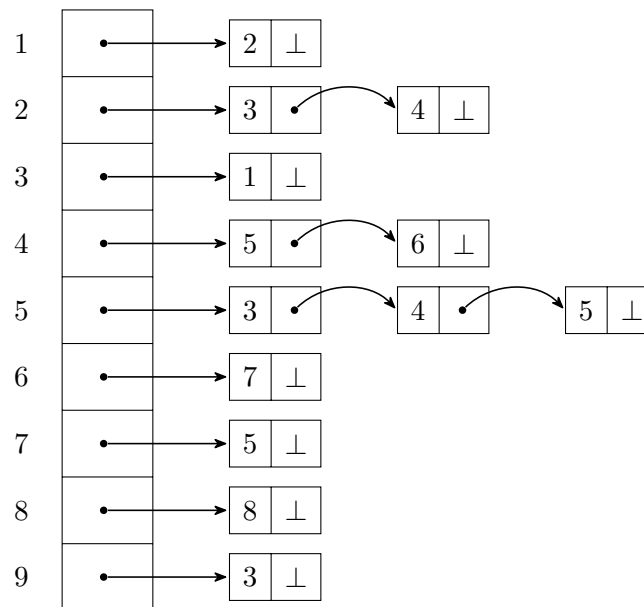


Figura 6.4: Esempio di grafo rappresentato con lista di adiacenti

Attenzione. L'ordine dei nodi nella lista degli adiacenti di un determinato vertice non è significativo.

Ad esempio è possibile rappresentare gli adiacenti del vertice 5 come fatto in figura, oppure scambiando il nodo che contiene 3 con il nodo che contiene 4.

6.5.1 Vantaggi e svantaggi della rappresentazione con lista di adiacenti

Quali sono i vantaggi di una rappresentazione del genere?

È ovvio che qui stiamo rappresentando solo gli archi, non abbiamo alcuna informazione dei non archi; l'informazione c'è ma è implicita, i non archi sono semplicemente ciò che non è presente nelle liste, non li stiamo rappresentando concretamente.

Quindi se abbiamo pochi archi allora si avranno pochi elementi nella rappresentazione con liste di adiacenti, a differenza della rappresentazione con matrice di adiacenza in cui invece c'è una marea di zero che dobbiamo comunque rappresentare.

In effetti, una rappresentazione del genere quanto costa in spazio?

Con la precedente rappresentazione abbiamo visto che serve una memoria che è $\Theta(|V|^2)$.

Con questa rappresentazione, invece non abbiamo come spazio occupato sempre $\Theta(|V|^2)$.

In generale, se abbiamo V vertici e E archi quello che ci serve come spazio è esattamente lineare nella somma delle cardinalità di V ed E .

In termini di formule, abbiamo che il footprint in memoria è $\Theta(|V| + |E|)$

Ovviamente $|V| + |E|$ può essere quadratico rispetto a $|V|$ e in quel caso conviene la rappresentazione tramite matrice visto che il grafo è denso; ma se il grafo è sparso, quindi $|E|$ è lineare in $|V|$, anche $|V| + |E|$ è lineare in $|V|$ e quindi scegliamo la rappresentazione mediante lista di adiacenza.

Quindi uno dei vantaggi è il fatto di evitare la memorizzazione degli 0 (i non archi) che ci permette di avere in generale una complessità spaziale minore.

Vediamo le complessità delle operazioni di inserimento e cancellazione.

Inserire un arco come nella rappresentazione matriciale si può fare a tempo costante (inserimento in testa) quindi ha la stessa complessità, ovvero $\Theta(1)$.

Cancellare un arco ha invece una complessità lineare nella dimensione della stella uscente dell'arco stesso.

La cancellazione o inserimento di un vertice cosa richiederebbe?

Se dobbiamo ad esempio cancellare un vertice bisogna eliminare l'intera lista che si origina da questo poi dobbiamo ovviamente andare a cancellare gli incidenti e poi riallocare il vettore dei vertici, la qual cosa però costa lineare, non quadratico.

Aggiungere un altro vertice significa semplicemente estendere il vettore dei vertici; e abbiamo visto tecniche che permettono che in media tale operazione sia a tempo costante.

Quindi inserimento e cancellazione invece di aver costo quadratico nel numero di nodi hanno costo lineare.

Ci sono due cose che però sono meno efficienti.

In primo luogo identificare se un certo arco esiste. Nella matrice era fatto a tempo costante, qui siamo costretti a scorrere la lista e quindi avrà tempo lineare sul massimo grado del grafo (se il massimo grado del grafo è basso

sebbene la ricerca non sia effettivamente a tempo costante possiamo assumere che lo sia).
Identico ragionamento vale anche per la cancellazione di un certo arco ovviamente.

Altra cosa negativa è che a differenza della rappresentazione con matrice di adiacenza non abbiamo immediato accesso agli incidenti.

Sottolineiamo che è possibile aggiungere informazione a tale rappresentazione per ottenere anche un accesso diretto agli incidenti, ma si tratta di questioni implementative che esulano dallo scopo del corso.

Con questa semplice rappresentazione quello che si deve fare se si volesse avere accesso agli incidenti è costruire il grafo trasposto.

Come si fa a fare il grafo trasposto?

Deve avere esattamente gli stessi vertici, quindi costruiamo un vettore di vertici identico e poi si scorre ogni lista e quando troviamo che c'è un arco da i a j andiamo a inserire dall'altra parte un arco da j ad i .

Quasi sempre saremo agnostici sulla rappresentazione.

6.6 Percorso in un grafo

Un concetto fondamentale all'interno della teoria dei grafi che noi sfutteremo tantissimo è il concetto di percorso.

Un percorso in un grafo è semplicemente una sequenza di nodi adiacenti.

Formalmente un percorso è una sequenza di vertici tale che per ogni vertice interno al percorso esiste un arco da tale vertice al vertice successivo del percorso.

Matematicamente.

$$\pi \in Path(G) \iff \pi \in V^+ (\forall i \in [0, |\pi| - 1) ((\pi)_i, (\pi)_{i+1}) \in E)) \quad (6.7)$$

Dove $Path(G)$ indica l'insieme di tutti i percorsi (path) possibili nel grafo G

In generale dato un grafo i percorsi che si dipanano da un vertice di partenza ad uno di arrivo non è detto siano unici, come non è detto che tra due vertici esista un percorso.

Dobbiamo, inoltre, distinguere i percorsi semplici da quelli non semplici.

Un percorso semplice è un percorso che non attraversa mai un ciclo.

Matematicamente.

$$\pi \in SPath(G) \iff \pi \in Path(G) \wedge \nexists i, j \in [0, |\pi|) ((\pi)_i = (\pi)_j \wedge i \neq j) \quad (6.8)$$

Dove $SPath(G)$ indica l'insieme di tutti i percorsi semplici (simple path) possibili nel grafo G .

Un ciclo può essere definito come un percorso chiuso. Un grafo che non contiene cicli viene detto aciclico.

Formalmente diremo che un percorso è un ciclo se è un percorso e il nodo di partenza e di arrivo sono lo stesso nodo.

Matematicamente.

$$\pi \in Cyc(G) \iff \pi \in Path(G) \wedge first(\pi) = (\pi)_0 = last(\pi) = (\pi)_{|\pi| - 1} \quad (6.9)$$

Dove $Cyc(G)$ indica l'insieme di tutti i percorsi ciclici (cycle) possibili nel grafo G .

È facile dimostrare che ogni volta che un percorso contiene un ciclo esiste un percorso più breve che non contiene tale ciclo, basta rimuovere il ciclo dal percorso lasciando il passaggio per il nodo che inizia e chiude il ciclo stesso.

È sempre possibile pertanto, partendo da un percorso generico, ottenere il percorso semplice associato.

Questo concetto di percorso semplice è molto importante, però attenzione, noi nonostante dichiareremo e andremo ad analizzare tante proprietà dei percorsi, in particolare dei percorsi semplici, non li enumereremo mai.

Non possiamo enumerare i percorsi per cercare di risolvere un problema perché in generale i percorsi in un grafo sono infiniti. E non è possibile nemmeno restringere la nostra attenzione ai soli percorsi semplici in quanto i percorsi semplici sono comunque in generale esponenziali nel numero di vertici.

6.6.1 Esempio famiglia di grafi con numero di percorsi semplici esponenziale

Vediamo ora una famiglia di grafi in cui mancano i cicli ed il numero dei percorsi semplici è esponenziale rispetto al numero di vertici.

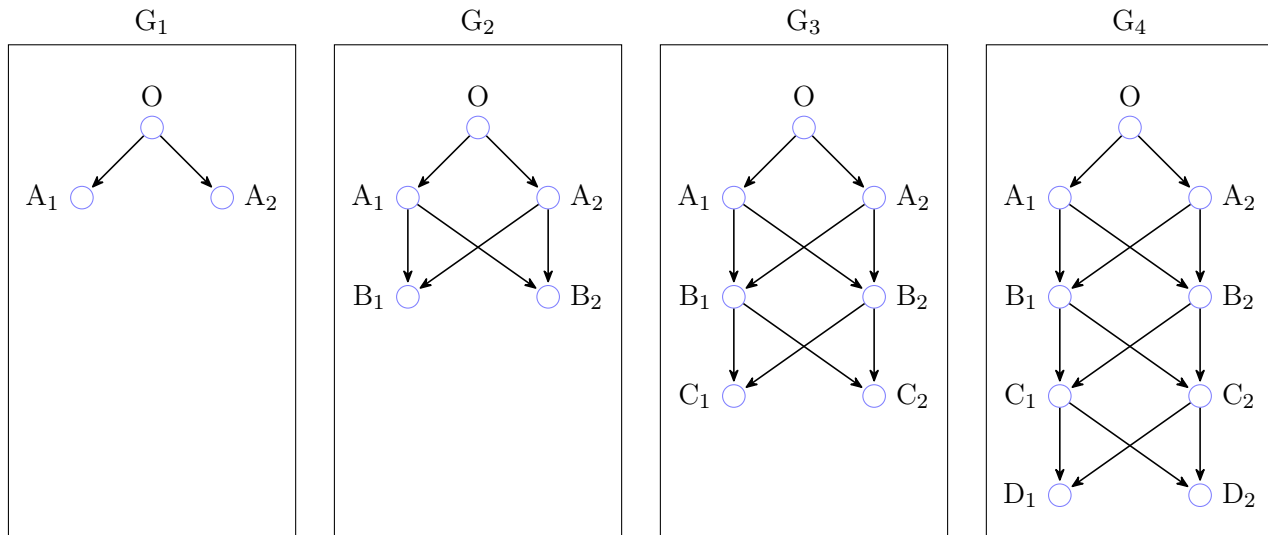


Figura 6.5: Esempio di famiglia di grafi con numero di simple path esponenziale

Dato un i positivo, ovvero $i \in \mathbb{N}^*$, sapendo che in generale chiedere la cardinalità di un grafo equivale a chiedere la cardinalità dell'insieme dei vertici del grafo stesso, possiamo dire che, nel caso di un grafo appartenente a questa famiglia, la sua cardinalità è uguale a due volte i più uno, ovvero $|G_i| = |V_i| = 2i + 1$.

Per quanto riguarda il numero di archi, ovvero $|E|$, possiamo osservare che partendo da G_1 , il cui numero di archi è 2, ad ogni incremento di uno di i notiamo come il numero di archi ha sempre 4 archi in più al precedente, ovvero cresce di 4 in 4 cioè ha un andamento lineare.

$$\text{Quindi } |E| = 2 + 4 \cdot (i - 1) = 4i - 2$$

Osserviamo, dunque, che sia il numero di vertici che di archi è lineare rispetto a i .

Andiamo adesso a considerare il numero di percorsi semplici.

In G_1 i percorsi semplici partendo da O sono 2.

In G_2 i percorsi semplici partendo da O sono 4. In G_3 i percorsi semplici partendo da O sono 8. E così via.

Osservando che la crescita è esponenziale possiamo dire che $|SPth(G_i)| = |Path(G_i)| = 2^i$.

Dove $|SPth(G_i)| = |Path(G_i)|$ vale perché la famiglia considerata, è una famiglia di grafi aciclici.

Attenzione. Se un problema proposto richiede delle proprietà sui percorsi ed é richiesto esplicitamente di trovare un algoritmo che risolva il problema in tempo lineare, non possiamo pensare di costruire l'algoritmo andando ad esplorare i percorsi, poiché potranno essere esponenziali.

Quindi bisognerà sfruttare delle conoscenze sui grafi che vedremo in seguito.

6.7 Raggiungibilità in un grafo

Definiamo ora un'ultima nozione prima di introdurre gli algoritmi sui grafi, si tratta di un concetto strettamente connesso con quello di percorso e viene detto raggiungibilità.

Cioè non vogliamo creare l'insieme di percorsi che si dipartono da un certo vertice, ma ci interessa solo guardare quali sono i vertici che si possono raggiungere, ecco perché è chiamato raggiungibilità, partendo da un certo nodo.

Come abbiamo visto i percorsi possono essere tanti tra due nodi, ma se a noi interessa solo guardare i nodi che possiamo raggiungere non andremo a esplorare tutti i percorsi, non appena troveremo un percorso che connette i due nodi ci possiamo fermare e non esploreremo altri percorsi tra i due nodi.

Questo approccio è fondamentale per mantenere una complessità bassa quando si lavora con i grafi.

Formalmente la relazione di raggiungibilità ($Reach(G)$) è una relazione binaria sull'insieme dei vertici ed è formata dall'insieme di tutte quelle coppie (i, j) tali che esista un percorso dal vertice i al vertice j .

Matematicamente.

$$Reach(G) = \{(v, w) \in V \times V \mid \exists \pi \in Path(G)(first(\pi) = v \wedge last(\pi) = w)\} \subseteq V \times V \quad (6.10)$$

Vediamo ora, prendendo un grafo G , e la relazione di raggiungibilità su di esso, $Reach(G)$, come sia possibile calcolare l'insieme $Reach(G)$.

Tale insieme può essere calcolato semplicemente componendo la relazione di transizione tante volte con se stessa fino a coprire tutto ciò che è possibile coprire.

Inoltre, nonostante nella descrizione formale della relazione di raggiungibilità non si faccia riferimento a percorsi semplici, sappiamo che se esiste un percorso tra due vertici di un grafo esisterà necessariamente un percorso semplice tra i due suddetti vertici.

Questa osservazione risulta fondamentale in quanto in un grafo con n nodi i percorsi semplici non possono essere più lunghi di n ovviamente, in quanto altrimenti ci sarebbe necessariamente un ciclo, e quindi andando a comporre la relazione di arco del grafo con se stessa al più n volte avremo totalmente definito l'insieme di tutti i nodi del grafo che possono essere raggiunti.

Non sarà necessario quindi comporre tale relazione con se stessa per più di n volte.

Di seguito è proposto un grafo che sfrutteremo successivamente per degli esempi sulla raggiungibilità.

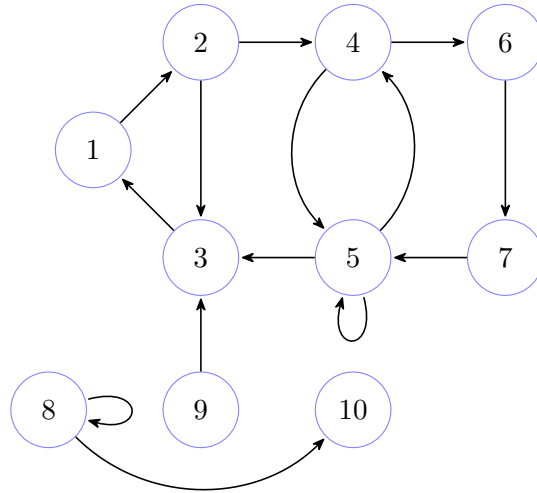


Figura 6.6: Esempio 2 di grafo

Sappiamo per definizione E^0 è l'identità.

Formalmente.

$$E^0 = \{(v, v) \mid v \in V\} \quad (6.11)$$

Se andiamo a prendere i percorsi di lunghezza 0, quindi, saranno fatti di singoli nodi.

Attenzione. Stiamo considerando la lunghezza di un percorso come il numero di archi di cui è composto; è possibile anche considerare il numero di vertici.

Per quanto riguarda E^1 , cioè E , esso è formato dagli archi semplici.

Quindi in E^1 avremo esattamente le coppie di vertici della relazione E del grafo.

Si tratta proprio dei percorsi di lunghezza 1.

Possiamo definire in generale E^i .

$$E^i = \begin{cases} Id & , \text{ se } i = 0 \\ E^{i-1} \circ E & , \text{ altrimenti} \end{cases} \quad (6.12)$$

Dove per $\circ$ si intende la composizione tra due relazioni, definita nel seguente modo:

$$(v, w) \in E^{i-1} \circ E \iff \exists u \in V ((v, u) \in E^{i-1} \wedge (u, w) \in E) \quad (6.13)$$

Dove $i \in \mathbb{N}^*$

Facciamo adesso un esempio pratico sull'insieme E^2 utilizzando il grafo disegnato precedentemente.

Per la definizione appena data possiamo dire che:

$$(v, w) \in E^2 \iff \exists u \in V ((v, u) \in E^1 \wedge (u, w) \in E) \quad (6.14)$$

L'insieme E^2 sarà quindi uguale a:

$$E^2 = \{(1, 3), (1, 4), (2, 1), (2, 6), (2, 5), (3, 2), (4, 3), (4, 4), (4, 7), (5, 1), (5, 5), (5, 6), \\ (6, 5), (7, 3), (7, 4), (8, 8), (8, 10), (9, 1)\}$$

E^2 è quindi formato da tutte quelle coppie di vertici che sono rispettivamente sorgente e destinazione di un percorso di lunghezza 2.

Quindi per ottenere l'insieme Reach ci basta calcolare i vari insiemi E^i con i che va da 0 ad n e poi effettuarne l'unione.

Non sarà necessario comporre la relazione E con se stessa più di n volte visto che come abbiamo già detto, se esiste un percorso tra due vertici è sempre possibile ottenere un percorso semplice tra gli stessi vertici.

Si può quindi dimostrare facilmente per induzione che l'insieme Reach si può formalizzare nel seguente modo:

$$Reach(G) = \bigcup_{k \in [0, n)} E^k \quad (6.15)$$

È semplice ottenere una rappresentazione dell'insieme E^i di un grafo nel caso della rappresentazione mediante la matrice di adiacenza, basta infatti effettuare il prodotto riga per colonna della matrice che rappresenta il grafo per se stessa per i volte, con l'accortezza che qualora il valore calcolato per una certa posizione (k, j) fosse maggiore di 0 venga tramutato in 1.

Sottolineiamo che se non viene tramutato il valore calcolato in 1, esso rappresenta il numero di percorsi semplici di lunghezza i aventi origine il nodo k e destinazione il nodo j .

Vediamo un esempio di calcolo della raggiungibilità nel caso di un grafo rappresentato con matrice di adiacenza.

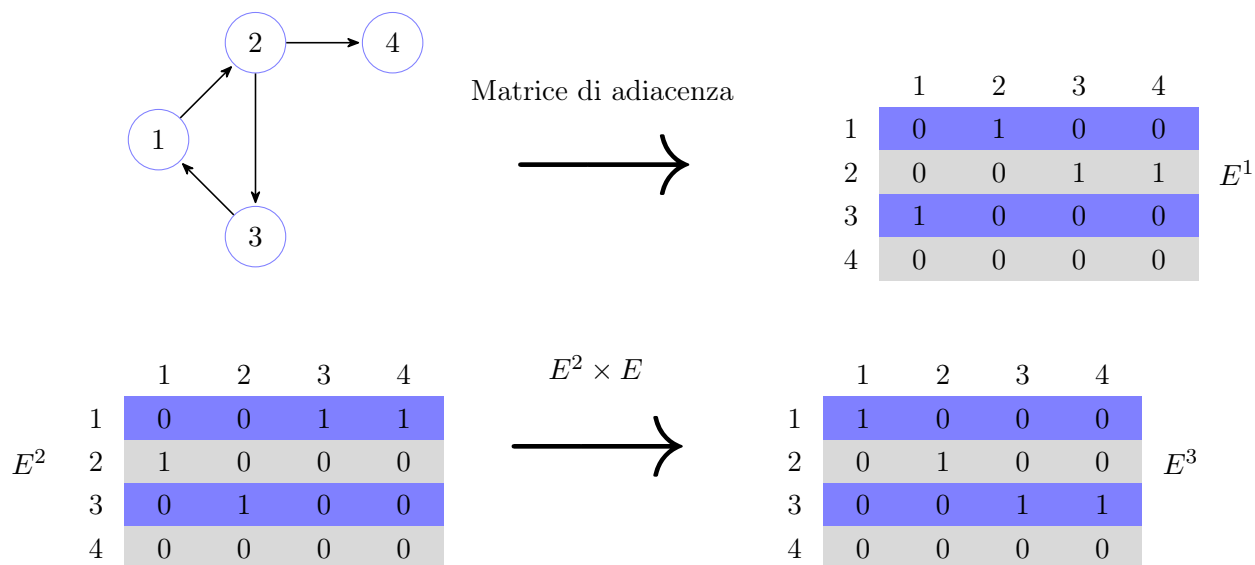


Figura 6.7: Esempio di calcolo della raggiungibilità con prodotto tra matrici

Vediamo ora un modo più efficiente per calcolare l'insieme *Reach* dato un grafo, in quanto il prodotto tra matrici ha una complessità cubica nella dimensione della matrice, sebbene esistano algoritmi migliori ma che sono almeno quadratici.

Il metodo che vediamo ora risulta essere, invece, lineare nella dimensione del grafo.

Vedremo infatti che è possibile non effettuare tanti calcoli che risultano effettivamente inutili per i nostri scopi, portando la complessità di tempo ad essere lineare.

Nello specifico se poi ci interessa verificare quali sono i percorsi che si dipartono da un certo nodo, in particolare quindi partendo da un nodo quali nodi possiamo raggiungere, allora possiamo fare molto di meglio.

6.8 Relazione di connessione in un grafo

Una volta compreso il concetto di *Reach* ovviamente possiamo immediatamente capire se il grafo è una componente connessa.

Definiamo prima ovviamente il concetto di connessione per un grafo.

Un grafo non orientato viene detto connesso se ogni vertice del grafo raggiunge ogni altro vertice.

Abbiamo specificato non orientato perché quando abbiamo un grafo non orientato un arco è percorribile ambo i versi, cosa non vera come sappiamo per i grafi orientati.

Dato un grafo orientato possiamo completarlo per ottenere il grafo non orientato ad esso associato.

Quindi un grafo non orientato viene detto connesso se c'è un'unica componente cioè se tutti i vertici vanno dappertutto, altrimenti è decomponibile in componenti connesse massimali, cioè quelle che non possono essere estese ulteriormente perché altrimenti non sono connesse.

Formalmente.

$$G \text{ non orientato è connesso} \iff \forall v, w \in V((v, w) \in Reach(G)) \quad (6.16)$$

Quando però consideriamo gli archi per il loro verso e quindi il grafo orientato, la situazione è leggermente diversa e si ottiene il concetto di componente fortemente connessa.

In un grafo orientato una componente fortemente connessa è un insieme di nodi per cui da ogni nodo possiamo raggiungere ogni altro, seguendo la direzione degli archi orientati.

Diremo che una componente fortemente connessa è banale se è formata da un singolo nodo senza possibilità di percorsi non nulli.

$$G \text{ orientato è fortemente connesso} \iff \forall v, w \in V((v, w) \in Reach(G) \wedge (w, v) \in Reach(G)) \quad (6.17)$$

Se un grafo è fortemente connesso, allora è anche connesso; il viceversa non è vero.

Vediamo adesso qualche esempio di grafo non orientato connesso, e grafo orientato fortemente connesso.

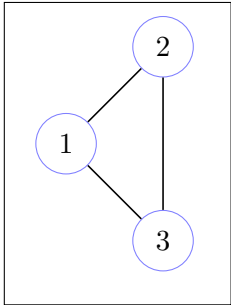
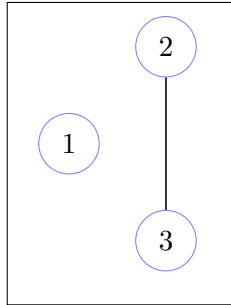
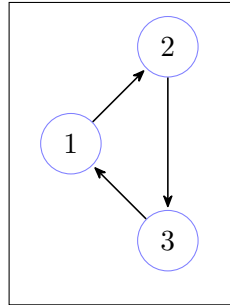
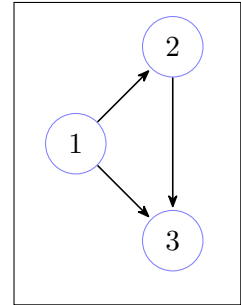
Grafo non orientato
connessoGrafo non orientato
non connessoGrafo orientato
fortemente connessoGrafo orientato
non fortemente connesso

Figura 6.8: Esempi di grafi connessi e fortemente connessi

Introduciamo che le componenti fortemente connesse sono tra di loro parzialmente ordinate nel senso che da alcune possiamo andare in altre, ma non possiamo effettuare lo spostamento nel verso opposto.

Diamo infine il concetto di sottografo.

Dato $G = \langle V, E \rangle$ e $G' = \langle V', E' \rangle$.

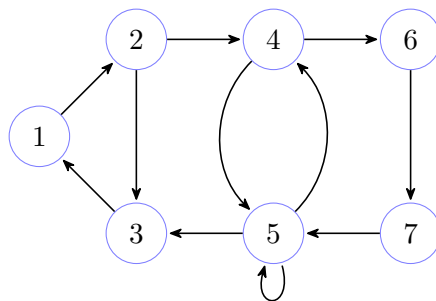
$$G' \text{ è sottografo di } G \iff V' \subseteq V \wedge E' \subseteq E \quad (6.18)$$

Esiste anche un altro concetto di sottografo, quello di sottografo indotto, che è il più grande sottografo, fissati dei vertici, che un grafo possa avere.

$$G' \text{ è sottografo indotto di } G \iff V' \subseteq V \wedge E' = E \cap V' \times V' \quad (6.19)$$

Vediamo adesso qualche esempio di sottografo e sottografo indotto.

Utilizzeremo come base il seguente grafo, che indicheremo con G_{ex} .



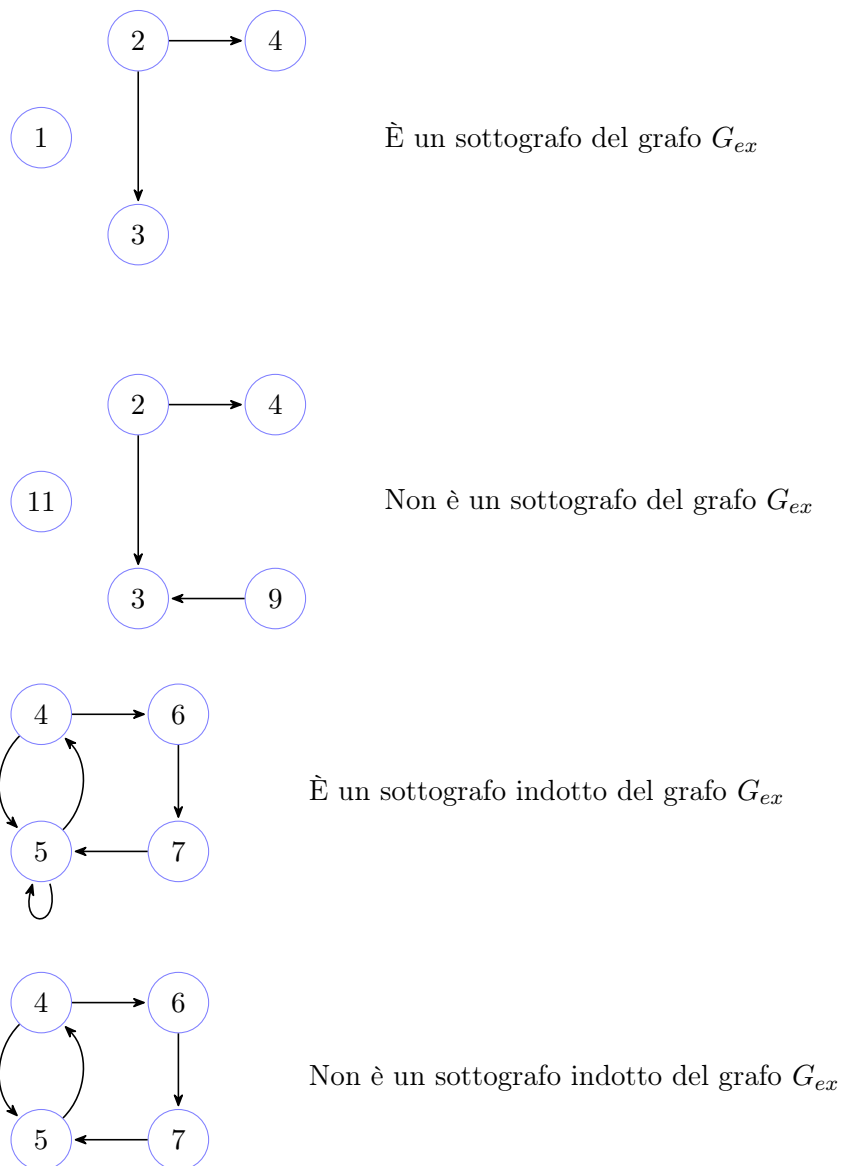


Figura 6.9: Esempi di sottografi e sottografi indotti

Accenniamo soltanto al concetto di similitudine tra grafi.

Si tratta di un concetto fondamentale e con numerose applicazioni pratiche. Esistono diverse definizioni di similitudine, noi considereremo due grafi simili se esiste un isomorfismo tra di loro.

È un problema difficile, di cui attualmente non è nota l'effettiva complessità.

6.9 Distanza in un grafo

Un'ultima nozione importante e necessaria prima di affrontare gli algoritmi è il concetto di distanza.

Diremo che la distanza tra due nodi di un grafo è la lunghezza (intesa come numero di archi) del percorso semplice minimale tra di essi.

Formalmente.

Se stiamo considerando con $|\pi|$ il numero di archi, la distanza tra due nodi $v, w \in V$ (che indicheremo con $\delta(v, w)$) è:

$$\delta(v, w) = \min_{\pi \in Path(G, v, w)} |\pi| \quad (6.20)$$

Se stiamo considerando con $|\pi|$ il numero di nodi allora è:

$$\delta(v, w) = \min_{\pi \in Path(G, v, w)} |\pi| - 1 \quad (6.21)$$

Ricordiamo che con $Path(G)$ stiamo prendendo l'insieme di tutti i possibili percorsi in G ; con $Path(G, v)$ l'insieme di tutti i percorsi che originano da v ; con $Path(G, v, w)$ l'insieme di tutti i percorsi che originano da v e terminano in w .

Inoltre, nel caso di $v, w \in V$ con $(v, w) \notin Reach(G)$ la distanza tra questi due nodi è infinito, ovvero $\delta(v, w) = \infty$

Esempio di matrice delle distanze

Vediamo la matrice delle distanze per il grafo 6.6.

Ovviamente tale matrice sarà simmetrica per i grafi non orientati.

	1	2	3	4	5	6	7	8	9	10
1	0	1	2	2	3	3	4	∞	∞	∞
2	2	0	1	1	2	2	3	∞	∞	∞
3	1	2	0	3	4	4	5	∞	∞	∞
4	3	4	2	0	1	1	2	∞	∞	∞
5	2	3	1	1	0	2	3	∞	∞	∞
6	4	5	3	3	2	0	1	∞	∞	∞
7	3	4	2	2	1	3	0	∞	∞	∞
8	∞	∞	∞	∞	∞	∞	∞	0	∞	1
9	2	3	1	4	5	5	6	∞	0	∞
10	∞	∞	∞	∞	∞	∞	∞	∞	∞	0

Per individuare in tale matrice due vertici (i, j) che fanno parte della stessa componente fortemente connessa basterà valutare che entrambe le celle (i, j) e (j, i) siano diverse da infinito, in caso positivo i vertici fanno parte della stessa componente fortemente connessa.

6.10 Visita in ampiezza su un grafo

La prima visita che andremo ad esplorare è la visita in ampiezza, cioè la visita che parte da un vertice ed esplora il grafo non andando lungo un percorso fino in fondo per poi tornare indietro ma cerca di esplorare il grafo in ampiezza esplorando tutto il vicinato prima e poi si sposta a un vicinato leggermente più ampio e così via.

Si tratta cioè di visitare il grafo partendo da un certo vertice esplorando l'orizzonte sempre più ampio, verificando l'orizzonte di distanza zero immediatamente, di distanza uno esplorando tutti i vertici a distanza uno, poi a distanza due, e così via.

Facciamo un esempio di visita in ampiezza su un grafo proposto in un precedente esempio.

Dato il seguente grafo 6.6, simuliamo la visita in ampiezza partendo dal vertice 1.

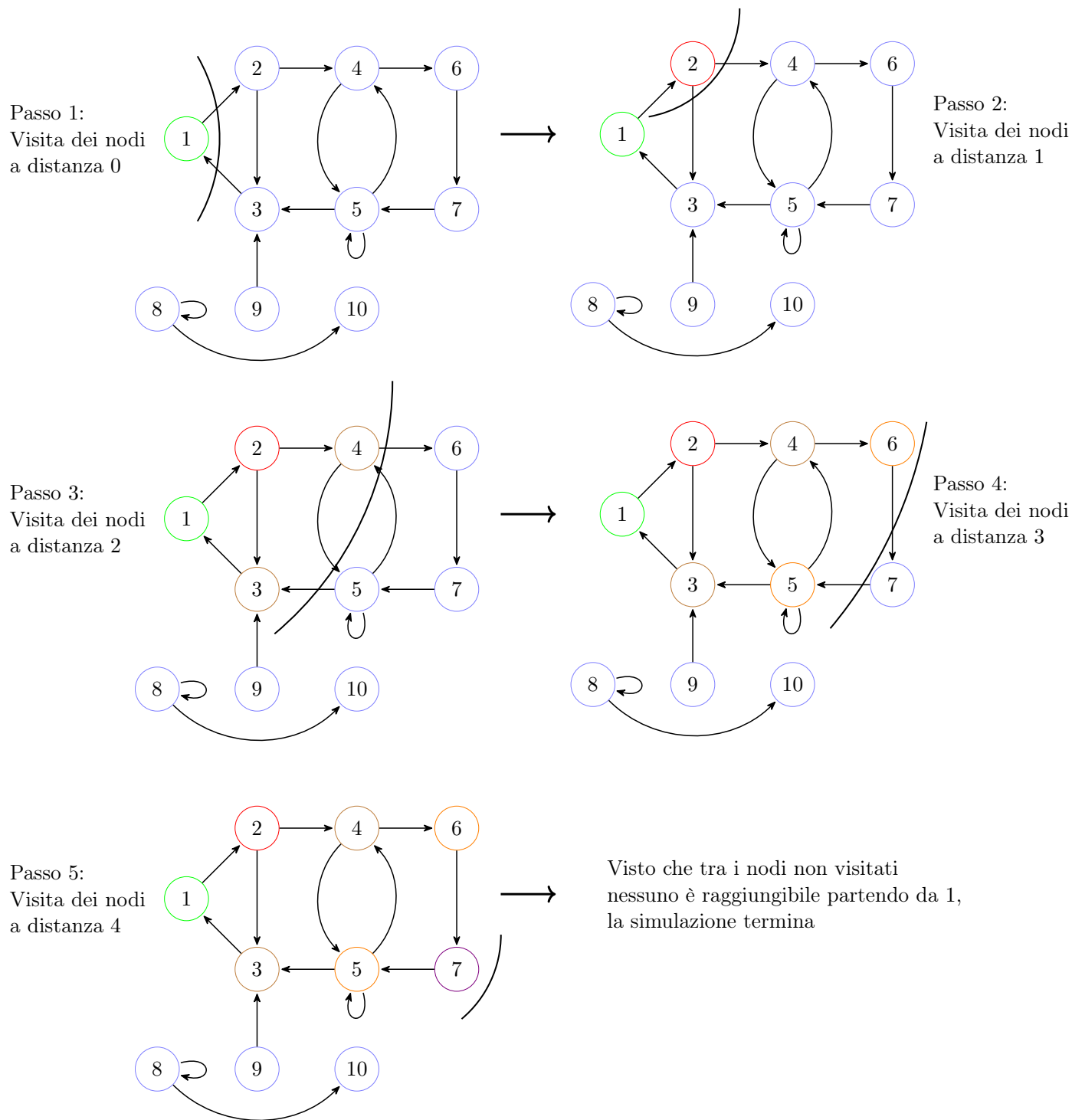


Figura 6.10: Esempio di visita in ampiezza su un grafo

Legenda:

- I nodi con bordo blu sono, i nodi con distanza da computare;
- I nodi con bordo verde sono, i nodi a distanza 0;
- I nodi con bordo rosso sono, i nodi a distanza 1;
- I nodi con bordo marrone sono, i nodi a distanza 2;
- I nodi con bordo arancione sono, i nodi a distanza 3;
- I nodi con bordo violetto sono, i nodi a distanza 4.

È chiaro che, se nel corso della visita si incontra un vertice già visitato si potrà non considerare tale percorso. Se quindi ad un certo punto tutti i percorsi ci riportano a nodi precedentemente visitati o non ci sono più percorsi disponibili, possiamo interrompere la visita.

In generale questo è possibile memorizzando informazioni sull'ultimo livello di ampiezza visitato, in quanto il nostro scopo è trovare percorsi semplici di dimensione via via crescente e quindi non ci interessa tenere traccia di livelli precedenti all'ultimo visitato.

Questo ci permette di limitare i calcoli inutili ed è dal punto di vista concettuale il motivo per il quale la complessità di tale algoritmo riesce ad essere lineare.

Notiamo che il fatto di non visitare nodi già visitati ci garantisce che l'algoritmo vada a termine, non entrando in loop dal quale non sarebbe possibile uscire.

In effetti, ricordare cosa sia stato visitato rispetto a ciò che ancora si deve visitare è fondamentale non solo per assicurare la terminazione dell'algoritmo ma anche per garantire una complessità lineare nel numero di nodi.

A noi interessa solo se è possibile raggiungere un certo nodo a partire da un altro nodo; pertanto una volta trovato un percorso non ha senso esplorarne altri, che per altro sarebbero anche di distanza maggiore, pertanto il primo percorso individuato tra due vertici è anche quello a distanza minore.

Per distinguere i vertici visitati da quelli non visitati faremo uso di una colorazione. Di solito utilizzeremo bianco per indicare un nodo che non è stato mai visitato; nero per dire che si tratta di un nodo che abbiamo scoperto e visitato; grigio per indicare un nodo che è stato solo scoperto ma non visitato.

Quando parliamo di nodo visitato significa che è stato effettuato del lavoro su quel nodo.

6.10.1 Algoritmo: Breadth First Search in un grafo

Ecco l'algoritmo di visita in ampiezza di un grafo.

Esso, prende in input un grafo G e il nodo v dal quale deve originare la visita.

Algoritmo 80: Breadth First Search in un grafo: BFS

```

1 function BFS(G, v)
    // c, d, p sono array lunghi quanto la cardinalità del grafo
    // c è un array di colori
    // d è un array di distanze dal nodo in input
    // p è un array di nodi che dice qual è il predecessore di un nodo
    // sul percorso minimo a partire dal nodo in input
    // tali array vengono inizializzati mediante una funzione accessoria
2   (c, d, p) ← init(G)
    // Inizializzo la coda inserendo il nodo in input
    // setto il colore del nodo a grigio in quanto è stato scoperto
    // ma non è ancora stato effettuato del lavoro
    // e la distanza da se stesso a 0
    // non setto il predecessore in quanto il minimo percorso è di lunghezza 0
    // e quindi ⊥ è corretto
3   (Q, c[v], d[v]) ← (enqueue(∅, v), GRAY, 0)
    // visto che per costruzione la coda non è vuota
    // utilizziamo un costrutto repeat until
4   repeat
    // prendo l'elemento in testa alla coda
5     (Q, v) ← head_&_dequeue(Q)
    // per ciascun nodo nella stella uscente del nodo appena prelevato
6     foreach (w in Adj[v]) do
    // se non è mai stato scoperto
7       if (c[w] = WHITE) then
    // lo inserisco in coda, setto che è stato scoperto,
    // setto la distanza come la successiva della corrente
    // e indico che il predecessore è il nodo di cui stiamo
    // valutando la stella uscente
8       (Q, c[w], d[w], p[w]) ← (enqueue(Q, w), GRAY, d[v] + 1, v)
    // una volta valutata la stella uscente del nodo
    // abbiamo terminato il lavoro da effettuare e possiamo indicare
    // che è stato completamente visitato colorandolo di nero
9     c[v] ← BLACK
    // fino a quando la coda non è vuota
10  until (is_empty(Q))
    // restituisco i tre vettori
    // il vettore c darà indicazione se un nodo è raggiungibile o meno
    // se colorato di bianco non raggiungibile, nero raggiungibile
    // grigio non è possibile perché una volta scoperto il nodo verrà visitato
    // il vettore d ci darà la distanza minima tra un nodo e quello in input
    // il vettore p ci darà indicazione effettiva del percorso minimo
11  return (c, d, p)

```

Di seguito mostriamo la funzione accessoria `init` che prende in input un grafo G

Algoritmo 81: Funzione di inizializzazione (BFS)

```
1 function init( $G$ )  
    // la funzione di inizializzazione dei vettori di colori,  
    // distanze e predecessori  
    // setta inizialmente per ogni nodo  
2 foreach ( $v$  in  $V$ ) do  
    // il colore a bianco, cioè nodo non visitato  
    // la distanza ad infinito, assumendo non esista un percorso  
    // e il predecessore nel percorso minimo a  $\perp$   
3    ( $c[v]$ ,  $d[v]$ ,  $p[v]$ )  $\leftarrow$  (WHITE,  $\infty$ ,  $\perp$ )  
    // resituisce i vettori inizializzati  
4 return ( $c$ ,  $d$ ,  $p$ )
```

6.10.2 Algoritmo: Percorso minimo in un grafo

Vediamo ora un algoritmo che è fatto di due parti, una parte non ricorsiva ed una parte ricorsiva, che è quello che estrae dal vettore dei precedenti il percorso minimo, se esiste, che porta da un vertice v ad un qualunque vertice w .

Il percorso minimo viene restituito sotto forma di uno stack.

In pratica quello che restituirà sarà uno stack, in cui in testa ci sarà il nodo dal quale origina il percorso, e man mano, togliendo dallo stack i vari vertici, raggiungeremo quello di destinazione, che sarà posto in fondo allo stack.

Questa funzione prende in input il grafo G , il nodo di partenza v e il nodo di arrivo w .

Algoritmo 82: Percorso minimo in un grafo

Signature `min_path`: $Graph(D) \times V \times V \rightarrow Stack(D)$

```

function min_path(G, v, w)
    // inizializzo uno stack vuoto
1   S ← empty_stack()
    // riempio i vettori di colorazione, distanza e predecessore
    // mediante una visita in ampiezza a partire dal nodo di origine
    // non ci interessa in tale algoritmo dell'array delle distanze
2   (c, _, p) ← BFS(G, v)
    // se esiste un percorso dal nodo di origine a quello di destinazione
3   if (c[w] = BLACK) then
        // mediante una funzione accessoria costruisco il percorso minimo
        // che inserisco nello stack
4       S ← build_min_path(S, p, v, w)
    // restituisco lo stack
5   return S

```

La funzione `build_min_path` prende in input lo stack da popolare, il vettore dei predecessori, il nodo di partenza e quello di arrivo.

Intuitivamente, partendo dal presupposto che il nodo di destinazione sia nero, scorre il vettore dei predecessori a ritroso fino a ritrovare il nodo di partenza.

A questo punto ha ricostruito il percorso.

Ecco l'algoritmo.

Algoritmo 83: Funzione per la costruzione del percorso minimo in un grafo

Signature `build_min_path`: $Stack(D) \times Array(D) \times V \times V \rightarrow Stack(D)$ **function** `build_min_path`(`S`, `p`, `v`, `w`)

```
1  // inserisco nello stack il nodo di destinazione corrente
   S  $\leftarrow$  push(S, w)
   // se non ho ancora raggiunto l'origine del percorso
2  if (v  $\neq$  w) then
   |   // chiamo ricorsivamente la funzione avanzando a ritroso lungo
   |   // il vettore dei predecessori
3  |   S  $\leftarrow$  build_min_path(S, p, v, p[w])
   |   // restituisco lo stack
4  return S
```

6.10.3 Correttezza dell'algoritmo BFS

Ora dobbiamo però dimostrare che questi algoritmi siano corretti.

Dobbiamo cioè dare delle proprietà formali che stiano ad indicare che effettivamente quello che calcola l'algoritmo BFS, cioè i vettori c , d e p , siano effettivamente dei valori che hanno senso.

In particolare in c , che denota la colorazione, ci aspettiamo che un vertice sia colorato nero se e soltanto se quel vertice è raggiungibile dall'origine.

In d , il vettore delle distanze, ci aspettiamo che i valori contenuti indichino esattamente le distanze minime dei vertici dal vertice v di origine.

Ci aspettiamo, poi, che p sia effettivamente il vertice dei predecessori, cioè che appunto esista un percorso minimo in cui p indica esattamente il predecessore di ogni vertice che andiamo a campionare.

Il seguente teorema andrà a formalizzare esattamente le proprietà appena descritte.

La prima sarà una proprietà sulla colorazione, la seconda sulle distanze e la terza sui predecessori.

Fissato un grafo G ed un vertice v e l'algoritmo BFS.

La prima proprietà dice che per ogni vertice w all'interno dell'insieme V di vertici del grafo G , w è raggiungibile da v se e solo se alla fine dell'algoritmo $c[w]$ sarà colorato di nero.

Quindi se un percorso esiste l'algoritmo BFS deve dircelo, tale proprietà viene detta completezza; allo stesso tempo se qualcosa ci viene detto dall'algoritmo quel qualcosa è vero, cioè l'algoritmo è corretto.

La seconda proprietà dice che per ogni vertice w in V , alla fine dell'algoritmo, $d[w]$ sia proprio la distanza tra il vertice v e il vertice w .

Infine, la terza proprietà dice che per ogni vertice w in V , se $p[w]$ è diverso da $\perp$ allora $p[w]$ è il predecessore di w lungo un percorso minimo da v a w .

Per dimostrare questo teorema avremo bisogno di alcuni lemma preliminari.

Il primo lemma descrive una proprietà molto semplice. Afferma la seguente cosa.

Se prendiamo tre vertici dell'insieme di vertici V di un grafo G e tra due di questi c'è un arco, allora se andiamo a prendere la distanza tra l'altro vertice e la destinazione di questo arco questa non può essere superiore alla distanza dello stesso vertice dall'origine dell'arco più uno (distanza tra due vertici connessi da un arco).

Mostriamo graficamente la condizione del lemma da dimostrare.

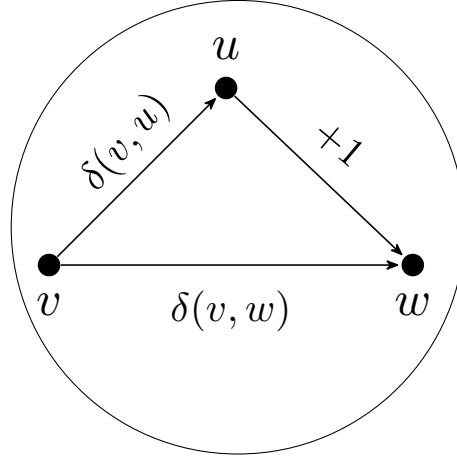


Figura 6.11: Esempio grafico del lemma delle distanze dei grafi semplici

Formalmente.

Lemma 6.10.1 (Distanze grafi semplici).

$$\forall v, w, u \in V. \quad (u, w) \in E \implies \delta(v, w) \leq \delta(v, u) + 1.$$

Dimostrazione.

Dimostriamo la tesi per assurdo; quindi supponiamo $\delta(v, w) > \delta(v, u) + 1$.

Sia $\pi \in \text{MinPath}(G, v, w)$. Conseguentemente, considerando la lunghezza dei percorsi nel numero dei nodi, avremo $|\pi| = \delta(v, w) + 1$.

Ovviamente, dall'ipotesi, $\exists \pi' \in \text{MinPath}(G, v, u)$ ($\pi' \circ w \in \text{Path}(G, v, w)$).

Considerando le lunghezze dei percorsi

$$|\pi' \circ w| = |\pi'| + 1 = \delta(v, u) + 2$$

Chiaramente segue immediatamente la contraddizione con la supposizione assurda

$$\delta(v, w) + 1 \leq \delta(v, u) + 2 \iff \delta(v, w) \leq \delta(v, u) + 1$$

□

Il secondo lemma, a differenza del primo che definisce una proprietà astratta, darà delle proprietà relative all'algoritmo.

Si compone di due punti.

Il punto a) afferma che per ogni nodo w nell'insieme dei vertici V di un grafo G il valore di $d[w]$ al termine dell'algoritmo BFS è una sovrastima della distanza dal nodo di origine, v , a w .

Il punto b) definisce una proprietà dei vertici contenuti in coda in un qualunque momento di esecuzione dell'algoritmo.

Sia Q una istantanea della coda Q durante l'esecuzione di $\text{BFS}(G, v)$.

I valori che l'algoritmo assegna al vettore d dei vertici contenuti in coda sono monotoni crescenti, e non solo, non potranno essere distanziati di un valore troppo grande, lo scarto massimo potrà essere al più di una unità.

Ricordiamo che l'algoritmo BFS assegna i valori delle distanze ai nodi prima di inserirli in coda, ed una volta che le distanze vengono assegnate non verranno modificate.

Formalmente.

Lemma 6.10.2 (Invarianti BFS).

Considerando l'esecuzione dell'algoritmo $\text{BFS}(G, v)$

- a) $\forall w \in V (d[w] \geq \delta(v, w))$;
- b) Sia $Q = v_0 v_1 \dots v_{k-1} v_k$, successione monotona crescente, una istantanea della coda Q durante l'esecuzione di $\text{BFS}(G, v)$.
 - i) $\forall i \in [0, k) (d[v_i] \leq d[v_{i+1}])$;
 - ii) $d[v_k] \leq 1 + d[v_0]$

Dimostrazione.

Dimostriamo in prima battuta il punto a) per assurdo.

Ricordiamo che il settaggio iniziale delle distanze è a ∞ ; pertanto all'inizio dell'algoritmo la proprietà è banalmente verificata.

Supponiamo, per assurdo, che $\exists w \in V (d[w] < \delta(v, w))$. E sia w^* il primo nodo (in ordine temporale) con tale caratteristica.

Necessariamente $\exists u \in V (w^* \in \text{Adj}[u])$. E chiaramente u è stato scoperto prima di w^* , pertanto $d[u] \geq \delta(v, u)$.

Dalla supposizione assurda e dal lemma precedente segue la contraddizione.

$$\delta(v, w^*) > d[w^*] = d[u] + 1 \geq \delta(v, u) + 1 \geq \delta(v, w^*)$$

Passiamo, ora, a dimostrare il punto b) per induzione sul numero di iterazioni, che indicheremo con I , del ciclo **repeat...until** dell'algoritmo BFS.

Il caso base risulta banale. Quando $I = 0$ avremo $Q = v_0$ e chiaramente il punto i) è vacuamente verificato, ed il punto ii) banalmente verificato.

Passiamo, quindi, al caso induttivo.

Sottolineiamo che ci sono due situazioni nelle quali la proprietà potrebbe essere violata: quando andiamo a rimuovere un elemento dalla coda, o quando andiamo ad inserire un elemento.

Sia $Q = v_0 v_1 \dots v_{k-1} v_k$ la coda prima di essere modificata. Pertanto, per ipotesi induttiva la proprietà risulta verificata. Analizziamo separatamente il caso in cui al passo successivo avviene una rimozione, e il caso in cui al passo successivo avviene un inserimento. Per semplificare la trattazione indicheremo con Q' la coda a seguito della modifica.

- Rimozione v_0 da $Q \implies Q' = v_1 \dots v_{k-1} v_k$.

In generale, una proprietà universale su un insieme non può essere violata rimuovendo un elemento di tale insieme.

Infatti, che il punto i) rimanga verificato è banale.

Similmente, il punto ii) rimane vero

$$d[v_k] \leq 1 + d[v_0] \leq 1 + d[v_1]$$

- Inserimento v_{k+1} in $Q \implies Q' = v_1 \dots v_{k-1} v_k v_{k+1}$

Consideriamo v_0 il nodo precedentemente rimosso.

In questo caso basta considerare l'algoritmo.

Quando un nodo viene inserito in coda necessariamente sarà un adiacente del nodo che si trova in prima posizione nella coda stessa; pertanto avremo

$$d[v_{k+1}] = d[v_0] + 1 \geq d[v_1] + 1 \geq d[v_1] \geq d[v_k]$$

□

Possiamo finalmente dimostrare il teorema.

Teorema 6.10.1 (Correttezza BFS(G, v)).

1. $\forall w \in V ((v, w) \in \text{Reach}(G) \iff c[w] = \text{BLACK});$
2. $\forall w \in V (d[w] = \delta(v, w));$
3. $\forall w \in V (p[w] \neq \perp \implies \exists \pi \in \text{MinPath}(G, v, w) ((\pi)_{|\pi|-2} = p[w]));$

Dimostrazione.

La dimostrazione verrà separata in due parti:

- Caso in cui il vertice w è a distanza infinita, $\delta(v, w) = \infty$
- Caso in cui il vertice w è a distanza finita, $\delta(v, w) < \infty$

Consideriamo prima il caso in cui il vertice w è a distanza infinita.

Come sappiamo, se $\delta(v, w) = \infty$ significa che il vertice w non è raggiungibile dal vertice v .

Supponiamo, per assurdo, che il vertice w venga colorato di nero, quindi $c[w] = BLACK$ (chiaramente non potrà essere colorato di grigio in quanto come sappiamo alla fine dell'algoritmo un nodo o è bianco o è nero).

Guardiamo l'algoritmo.

Se w viene colorato di nero significa che un qualche nodo u lo deve aver inserito in coda, cioè $w \in Adj[u]$; ma questo comporta il fatto che $d[w] = d[u] + 1 \neq \infty$. Inoltre, visto che il primo nodo inserito è chiaramente v significa che v raggiunge w , visto che andando a ritroso da u necessariamente arriviamo a v .

Chiaramente assurdo per il lemma precedente.

Abbiamo in questo modo provato i punti 1 e 2 del teorema. Il punto 3 è vacuamente vero.

Consideriamo, adesso, il caso in cui il vertice w è a distanza finita.

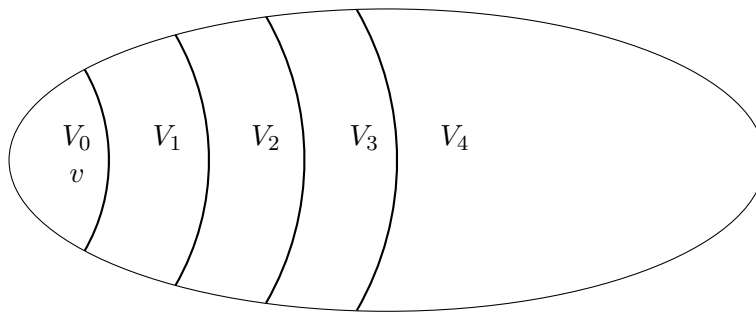
Questo caso lo dimostreremo per induzione.

Definiamo un insieme V_i formato da tutti i vertici del grafo G tali per cui la distanza di tali vertici da v è proprio i .

$$V_i = \{u \in V \mid \delta(v, u) = i\}$$

Stiamo in un certo senso esplorando il grafo, a partire da un vertice, in ampiezza andando su un vicinato a distanza via via maggiore.

Notiamo che $V_0 = \{v\}$



Andiamo a ragionare per induzione su i .

Il caso base, quando $i = 0$ è banale.

Chiaramente v raggiunge se stesso e viene colorato di nero; la sua distanza viene settata a 0; ed avendo come predecessore $\perp$ il punto 3 è vacuamente vero.

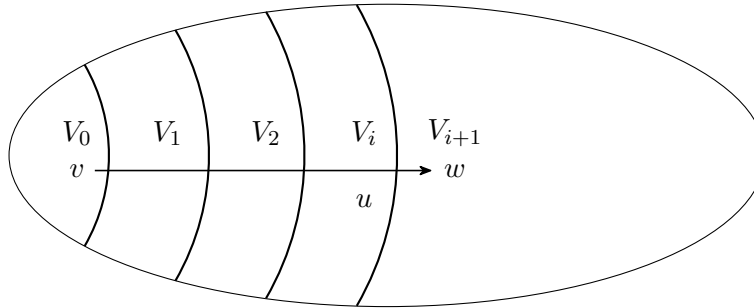
Passiamo al caso induttivo. Supponiamo il teorema vero per un certo generico ma fissato i e consideriamo il caso $i + 1$.

Se $w \in V_{i+1} \implies \exists u \in V_i ((u, w) \in E)$.

Il punto 1 risulta quindi immediato.

Per ipotesi induttiva sappiamo che l'algoritmo è corretto fino ad i .

Se fosse proprio il nodo u a scoprire il nodo w , vista l'ipotesi induttiva, l'algoritmo setterebbe la distanza del nodo w in modo corretto; e quindi il punto 2 sarebbe verificato. Inoltre, si avrebbe $p[w] = u$ e quindi banalmente anche il punto 3.



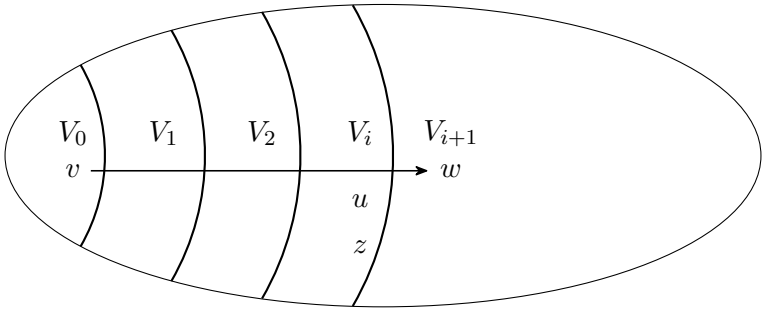
Pertanto, se è proprio il nodo u a scoprire il nodo w la dimostrazione termina. Notiamo, però, che non è detto che sia proprio il nodo u a scoprire il nodo w ; potrebbe essere un altro nodo z . In questo caso ci basta far vedere che anche il nodo z appartiene a V_i per terminare la dimostrazione.

Ma che $z \in V_i$ è immediato.

Se $z \in V_j$, con $j > i$, avremmo un assurdo in quanto per ipotesi induttiva sappiamo che l'algoritmo lavora bene fino ad i , ed inoltre il nodo z dovrebbe necessariamente venire scoperto dopo il nodo u , ulteriore assurdo in quanto stiamo supponendo che sia il nodo z a scoprire il nodo w . Pertanto $z \in V_j$, con $j \leq i$.

Se $z \in V_j$, con $j < i$, per la monotonicità della coda dell'algoritmo, dimostrata nel lemma precedente, avremo necessariamente che $w \in V_i$; ma abbiamo scelto $w \in V_{i+1}$, quindi abbiamo un assurdo. Pertanto $z \in V_j$, con $j \geq i$.

Quindi, $z \in V_i$.



□

6.10.4 Complessità dell'algoritmo BFS

Dimostrata la correttezza dell'algoritmo BFS su un grafo G con una certa sorgente v cerchiamo di calcolarne la complessità.

Ovviamente, la complessità in questo algoritmo non è immediata da calcolare perché abbiamo un ciclo **while** (equivalentemente un **repeat...until**) e come abbiamo già detto precedentemente i cicli **while** sono dei cicli che di per sé non ci danno un'immediata idea di quante iterazioni possono essere compiute.

In un ciclo **for** (intendiamo un ciclo **for** alla C), il numero di cicli è descritto esplicitamente. Ma in un ciclo **while** c'è la condizione che determina l'uscita o meno dal ciclo.

Capire quante volte un ciclo **while** viene fatto di principio può essere qualcosa di estremamente complesso in generale e infatti è il motivo per cui la terminazione degli algoritmi è un problema indecidibile.

Fortunatamente, nel nostro caso dell'algoritmo di visita in ampiezza, invece è estremamente semplice calcolare la complessità. Perché?

Perché il ciclo **while** dell'algoritmo è vero che è un ciclo che termina in base a una condizione sulla coda, però possiamo osservare che gli elementi che sono in coda sono necessariamente tutti quanti colorati di grigio, perché nel momento in cui inseriamo un valore in coda lo coloriamo seduta stante di grigio.

Inoltre, quando andiamo ad inserirlo, prima di inserirlo controlliamo che il colore originale sia bianco. Quindi, inseriamo in coda un nodo che è bianco e nel momento in cui lo inseriamo lo trasformiamo a grigio.

Dopodiché, appena tiriamo fuori il nodo dalla coda, oltre al lavoro che facciamo sulla stella uscente, quel nodo verrà poi posto a nero. Non viene reinserito in coda. Quello che verrà inserito sono gli adiacenti bianchi.

Ciò significa che un nodo, a livello di colorazione, parte bianco, permane bianco per un certo tempo, potenzialmente fino alla fine, se non è raggiungibile dal vertice iniziale.

Nel caso in cui questo sia raggiungibile, a un certo punto diventerà grigio nel momento di entrata in coda. Per la permanenza in coda rimane grigio perché nessuno lo tocca. Infatti i nodi in coda, una volta inseriti, non vengono toccati se non nel momento in cui vengono estratti. Quindi rimane grigio per un certo tempo.

Dopodiché, nel momento in cui viene estratto dalla coda, diventerà nero.

Ci rendiamo facilmente conto quindi che un nodo può essere bianco nel caso in cui non venga toccato, o diventa bianco, grigio, nero. Ma per poter essere inserito in coda, deve essere per forza bianco.

Il che significa che un nodo, una volta inserito in coda, sempre che questo entri, non vi entrerà mai più.

Quindi nella coda, ogni nodo può comparire al più una sola volta. Significa che la coda, dopo n passi, al più, si dovrà svuotare.

Quindi il numero massimo di interazioni è pari al numero di nodi.

Da questo, però, non segue immediatamente che la complessità sia lineare nel numero di nodi perché non è l'unica operazione che facciamo.

All'interno del ciclo, c'è un'analisi della stella uscente, che è implementata mediante un ciclo **for** (supponendo un grafo rappresentato mediante lista di adiacenza. Con una rappresentazione matriciale questo **for** costerebbe sempre n) che con una rappresentazione con lista di adiacenza costa esattamente quanti elementi ha la lista considerata.

Inoltre il corpo del **for** è costituito da operazioni costanti.

Osserviamo che un arco se appartiene a una stella uscente non appartiene a un'altra stella uscente. Un arco è identificato dalla destinazione, ma anche dalla sorgente.

Quindi un arco viene preso solo e soltanto in una esecuzione del ciclo **while** all'interno di una certa esecuzione del ciclo **for**.

Significa che complessivamente il ciclo **for**, se andiamo a coprire ogni ciclo del **while**, può coprire al più tutti gli archi del grafo perché un arco verrà preso una volta soltanto all'interno del **for**.

Quindi la complessità totale dell'algoritmo non è altro che $O(|E|)$, per il ciclo **while**, visto che nel caso peggiore percorreremo tutti gli archi, a cui va sommata la complessità per il lavoro iniziale di settaggio su tutti i vertici del grafo (funzione di **init**), che a prescindere andrà sempre fatta, cioè $\Omega(|V|)$.

Risulta pertanto chiaro che nell'insieme questo algoritmo abbia una complessità di $O(|E| + |V|)$, risulta pertanto lineare nella dimensione del grafo.

Questo ragionamento è comune poi al ragionamento che faremo per il calcolo della complessità delle altre visite su grafo.

6.11 Visite in profondità sui grafi

Abbiamo visto quindi la visita in ampiezza di un grafo a partire da un vertice di origine; ma così come sugli alberi siamo andati anche ad effettuare delle visite in profondità con algoritmi ricorsivi, su grafi faremo lo stesso.

Vi sono però delle differenze con le visite in profondità negli alberi. Così come per la visita in ampiezza avremo bisogno della colorazione.

Un'altra differenza rispetto agli alberi è che non c'è un concetto di visita in order visto che stiamo considerando grafi generici dove un nodo può avere un numero arbitrario (limitato ovviamente) di adiacenti.

Non è possibile definire quindi un concetto canonico di visita in order.

Quindi noi vedremo un unico algoritmo che lavorerà in pratica con una versione che è sia preorder che postorder; contempla allo stesso tempo quindi sia un lavoro in pre ordine sia un lavoro in post ordine.

Ha chiaramente senso parlare di visita in profondità pre order e post order in un grafo.

Vediamo ora come può essere definita una visita in profondità di un grafo.

La prima cosa da osservare è che poiché non c'è un'unica radice, come nel caso degli alberi, ci sono due possibili visite in profondità.

La visita effettuata partendo da un nodo, un po' come quella in ampiezza e la visita che cerca di vedere tutto ciò che è visitabile da tutti i nodi.

In effetti noi utilizzeremo per la visita in profondità molto spesso quella che parte da tutti i nodi.

Quindi ci sarà una prima parte dell'algoritmo che semplicemente scorre l'insieme dei vertici e non appena trova un vertice bianco fa partire una visita da quel vertice in avanti.

Nell'algoritmo in ampiezza noi non avevamo questa parte perché chiamavamo direttamente l'algoritmo sul grafo dato un vertice, mentre l'algoritmo di visita in profondità che vedremo ora parte solo dal grafo e prova a esplorare tutto ciò che è esplorabile da tutti i vertici.

In questo caso non ci sarà un concetto di distanza.

Il concetto di distanza non si può valutare con una visita in profondità, le distanze vanno calcolate sempre e soltanto con una visita in ampiezza perché quando visitiamo un percorso dobbiamo assicurarci, se vogliamo le distanze, di prendere il percorso più corto e l'unico modo per essere sicuri di raggiungere un nodo con il percorso più corto è esplorare a sfoglia di cipolla, cioè prima partiamo da un vertice esploriamo il suo vicinato di distanza zero, poi esploriamo tutto ciò che è visitabile a distanza uno, e così via.

Con la visita in profondità possiamo sfortunatamente incappare in un percorso estremamente lungo e una volta che abbiamo colorato un nodo non possiamo rivalutare le cose.

Bisogna però dire che, a differenza della visita in ampiezza, la visita in profondità ci permetterà di identificare l'esistenza di cicli, cosa che invece non è assolutamente identificabile con le visite in ampiezza.

6.11.1 Algoritmo: Depth First Search su un grafo

Vediamo ora l'algoritmo di visita in profondità in un grafo, DFS (Depth First Search).

Anche in questo caso avremo un vettore dei colori.

Avremo un vettore dei predecessori che però a differenza di quello visto nella visita in ampiezza non ha niente a che fare con il percorso minimo, visto che, come abbiamo detto, in questo caso il concetto di distanza non ha senso. Esso servirà a costruire quella che poi vedremo essere la cosiddetta foresta di visita.

Cioè in pratica identifica un sottografo indotto del grafo originale che è una foresta.

Foresta significa un insieme di alberi. Quindi un grafo completamente aciclico e potenzialmente però non connesso perché è fatto da tanti alberi.

In aggiunta vedremo due vettori numerici, i vettori di tempo di inizio e tempo di fine visita.

Tali vettori non servono al funzionamento dell'algoritmo ma servono solo a noi per dimostrare le proprietà dell'algoritmo stesso.

Ecco l'algoritmo

Algoritmo 84: Visita in profondità di un grafo: DFS

Signature DFS: $Graph(D) \rightarrow Array(Color) \times Array(D) \times Array(N) \times Array(N^*)$

function DFS(G)

```

    // c: vettore colori
    // p: vettore predecessori
    // s: vettore tempi inizio visita
    // e: vettore tempi fine visita
    // t: variabile che funge da contatore dei tempi
    // setto inizialmente mediante una funzione accessoria i vettori
    // e setto il tempo a 0 indicando che l'algoritmo inizia

```

1 **(c, p, s, e, t) ← (init(G), 0)**

```

    // per ogni nodo del grafo

```

2 **foreach (v in V) do**

```

        // se non è mai stato scoperto

```

3 **if (c[v] = WHITE) then**

```

            // aggiorni i vettori e il contatore dei tempi

```

```

            // mediante una funzione accessoria ricorsiva

```

```

            // che esplora il grafo a partire dal nodo in questione

```

4 **(c, p, s, e, t) ← DFS(G, v, c, p, s, e, t)**

```

        // restituisco i vettori

```

5 **return (c, p, s, e)**

Vediamo adesso la funzione accessoria `init`.

Algoritmo 85: Funzione di inizializzazione (DFS)

Signature `init`: $Graph(D) \rightarrow Array(Color) \times Array(D) \times Array(\mathbb{N}) \times Array(\mathbb{N}^*)$

```

function init(G)
  // alloca i vettori e li inizializza
1  foreach (v in V) do
    // ponendo per ciascun nodo il colore a bianco
    // e il predecessore a  $\perp$ 
    // non serve inizializzare i vettori dei tempi
    // visto che saranno sicuramente sovrascritti
    // durante l'esecuzione dell'algoritmo
2    (c[v], p[v])  $\leftarrow$  (WHITE,  $\perp$ )
  // restituisce i vettori
3  return (c, p, s, e)
  
```

Vediamo adesso la funzione `DFS` che svolge la parte ricorsiva dell'algoritmo di visita in profondità.

Algoritmo 86: Funzione accessoria ricorsiva per la DFS in un grafo

Signature DFS: $Graph(D) \times D \times Array(Color) \times Array(D) \times Array(N) \times Array(N^*) \times N \rightarrow$
 $Array(Color) \times Array(D) \times Array(N) \times Array(N^*) \times N$

```

function DFS(G, v, c, p, s, e, t)
    // indico che il nuovo nodo è stato scoperto
    // settando il colore a grigio
    // setto anche il tempo di inizio visita al corrente
    // ed incremento il contatore dei tempi
1   (c[v], s[v], t) ← (GRAY, t, t+1)
    // per tutti i nodi della stella uscente del nodo in input
2   foreach (w in Adj[v]) do
        // se il nodo nella stella uscente non è mai stato scoperto
3       if (c[w] = WHITE) then
            // ne setto il predecessore al nodo in input
4             p[w] ← v
            // aggiorni i vettori ed il contatore dei tempi
            // richiamando la funzione ricorsiva sul nodo appena scoperto
5             (c, p, s, e, t) ← DFS(G, w, c, p, s, e, t)
    // una volta terminato il lavoro sul nodo
    // indico che è stato visitato, settandone il colore a nero
    // setto il tempo di fine visita al tempo corrente
    // ed incremento il contatore dei tempi
6   (c[v], e[v], t) ← (BLACK, t, t+1)
    // restituisco i vettori e il contatore dei tempi
7   return (c, p, s, e, t)

```

Sottolineiamo ancora, come abbiamo già detto, che i vettori dei tempi, così come il contatore dei tempi, non hanno una effettiva utilità per il funzionamento dell'algoritmo ma sono importanti per effettuarne un'analisi perché ci daranno un modo per analizzare cosa sta facendo l'algoritmo al procedere della visita.

Come preannunciato l'algoritmo di visita in profondità in un grafo effettua sia del lavoro in pre ordine, cioè il settaggio dei tempi di inizio visita, e del lavoro in post ordine, cioè il settaggio dei tempi di fine visita.

6.11.2 Complessità dell'algoritmo DFS

Il calcolo della complessità dell'algoritmo di visita in profondità è praticamente identico a quello che abbiamo visto per la BFS.

Va considerata la complessità della funzione di `init`, che chiaramente è $\Theta(|V|)$. Ovviamente anche in questo caso il `for each` dell'algoritmo viene eseguito una volta per ciascun arco, ma a differenza della visita in ampiezza nella quale era possibile trovare archi che non vengono esplorati, in questo caso tutti gli archi verranno considerati dall'algoritmo visto che l'algoritmo esplora ogni nodo una sola volta e per ciascun nodo va ad esplorare la stella uscente una sola volta, pertanto la complessità del ciclo `while` sarà $\Theta(|E|)$.

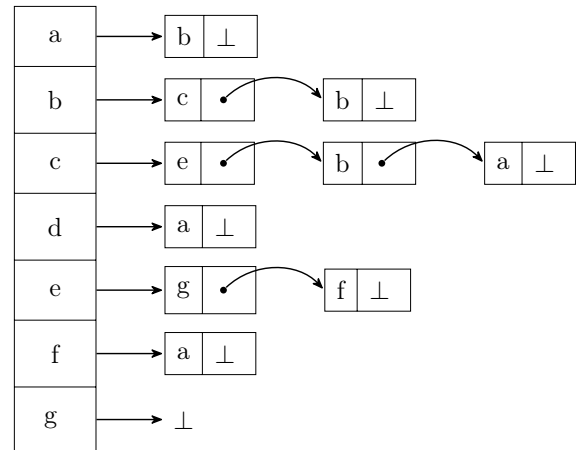
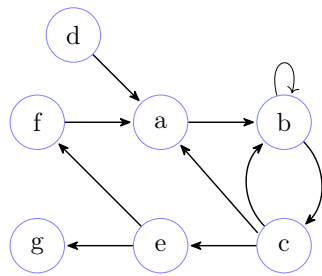
Conseguentemente nel complesso la visita in profondità di un grafo ha complessità $\Theta(|E| + |V|)$.

6.12 Analisi algoritmo DFS e relativa foresta di visita

Vediamo adesso con un esempio il funzionamento dell'algoritmo di visita in profondità in un grafo.

Nell'esempio useremo un ordine lessicografico per questioni di comodità ma non è necessario ovviamente.

Dato il seguente grafo, rappresentato sia con metodo grafico che con lista di adiacenti, simuliamo la sua visita in profondità.



Inizio simulazione

Passo 1: Istante $t = 0$

t	0
v	a
s	0
e	
p	⊥

Passo 2: Istante $t = 1$

t	0	1
v	a	b
s	0	1
e		
p	⊥	a

Passo 3: Istante $t = 2$

t	0	1	2
v	a	b	c
s	0	1	2
e			
p	⊥	a	b

Passo 4: Istante $t = 3$

t	0	1	2	3
v	a	b	c	e
s	0	1	2	3
e				
p	⊥	a	b	c

Passo 5: Istante $t = 4$

t	0	1	2	3	4
v	a	b	c	e	g
s	0	1	2	3	4
e					
p	⊥	a	b	c	e

Passo 6: Istante $t = 5$

t	0	1	2	3	4	5
v	a	b	c	e	g	g
s	0	1	2	3	4	—
e						5
p	⊥	a	b	c	e	—

Passo 7: Istante $t = 6$

t	0	1	2	3	4	5	6
v	a	b	c	e	g	g	f
s	0	1	2	3	4	—	6
e						5	
p	⊥	a	b	c	e	—	e



Passo 8: Istante $t = 7$

t	0	1	2	3	4	5	6	7
v	a	b	c	e	g	g	f	f
s	0	1	2	3	4	—	6	—
e						5		7
p	$\perp$	a	b	c	e	—	e	—

Passo 9: Istante $t = 8$

t	0	1	2	3	4	5	6	7	8
v	a	b	c	e	g	g	f	f	e
s	0	1	2	3	4	—	6	—	—
e						5		7	8
p	$\perp$	a	b	c	e	—	e	—	—

Passo 10: Istante $t = 9$

t	0	1	2	3	4	5	6	7	8	9
v	a	b	c	e	g	g	f	f	e	c
s	0	1	2	3	4	—	6	—	—	—
e						5		7	8	9
p	$\perp$	a	b	c	e	—	e	—	—	—

Passo 11: Istante $t = 10$

t	0	1	2	3	4	5	6	7	8	9	10
v	a	b	c	e	g	g	f	f	e	c	b
s	0	1	2	3	4	—	6	—	—	—	—
e						5		7	8	9	10
p	$\perp$	a	b	c	e	—	e	—	—	—	—



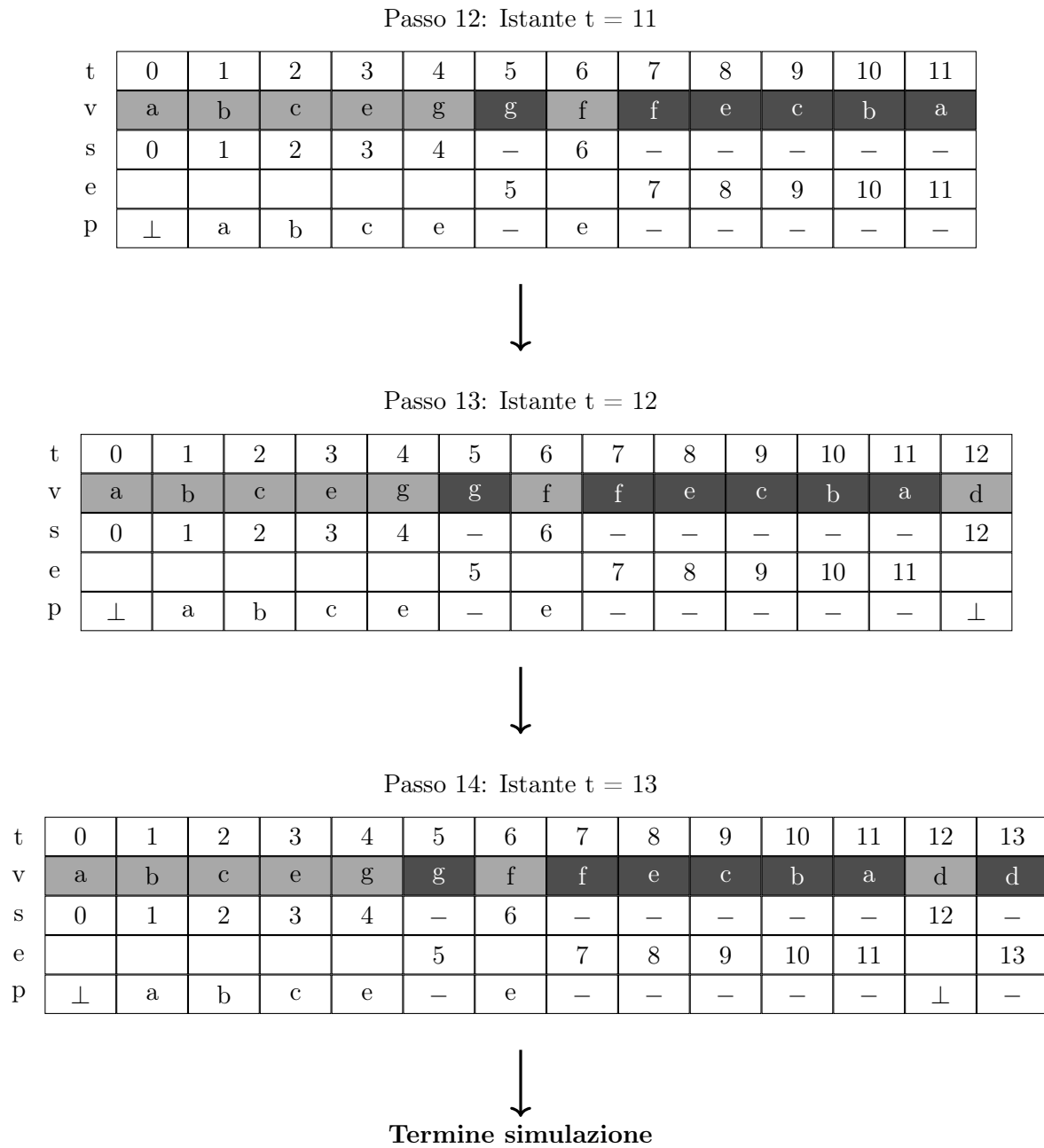


Figura 6.12: Esempio 1 di visita in profondità del grafo

Sottolineiamo che il tutto è stato dettato da come abbiamo iniziato a visitare, dall'ordine di visita. Osserviamo cosa succede invertendo l'ordine di visita.

Partiremo quindi visitando per il primo il nodo g.

Inizio simulazione

Passo 1: Istante $t = 0$

t	0
v	g
s	0
e	
p	$\perp$



Passo 2: Istante $t = 1$

t	0	1
v	g	g
s	0	—
e		1
p	$\perp$	—



Passo 3: Istante $t = 2$

t	0	1	2
v	g	g	f
s	0	—	2
e		1	
p	$\perp$	—	$\perp$



Passo 4: Istante $t = 3$

t	0	1	2	3
v	g	g	f	a
s	0	—	2	3
e		1		
p	$\perp$	—	$\perp$	f



Passo 5: Istante $t = 4$

t	0	1	2	3	4
v	g	g	f	a	b
s	0	—	2	3	4
e		1			
p	$\perp$	—	$\perp$	f	a



Passo 6: Istante $t = 5$

t	0	1	2	3	4	5
v	g	g	f	a	b	c
s	0	—	2	3	4	5
e		1				
p	$\perp$	—	$\perp$	f	a	b



Passo 7: Istante $t = 6$

t	0	1	2	3	4	5	6
v	g	g	f	a	b	c	e
s	0	—	2	3	4	5	6
e		1					
p	$\perp$	—	$\perp$	f	a	b	c



Passo 8: Istante $t = 7$

t	0	1	2	3	4	5	6	7
v	g	g	f	a	b	c	e	e
s	0	—	2	3	4	5	6	—
e		1						7
p	$\perp$	—	$\perp$	f	a	b	c	—



Passo 9: Istante $t = 8$

t	0	1	2	3	4	5	6	7	8
v	g	g	f	a	b	c	e	e	c
s	0	—	2	3	4	5	6	—	—
e		1						7	8
p	$\perp$	—	$\perp$	f	a	b	c	—	—



Passo 10: Istante $t = 9$

t	0	1	2	3	4	5	6	7	8	9
v	g	g	f	a	b	c	e	e	c	b
s	0	—	2	3	4	5	6	—	—	—
e		1						7	8	9
p	$\perp$	—	$\perp$	f	a	b	c	—	—	—



Passo 11: Istante $t = 10$

t	0	1	2	3	4	5	6	7	8	9	10
v	g	g	f	a	b	c	e	e	c	b	a
s	0	—	2	3	4	5	6	—	—	—	—
e		1						7	8	9	10
p	$\perp$	—	$\perp$	f	a	b	c	—	—	—	—

Passo 12: Istante $t = 11$

t	0	1	2	3	4	5	6	7	8	9	10	11
v	g	g	f	a	b	c	e	e	c	b	a	f
s	0	—	2	3	4	5	6	—	—	—	—	—
e		1						7	8	9	10	11
p	$\perp$	—	$\perp$	f	a	b	c	—	—	—	—	—

Passo 13: Istante $t = 12$

t	0	1	2	3	4	5	6	7	8	9	10	11	12
v	g	g	f	a	b	c	e	e	c	b	a	f	d
s	0	—	2	3	4	5	6	—	—	—	—	—	12
e		1						7	8	9	10	11	
p	$\perp$	—	$\perp$	f	a	b	c	—	—	—	—	—	$\perp$

Passo 14: Istante $t = 13$

t	0	1	2	3	4	5	6	7	8	9	10	11	12	13
v	g	g	f	a	b	c	e	e	c	b	a	f	d	d
s	0	—	2	3	4	5	6	—	—	—	—	—	12	—
e		1						7	8	9	10	11		13
p	$\perp$	—	$\perp$	f	a	b	c	—	—	—	—	—	$\perp$	—

**Fine simulazione**

Figura 6.13: Esempio 2 di visita in profondità del grafo

Vediamo ora le foreste di visita relative agli esempi.

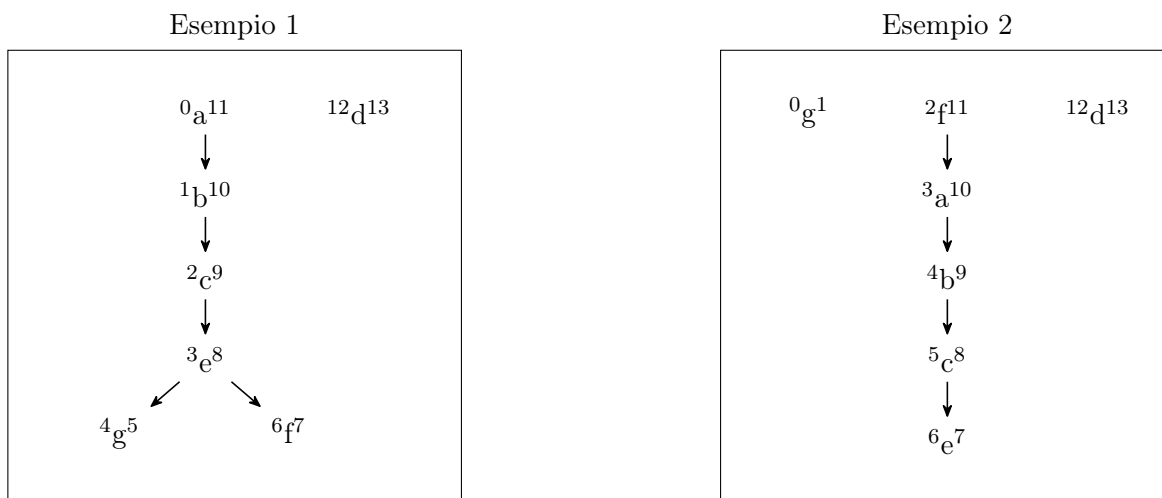


Figura 6.14: Foreste di visita degli esempi 1 e 2 della DFS

Si nota chiaramente che due visite dello stesso grafo con ordine dei nodi diverso portano a scoprire i vertici e ad analizzarli in ordine completamente diverso e le foreste di visita risultanti saranno conseguentemente diverse.

Attenzione. Non possiamo pretendere che l'algoritmo di visita in profondità segua un ordine specifico, seguirà un ordine a caso che noi non potremo forzare; di conseguenza a seconda di come vengono scelti i vertici ci sarà una certa visita del grafo e una sottostante implicita costruzione delle foreste di visita.

Visto che ogni nodo ha tempi di inizio e fine visita differenti, in generale possiamo dire che l'incremento dei tempi per ciascun nodo è di 2 unità e pertanto indipendentemente dall'ordine di visita, avremo che i tempi andranno sempre da 0 a $2n - 1$, dove n è il numero di vertici del grafo in questione.

Una cosa importante è che la struttura della foresta di visita indotta è salvata all'interno del vettore dei predecessori.

Quindi noi sapremo al termine della visita quale è stato l'ordine ma non possiamo forzarlo a priori.

O meglio, non è possibile forzarlo senza determinare un aumento della complessità.

Vediamo ora quali sono le proprietà che possiamo determinare analizzando i vettori dei tempi e le strutture di albero e foresta che risultano dall'esecuzione dell'algoritmo.

6.12.1 Proprietà algoritmo DFS

Una delle prime proprietà da osservare è la seguente cosa.

Guardiamo un attimo la sequenza di visita dell'algoritmo e andiamo a posizionare delle parentesi aperte e chiuse a seconda dell'inizio e fine visita dello stesso nodo.

Esempio 1

	(a	(b	(c	(e	(g	g)	(f	f)	e)	c)	b)	a)	(d	d)
t	0	1	2	3	4	5	6	7	8	9	10	11	12	13
v	a	b	c	e	g	g	f	f	e	c	b	a	d	d
s	0	1	2	3	4	—	6	—	—	—	—	—	12	—
e						5		7	8	9	10	11		13
p	⊥	a	b	c	e	—	e	—	—	—	—	—	⊥	—

Esempio 2

	(g	g)	(f	(a	(b	(c	(e	e)	c)	b)	a)	f)	(d	d)
t	0	1	2	3	4	5	6	7	8	9	10	11	12	13
v	g	g	f	a	b	c	e	e	c	b	a	f	d	d
s	0	—	2	3	4	5	6	—	—	—	—	—	12	—
e		1						7	8	9	10	11		13
p	⊥	—	⊥	f	a	b	c	—	—	—	—	—	⊥	—

Figura 6.15: Esempio di struttura a parentesi della DFS

Quello che possiamo notare facilmente è che indipendentemente dall'ordine di visita le parentesi sono ben bilanciate.

Non si tratta chiaramente di un caso, bensì di un teorema.

Il teorema dice che, considerando i tempi di inizio e fine visita dei vertici del grafo, o capita che i tempi di inizio e fine visita di due vertici distinti sono completamente disgiunti, oppure sono innestati correttamente gli uni negli altri.

Tale teorema prende il nome di teorema della struttura a parentesi dell'algoritmo di visita in profondità nei grafi.

Intuitivamente guardando una sequenza di visita dell'algoritmo possiamo notare che in generale un nodo ha un tempo di inizio visita che precede tutti i tempi di inizio e fine visita di tutti i nodi che può raggiungere e termina solo dopo questi; quando invece si crea un fork nell'albero di visita, i bracci del fork saranno parentesizzazioni indipendenti.

Con l'analisi empirica dell'algoritmo della visita in profondità in un grafo abbiamo osservato che a livello strutturale quello che otteniamo se andiamo a prendere i tempi di inizio visita e fine visita e andiamo a collegarli con una parentesizzazione, in particolare ad ogni tempo di inizio visita facevamo partire una parentesi di tipo il nodo, cioè il nome, la cui tipologia era data dal nome del nodo che veniva scoperto in quel momento e a tempo di fine visita associavamo una parentesi chiusa con lo stesso nome, allora quello che otteniamo è sempre una parentesizzazione fatta bene, cioè ben bilanciata.

Formalmente, quello che abbiamo osservato empiricamente si formalizza nel seguente modo.

Prendiamo due vertici, v e w , in cui semplicemente abbiamo che il tempo di inizio visita di v , $s[v]$ è precedente al tempo di inizio visita w , $s[w]$.

Questo è arbitrario perché tanto i vertici li possiamo scegliere come vogliamo, sono entrambi quantificati diversamente, quindi questa è una scelta senza perdita di generalità.

Allora, quello che può accadere sono soltanto due casi e mai un terzo.

Il primo caso afferma che iniziamo prima la visita di v , dopodiché iniziamo la visita di w , terminiamo quella di w e poi solo dopo terminiamo quella di v .

Il secondo caso afferma che iniziamo la visita di w solo dopo che la visita di v è terminata.

Non ci ritroveremo mai nel terzo caso, cioè quello in cui iniziamo la visita di v , iniziamo la visita di w e poi terminiamo prima la visita di v e solo dopo quella di w .

Questo è un teorema, detto Teorema della struttura a parentesi della DFS.

Prima di affrontare la dimostrazione cerchiamo di capire perché intuitivamente quello che abbiamo affermato risulta vero.

Il motivo è che l'algoritmo DFS è ricorsivo, il cui funzionamento è basato implicitamente sull'utilizzo dello stack, lo stack di attivazione e come sappiamo ad uno stack si accede sempre in testa.

Per poter accedere a qualcosa che abbiamo messo nel passato nello stack, bisogna prima terminare ciò che si trova sopra. Non è possibile accedere a ciò che sta nel fondo dello stack se non abbiamo svuotato la parte in alto.

Ma quando svuotiamo la parte in alto, significa che abbiamo terminato il lavoro sulla parte in alto, l'abbiamo già colorata di nero.

Formalmente.

Teorema 6.12.1 (Struttura a parentesi della DFS).

$\forall v, w \in V$, con $s[v] < s[w]$, una delle seguenti è vera:

- a) $s[v] < s[w] < e[w] < e[v]$;
- b) $s[v] < e[v] < s[w] < e[w]$.

Ovvero, è impossibile che $s[v] < s[w] < e[v] < e[w]$.

Dimostrazione.

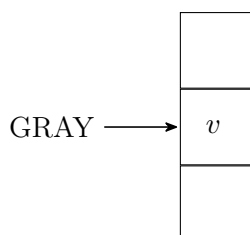
La dimostrazione si basa su un ragionamento per assurdo.

Supponiamo, per assurdo, il teorema falso. Quindi di trovarci in una situazione in cui, dati i presupposti dell'ipotesi, sia vero che $s[v] < s[w] < e[v] < e[w]$.

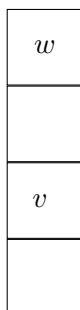
Andiamo ad analizzare lo stack di attivazione nel momento in cui il nodo v viene scoperto. Ovviamente semplificheremo la rappresentazione considerando solo i vertici.

Ricordiamo che, come si nota immediatamente dall'algoritmo, quando un nodo viene scoperto viene colorato di grigio.

Pertanto la situazione, quando viene scoperto il nodo v è la seguente.



Successivamente, viene scoperto il vertice w .



Data la nostra supposizione assurda stiamo immaginando di settare il tempo di fine visita del nodo v , e quindi di terminare la visita sul nodo v prima di terminare quella su w .

Come sappiamo, ad una struttura dati di tipo stack è possibile accedere solo in testa.

Pertanto non è possibile rimuovere v dallo stack senza prima rimuovere w .

L'unica possibilità è quindi che ci sia stata un'altra chiamata sul vertice v , a seguito di quella sul nodo w che vada quindi a terminare prima della fine della visita sul nodo w ; ma come sappiamo questo non è possibile visto che se un nodo non è colorato di bianco allora non viene preso in considerazione dall'algoritmo.

Siamo quindi arrivati ad una contraddizione a seguito della supposizione per assurdo. □

Passiamo adesso ad un importante lemma.

Prima di formalizzarlo cerchiamo di darne un'intuizione.

Se ci troviamo nell'eventualità a) del teorema precedente allora siamo sicuri che esista un percorso nella foresta di visita che da v vada a w .

Cioè intuitivamente w è stato scoperto tramite nodi che sono raggiungibili da v .

Viceversa, se c'è un percorso nella foresta di visita tra due nodi, allora necessariamente se andiamo a guardare i tempi della foresta di visita dei nodi considerati ci troveremo nell'eventualità a) del teorema precedente, non nell'eventualità b).

L'eventualità b) del teorema della struttura a parentesi della DFS ci dice solo invece che i due nodi fanno parte di sottoalberi o alberi diversi della foresta di visita.

In generale se consideriamo il nodo con tempo di inizio visita minore come la radice di un sottoalbero nella foresta di visita, se il secondo nodo è raggiungibile a partire dal primo ci troveremo nel primo caso, altrimenti nel secondo.

Quindi i due punti del teorema precedente ci dicono come sono stati scoperti i nodi e che relazione c'è tra questi nodi.

Prima di affrontare il lemma dobbiamo però formalizzare il concetto di foresta di visita indotta dall'algoritmo DFS.

Dato un grafo G e dato un vettore dei predecessori ottenuto a seguito dell'esecuzione dell'algoritmo DFS, la foresta di visita indotta sarà una coppia formata dall'insieme dei vertici del grafo G e gli archi contenuti nel vettore dei predecessori.

Formalmente.

Dato $G = \langle V, E \rangle$ e p vettore dei predecessori di $\text{DFS}(G)$ avremo che la foresta di visita $\mathcal{F}$ è:

$$\mathcal{F} = \langle V, \{(p[v], v) \mid v \in V \wedge p[v] \neq \perp\} \rangle \quad (6.22)$$

Passiamo, adesso, al lemma.

Lemma 6.12.1 (Percorsi nella foresta di visita della DFS).

$$\forall v, w \in V (s[v] < s[w] < e[w] < e[v] \iff \exists \pi \in \text{Path}(\mathcal{F}_p, v, w))$$

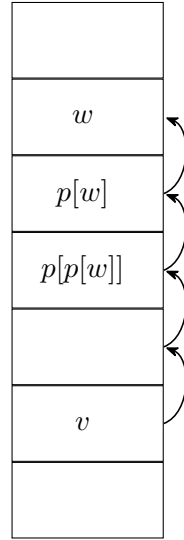
Dimostrazione.

Dimostriamo il lemma verificando le due implicazioni.

($\implies$)

Se è vero che $s[v] < s[w] < e[w] < e[v]$, nel momento in cui abbiamo scoperto il nodo w lo stack avrà una rappresentazione del genere.

Ricordiamo, come si nota dall'algoritmo, che non appena un nodo viene scoperto viene settato anche il predecessore.



Osservando che, dalla definizione di foresta di visita, inseriremo nella foresta di visita un arco che, per ogni nodo u , va da $p[u]$ a u l'implicazione risulta banale.

($\Leftarrow$)

Dimostreremo l'implicazione inversa facendo induzione sulla lunghezza del percorso della foresta di visita.

Sia $\pi \in Path(\mathcal{F}_p)$. Avremo $\pi = \pi_0, \pi_1, \pi_2, \dots, \pi_{k-2}, \pi_{k-1}, \pi_k$. Ma π è un percorso della foresta di visita, pertanto $\pi = p[\pi_1], p[\pi_2], p[\pi_3], \dots, p[\pi_{k-1}], p[\pi_k], \pi_k$.

Ricordiamo, come si nota dall'algoritmo, che non appena un nodo viene scoperto esso passa da bianco a grigio, viene settato il tempo di inizio visita e anche il predecessore; quando poi la visita sul nodo termina il colore passa a nero e viene settato il tempo di fine visita.

Ragioniamo per induzione sulla lunghezza del percorso π , k .

Il caso base risulta ovvio. Se $k = 1 \implies \pi = v, w$ e chiaramente, visto il comportamento dell'algoritmo $s[v] < s[w] < e[w] < e[v]$.

Consideriamo il caso induttivo. Sia pertanto $\pi = p[\pi_1], p[\pi_2], p[\pi_3], \dots, p[\pi_{k-1}], p[\pi_k], p[\pi_{k+1}], \pi_{k+1}$.

Come abbiamo ripetuto diverse volte se un nodo viene scoperto necessariamente doveva essere bianco e quindi risulta $s[\pi_k] < s[\pi_{k+1}] < e[\pi_{k+1}] < e[\pi_k]$. Per ipotesi induttiva la tesi.

□

C'è un ultimo teorema che dobbiamo dimostrare, il teorema del percorso bianco.

Andiamo prima a descriverlo e poi andremo a formalizzarlo e dimostrarlo.

Prendiamo due vertici di un grafo, v e w . Quello che andremo a mostrare è un'equivalenza tra due proprietà. La prima proprietà sarà relativa all'esistenza di un percorso nella foresta di visita, quindi w è discendente di v

nella foresta, che altro non vuol dire che esiste un percorso nella foresta di visita da v a w .

La seconda proprietà afferma che al tempo $s[v]$, cioè quando scopriamo v , che è l'origine del percorso, andando a guardare il grafo al tempo $s[v]$ e osservandone la colorazione quello che dobbiamo necessariamente osservare è che nel grafo esiste un percorso bianco che da v va a w (supponiamo di guardare il grafo all'istante $s[v]$ e prima che il nodo v venga colorato a grigio).

Intuitivamente questo teorema sta dicendo che se c'è un percorso nella foresta di visita, ad esempio da un nodo v ad un nodo w , allora necessariamente nel momento in cui abbiamo scoperto v doveva esistere un percorso che raggiunge w nel grafo originale tutto bianco perché se questo percorso non dovesse esistere avremmo tutti i nodi neri ma già sappiamo che l'algoritmo quando trova qualcosa di non bianco si ferma e quindi non sarebbe possibile alla fine dell'algoritmo avere un percorso da v a w nella foresta di visita.

Dualmente, se in un certo istante dell'esecuzione dell'algoritmo c'è almeno nel grafo un percorso bianco che da v va a w , sappiamo che l'algoritmo percorrerà uno di questi percorsi bianchi che sarà proprio il percorso che andrà a far parte della foresta di visita indotta.

Non abbiamo però modo di decidere quale dei diversi percorsi bianchi verrà effettivamente selezionato dall'algoritmo (a meno di non inficiare la complessità).

Formalmente.

Teorema 6.12.2 (Percorso Bianco).

$\forall v, w \in V (\exists \pi_F \in Path(\mathcal{F}_p, v, w) \iff \text{al tempo } s[v], \exists \pi_G \in Path(G, v, w) (\forall i \in [0, |\pi_G|) (c[\pi_i] = \text{WHITE})))$.

Dimostrazione.

Dimostriamo il teorema verificando le due implicazioni singolarmente.

($\implies$)

Se $\exists \pi_F \in Path(\mathcal{F}_p, v, w)$, per il lemma precedente avremo necessariamente $s[\pi_0] < s[\pi_1] < \dots < s[\pi_k]$, con $k = |\pi| - 1$.

Risulta immediato che al tempo di scoperta di v , tutti i nodi del percorso siano bianchi; e visto che gli archi della foresta di visita sono archi del grafo originale abbiamo trovato un percorso bianco nel grafo.

($\impliedby$)

Sappiamo che, al tempo di scoperta del nodo v , $\exists \pi_G \in Path(G, v, w) (\forall i \in [0, |\pi_G|) (c[\pi_i] = \text{WHITE}))$.

Tuttavia, non è detto che sia l'unico percorso bianco nel grafo e non sappiamo se sarà proprio il percorso che andrà a far parte della foresta di visita indotta. Come abbiamo già detto non abbiamo modo di sapere effettivamente quale percorso l'algoritmo andrà ad inserire nella foresta di visita; pertanto non possiamo utilizzare un approccio costruttivo per la dimostrazione di questa implicazione.

Andremo a ragionare per assurdo, supponendo che nessuno dei percorsi bianchi del grafo da v a w , disponibili al tempo di scoperta del nodo v , venga effettivamente scelto.

Quindi, supponiamo, per assurdo, che $\nexists \pi_F \in Path(\mathcal{F}_p, v, w)$.

Chiaramente l'origine del percorso, v , deve esistere.

Allora, necessariamente $\exists i \in (0, |\pi_G|)((\pi_G)_i$ non è discendente di v in $\mathcal{F}_p$). Sia $u \in V$ il primo di tali nodi.

Quindi, logicamente segue che $\nexists \pi' \in \text{Path}(\mathcal{F}_p, v, u)$. Dal lemma precedente questo equivale a dire che $s[v] < e[v] < s[u] < e[u]$.

Sia ora $z \in V$ ($z = (\pi_G)_{i-1}$). Sappiamo che questo nodo deve esistere, in quanto ovviamente v è discendente di se stesso e sappiamo che u non discenda da v , quindi, ovviamente, avremo che z è discendente di v in $\mathcal{F}_p$; pertanto $\exists \pi'' \in \text{Path}(\mathcal{F}_p, v, z) \iff s[v] < s[z] < e[z] < e[v]$.

Integrando le nostre osservazioni abbiamo $s[v] < s[z] < e[z] < e[v] < s[u] < e[u]$, nello specifico $s[z] < e[z] < s[u] < e[u]$

Ma per come abbiamo scelto z , visto che $z = (\pi_G)_{i-1}$ e $u = (\pi_G)_i$, si ha necessariamente $((\pi_G)_{i-1}, (\pi_G)_i) = (z, u) \in E$; quindi $s[z] < s[u] < e[u] < e[z]$.

Abbiamo quindi trovato una contraddizione a seguito della supposizione per assurdo.

Vediamo adesso un esempio grafico della dimostrazione appena fatta.

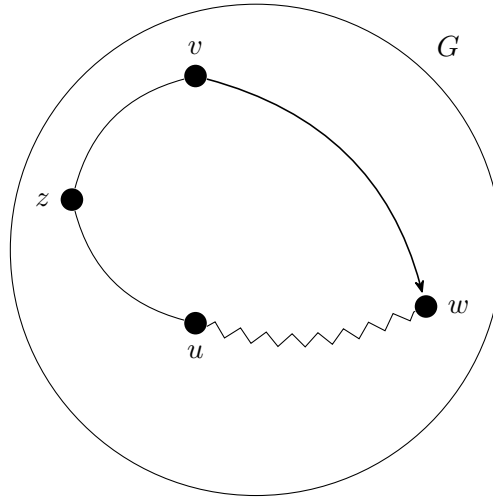


Figura 6.16: Esempio della dimostrazione sul percorso bianco

□

6.13 Partizionamento archi in un grafo mediante algoritmo DFS

Andiamo ora ad analizzare del grafo quali archi sono presenti e quali non sono presenti nella foresta di visita cercando in particolare di dare una categorizzazione degli archi non presenti.

Ovviamente questo concetto è un concetto che è dato una foresta; foreste diverse per lo stesso grafo daranno atto a diverse classificazioni degli archi. Non si tratta di un concetto assoluto.

Diamo una iniziale classificazione generale degli archi di un grafo data una foresta.

Indicheremo con F gli archi della foresta di visita.

Indicheremo con I, gli archi all'indietro. Sono archi che quando l'algoritmo prova ad esplorare trova il nodo di destinazione grigio; vengono chiamati in questo modo in quanto graficamente se andiamo a guardare la foresta di visita, di solito, tendono ad andare verso l'alto nello stesso albero o a rimanere nello stesso livello (self loop); sono gli archi che sfrutteremo per l'algoritmo di determinazione dei cicli.

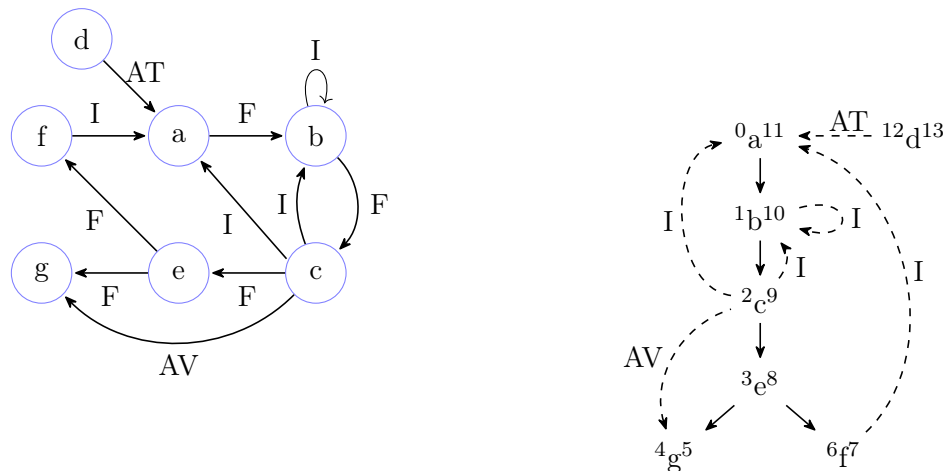
Se nel grafo è presente un ciclo dovrà esistere necessariamente un arco all'indietro in una qualunque foresta di visita.

Indicheremo con AT gli archi di attraversamento, chiamati così perché sono archi che attraversano alberi differenti della foresta; si generano quando l'algoritmo tenta di percorrere un arco con destinazione colorata di nero.

Se nella rappresentazione grafica della foresta posizioniamo a destra via via alberi che hanno tempi di inizio visita crescenti gli archi di attraversamento vanno sempre da destra verso sinistra.

Indicheremo con AV gli archi in avanti, anche questi si formano quando l'algoritmo esplora un arco con destinazione nera ma tale nodo fa parte dello stesso albero del nodo di origine. Graficamente sono archi che vanno da un nodo di un albero della foresta verso un nodo più in basso.

Consideriamo il seguente grafo e la seguente foresta di visita indotta dal DFS sul grafo.



Abbiamo quindi decomposto l'intero grafo, l'insieme degli archi del grafo, in un insieme dato dagli archi di foresta, gli archi all'indietro, che sono gli archi da nodi di un albero a nodi dello stesso albero più in alto che corrispondono

ad archi che nel momento in cui l'algoritmo è andato ad esplorare ha trovato con destinazione grigia, poi ci sono due tipologie di archi che l'algoritmo trova quando esplora archi con destinazione nera e che quindi di per sé non è in grado di distinguere, gli archi di attraversamento che sono gli archi che vanno da un albero ad un altro, e archi in avanti che invece sono archi da nodi dell'albero a nodi più in basso dello stesso albero da loro raggiungibili.

Notiamo che in questo modo abbiamo effettivamente partizionato gli archi del grafo.

Grazie a questa classificazione abbiamo ora un modo per definire un algoritmo che vada alla ricerca di cicli in un grafo.

Diamo ora una formalizzazione del concetto di classificazione degli archi di un grafo data una foresta di visita indotta su di esso dall'algoritmo DFS.

In generale prendiamo qualunque arco del grafo in considerazione sul quale l'algoritmo DFS ha indotto una foresta di visita.

Dobbiamo guardare gli archi del grafo e non della foresta perché vogliamo partizionare gli archi del grafo in quattro diverse classiche.

Controlliamo al tempo $s[v]$, cioè quando l'origine dell'arco viene scoperta, il colore del nodo di destinazione dell'arco.

Se questo è bianco l'arco sarà necessariamente un nodo che apparterrà alla foresta, e di conseguenza viene detto arco della foresta.

Se invece il colore è grigio quello verrà detto arco all'indietro, perché conetterà un nodo nella foresta a un suo avo in quanto quel nodo a cui si connette non ha ancora terminato la visita.

Se invece il colore è nero ci troviamo nel caso di o avere un arco in avanti o di avere un arco di attraversamento e distinguiamo questi due casi in base al fatto che ovviamente nel caso di arco in avanti il nodo di destinazione è raggiungibile nella foresta da un certo percorso partendo dal nodo di origine dell'arco, se invece il percorso che connette il nodo di origine e quello di destinazione nella foresta non esiste quello è detto arco di attraversamento.

Attenzione che gli archi di attraversamento quindi non sono solo archi tra alberi diversi della stessa foresta ma anche tra sottoalberi diversi dello stesso albero.

Formalmente.

Dato $\mathcal{F}_p$ la foresta di visita $G < V, E >$ il grafo e p il vettore dei predecessori, possiamo dire che $\forall (v, w) \in E$.

- a) arco di foresta se $(v, w) \in \mathcal{F}_p$, al tempo $s[v]$, $c[w] = \text{WHITE}$;
- b) arco all'indietro se al tempo $s[v]$, $c[w] = \text{GRAY}$;
- c) arco in avanti se al tempo $s[v]$, $c[w] = \text{BLACK}$ e w è discendente di v in $\mathcal{F}_p$;
- d) arco di attraversamento se al tempo $s[v]$, $c[w] = \text{BLACK}$ e w non è discendente di v in $\mathcal{F}_p$.

6.14 Cicli in un grafo

Quello che vedremo ora è un algoritmo che identifica la proprietà di aciclicità o ciclicità di un grafo.

Si tratta di una banale variazione dell'algoritmo di visita in profondità.

A differenza dell'algoritmo di visita in profondità, poiché non è necessario agli scopi per il check di aciclicità, non avremo né i tempi di inizio e fine visita che servivano solo come strumentazione per dimostrare proprietà, né tantomeno il vettore di predecessori.

Quindi l'unica cosa che avremo è il vettore dei colori.

6.14.1 Algoritmo: ricerca cicli in un grafo

Ecco l'algoritmo.

Anche in questo caso sarà composto da una parte che chiama una funzione ricorsiva e da una parte ricorsiva.

Algoritmo 87: Funzione che verifica l'aciclicità di un grafo

Signature `acyclic`: $Graph(D) \rightarrow \mathbb{B}$

```

function acyclic(G)
    // mediante una funzione accessoria
    // inizializzo il vettore dei colori a bianco
1   c  $\leftarrow$  init(G)
    // per ogni vertice del grafo
2   foreach (v in V) do
        // se il nodo non è mai stato scoperto
3       if (c[v] = WHITE) then
            // se ho trovato un ciclo
            // mediante una funzione accessoria ricorsiva
            // che esplora il grafo dal nodo in questione
4           if (NOT acyclic(G, v)) then
                // possiamo terminare perché sicuramente
                // il grafo non sarà aciclico e quindi
                // restituisco falso
5           return false
        // se non ho trovato cicli
        // esplorando il grafo a partire da tutti i nodi bianchi incontrati
        // il grafo è aciclico e restituisco vero
6   return true

```

Ecco la parte ricorsiva che va alla ricerca di un ciclo nel grafo nella parte raggiungibile dal nodo in input.

Algoritmo 88: Funzione accessoria per la verifica dell'aciclicità di un grafo, parte ricorsiva

Signature `acyclic`: $Graph(D) \times D \rightarrow \mathbb{B}$

```

function acyclic(G, v)
  // indico che ho scoperto il nodo in input
  // colorandolo di grigio
1  c[v] ← GRAY
  // per ogni nodo nella sua stella uscente
2  foreach (w in Adj[v]) do
    // se è un nodo che non è mai stato scoperto
3    if (c[w] = WHITE) then
      // richiamo ricorsivamente la funzione
      // sul nodo in questione
      // e se trovo un ciclo
4      if (NOT acyclic(G, w)) then
        // posso terminare in quanto il grafo
        // non sarà aciclico e quindi
        // restituisco falso
5        return false
    // se il nodo in questione è stato scoperto ma non visitato
6    else if (c[w] = GRAY) then
      // ho trovato un arco all'indietro
      // che equivale a trovare un ciclo nel grafo
      // quindi restituisco falso
7      return false
  // una volta effettuato il lavoro sul nodo
  // setto che è stato visitato colorandolo di nero
8  c[v] ← BLACK
  // se non ho trovato cicli restituisco true
9  return true
  
```

6.14.2 Correttezza algoritmo ricerca cicli in un grafo

Risulta immediato che assumendo la correttezza della parte ricorsiva tutto l'algoritmo sia corretto. Pertanto per dimostrare la correttezza dell'algoritmo possiamo focalizzarci sulla funzione ricorsiva.

Quello che dimostriamo è semplicemente che se c'è un ciclo la parte ricorsiva dell'algoritmo restituirà falso, e che se la parte ricorsiva restituisce falso allora c'è un ciclo.

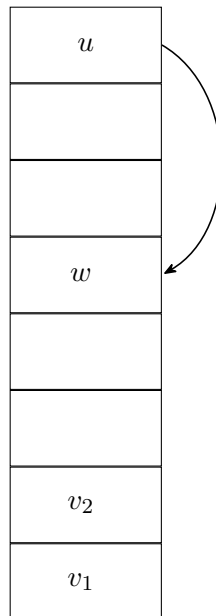
Teorema 6.14.1 (Dimostrazione correttezza algoritmo di aciclicità).

Dimostrazione.

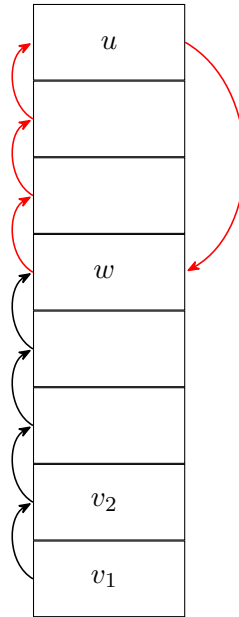
Per dimostrare la correttezza dell'algoritmo mostreremo che se nel grafo c'è un ciclo l'algoritmo in questione restituirà falso e che se l'algoritmo restituisce falso allora c'è un ciclo nel grafo.

Supponiamo che l'algoritmo ricorsivo restituisca per una certa chiamata falso. Osserviamo lo stack di attivazione.

Sottolineiamo che tutti i nodi nello stack sono per forza grigi.



Di conseguenza abbiamo trovato l'esistenza di un arco dall'ultimo nodo inserito nello stack in uno dei nodi già presenti nello stack. Come sappiamo da lemma precedentemente dimostrati esiste un percorso tra i nodi presenti nello stack e quindi, avendo trovato un arco tra un nodo dello stack ed un nodo precedentemente inserito nello stack, avremo trovato un ciclo.



Supponiamo, invece, che nel grafo esista un ciclo.

Abbiamo visto che nella foresta di visita partizioniamo gli archi come archi di foresta, archi all'indietro, archi di attraversamento, e archi in avanti.

Nello specifico abbiamo già osservato che se in un grafo è presente un ciclo necessariamente, nella foresta di visita, indipendentemente dalla foresta, troveremo almeno un arco all'indietro.

Chiaramente tutti i nodi del grafo sono presenti nella foresta, come anche gli archi. Infatti una qualunque foresta di visita, una volta completata con tutti gli archi è identica al grafo originale.

Di conseguenza, visto che abbiamo osservato che un arco all'indietro equivale alla presenza di un ciclo nel grafo, ed abbiamo anche notato che è possibile trovare un arco all'indietro solo se un nodo ha come adiacente un nodo precedentemente scoperto ma non ancora visitato, quindi di colore grigio, segue direttamente che se un nodo, esplorando la propria stella uscente trova un nodo grigio avremo la presenza di un arco all'indietro e quindi, equivalentemente, di un ciclo.

Pertanto risulta chiaro che in presenza di un ciclo nel grafo la funzione ricorsiva restituisce falso.

□

6.14.3 Complessità algoritmo ricerca cicli in un grafo

La complessità dell'algoritmo è analoga a quella del DFS. O meglio, il caso peggiore, quando il grafo è aciclico saremo costretti a percorrere tutti gli archi del grafo, pertanto ha una complessità identica a quella del DFS; nel caso migliore l'algoritmo trova subito un ciclo (un self loop nel primo nodo analizzato), ma va comunque considerato il costo dell'inizializzazione del vettore dei colori.

Quindi la complessità dell'algoritmo descritto è $O(|E| + |V|)$ e $\Omega(|V|)$.

6.15 Ordinamento topologico per un grafo

L'algoritmo che ricerca la ciclicità in un grafo risulta essere particolarmente importante, ha infatti numerose applicazioni pratiche (gestione delle dipendenze circolari in un compilatore, meccanismi di scheduling, etc ...).

Facciamo un esempio generale che può essere considerato un'astrazione di numerosi processi della vita quotidiana nel quale un algoritmo di ricerca di cicli in un grafo risulta utile.

Immaginiamo di avere un insieme di entità che sono tra di loro parzialmente ordinate che debbano essere però processate una alla volta.

Dobbiamo trovare quindi un ordine totale compatibile con l'ordinamento parziale tra le entità che ci permetta di processare le entità in sequenza rispettando le varie dipendenze tra entità.

Dobbiamo trovare la cosiddetta linearizzazione di un ordine parziale.

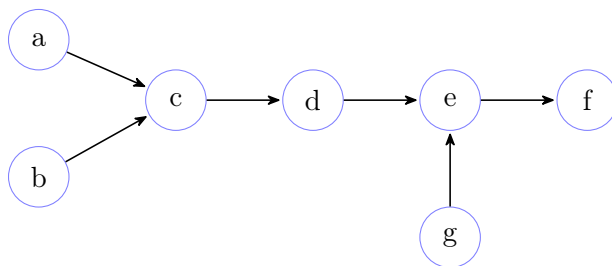
Si tratta cioè di prendere un ordine parziale ed incastrarlo in un ordine totale in modo tale da permettere di processare le varie entità una alla volta rispettando i vari vincoli.

Questi vincoli, affinché sia possibile trovare tale linearizzazione, devono formare un grafo aciclico. Non vi è modo infatti di linearizzare una dipendenza circolare.

Quindi è possibile prendere in considerazione questo insieme di entità, e vederle in modo astratto come nodi, e le varie dipendenze tra tali entità (ordinamenti parziali, totali, vincoli) vederle come archi, e costruire un grafo.

Il grafo così ottenuto viene chiamato proprio grafo delle dipendenze. Si tratta di un grafo aciclico, di cui potremmo voler trovare una cosiddetta linearizzazione; cioè un ordine dei vertici del grafo che rispetti i vincoli.

Esempio di linearizzazione su un grafo aciclico



Una delle possibili linearizzazioni di questo grafo aciclico è: a-b-c-d-g-e-f.

Attenzione che l'entità g non può essere né dopo l'entità e, né dopo l'entità f, poiché per i vincoli (gli archi), l'entità g deve computare prima di e ed f; inoltre, essa può essere anche la prima a processare, poiché l'entità g non dipende da nessun'altra entità.

Esistono quindi tante possibili linearizzazioni dato un grafo delle dipendenze.

Vedremo adesso come fare, a partire da un grafo aciclico, a trovare una delle possibili linearizzazioni, che viene detto ordinamento topologico del grafo.

Un ordinamento topologico per un grafo esisterà sempre e solo se il grafo è aciclico. Ecco perché l'algoritmo di ricerca di cicli in un grafo risulta essenziale in tutti questi discorsi.

Se il grafo è aciclico, infatti, ci sarà sempre almeno un ordinamento topologico.

Quello che faremo noi è, a partire da un grafo aciclico, ottenere uno dei possibili ordinamenti topologici.

Facciamo prima di tutto un'osservazione.

Quanti ordinamenti topologici ci sono in un grafo che è una sequenza di nodi totalmente ordinata?

Chiaramente uno soltanto, che è il grafo stesso.

Quanti ordinamenti topologici ci sono in un grafo che è formato da n nodi senza neanche un arco?

Vista l'assenza di vincoli una qualsiasi permutazione degli n nodi è un ordinamento topologico, pertanto $n!$.

Da questa osservazione segue chiaramente che considerando un grafo aciclico di n nodi le possibili linearizzazioni andranno da un minimo di 1 ad un massimo di $n!$.

Pertanto sicuramente non è possibile tentare di percorrere tutte le possibilità, bisogna trovare un modo intelligente di farlo.

Vediamo due diversi approcci per ottenere da un grafo aciclico un ordinamento topologico.

Il primo approccio è un approccio che segue una vista in profondità.

L'intuizione è la seguente. Assumiamo ovviamente che il grafo sia aciclico. Se esploriamo il grafo quello che staremo costruendo sarà una foresta in cui necessariamente abbiamo che gli archi che abbiamo esplorato in parte saranno parte della foresta. In particolare quello che andremo a esplorare come ultimo nodo sarà un nodo che nel momento in cui si analizza la sua stella uscente o la troviamo vuota oppure trova nodi già colorati di nero, visto che abbiamo detto che il grafo è assunto aciclico e se trovassimo nella stella uscente un nodo grigio significherebbe avere un arco all'indietro che equivale alla presenza di un ciclo.

Significa che grazie al partizionamento degli archi i nodi sono neri o perché sono parti di alberi valutati prima oppure sono teoricamente archi in avanti.

Ora, prendendo quindi l'ordine a ritroso, quello che otteniamo è che l'ultimo nodo che valutiamo è quello in cui tutte le dipendenze vanno ma non è sorgente di dipendenze verso altri. Tolto questo nodo quello subito prima sarà un nodo che può avere dipendenze in ingresso ma non è sorgente di dipendenze, e così via.

Quindi utilizzando l'ordine inverso rispetto a quello che ci viene dato dalla DFS otterremo automaticamente una linearizzazione che rispetta le dipendenze, ottenendo quindi un ordinamento topologico.

Ovviamente prima di vedere l'algoritmo è necessario formalizzare il concetto di ordinamento topologico.

Un ordinamento topologico di un grafo G è una permutazione dei vertici di G tale che per ogni coppia di vertici v, w tra cui esiste un arco il nodo v compare prima del nodo w nella permutazione.

Una permutazione di un insieme di elementi è una sequenza di quegli elementi in cui ogni elemento compare una e una sola volta.

Formalmente.

Un ordinamento topologico di un grafo G è una permutazione dei vertici di G tale che $\forall (v, w) \in E (v <_{\pi} w)$.
Dove

$$v <_{\pi} w : \Longleftrightarrow \exists i, j \in [0, |\pi|) (i < j \wedge (\pi)_i = v \wedge (\pi)_j = w) \quad (6.23)$$

Prima di passare all'algoritmo dimostriamo un semplice lemma che afferma che se G è un grafo ciclico, allora non esiste un ordinamento topologico per G .

L'intuizione è che in un ciclo non è possibile definire un ordine totale tra gli elementi che rispetti la ciclicità, non è possibile nemmeno definire un punto di partenza o di arrivo di una permutazione.

Lemma 6.15.1 (Non esistenza ordinamento topologico in grafo aciclico).

Se G è ciclico allora non esiste un ordinamento topologico per G .

Dimostrazione.

Dimostriamo il lemma per assurdo.

Supponiamo per assurdo che, nonostante il grafo G sia ciclico, esista un ordinamento topologico per G .

Chiaramente, G è ciclico $\iff \exists \pi \in Path(G) (first(\pi) = last(\pi))$. Dove, $\pi = \pi_0, \pi_1, \dots, \pi_{k-1}, \pi_k$, e $\pi_0 = \pi_k$.

Sia ρ un ordinamento topologico per G . Per definizione ρ contiene tutti i vertici del grafo, e in particolare contiene π_0 .

Visto l'esistenza del percorso π necessariamente l'ordinamento topologico deve rispettare i vincoli del percorso; quindi avremo $\rho = v_0, \dots, \pi_0, \dots, \pi_1, \dots, \pi_{k-1}, \dots, \pi_k, \dots$, ma come abbiamo osservato $\pi_0 = \pi_k$; siamo giunti ad un assurdo.

□

6.15.1 Algoritmo: ordinamento topologico in un grafo basato algoritmo DFS

Il seguente algoritmo risulta particolarmente importante perché funzionerà non solo per costruire un ordinamento topologico su un grafo aciclico ma anche su grafi ciclici dove la semantica non è restituire un ordinamento topologico, ma sarà una sorta di permutazione topologica di un sottografo, quindi per le componenti fortemente connesse.

Essendo una visita in profondità l'algoritmo è formato da una parte non ricorsiva e da una parte ricorsiva.

Il risultato dell'algoritmo sarà uno stack.

Ovviamente uno stack si può leggere dalla testa verso la coda e l'ordine in cui andremo a leggere sarà esattamente l'ordine dell'ordinamento topologico.

Quindi in testa nello stack avremo il nodo che è quello che vincola di più potenzialmente gli altri e in coda, in fondo a tutto, ci sarà quello che è il più vincolato ma che non vincola nessuno.

Si tratta in ogni caso dell'ennesima variante di una visita in profondità.

Ecco l'algoritmo.

Algoritmo 89: Funzione per l'ordinamento topologico in un grafo (DFS)

Signature `topological_ordering`: $Graph(D) \rightarrow Stack(D)$

```

function topological_ordering(G)
    // inizializzo il vettore di colori a bianco con una funzione accessoria
    // e creo uno stack vuoto
1   (c, S) ← (init(G), empty_stack())
    // per ogni nodo del grafo
2   foreach (v in V) do
        // se è un nodo da scoprire
3       if (c[v] = WHITE) then
            // chiamo la parte ricorsiva
            // che esplora il grafo a partire dal nodo in questione
            // e aggiorna lo stack
4       S ← topological_ordering(G, S, v)
    // restituisce lo stack
5   return S

```

Ecco la parte ricorsiva dell'algoritmo.

Algoritmo 90: Funzione accessoria per l'ordinamento topologico in un grafo, parte ricorsiva (DFS)

Signature `topological_ordering`: $Graph(D) \times Stack(D) \times D \rightarrow Stack(D)$

```

function topological_ordering(G, S, v)
    // indico che il nodo in questione è stato scoperto
    // colorandolo di grigio
1   c[v] ← GRAY
    // per ogni nodo nella stella uscente del nodo in input
2   foreach (w in Adj[v]) do
        // se il nodo è da scoprire
3       if (c[w] = WHITE) then
            // richiamo ricorsivamente la funzione
            // sul nodo in questione aggiornando lo stack
4       S ← topological_ordering(G, S, w)
    // indico che il nodo è stato visitato
    // colorandolo a nero
    // e lo inserisco nello stack
5   (c[v], S) ← (BLACK, push(S, v))
    // restituisco lo stack
6   return S
  
```

Essenzialmente l'unica cosa che fa questo algoritmo è esplorare ciò che può esplorare del grafo e mettere in uno stack i vertici nell'ordine di fine visita.

Infatti, come è immediato notare, l'algoritmo inserisce i nodi nello stack non appena questi vengono colorati di nero.

Quindi andranno nello stack prima i nodi che terminano la visita prima e dopo quelli che terminano la visita dopo.

Significa che in testa allo stack avremo il nodo con il tempo di fine visita più alto.

Se ricordiamo la rappresentazione delle parentesi che abbiamo visto, il nodo che viene scoperto per primo in un albero della foresta di visita è quello che termina dopo, tutti gli altri saranno dipendenti da questo nodo, perché appunto c'è un percorso nella foresta di visita.

Quindi significa che l'algoritmo inserirà come primo elemento dello stack il vertice con maggiore necessità di dipendenze, poi verrà inserito un nodo che non ha bisogno di quello già presente nello stack, e così via fino all'ultimo nodo che è quello che non ha bisogno di alcun nodo.

Quindi estraendo gli elementi dallo stack così costruito avremo proprio un ordinamento topologico sul grafo in input.

È importante sottolineare che l'ordinamento topologico risultante deriva ovviamente dall'ordine di visita dei nodi del grafo, sul quale, come già detto, non abbiamo nessun controllo; pertanto la linearizzazione risultante sarà solo una delle possibili, ma non sarà possibile prevedere né decidere quale.

6.15.2 Correttezza algoritmo `topological_ordering` (DFS)

Quello che vedremo è che quello che l'algoritmo tirerà fuori sarà uno stack il cui ordine è compatibile con quello dell'ordinamento topologico.

Questo è un modo costruttivo per dimostrare che se il grafo è aciclico l'ordinamento topologico esiste.

Quello che vogliamo dimostrare è la seguente cosa.

Prendendo lo stack risultante dall'algoritmo appena descritto, l'ordine dello stack è un ordinamento topologico.

Formalmente.

Proposizione 6.15.1 (Correttezza `topological_ordering` (DFS)).

Sia S lo stack risultante dall'algoritmo `topological_ordering`.

$$S = v_0 v_1 \dots v_k = \pi \implies \forall (u, w) \in E (\exists i, j \in [0, k] (v_i = u \wedge v_j = w \wedge i < j)).$$

Dimostrazione.

Sia $S = v_0 v_1 \dots v_k$ lo stack ottenuto dall'algoritmo `topological_ordering`.

Visto il funzionamento dell'algoritmo avremo chiaramente che nello stack verranno posti prima i vertici con tempo di fine visita minore e via via nodi con tempo di fine visita maggiore; pertanto avremo $e[v_0] > e[v_1] > \dots > e[v_k]$.

Quindi per dimostrare la proposizione ci basterà far vedere che $\forall u, w \in V ((u, w) \in E \implies e[u] > e[w])$.

Siano ora, $u, w \in V ((u, w) \in E)$.

Poniamoci all'istante $s[u]$, e consideriamo il comportamento dell'algoritmo.

Visto che il grafo è aciclico il vertice w sicuramente non sarà grigio.

Se w è bianco avremo un percorso bianco da u a w , e quindi, per i lemma precedentemente dimostrati, avremo $s[u] < s[w] < e[w] < e[u]$; visto che sappiamo che w sarà discendente di u nella foresta di visita, in quanto l'arco tra u e w è un arco di foresta.

Se, invece, w è nero l'arco tra u e w è necessariamente un arco di attraversamento. Avremo che nel momento $s[u]$ il nodo w è già stato visitato; pertanto, per il teorema delle parentesi, $s[w] < e[w] < s[u] < e[u]$.

□

6.15.3 Algoritmo: ordinamento topologico in un grafo basato grado entrante

Come preannunciato vedremo ora un altro algoritmo per la risoluzione dello stesso problema, trovare un ordinamento topologico su un grafo, che si basa però su una visita che si può dire in qualche modo più ispirata, anche se non è una visita, alla visita in ampiezza rispetto alla visita in profondità.

Prendiamo come esempio un grafo aciclico, perché come sappiamo in un grafo aciclico esiste almeno un ordinamento topologico.

Vediamo adesso un esempio di grafo, in cui sopra ogni nodo riporteremo il loro grado entrante.

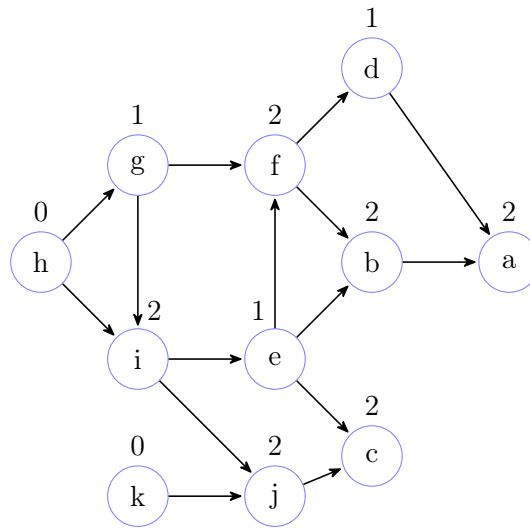


Figura 6.17: Esempio di grafo aciclico con grado entrante

Cominciamo ad osservare delle cose.

Sicuramente un grafo aciclico, per quanto possa essere complesso, deve necessariamente avere almeno un vertice con grado entrante zero; può averne più di uno ma ne deve avere necessariamente uno.

Perché intuitivamente in un grafo finito se nessun nodo ha grado entrante zero noi possiamo creare un ciclo.

Quindi quello che osserviamo come prima cosa è che in un grafo aciclico finito deve esistere almeno un nodo con grado entrante zero; dualmente deve esistere almeno un nodo con grado uscente zero.

L'algoritmo di visita in profondità che abbiamo utilizzato per il primo algoritmo non sfrutta il grado uscente zero ma in effetti quello che fa è mettere in fondo a uno stack necessariamente un nodo il cui grado uscente è zero e va a ritroso.

L'algoritmo che vedremo ora fa esattamente l'opposto, cercherà di partire da un nodo con grado entrante zero; se il nodo ha grado entrante zero significa che è libero da qualunque vincolo e quindi può essere messo come primo elemento dell'ordinamento topologico (in caso di più nodi con grado entrante zero la scelta è indifferente

in quanto tali nodi non subiscono vincoli, tantomeno tra di loro).

Se una volta identificato questo nodo andiamo a rimuoverlo dal grafo, quello che otterremo sarà un sottografo, in particolare sarà un sottografo indotto perché togliamo un nodo e tutti i suoi archi uscenti.

È molto facile dimostrare che un sottografo indotto di un grafo aciclico è a sua volta aciclico perché ovviamente se ci fosse un ciclo nel sottografo, poichè si preservano tutti gli archi, cioè tutti gli archi del sottografo sono archi del grafo originale se abbiamo un ciclo del sottografo dovremmo avere un ciclo anche nel grafo originale, il che è assurdo visto che per assunto il grafo originale è aciclico.

Allora cosa significa?

Significa che nel sottografo ci troviamo esattamente nella stessa identica situazione del grafo originale, cioè un grafo aciclico e pertanto deve avere anche lui almeno un nodo con grado entrante zero.

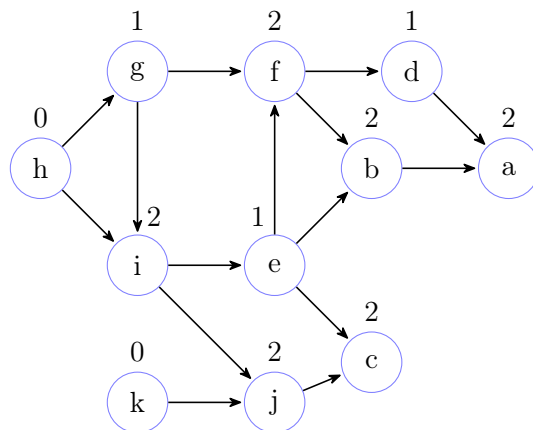
Quindi possiamo reiterare questa idea di mettere nella permutazione i nodi con grado entrante zero partendo da uno dei nodi del grafo originale, man mano eliminando questi nodi nel grafo e alla fine, ovviamente eliminando ad ogni iterazione un nodo, visto che il grafo è finito, prima o poi termineremo arrivando alla sequenza completa.

Vediamo adesso un'esecuzione dell'algoritmo sfruttando la nostra idea intuitiva.

Passo 1:

Ci sono due nodi con grado entrate 0 (h,k),
scegliamo arbitrariamente uno dei due.

Nel nostro caso specifico è stato scelto h

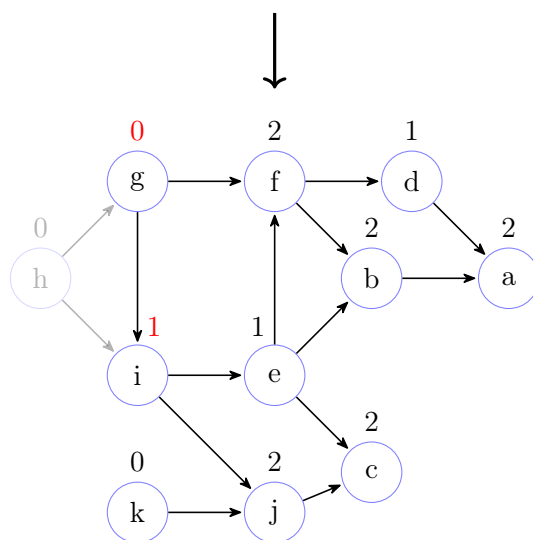


Passo 2:

Dopo aver eliminato il nodo h e i suoi archi uscenti
(dopo ogni eliminazione, in figura, essi verranno sbiaditi),
e corretto il grado entrate delle loro destinazioni
(ad ogni passo, i gradi entranti corretti
saranno colorati di rosso).

Ci sono due nodi con grado entrate 0 (g, k),
scegliamo arbitrariamente uno dei due.

Nel nostro caso specifico è stato scelto g

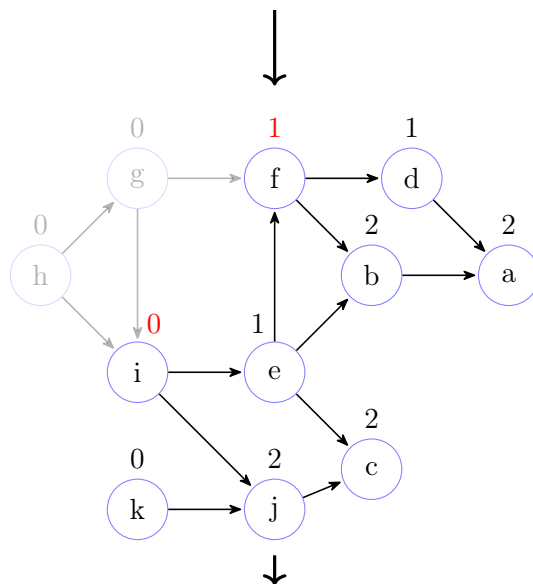


Passo 3:

Dopo aver eliminato il nodo g e i suoi archi uscenti
e corretto il grado entrate delle loro destinazioni

Ci sono due nodi con grado entrate 0 (i, k),
scegliamo arbitrariamente uno dei due.

Nel nostro caso specifico è stato scelto i

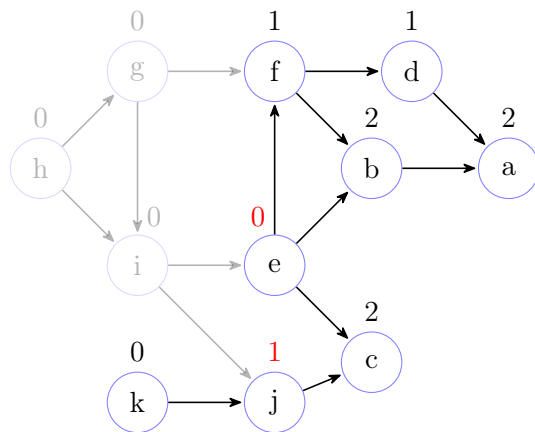


Passo 4:

Dopo aver eliminato il nodo i e i suoi archi uscenti e corretto il grado entrate delle loro destinazioni

Ci sono due nodi con grado entrate 0 (e, k), scegliamo arbitrariamente uno dei due.

Nel nostro caso specifico è stato scelto e

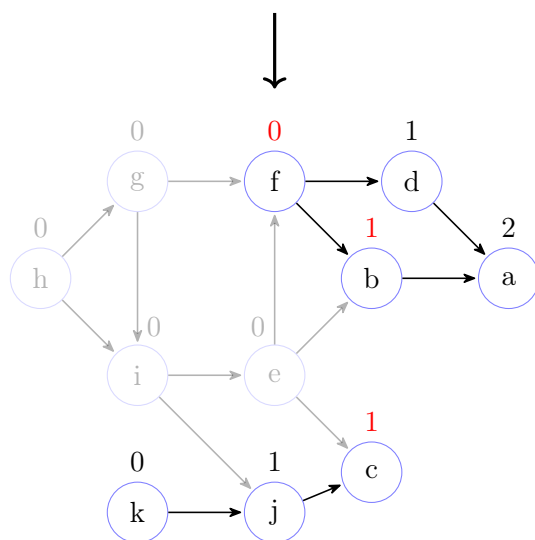


Passo 5:

Dopo aver eliminato il nodo e e i suoi archi uscenti e corretto il grado entrate delle loro destinazioni

Ci sono due nodi con grado entrate 0 (f, k), scegliamo arbitrariamente uno dei due.

Nel nostro caso specifico è stato scelto f

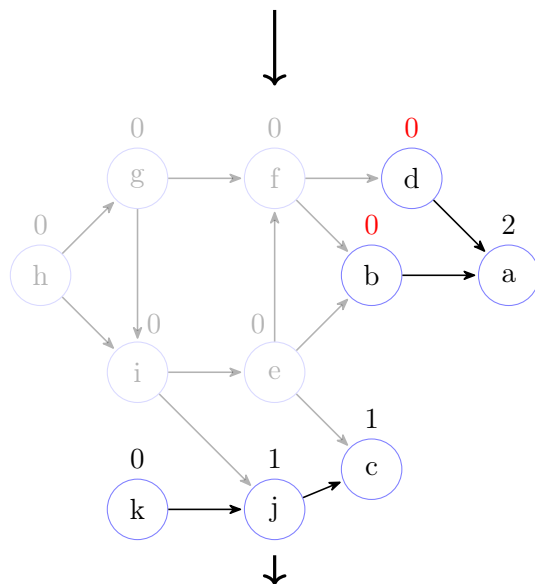


Passo 6:

Dopo aver eliminato il nodo f e i suoi archi uscenti e corretto il grado entrate delle loro destinazioni

Ci sono tre nodi con grado entrate 0 (d, b, k), scegliamo arbitrariamente uno dei due.

Nel nostro caso specifico è stato scelto d

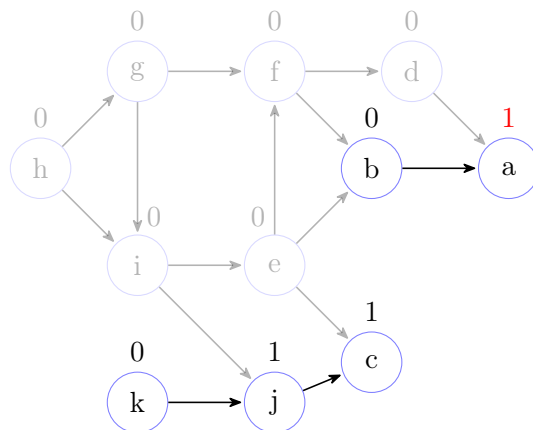


Passo 7:

Dopo aver eliminato il nodo d e i suoi archi uscenti e corretto il grado entrate delle loro destinazioni

Ci sono due nodi con grado entrate 0 (b, k), scegliamo arbitrariamente uno dei due.

Nel nostro caso specifico è stato scelto b

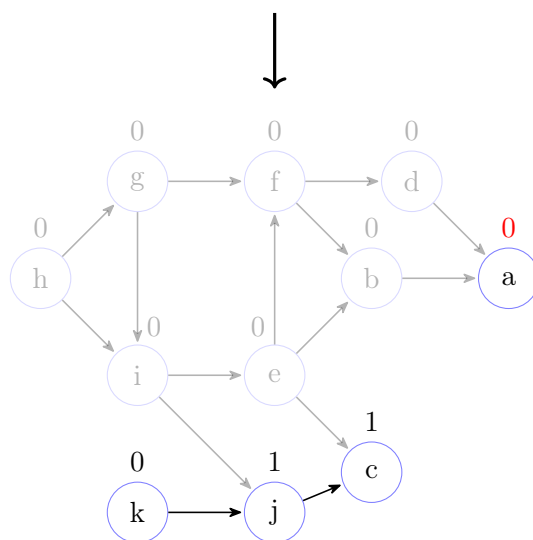


Passo 8:

Dopo aver eliminato il nodo b e i suoi archi uscenti e corretto il grado entrate delle loro destinazioni

Ci sono due nodi con grado entrate 0 (a, k), scegliamo arbitrariamente uno dei due.

Nel nostro caso specifico è stato scelto a

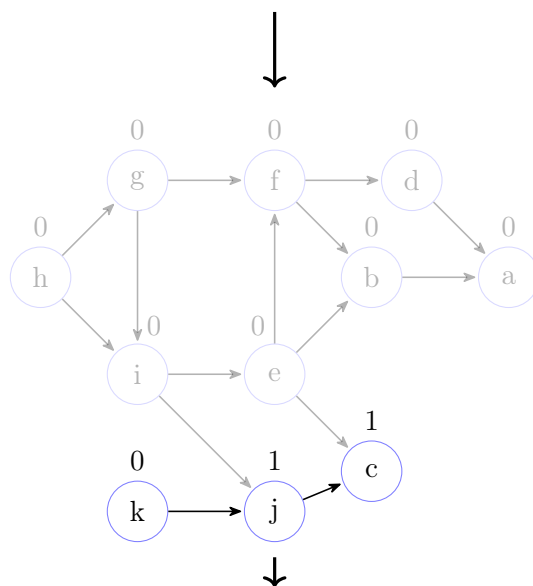


Passo 9:

Dopo aver eliminato il nodo a e i suoi archi uscenti e corretto il grado entrate delle loro destinazioni

C'è un nodo con grado entrate 0 (k).

Siamo quindi obbligati a scegliere k

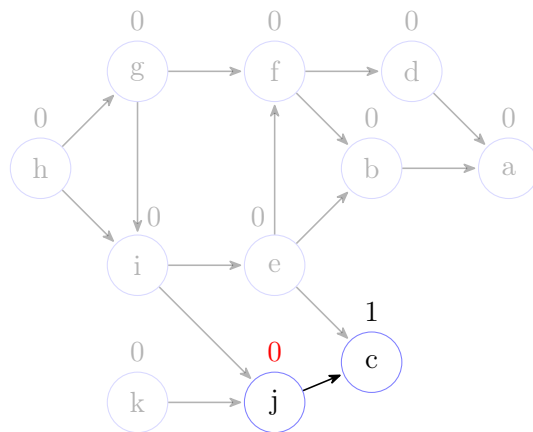


Passo 10:

Dopo aver eliminato il nodo k e i suoi archi uscenti
e corretto il grado entrate delle loro destinazioni

C'è un nodo con grado entrate 0 (j).

Siamo quindi obbligati a scegliere j

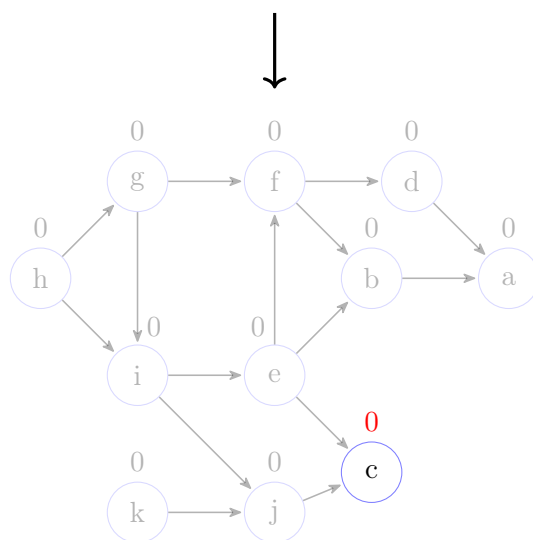


Passo 10:

Dopo aver eliminato il nodo j e i suoi archi uscenti
e corretto il grado entrate delle loro destinazioni

C'è un nodo con grado entrate 0 (c).

Siamo quindi obbligati a scegliere c

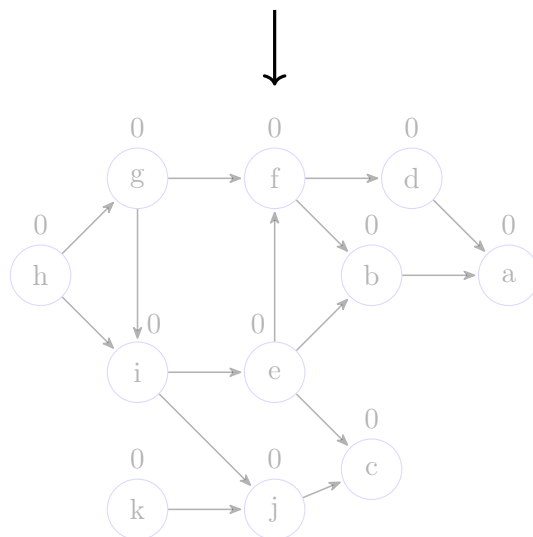


Passo 11:

Dopo aver eliminato il nodo c e i suoi archi uscenti
e corretto il grado entrate delle loro destinazioni

Poiché non ci sono più altri nodi, la simulazione termina.

L'ordine topologico creato da questa simulazione è
il seguente: h - g - i - e - f - d - b - a - k - j - c



Grazie all'idea intuitiva abbiamo visto un esempio di esecuzione; quello che farà l'algoritmo è esattamente la stessa identica cosa ma cercherà di sfruttare un trucco per evitare di calcolare il sottografo, perché creare un sottografo significa implementare una nuova struttura in memoria dove riversare informazioni ecc. Invece di creare esplicitamente il sottografo si terrà traccia del grado entrante in un opportuno vettore. Si calcola questo grado entrante inizialmente andando ad esplorare per ogni nodo la stella uscente e ad ogni arco si incrementerà il contatore della destinazione.

Per quello che abbiamo detto sappiamo che esiste almeno un nodo con grado entrante 0. Prendiamo e collezioniamo tutti i nodi con grado entrante 0 manualmente, scorrendo questo vettore alla ricerca di 0 per ogni iterazione e spostiamo il nodo corrispondente in una coda (va bene una qualsiasi struttura di supporto), ovviamente per ogni nodo eliminato andranno aggiornati i contatori nel vettore, decrementando di una unità i contatori associati ai nodi che si trovano nella stella uscente del nodo rimosso.

In questo modo possiamo evitare quindi di costruire un sottografo per ogni iterazione.

Anche questo algoritmo, come il precedente, estrae uno dei possibili ordinamenti topologici ma non ci permette di prevedere né scegliere quale sia l'ordinamento topologico in output.

Possiamo però essere sicuri che restituisca un ordinamento topologico perché si procede dai nodi meno vincolati e man mano che si tolgono quei nodi si eliminano i relativi vincoli.

È interessante sottolineare che tutti i ragionamenti fatti per il grado entrante possono essere fatti tal quali considerando il grado uscente.

Prima però di passare all'implementazione di quest'idea nell'algoritmo vediamo i due lemma su cui si basa.

Il primo lemma afferma che preso un grafo aciclico e finito (è fondamentale che sia finito in quanto esistono grafi aciclici e infiniti con nodi avente tutti grado entrante non nullo) esiste almeno un nodo del grafo avente grado entrante zero.

Formalmente.

Lemma 6.15.2 (Grado entrante grafo aciclico finito).

Sia G grafo aciclico finito $\implies \exists v \in V (ed(v) = |\{(w, v) \in E \mid w \in V\}| = 0)$.

Dimostrazione.

Dimostriamo il lemma per assurdo negando la tesi.

Supponiamo, quindi, che se G è un grafo aciclico finito $\implies \forall v \in V (ed(v) > 0)$.

Chiaramente, visto che il grafo è aciclico, i self loop sono esclusi.

Senza ledere in generalità, supponiamo che $|V| = n$, con $n \in \mathbb{N}$ (il caso con $n = 0$ risulta vacuamente vero).

Abbiamo supposto che ogni nodo ha grado entrante non nullo e, pertanto possiamo immaginare, una sequenza di archi tra i vertici del grafo (l'ordine risulta chiaramente indifferente ai nostri scopi).

Prendiamo in considerazione v_{n-1} . Visto che non possono essere presenti self loop, supponiamo che esista un arco da v_{n-2} ad v_{n-1} .

$$v_{n-2} \longrightarrow v_{n-1}$$

In generale, possiamo iterare tale ragionamento fino a v_0 . Quindi, $\forall i \in [0, n-1) (\exists (v_i, v_{i+1}) \in E)$.

$$v_0 \longrightarrow v_1 \longrightarrow \cdots \longrightarrow v_{n-2} \longrightarrow v_{n-1}$$

Ma anche v_0 ha grado entrante non nullo e qualsiasi scelta possiamo fare per definire un arco entrante in v_0 porta alla genesi di un ciclo.

□

Il secondo lemma afferma invece che preso un grafo aciclico G , qualsiasi sottografo G' di G è a sua volta aciclico.

Lemma 6.15.3 (Sottografo di un grafo aciclico).

Sia G grafo aciclico $\implies \forall G' \sqsubseteq G$ (G' è aciclico).

Dimostrazione.

Dimostriamo il lemma per assurdo negando la tesi.

Supponiamo quindi che se G è un grafo aciclico $\implies \exists G' \sqsubseteq G$ (G' è ciclico).

Ma, se G' è ciclico $\implies \exists \pi \in \text{Cyclic}(G') \implies \pi \in \text{Path}(G) \implies \pi \in \text{Cyclic}(G) \implies G$ è ciclico.

□

Grazie a questi lemma abbiamo modo di costruire l'algoritmo e anche dimostrarne la correttezza.

Vediamo adesso come è fatto l'algoritmo.

La prima cosa che dobbiamo fare è andare a definire una funzione che è quella che calcola il grado entrante sul grafo iniziale.

Quello che dobbiamo fare è semplicemente restituire in output un vettore lungo quanto la cardinalità dell'insieme di vertici del grafo al cui interno è contenuto per ogni vertice il grado entrante relativo.

Algoritmo 91: Funzione ausiliaria per il calcolo del grado entrante sul grafo

Signature entering_degree: $Graph(D) \rightarrow Array(\mathbb{N})$

```

function entering_degree(G)
    // ed: vettore contenente i contatori per il grado entrante
    // di ciascun vertice del grafo
    // setto a zero tutti i contatori
1  foreach (v in V) do
2    | ed[v] ← 0
    // per ogni nodo del grafo
3  foreach (v in V) do
    | // ne esploro la stella uscente
4    | foreach (w in Adj[v]) do
    | | // ed incremento il contatore
    | | // del vertice destinazione di un arco
5    | | ++ed[w]
    // restituisco il vettore
6  return ed

```

Attenzione che i due cicli **for** dell'algoritmo non possono essere fusi in quanto è necessario azzerare prima tutti i contatori per poi essere sicuri che il conteggio alla fine sia corretto.

Risulta immediato che il primo ciclo **for** ha complessità $\Theta(|V|)$, mentre il secondo visto che esplora tutti gli archi avrà complessità $\Theta(|E|)$; quindi nel complesso tale funzione ha complessità $\Theta(|E| + |V|)$.

Vediamo adesso un'altra funzione nella quale, dopo aver calcolato il vettore dei gradi entranti, lo scorriamo alla ricerca dei nodi con grado entrante zero, che sappiamo esistere.

Di conseguenza restituiranno sempre una coda Q non vuota, eccezion fatta se il grafo è vuoto.

Algoritmo 92: Funzione ausiliaria per trovare e restituire l'insieme dei vertici con grado entrante zero

Signature `init_queue`: $Graph(D) \times Array(\mathbb{N}) \rightarrow Queue(D)$

```

function init_queue(G, ed)
    // creo una coda vuota
1   Q  $\leftarrow$  empty_queue()
    // per ogni vertice nel grafo
2   foreach (v in V) do
        // se il grado entrante del vertice è 0
3       if (ed[v] = 0) then
            // lo inserisco in coda
4         Q  $\leftarrow$  enqueue(Q, v)
    // restituisco la coda
5   return Q
  
```

Chiaramente tale funzione deve scorrere tutto il vettore in input che sappiamo avere dimensione pari alla cardinalità dell'insieme dei vertici del grafo, pertanto la complessità di tale funzione sarà $\Theta(|V|)$.

Vediamo ora l'ultima parte dell'algoritmo.

Algoritmo 93: Funzione per l'ordinamento topologico in un grafo aciclico basato su grado entrante

Signature `topological_ordering: Graph(D) → List(D)`

```

function topological_ordering(G)
    // mediante una funzione accessoria
    // riempio il vettore dei gradi entranti
1  ed ← entering_degree(G)
    // mediante una funzione accessoria
    // riempio una coda con i nodi avente grado entrante 0
2  Q ← init_queue(G, ed)
    // visto che il grafo potrebbe essere vuoto
    // anche la coda potrebbe esserlo
    // quindi è necessario un ciclo while
    // fino a quando la coda non è vuota
3  while (NOT is_empty(Q)) do
    // rimuovo un nodo dalla coda
4  (Q, v) ← head_&_dequeue(Q)
    // per ogni nodo della stella uscente
    // del nodo rimosso dalla coda
    // devo aggiornare il contatore del grado entrante
    // mimando la rimozione del nodo dal grafo
5  foreach (w in adj(v)) do
    // decremento di una unità
    // il valore del contatore del grado entrante
6  ed[w] ← ed[w] - 1
    // se dopo il decremento il grado del nodo in questione è 0
7  if (ed[w] = 0) then
    // inserisco il nodo nella coda dei nodi
    // con grado entrante 0
8  Q ← enqueue(Q, w)
    // inserisco il nodo rimosso dalla coda in coda ad una lista
    // che rappresenta l'ordinamento topologico da restituire
9  L ← append(L, v)
    // restituisco la lista
10 return L
  
```

Notiamo che per i ragionamenti fatti in precedenza ogni nodo del grafo prima o poi entrerà in coda e quindi il ciclo `while` verrà eseguito esattamente tante volte quanti sono i vertici del grafo; mentre il ciclo `for` interno al `while` permette di percorrere tutti gli archi; quindi tutto il ciclo `while` ha una complessità che è $\Theta(|E| + |V|)$.

Considerando anche le funzioni che vengono chiamate prima del ciclo `while`, di cui abbiamo già espresso la complessità, otteniamo che asintoticamente la complessità non cambia; quindi nel complesso l'algoritmo ha una complessità che è $\Theta(|E| + |V|)$.

6.16 Componenti connesse e fortemente connesse in un grafo

Ci concentreremo ora sul calcolo delle componenti connesse e delle componenti fortemente connesse.

Cominciamo quindi con dei grafi non orientati.

Immaginiamo di voler trovare un algoritmo che ci permetta di calcolare quali sono le componenti connesse.

Una componente connessa è massimale se è una componente connessa e allo stesso tempo non possiamo aggiungervi un altro nodo del grafo facendo in modo che la proprietà di essere una componente connessa si preservi.

Vediamo qualche esempio per capire meglio il concetto.

I grafi proposti saranno non orientati.

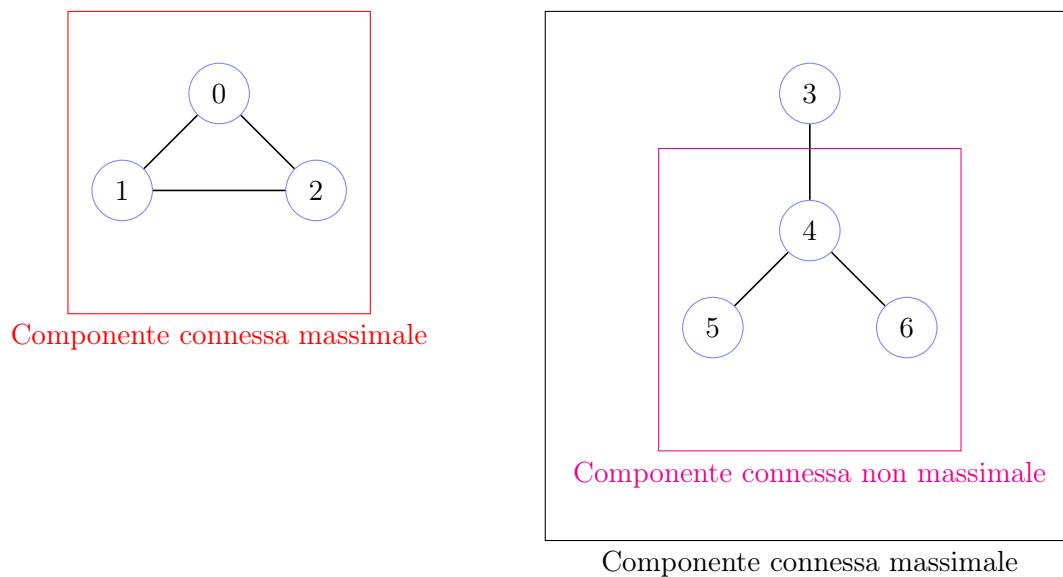


Figura 6.18: Esempio di componenti connesse e componenti connesse massimali

L'algoritmo che vedremo calcola le componenti connesse massimali.

Risulta immediato che tra gli algoritmi che abbiamo visto quello che ci può dare tale informazione è la visita in profondità in quanto come sappiamo crea la foresta di visita indotta sul grafo. E la foresta di visita ci permette di dire quali nodi possiamo raggiungere da un altro nodo.

Se ora andassimo a vedere un grafo non orientato come un grafo orientato con gli archi ambo i versi, la foresta che costruirebbe l'algoritmo di visita in profondità, ci permetterebbe di dire, ovviamente, che dal nodo radice di ogni albero della foresta è possibile raggiungere ogni nodo dello stesso albero; ma visto che per ogni arco esiste anche l'arco di verso opposto significa anche che ogni nodo di un albero della foresta raggiunge la radice, e quindi tutti i nodi dello stesso albero possono raggiungersi a vicenda.

Questo significa, quindi, che se prendiamo un grafo non orientato, lo interpretiamo come orientato in cui gli archi li abbiamo ambo i versi e applichiamo una visita in profondità ed estraiamo dal vettore dei predecessori la foresta di visita, ogni albero della foresta è una componente connessa massimale.

Perché massimale?

Sappiamo che gli archi del grafo sono tutti quanti necessariamente o archi della foresta, o archi all'indietro, o archi in avanti, o archi di attraversamento.

Ovviamente, se il grafo è non orientato e lo vediamo come un grafo orientato ambiverso ovviamente ci sono tanti cicli corrispondenti ad archi all'indietro e potrebbero esserci anche diversi archi in avanti.

Gli archi che però non è possibile trovare in un grafo non orientato sono gli archi di attraversamento. Perché?

Supponiamo per assurdo l'esistenza di un arco di attraversamento, visto che gli archi sono ambiversi necessariamente avremo che nell'esplorazione del grafo troveremo uno dei due estremi colorati di bianco (a differenza di quanto può accadere in un grafo orientato) e pertanto non sarebbe classificato come arco di attraversamento.

Quindi ogni albero della foresta è una componente connessa massimale.

Quindi una semplice visita in profondità sulla versione orientata del grafo non orientato ci permette di calcolare immediatamente le componenti connesse massimali.

Di conseguenza siamo in grado di partizionare il grafo non orientato nei pezzi che lo compongono e che sono tra di loro indipendenti.

E nel caso di un grafo orientato?

Possiamo sperare che in un grafo orientato, quindi in cui la proprietà che ad ogni arco corrisponde l'inverso non è assicurata, che sia sufficiente semplicemente fare una visita in profondità e ottenere la foresta di visita per capire subito, così come nel caso dei grafi non orientati, qual è il partizionamento ed equivalentemente le componenti fortemente connesse?

Attenzione. Nota importante. Sebbene in algebra, nella teoria dei grafi, in un grafo orientato una componente fortemente connessa è un qualunque sottoinsieme di vertici del grafo che percorrendo gli archi secondo il loro verso riescono a raggiungersi vicendevolmente, in computer science quando si parla di componente fortemente connessa di un grafo orientato si fa riferimento ad una componente fortemente connessa massimale.

Pertanto, quando parleremo di componente fortemente connessa di un grafo orientato, scc (strongly connected component), intendiamo una componente fortemente connessa massimale del grafo orientato.

Facciamo un esempio considerando un grafo orientato ed un ordine di scoperta dei vertici arbitrario.

Dati i seguenti grafi, mostreremo le loro foreste di visita con partizionamento degli archi del grafo.

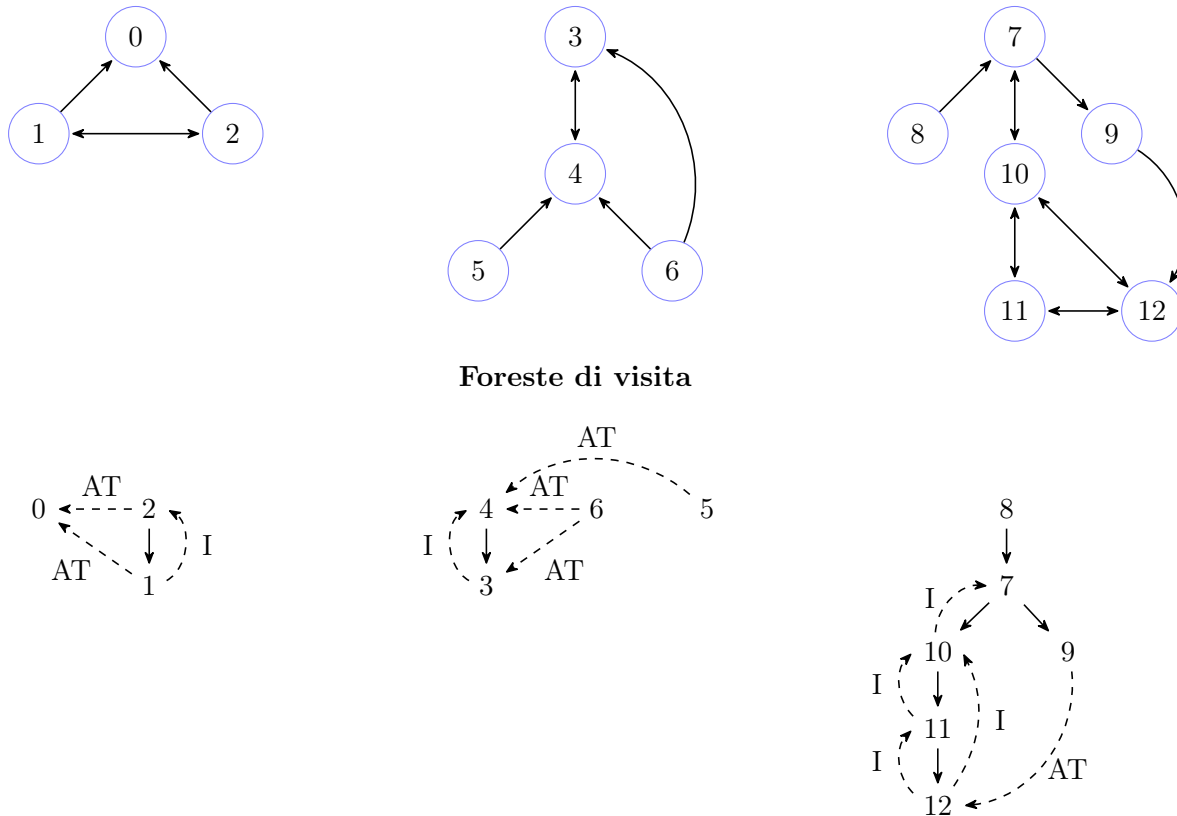


Figura 6.19: Esempio di foreste di visita partendo da grafi orientati

Risulta evidente che la foresta in qualche modo ci può dare informazioni, almeno parziali, su qual è la struttura delle componenti.

Infatti se due nodi appartengono ad alberi diversi questi non possono essere parte della stessa componente perché se troviamo un arco di attraversamento è vero che c'è un percorso tra i due nodi, ma tale percorso non è in ambo i sensi, altrimenti durante l'esplorazione del grafo da parte dell'algoritmo avremmo trovato un percorso bianco e pertanto i due nodi farebbero parte dello stesso albero.

È lo stesso criterio che abbiamo usato prima e che deriva dal teorema del percorso bianco.

Tuttavia, se rappresentiamo la foresta nell'ordine temporale di scoperta, possiamo avere archi da alberi ad alberi diretti da destra verso sinistra, ma non il contrario. Possiamo quindi, seguendo una rappresentazione temporale, avere solo archi che vanno da alberi che sono stati scoperti dopo verso alberi che sono stati scoperti prima.

Il che significa quindi che da una componente potenziale possiamo raggiungere altre componenti già scoperte ma non il viceversa.

Il discorso sembrerebbe simile a quello che abbiamo affrontato per i grafi non orientati, ma non è esattamente identico perché all'interno di un albero potremmo avere più componenti.

In un albero, ora, non è più detto che un nodo qualsiasi raggiunga sempre almeno la radice, visto che gli archi non sono necessariamente bidirezionali.

Gli alberi sono infatti percorribili, come sempre, dalla radice verso le foglie; ma non è detto che sia vero anche il contrario.

Questo è il motivo per il quale all'interno di un albero possiamo avere più componenti.

Quello che mostreremo è che le componenti fortemente connesse in un grafo orientato sono necessariamente tutte contenute all'interno di uno stesso albero della foresta di visita, non possono essere separate in alberi perché altrimenti avremmo archi tra alberi che vanno anche da sinistra verso destra, che come abbiamo già detto è impossibile.

Prendiamo le foreste di visita dell'esempio precedente e individuiamo le componenti connesse.

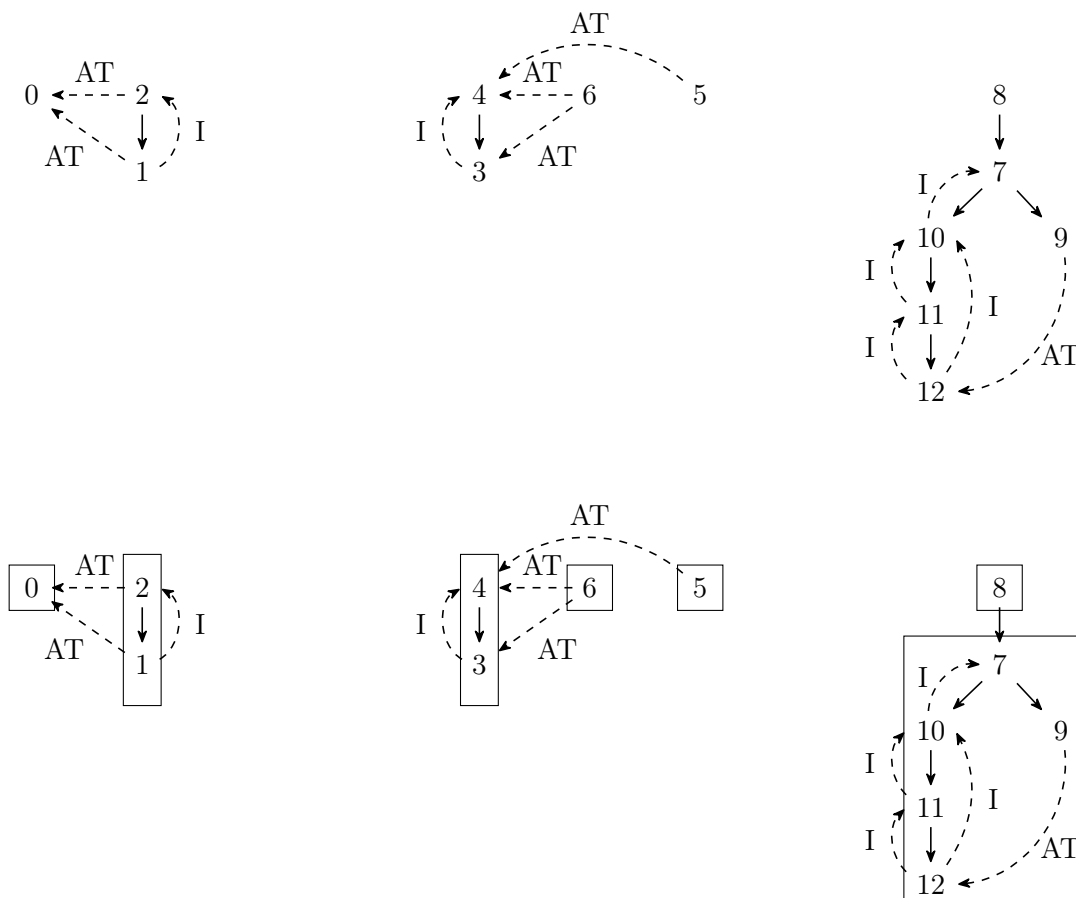


Figura 6.20: Esempio di componenti connesse nelle foreste di visita

Vediamo un ulteriore esempio che ci servirà ad introdurre poi l'algoritmo.

Questo è il grafo.

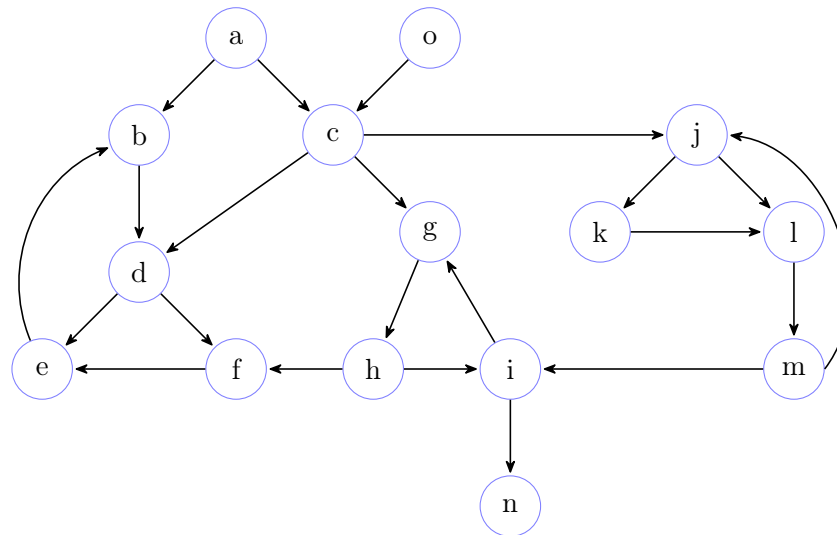
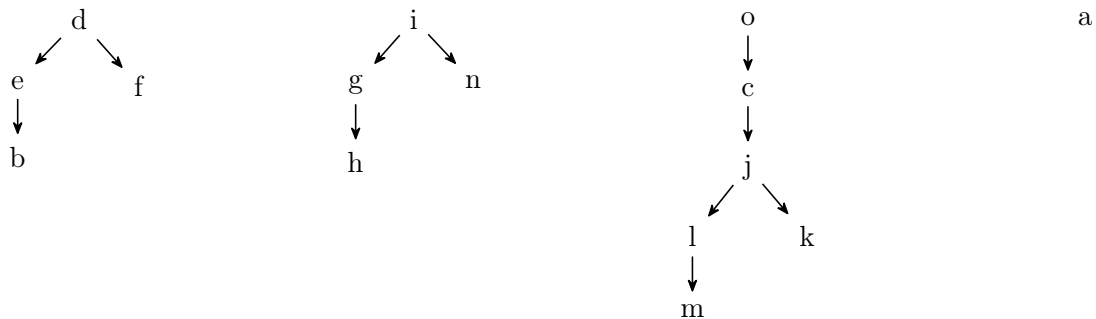
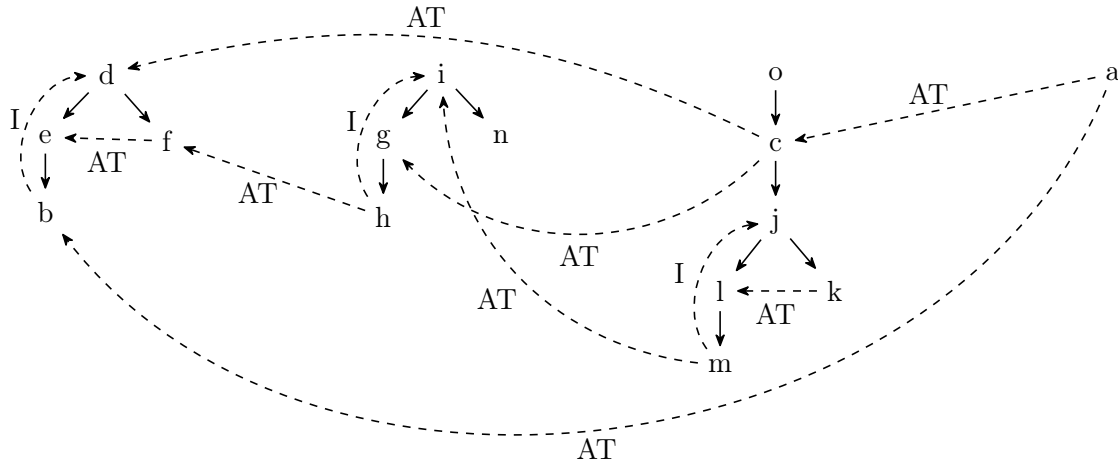


Figura 6.21: Ulteriore esempio di grafo

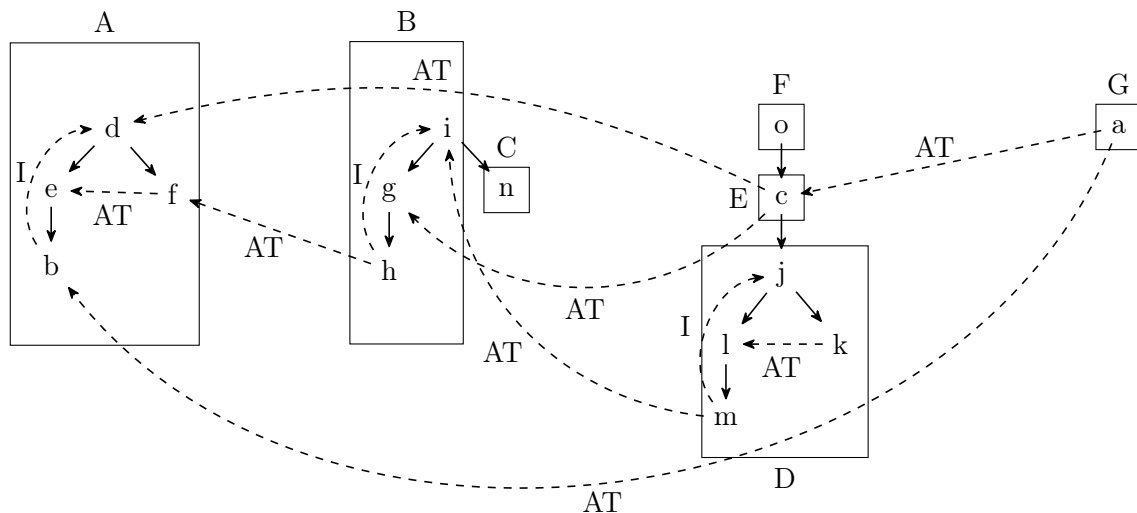
Come foresta di visita considereremo la seguente.



La foresta di visita con gli archi del grafo partizionati è la seguente.



Troviamo adesso le componenti fortemente connesse di questo grafo.



Ripetiamo che la relazione di connessione in un grafo è una relazione binaria sul grafo che risulta essere riflessiva, simmetrica e transitiva, è pertanto una relazione di equivalenza.

Pertanto ogni componente connessa è una classe di equivalenza.

Possiamo quindi astrarre tale concetto e far diventare ogni componente un nodo.

In questo modo otterremo quello che viene chiamato il grafo delle componenti.

Mostreremo che il grafo delle componenti è necessariamente un grafo aciclico perché come abbiamo già detto da una componente potenzialmente possiamo andare in un'altra ma non anche il viceversa, in quanto altrimenti sarebbero la stessa componente connessa.

In generale possiamo andare da componenti connesse scoperte dopo verso componenti connesse scoperte prima.

Tale proprietà risulta essere fondamentale per il funzionamento dell'algoritmo che vedremo a breve.

Vediamo adesso il grafo delle componenti dell'esempio precedente.

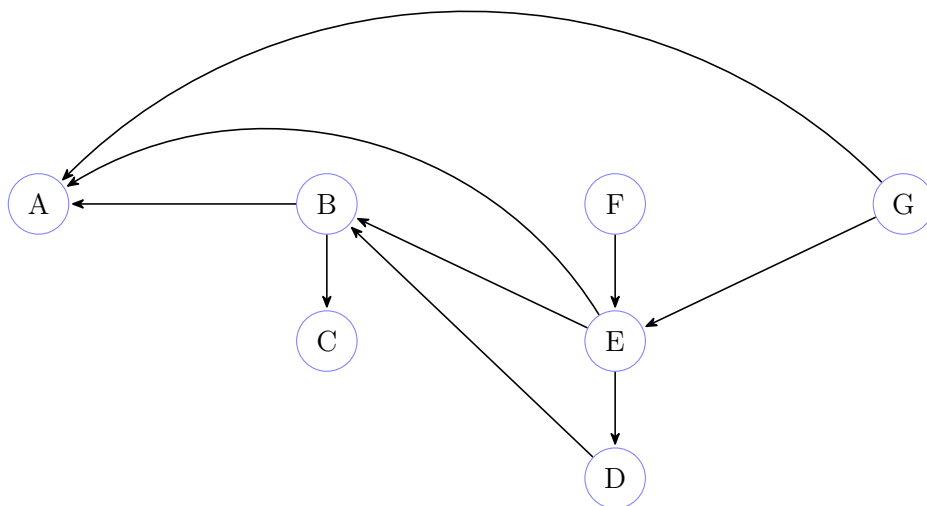


Figura 6.22: Esempio di grafo delle componenti

Quindi il grafo delle componenti, una volta che astraiamo la componente connessa e la vediamo come un unico nodo, è necessariamente un grafo aciclico perché possiamo percorrere gli archi in una sola direzione.

Vedremo quindi come sfruttare implicitamente l'ordine che otteniamo sul grafo delle componenti, che è un ordine topologico, per analizzare a ritroso il grafo, cioè andando proprio sul grafo trasposto, e da questo estrarre poi le componenti.

Al momento abbiamo solo una classificazione approssimata. Sappiamo che una componente non può far parte di alberi diversi ma uno stesso albero può contenere componenti diverse; quello che faremo è andare a ritroso sugli archi del grafo per distinguere nodi che fanno parte dello stesso albero ma di componenti diverse e che al momento però non sappiamo come distinguere.

Inseriamo nell'esempio del grafo con gli archi partizionati in base alla visita in profondità i tempo di inizio e fine visita.

Come sappiamo i tempi di inizio visita vengono calcolati in preorder, quelli di fine visita in postorder.

Vediamo di seguito l'esempio.

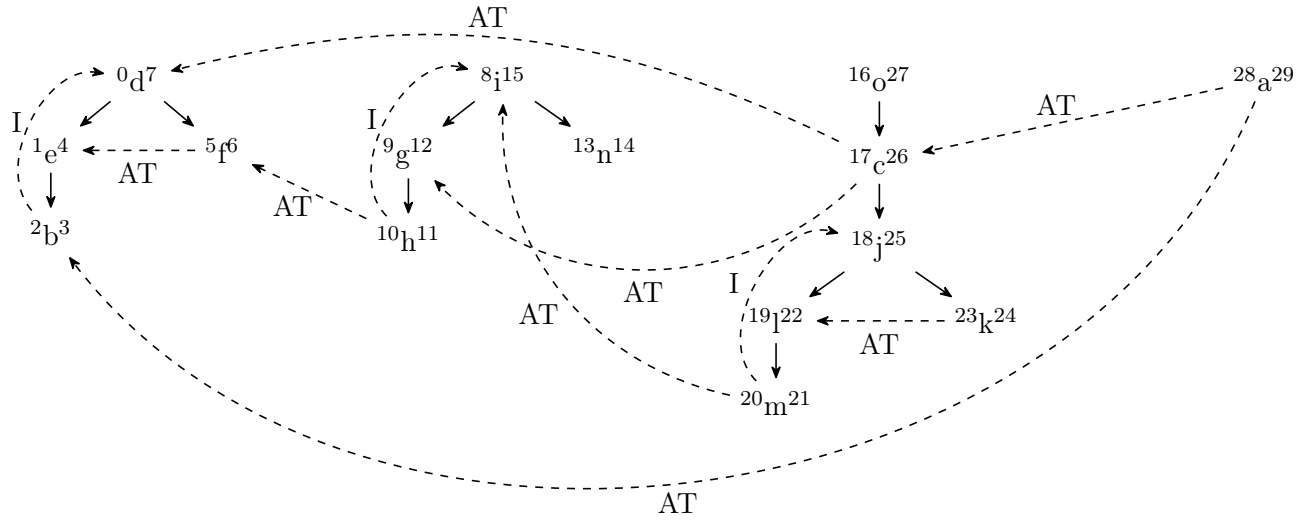
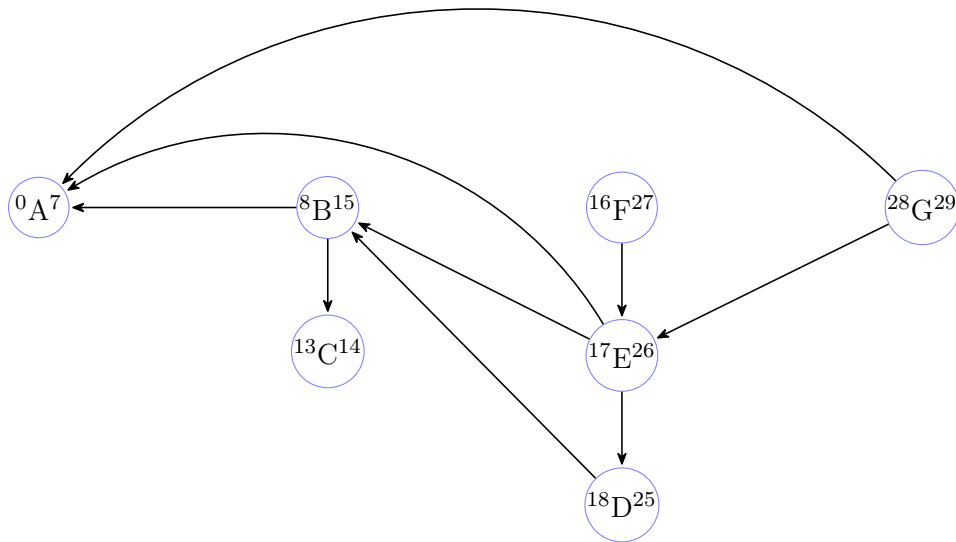


Figura 6.23: Esempio di foresta di visita per componenti massimali, con tempi di visita

Possiamo fare lo stesso anche con il grafo delle componenti fortemente connesse massimali.



Come abbiamo più volte detto, il grafo delle componenti connesse deve essere necessariamente un grafo aciclico. Non è detto che sia un albero, ma sicuramente deve essere un grafo aciclico.

Ora, cosa possiamo osservare dalla visita?

Già avevamo osservato, e adesso andremo a dimostrarlo, che se c'è una componente questa deve essere necessariamente tutta contenuta in un albero.

Dall'esempio è evidente, perché si nota immediatamente che le varie componenti fortemente connesse sono tutte parti di uno stesso albero.

Quindi ogni componente è contenuta all'interno di un albero.

Questo era vero già prima, quando abbiamo lavorato sui grafi non orientati. Lì però avevamo la proprietà che un albero copriva un'intera componente connessa.

Questa proprietà però non è vera in generale quando l'albero è un grafo orientato.

Ora, come possiamo distinguere le componenti all'interno di un albero?

È chiaro che la visita che abbiamo fatto ci permette di passare da componenti di un albero ad altre componenti dello stesso albero, o addirittura da un albero ad alberi precedenti perché ci sono gli archi di attraversamento.

Tuttavia, se noi andassimo a prendere il grafo con gli archi orientati in senso opposto, cioè il grafo trasposto, cosa potremmo fare?

Vediamo di seguito il trasposto del grafo 6.21.

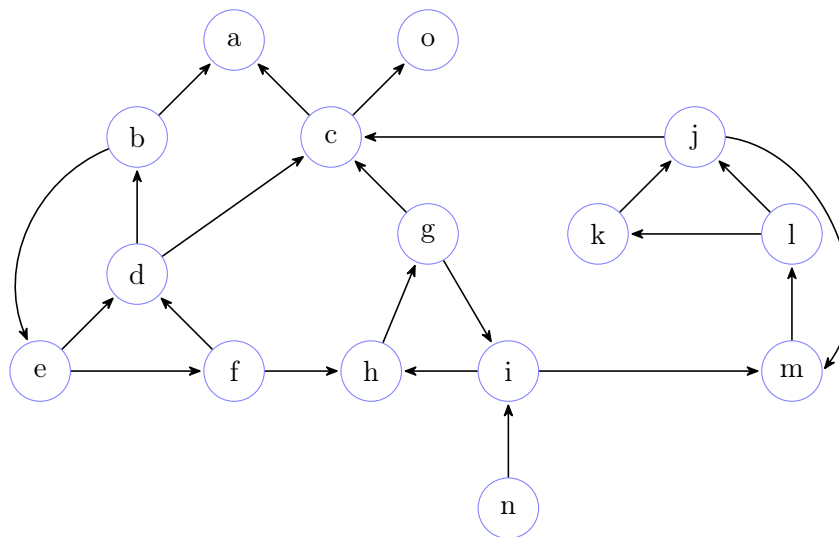


Figura 6.24: Grafo trasposto dell'esempio 3 di grafo

In particolare guardiamo le componenti connesse in ordine inverso di fine vista.

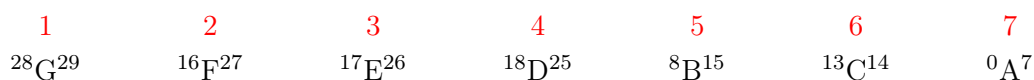


Figura 6.25: Ordine per fine visita delle componenti connesse dell'esempio 3 di grafo

Cioè, lo stesso identico ordine che veniva calcolato dall'algoritmo dell'ordinamento topologico basato su DFS. Noi cosa facevamo nell'algoritmo DFS?

Inserivamo nello stack i nodi in ordine inverso rispetto all'ordine di scoperta. Mettevamo in fondo gli ultimi nodi che venivano scoperti e poi man mano andavamo ad accatastare tutti gli altri nello stack.

Quindi quello che trovavamo in testa allo stack era l'ultimo nodo da cui terminavamo la visita, quindi quello col tempo di fine visita più alto.

Vediamo adesso un esempio di stack alla fine della DFS sul grafo 6.21

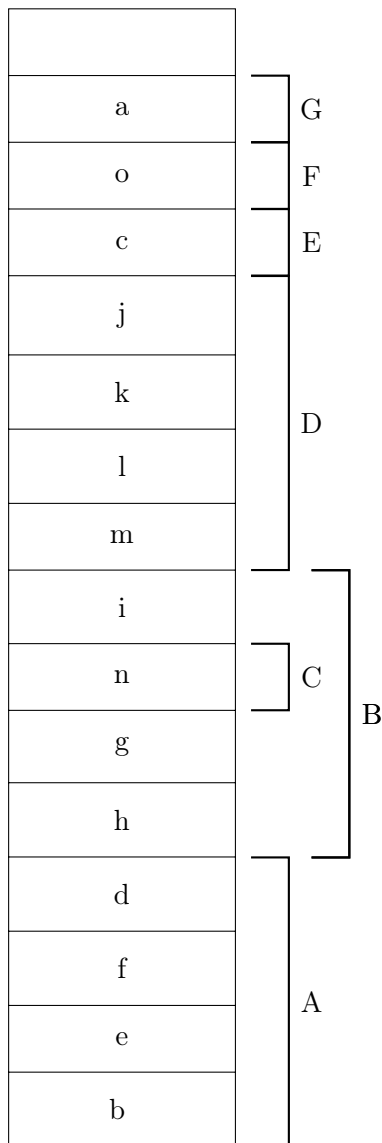


Figura 6.26: Stack alla fine della DFS sull'esempio 3 di grafo

Ovviamente stiamo facendo un'osservazione su un esempio concreto che poi, nel caso dovesse essere vera in generale, va dimostrata con opportuni teoremi.

Osserviamo che se da un nodo possiamo raggiungere dei nodi sia nel grafo originale sia nel grafo trasposto significa che i nodi che raggiungiamo nel grafo trasposto sono i nodi che raggiungono il nodo in questione nel grafo originale.

Quindi se un nodo raggiunge dei nodi nel grafo originale e anche in quello trasposto significa che nel grafo originale è raggiunto da tali nodi e quindi tali nodi per definizione faranno parte della stessa componente fortemente connessa.

Quindi facendo una visita sul grafo trasposto in cui l'ordine delle radici di ricerca è prestabilito dall'ordine di fine visita della ricerca precedente (è uno dei rari casi in cui è necessario un preciso ordine di visita), cioè andando ad analizzare radici di ricerca a ritroso a livello temporale rispetto a quello che ci viene detto dalla prima visita, potremo identificare tutte e sole le componenti fortemente connesse.

Questo perché una componente che nel grafo originale raggiunge un'altra sul grafo trasposto non potrà fare l'inverso. Non potrà raggiungere componenti precedentemente scoperte perché il grafo appunto è trasposto, gli archi di attraversamento non sono percorribili, e quelle che sono state già analizzate, appunto, sono state già analizzate, quindi è come se le escudessimo dal grafo.

Quindi l'algoritmo delle componenti fortemente connesse è un algoritmo estremamente semplice.

Ha una prima parte che consiste in una DFS, che è esattamente identica all'algoritmo del calcolo dell'ordinamento topologico tramite DFS che abbiamo visto.

Dalla DFS tirerà fuori come risultato uno stack che come sappiamo ripropone i nodi in ordine di fine visita, quindi in testa avremo quello con tempo di fine visita più alto, in fondo allo stack ci sarà quello con tempo di fine visita più basso.

Dopodiché calcoliamo il trasposto del grafo. Semplicemente creiamo un nuovo grafo dove per ogni nodo viene creato un altro nodo e per ogni arco aggiungiamo un arco invertito nel grafo nuovo.

È un algoritmo lineare sulla dimensione del grafo.

Fatto questo facciamo una variante della visita dove la parte esterna dell'algoritmo invece che scorrere tutti i vertici, come farebbe nella classica visita, prende i vertici dallo stack tramite una `top_&_pop` e man mano procede.

Una piccola variante che di solito mettiamo in atto per creare il grafo delle componenti connesse implicitamente è quello di avere un ulteriore vettore, spesso chiamato `scc`, indicizzato con i nodi del grafo originale a cui daremo all'interno un nome della componente.

Questo nome può essere un numero, o una lettera, o altre volte può essere comodo assegnare a una componente uno dei nodi della componente.

Visto che le componenti connesse solo classi di equivalenza, possiamo usare uno qualunque dei rappresentanti della classe stessa; quindi quello che si può fare è un algoritmo che associa ad ogni classe uno dei nodi che gli appartiene, di solito viene assegnato il primo nodo che si scopre nella classe.

6.16.1 Algoritmo: calcolo componenti fortemente connesse in un grafo orientato

Andiamo quindi a vedere l'algoritmo, dopodiché andremo a dimostrarne le proprietà di correttezza e completezza.

Algoritmo 94: Funzione accessoria per fare il trasposto di un grafo

Signature transpose: $Graph(D) \rightarrow Graph(D)$

function transpose(G)

```

1  // effettuo una copia dei vertici del grafo in input
   V' ← V
   // per ogni nodo del grafo in input
2  foreach (v in V) do
   // ne esploro la stella uscente
   // e per ogni nodo della stella uscente
3  foreach (w in Adj[v]) do
   // inserisco negli adiacenti del nodo in questione
   // il nodo che nel grafo in input ha il nodo in questione
   // come adiacente
   // creo in questo modo un arco inverso
4  Adj'[w] ← insert(Adj'[w], v)
   // restituisco la coppia dei vertici e degli adiacenti ai vertici creati
   // che rappresentano il grafo trasposto
5  return (V', Adj')
```

Osserviamo che un algoritmo del genere è chiaramente lineare nella dimensione del grafo.

La copia dei vertici ha complessità $\Theta(|V|)$, la creazione dei nuovi archi ovviamente $\Theta(|E|)$; quindi la complessità totale è $\Theta(|E| + |V|)$.

Tale calcolo della complessità si basa sull'assunto che l'operazione di inserimento nella struttura che conserva gli adiacenti per ogni nodo sia a tempo costante.

Tale algoritmo per quanto semplice risulta molto importante perché sono numerosi i problemi che necessitano di una doppia visita, una visita sul grafo originale e una visita sul grafo trasposto per essere risolti con una certa complessità.

Vediamo ora l'algoritmo delle componenti fortemente connesse (strongly connected component, scc).

Algoritmo 95: Funzione per trovare le componenti fortemente connesse di un grafo

Signature SCC: $Graph(D) \rightarrow Array(D)$
function SCC(G)

```

    // mediante una funzione accessoria
    // che è lo stesso algoritmo DFS per l'ordinamento topologico
    // popolo uno stack con i nodi nell'ordine inverso di visita
1   S ← DFS1(G)
    // mediante una funzione accessoria
    // creo il grafo trasposto del grafo in input
2   G_t ← transpose(G)
    // mediante una funzione accessoria
    // che è una variante dell'algoritmo DFS
    // che viene eseguito con un particolare ordine
    // creo un vettore che ad ogni nodo del grafo in input
    // assegna il nome della componente fortemente connessa di appartenenza
3   scc ← DFS2(G_t, S)
    // restituisco il vettore
4   return scc

```

Vediamo ora la versione modificata dell'algoritmo DFS che in input prende anche uno stack in quanto è fondamentale che l'ordine di visita sia ben definito.

Come al solito si compone di una parte non ricorsiva e di una parte ricorsiva.

Algoritmo 96: Funzione accessoria per la visita in profondità di un grafo, dato un determinato ordine

Signature DFS2: $Graph(D) \times Stack(D) \rightarrow Array(D)$

```

function DFS2(G, S)
    // mediante una funzione accessoria
    // inizializzo il vettore dei colori settando tutti i nodi da scoprire
1   c ← init(G)
    // fino a quando non si è svuotato lo stack in input
2   while (NOT is_empty(S)) do
        // prelevo il nodo in testa allo stack
        // in modo da seguire un ordine preciso
3       (S, v) ← top_&_pop(S)
        // se il nodo in questione è da scoprire
4       if (c[v] = WHITE) then
            // mediante una funzione ricorsiva accessoria
            // che prende in input il grafo in input
            // il vettore delle componenti che andrà a riempire
            // il vettore dei colori
            // il nodo corrente dal quale parte l'esplorazione
            // il rappresentante della classe cui appartiene il nodo corrente
5       DFS2(G, scc, c, v, v)
    // restituisco il vettore delle componenti
6   return scc
  
```

Vediamo la parte ricorsiva.

Tale funzione prende in input il grafo, il vettore delle componenti che assegna ad ogni nodo del grafo la classe di connessione di appartenenza, il nodo dal quale far partire l'esplorazione e un rappresentante della classe cui appartiene il nodo in input.

Algoritmo 97: Funzione accessoria per la visita in profondità di un grafo, dato un determinato ordine (ricorsiva)

Signature DFS2: $Graph(D) \times Array(D) \times Array(Color) \times D \times D \rightarrow \emptyset$

```

function DFS2(G, scc, c, v, s)
    // indico che il nodo in input viene scoperto
    // colorandolo di grigio
1   c[v] ← GRAY
    // esploro la stella uscente del nodo in input
2   foreach (w in Adj[v]) do
        // se trovo un nodo da scoprire
3       if (c[w] = WHITE) then
            // richiamo la funzione ricorsiva
            // sul nodo corrente con la stessa classe di equivalenza
4         DFS2(G, scc, c, w, s)
    // setto la classe di appartenenza del nodo in input
    // con il rappresentante della classe in input
    // tale istruzione poteva essere anche posta in preorder
    // visto che non è rilevante quando effettuiamo tale istruzione
5   scc[v] ← s
    // indico che il nodo è stato visitato
    // colorandolo di nero
6   c[v] ← BLACK

```

Questo algoritmo funzionerebbe tal quale se invece del rappresentante della classe si utilizza un nome per la classe (un valore numerico ad esempio che viene fatto incrementare nel ciclo **while**).

Risulta inoltre ovvio, considerando tutte le discussioni su i vari algoritmi su grafi simili, che tale algoritmo (ci riferiamo all'algoritmo **SCC**) abbia una complessità lineare nella dimensione del grafo, cioè $\Theta(|E| + |V|)$.

Per esemplificare il comportamento dell'algoritmo mostriamo il contenuto del vettore **scc** sull'esempio già proposto in precedenza.

Al termine dell'algoritmo sul grafo 6.21 il vettore **scc** è il seguente.

Indicheremo nella prima riga, il vertice considerato, e nella seconda la componente fortemente connessa massimale a cui appartiene indicandone la sua rappresentante di classe.

V	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
scc	a	d	c	d	d	d	i	i	i	j	j	j	j	n	o

6.16.2 Correttezza algoritmo calcolo componenti fortemente connesse in un grafo orientato

Vediamo quindi i teoremi di correttezza e completezza per l'algoritmo appena descritto.

Un primo lemma definisce una proprietà dei grafi, non dell'algoritmo.

Esso afferma che presa una componente fortemente connessa massimale C di un grafo orientato G e due nodi del grafo, v e w , appartenenti alla componente C , qualsiasi percorso nel grafo da v a w appartiene alla componente C .

Formalmente.

Lemma 6.16.1 (Percorsi di una componente connessa).

Sia C una scc massimale di G , $\forall v, w \in C \implies \forall \pi \in Path(G, v, w) (\pi \subseteq C)$.

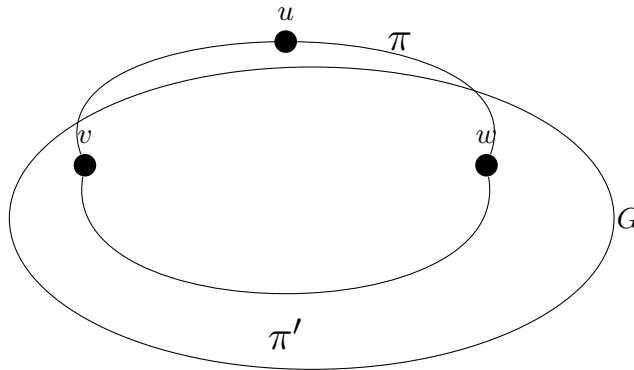
Dimostrazione.

Dimostriamo il lemma per assurdo negando la tesi.

Supponiamo per assurdo che se C è una scc massimale di G , $\forall v, w \in C \implies \exists \pi \in Path(G, v, w) (\pi \not\subseteq C)$.

Significa, necessariamente, che $\exists u \in V (u \notin C \wedge u \in \pi)$. Chiaramente visto che $v, w \in C$ significa che possono raggiungersi mutuamente.

Dato che $u \in \pi$ abbiamo necessariamente che v raggiunge u e che u raggiunge w ; ma come detto anche w raggiunge v . Necessariamente quindi $u \in C$, che è una contraddizione visto che C è una componente fortemente connessa massimale di G .



□

Attenzione che questo teorema non sta dicendo nulla dei percorsi che raggiungono la componente o dei percorsi che dalla componente escono fuori.

Quindi stiamo parlando solo dei percorsi tra nodi della stessa componente.

Utilizziamo adesso il lemma appena visto per dimostrare un'affermazione che abbiamo ripetuto diverse volte, cioè che ogni componente connessa deve far parte dello stesso albero della foresta e non può quindi essere distribuita tra alberi diversi.

Il lemma che vogliamo dimostrare afferma che presa una componente fortemente connessa massimale C di un grafo G e p il vettore dei predecessori ottenuto con l'algoritmo DFS, allora avremo che la componente è inclusa in qualche albero della foresta di visita.

Formalmente.

Lemma 6.16.2 (Componenti connesse massimali nella foresta di visita della DFS).

Sia C una scc di G e p un vettore dei predecessori calcolato con la DFS $\implies \exists$ albero $T \in \mathcal{F}_p$ ($C \subseteq T$).

Dimostrazione.

Una componente connessa massimale per definizione non è vuota, quindi $C \neq \emptyset$.

Sia, quindi, $v \in C$, il nodo con tempo di inizio visita minore, cioè $\forall w \in C \setminus \{v\} (s[v] < s[w])$.

Conseguentemente, al tempo $s[v]$ tutti gli altri nodi di C saranno bianchi e per il teorema del percorso bianco sappiamo che tutti i nodi della componente C sono raggiungibili da v nella foresta di visita. Ciascuno di questi percorsi, e quindi ciascuno dei nodi facente parte del percorso, fa parte dell'albero della foresta di visita in cui si trova v e per il lemma precedente sappiamo che sono tutti interni a C .

Abbiamo quindi che, $\forall u \in C \implies u \in T \in \mathcal{F}_p$.

□

Vediamo ora la dimostrazione di correttezza dell'algoritmo per ottenere le componenti fortemente connesse di un grafo orientato.

Teorema 6.16.1 (Dimostrazione correttezza algoritmo SCC).

Dimostrazione.

Facciamo una prima osservazione. All'interno della foresta di visita costruita su un grafo orientato le uniche componenti fortemente connesse massimali che non hanno cicli sono ovviamente le componenti banali; tutte le altre componenti fortemente connesse hanno almeno un arco di ritorno.

A questo punto possiamo osservare che se da un nodo v troviamo un percorso bianco verso un certo nodo w , sicuramente il nodo v raggiunge il nodo w .

Tuttavia, ci manca ancora il percorso inverso, che è la parte relativa al grafo trasposto. Infatti, non tutto quello che il nodo v riesce a raggiungere è detto sia parte della componente fortemente connessa massimale cui fa parte v .

Vediamo cosa possiamo dire a riguardo.

Se il nodo w non raggiunge il vertice v nel grafo originale, allora nel grafo trasposto v non raggiunge w .

Se $\exists \pi \in \text{Path}(G, v, w) \implies \exists \pi^T \in \text{Path}(G^T, w, v)$; ed equivalentemente se $\nexists \pi^T \in \text{Path}(G^T, w, v) \implies \nexists \pi \in \text{Path}(G, v, w)$.

Quindi, la prima DFS vede cosa è raggiungibile da un certo nodo v . Sul trasposto andremo poi a percorrere i percorsi che possono esistere dai vertici che sono raggiunti da v nel grafo originale, ma visto che ci troviamo sul grafo trasposto li percorreremo a partire da v .

Se esistono archi all'indietro tali percorsi saranno percorribili nel grafo trasposto a partire da v , e quindi otterremo tutti i nodi che raggiungono v nel grafo originale, guardando nel grafo trasposto i vertici che è possibile raggiungere da v .

Allo stesso tempo, se $\nexists \pi \in \text{Path}(G, v, w) \implies \nexists \pi^T \in \text{Path}(G^T, w, v)$; ed equivalentemente se $\exists \pi^T \in \text{Path}(G^T, w, v) \implies \exists \pi \in \text{Path}(G, v, w)$.

Inoltre, quello che è raggiungibile da un certo nodo v nel grafo originale mediante gli archi di attraversamento, in teoria raggiunge v nel grafo trasposto, ma visto che la visita sul grafo trasposto procede in ordine di tempo di fine visita secondo la prima DFS, v sarà già colorato di nero quando la visita proverà a percorrere in senso inverso gli archi che nel grafo originale sono di attraversamento e che originano da v , e visto che nell'algoritmo esploriamo solo nodi che sono bianchi tali parti del grafo non entreranno nelle componenti fortemente connesse massimali di un certo vertice v .

Gli unici archi accessibili sul grafo trasposto sono, ovviamente, quelli che nel grafo originale sono archi di attraversamento, e quelli che nel grafo originale sono archi all'indietro; chiaramente in direzione opposta.

Come abbiamo detto, visto che la visita sul grafo trasposto viene eseguita secondo l'ordine dato dalla prima DFS avremo che gli archi che nel grafo originale sono di attraversamento non saranno percorribili, e pertanto gli unici archi realmente percorribili sono quelli che nel grafo originale sono archi all'indietro.

I nodi che fanno parte di alberi di visita diversi e che hanno un tempo di fine visita minore non saranno raggiungibili nel trasposto da un certo nodo v in quanto gli archi di attraversamento saranno entranti nel trasposto, e noi esploriamo la stella uscente.

I nodi che fanno parte di alberi di visita diversi e che hanno un tempo di fine visita maggiore non saranno raggiungibili nel trasposto da un certo nodo v in quanto gli archi di attraversamento, sebbene uscenti, non saranno percorribili perché tali nodi saranno già colorati di nero, visto che l'esplorazione del grafo trasposto avviene secondo l'ordine inverso di fine visita della prima DFS.

Inoltre, i nodi che a partire da un certo vertice v sono raggiungibili nel grafo originale e che fanno parte dello stesso albero della foresta di visita, ma che non raggiungono il nodo v , come detto, nel trasposto non sono raggiungibili.

Di conseguenza, visto che abbiamo dimostrato che le componenti fortemente connesse sono sempre incluse in un

albero della foresta di visita e abbiamo visto che l'esplorazione del grafo trasposto prende sicuramente tutte le componenti dell'albero, possiamo essere certi che nell'algoritmo SCC non è possibile che venga assegnato in modo erroneo un vertice ad una componente fortemente connessa.

Quindi, necessariamente, tutto ciò che è raggiungibile a partire da un nodo v nel grafo trasposto, nell'algoritmo SCC, è solo ciò che nel grafo originale è raggiunto da v e allo stesso tempo raggiunge v ; cioè i nodi facente parte della componente fortemente connessa massimale cui appartiene anche v .

L'algoritmo SCC risulta, pertanto, corretto.

□

Con questo terminiamo la parte relativa ai grafi non pesati o grafi classici.

6.17 Grafi pesati

Passiamo adesso ai cosiddetti grafi pesati.

Per prima cosa è lecito chiedersi: a cosa servono i grafi pesati?

I grafi pesati servono a dare una rappresentazione di una parte di realtà con un maggior grado di dettaglio.

Facciamo un esempio estremamente semplice.

Immaginiamo ad esempio di avere una rete stradale con un punto da cui vogliamo partire e un punto che vogliamo raggiungere.

Le strade come noto sono tra di loro diverse e un modo di classificarle per esempio è la lunghezza.

Potremmo preferire ad esempio il percorso più corto.

Si potrebbe, quindi, costruire un grafo avente per vertici i punti possibili di arrivo e partenza della rete stradale, per archi le varie strade che connettono i punti e come peso sugli archi una misura della lunghezza delle strade.

Con un grafo del genere è possibile facilmente trovare il percorso (o i percorsi) più breve in termini di lunghezza complessiva.

Attenzione però che in generale il peso dato agli archi non necessariamente è una misura di una distanza.

Sono possibili i più svariati criteri.

In generale quindi in un grafo pesato il percorso minimo non va inteso come minimo in termini di distanze o di numero di archi percorsi, ma minimo in riferimento al costo del peso assegnato agli archi.

Il primo algoritmo che vedremo, che è appunto l'algoritmo di Dijkstra, cercherà esattamente di identificare qual è il percorso più piccolo, minimale, in un grafo pesato, in cui appunto abbiamo dato dei pesi agli archi.

Quindi il percorso minimo in un grafo pesato potrebbe essere, considerando ad esempio il numero di archi, il più lungo e tuttavia quello con un costo più basso.

L'algoritmo di Dijkstra è una variante dell'algoritmo di visita in ampiezza.

Perché?

Immaginiamo di avere un grafo in cui i pesi corrispondono esattamente agli archi; l'algoritmo di Dijkstra dovrà dare esattamente la stessa risposta che dà l'algoritmo di visita in ampiezza.

Quindi sarà una variante della visita in ampiezza dove però l'esplorazione del vicinato, che noi nell'algoritmo di visita in ampiezza basavamo semplicemente sulla stella uscente, ora sarà basata su quanto pesano gli archi.

Quindi cercheremo di esplorare un orizzonte che a livello di archi avrà una forma probabilmente del tutto arbitraria ma che a livello di pesi, cioè di quanto costa percorrere quel pezzo di percorso, saranno tutti quanti simili e man mano andremo a rendere più pesante il costo via via che ci allontaniamo.

Se esploriamo i nodi in questo modo, così come per la visita in ampiezza siamo riusciti a ottenere percorsi più corti esplorando man mano orizzonti sempre più lontani, così qui esploreremo orizzonti più lontani secondo una diversa misura, cioè il peso assegnato ai vari archi.

La differenza sta ovviamente nel fatto che a differenza di quello che accade nella visita in ampiezza dove è sufficiente coprire un arco una sola volta e in particolare quando andiamo a settare una distanza è definitiva,

ora non sarà così perché potremmo avere un arco diretto che ci porta in un nodo e, per il momento, quando scopriamo quell'arco non sappiamo se ci sono strade migliori, poi però esploriamo un percorso molto lungo ma con costo basso che porta allo stesso nodo, trovando un percorso migliore dal punto di vista del peso e quindi sarà necessario effettuare degli aggiornamenti.

Vedremo che per fare questo in modo efficiente dovremo utilizzare una struttura dati, per noi nuova, che si chiama Heap.

6.17.1 Definizione di grafo pesato

Vediamo adesso più formalmente cosa è un grafo pesato.

Sappiamo che in generale un grafo è un insieme di vertici ed una relazione binaria su di questi.

Nel caso di grafo pesato abbiamo anche una funzione di peso, weight, wg .

Quindi un grafo pesato G è:

$$G = \langle V, E, wg \rangle \quad (6.24)$$

Questa funzione di peso, assegna ad ogni arco un peso, quindi è una funzione che da E , che sappiamo essere l'insieme degli archi del grafo, mappa nei numeri reali.

La signature della funzione wg è infatti:

$$wg : \begin{cases} E \rightarrow \mathbb{R}_{\geq 0} & , \text{ se si considerano solo i pesi non negativi} \\ E \rightarrow \mathbb{R} & , \text{ altrimenti} \end{cases} \quad (6.25)$$

A livello algoritmico, in verità, quello che si fa è assegnare ad ogni arco un peso non appartenente ai numeri reali in generale, ma ai numeri razionali. Perché come noto in un calcolatore i reali in generale non sono rappresentabili, i razionali invece sono rappresentabili come rapporto di due naturali, in particolare con il denominatore non nullo.

Ma come detto più volte, noi stiamo sempre cercando di astrarre il più possibile e di non curarci delle questioni puramente implementative, pertanto noi possiamo senza alcun problema utilizzare i numeri reali come peso per gli archi in un grafo pesato.

Vedremo inizialmente una funzione wg che assegna pesi non negativi e infatti il primo algoritmo che tratteremo, l'algoritmo di Dijkstra, risolvere il problema dell'esistenza del percorso con minimo peso, sfruttando l'assunzione in cui i pesi sono reali non negativi, potrà funzionare solo con tale assunzione.

L'algoritmo di Bellman-Ford, che vedremo poi successivamente, invece potrà utilizzare l'intero insieme dei reali, ma costerà di più.

6.17.2 Minimo percorso in un grafo pesato

Vedremo che in generale se assegniamo valori reali arbitrari, quindi anche negativi, il concetto di minimo percorso può perdere di significato.

Mostriamo un semplice esempio per chiarire questo concetto.

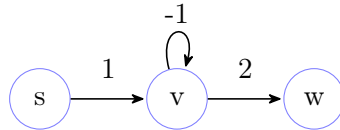


Figura 6.27: Esempio di grafo pesato

È chiaro che nell'esempio proposto non esiste un percorso minimo.

Un percorso semplice di lunghezza minima c'è, ma non è il minimo percorso.

Se consideriamo tutti i percorsi con pesi non semplici, in generale possiamo fare percorsi di lunghezza arbitraria con peso arbitrariamente piccolo.

Quindi il concetto di percorso minimo può perdere di significato.

Vediamo come si rappresentano i grafi pesati.

Abbiamo visto che nei grafi normali ci sono due rappresentazioni canoniche, abbiamo discusso anche di alcune varianti, ma le due canoniche sono matrice di adiacenza e lista di adiacenza.

La matrice di adiacenza abbiamo visto che non è altro che una matrice booleana dove una cella contiene un 1 se e soltanto se c'è un arco che dal nodo di riga va nel nodo di colonna.

Quello che si può fare è utilizzare quella cella non per contenere semplicemente un valore booleano, ma per rappresentare il peso.

Ovviamente vista la necessità di rappresentare anche lo 0 come peso abbiamo bisogno di un valore artificiale, esterno all'insieme dei numeri reali, per indicare l'assenza di un arco.

Nel nostro caso utilizzeremo il simbolo $\perp$.

Vediamo adesso un esempio di rappresentazione del grafo 6.27 tramite matrice di adiacenza.

	s	v	w
s	$\perp$	1	$\perp$
v	$\perp$	-1	2
w	$\perp$	$\perp$	$\perp$

Figura 6.28: Esempio di rappresentazione di un grafo pesato tramite matrice di adiacenza

Questa è la prima possibilità.

Un'altra possibile rappresentazione è semplicemente avere il solito vettore dei nodi e una lista, soltanto che ora nella lista non solo assegniamo il nodo che raggiungiamo, ma dobbiamo ricordarci anche del peso dell'arco.

Quindi un nodo della lista, conterrà due campi dato, uno per il nodo che viene raggiunto e uno per il peso dell'arco.

Vediamo adesso un esempio di rappresentazione del grafo 6.27 tramite lista di adiacenti.

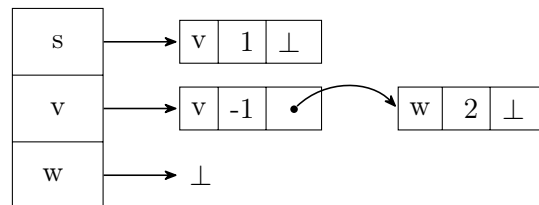


Figura 6.29: Esempio di rappresentazione di un grafo pesato tramite lista di adiacenti

6.17.3 Peso di un percorso in un grafo pesato

La prima cosa che ci interessa è andare a definire il concetto di peso di un percorso.

Sottolineiamo in primo luogo che nei grafi pesati, tutti i concetti che abbiamo già visto valgono tal quali.

Quindi le definizioni di percorso, di ciclo, percorso semplice, ciclo semplice, di distanza, tutto quello che abbiamo visto vale tale quale. Tanto non fanno uso della funzione *weight*.

Quello che manca è il concetto di peso di un percorso.

Andiamo a definirlo.

Ci sono due possibili notazioni.

O facciamo overloading della funzione *wg*, o altre volte si può utilizzare per esempio δ_{wg} ; per uniformità e maggiore chiarezza useremo la seconda notazione.

Delta sta ad indicare non banalmente quella delle distanze che abbiamo usato per la visita in ampiezza, ma semplicemente la distanza pesata, il peso.

Quindi, qual è il peso di un percorso?

Il peso di un percorso è la somma dei pesi degli archi che formano il percorso.

Ovviamente, se il percorso è quello banale che contiene soltanto un nodo, allora lì non c'è nessun arco e il peso sarà per definizione zero.

Formalmente.

Dato un percorso π avremo che:

$$\delta_{wg}(\pi) = \begin{cases} |\pi| - 2 \\ \sum_{i=0}^{|\pi|-2} wg((\pi)_i, (\pi)_{i+1}) \end{cases} \quad , \text{ se } |\pi| \geq 2 \quad (6.26)$$

$$0 \quad , \text{ altrimenti}$$

Ricordiamo la definizione di distanza tra due nodi.

È semplicemente la lunghezza del minimo percorso, che da un primo nodo va al secondo nodo.

Formalmente.

$$\delta(s, v) = \min_{\pi \in Path(G, s, v)} |\pi| \quad (6.27)$$

Definiamo il concetto di distanza di un percorso, cioè la lunghezza del percorso, intesa come numero di archi che compongono il percorso.

Formalmente.

$$|\pi| = \min_{\pi \in Path(G, s, v)} \delta(\pi) \quad (6.28)$$

Se invece di usare la distanza geometrica, cioè il numero di archi, usassimo il peso degli archi che formano il percorso, avremmo la minima distanza pesata, o il minimo peso, che intercorre tra una sorgente e una certa

destinazione.

È proprio quello che faremo per definire la minima distanza pesata.

Formalmente.

$$\delta_{wg}(s, v) = \min_{\pi \in \text{Path}(G, s, v)} \delta_{wg}(\pi) \quad (6.29)$$

Facciamo degli esempi per rendere più chiaro il concetto.

Considerando il grafo 6.27 possiamo notare che a parte le minime distanze pesate banali ($\delta_{wg}(s, s) = 0$ e ecc...) le altre minime distanze pesate non sono definite.

Immaginiamo di voler calcolare $\delta_{wg}(s, v)$.

Potremmo iniziare notando che il peso dell'arco da s a v è 2. Abbiamo ottenuto la minima distanza pesata?

No, perché v ha un arco su sé stesso che ha peso -1 , e quindi percorrendolo il peso generale del percorso diminuisce.

Dunque decrementiamo di 1 il peso totale.

Adesso, abbiamo ottenuto la minima distanza pesata?

No, perché possiamo di nuovo ripercorrere l'arco che v ha su sé stesso, che ci porterà ad un peso generale del percorso più piccolo.

E così via ...

Seguendo questo criterio è immediato notare che, il calcolo della distanza minima pesata tra s e v non è definito.

Consideriamo adesso il seguente grafo.

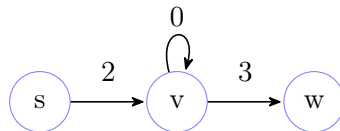


Figura 6.30: Esempio di grafo pesato con pesi non negativi

In questo caso, la distanza minima pesata tra s e v è definita, infatti $\delta_{wg}(s, v) = 2$, poiché percorriamo l'arco da s a v di peso 2 e ci fermiamo, non percorrendo l'arco che v ha con sé stesso perché non diminuisce il peso totale del percorso.

Lo stesso principio vale anche per la minima distanza pesata tra s e w che sappiamo essere $\delta_{wg}(s, w) = 5$, poiché percorriamo l'arco da s a v , non percorriamo il self-loop su v , percorriamo l'arco da v a w e ci fermiamo.

Attenzione che la funzione δ_{wg} su grafi pesati arbitrari è parziale. Cioè, ci sono casi in cui non è definita.

In alcuni casi si può dire che la distanza è meno infinito, ma non è un qualcosa che è sempre condiviso da tutti i testi perché effettivamente non c'è un percorso con peso meno infinito; ci sono infiniti percorsi con peso sempre più basso ma non ce n'è mai uno con peso meno infinito.

Ecco perché non tutti sono d'accordo a porre, quando un minimo non esiste, il minimo uguale a meno infinito.

A volte, matematicamente, può essere comodo, ma noi quando non c'è il minimo, diremo che la funzione δ_{wg} non è definita; pertanto è una funzione parziale.

Quindi il problema è che il minimo percorso non è definito se consideriamo archi con peso negativo in un grafo avente cicli.

In realtà, per essere precisi, se il grafo fosse aciclico oppure se il grafo ha cicli ma il peso dei cicli è non negativo allora il concetto di minimo è sempre definito.

Vediamo ora alcune proprietà molto semplici ma allo stesso tempo fondamentali relative ai percorsi minimi per poter dimostrare la correttezza dell'algoritmo di Dijkstra.

Queste proprietà per altro le abbiamo già viste, nel senso che le abbiamo viste per percorsi in cui la distanza era quella geometrica; dobbiamo quindi generalizzare tali proprietà al caso di percorsi pesati.

Prima di introdurre le proprietà andiamo a definire l'insieme $MinWPath$ come il minimo percorso pesato (ovviamente come già visto possiamo definirlo considerando un grafo G ; un grafo G ed un'origine s ; ed un grafo G , un'origine s ed una destinazione v).

Formalmente.

$$\begin{aligned} MinWPath(G, s, v) &\subseteq Path(G, s, v) \\ \pi \in MinWPath(G, s, v) &\iff \delta_{wg}(\pi) = \delta_{wg}(s, v) \end{aligned} \tag{6.30}$$

La prima proprietà che vogliamo dimostrare è definita dal seguente lemma che afferma che preso un grafo pesato G con pesi non negativi e un minimo percorso pesato in G , arbitrario tra due nodi, π , allora qualunque sottopercorso di π questo è necessariamente il minimo percorso pesato tra l'origine e la destinazione del sottopercorso.

Lemma 6.17.1 (Sottopercorso di un percorso pesato minimo).

Sia G un grafo pesato con pesi non negativi.

Sia $\pi \in MinWPath(G) \implies \forall i, j \in [0, |\pi|) (i \leq j \wedge \pi[i, j] \in MinWPath(G, (\pi)_i, (\pi)_j))$.

Con $\pi[i, j]$ si intende il percorso da posizione $(\pi)_i$ a $(\pi)_j$ in π , estremi inclusi.

Dimostrazione.

Dimostriamo il lemma per assurdo, negando la tesi.

Supponiamo, quindi, che $\exists i, j \in [0, |\pi|) (i \leq j \wedge \pi[i, j] \notin MinWPath(G, (\pi)_i, (\pi)_j))$.

Significa che $\exists \pi' \in MinWPath(G, (\pi)_i, (\pi)_j) (\delta_{wg}(\pi') < \delta_{wg}(\pi[i, j]))$.

Avremo, ovviamente, $\pi = \pi[0, i), \pi[i, j], \pi(j, |\pi|)$.

Visto che $first(\pi[i, j]) = first(\pi') \wedge last(\pi[i, j]) = last(\pi')$, possiamo sostituire $\pi[i, j]$ con π' ed ottenere un percorso con stessa origine e destinazione di π , ma con peso minore.

Quindi, $\pi'' = \pi[0, i), \pi', \pi(j, |\pi|)$. Risulta, chiaramente, $\pi'' \in Path(G, first(\pi), last(\pi))$, ed inoltre, $\delta_{wg}(\pi'') < \delta_{wg}(\pi)$. Abbiamo pertanto raggiunto una contraddizione partendo da una supposizione per assurdo.

□

Quindi questo primo lemma altro non fa che generalizzare quello che abbiamo visto nei percorsi di cui calcolavamo la distanza geometrica.

Un immediato corollario del precedente lemma è il seguente. Sia G un grafo pesato con pesi non negativi e π un percorso minimo dal nodo sorgente s al nodo destinazione w .

Se andiamo a scomporre il percorso π come formato dal percorso π' e l'arco tra i nodi v e w , quindi separando l'ultimo arco del percorso, allora il peso di π è necessariamente la somma tra i pesi del percorso che da s va a v e del peso dell'arco tra v e w .

Corollario 6.17.1 (Scomposizione sottopercorso di un percorso pesato minimo).

Sia G un grafo pesato con pesi non negativi.

Sia $\pi = \pi' \circ (v, w) \in \text{MinWPath}(G, s, w) \implies \delta_{wg}(\pi) = \delta_{wg}(s, v) + wg(v, w)$.

Con $\pi' \in \text{Path}(G, s, v)$.

Dimostrazione.

Per definizione, $\delta_{wg}(\pi) = \delta_{wg}(\pi') + wg(v, w)$.

Per il lemma precedente, $\pi' \in \text{MinWPath}(G, s, v)$, quindi la tesi. □

Vediamo ora un'altra generalizzazione di un lemma che abbiamo già visto; ovviamente come per il lemma precedente anche qui faremo riferimento alle distanze pesate.

Il lemma afferma che preso un grafo pesato con pesi non negativi e tre vertici del grafo, s , v e w , dove v e w sono connessi da un arco, allora la distanza pesata da s a w è necessariamente inferiore o uguale alla distanza da s a v sommata al peso dell'arco da v e w .

Lemma 6.17.2 (Distanze grafi pesati).

Sia G grafo pesato con pesi non negativi.

$\forall s, v, w \in V, (v, w) \in E (\delta_{wg}(s, w) \leq \delta_{wg}(s, v) + wg(v, w))$

Dimostrazione.

Dimostriamo il lemma per assurdo, negando la tesi.

Supponiamo, quindi, che $\exists s, v, w \in V, (v, w) \in E (\delta_{wg}(s, w) > \delta_{wg}(s, v) + wg(v, w))$.

Di conseguenza, $\exists \pi \in \text{MinWPath}(G, s, w), \pi' \in \text{MinWPath}(G, s, v) (\delta_{wg}(\pi) = \delta_{wg}(s, w) \wedge \delta_{wg}(\pi') = \delta_{wg}(s, v))$.

Sostituendo, otteniamo

$$\text{MinWPath}(G, s, w) \ni \delta_{wg}(\pi) > \delta_{wg}(\pi') + wg(v, w) = \delta_{wg}(\pi' \circ w) \in \text{MinWPath}(G, s, w)$$

□

6.17.4 Calcolo del percorso minimo in un grafo pesato

Grazie a queste generalizzazioni vedremo ora, prima a livello empirico su un esempio, e poi dopo a livello algoritmico, qual è l'approccio che va utilizzato per risolvere il problema del calcolo del percorso minimo da una certa sorgente in un grafo pesato con pesi non negativi.

Attenzione. Questi sono tutti i problemi che richiedono una sorgente; risolvere il problema di tutte le distanze tra tutti i nodi è più complesso e noi non lo vedremo.

Quello che vedremo è praticamente una generalizzazione della visita di ampiezza, tuttavia la visita in ampiezza analizza il grafo estendendo sempre di più l'orizzonte un passo alla volta cercando prima i percorsi con il minimo numero di archi e poi man mano essi vengono estesi.

Ora, non possiamo fare esattamente la stessa cosa nel caso dei grafi pesati perché una volta che assegnamo ad un arco un peso superiore all'unità o comunque superiore al peso di altri archi (se tutti gli archi fossero ugualmente pesati il problema sarebbe esattamente quello della distanza) non è detto che un percorso più breve nel numero degli archi sia il percorso minimo in termini di peso, e pertanto risulta fondamentale aggiornare quelle che sono le distanze, cosa che sappiamo non è possibile nella visita in ampiezza che abbiamo descritto.

L'idea intuitiva è provare ad esplorare l'orizzonte esplorando ciò che è possibile raggiungere al minimo peso, e continuare in questo modo incrementando man mano il peso di una unità.

Ovviamente tale ragionamento funziona con pesi interi, infatti l'algoritmo che vedremo fa qualcosa di leggermente diverso e più fine per essere più generale.

Sottolineiamo che quello appena descritto è un potenziale approccio, non è il migliore, funziona solo con distanze discrete ed è poco efficiente.

È importante, però, per rendere l'idea generale del processo, cioè quello di valutare comunque la stella uscente, ma non esplorando tutti gli archi ma solo quelli che ci permettono di avere il minimo incremento della distanza, e potenzialmente anche di percorrere archi di nodi differenti perché permettono di attraversare il grafo mantenendo comunque il minimo peso.

Vediamo di seguito un esempio di applicazione di questa idea intuitiva naïve, dato il seguente grafo pesato e partendo dal nodo etichettato S (Sorgente).

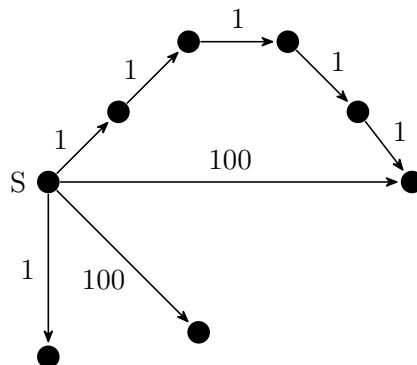
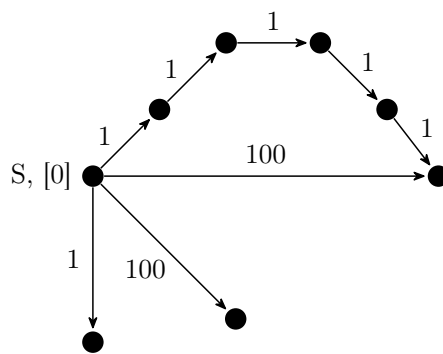


Figura 6.31: Esempio 2 di grafo pesato con pesi non negativi

Passo 0:

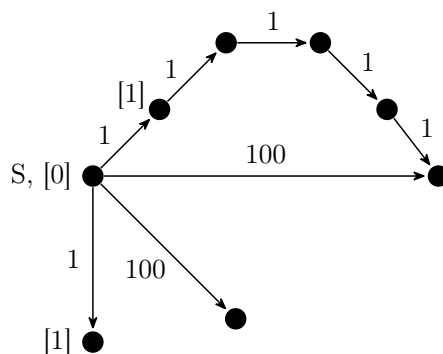
Esplorazione di distanze minime con peso 0.
Indicheremo la minima distanza pesata con
un numero racchiuso da due parentesi quadre
vicino al nodo in questione.

Nel nostro caso esploriamo solo la sorgente.



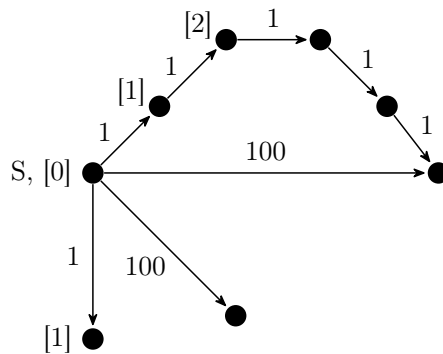
Passo 1:

Esplorazione di distanze minime con peso 1.
Esploriamo i nodi che sono collegati alla sorgente
da un percorso di peso 1.



Passo 2:

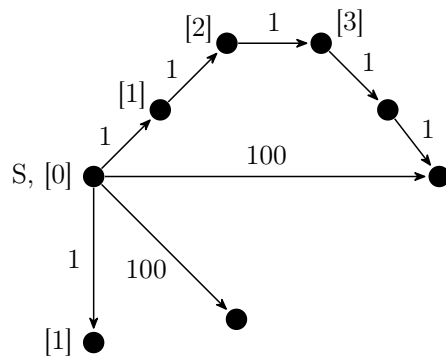
Esplorazione di distanze minime con peso 2.
Esploriamo i nodi che sono collegati alla sorgente
da un percorso di peso 2.



Passo 3:

Esplorazione di distanze minime con peso 3.

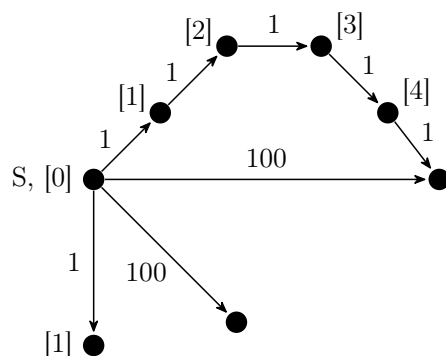
Esploriamo i nodi che sono collegati alla sorgente da un percorso di peso 3.



Passo 4:

Esplorazione di distanze minime con peso 4.

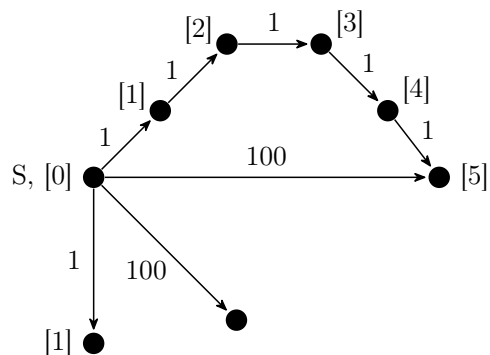
Esploriamo i nodi che sono collegati alla sorgente da un percorso di peso 4.



Passo 5:

Esplorazione di distanze minime con peso 5.

Esploriamo i nodi che sono collegati alla sorgente da un percorso di peso 5.



...

...

Passo 100:

Esplorazione di distanze minime con peso 100.

Esploriamo i nodi che sono collegati alla sorgente da un percorso di peso 100.

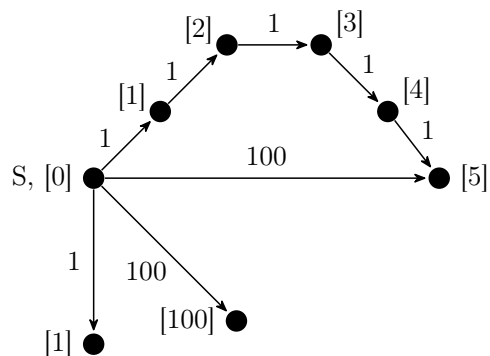


Figura 6.32: Esempio di applicazione dell'idea intuitiva naïve per il calcolo del percorso minimo in un grafo pesato

Tuttavia, come detto tale approccio funziona solo se abbiamo pesi discreti e inoltre è estremamente lento perché va per tentativi.

Quale può essere l'idea per migliorare l'approccio precedente?

Un'altra possibilità può essere quella di esplorare l'intera stella uscente, a differenza dell'approccio precedente, ma senza settare la distanza una volta per tutte, si tratterà comunque di una stima della distanza, ma è provvisoria, lasciando la libertà di modificarla e migliorarla potenzialmente tante volte.

Quindi quello che faremo è partire dal nodo sorgente settando la distanza a 0, poi andremo ad esplorare l'intera stella uscente settando i valori dei nodi della stella uscente con i valori attuali delle distanze che però saranno provvisori; se nel corso dell'esplorazione trovassimo un percorso più breve verso un nodo allora andremo ad aggiornare la distanza.

Solo al termine dell'intera visita potremo asserire che le nostre stime per le distanze sono effettivamente quelle corrette e definitive.

Non possiamo inoltre basarci, come in molti algoritmi precedenti, su un meccanismo di colorazione dei nodi, in quanto altrimenti non saremmo in grado di tornare su un nodo già scoperto per potenzialmente aggiornarne la distanza.

Vediamo però che ad ogni passo possiamo indicare alcune distanze come definitive, sono proprio le distanze minime che incontriamo ogni volta che esploriamo una stella uscente.

Quello che faremo sarà quindi proprio andare a percorrere solo i percorsi di distanza minima, simulando l'idea dell'approccio precedente di esplorare man mano solo i percorsi a distanza minima, ma rendendo l'approccio più astratto e generale.

Esplorando in questo modo siamo sicuri che le distanze che andremo ad indicare come definitive non saranno appunto migliorabili e che alla fine di tutta la visita avremo necessariamente le distanze corrette e definitive, proprio per i lemmi precedentemente dimostrati.

Ricordiamo che è fondamentale che i grafi siano grafi pesati con pesi non negativi.

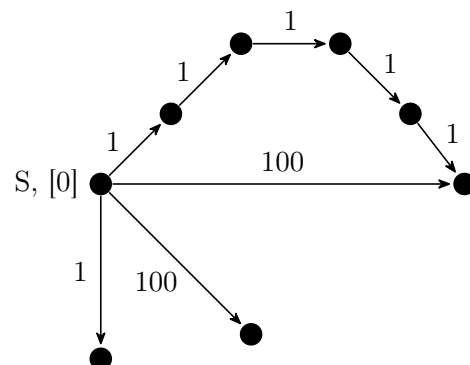
Utilizzando il grafo 6.31 applichiamo con un esempio l'idea intuitiva smart.

Partiremo sempre dalla stessa sorgente dell'esempio precedente.

Passo 0:

Partiamo con il nodo sorgente settando la distanza provvisoriamente a 0.

La distanza provvisoria è indicata da un valore racchiuso in parentesi quadre.

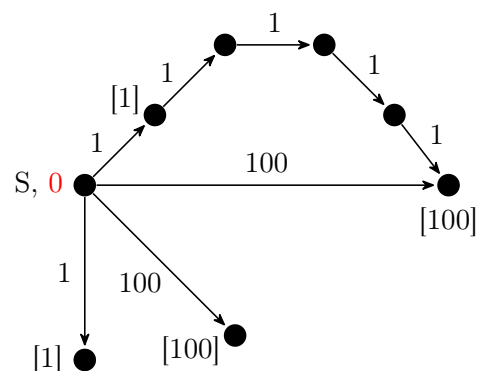


Passo 1:

Prendiamo la minima stima provvisoria delle distanze trovate fino ad ora, e mettiamo tale stima come definitiva (a livello grafico sarà indicata da un valore senza parentesi quadre).

Una volta fatto ciò esploriamo l'intera stella uscente, del nodo con stima definitiva, e settiamo le distanze provvisorie ai suoi adiacenti.

Nel nostro caso mettiamo come definitiva la sorgente, ed esploriamo la sua stella uscente.

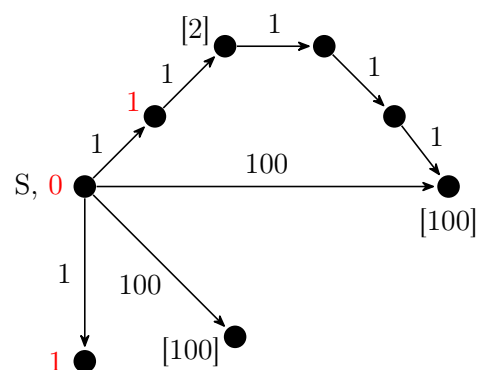


Passo 2:

Prendiamo la minima stima provvisoria delle distanze trovate fino ad ora, e mettiamo tale stima come definitiva.

Una volta fatto ciò esploriamo l'intera stella uscente, dei nodi in cui abbiamo appena posto la stima come definitiva, e settiamo le distanze provvisorie ai suoi adiacenti.

Nel nostro caso mettiamo come definitiva i nodi a distanza 1 (ce ne sono due), ed esploriamo le loro stelle uscenti.

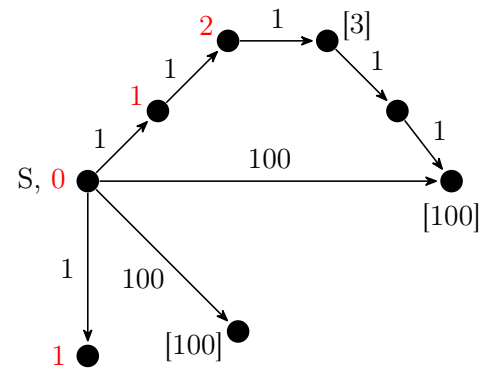


Passo 3:

Prendiamo la minima stima provvisoria delle distanze trovate fino ad ora, e mettiamo tale stima come definitiva.

Una volta fatto ciò esploriamo l'intera stella uscente, dei nodi in cui abbiamo appena posto la stima come definitiva, e settiamo le distanze provvisorie ai suoi adiacenti.

Nel nostro caso mettiamo come definitiva il nodo a distanza 2, ed esploriamo la sua stella uscente.

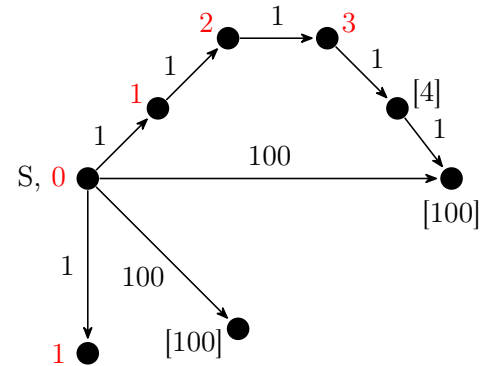


Passo 4:

Prendiamo la minima stima provvisoria delle distanze trovate fino ad ora, e mettiamo tale stima come definitiva.

Una volta fatto ciò esploriamo l'intera stella uscente, dei nodi in cui abbiamo appena posto la stima come definitiva, e settiamo le distanze provvisorie ai suoi adiacenti.

Nel nostro caso mettiamo come definitiva il nodo a distanza 3, ed esploriamo la sua stella uscente.

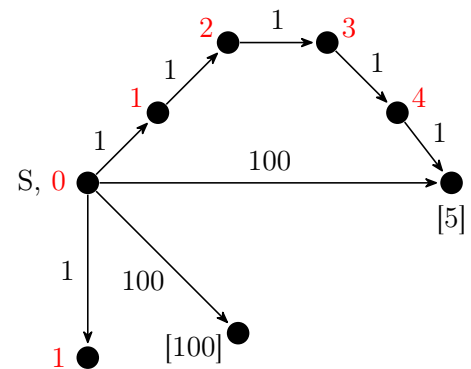


Passo 5:

Prendiamo la minima stima provvisoria delle distanze trovate fino ad ora, e mettiamo tale stima come definitiva.

Una volta fatto ciò esploriamo l'intera stella uscente, dei nodi in cui abbiamo appena posto la stima come definitiva, e settiamo le distanze provvisorie ai suoi adiacenti.

Nel nostro caso mettiamo come definitiva il nodo a distanza 4, ed esploriamo la sua stella uscente.

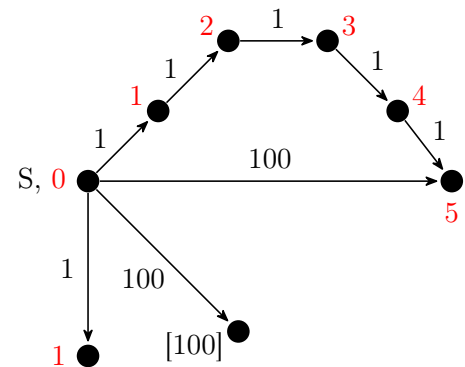


Passo 6:

Prendiamo la minima stima provvisoria delle distanze trovate fino ad ora, e mettiamo tale stima come definitiva.

Una volta fatto ciò esploriamo l'intera stella uscente, dei nodi in cui abbiamo appena posto la stima come definitiva, e settiamo le distanze provvisorie ai suoi adiacenti.

Nel nostro caso mettiamo come definitiva il nodo a distanza 5, ed esploriamo la sua stella uscente.



Passo 7:

Prendiamo la minima stima provvisoria delle distanze trovate fino ad ora, e mettiamo tale stima come definitiva.

Una volta fatto ciò esploriamo l'intera stella uscente, dei nodi in cui abbiamo appena posto la stima come definitiva, e settiamo le distanze provvisorie ai suoi adiacenti.

Nel nostro caso mettiamo come definitiva il nodo a distanza 100, ed esploriamo la sua stella uscente.

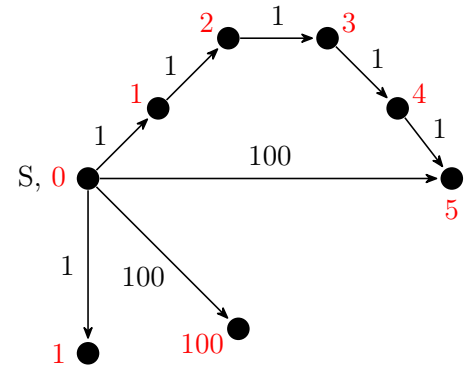


Figura 6.33: Esempio di applicazione dell'idea intuitiva smart per il calcolo del percorso minimo in un grafo pesato

Nell'approccio appena descritto quindi dobbiamo andare a prendere il minimo tra i vari pesi provvisori per capire come esplorare il grafo.

Come facciamo a far sì che questo prendere il minimo non costi troppo?

In effetti un approccio molto banale sarebbe quello di tenere i pesi all'interno di una lista o di un vettore ma sarebbe comunque dispendioso cercare il minimo, come ben sappiamo infatti la ricerca del minimo in una struttura dati lineare può avere un costo lineare nella dimensione della struttura.

Si può però fare di meglio rispetto a lineare.

C'è un'opportuna struttura dati, che è il concetto di Heap, che fa sì che il minimo si possa estrarre a tempo costante.

Quello che però costerà un po' di più di tempo costante, costerà logaritmico, sarà l'inserimento dei nuovi valori.

Quindi per guadagnare qualcosa perderemo sull'inserimento, perché come detto l'inserimento sarà logaritmico, ma allo stesso tempo la lettura del minimo sarà a tempo costante.

Vediamo, ora, un esempio più complesso, prima di passare all'algoritmo, che ci permetterà di introdurre il concetto di rilassamento.

Cioè quando la scoperta di un arco migliora una stima già effettuata, viene appunto detto che tale arco rilassa la stima.

Dato il seguente grafo.

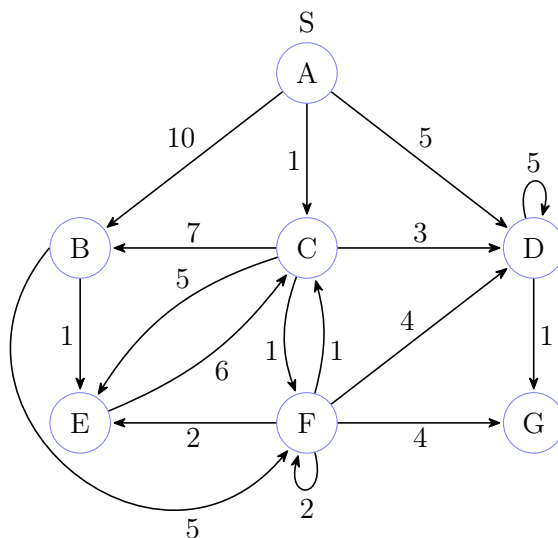


Figura 6.34: Esempio 3 di grafo pesato con pesi non negativi

Indicheremo con num^* (num è la stima) dove ci troviamo attualmente nell'esecuzione, e quando verrà settata una stima a definitiva, sostituiremo l'asterisco con una D.

La sorgente è il nodo A.

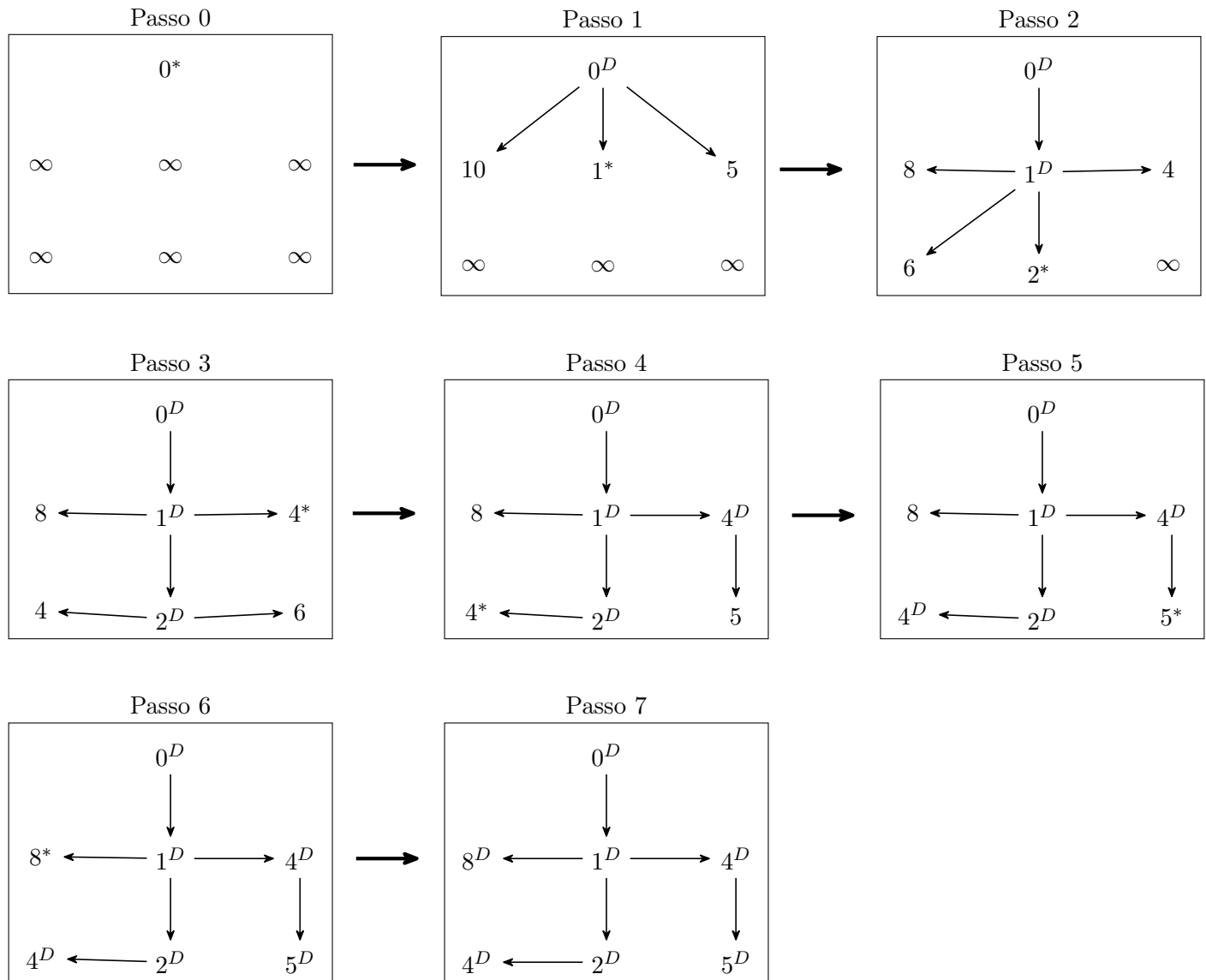


Figura 6.35: Esempio complesso per il calcolo del percorso minimo in un grafo pesato

6.17.5 Algoritmo di Dijkstra

Passiamo finalmente all'algoritmo.

Vediamo prima di tutto il codice di due funzioni ausiliarie che saranno fondamentali per l'algoritmo di Dijkstra.

La prima funzione è la funzione `init` che serve ad inizializzare correttamente i vettori necessari all'algoritmo, ponendo a $\perp$ il vettore dei predecessori e ad ∞ il vettore delle distanze, eccezion fatta per la distanza del nodo sorgente che verrà settata a 0.

Algoritmo 98: Funzione accessoria per l'inizializzazione di un grafo pesato

Signature `init`: $Graph(D) \times V \rightarrow Array(\mathbb{N} \cup \{+\infty\}) \times Array(D \cup \{\perp\})$

```

function init(G, s)
    // per ogni nodo del grafo in input
1   foreach (v in V) do
        // setta il predecessore a  $\perp$ 
2       p[v]  $\leftarrow$   $\perp$ 
        // se si tratta del nodo in input
3       if (v = s) then
            // setto la distanza a 0
4           d[v] = 0
5       else
            // altrimenti a  $+\infty$ 
6           d[v] =  $+\infty$ 
7   return (d,p)

```

Notiamo che non c'è il vettore dei colori perché come abbiamo già accennato siamo costretti a scoprire dei nodi già scoperti e pertanto il meccanismo della colorazione in questo caso non risulterebbe utile.

Dobbiamo infatti poi capire perché l'algoritmo termina visto che non essendoci alcuna colorazione esploriamo più potenziali percorsi che raggiungono un nodo.

Anticipiamo che possiamo essere sicuri che l'algoritmo termini perché effettivamente ad ogni iterazione un nodo diventa definitivo, e visto che lavoriamo sempre su grafi finiti questo basta per assicurare la terminazione dell'algoritmo. Lo dimostreremo a breve.

Vediamo ora la funzione che esegue invece la funzione di rilassamento.

Qual è l'idea?

Immaginiamo di aver analizzato un arco e di avere una stima per la distanza del nodo destinazione rispetto al

nodo sorgente. Se tale stima risulta migliorativa, cioè strettamente minore in termini quantitativi, allora andremo ad aggiornare il peso nel nodo.

Risulta chiaro che quindi tale funzione necessita del grafo in input e dell'arco in questione.

Algoritmo 99: Funzione accessoria per il rilassamento di un arco in un grafo pesato

Signature relax: $Graph(D) \times E \rightarrow \emptyset$

function relax(G, (v, w))

 // se la stima del peso del nodo destinazione

 // considerando l'arco in input

 // è migliorativa rispetto a quella attuale

1 **if** ($d[v] + wg(v, w) < d[w]$) **then**

 // aggiorniamo il peso del nodo di destinazione corrente

2 $d[w] \leftarrow d[v] + wg(v, w)$

 // e ne aggiorniamo il predecessore considerando l'arco in input

3 $p[w] \leftarrow v$

Notiamo che a parità di peso non modifichiamo nulla, in questo modo sceglieremo a parità di peso il percorso che attraversa un numero minore di archi.

Arriviamo finalmente all'algoritmo di Dijkstra.

Algoritmo 100: Algoritmo di Dijkstra

Signature `dijkstra`: $Graph(D) \times V \rightarrow Array(\mathbb{N} \cup \{+\infty\}) \times Array(D \cup \{\perp\})$ **function** `dijkstra`(`G`, `s`) // `d`: vettore distanze (pesi) dal nodo sorgente in input // `p`: vettore predecessori

// mediante una funzione accessoria

 // inizializzo i vettori delle distanze a $+\infty$ (eccezione nodo sorgente a 0) // e quello dei predecessori a $\perp$ **1** (`d`, `p`) $\leftarrow$ `init`(`G`, `s`)

// creo un insieme a partire dai nodi del grafo in input

// è importante che ci sia possibilità di ricercare il minimo

// e di effettuare inserimento

// tale insieme nella versione più efficiente sarà una struttura dati Heap

2 `Q` $\leftarrow$ `V`

// fino a quando ci sono elementi nell'insieme

3 **repeat** // rimuovo dall'insieme `Q` uno dei nodi con minimo peso

// ciò equivale a rendere il peso del nodo definitivo

4 (`Q`, `v`) $\leftarrow$ `remove_min`(`Q`, `d`)

// esploro la stella uscente del nodo rimosso

5 **foreach** (`w` in `Adj`[`v`]) **do**

// per ciascun nodo della stella uscente

// tento di rilassare il nodo in questione

6 `relax`(`G`, (`v`, `w`))**7** **until** (`is_empty`(`Q`))

// restituisco i vettori delle distanze e dei predecessori

8 **return** (`d`, `p`)

6.17.6 Complessità algoritmo di Dijkstra

Quanto costa tale algoritmo?

Come già sappiamo la funzione **init** costa lineare sul numero di nodi, pertanto $\Theta(|V|)$. Stessa complessità anche per la costruzione dell'insieme **Q** a partire dall'insieme dei vertici del grafo.

Il ciclo **repeat...until** visto che ad ogni iterazione rimuove un vertice dall'insieme **Q** verrà eseguito esattamente un numero di volte pari alla cardinalità dell'insieme di vertici del grafo e per ciascuno di essi va ad esplorare la stella uscente; risulta chiaro quindi che verranno eseguite un numero di operazioni pari al numero di archi del grafo, pertanto ha una complessità lineare nel numero di archi del grafo, quindi $\Theta(|E|)$.

Quindi globalmente tale algoritmo costa $\Theta(|E| + |V|)$.

In realtà l'analisi che abbiamo fatto risulta imprecisa. Sarebbe corretta se l'istruzione di rimozione del minimo fosse a tempo costante.

È quindi fondamentale capire tale istruzione quanto costi.

Come abbiamo già accennato, abbiamo modo di rendere tale operazione a tempo costante, è necessario utilizzare una struttura dati Heap.

Purtroppo, per poterlo fare, dobbiamo anche inserire un'ulteriore istruzione nella funzione **relax** che serve a dire alla coda a priorità (Heap) se un nodo debba migliorare o meno.

Tale miglioramento rende effettivamente l'operazione di estrazione del minimo costante, ma aggiunge un costo nella funzione **relax** che è logaritmico nel numero di vertici del grafo.

Quindi la versione con la coda a priorità dell'algoritmo di Dijkstra sarà $\Theta(\log(|V|) \cdot |E| + |V|)$.

Se invece di una coda a priorità utilizzassimo una lista non ci sarebbe un costo aggiuntivo nella funzione di **relax** ma l'estrazione del minimo sarebbe lineare e pertanto la complessità dell'algoritmo sarebbe $\Theta(|V| \cdot |E| + |V|)$.

Facciamo ora una stima più precisa di quella che è la complessità generale dell'algoritmo di Dijkstra.

Abbiamo infatti dato una stima sicuramente corretta ma è per eccesso.

In effetti una stima più precisa si può dare perché se osserviamo l'istruzione che fa la rimozione del minimo, essa è contenuta fuori dal ciclo **for** però viene eseguita $|V|$ volte, in quanto ad ogni iterazione viene sempre rimosso uno ed un solo nodo dall'insieme **Q** e pertanto dopo necessariamente $|V|$ iterazioni tale ciclo **repeat...until** dovrà terminare.

Quindi andando a fare l'analisi precisa, il ciclo **repeat...until**, nell'algoritmo senza coda a priorità, costa $|V| \cdot |V| + |E|$, cioè $|V|^2 + |E|$.

Facciamo quindi un'osservazione importante.

Se il grafo è denso, cioè quando $|E|$ è $\Theta(|V|^2)$, quindi quando abbiamo tanti archi, effettivamente la versione più efficiente è quella senza la coda a priorità perché comunque avremo una complessità pari a $|V|^2$ perché $|E|$ è prossimo a $|V|^2$.

Formalmente. Indipendentemente se utilizziamo o non utilizziamo una coda a priorità avremo:

$$|E| = \Theta(|V|^2) \implies \text{l'algoritmo ha complessità } \Theta(|V|^2) \quad (6.31)$$

Dunque la versione dell'algoritmo senza coda di priorità risulta più efficiente.

Nel momento in cui invece utilizziamo la coda a priorità, quello che otteniamo è un costo di $|V| + |E| \cdot \log(|V|)$.

Quindi se il grafo non è denso conviene utilizzare la coda a priorità perché ovviamente non c'è un termine paragonabile a $|V|^2$ e chiaramente conviene non inserirlo per colpa dell'istruzione di estrazione del minimo.

Infatti se $|E|$ è nettamente più piccolo di $|V|^2$, anche moltiplicarlo per un termine pari a $\log(|V|)$ non porterà a $|V|^2$.

Formalmente.

$$\text{grafo non è denso} \implies \text{l'algoritmo ha complessità } \Theta(|V| + |E| \cdot \log(|V|)) \quad (6.32)$$

Se invece il grafo è denso e quindi $|E|$ è paragonabile a $|V|^2$ non conviene usare la coda a priorità in quanto in ogni caso si avrebbe una complessità perfino maggiore di $|V|^2$ e quindi è preferibile utilizzare una struttura dati, come una lista, che è notevolmente più semplice da gestire, ottenendo anche una complessità migliore.

Formalmente.

$$\begin{aligned} \text{grafo denso} &\implies \text{l'algoritmo senza utilizzare una coda a priorità ha complessità } \Theta(|V|^2) \\ \text{grafo denso} &\implies \text{l'algoritmo con l'utilizzo di una coda a priorità ha complessità } \Theta(|E| \cdot \log(|V|)) \end{aligned} \quad (6.33)$$

Come al solito tutti i ragionamenti che facciamo non sono fini a sé stessi per quanto riguarda l'algoritmo, servono a capire che lo stesso algoritmo a livello astratto implementato poi in modo diverso, perché utilizza strutture diverse, è in pratica un software diverso con efficienze diverse, che può essere valido in un contesto e non valido in un altro, e viceversa.

È estremamente importante prendere tutte le informazioni di cui si è in possesso in considerazione, altrimenti si corre il rischio di usare per lo stesso problema lo stesso algoritmo, ma implementato in modo erraneo per quella particolare applicazione.

Lo scopo di questo manuale non è semplicemente dare una lista di algoritmi, serve per far ragionare, e per insegnare a ragionare.

6.17.7 Correttezza algoritmo di Dijkstra

Vediamo adesso la correttezza dell'algoritmo.

Per dimostrare la correttezza, quello che andremo a fare è in primo luogo dimostrare un insieme di tre lemma, e poi, in secondo luogo, utilizzare i lemma appena provati per dimostrare la correttezza dell'algoritmo di Dijkstra.

Il primo lemma intuitivamente dice la seguente cosa, cioè che dopo un'applicazione della funzione di **relax** su un certo arco (v, w) , necessariamente avremo che la stima del peso del nodo w , cioè $d[w]$, non può essere peggiore, cioè non può essere maggiore della stima di $d[v]$ più il peso dell'arco (v, w) .

Formalmente.

Lemma 6.17.3 (Invariante **relax**).

Sia G grafo pesato.

Dopo un'applicazione di **relax**($G, (v, w)$), con $(v, w) \in E$, si ha che: $d[w] \leq d[v] + wg(v, w)$

Dimostrazione.

Risulta immediato, osservando l'algoritmo della funzione **relax**, che effettivamente il rilassamento dell'arco avviene solo se la nuova stima del peso risulta essere migliorativa (strettamente minore) rispetto a quella corrente. Pertanto se viene effettuato un rilassamento avremo che la nuova stima sarà strettamente minore della precedente; se invece la nuova ipotetica stima risulta essere maggiore o uguale della corrente non vi sarà rilassamento.

□

Sebbene si tratti di una ovvietà è importante osservarla perché nel ragionamento per dimostrare la correttezza dell'algoritmo di Dijkstra a un certo punto la useremo ed inoltre è una proprietà caratterizzante della funzione **relax**.

Un conto è capire come fare una certa cosa e un altro è capire quella particolare cosa che proprietà ha e come sfruttarne le proprietà per altro.

Dobbiamo disaccoppiare sempre il sapere come dimostrare o come assicurarsi che qualcosa ha delle proprietà, dall'utilizzo delle proprietà in un'applicazione più generale.

Passiamo ad un altro lemma che ci permetterà di dire che se eseguiamo una serie di rilassamenti possiamo migliorare la stima, non peggiorarla.

Questa è una cosa che abbiamo ancora una volta visto in una versione semplificata per la visita in ampiezza ed avevamo osservato che l'algoritmo poteva solo migliorare la stima, quindi effettivamente crea un limite superiore al valore della distanza.

La stessa cosa facciamo in questo caso, soltanto che ora la stima sarà sulla distanza pesata, non sulla distanza geometrica.

Formalmente.

Lemma 6.17.4 (Limite stima distanza pesata).

Sia G grafo pesato.

Assumendo l'esecuzione di $\text{init}(G, s)$, dove viene impostato $d[s] = 0$ e $\forall v \in V \setminus \{s\} (d[v] = +\infty)$, ogni sequenza arbitraria $\text{relax}(G, (v_o, w_o)), \dots, \text{relax}(G, (v_k, w_k))$ ha come invariante che $d[v] \geq \delta_{wg}(s, v)$; inoltre, appena si ottiene $d[v] = \delta_{wg}(s, v)$ sia $d[v]$ che $p[v]$ non cambieranno più.

La sequenza di relax è arbitraria, non stiamo richiedendo che il nodo di destinazione di una operazione di relax sia il nodo di origine dell'operazione di relax successiva, e così via.

Dimostrazione.

Dimostriamo il lemma per induzione sulla lunghezza della sequenza di relax , i .

Il caso base, $i = 0$, è banale.

- $w = s \implies d[w] = 0 = \delta_{wg}(s, s)$
- $w \neq s \implies d[w] = \infty \geq \delta_{wg}(s, w)$

Passiamo al caso induttivo, consideriamo il lemma vero per un generico ma fissato i , e consideriamo il caso $i + 1$.

Se al passo $i + 1$ non avviene effettivamente un rilassamento, visto che non va a cambiare nulla, per ipotesi induttiva, le condizioni rimangono verificate.

Se, invece, al passo $i + 1$ avviene un rilassamento, avremo (considerando anche un lemma precedentemente dimostrato)

$$d[w_{i+1}] = d[v_{i+1}] + wg(v_{i+1}, w_{i+1}) \geq \delta_{wg}(s, v_{i+1}) + wg(v_{i+1}, w_{i+1}) \geq \delta_{wg}(s, w_{i+1})$$

Questo termina la dimostrazione per induzione.

Concludiamo osservando che se $d[v] = \delta_{wg}(s, v)$ per il lemma precedente e per quanto appena dimostrato otteniamo immediatamente la tesi.

□

Come corollario immediato di questo lemma abbiamo la seguente cosa.

Cioè che preso un grafo pesato se un nodo w non è raggiungibile a partire da un nodo sorgente s , allora per qualunque sequenza arbitraria di relax avremo sempre che $d[w] = \delta_{wg}(s, w) = +\infty$.

Formalmente.

Corollario 6.17.2 (Corollario limite stima distanza pesata).

Sia G un grafo pesato e sia $Path(s, w) = \emptyset$. Assumendo $\text{init}(G, s)$.

Per ogni sequenza di relax , $\text{relax}(G, (v_o, w_o)), \dots, \text{relax}(G, (v_k, w_k))$, si avrà $d[w] = \delta(s, w) = +\infty$

Dimostrazione.

Osserviamo che visto che s non raggiunge w , abbiamo che $s \neq w$.

Banalmente, basta osservare che per il lemma precedente l'invariante da mantenere è che ad ogni istante della sequenza di relax si abbia $d[w] \geq \delta_{wg}(s, w) = \infty$

□

Arriviamo finalmente all'ultimo lemma.

Esso afferma che preso un grafo pesato ed un percorso minimo, π appartenente a $MinWPath(G, s, w)$, che scomponiamo nel sottopercorso π' e nell'ultimo arco del percorso π , (v, w) , e data una sequenza di relax , se prima del rilassamento dell'arco (v, w) si ha che la stima del peso del nodo v sia definitiva, cioè $d[v] = \delta_{wg}(s, v)$, allora a seguito del rilassamento dell'ultimo arco del percorso π , (v, w) , avremo necessariamente che $d[w] = \delta_{wg}(s, w)$.

Formalmente.

Lemma 6.17.5 (Stima definitiva peso sequenza relax).

Sia G un grafo pesato, sia $\pi = \pi' \circ (v, w) \in MinWPath(G, s, w)$, e consideriamo una sequenza di relax , $\text{relax}(G, (v_0, w_0)), \dots, \text{relax}(G, (v, w)), \dots, \text{relax}(G, (v_k, w_k))$, con $d[v] = \delta_{wg}(s, v)$ prima di $\text{relax}(G, (v, w))$.

Dopo $\text{relax}(G, (v, w))$, si ha $d[w] = \delta_{wg}(s, w)$

Dimostrazione.

Considerando l'ipotesi, lemma e corollari precedentemente dimostrati sappiamo che

$$\pi' \in MinWPath(G, s, v) \implies \delta_{wg}(\pi) = \delta_{wg}(\pi') + wg(v, w) = \delta_{wg}(s, v) + wg(v, w) = d[v] + wg(v, w)$$

Conseguentemente, possiamo avere 2 possibilità:

1. la $\text{relax}(G, (v, w))$ non viene applicata su w perché è stato trovato un percorso ottimale precedentemente, quindi abbiamo già $d[w] = \delta_{wg}(s, w)$
2. la $\text{relax}(G, (v, w))$ viene applicata e avremo $d[w] = d[v] + wg(v, w) = \delta_{wg}(s, w)$

Quindi, considerando il lemma precedente, la tesi.

□

A questo punto possiamo finalmente concentrarci sulla dimostrazione di correttezza dell'algoritmo di Dijkstra.

La dimostrazione è, per l'ennesima volta, qualcosa di ispirato alla dimostrazione di correttezza dell'algoritmo di visita in ampiezza.

Ovviamente, visto che l'algoritmo non procede nello stesso identico modo, sarà una variante un po' più complessa.

Essenzialmente quello che vogliamo dimostrare è che quando un generico nodo del grafo pesato, v , viene eliminato dall'insieme Q , l'algoritmo ha un valore corretto del peso del nodo v , cioè $d[v] = \delta_{wg}(s, v)$. Ovviamente supponiamo il nodo s del grafo come nodo sorgente.

In questo modo ci si assicura che poiché tale nodo non verrà più preso in considerazione abbia un valore di peso corretto e allo stesso tempo qualunque cosa sia stata fatta tramite di esso era corretta.

Teorema 6.17.1 (Dimostrazione correttezza algoritmo di Dijkstra).

Nell'esecuzione dell'algoritmo di Dijkstra, $\text{dijkstra}(G, v)$.

$\forall v \in V (v \text{ eliminato da } Q) \implies d[v] = \delta_{wg}(s, v)$

Dimostrazione.

Dimostriamo la correttezza dell'algoritmo di Dijkstra per assurdo, contraddicendo la correttezza.

Supponiamo, quindi, per assurdo, che $\exists v (v \text{ eliminato da } Q \wedge d[v] \neq \delta_{wg}(s, v))$. Ovviamente, abbiamo che $s \neq v$.

Per un corollario precedentemente dimostrato, $v \in \text{Reach}(G, s) \implies \text{Path}(G, s, v) \neq \emptyset$.

Inoltre, ovviamente, $s \neq v \implies \forall v (\delta(s, v) \geq 1)$.

Chiaramente, per un lemma precedentemente dimostrato

$$\exists v (v \text{ eliminato da } Q \wedge d[v] \neq \delta_{wg}(s, v)) \iff \exists v (v \text{ eliminato da } Q \wedge d[v] > \delta_{wg}(s, v))$$

Sia v^* il primo dei nodi del grafo (inteso in relazione all'esecuzione dell'algoritmo) con tali proprietà.

Consideriamo, $\pi^* \in \text{MinWPath}(G, s, v^*)$, quindi, come dimostrato i sottopercorsi di π^* saranno minimi.

Sia $y \in \pi^*$ ($y \in Q$ dopo la rimozione di v^*).

Visto che $v^* \neq s \implies \exists x \in V (x \notin Q \text{ dopo la rimozione di } v^* \wedge x \text{ predecessore di } y \text{ in } \pi^*)$.

Quindi, $\pi^* = s \circ \dots \circ x \circ y \circ \dots \circ v^*$.

Facciamo un'osservazione. Visto che il grafo è a pesi non negativi all'aumentare della lunghezza del percorso il peso del percorso potrà aumentare, o al massimo rimanere identico, ma di certo non diminuire.

Conseguentemente, $\delta_{wg}(s, v^*) \geq \delta_{wg}(s, y)$.

Facciamo un'ulteriore osservazione. x è un predecessore di y in $\pi^* \implies (x, y) \in E$, ed inoltre x viene rimosso da Q prima di v^* .

Pertanto $d[x] = \delta_{wg}(s, x)$; per il lemma precedente, quindi $d[y] = \delta_{wg}(s, y)$.

Infine, visto che nell'algoritmo eliminiamo un nodo da Q selezionando il nodo avente peso minimo nell'iterazione considerata, mediante la funzione `remove_min`; quindi, necessariamente $d[y] \geq d[v^*]$.

Integrando le nostre osservazioni, otteniamo una contraddizione

$$\delta_{wg}(s, v^*) \geq \delta_{wg}(s, y) = d[y] \geq d[v^*]$$

□

Questo conclude la discussione sull'algoritmo di Dijkstra.

Sottolineiamo che, come abbiamo ripetuto diverse volte, per l'algoritmo di Dijkstra è necessario che i pesi degli archi del grafo siano non negativi, ed è proprio nella dimostrazione di correttezza che tale proprietà risulta necessaria.

Infatti, i lemmi e il corollario precedenti funzionano anche su grafi pesati in generale.

6.17.8 Algoritmo di Bellman-Ford

L'algoritmo che vedremo a breve, detto algoritmo di Bellman-Ford, cerca di esplorare tutti i percorsi semplici, in modo intelligente, e applica la **relax** su tutti gli archi un numero di volte pari alla massima lunghezza di un percorso semplice, che in un grafo con n vertici sappiamo essere $n - 1$.

Fatto ciò, se non ci sono cicli a peso negativo, avremo calcolato le distanze ottimali.

Se invece nel grafo esiste almeno un ciclo a peso negativo aver esplorato nel grafo tutti i percorsi semplici non ci assicura di aver trovato l'ottimo perché basterebbe percorrere il ciclo negativo per diminuire il peso del percorso.

Attenzione. Abbiamo detto cicli a peso negativo, non archi negativi.

L'algoritmo di Bellman-Ford infatti, a differenza di quello di Dijkstra, funziona su grafi pesati anche con archi avente peso negativo a patto che non esistano cicli che complessivamente abbiano un peso negativo.

Quindi l'algoritmo di Bellman-Ford sarà fatto da una parte di calcolo iniziale, che è la parte più complessa dal punto di vista computazionale dell'algoritmo, e poi di un check, leggermente meno costoso dal punto di vista computazionale, per verificare se nel grafo in questione è presente un ciclo negativo.

In caso di presenza di un ciclo a peso negativo allora l'algoritmo non può aver calcolato le minime distanze perché come abbiamo visto, con cicli negativi le minime distanze perdono di significato.

Se invece nel grafo non ci sono cicli negativi ovviamente non c'è modo di diminuire il peso di un percorso mediante l'attraversamento di un ciclo e quindi l'algoritmo restituisce effettivamente le minime distanze.

Quindi intuitivamente quello che fa l'algoritmo di Bellman-Ford è esplorare il grafo per $|V| - 1$ passi calcolando le stime dei pesi dei percorsi.

Dopodiché esegue un'ulteriore iterazione. È chiaro che a seguito di tale iterazione andrà a percorrere un ciclo, visto che come sappiamo in un grafo di $|V|$ vertici i percorsi semplici possono essere lunghi al più $|V| - 1$.

Se quindi all' $|V|$ -esima iterazione troviamo un nodo che, a seguito della **relax**, è tentato ad abbassare il proprio peso è perché abbiamo trovato un ciclo a peso negativo.

Quindi a seguito dell'ultima iterazione saremo sicuramente in grado di capire se l'algoritmo restituirà un valore corretto o se il concetto di minimo percorso nel grafo in questione non ha senso.

In verità, per essere precisi, anche in presenza di cicli a peso negativo nel grafo orientato non è detto che tutti i valori calcolati dall'algoritmo di Bellman-Ford siano inutili, ma l'algoritmo così come è fatto non è in grado di discriminare quali siano corretti e quali no.

Ci possono essere alcuni dei valori che sono corretti se per raggiungere quei nodi non ci sono cicli negativi, ma la versione che vedremo noi dell'algoritmo di Bellman-Ford non effettua tale discriminazione e non ci darà quindi modo per distinguere ciò che può essere corretto da ciò che non lo è.

Tale algoritmo, per l'approccio utilizzato, rientra nella categoria dei cosiddetti algoritmi di bruteforce, in quanto applica, sebbene in modo intelligente, lo stesso ragionamento su tutti i percorsi per un numero fissato e predefinito di volte.

Infatti, anche se nel grafo i percorsi semplici fossero tutti formati da un solo arco, l'algoritmo farebbe comunque $|V|$ iterazioni, in quanto si pone nel caso peggiore, nel quale sappiamo che un percorso semplice è formato da $|V| - 1$ archi; non analizza pertanto la situazione man mano, ma solo alla fine.

Inoltre, l'algoritmo, ad ogni iterazione tenterà la **relax** per tutti i nodi essenzialmente alla cieca, senza un ordine prestabilito.

Ci sono varianti dell'algoritmo di Bellman-Ford più intelligenti, ma noi vedremo una versione semplice. Vediamo, come al solito, un esempio di esecuzione prima di passare al vero e proprio algoritmo.

Dato il seguente grafo.

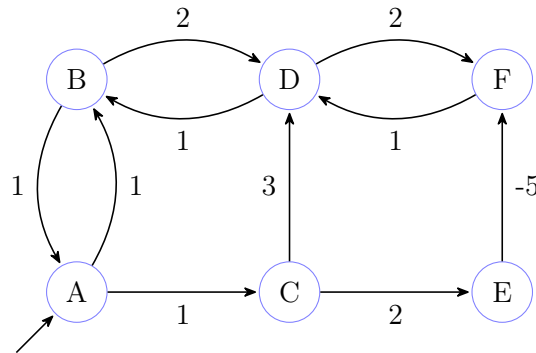


Figura 6.36: Esempio di grafo pesato senza cicli a pesi negativi

Visto che ci sono sei nodi, eseguiremo prima cinque iterazioni e alla sesta ci sarà il check per verificare se vi è presente un ciclo a peso negativo.

Sottolineiamo che l'ordine dei nodi è stato scelto in modo casuale.

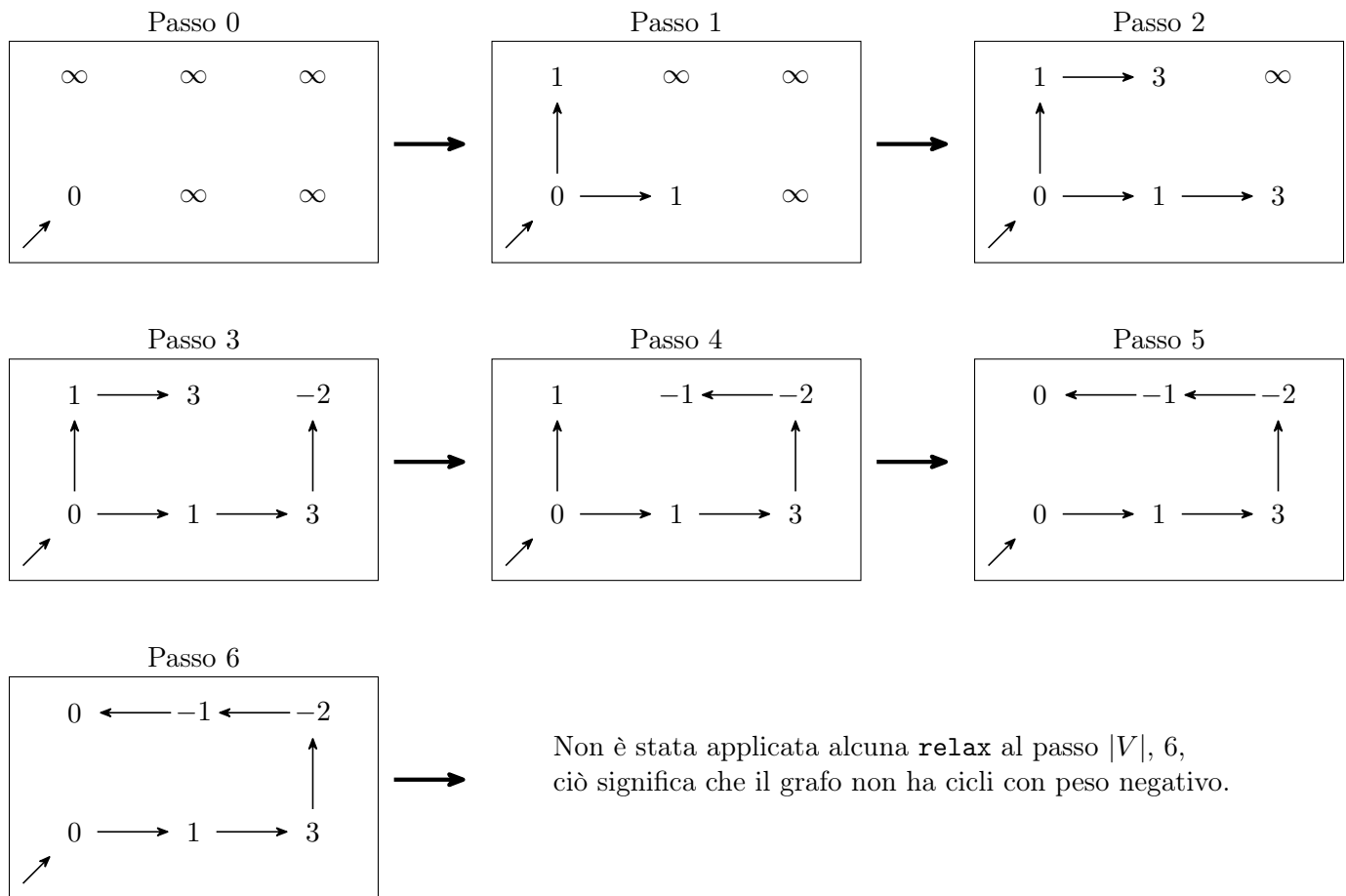


Figura 6.37: Esempio di esecuzione dell'algoritmo di Bellman-Ford su un grafo pesato senza cicli negativi

Vediamo adesso cosa sarebbe successo se avessimo modificato il peso dell'arco da B verso A a -1 .

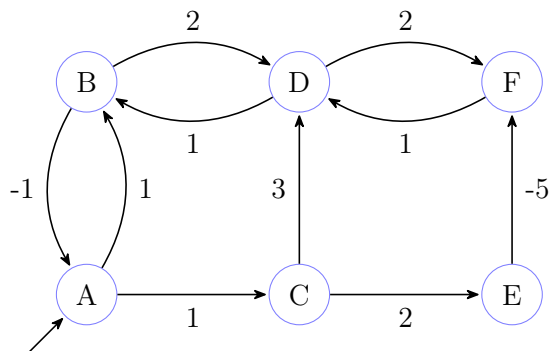


Figura 6.38: Esempio di grafo pesato con cicli a pesi negativi

Simuliamo solo il passo 6 poiché si può notare che i primi cinque passi sono equivalenti a quelli dell'esempio precedente

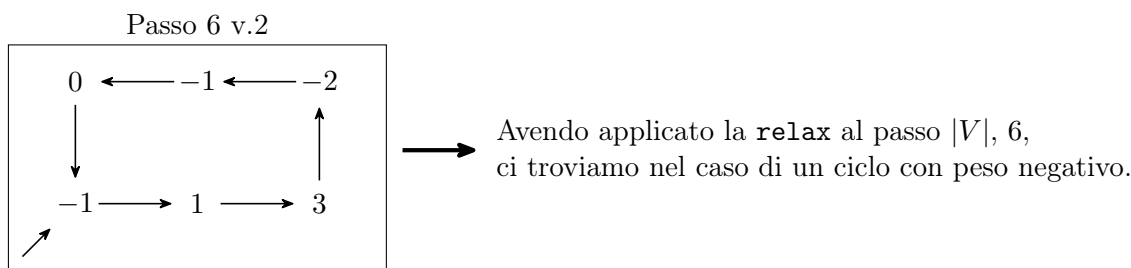


Figura 6.39: Esempio complesso per il calcolo del percorso minimo in un grafo pesato

Vediamo ora l'algoritmo.

La funzione di **init** è la stessa vista nell'algoritmo di Dijkstra.

Sottolineiamo inoltre che il vettore dei predecessori, **p**, ci permetterà alla terminazione dell'algoritmo, se non ci sono cicli negativi nel grafo, di costruire un albero e di ottenere esattamente i percorsi minimi.

Noi lo daremo per scontato, ma la dimostrazione che ciò che si ottiene dal vettore dei predecessori sia effettivamente un albero, e nello specifico un albero dei percorsi minimi pesati, non è una dimostrazione del tutto banale. Noi non la vedremo.

Per altro anche nell'algoritmo di Dijkstra, se nel grafo non sono presenti archi a peso negativo, dal vettore dei predecessori sarà possibile costruire un albero dei percorsi minimi pesati.

Algoritmo 101: Algoritmo di Bellman-Ford su grafi pesati

Signature `bellman_ford`: $Graph(D) \times V \rightarrow \perp \cup (Array(\mathbb{N} \cup \{+\infty\}) \times Array(D \cup \{\perp\}))$

function `bellman_ford`(`G`, `s`)

```

    // d: vettore distanze (pesi) dal nodo sorgente in input
    // p: vettore predecessori
    // mediante una funzione accessoria
    // inizializzo i vettori delle distanze a  $+\infty$  (eccezione nodo sorgente a 0)
    // e quello dei predecessori a  $\perp$ 
1   (d, p)  $\leftarrow$  init(G, s)
    // prima parte di calcolo iniziale
    // ripeto per un numero di volte pari alla massima lunghezza
    // di un percorso semplice nel grafo in input
2   loop (|V| - 1) times
    // per ciascun nodo del grafo in input
3   foreach (v in V) do
    // esploro la stella uscente
4   foreach (w in Adj[v]) do
    // tento il rilassamento dell'arco in questione
5   relax(G, (v, w))
    // seconda parte di check
    // per ogni nodo del grafo in input
6   foreach (v in V) do
    // esploro la stella uscente
7   foreach (w in Adj[v]) do
    // se la possibile stima del peso del nodo destinazione
    // è migliorativa rispetto a quella attuale
8   if (d[v] + wg(v, w) < d[w]) then
    // allora necessariamente è stato trovato
    // un ciclo a peso negativo
    // e restituisco un indicatore di errore
9   return  $\perp$ 
    // se il controllo è andato a buon termine
    // restituisco i vettori delle distanze e dei predecessori
10  return (d, p)

```

Stiamo assumendo di poter restituire i cosiddetti valori di tipo optional, nello specifico $\perp$ nel caso ci fosse un ciclo negativo nel grafo, due vettori, quello delle distanze (pesi) e quello dei predecessori, nel caso di assenza di cicli

negativi.

Non ci soffermeremo su come si possa implementare un meccanismo del genere visto che, come ripetuto più volte, ragioniamo ad un livello astratto.

Basti però pensare, per esempio alle union nel linguaggio di programmazione C per avere una possibile idea.

Complessità algoritmo di Bellman-Ford

Prima di concludere il discorso sull'algoritmo di Bellman-Ford valutiamone la complessità.

Come già sappiamo la funzione di `init` ha un costo lineare nel numero di vertici del grafo, quindi $\Theta(|V|)$.

La prima parte dell'algoritmo, quella di calcolo, è quella pesante dal punto di vista computazionale. Il ciclo `for` interno al `loop` viene chiaramente eseguito un numero di volte pari alla dimensione del grafo, quindi ha una complessità pari a $\Theta(|E| + |V|)$ ma tale ciclo viene eseguito per $|V| - 1$ volte.

Pertanto nel complesso avremo che la complessità della prima parte dell'algoritmo di Bellman-Ford sarà $\Theta(|V| \cdot |E| + |V|^2)$.

La seconda parte dell'algoritmo, quella di controllo, potrebbe terminare dopo aver testato il primo arco, ma se non ci sono cicli negativi testerà ogni arco; pertanto avremo una complessità pari a $O(|E|)$ e $\Omega(1)$.

Quindi nel complesso l'algoritmo di Bellman-Ford ha una complessità pari a $\Theta(|V| \cdot |E| + |V|^2)$.

Osserviamo quindi che nel caso di un grafo denso ha una complessità cubica nel numero di vertici del grafo; se invece grafo è sparso risulta essenzialmente quadratico nel numero di nodi del grafo.

Con questo terminiamo la nostra trattazione sui grafi.

Capitolo 7

Algoritmi di Ordinamento

"It's the question that drives us, Neo. It's the question that brought you here. You know the question, just as I did"

– Trinity (The Matrix)

7.1 Introduzione

Cominciamo adesso una parte relativa all'analisi degli algoritmi di ordinamento, partendo però da una parte leggermente più matematica che riguarda come capire se una certa funzione f , su un certo parametro, sia o meno O -grande di un'altra funzione g , oppure omega di un'altra funzione g , o theta di un'altra funzione g .

Fino a questo momento abbiamo effettuato sempre un'analisi un po' approssimata delle cose, sebbene tutte le nostre analisi erano basate su dei teoremi.

In particolare, ad esempio, abbiamo sempre detto che se abbiamo un'espressione polinomiale nella variabile n di grado k , allora l'intera espressione polinomiale sarà $\Theta(n^k)$, quindi possiamo omettere tutti i termini di grado inferiore al massimo; ma non abbiamo giustificato perché questo fatto fosse o meno vero, l'abbiamo dato per scontato.

7.2 Complessità asintotica

Come fare quindi, formalmente, a dire, avendo una funzione f e una funzione g , se $f = O(g)$, $f = \Omega(g)$, o $f = \Theta(g)$?

Il teorema che vedremo, che non è niente altro che una banale applicazione del concetto di limite, ci permetterà di capire quale di queste alternative sia vera e giustificherà finalmente ciò che fino a questo momento abbiamo dato per scontato.

È doveroso un piccolo appunto.

Abbiamo sempre ipotizzato le funzioni come funzioni positive perché nella nostra trattazione rappresentano i tempi di un algoritmo, e chiaramente un tempo negativo non ha senso. Inoltre le abbiamo sempre assunte monotone crescenti, cioè al crescere dell'input immaginiamo che l'algoritmo possa impiegare più tempo, non meno.

Quindi abbiamo sempre dato per scontato che in generale entrambe le funzioni, quella in analisi e quella di confronto, fossero definitivamente positive e crescenti.

Formalmente.

$$\exists n_0 \in \mathbb{N} (\forall n \geq n_0, f(n), g(n) > 0 (\forall n_1, n_2 \geq n (n_1 < n_2 \implies f(n_1) \leq f(n_2) \wedge g(n_1) \leq g(n_2)))) \quad (7.1)$$

A parte questa assunzione che abbiamo dato sempre per scontato, ricordiamo quali sono le definizioni fondamentali per il discorso che stiamo per affrontare.

Siano f e g due funzioni discrete da $\mathbb{N}$ in $\mathbb{N}$, definitivamente positive e crescenti.

$$\begin{aligned} f(n) = O(g(n)) &\iff \exists c \in \mathbb{R}^+, \exists n_o \in \mathbb{N} (\forall n \geq n_o (f(n) \leq c \cdot g(n))) \\ f(n) = \Omega(g(n)) &\iff \exists c \in \mathbb{R}^+, \exists n_o \in \mathbb{N} (\forall n \geq n_o (f(n) \geq c \cdot g(n))) \\ f(n) = \Theta(g(n)) &\iff \exists c_1, c_2 \in \mathbb{R}^+, \exists n_o \in \mathbb{N} (\forall n \geq n_o (c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n))) \end{aligned} \quad (7.2)$$

Ovviamente è evidente che dalla definizione di Θ si possono immediatamente dedurre entrambe le altre; il viceversa che, intuitivamente è ovvio, è vero, però bisogna stare attenti ad un piccolo dettaglio.

Se sono vere entrambe le prime due definizioni, quella di O -grande e di Ω , dobbiamo stare attenti che esistono potenzialmente due n_0 ; basterà prendere il massimo tra i due e considerarlo come n_0 per la terza definizione deducendola logicamente.

Come fare quindi avendo due funzioni, la funzione di analisi $f(n)$ e quella di confronto $g(n)$, a capire in quale caso ci troviamo?

Quello che andremo a fare è studiare il limite per n che tende a infinito del rapporto della funzione di analisi e quella di confronto.

Intuitivamente se andando a fare il rapporto di $f(n)$ e $g(n)$ al limite otteniamo un valore finito, costante, e diverso da zero, significa che al limite per n all'infinito avremo, nella funzione rapporto, $f(n)/g(n)$, un asintoto orizzontale; che significa che asintoticamente le due funzioni si stanno comportando esattamente allo stesso modo.

Se invece il limite del rapporto, $f(n)/g(n)$, tende ad infinito, significa intuitivamente che $f(n)$ cresce più velocemente e allora ci troveremo nel secondo caso, cioè $f(n)$ è $\Omega(g(n))$.

Viceversa se il limite del rapporto, $f(n)/g(n)$, tende a 0, cioè tende a un valore finito ma nullo, allora è la funzione $g(n)$ che cresce più velocemente di $f(n)$, e quindi ci troveremo nel primo caso, cioè $f(n)$ è $O(g(n))$.

Questo è intuitivamente il teorema che ora andremo a dimostrare semplicemente andando ad espandere la definizione di limite.

Formalmente.

Teorema 7.2.1 (Definizione complessità asintotiche).

Hp: Sia $h(n) \triangleq \frac{f(n)}{g(n)}$.

Th:

1. Se $\lim_{n \rightarrow +\infty} h(n) \in \mathbb{R}^+ \implies f(n) = \Theta(g(n))$;
2. Se $\lim_{n \rightarrow +\infty} h(n) = \infty \implies f(n) = \Omega(g(n))$;
3. Se $\lim_{n \rightarrow +\infty} h(n) = 0 \implies f(n) = O(g(n))$;

Dimostrazione.

Dimostriamo il teorema espandendo la definizione di limite.

Cominciamo con i casi in cui il valore del limite è finito (caso 1 e caso 3).

$$\lim_{n \rightarrow +\infty} h(n) = L \in \mathbb{R} \iff \forall \epsilon > 0 (\exists n_0 \in \mathbb{N} (\forall n \geq n_0 (|h(n) - L| < \epsilon)))$$

Segue, quindi, espandendo il valore assoluto

$$L - \epsilon \leq h(n) \leq L + \epsilon \iff L - \epsilon \leq \frac{f(n)}{g(n)} \leq L + \epsilon \iff (L - \epsilon) \cdot g(n) \leq f(n) \leq (L + \epsilon) \cdot g(n)$$

Fissiamo un certo ϵ .

- Caso 1. $L \in \mathbb{R}^+ \implies c1 = L - \epsilon \wedge c2 = L + \epsilon$
- Caso 3. $L = 0 \implies c = L + \epsilon$, in quanto la prima parte della disuguaglianza a tre vie non ha senso in questo caso date le ipotesi

Passiamo al caso in cui il valore del limite non è finito.

$$\lim_{n \rightarrow +\infty} h(n) = \infty \iff \forall c \in \mathbb{R}^+ (\exists n_0 \in \mathbb{N} (\forall n \geq n_0 (h(n) \geq c)))$$

Segue

$$h(n) \geq c \iff \frac{f(n)}{g(n)} \geq c \iff f(n) \geq c \cdot g(n)$$

□

7.2.1 Esempio analisi asintotica data una funzione

Il ragionamento appena visto per dimostrare il teorema lo si può utilizzare per identificare le costanti.

Lo vedremo solo in un caso particolare, cioè quando le funzioni f e g da un certo punto in poi sono crescenti e non oscillanti.

Anche perché che il tempo sia oscillante è abbastanza irragionevole. Quindi le funzioni che analizzeremo saranno da un certo punto in poi sempre monotone.

In verità le costanti si possono identificare anche quando le funzioni sono oscillanti ma noi non lo vedremo.

Facciamo un semplice esempio per capire perché il teorema appena visto ci permette di prendere solo il grado massimo di una certa funzione senza considerare i termini di grado minore del massimo.

Indicheremo con $h(n)$ la funzione rapporto.

Presa questa funzione $f(n) = n^3 + 25n \cdot \log(n) + n^{\frac{1}{2}}$.

Sappiamo che $f(n) = \Theta(n^3)$, quindi la funzione $h(n)$, sapendo che la funzione $g(n) = n^3$, sarà $h(n) = 1 + \frac{25 \log(n)}{n^2} + n^{-\frac{5}{3}}$.

Se andiamo a calcolare il limite di n che tende all'infinito di $h(n)$, avremo che $\frac{\log(n)}{n^2}$ tenderà a 0, $n^{-\frac{5}{3}}$ è uguale a $\frac{1}{n^{\frac{5}{3}}}$ che tende a 0, e dunque $h(n)$ tenderà a 1, un valore reale positivo come da teorema.

Continuiamo espandendo l'esempio precedente e mostrando come ottenere le costanti.

Risulta chiaro che la funzione di partenza è crescente da un certo punto in poi.

Prendiamo quindi la funzione h ed andiamo a studiarla.

Sappiamo grazie al limite calcolato nell'esempio appena visto che ha un asintoto orizzontale, bisogna capire se tende a tale limite dal basso o dall'alto.

Come noto bisogna calcolarne la derivata.

$$\begin{aligned} \frac{d}{dn} \left(1 + \frac{25 \log(n)}{n^2} + n^{-\frac{5}{3}} \right) &= 0 + 25 \cdot \frac{d}{dn} (\log(n) \cdot n^{-2}) + \frac{d}{dn} n^{-\frac{5}{3}} = 0 + 25 \left(\frac{1}{n} \cdot n^{-2} + \log(n) \cdot (-2n^{-3}) \right) - \frac{5}{3} n^{-\frac{8}{3}} = \\ &= \frac{25}{n^3} - \frac{50 \log(n)}{n^3} - \frac{5}{3} n^{-\frac{8}{3}} \end{aligned}$$

Come noto, se la derivata di una funzione positiva è maggiore di 0 da un certo punto, la funzione sarà crescente; viceversa, se la derivata è minore di 0, la funzione sarà decrescente.

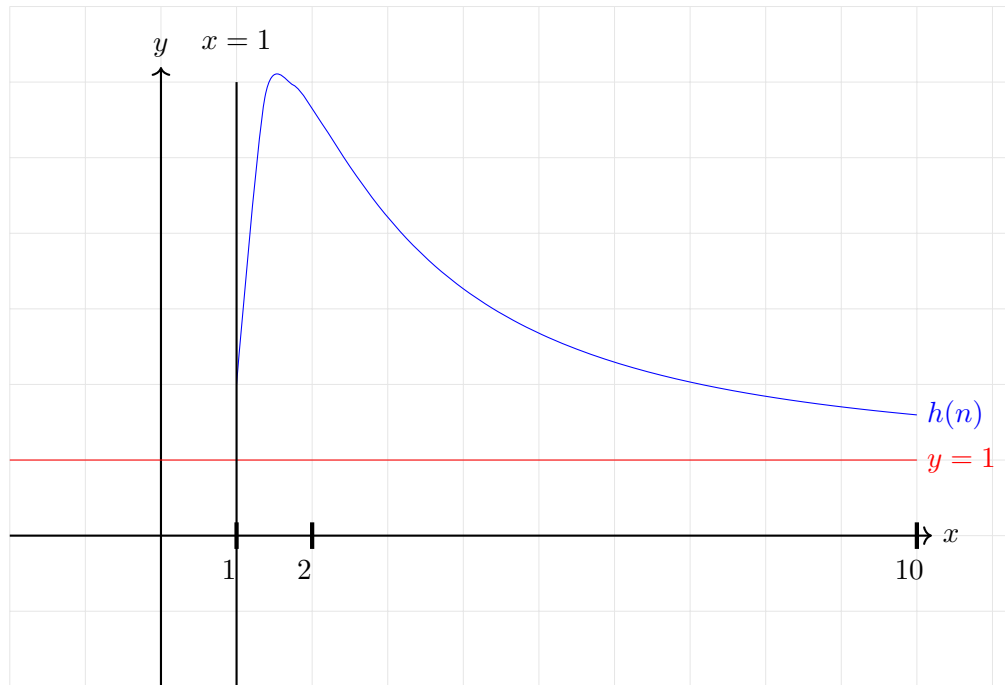
Vediamo dunque il segno della derivata calcolata precedentemente.

La derivata è negativa da un certo punto in poi, poiché tranne nel caso di $n = 1$, $\frac{50 \log(n)}{n^3}$, termine negativo, sarà predominante su $\frac{25}{n^3}$, termine positivo ed inoltre il termine negativo è anche aiutato dall'altro termine negativo $\frac{5}{3} n^{-\frac{8}{3}}$.

Possiamo quindi dire che: $\forall n > 1. \quad \frac{25}{n^3} - \frac{50 \log(n)}{n^3} - \frac{5}{3} n^{-\frac{8}{3}} < 0$

Attenzione. Il segno della funzione è stato trovato tramite osservazione senza utilizzare il metodo classico che generalmente si usa per trovare il segno di una funzione.

Notiamo che visto che la funzione $h(n)$ in questo caso è decrescente e tende all'asintoto orizzontale, senza mai raggiungerlo ovviamente, avremo che sarà sempre maggiore dell'asintoto orizzontale.



Quindi scomponendo la funzione $h(n)$ nel rapporto tra la funzione $f(n)$ e $g(n)$ otterremo che il limite è già una delle due costanti cercate.

In particolare, nel caso specifico la funzione rapporto tende all'asintoto orizzontale dall'alto perché è decrescente, quindi otterremo che la funzione calcolata, $f(n)$, è maggiore o uguale alla funzione di test, $g(n)$, moltiplicata per il valore dell'asintoto orizzontale, che è quindi una delle costanti cercate, nello specifico la costante inferiore, quella che abbiamo indicato con c_1 .

Abbiamo quindi $h(n) > 1$. Poiché $h(n) = \frac{f(n)}{g(n)}$, allora $f(n) \geq 1 \cdot g(n)$

Quindi il limite già ci dà una delle due costanti, quale delle due costanti lo capiamo da come la funzione rapporto approccia il limite.

Se la funzione rapporto approccia al limite asintotico dall'alto la costante che otteniamo dal limite è la costante inferiore, c_1 ; se invece la funzione rapporto approccia l'asintoto orizzontale dal basso perché è crescente il ragionamento sarà opposto perché la funzione rapporto, $h(n)$, sarà minore dell'asintoto e quindi otterremo la costante superiore, c_2 .

Quindi una volta che sappiamo se la funzione rapporto approccia dall'alto o dal basso il limite abbiamo già una delle due costanti.

Per il calcolo dell'altra costante quello che bisogna fare è semplicemente scegliere un qualunque punto a caso, cercando di essere però scaltri nella scelta in modo da semplificare i calcoli, cosa che spesso dipende dalla funzione considerata, ovviamente.

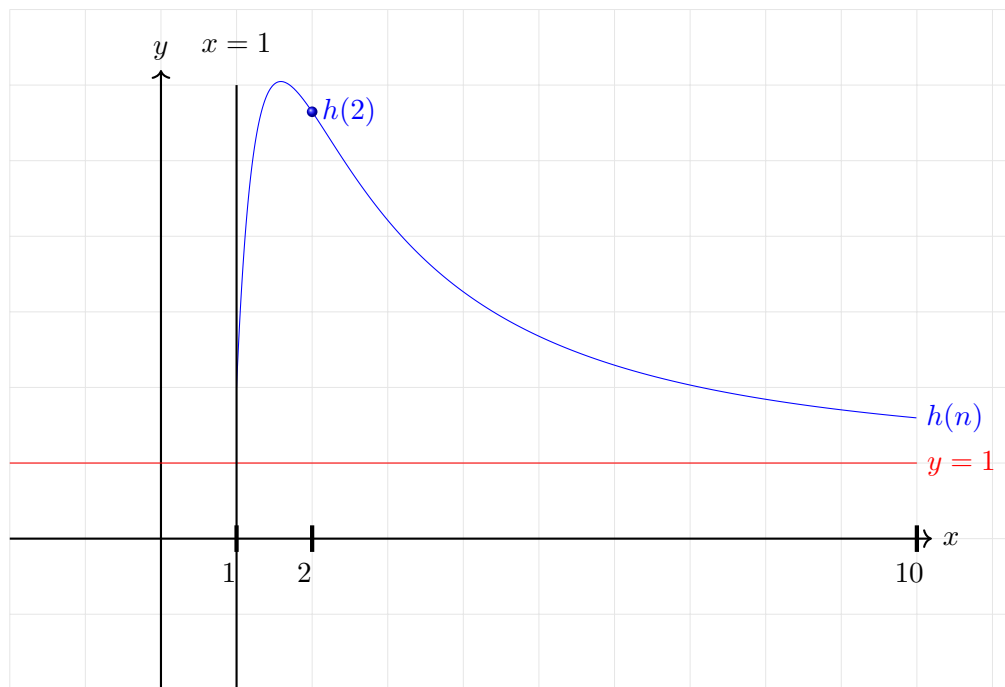
Continuando a considerare l'esempio, possiamo notare facilmente che la funzione $h(n)$ a partire da $n = 1$ è decrescente.

Quindi basta calcolare il valore della funzione h in qualunque punto a partire da 1, ad esempio $h(2)$, perché visto che come abbiamo più volte ripetuto si tratta di una funzione monotona decrescente siamo sicuri che per valori maggiori del punto scelto, 2, sarà sempre inferiore ad $h(2)$.

Quindi nel caso specifico possiamo scegliere come costante superiore, c_2 , proprio il valore di $h(2)$.

Abbiamo scelto quindi il valore $h(2)$, e va bene come valore perché essendo la funzione monotona decrescente vale che $\forall n > 2 (h(n) < h(2))$.

Vediamo adesso graficamente la funzione.



Il ragionamento è analogo ma duale nel caso di una funzione crescente, nel quale andremo a scegliere però ovviamente la costante inferiore, c_1 .

Questo è ciò che ci interessa per quanto riguarda la parte asintotica.

7.3 Algoritmi di ordinamento basati su confronti

A questo punto andiamo invece a cominciare la parte relativa agli algoritmi di ordinamento.

Vedremo diversi algoritmi di ordinamento, non sarà una trattazione finalizzata al solo algoritmo ma più che altro servirà soprattutto per esemplificare un po' di tecniche di calcolo della complessità.

Andremo a studiare tre algoritmi iterativi di ordinamento:

- Insertion sort;
- Selection sort;
- Heap sort.

Dopodichè passeremo agli algoritmi ricorsivi di ordinamento:

- Merge sort;
- Quick sort.

Vedremo anche il perché gli algoritmi di ordinamento non possono avere complessità inferiore a $n \cdot \log(n)$, ovviamente assumendo di fare soltanto confronti.

Partiamo con l'insertion sort.

7.4 Insertion sort

L'insertion sort è un algoritmo sicuramente non brillante ma utile perché si studia solitamente come primo approccio al calcolo della complessità.

Vedremo due versioni dell'algoritmo, che risolvono esattamente lo stesso problema.

Una versione iterativa ed una versione ricorsiva.

L'insertion sort è un algoritmo che cerca in qualche modo di prendere una sequenza e grazie ad inserimenti andare a riordinarla.

Prima però di descrivere effettivamente l'algoritmo di insertion sort e gli altri algoritmi di ordinamento dobbiamo capire cosa significa effettivamente algoritmo di ordinamento.

Cerchiamo di formalizzare il concetto.

L'input del problema ovviamente è una sequenza, che a livello algoritmico poi sarà semplicemente un vettore di elementi, di una certa lunghezza, i cui elementi fanno parte di un certo insieme di cui conosciamo un ordinamento totale.

L'output è una permutazione, cioè una sequenza della stessa lunghezza che contiene gli stessi dati della sequenza in input con la stessa molteplicità, che però deve essere anche ordinata.

Formalmente.

$$\begin{aligned} \text{Input: Sequenza } \vec{A} \text{ di lunghezza } n \text{ di elementi di un insieme } D \text{ totalmente ordinato } (\leq). \\ \text{Output: Permutazione } \vec{A}' \text{ di } \vec{A} \text{ tale che } \forall i, j \in [0, n) \left(i < j \wedge A'[i] \leq A'[j] \right). \end{aligned} \quad (7.3)$$

Ovviamente stiamo assumendo ordinato in modo crescente. Quello che vedremo sarà del tutto trasparente rispetto a che senso di ordinamento ci interessa.

Assumeremo quindi, per semplicità, che gli algoritmi che vedremo devono ordinare in modo crescente.

Quindi un algoritmo di ordinamento è qualcosa che risponde al problema descritto.

Cioè avremo quindi in input una sequenza di una certa lunghezza di dati ordinabili in modo totale e quello che deve restituire è una permutazione della sequenza originale in cui i dati sono ordinati.

Attenzione che è possibile avere copie dello stesso dato in più istanze.

7.4.1 Algoritmo: insertion sort ricorsivo

Vediamo, quindi, l'algoritmo di insertion sort nella variante ricorsiva.

L'idea di base dell'algoritmo è quella di eliminare dalla sequenza in input un elemento alla volta fino a raggiungere il caso in cui essa è formata da un solo elemento, nella quale sarà banalmente ordinata.

A questo punto uno alla volta gli elementi rimossi dalla sequenza saranno inseriti man mano in modo ordinato.

Algoritmo 102: Algoritmo di Insertion sort versione ricorsiva

Signature `insertion_sort`: $Array(D) \times \mathbb{N} \rightarrow \emptyset$

```
function insertion_sort(A, n)
    // se non ho raggiunto il caso base
    // cioè il sottoarray in input è di dimensione maggiore di 1
    // la sequenza (array in input) potrebbe non essere ordinata
1  if (n > 1) then
    // chiamo ricorsivamente la funzione
    // sull'array in input a cui rimuovo logicamente
    // l'ultimo elemento
2    insertion_sort(A, n-1)
    // al ritorno dalla chiamata ricorsiva
    // quando il sottoarray considerato è ordinato
    // ad eccezione dell'elemento rimosso
    // mediante una funzione accessoria lo inserisco in modo ordinato
3  insert(A, n-1)
```

Vediamo adesso la funzione accessoria `insert`.

Algoritmo 103: Funzione accessoria per l'insertion sort versione ricorsiva

Signature insert: $Array(D) \times \mathbb{N}^* \rightarrow \emptyset$ **function insert**(A, i)

// A: array ordinato in input fino alla posizione i-1

// salvo in una variabile accessoria

// il valore dell'elemento dell'array

// da inserire in modo ordinato

// è necessario in quanto a causa degli shift verso destra

// tale valore potrebbe andar perso

1 **x** $\leftarrow$ **A[i]**

// setto la posizione corrente alla precedente

// dell'indice dell'elemento da inserire

// cioè all'ultima posizione ordinata dell'array in input

2 **j** $\leftarrow$ **i-1**

// fino a quando non ho scorso tutto l'array

// oppure l'elemento da inserire risulta più piccolo del corrente

3 **while** ((**j** $\geq$ 0) **AND** (**x** < **A[j]**)) **do**

// spostato l'elemento corrente di una posizione verso destra

4 **A[j+1]** $\leftarrow$ **A[j]**

// e decremento l'indice della posizione corrente

5 **j** $\leftarrow$ **j-1**

// una volta terminato il ciclo while

// avrò spostato a destra tutti gli elementi dell'array maggiori

// dell'elemento da inserire

// che pertanto potrà essere posizionato correttamente

6 **A[j+1]** $\leftarrow$ **x**

7.4.2 Analisi dell'algoritmo insertion sort

Come abbiamo già anticipato, e come è facile notare, l'insertion sort sicuramente non è tra i più brillanti algoritmi di ordinamento.

Risulta però utile, proprio per la sua semplicità, come primo approccio per capire come analizzare un problema in modo po' più formale rispetto a quello che abbiamo fatto fino ad ora.

In primo luogo osserviamo che l'input di questo problema è ora più complesso.

In generale infatti potremmo avere diversi array da ordinare in input, della stessa dimensione, e tuttavia in base a come è fatto l'array l'algoritmo potrebbe avere efficienze diverse.

Immaginiamo di fissare la size degli array in input a un certo valore naturale n e di prendere diversi array di dimensione n .

Possiamo prendere un array già ordinato, un array ordinato ma in modo opposto (nel nostro caso quindi ordinato in modo decrescente), o possiamo prendere un array in cui gli elementi sono disposti in modo randomico.

Quello che si può osservare è che l'algoritmo di insertion sort su queste tre differenti istanze del problema, sebbene di uguale dimensione, si comporterà in modo differente e avrà diverse complessità.

Vedremo infatti che potremo caratterizzare il cosiddetto caso peggiore, così come il caso migliore, ed anche il caso medio.

Prendiamo il caso di una sequenza completamente ordinata.

Ovviamente l'algoritmo cercherà di ordinare l'array sebbene sia già ordinato.

L'insertion sort in questo caso specifico si comporta bene, vediamo il perché.

Supponiamo quindi che l'array iniziale sia già ordinato.

La parte ricorsiva dell'algoritmo di insertion sort scompone l'array nell'ultimo elemento e nel resto e si richiama. La seconda chiamata scomporrà l'array rimanente, di dimensione $n - 1$, nell'ultimo elemento più il resto, e così via fino a quando l'array non arriverà a dimensione 1.

Arrivati a questo punto, quando l'array ha dimensione 1, l'algoritmo non fa nulla perché la sola cella in posizione 0 è già ordinata; quindi restituisce il controllo senza far nulla e lo restituisce al caso in cui l'array considerato è di dimensione 2.

A questo punto viene chiamata la funzione di **insert**, ma visto che l'array in input è già ordinato avremo che l'elemento in posizione 1 dell'array sarà maggiore o uguale di quello in posizione 0, pertanto la funzione non entra nel ciclo **while** in quanto non supera il primo controllo.

Di conseguenza la funzione **insert** avrà fatto un numero di istruzioni costanti.

Il controllo viene quindi restituito, dopo un numero di operazioni costanti, alla chiamata ricorsiva del caso in cui l'array considerato è di dimensione 3.

Nuovamente viene chiamata la funzione **insert**, ma anche in questo caso, per gli stessi motivi di prima dopo un numero di operazioni costanti il controllo verrà restituito alla chiamata ricorsiva facente capo all'array di dimensione 4; e così via per tutte le chiamate ricorsive innestate, che sappiamo essere al più una per ogni elemento eliminato logicamente dall'array di partenza.

Di conseguenza avremo che in caso di array ordinato in input la funzione di **insert** avrà sempre tempo costante e verrà eseguita un numero di volte pari alle chiamate ricorsive della funzione di **insertion_sort**, che sappiamo essere n , dove n è la dimensione dell'array in input.

Quindi avremo che nel caso di array ordinato l'algoritmo di insertion sort nel complesso impiegherà tempo lineare.

Quindi nel caso di array ordinato meno che lineare la complessità non potrà essere, anche solo perché per fare la discesa ricorsiva l'algoritmo impiega n step, pertanto avrà una complessità pari a $\Theta(n)$.

Vediamo, invece, il caso in cui l'array in input è ordinato ma all'opposto, cioè invece che essere crescente è decrescente.

Come nel caso descritto in precedenza si effettua la discesa fino al caso base.

Il caso base non fa nulla. Consideriamo quindi il caso in cui abbiamo due elementi nell'array considerato.

Viene chiamata la funzione **insert** e questa volta però entriamo nel ciclo **while** visto che l'elemento in posizione 1 dell'array risulta essere minore dell'elemento in posizione 0 (stiamo leggermente semplificando non considerando il caso in cui l'array sia formato da tutti elementi identici).

Quindi viene effettuato lo shift a destra di una posizione dell'elemento in posizione 0 dell'array e all'uscita del ciclo **while** l'elemento che era in posizione 1 viene inserito in posizione 0. A questo punto il controllo viene restituito alla chiamata ricorsiva che considera l'array di dimensione 3.

Ancora una volta viene chiamata la funzione di **insert** e come sempre il valore dell'elemento da inserire viene salvato in una variabile accessoria.

Visto che però l'array è ordinato in senso inverso avremo che l'elemento da inserire è minore di tutti gli elementi posti in posizioni precedenti dell'array; pertanto tali elementi verranno shiftati di una posizione verso destra e infine l'elemento da inserire verrà posto in posizione 0 dell'array.

Alla successiva chiamata della funzione **insert** la situazione sarà analoga in quanto ogni elemento che andremo ad inserire sarà minore di tutti gli elementi già inseriti e quindi ogni volta il ciclo **while** verrà eseguito il numero massimo di volte possibili. Questo fino a terminazione dell'algoritmo ovviamente.

Globalmente, nel caso di un array ordinato in modo inverso, l'algoritmo di insertion sort, oltre alle n chiamate ricorsive andrà ad eseguire un numero di operazioni nella funzione di **insert** che saranno via via maggiori e direttamente proporzionali alla dimensione dell'array considerato ($1 + 2 + \dots + n$), vedremo che il numero di operazioni eseguite nella funzione di **insert** sarà pari alla sommatoria per i che va da 1 ad n di i ($\sum_{i=1}^n i$).

Come noto tale sommatoria può essere scritta come $\frac{n \cdot (n+1)}{2}$ che per il teorema precedentemente dimostrato ci porta ad affermare che nel caso di un array ordinato in modo inverso l'algoritmo di insertion sort ha una complessità che è $\Theta(n^2)$.

Sottolineiamo che abbiamo analizzato i due casi estremi.

Il caso in cui l'array in input è ordinato, che è il migliore; e il caso in cui l'array è ordinato in senso inverso, che invece è il peggiore.

È facile osservare che, indipendentemente dall'array in input, l'algoritmo dovrà in ogni caso effettuare un numero di chiamate ricorsive lineare rispetto alla dimensione dell'array; pertanto la discriminante per caratterizzare il caso peggiore o migliore è data dalla complessità della funzione **insert**.

Risulta immediato quindi, dall'analisi appena effettuata, che quando l'array è ordinato siamo nel caso migliore visto che il numero di operazioni eseguite dalla funzione **insert** è costante.

Quando, invece, in input abbiamo un array ordinato in senso inverso ci troviamo nel caso peggiore perché il numero di operazioni della funzione **insert** è il massimo possibile dato che saturiamo ad ogni chiamata tutte le possibili iterazioni del ciclo **while**, effettuando quindi il massimo numero di operazioni possibili.

È lecito quindi porsi una domanda.

Se andassimo a generare un array, sempre a parità di dimensione, in modo casuale, l'algoritmo di insertion sort si comporterebbe più come il caso migliore o come il caso peggiore?

La risposta a tale domanda definisce quello che viene solitamente chiamato caso medio.

Nel caso medio l'algoritmo di insertion sort, anticipiamo, si comporta più come il caso peggiore.

Vedremo che, in generale, spesso per valutare un algoritmo il caso medio risulta essere quello più significativo.

Putroppo, frequentemente risulta essere molto complicato da analizzare.

L'analisi del caso medio necessita infatti conoscenze di probabilità e statistica e pertanto ne daremo, a volte, solo un accenno e non andremo ad approfondirla come faremo invece per i casi peggiore e migliore.

7.4.3 Algoritmo: insertion sort iterativo

Prima di formalizzare i ragionamenti effettuati riguardanti l'analisi della complessità dell'algoritmo di insertion sort andiamo a dare anche la versione iterativa di tale algoritmo, che è ovviamente equivalente alla versione ricorsiva.

La funzione `insert` è la stessa della versione ricorsiva dell'algoritmo.

Algoritmo 104: Algoritmo di Insertion sort versione iterativa

Signature `insertion_sort`: $\text{Array}(D) \times \mathbb{N}^* \rightarrow \emptyset$

```

function insertion_sort(A, n)
    // visto che l'array di dimensione 1 è già ordinato
    // inizio a scorrere gli elementi da inserire a partire
    // da quello di posizione 1
1   for (i = 1 to n-1) do
        // per ciascuno di essi
        // mediante una funzione accessoria
        // effettuo un inserimento ordinato in un array in cui
        // tutte le posizioni precedenti sono ordinate
2   insert(A, i)
  
```

Tale algoritmo lavora in modo identico concettualmente, ma in modo diretto, rispetto alla versione ricorsiva. Invece di rimuovere un elemento dell'array alla volta come la versione ricorsiva per poi reinserire l'elemento, scorre direttamente l'array in avanti.

Sono chiaramente due algoritmi che fanno esattamente la stessa cosa. Hanno inoltre le medesime complessità, sebbene come sappiamo in generale non è detto che due algoritmi equivalenti dal punto di vista funzionale abbiano la stessa complessità. Sono comunque due algoritmi diversi, o meglio per essere precisi sono due differenti implementazioni di uno stesso algoritmo astratto.

7.4.4 Complessità dell'algoritmo insertion sort versione iterativa

Vediamo ora come formalizzare il calcolo della complessità per l'algoritmo di insertion sort.

Andiamo prima di tutto a calcolare il costo dell'istruzione comune alle due versioni dell'algoritmo, cioè la chiamata della funzione `insert`.

Diamo, per semplificare l'analisi, al costo in termini di tempo, ovviamente, di tale istruzione un nome. $T_{insert}(A, i)$. Che è quindi il tempo che impiega un'istruzione di chiamata della funzione `insert` su un array A e su indice i .

Risulta quindi immediato che il costo della versione iterativa dell'algoritmo di insertion sort non è niente altro che una somma dei vari termini $T_{insert}(A, i)$ al variare dell'indice i , a cui va anche, per dovere di precisione, aggiunto il costo dell'incremento dell'indice i nel ciclo `for` e anche il controllo che esso sia minore o uguale di un certo limite ($n - 1$); sebbene per ogni iterazione si tratti di un numero di operazioni costante.

Facciamo prima però un'importante osservazione.

La sommatoria e l'operatore di classe theta commutano.

Utilizzando l'operatore theta stiamo implicitamente affermando che esistono due costanti, c_1 e c_2 , tale che la funzione considerata che sappiamo appartenere a questa classe di complessità, è bounded sia inferiormente, da una costante c_1 , che superiormente, da una costante c_2 , quindi in pratica la funzione è costante.

Come noto se sommiamo una costante, quello che otterremo sarà qualcosa che dipende da quante volte effettuiamo la somma, quindi lineare nel numero di volte che tale somma viene effettuata. Se poi non ci interessa la costante e la portiamo fuori dalla sommatoria quello che otteniamo è la commutazione degli operatori.

$$\Theta(f(i) + g(i)) = \Theta(f(i)) + \Theta(g(i)) \quad (7.4)$$

Ovviamente lo stesso vale per il prodotto.

$$\Theta(f(i) \cdot g(i)) = \Theta(f(i)) \cdot \Theta(g(i)) \quad (7.5)$$

Ritornando all'analisi precedente, abbiamo che formalmente.

$$T_{IS_I}(A, n) = \sum_{i=1}^{n-1} (T_{insert}(A, i) + \Theta(1)) \quad (7.6)$$

Applichiamo semplici operazioni per semplificare tale sommatoria.

Sottolineiamo che alcuni passaggi che faremo saranno giustificati dal teorema visto in precedenza.

$$\sum_{i=1}^{n-1} (T_{insert}(A, i) + \Theta(1)) = \sum_{i=1}^{n-1} T_{insert}(A, i) + \sum_{i=1}^{n-1} \Theta(1) = [\dots] + \Theta\left(\sum_{i=1}^{n-1} 1\right) = [\dots] + \Theta(n-1) = [\dots] + \Theta(n)$$

Possiamo vedere quindi che il solo ciclo **for** contribuisce di un termine pari a $\Theta(n)$.

Se anche la parte dovuta alle chiamate della funzione **insert** dovesse rimanere $\Theta(n)$ avremmo un algoritmo che complessivamente ha una complessità che è $\Theta(n)$.

Questo è vero nel caso migliore ma, come sappiamo, non è sempre vero.

Abbiamo già osservato che se A è un array ordinato allora, indipendentemente dall'indice i , la funzione **insert** fa un numero di operazioni costante e quindi il tempo che impiega una chiamata della funzione **insert** è a sua volta costante.

Se invece A è un array ordinato in senso inverso la funzione di **insert** farà un numero di operazioni direttamente proporzionali all'indice i in input e lo stesso varrà quindi per il tempo.

Abbiamo quindi formalmete.

$$T_{insert}(A, i) = \begin{cases} \Theta(1) & , \text{ se l'array è ordinato.} \\ \Theta(i) & , \text{ se l'array è ordinato in senso inverso.} \end{cases} \quad (7.7)$$

In generale abbiamo che la complessità di $T_{insert}(A, i)$ è $\Omega(1)$ e $O(i)$.

Possiamo quindi ora facilmente notare le differenze nei due casi, il caso migliore quando in input abbiamo un array ordinato, e il caso peggiore, quando in input abbiamo un array ordinato in senso inverso.

Caso migliore

$$T_{IS_I}^B(A, n) = \sum_{i=1}^{n-1} (T_{insert}(A, i)) + \Theta(n) = \sum_{i=1}^{n-1} \Theta(1) + \Theta(n) = \Theta\left(\sum_{i=1}^{n-1} 1\right) + \Theta(n) = \Theta(n-1) + \Theta(n) = \Theta(n).$$

Caso peggiore

$$\begin{aligned} T_{IS_I}^W(A, n) &= \sum_{i=1}^{n-1} (T_{insert}(A, i)) + \Theta(n) = \sum_{i=1}^{n-1} \Theta(i) + \Theta(n) = \Theta\left(\sum_{i=1}^{n-1} i\right) + \Theta(n) = \\ &= \Theta\left(\frac{n \cdot (n-1)}{2}\right) + \Theta(n) = \Theta(n^2) + \Theta(n) = \Theta(n^2) \end{aligned}$$

Dall'analisi formale fatta sulla versione iterativa dell'algoritmo di insertion sort abbiamo, quindi, dimostrato che nel caso migliore, cioè quando in input abbiamo un array ordinato l'algoritmo ha una complessità pari a $\Theta(n)$; quando ci troviamo nel caso peggiore invece, cioè quando in input abbiamo un array ordinato in senso opposto, la complessità dell'algoritmo è $\Theta(n^2)$.

Inoltre, risulta immediato, che dato un generico array in input la complessità dell'algoritmo di insertion sort, dato l'input, sarà necessariamente tra quello del caso peggiore e quello del caso migliore.

$$T_{IS_I}^B(A, n) = \Theta(n) \leq T_{IS_I}(A, n) \leq \Theta(n^2) = T_{IS_I}^W(A, n)$$

Quindi, in generale possiamo dire che la complessità della versione iterativa dell'algoritmo di insertion sort è $\Omega(n)$ e $O(n^2)$.

E nel caso medio? Se dovessimo generare un array in modo randomico l'algoritmo si comporterebbe più come il caso peggiore, più come il caso migliore, o starebbe al centro?

Nel caso specifico di questo algoritmo si comporta praticamente sempre come il caso peggiore.

Questo fatto si può formalizzare bene con qualche nozione di matematica di probabilità generale. Noi lo daremo molto intuitivamente.

Quello che osserviamo è questo.

Se prendiamo un vettore random ci saranno con equa probabilità sufficienti coppie di elementi che saranno ordinate in modo corretto ed altrettante coppie ordinate in modo scorretto.

Sottolineiamo che per semplicità di analisi evitiamo di considerare array aventi elementi duplicati, ma si può fare la stessa analisi anche con i duplicati.

Quindi avremo tante coppie in cui i valori sono nell'ordine corretto e tante coppie in cui sono nell'ordine opposto.

Vediamone un esempio grafico.



Figura 7.1: Esempio di vettore random per l'insertion sort

Questo significa che l'algoritmo di insertion sort, e nello specifico la funzione `insert`, il 50% delle volte si troverà nel caso in cui il valore da inserire è ben posizionato e non entrerà nel ciclo `while`; ma il rimanente 50% delle volte non sarà così e quindi il numero di operazioni non sarà costante.

Significa che mediamente l'algoritmo `insert` effettuerà un numero di operazioni che saranno la metà del numero di operazioni che vengono eseguite nel caso peggiore. Quindi, rispetto al caso peggiore l'unica cosa che cambia è la costante moltiplicativa, che, come abbiamo detto più volte, non è rilevante ai fini del calcolo asintotico della complessità.

$$T_{insert}(A, i) = \Theta\left(\sum_{i=1}^{n-1} \frac{i}{2}\right) = \Theta\left(\frac{1}{2} \cdot \frac{n \cdot (n-1)}{2}\right) = \Theta(n^2)$$

Pertanto la complessità del caso medio dell'algoritmo di insertion sort è identica a quella del caso peggiore.

$$T_{IS_I}^{AVG} = \Theta(n^2) \quad (7.8)$$

Risulta chiaro che, come anticipato, l'analisi del caso medio serve solo a dare l'intuizione e non ha lo scopo di essere completa ed esaustiva.

7.4.5 Complessità dell'algoritmo insertion sort versione ricorsiva

Vediamo adesso il calcolo della complessità della versione ricorsiva dell'algoritmo di insertion sort.

A seguito vedremo la metodologia per provare la correttezza degli algoritmi di ordinamento.

Andiamo, in primo luogo, a calcolare il tempo della funzione ricorsiva di insertion sort perché come sappiamo nonostante l'idea dell'algoritmo sia la stessa, i tempi della versione iterativa e ricorsiva potrebbero in principio essere diversi (abbiamo già anticipato che non sarà questo il caso).

Questa formalizzazione risulterà importante in quanto fungerà da introduzione al calcolo delle equazioni di ricorrenza, che vedremo in modo più approfondito successivamente.

Guardando l'algoritmo ricorsivo risulta chiaro che se $n \leq 1$ non verrà eseguito nulla nel corpo, viene eseguita soltanto l'istruzione di controllo.

Conseguentemente se l'array è vuoto o contiene un solo elemento l'algoritmo eseguirà soltanto un controllo, che è costante, e terminerà.

Nota. Spesso al posto di $\Theta(1)$ per indicare un numero di operazioni costante si usa proprio una variabile, ad esempio c o perfino un valore numerico vero e proprio, per esemplificare maggiormente tale concetto.

Se invece l'array ha dimensione $n > 1$ l'algoritmo andrà comunque ad eseguire il controllo iniziale a cui dobbiamo sommare il tempo della chiamata ricorsiva su un'istanza di dimensione $n - 1$ ed anche il tempo della chiamata della funzione **insert**.

$$T_{IS_R}(A, n) = \begin{cases} \Theta(1) & , \text{ se } n \leq 1 \\ \Theta(1) + T_{IS_R}(A, n - 1) + T_{insert}(A, n - 1) & , \text{ altrimenti} \end{cases} \quad (7.9)$$

Quindi in generale abbiamo formalizzato il tempo dell'algoritmo considerando separatamente il caso base e il tempo che impiega internamente l'algoritmo a cui sommiamo il tempo della chiamata ricorsiva.

Ovviamente l'equazione di ricorrenza appena costruita dovrà essere svolta separatamente considerando le complessità del caso peggiore e del caso migliore, che come sappiamo influenzano il tempo della funzione **insert**.

Caso migliore.

$$T_{IS_R}^B(A, n) = \begin{cases} \Theta(1) & , \text{ se } n \leq 1 \\ \Theta(1) + T_{IS_R}^B(A, n - 1) & , \text{ altrimenti} \end{cases} \quad (7.10)$$

Caso peggiore.

$$T_{IS_R}^W(A, n) = \begin{cases} \Theta(1) & , \text{ se } n \leq 1 \\ \Theta(n) + T_{IS_R}^W(A, n - 1) & , \text{ altrimenti} \end{cases} \quad (7.11)$$

Vediamo il ragionamento intuitivo per la risoluzione di tale equazione di ricorrenza.

Risolveremo prima l'equazione di ricorrenza ottenuta nel caso migliore, poi quella del caso peggiore.

Se $n \leq 1$ il costo ce lo dice direttamente l'equazione di ricorrenza.

Concentriamoci quindi solo sul caso in cui $n > 1$.

$$T_{IS_R}^B(A, n) = T_{IS_R}^B(A, n-1) + c$$

Ovviamente potremo avere solo due alternative per il valore di $n-1$. Se $n-1 \leq 1$ allora finiremo nel caso base, che come abbiamo detto, è noto. Altrimenti possiamo iterare il ragionamento appena effettuato.

$$T_{IS_R}^B(A, n-1) = T_{IS_R}^B(A, n-2) + c$$

Stesso identico approccio su $n-2$, e così via.

Di conseguenza dopo $n-1$ passi necessariamente raggiungeremo il caso base.

$$T_{IS_R}^B(A, 1) = +c$$

In effetti i valori delle costanti nei vari step possono essere diverse ma, come sappiamo, a livello asintotico sono tutte $\Theta(1)$ e pertanto possiamo usare senza problemi la stessa costante.

Andiamo ad effettuare una semplice somma.

$$\sum_{i=1}^n T_{IS_R}^B(A, i) = \sum_{i=1}^{n-1} (T_{IS_R}^B(A, i) + c) + c$$

Svolgendo.

$$\begin{aligned} \sum_{i=1}^n T_{IS_R}^B(A, i) &= \sum_{i=1}^{n-1} (T_{IS_R}^B(A, i) + c) + c \\ \sum_{i=1}^n T_{IS_R}^B(A, i) &= \sum_{i=1}^{n-1} (T_{IS_R}^B(A, i)) + c \cdot \sum_{i=1}^{n-1} 1 + c \\ \sum_{i=1}^n T_{IS_R}^B(A, i) &= \sum_{i=1}^{n-1} (T_{IS_R}^B(A, i)) + c \cdot (n-1) + c \\ \sum_{i=1}^n T_{IS_R}^B(A, i) - \sum_{i=1}^{n-1} (T_{IS_R}^B(A, i)) &= c \cdot n \\ T_{IS_R}^B(A, n) &= c \cdot n = \Theta(n) \end{aligned}$$

Quindi la versione ricorsiva dell'algoritmo di insertion sort nel caso migliore ha una complessità che è $\Theta(n)$, come appunto la versione iterativa.

Passiamo ora al caso peggiore.

$$\begin{aligned}
 T_{IS_R}^W(A, n) &= T_{IS_R}^W(A, n-1) + c \cdot n \\
 T_{IS_R}^W(A, n-1) &= T_{IS_R}^W(A, n-2) + c \cdot (n-1) \\
 &\vdots \\
 &\vdots \\
 T_{IS_R}^W(A, 2) &= T_{IS_R}^W(A, 1) + c \cdot (2) \\
 T_{IS_R}^W(A, 1) &= c \cdot (1)
 \end{aligned}$$

Sommiamo tutti gli step, come fatto per il caso migliore.

$$\sum_{i=1}^n T_{IS_R}^W(A, i) = \sum_{i=1}^{n-1} (T_{IS_R}^W(A, i) + c \cdot (i+1)) + c$$

Svolgendo.

$$\begin{aligned}
 \sum_{i=1}^n T_{IS_R}^W(A, i) &= \sum_{i=1}^{n-1} (T_{IS_R}^W(A, i) + c \cdot (i+1)) + c \\
 \sum_{i=1}^n T_{IS_R}^W(A, i) &= \sum_{i=1}^{n-1} (T_{IS_R}^W(A, i)) + c \cdot \sum_{i=1}^{n-1} (i+1) + c \\
 \sum_{i=1}^n T_{IS_R}^W(A, i) &= \sum_{i=1}^{n-1} (T_{IS_R}^W(A, i)) + c \cdot \left(\sum_{i=2}^n i + 1 \right) \\
 \sum_{i=1}^n T_{IS_R}^W(A, i) - \sum_{i=1}^{n-1} (T_{IS_R}^W(A, i)) &= c \cdot \left(\sum_{i=1}^n i \right) \\
 T_{IS_R}^W(A, n) &= c \cdot \frac{n \cdot (n+1)}{2} = \Theta(n^2)
 \end{aligned}$$

Quindi la versione ricorsiva dell'algoritmo di insertion sort nel caso peggiore ha una complessità che è $\Theta(n^2)$, come appunto la versione iterativa.

Come avevamo annunciato le complessità delle due versioni dell'algoritmo di insertion sort sono identiche. Questo approccio, che è una versione molto semplificata di un approccio più completo che vedremo successivamente, ci permette già di capire come risolvere equazioni lineari con una sola chiamata ricorsiva, in particolare quando il coefficiente moltiplicativo della chiamata ricorsiva è 1.

7.4.6 Correttezza algoritmo insertion sort

C'è un'ultima cosa sulla quale è importante soffermarci.

Noi abbiamo dato per scontato che questi due algoritmi effettivamente andassero a ordinare l'array.

Come facciamo a capire effettivamente se è vero che questi algoritmi ordinano un generico array in input?

Sicuramente la dimostrazione più semplice di correttezza è sull'algoritmo ricorsivo perché possiamo applicare una banalissima dimostrazione per induzione.

Teorema 7.4.1 (Dimostrazione di correttezza dell'insertion sort ricorsivo).

Dimostrazione.

Dimostriamo la correttezza dell'algoritmo di insertion sort ricorsivo per induzione sulla dimensione dell'array, n .

Nei casi base, $n = 0 \vee n = 1$, chiaramente l'algoritmo non fa nulla e la correttezza è banale.

Passiamo al caso induttivo, considerando la tesi vera per un generico ma fissato n , considerando il caso per $n + 1$.

Per ipotesi induttiva sappiamo, quindi, che l'algoritmo ordina correttamente l'array di dimensione $n + 1$ fino alla dimensione n ; quindi ad eccezione dell'ultima cella dell'array tutte le altre celle saranno ordinate.

A questo punto l'algoritmo applica la funzione `insert` proprio sull'ultimo elemento dell'array.

Come sappiamo la funzione `insert` prende un elemento e lo va a posizionare esattamente nella cella successiva al primo elemento dell'array che risulta essere minore o uguale all'elemento da confrontare; spostando di una cella verso destra tutti gli elementi che sono strettamente maggiori dell'elemento da confrontare, preservandone l'ordine relativo.

Quindi, indicando con x l'elemento di posto n nell'array di dimensione $n - 1$, tutti gli elementi che sono minori o uguali di x si troveranno alla sua sinistra, e tutti quelli strettamente maggiori di x si troveranno alla sua destra.

Per ipotesi induttiva abbiamo la tesi.

□

La dimostrazione per induzione in particolare quando gli algoritmi sono induttivi è una delle cose più semplici da fare per dimostrare correttezza dell'algoritmo perché praticamente dobbiamo focalizzare la nostra attenzione solo su ciò che l'algoritmo fa fuori dalla chiamata ricorsiva; sulla chiamata ricorsiva possiamo assumere che l'algoritmo si comporti bene.

Dimostrare la correttezza per gli algoritmi iterativi è in generale più complesso.

Vediamo la dimostrazione di correttezza per la versione iterativa dell'algoritmo di insertion sort.

Teorema 7.4.2 (Dimostrazione di correttezza dell'insertion sort iterativo).

Dimostrazione.

Per dimostrare la correttezza dell'algoritmo di insertion sort iterativo dobbiamo identificare un invariante, cioè una proprietà che è vera ad ogni esecuzione della parte iterativa; nello specifico del ciclo **for** dell'algoritmo di insertion sort iterativo.

Si può osservare facilmente, che nell'algoritmo, al variare di i , tutto ciò che sta strettamente prima di i è già ordinato; ecco il nostro invariante.

Per dimostrare la correttezza dell'algoritmo ci basta quindi dimostrare che l'invariante che abbiamo identificato sia valido, in quanto se risulta verificato ovviamente avremo che quando i è uguale alla dimensione dell'array in input, allora l'array sarà necessariamente ordinato; lo faremo per induzione su i .

Il caso base, $i \leq 1$, è banalmente verificato.

Passiamo al caso induttivo, considerando vero l'invariante per un generico ma fissato i , e considerando il caso per $i + 1$.

Notiamo che, come nel caso dell'algoritmo di insertion sort ricorsivo, anche in questo caso viene effettuata una chiamata della funzione **insert** sull'elemento di indice $i + 1$; pertanto, per i medesimi ragionamenti fatti nella dimostrazione di correttezza dell'algoritmo di insertion sort ricorsivo abbiamo la tesi.

□

7.5 Selection sort

Vediamo ora un altro algoritmo che è addirittura in un certo senso, dal punto di vista dell'efficienza, peggiore dell'insertion sort in quanto sia il caso peggiore, sia il caso migliore, che il caso medio, risultano avere tutti complessità $\Theta(n^2)$.

È il cosiddetto algoritmo di selection sort.

Tuttavia, il fatto di partire da un algoritmo che non è particolarmente efficiente non significa che poi questo non possa portare all'algoritmo asintoticamente migliore che c'è, solo facendo un'osservazione molto semplice e sfruttando una struttura dati che permetta a una serie di operazioni di passare da tempo lineare a un tempo logaritmico.

Quindi sfruttando un'osservazione ed utilizzando un'opportuna struttura dati, che è lo Heap (abbiamo già visto potrebbe essere utile per l'algoritmo di Dijkstra) vedremo che l'algoritmo di selection sort può essere migliorato trasformandolo in quello che è chiamato heap sort.

Vedremo poi che l'algoritmo di heap sort è un algoritmo che nel caso peggiore, caso migliore e caso medio ha una complessità di $\Theta(n \cdot \log(n))$, che è il meglio che si può ottenere per l'ordinamento di dati in cui l'ordinamento è basato soltanto su confronti.

7.5.1 Algoritmo: selection sort

Ecco il codice dell'algoritmo di selection sort.

Vediamo prima la funzione accessoria `max`.

Algoritmo 105: Funzione accessoria per il selection sort

Signature `max`: $Array(D) \times \mathbb{N}^* \rightarrow \mathbb{N}$

```

function max(A, i)
    // setto inizialmente come indice
    // dell'elemento di valore massimo
    // dell'array in input la prima posizione
1   m ← 0
    // scorro tutto l'array a partire dalla seconda posizione
    // fino alla posizione corrispondente all'indice in input
2   for (j from 1 to i) do
        // se trovo un valore maggiore del massimo attuale
3       if (A[m] < A[j]) then
            // aggiorno la posizione del massimo del sottoarray
4         m ← j
    // restituisco la posizione dell'elemento massimo
    // del sottoarray considerato fino alla posizione data in input
5   return m
  
```

Risulta immediato che, indipendentemente dall'array in input, la funzione `max` andrà ad eseguire il ciclo `for` per $i - 1$ volte, effettuando quindi sempre un numero di operazioni linearmente proporzionali all'indice in input. Ha pertanto una complessità che è $\Theta(i)$ in ogni caso.

Vediamo adesso la funzione `selection_sort`.

Algoritmo 106: Algoritmo di Selection sort

```

Signature selection_sort:  $Array(D) \times \mathbb{N} \rightarrow \emptyset$ 
function selection_sort(A, i)
    // scorro l'array in input a ritroso
1   for (i from n-1 down to 1) do
        // ad ogni iterazione mediante una funzione accessoria
        // prendo l'indice del massimo elemento del sottoarray
        // fino all'indice corrente
2       m  $\leftarrow$  max(A, i)
        // posiziono il massimo del sottoarray considerato
        // nella posizione corrente scambiandolo con l'elemento
        // posto in posizione corrente
3       swap(A, i, m)

```

Anche in questo caso avremo che indipendentemente dall'array in input il ciclo **for** verrà eseguito per $n - 1$ volte.

Possiamo quindi facilmente calcolare la complessità temporale dell'algoritmo di selection sort.

$$T_{SS}(n) = \sum_{i=1}^{n-1} \Theta(i) = \Theta\left(\sum_{i=1}^{n-1} i\right) = \Theta\left(\frac{n \cdot (n-1)}{2}\right) = \Theta(n^2) \quad (7.12)$$

Notiamo che l'analisi formale è indipendente dall'array, infatti l'algoritmo di selection sort ha una complessità pari a $\Theta(n^2)$ sia nel caso peggiore, che in quello migliore; e pertanto anche nel caso medio.

Di conseguenza, come anticipato, l'algoritmo di selection sort non risulta essere particolarmente efficiente.

Tuttavia, questo algoritmo nonostante tutto è basato su un'idea intelligente che è la ricerca del massimo.

Se avessimo modo di identificare il massimo a tempo costante questo algoritmo diventerebbe addirittura lineare.

Il problema è che se cerchiamo di calcolare il massimo su un array che non ha alcuna struttura non abbiamo alcuna speranza di fare questo, ma se tuttavia riuscissimo a organizzare l'array in modo tale che, ad esempio, nella sottoparte che ancora dobbiamo ordinare il massimo sia sempre in posizione zero, allora potremmo identificare il massimo a tempo costante.

Come abbiamo però già detto esiste un teorema che afferma che non è possibile avere una complessità inferiore a $\Theta(n \cdot \log(n))$ per algoritmi di ordinamento basati solo su confronti.

Infatti, per strutturare l'array in modo tale da avere il massimo sempre in posizione zero è necessario spendere del tempo inizialmente per organizzare l'array in modo opportuno e poi ad ogni scambio dovremo aggiustare le cose perché l'istruzione di scambio andrà a danneggiare la struttura interna che quindi dovrà essere recuperata.

Fortunatamente il danneggiamento dovuto allo scambio è limitato, tanto che la correzione della struttura potrà avvenire a tempo logaritmico.

Conseguentemente, se abbiamo un ciclo `for` che viene eseguito $n-1$ volte in cui la ricerca del massimo sarà a tempo costante e l'istruzione di `swap` insieme alla successiva correzione avviene a tempo logaritmico, complessivamente avremo un algoritmo con complessità di $\Theta(n \cdot \log(n))$, indipendentemente dall'array.

Ovviamente questa è l'idea intuitiva. Vediamo come è possibile metterla in pratica.

Vediamo, quindi, prima di tutto come rimodellare un array, mantenendo invariati ovviamente i dati, ma facendo sì che in prima posizione ci sia sempre il massimo e che poi a seguito dello spostamento del massimo siamo in grado di riaggiustare la struttura a tempo logaritmico.

Ovviamente dovremo costruire in modo opportuno l'array prima dell'esecuzione dell'algoritmo con una specifica procedura che avrà tempo lineare nella dimensione dell'array.

Quindi ci sarà un preprocessamento, che avrà tempo lineare, di costruzione dell'array.

Come abbiamo detto l'algoritmo di correzione della struttura avrà complessità pari a $\Theta(\log(n))$.

E, complessivamente avremo una complessità di $\Theta(n \cdot \log(n))$.

Dunque l'idea è praticamente implementare in modo più smart un algoritmo di selection sort, sfruttando una struttura dati più efficiente dell'array non ordinato per la ricerca del massimo.

Quello che andremo a costruire sarà un array parzialmente ordinato in cui in posizione 0 ci sarà sempre il massimo. Costruiremo il cosiddetto max heap (max perché appunto in 0 c'è il massimo).

Chiaramente il ragionamento vale tal quale dualmente se volessimo ricercare il minimo.

7.6 Alberi Completi

Come prima cosa andiamo a definire il concetto albero completo.

Introduciamo una nuova famiglia di alberi, gli alberi completi.

Gli alberi completi sono alberi binari con la proprietà che ogni nodo interno tranne al più uno deve avere due figli e tutte le foglie sono a profondità h o $h - 1$, dove h è l'altezza dell'albero.

Quindi un albero è un albero completo se:

1. È un albero binario;
2. Tutti i nodi interni tranne al più uno hanno 2 figli;
3. Tutte le foglie sono a profondità h o $h - 1$, dove $h = h(T)$.

Facciamo alcuni esempi di alberi valutando se siano o meno alberi completi.

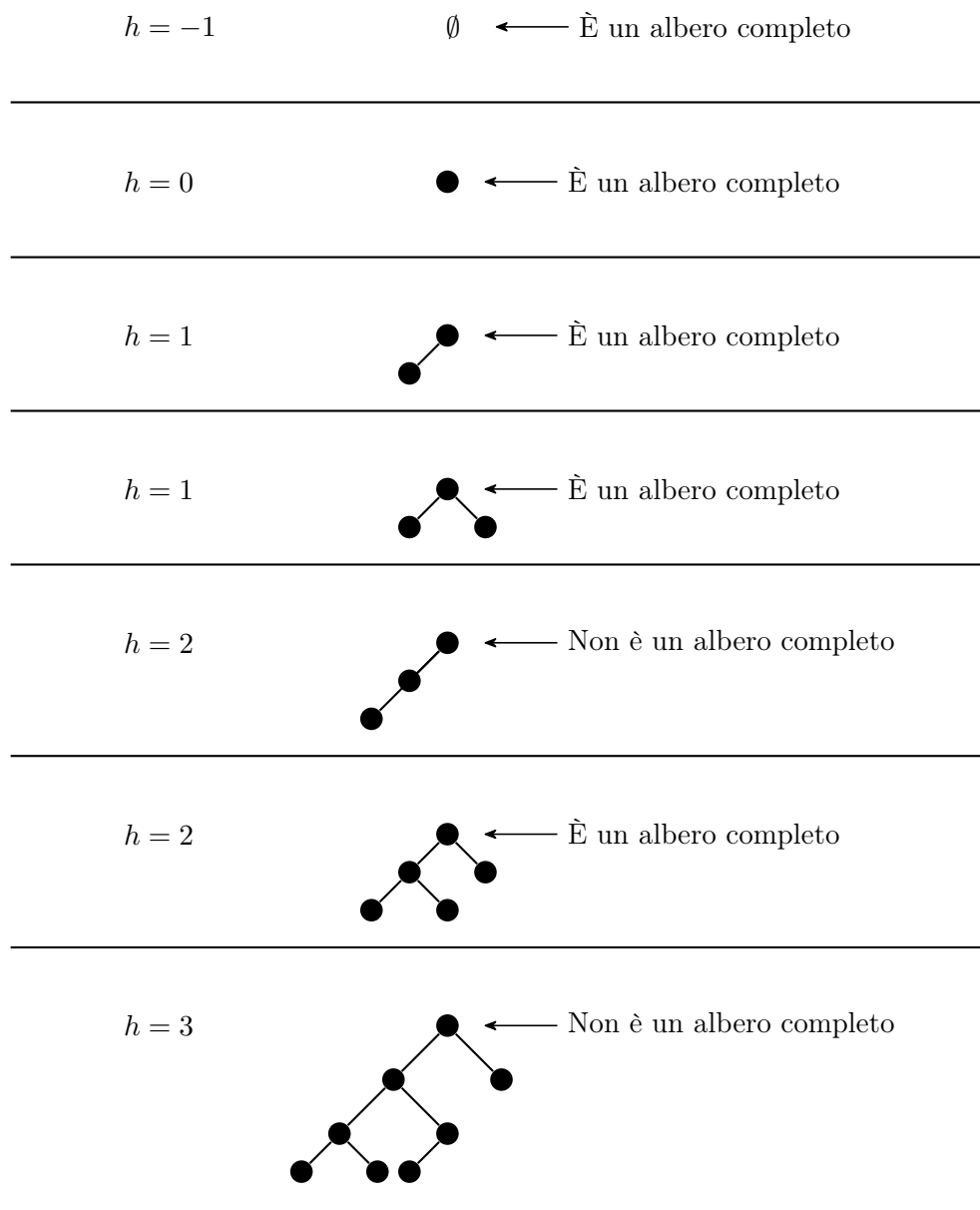


Figura 7.2: Esempio di individuazione di alberi completi

Si nota facilmente che un albero completo, grazie alle sue proprietà, è per forza un albero bilanciato. Inoltre, assumendo di mettere le foglie che sono di profondità massima verso sinistra e poi dopo le foglie di profondità non massima verso destra, strutturalmente un albero completo ha una forma caratteristica.

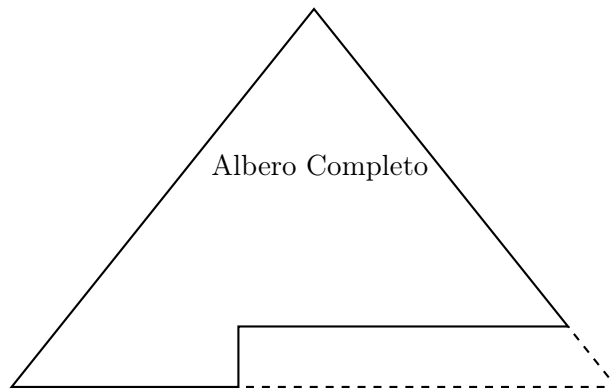


Figura 7.3: Forma caratteristica di un albero completo

Osserviamo che un albero completo è sempre in mezzo a un albero pieno più piccolo che è contenuto in esso ed un albero pieno più grande che lo contiene.

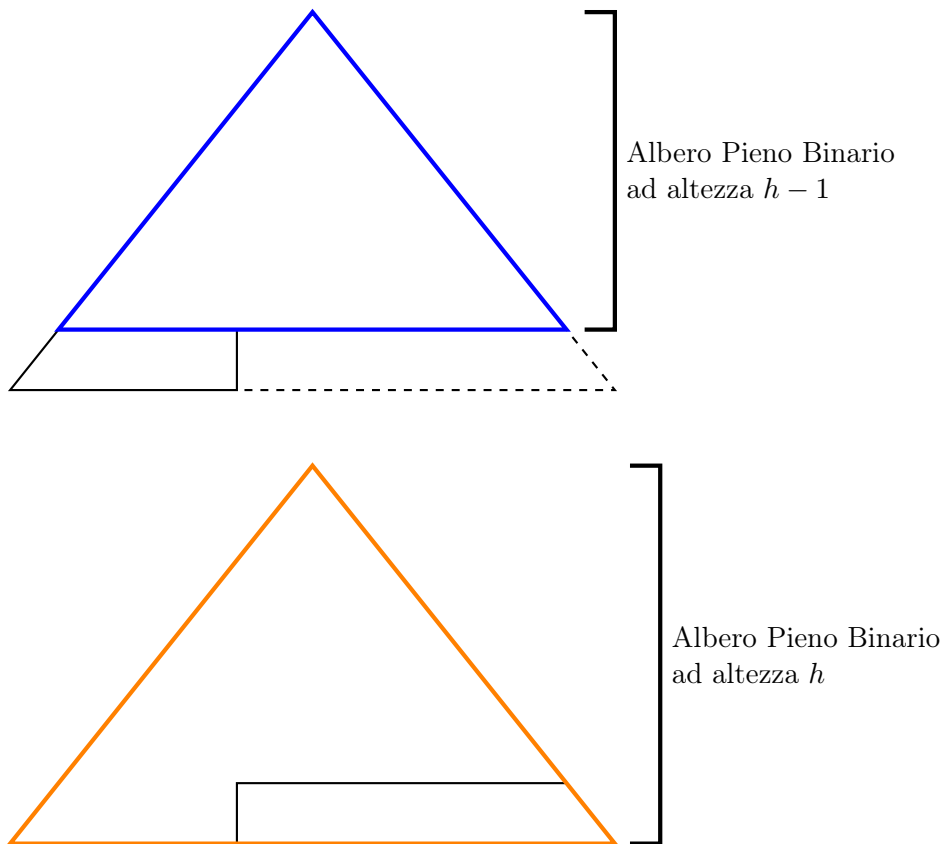


Figura 7.4: Visualizzazione degli alberi pieni nell'albero completo

Considerando il numero di nodi un albero completo avremo quindi necessariamente che sarà compreso tra il numero di nodi dell'albero pieno che contiene e di quello che lo contiene.

$$2^h - 1 < \#C_h \leq 2^{h+1} - 1 \quad (7.13)$$

Notiamo che il numero di nodi di un albero completo di altezza h deve essere strettamente maggiore del numero di nodi dell'albero pieno di altezza $h - 1$ che contiene, in quanto altrimenti sarebbe un albero completo di altezza $h - 1$ e non h .

Segue pertanto che un albero completo è un albero bilanciato.

Teorema 7.6.1 (Altezza albero completo).

L'altezza di un albero completo è logaritmica nel numero di nodi.

Dimostrazione.

Andando a risolvere la precedente disuguaglianza si ottiene immediatamente che quindi un albero completo è necessariamente bilanciato.

$$h = O(\log(n))$$

□

Ora che sappiamo che un albero completo è bilanciato, quindi che la sua altezza è logaritmica nel numero di nodi, se utilizziamo questa struttura e facciamo in modo tale che ogni operazione su tale struttura richieda solo un lavoro lungo un percorso di quest'albero, mai sull'interezza dell'albero, allora possiamo essere sicuri che le operazioni in questione avranno costo lineare nell'altezza dell'albero e quindi logaritmica nel numero di nodi.

Questa è l'intuizione che viene sfruttata negli alberi heap.

7.7 Alberi Heap

L'idea intuitiva è quindi di avere un albero che strutturalmente è un albero completo in cui le operazioni lavorano solo su percorsi dell'albero e in cui in radice, nel caso di un max heap, sarà sempre contenuto il massimo globale.

I due figli della radice potranno avere valori pari alla radice ma non potranno essere mai superiori. Cioè il valore di chiave in radice sarà sempre maggiore o uguale del figlio sinistro e anche maggiore o uguale del figlio destro.

Questa proprietà fondamentale sarà propria non solo della radice, ma di ogni nodo.

Quindi ogni nodo avrà una chiave maggiore o uguale della chiave del figlio sinistro e maggiore o uguale della chiave del figlio destro.

Non è quindi richiesta direttamente alcuna proprietà tra un nodo e i suoi discendenti, in quanto è implicita. Avremo infatti per forza di cose che in radice si trovi il nodo con il massimo globale.

Attenzione che però non vi è alcuna relazione tra nodi appartenenti a sottoalberi diversi.

Vediamo adesso un esempio di albero max heap e min heap.

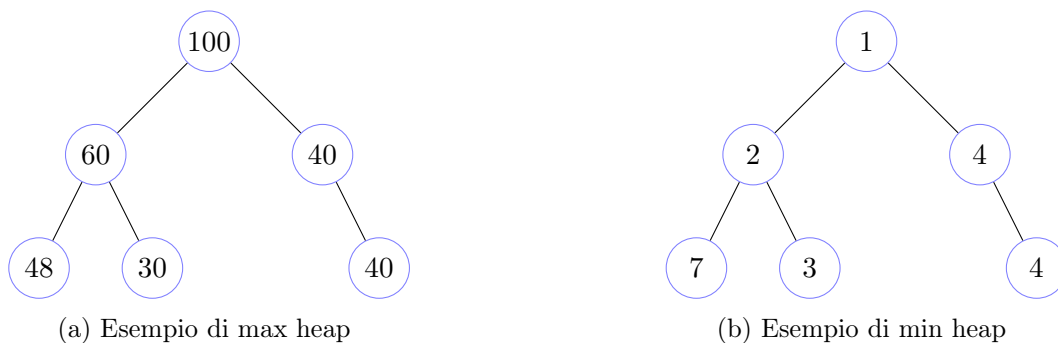


Figura 7.6: Esempi di alberi heap

La struttura del max heap verrà, come vedremo a breve, costruita in place nell'array originario.

È necessario, quindi, capire come rappresentare un albero binario in un array.

Quindi un max heap tree è un albero completo in cui ogni nodo è maggiore o uguale di entrambi i figli.

In un albero T max heap valgono le seguenti condizioni.

1. T è un albero completo;
2. $\forall x \in T (x.dat \geq x.l.dat \wedge x.dat \geq x.r.dat)$.

Vediamo ora come rappresentare un albero completo in un array.

L'idea è la seguente.

In posizione 0 avremo la radice dell'albero.

I due figli della radice andranno a occupare le immediate posizioni adiacenti; il figlio sinistro sarà in posizione 1 e il figlio destro sarà in posizione 2.

A questo punto ciò che è disponibile nell'array è solo a partire dalla posizione 3.

Quindi i figli del nodo in posizione 1 andranno ad occupare le posizioni 3 e 4; rispettivamente il figlio sinistro la posizione 3 e il figlio destro la posizione 4.

Ora le posizioni disponibili partono dalla 5, e quindi i figli del nodo in posizione 2 andranno in posizione 5 (il sinistro) e 6 (il destro).

E così via.

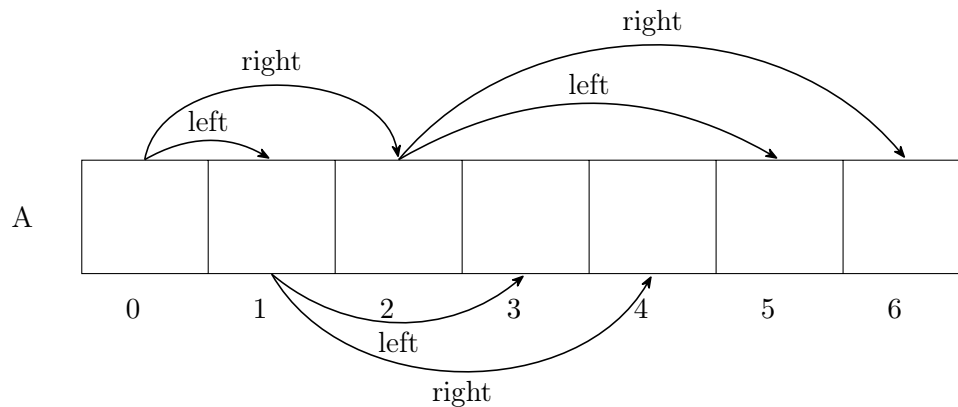


Figura 7.7: Esempio di rappresentazione di un albero completo in un array

Mostriamo una tabella in cui evidenziamo le posizioni dei figli per ciascun nodo.

	left	right
i	$2i+1$	$2i+2$
0	1	2
1	3	4
2	5	6

Si può facilmente osservare che ci sono due formule che dato l'indice di un nodo identificano il figlio sinistro e il figlio destro e lo calcolano a tempo costante.

Indice del figlio sinistro di un nodo di indice i è $2i + 1$

Indice del figlio destro di un nodo di indice i è $2i + 2$

Quindi in questo modo possiamo utilizzare direttamente un array per contenere un albero heap avente esattamente gli stessi dati.

Dovremo, però, partendo dall'array originale riposizionare gli elementi per fare in modo che le proprietà dell'albero heap vengano rispettate.

Per capire come questa fase di preprocessing possa avvenire, ed anche per comprendere le successive operazioni di ripristino della struttura, dobbiamo introdurre il concetto di albero quasi heap.

Un albero quasi heap è un albero completo in cui tutti i nodi, eccezion fatta per la radice, hanno un valore maggiore o uguale di entrambi i figli.

Essenzialmente, si tratta di un albero completo in cui i sottoalberi destro e sinistro della radice sono alberi heap, ma non è detto che la radice rispetti la proprietà di avere un valore maggiore o uguale di quello di entrambi i figli.

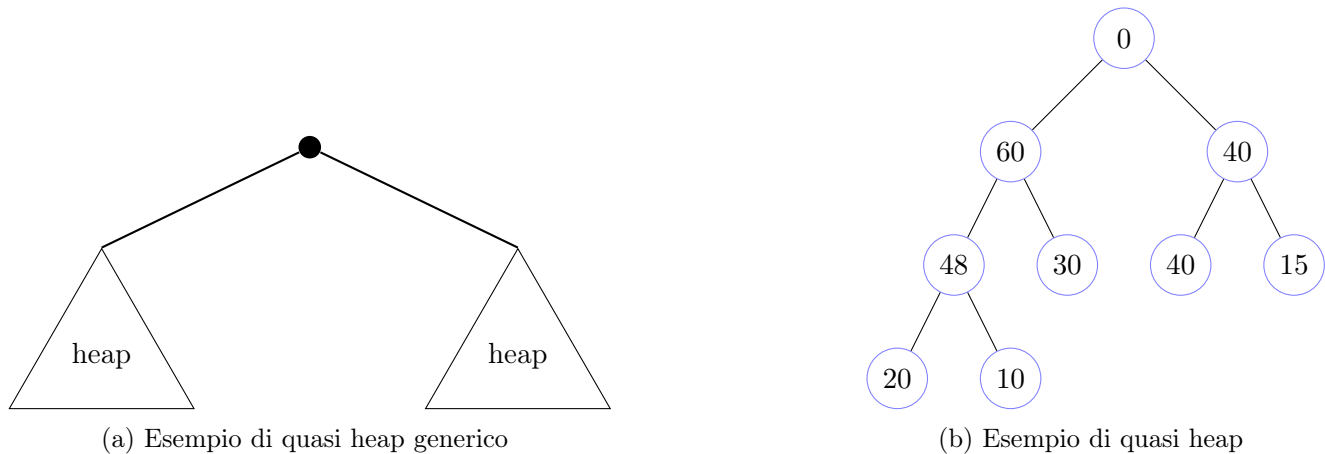


Figura 7.9: Esempi di alberi heap

Essendo però un albero completo è possibile rappresentarlo mediante un array come descritto in precedenza.

Vediamo il quasi heap della figura 7.9 rappresentato mediante array.

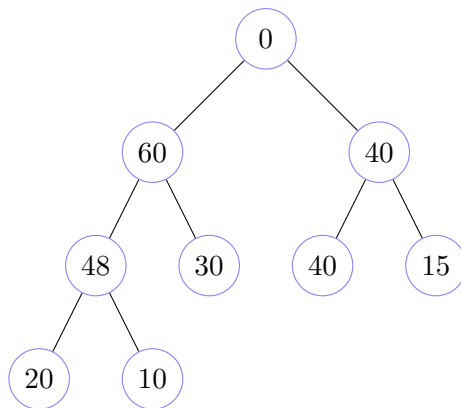
A	0	60	40	48	30	40	15	20	10
---	---	----	----	----	----	----	----	----	----

Figura 7.10: Esempio di rappresentazione di un albero completo in un array

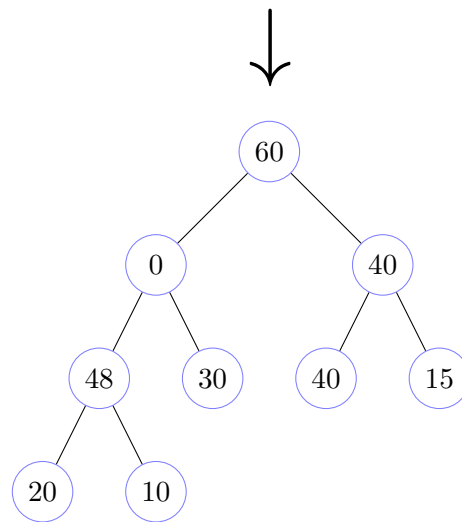
Se quindi l'unica cosa che distingue un albero heap da un albero quasi heap è la radice possiamo immaginare di correggere questo problema, per trasformare l'albero quasi heap in un albero heap.

L'idea intuitiva è quella di prendere il massimo tra i valori del nodo in radice e quello dei suoi due figli e di posizionarlo in radice mediante uno scambio.

Vediamolo applicato all'esempio di albero quasi heap in figura 7.9



Prendiamo il massimo tra 0, 60 e 40, e sostituiamo lo 0 al posto del nodo che ha valore massimo. In questo caso è il figlio sinistro che ha il valore massimo (60) e quindi sostituiamo il suo valore con 0.



Osserviamo una cosa importante.

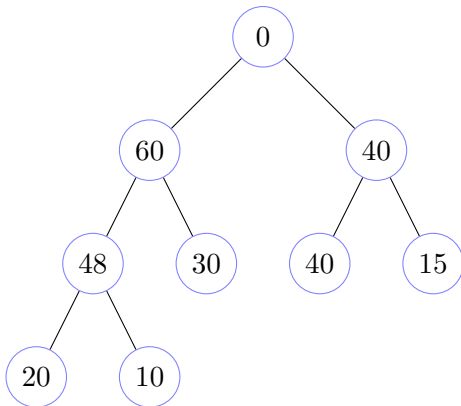
In questo modo, visto che partivamo da un albero quasi heap, il sottoalbero che non è coinvolto nello scambio non viene in alcun modo modificato e pertanto rimane ancora un albero heap.

Potremmo però aver reso il sottoalbero coinvolto nello scambio un albero quasi heap, visto che al più la violazione potrebbe essere in radice.

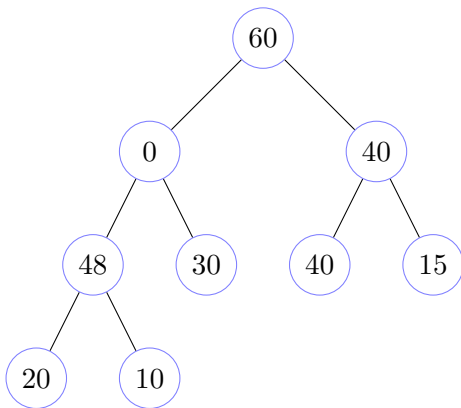
Basterà pertanto ripetere la stessa procedura sul sottoalbero in questione per correggerlo (se serve), ed iterare tale procedura, eventualmente, fino in foglia.

Attenzione. È facile osservare che questa procedura viene eseguita lungo un percorso; il che significa che tale procedura, che permette di correggere un albero quasi heap in un albero heap, impiegherà un tempo al più lineare nell'altezza dell'albero e quindi, poiché l'albero è completo e quindi bilanciato, al più logaritmico nel numero di nodi.

Dato l'albero quasi heap in figura 7.9 correggiamolo in modo da farlo diventare heap.

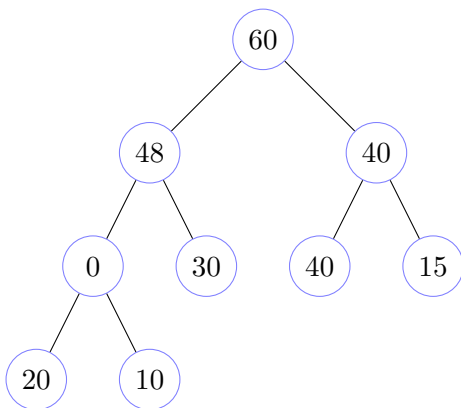


Prendiamo il massimo tra 0, 60 e 40, e sostituiamo lo 0 al posto del nodo che ha valore massimo.
In questo caso è il figlio sinistro che ha il valore massimo (60) e quindi sostituiamo il suo valore con 0.



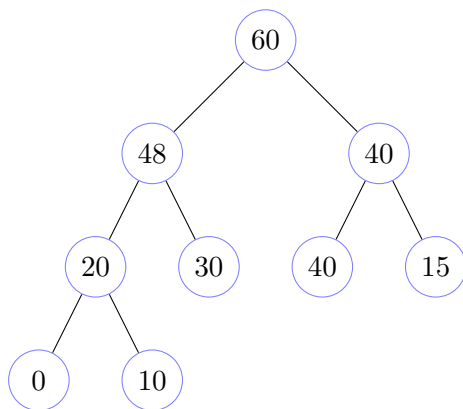
Consideriamo il sottoalbero in cui è avvenuto lo scambio.

Prendiamo il massimo tra 0, 48 e 30, e sostituiamo lo 0 al posto del nodo che ha valore massimo.
In questo caso è il figlio sinistro che ha il valore massimo (48) e quindi sostituiamo il suo valore con 0.



Consideriamo il sottoalbero in cui è avvenuto lo scambio.

Prendiamo il massimo tra 0, 20 e 10, e sostituiamo lo 0 al posto del nodo che ha valore massimo.
In questo caso è il figlio sinistro che ha il valore massimo (20) e quindi sostituiamo il suo valore con 0.



La simulazione di correzione è terminata. L'albero in questione adesso è diventato un max heap

Figura 7.11: Simulazione di correzione di un albero quasi heap in max heap

Quindi, possiamo anticipare, che tale procedura avrà una complessità che nel caso migliore sarà costante, nel caso peggiore logaritmica nella dimensione dell'array; pertanto $\Omega(1)$, $O(\log(n))$.

7.8 Heap sort

Quindi ora possiamo capire intuitivamente i passi da seguire.

Partiremo da un array generico, immaginandolo come un albero mappato come abbiamo precedentemente mostrato.

Chiaramente tale array non avrà nulla a che fare con un albero heap, se non per le foglie.

Un albero avente un solo nodo è banalmente un albero heap. Pertanto le foglie di un albero generico sono alberi heap.

Di conseguenza partiremo dai nodi padre delle foglie e applicheremo la procedura appena descritta per trasformare un albero quasi heap in un albero heap.

A questo punto passeremo al livello precedente, applicheremo la procedura ai padri dei padri delle foglie. E così via fino in radice.

Questo permetterà di prendere un array generico e trasformarlo in uno heap.

Infine, l'operazione di scambio dell'algoritmo di ordinamento andrà come sappiamo a prendere il massimo e a posizionarlo in coda all'array.

Quello che faremo è non considerare più il sottoarray a partire proprio dall'elemento scambiato. In questo modo la parte dell'array che ancora considereremo, visto che si trattava di un albero heap prima dello scambio, potrà essere al massimo un albero quasi heap e quindi semplicemente applicando la procedura descritta andremo a correggerlo se necessario.

Questa è in sintesi la descrizione complessiva del nuovo algoritmo di ordinamento che finalmente possiamo affrontare, il cosiddetto heap sort.

Con un'ulteriore funzione aggiuntiva, che serve ad estendere l'array, avremo tutto ciò che ci serve per creare la coda a priorità che permette di rendere, in caso di grafi non densi, l'algoritmo di Dijkstra più efficiente.

Prima di procedere è necessario effettuare alcune importanti osservazioni.

La rappresentazione vettoriale di un albero è la linearizzazione dell'albero visitato in ampiezza.

Pertanto risulta chiaro che una visita in preorder dell'array corrisponde ad una visita in ampiezza dell'albero.

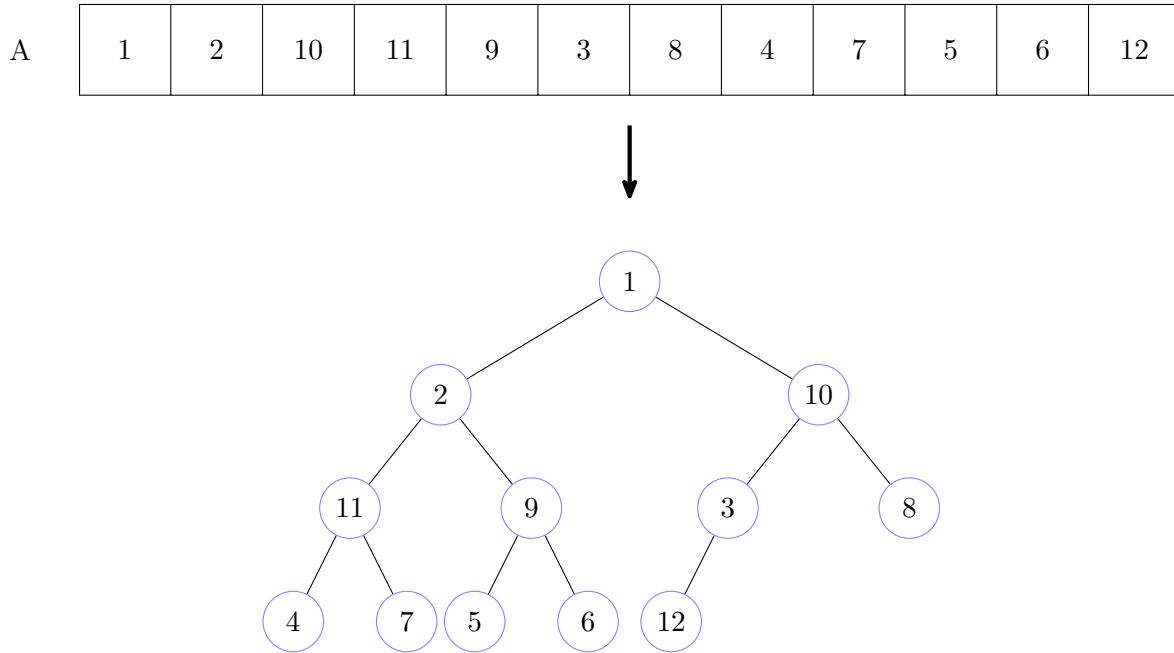


Figura 7.12: Esempio di linearizzazione dell'albero visitato in ampiezza in un array

In un albero completo il numero di foglie è uguale al numero di nodi interni, a parte un possibile scarto di una unità.

Formalmente.

Sia T un albero completo ed indichiamo con $leaf_T$ l'insieme delle foglie dell'albero T , e con $inner_T$ l'insieme dei nodi interni dell'albero T .

Pertanto

$$leaf_T = \{x \in T \mid x.l = x.r = \perp\}$$

$$inner_T = \{x \in T \mid x.l \neq \perp \vee x.r \neq \perp\}$$

Avremo che

$$|leaf_T| - |inner_T| \leq 1$$

Pertanto, visto che come abbiamo descritto, nella fase di rimodellamento iniziale dell'array partiremo dai padri delle foglie, significa che a metà dell'array avremo il primo nodo, a partire dalla fine dell'array, che è padre di foglia.

Se prendiamo l'esempio precedente, notiamo come il primo padre di foglia (3), a partire dalla fine dell'array, si trova a metà dell'array, alla sua destra si trovano solo foglie.

Inoltre, è chiaro che le formule che ci permettono di ottenere a partire da un nodo la posizione dei nodi figli nell'array possano essere invertite e quindi è anche possibile ottenere, partendo da un nodo, la posizione nell'array del nodo padre.

$$\text{Il padre di un nodo di indice } i \text{ è all'indice } \left\lfloor \frac{i-1}{2} \right\rfloor \quad (7.14)$$

Infine, ricordiamo che, come abbiamo detto, la prima fase dell'algoritmo di heap sort, cioè quella di preparazione dell'array, ha una complessità lineare nel numero di nodi; mentre invece mettere a posto l'array a seguito di una operazione di scambio ha un costo logaritmico nel numero di nodi.

Questo potrebbe sembrare poco intuitivo visto che la procedura che utilizziamo per ripristinare la struttura dell'array a seguito di uno scambio è la stessa che durante la prima fase viene applicata ripetutamente per portare un array generico allo stato iniziale corretto.

Anticipiamo, sebbene lo vedremo in dettaglio a breve, che la differenza sostanziale sta nel fatto che quando dobbiamo aggiustare l'albero a seguito di uno scambio partiamo sempre dalla radice e pertanto il costo sarà al più logaritmico come abbiamo già discusso; quando invece siamo nella fase iniziale, effettuiamo tale procedura a partire da tutti i padri di foglie, e così via fino in radice.

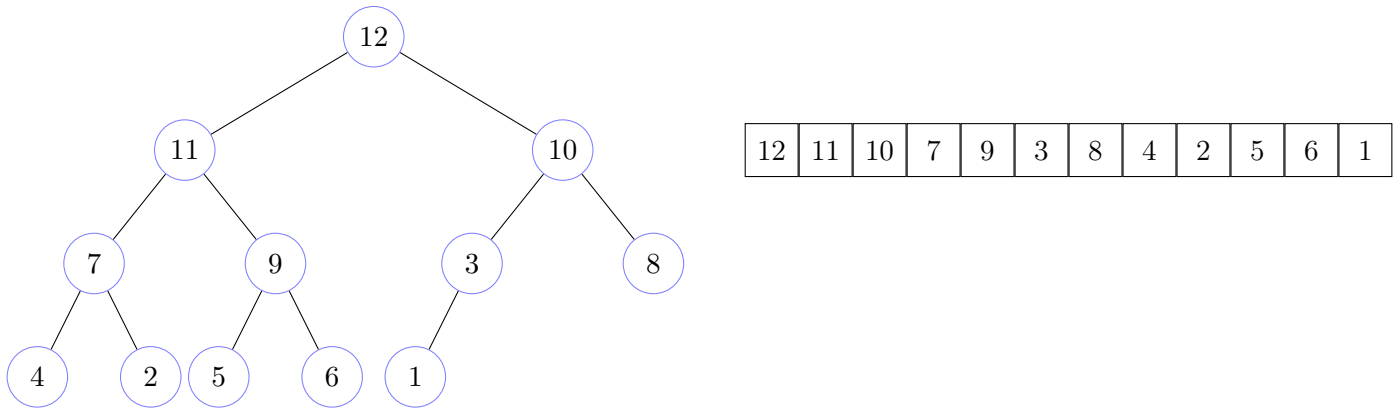
È proprio questa la differenza.

Avanzando in questo modo avremo che gli alberi bassi saranno tanti, visto che come abbiamo detto il numero di foglie è uguale (tralasciando lo scarto di una possibile unità) al numero di nodi interni, mentre quelli alti saranno pochi.

Quindi sebbene la procedura venga richiamata più volte essa verrà eseguita in media su alberi bassi e questo è il fattore principale che porta nel complesso la prima fase ad avere una complessità lineare nel numero di nodi dell'array.

Questa è solo l'idea intuitiva che formalizzeremo a breve.

Prima di passare a vedere l'algoritmo di heap sort facciamo un esempio del suo funzionamento partendo da un array generico.



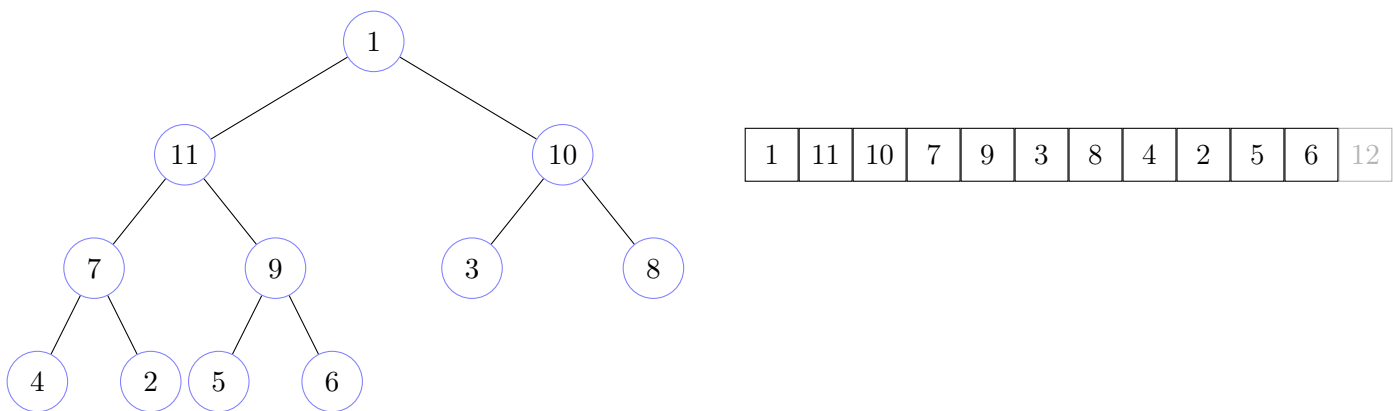
Passo 1: Scambiamo le celle dell'array di indice 0 e indice $n - 1$.

Poi applichiamo una correzione alla struttura dati, che a seguito dello scambio è diventata un quasi heap.

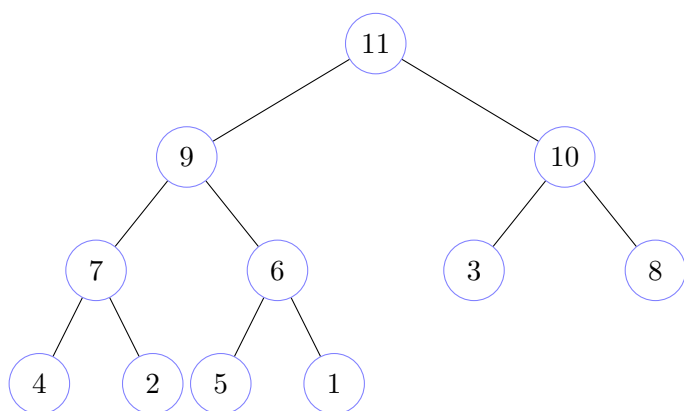
Attenzione. Dopo lo scambio il sottoarray dall'indice $n - 1$ in poi non deve essere più considerato per l'esecuzione dell'algoritmo.

Nel disegno ciò verrà rappresentato da delle celle sbiadite per l'array.

Per quanto riguarda lo heap, nei successivi passi, esso verrà considerato senza quella parte del sottoarray non più valido, per evitare errori di correzione della struttura.



Adesso correggiamo la struttura dati quasi heap, riportandola ad heap

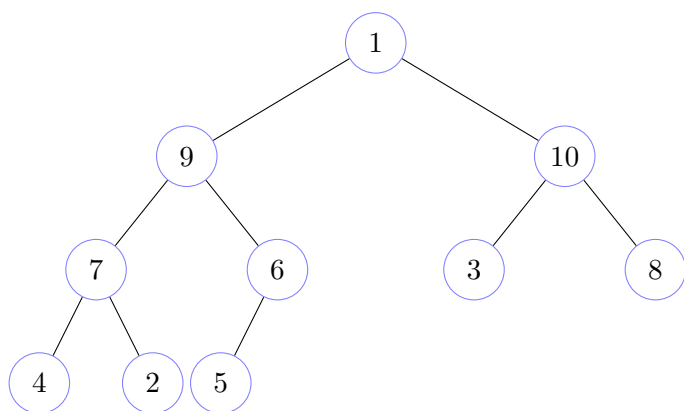


11	9	10	7	6	3	8	4	2	5	1	12
----	---	----	---	---	---	---	---	---	---	---	----

Passo 2: Scambiamo le celle dell'array di indice 0 e indice $n - 2$.

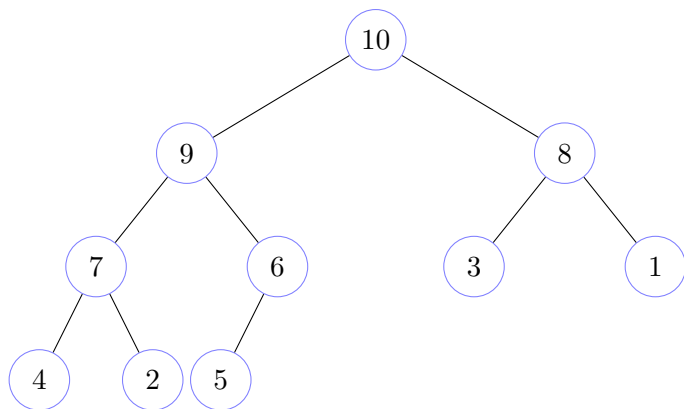
Poi applichiamo una correzione alla struttura dati, che a seguito dello scambio è diventata un quasi heap.

Attenzione. Dopo lo scambio il sottoarray dall'indice $n - 2$ in poi non deve essere più considerato per l'esecuzione dell'algoritmo.



1	9	10	7	6	3	8	4	2	5	11	12
---	---	----	---	---	---	---	---	---	---	----	----

Adesso correggiamo la struttura dati quasi heap, riportandola ad heap

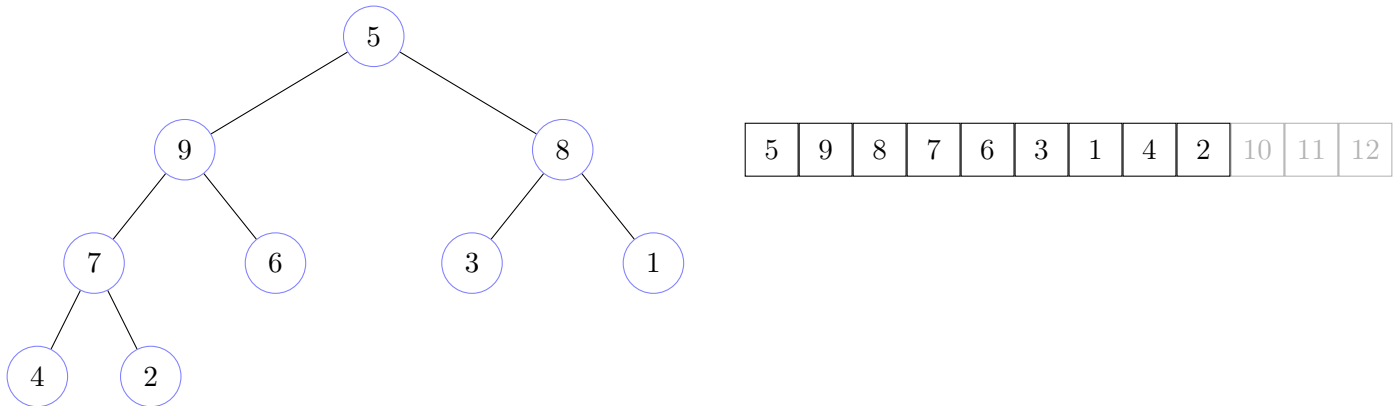


10	9	8	7	6	3	1	4	2	5	11	12
----	---	---	---	---	---	---	---	---	---	----	----

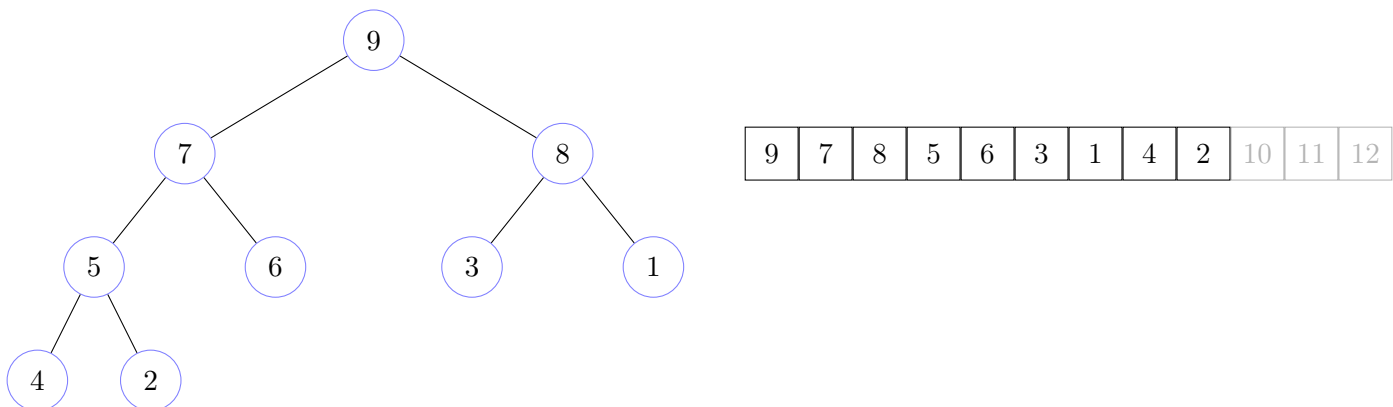
Passo 3: Scambiamo le celle dell'array di indice 0 e indice $n - 3$.

Poi applichiamo una correzione alla struttura dati, che a seguito dello scambio è diventata un quasi heap.

Attenzione. Dopo lo scambio il sottoarray dall'indice $n - 3$ in poi non deve essere più considerato per l'esecuzione dell'algoritmo.



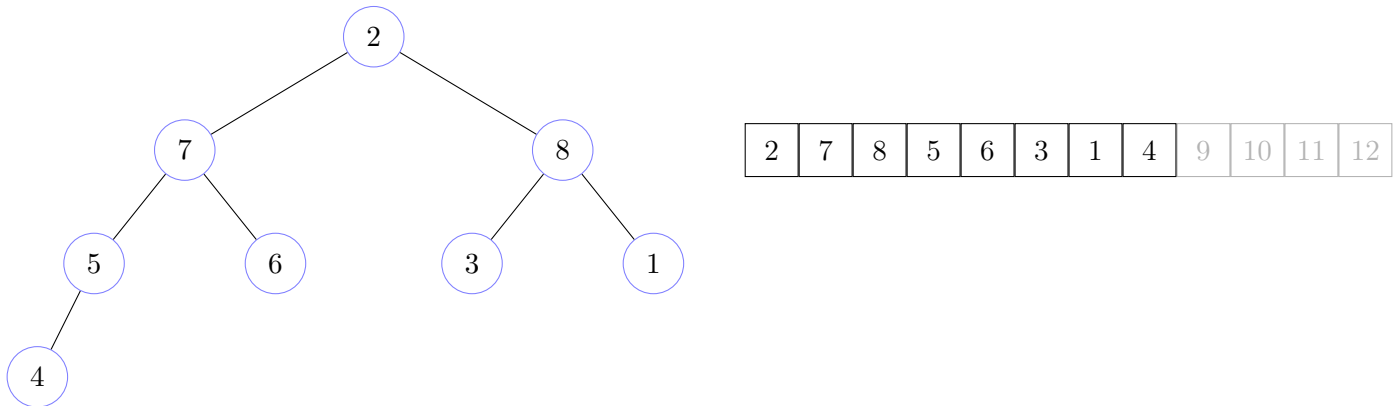
Adesso correggiamo la struttura dati quasi heap, riportandola ad heap



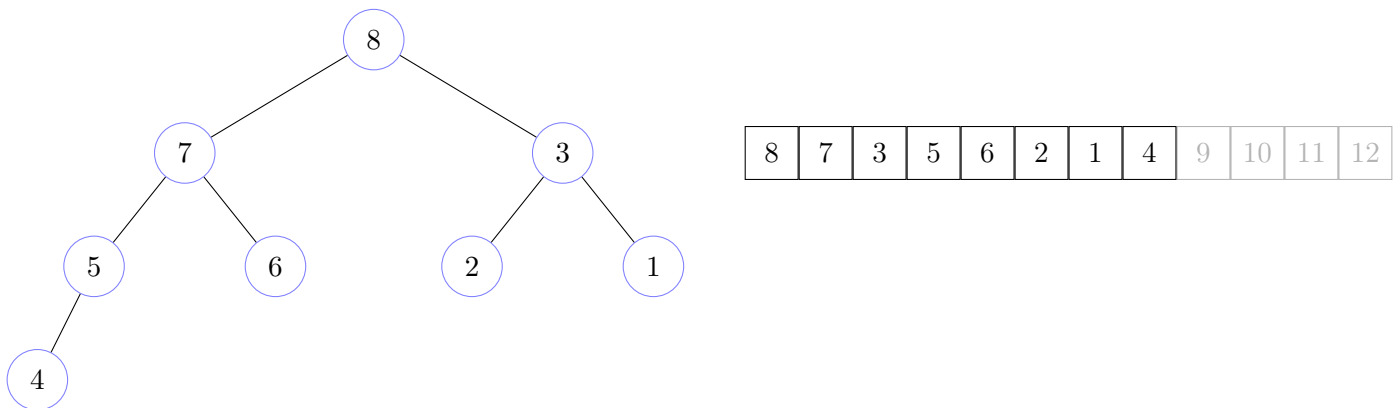
Passo 4: Scambiamo le celle dell'array di indice 0 e indice $n - 4$.

Poi applichiamo una correzione alla struttura dati, che a seguito dello scambio è diventata un quasi heap.

Attenzione. Dopo lo scambio il sottoarray dall'indice $n - 4$ in poi non deve essere più considerato per l'esecuzione dell'algoritmo.



Adesso correggiamo la struttura dati quasi heap, riportandola ad heap



E così via ...

Figura 7.13: Esempio di funzionamento dell'algoritmo heap sort

Facciamo una piccola parentesi doverosa.

L'albero heap non serve solo per l'algoritmo di heap sort, ovviamente.

Ricordiamo, infatti, che avevamo discusso delle code a priorità per l'algoritmo di Dijkstra. Si tratta di alberi heap.

Ovviamente, come sappiamo, nell'algoritmo di Dijkstra lavoriamo aggiornando le minime distanze, quindi serve un min heap e non un max heap.

L'idea intuitiva dell'utilizzo di un min heap nell'algoritmo di Dijkstra è che ad ogni relax effettivamente applicato viene diminuito il peso del nodo e visto che si sta utilizzando un min heap chiaramente tale aggiornamento potrebbe aver compromesso la struttura di albero heap che andrà ripristinata con le medesime correzioni appena viste, dualizzate per il min heap.

7.8.1 Algoritmo: heap sort

Passiamo finalmente all'algoritmo di heap sort. Esso è formato da tre parti.

La funzione `build_heap` che costruisce a partire da un array generico il corrispondente max heap.

Algoritmo 107: Funzione accessoria che costruisce un max heap partendo da un array

Signature `build_heap`: $\text{Array}(D) \times \mathbb{N} \rightarrow \emptyset$

function `build_heap(A, n)`

```

    // visto che le foglie sono già alberi heap
    // possiamo partire dai padri delle foglie
    // quindi a partire dall'indice medio dell'array in input
    // a decrescere
1   for (i from floor(n/2)-1 down to 0) do
        // mediante una funzione accessoria
        // correggo la struttura fino all'ultima posizione dell'array
        // a partire dalla posizione corrente
        // per trasformare l'albero quasi heap in albero heap
2   heapify(A, n, i)
```

La funzione `heapify` corregge eventuali distorsioni della struttura portando un albero quasi heap a raggiungere lo stato di albero heap.

Prende in input ovviamente l'array e due indici di posizione.

Il primo rappresenta il successivo dell'ultima posizione dell'array in input che la funzione andrà a considerare; il secondo è l'indice di posizione dal quale far originare il meccanismo di correzione.

Algoritmo 108: Funzione accessoria che corregge un albero quasi heap in albero heap**Signature** `heapify`: $\text{Array}(D) \times \mathbb{N} \times \mathbb{N} \rightarrow \emptyset$ **function** `heapify(A, n, i)`

```

    // m: indice di stima del massimo di tre nodi di interesse
    // assegno a delle variabili accessorie
    // il valore della posizione di inizio in input
    // e dei rispettivi figli destro e sinistro
    // supponendo che l'indice associato al valore massimo
    // tra i tre sia quello di partenza in input
1   (m, l, r) ← (i, 2*i+1, 2*i+2)
    // effettuiamo il calcolo del massimo
    // se la posizione del figlio sinistro è valida
    // e il valore associato ad essa è maggiore
    // di quella della stima corrente del massimo
2   if ((l < n) AND (A[m] < A[l])) then
        // aggiorna la posizione del massimo
        // a quella del figlio sinistro
        // dell'indice di partenza in input
3       m ← l
    // se la posizione del figlio destro è valida
    // e il valore associato ad essa è maggiore
    // di quella della stima corrente del massimo
4   if ((r < n) AND (A[m] < A[r])) then
        // aggiorna la posizione del massimo
        // a quella del figlio destro
        // dell'indice di partenza in input
5       m ← r
    // se la posizione del massimo è diversa
    // da quella dell'indice di partenza in input
6   if (m ≠ i) then
        // mediante una funzione accessoria
        // scambio i valori degli elementi
        // di posizione pari all'indice di partenza in input
        // e del massimo calcolato
        // per portare quindi il massimo in alto
        // se consideriamo la struttura ad albero
7       swap(A, i, m)
        // richiamo la funzione ricorsivamente
        // sullo stesso array
        // fino alla stessa posizione finale
        // cambiando la posizione di inizio a quella del massimo calcolata
        // per continuare a correggere nel sottoalbero coinvolto nello scambio
8       heapify(A, n, m)

```

Algoritmo 109: Algoritmo di heap sort

Signature `heap_sort`: $Array(D) \times \mathbb{N} \rightarrow \emptyset$ **function** `heap_sort`(`A`, `n`)

```

    // mediante una funzione accessoria
    // costruisco il max heap partendo dall'array generico in input
1  build_heap(A, n)
    // per ogni posizione dello heap in senso decrescente
    // Nota. Possiamo fermarci all'indice 1
    // perché una volta arrivati a quel punto per il funzionamento
    // dell'algoritmo avremo già ordinato l'array
2  for (i from n-1 down to 1) do
    // mediante una funzione accessoria
    // scambio l'elemento di posizione corrente
    // con il massimo del sottoarray fino alla posizione corrente
3  swap(A, 0, i)
    // mediante una funzione accessoria
    // correggo la struttura
    // che ora potrebbe essere diventata un albero quasi heap
    // fino alla posizione corrente
    // a partire dalla prima posizione dell'array in input
    // per farla tornare ad essere uno heap
4  heapify(A, i, 0)

```

Possiamo notare, come già anticipato, una differenza sostanziale tra le chiamate della funzione `heapify` in `heap_sort` e in `build_heap`.

Quando siamo in costruzione, quindi nella funzione `build_heap`, il secondo parametro della funzione `heapify`, che identifica il successivo dell'ultimo indice dell'array che stiamo considerando, non viene mai modificato ed è sempre n . Quello che viene modificato è il primo indice dell'array che consideriamo, infatti partiamo dal primo padre di foglia e andiamo a ritroso, fino ad arrivare in radice.

Ecco il motivo per il quale la funzione `build_heap` lavora prevalentemente con alberi bassi.

Quando invece chiamiamo la funzione `heapify` nella funzione `heap_sort` partiamo sempre dalla radice, ma andiamo a considerare un array che man mano si accorcia; infatti il secondo parametro della funzione `heapify` nella chiamata effettuata dalla funzione `heap_sort` segue la posizione del ciclo `for` che percorre le posizioni dell'array in senso decrescente.

Sottolineiamo che la funzione `heapify` riesce a ripristinare la struttura di albero heap solo e soltanto se viene chiamata su un albero quasi heap, dove quindi il problema è al massimo in radice.

Vediamo, adesso, la complessità dell'algoritmo di heap sort.

7.8.2 Complessità dell'algoritmo heap sort

Prendiamo in considerazione la funzione `build_heap`.

Ricordiamo infatti che abbiamo già discusso la complessità della procedura di correzione che porta un albero quasi heap a raggiungere lo stato di albero heap, che è proprio l'insieme di operazioni che vengono effettuate nella funzione `heapify`.

Abbiamo quindi che la funzione `heapify` ha una complessità pari a $\Omega(1)$ e $O(\log(n))$, dove n è il numero di nodi dell'albero sulla quale viene applicata.

Cioè nel caso peggiore ha una complessità lineare nell'altezza dell'albero considerato.

Abbiamo detto che in un albero completo, avente n nodi e altezza h , il numero delle foglie è uguale al numero di nodi interni, o addirittura possono essere maggiori di una unità.

Consideriamo, visto che a livello asintotico non cambia, il caso in cui in un albero completo con n nodi il numero delle foglie è uguale al numero di nodi interni.

$$\#_{foglie} = \frac{n}{2}$$

Conseguentemente i padri delle foglie saranno la metà delle foglie, in quanto per le proprietà degli alberi completi sappiamo che ogni nodo interno, tranne al più uno, deve avere entrambi i figli.

$$\#_{padri-foglie} = \frac{\#_{foglie}}{2} = \frac{n}{4}$$

Quindi, segue logicamente, che i nodi padri dei nodi padri delle foglie saranno a loro volta la metà dei nodi padri di foglie.

$$\#_{padri-padri-foglie} = \frac{\#_{padri-foglie}}{2} = \frac{\frac{\#_{foglie}}{2}}{2} = \frac{\frac{n}{4}}{2} = \frac{n}{8}$$

E così via.

Vediamo adesso una figura che sintetizza ciò che abbiamo appena enunciato.

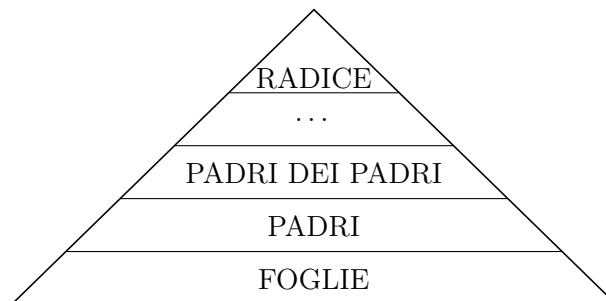


Figura 7.14: Categorizzazione del numero di foglie negli alberi completi

Risulta inoltre chiaro che il costo della funzione **heapify** per i nodi di un certo livello dell'albero è, nel caso peggiore, lo stesso. Visto che dipende dalla profondità del percorso considerato.

Pertanto possiamo facilmente calcolare il contributo alla complessità di un intero livello dell'albero andando semplicemente a moltiplicare il costo nel caso peggiore di una chiamata della funzione **heapify** di un nodo di un certo livello dell'albero per il numero di nodi presenti nello stesso livello.

Segue quindi immediatamente il calcolo della complessità della funzione di **build_heap** in quanto basta considerare la somma delle complessità dei vari livelli dell'albero, prendendo in considerazione l'altezza degli alberi sui quali viene applicata la funzione di **heapify**.

Quando la funzione verrà chiamata sui padri di foglia l'altezza del sottoalbero sarà quindi di 1, e man mano salendo i livelli dell'albero a ritroso andrà ad aumentare.

Quindi avremo che la complessità della funzione **build_heap** è:

$$T_{BH} = \sum_{i=1}^h \frac{n}{2^{i+1}} \cdot i \quad (7.15)$$

Andiamo a sviluppare tale sommatoria.

Alcuni passaggi andrebbero giustificati meglio, noi li daremo per scontati.

Per altro, visto che ci interessa la complessità asintotica, andremo a maggiorare in quanto a livello asintotico non farà differenza.

$$\sum_{i=1}^h \frac{n}{2^{i+1}} \cdot i = \frac{n}{4} \cdot \sum_{i=1}^h \left(\frac{1}{2^{i-1}} \cdot i \right) = \frac{n}{4} \cdot \sum_{i=1}^h i \cdot \left(\frac{1}{2} \right)^{i-1} = \frac{n}{4} \cdot \sum_{i=1}^h [ix^{i-1}]_{x=\frac{1}{2}} = \frac{n}{4} \cdot \left[\sum_{i=1}^h ix^{i-1} \right]_{x=\frac{1}{2}}$$

Notiamo che

$$ix^{i-1} = \frac{d}{dx} x^i$$

Quindi

$$\frac{n}{4} \cdot \left[\sum_{i=1}^h \frac{d}{dx} x^i \right]_{x=\frac{1}{2}} = \frac{n}{4} \cdot \left[\frac{d}{dx} \sum_{i=1}^h x^i \right]_{x=\frac{1}{2}}$$

Visto che ci interessa un limite superiore possiamo maggiorare

$$\frac{n}{4} \cdot \left[\frac{d}{dx} \sum_{i=1}^h x^i \right]_{x=\frac{1}{2}} \leq \frac{n}{4} \cdot \left[\frac{d}{dx} \sum_{i=0}^{\infty} x^i \right]_{x=\frac{1}{2}}$$

Possiamo applicare la formula semplificata della serie geometrica con ragione < 1 .

Attenzione. Tale formula si applica solo con una ragione < 1 .

$$\frac{n}{4} \cdot \left[\frac{d}{dx} \sum_{i=0}^{\infty} x^i \right]_{x=\frac{1}{2}} = \frac{n}{4} \cdot \left[\frac{d}{dx} \left(\frac{1}{1-x} \right) \right]_{x=\frac{1}{2}} = \frac{n}{4} \cdot \left[\frac{1}{(1-x)^2} \right]_{x=\frac{1}{2}} = \frac{n}{4} \cdot \frac{1}{\left(1 - \frac{1}{2}\right)^2} = n$$

Quindi abbiamo dimostrato che l'algoritmo di **build_heap** ha una complessità lineare nella dimensione dell'array (o nel numero di nodi dell'albero completo), pertanto è $\Theta(n)$.

Questa conclusione potrebbe sembrare sorprendente.

L'algoritmo di **build_heap**, a prima vista, potrebbe sembrare avere una complessità di $n \cdot \log(n)$, in quanto c'è un ciclo **for** eseguito $\frac{n}{2}$ volte all'interno del quale c'è un'operazione che al più è logaritmica.

Questa analisi è stata quindi particolarmente importante, non solo per capire la vera complessità della funzione di **build_heap**, ma anche per capire che la fase di preprocessamento dell'array che viene effettuata prima dell'effettivo ordinamento non costi già quanto l'algoritmo di ordinamento stesso; in quanto se così fosse tale algoritmo nel complesso non sarebbe un buon algoritmo in quanto sarebbero preferibili altri algoritmi con medesimo costo che però non hanno una fase di preprocessamento così pesante.

Fortunatamente abbiamo mostrato che non è questo il caso.

Quindi possiamo concludere che il vero costo dell'algoritmo di heap sort non è nella fase di preprocessamento, che per altro avendo costo lineare in una struttura lineare ha il minimo costo possibile.

Pertanto per calcolare la complessità dell'algoritmo di heap sort possiamo quindi effettuare una banale analisi. Nell'algoritmo della funzione **heap_sort** c'è un ciclo **for** che viene eseguito per n volte in cui c'è una chiamata della funzione **heapify**, che come abbiamo già detto è, nel caso peggiore, lineare nell'altezza dell'albero sulla quale viene chiamata.

È facile osservare che ogni volta che la funzione di **heapify** verrà chiamata partendo dalla radice fino alle foglie avrà un costo logaritmico nell'altezza dell'intero albero; e come abbiamo detto in un albero completo il numero di foglie è (eccezion fatta per il possibile scarto di una unità) pari alla metà del numero di nodi totale.

Quindi solo considerare le chiamate della funzione di **heapify** sulle foglie comporta una complessità che nel caso peggiore è $(\frac{n}{2}) \cdot \log(n)$, dove n è il numero di nodi dell'albero completo, o equivalentemente la dimensione dell'array. Di conseguenza da tale osservazione segue che l'algoritmo sicuramente è $\Omega(n \cdot \log(n))$.

È anche chiaro, però, che eseguendo tutto il ciclo **for**, mettendoci nel caso peggiore, avremo una complessità che è proprio $n \cdot \log(n)$; dal quale segue che l'algoritmo è anche $O(n \cdot \log(n))$.

Quindi, possiamo concludere, che in generale l'algoritmo di heap sort ha una complessità pari a $\Theta(n \cdot \log(n))$.

7.9 Heap nell'algoritmo di Dijkstra

Mostriamo adesso in che modo la struttura di albero heap viene utilizzata nell'algoritmo di Dijkstra.

Vediamo un esempio che mostra come viene cambiata la priorità in un albero heap.

Nell'algoritmo di Dijkstra, ovviamente, viene utilizzato un min heap in quanto, come sappiamo, l'aggiornamento dei pesi viene effettuato solo in senso migliorativo.

Pertanto non sarà possibile aggiornare un valore di un nodo con un valore maggiore ma solo con un valore strettamente minore.

Tale osservazione risulta fondamentale in quanto assicura un'unica direzionalità dell'algoritmo, in quanto in un min heap cambiare il valore di un nodo con un valore strettamente minore non potrà sicuramente aver generato un problema tra il suddetto nodo e i figli, ma solo tra il nodo e il padre.

Ci basterà, in caso di problema, scambiare il nodo modificato col padre per risolverlo.

Ovviamente in questo modo il problema non è detto che venga risolto ma potrebbe solo essere stato spostato verso l'alto. Ci basterà ripetere in sequenza, se necessario gli scambi fino ad arrivare in radice nel caso peggiore.

In realtà si può fare una funzione di cambio di priorità sia a crescere che a decrescere; ma per i nostri scopi non è rilevante.

Vediamo adesso un esempio di cambio di priorità in un min heap e la sua correzione.

Dato il seguente albero min heap, cambiamo il valore del nodo che aveva valore 15, con il valore 0.

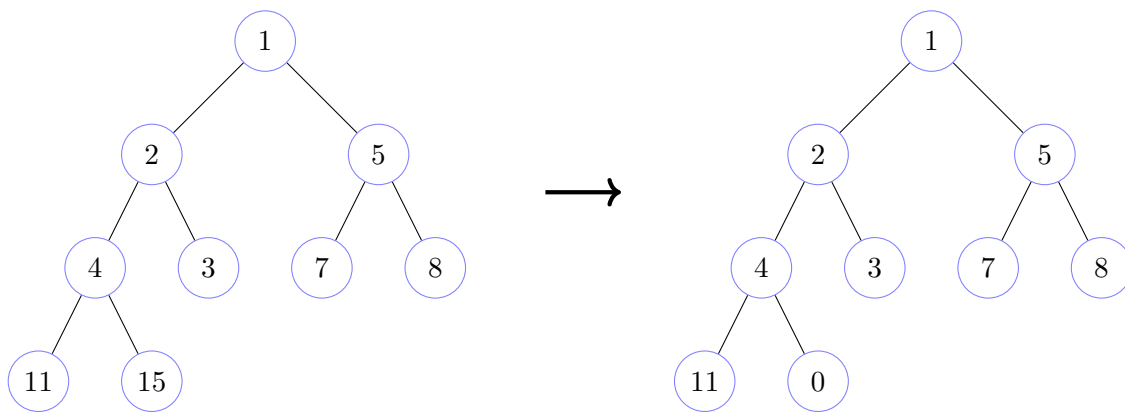
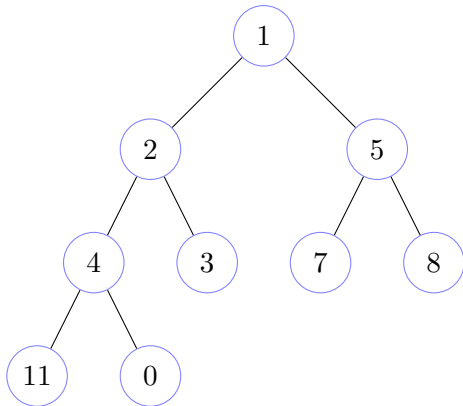
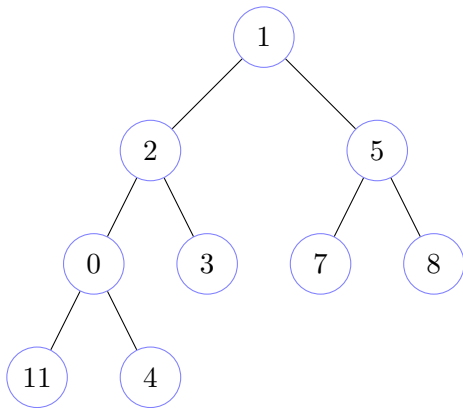


Figura 7.15: Esempio di cambio di priorità in un albero min heap

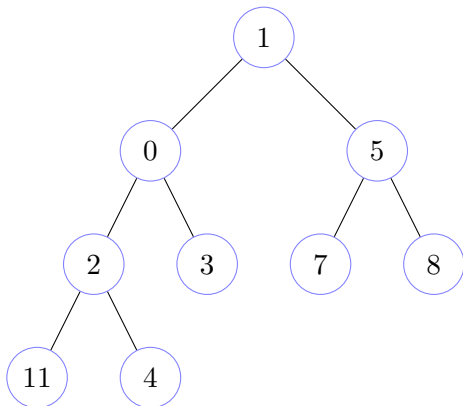
Correggiamo adesso l'albero quasi heap riportandolo in un min heap.



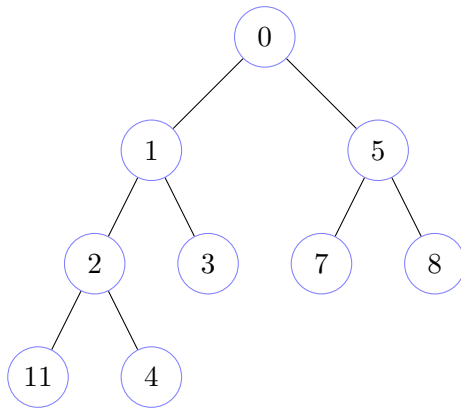
Guardiamo il padre del nodo che è stato aggiornato e se il valore del nodo è minore di quello del padre, scambiamo i due nodi. Nel nostro caso il nodo che è stato aggiornato ha valore 0, e il nodo padre ha valore 4, dobbiamo quindi poiché $0 < 4$ scambiare i due nodi.



Guardiamo il padre del nodo che è stato aggiornato e se il valore del nodo è minore di quello del padre, scambiamo i due nodi. Nel nostro caso il nodo che è stato aggiornato ha valore 0, e il nodo padre ha valore 2, dobbiamo quindi poiché $0 < 2$ scambiare i due nodi.



Guardiamo il padre del nodo che è stato aggiornato e se il valore del nodo è minore di quello del padre, scambiamo i due nodi. Nel nostro caso il nodo che è stato aggiornato ha valore 0, e il nodo padre ha valore 1, dobbiamo quindi poiché $0 < 1$ scambiare i due nodi.



La simulazione di correzione è terminata poiché l'ultima sostituzione è stata una sostituzione in radice. L'albero in questione adesso è diventato un min heap

Figura 7.16: Simulazione di correzione di un albero quasi heap in min heap

Ecco l'algoritmo che effettua il cambio di priorità nell'algoritmo di Dijkstra. Tale funzione dovrà essere chiamata a seguito di un avvenuto rilassamento.

Algoritmo 110: Funzione accessoria per l'algoritmo di Dijkstra utilizzando uno heap

Signature `change_priority`: $Array(D) \times \mathbb{N} \rightarrow \emptyset$

```

function change_priority(A, i)
    // fino a quando non sono arrivato in radice
    // e il nodo corrente è minore del padre
1  while (i > 0 AND A[i] < A[floor((i-1)/2)]) do
        // mediante una funzione accessoria
        // scambio il nodo corrente
        // con il nodo padre
        // portando il valore minore in alto
        // visto che lavoro su un min heap
2      swap(A, i, floor((i-1)/2))
3  i ← floor((i-1)/2

```

Risulta immediato che tale funzione può risalire, nel caso peggiore, l'albero da un nodo fino in radice lungo un percorso e pertanto ha una complessità pari a $\Omega(1)$ e $O(h)$, dove h è l'altezza dell'albero considerato, quindi visto che stiamo considerando alberi completi, avremo $\Omega(1)$ e $O(\log(n))$, dove n è il numero di nodi dell'albero, o equivalentemente la dimensione dell'array.

7.10 Merge sort

Passiamo, adesso, ad un altro algoritmo di ordinamento. Il merge sort.

Il merge sort è il primo algoritmo che vediamo che risolve il problema dell'ordinamento in modo naturale dal punto di vista strutturale ricorsivamente.

7.10.1 Algoritmo: merge sort

Abbiamo una semplice funzione di interfaccia che preso un array chiama la parte ricorsiva dell'algoritmo con dei parametri iniziali che denotano quale porzione dell'array andiamo a considerare.

Algoritmo 111: Algoritmo di merge sort: funzione di interfaccia

Signature merge_sort: $Array(D) \times \mathbb{N} \rightarrow \emptyset$

```
function merge_sort(A, n)
    // chiamo la funzione ricorsiva sull'array in input
    // dalla prima all'ultima posizione valida dell'array
1  merge_sort(A, 0, n-1)
```

Ecco la parte ricorsiva prende in input un array e due indici che denotano l'inizio e la fine del sottoarray che andiamo effettivamente a considerare.

Algoritmo 112: Algoritmo di merge sort: parte ricorsiva

Signature merge_sort: $Array(D) \times \mathbb{N} \times \mathbb{N} \rightarrow \emptyset$

```
function merge_sort(A, s, e)
    // se l'indice di inizio dell' sottoarray considerato
    // è minore dell'indice finale
    // significa che il sottoarray considerato è formato
    // da almeno 2 elementi e pertanto potrebbe essere necessario ordinarlo
1  if (s < e) then
    // calcolo la posizione dell'elemento mediano
    // del sottoarray considerato
2  m ← floor((s+e)/2)
    // richiamo la funzione ricorsivamente sulla prima metà
    // del sottoarray considerato
3  merge_sort(A, s, m)
    // richiamo la funzione ricorsivamente sulla seconda metà
    // del sottoarray considerato
4  merge_sort(A, m+1, e)
    // mediante una funzione accessoria
    // fondo in modo ordinato le due parti ordinate dell'array considerato
    // ordinando il sottoarray considerato
5  merge(A, s, m, e)
```

Infine, la funzione di `merge`, che prese due sottoparti ordinate di un array le fonde in modo pseudosincrono e ordinato.

Algoritmo 113: Algoritmo di merge sort: funzione merge**Signature merge:** $\text{Array}(D) \times \mathbb{N} \times \mathbb{N} \times \mathbb{N} \rightarrow \emptyset$ **function merge(A, s, m, e)**

```

    // faccio una copia delle due parti dell'array in input
    // in base agli indici in input
1  (L, R) ← (copy(A, s, m), copy(A, m+1, e))
    // inizializzo a 0 due variabili
    // che serviranno a scorrere le due parti dell'array considerato
    // l'indice i serve a scorrere la prima sottoparte dell'array considerato
    // l'indice j serve a scorrere la seconda sottoparte dell'array considerato
2  (i, j) ← (0, 0)

    // scorro l'array considerato
3  for (k from s to e) do
    // per ogni indice dell'array
    // l'indice i corrente è valido
    // e l'indice j corrente non è valido oppure
    // se l'indice j corrente è valido ma
    // il valore in posizione i della copia della prima parte dell'array in input
    // è minore o uguale del valore in posizione j della copia
    // della seconda parte dell'array in input
4  if ((i ≤ m-s) AND ((j ≥ e-m) OR (L[i] ≤ R[j]))) then
    // pongo al valore dell'indice corrente dell'array considerato
    // il valore dell'elemento corrente della copia della prima parte
    // dell'array in input
5    A[k] ← L[i]
    // incremento l'indice che scorre la copia della prima parte
    // dell'array in input
6    i ← i+1
7  else
    // se invece l'indice i corrente non è valido
    // o l'indice j corrente è valido e
    // se l'indice i corrente è valido ma
    // il valore in posizione i della copia della prima parte dell'array in input
    // è maggiore del valore in posizione j della copia
    // della seconda parte dell'array in input
    // pongo al valore dell'indice corrente dell'array considerato
    // il valore dell'elemento corrente della copia della seconda
    // parte dell'array in input
8    A[k] ← R[j]
    // incremento l'indice che scorre la copia della seconda parte
    // dell'array in input
9    j ← j+1

```

7.10.2 Correttezza dell'algoritmo merge sort

Prima di passare al calcolo della complessità dimostriamo la correttezza dell'algoritmo di merge sort.

Teorema 7.10.1 (Dimostrazione di correttezza del merge sort).

Dimostrazione.

Dimostriamo la correttezza dell'algoritmo di merge sort per induzione sulla dimensione dell'array, $n = e - s + 1$.

I casi base, $n = 0 \vee n = 1 \iff s = e - 1 \vee s = e$, sono banali. L'algoritmo non fa nulla e la correttezza risulta vera vacuamente.

Passiamo al caso induttivo, considerando l'algoritmo corretto fino ad un certo generico ma fissato n , e consideriamo il caso per $n + 1$.

Applichiamo l'algoritmo. Poniamoci nel caso in cui $s < e \iff n \geq 2$, altrimenti risulta banalmente verificato.

L'algoritmo calcola l'elemento mediano e richiama ricorsivamente la funzione `merge_sort` sugli intervalli $[s, m]$ e $[m + 1, e]$.

Risulta immediato, che per il calcolo dell'elemento mediano m , $|[s, m]| < n + 1$ e $|[m + 1, e]| < n + 1$; pertanto vale l'ipotesi induttiva.

A questo punto la funzione `merge`, che possiamo assumere ovviamente corretta (risulta immediato che se alla funzione `merge` viene passato in input un array in cui la sottoparte da s a m è ordinata e la sottoparte da $m + 1$ a e è ordinata, alla fine dell'algoritmo avremo un array ordinato dalla posizione s alla posizione e), non fa altro che prendere le due sottoparti dell'array ordinate e fonderle in modo ordinato.

Pertanto la tesi.

□

7.10.3 Complessità dell'algoritmo merge sort

Passiamo adesso ad analizzare la complessità dell'algoritmo di merge sort.

Vedremo, che calcolare la complessità di questo algoritmo è tutt'altro che banale in quanto nella parte ricorsiva dell'algoritmo non ci sono cicli.

Vedremo che sarà necessario risolvere un'equazione di ricorrenza per ottenere la complessità dell'algoritmo.

Risulta immediato il costo della funzione `merge`.

Ci sono due copie delle due sottoparti del sottoarray considerato che hanno costo lineare nella dimensione della parte di array da copiare.

Il ciclo `for` viene eseguito per tutta la lunghezza del sottoarray considerato e al suo interno abbiamo operazioni a tempo costante.

Pertanto risulta chiaro che ha una complessità che è lineare nella dimensione dell'array in input. Quindi $\Theta(n)$, con n la dimensione dell'array in input.

Banale è chiaramente la complessità della funzione di interfaccia in quanto chiama semplicemente la funzione ricorsiva. Si tratta chiaramente di un'istruzione a tempo costante; quindi avremo una complessità di $\Theta(1)$.

Concentriamoci, quindi, sulla funzione `merge_sort` ricorsiva.

In maniera simile a come fatto nel calcolo della complessità dell'algoritmo di ricerca binaria in un array ordinato consideriamo l'array in input e la dimensione dell'array, che possiamo facilmente ottenere mediante la differenza tra il valore dell'indice finale e quello iniziale del sottoarray considerato.

Possiamo, infatti, notare che l'algoritmo non dipende dagli specifici indici di posizione che riceve in input, ma solo dalla dimensione dell'array considerato.

Ecco il motivo per il quale la nostra analisi può essere indipendente dai valori specifici degli indici del sottoarray considerato e considerare solo la dimensione del sottoarray considerato.

Risulta chiaro che quando la funzione `merge_sort` ricorsiva viene chiamata su un array di dimensione minore o uguale di 1 non farà nulla se non il controllo iniziale, e avremo pertanto un tempo costante.

Quando invece viene chiamata su un array di dimensione almeno 2, come è facile notare dal codice, avremo il controllo iniziale, due chiamate ricorsive su una istanza dell'array avente dimensione pari alla metà di quella in input e la chiamata della funzione `merge`, che abbiamo appena visto essere lineare nella dimensione dell'array sul quale viene chiamata.

Sottolineiamo che la dimensione delle due chiamate ricorsive non hanno sempre la stessa dimensione, che per altro non è sempre esattamente la metà della dimensione della chiamata precedente.

È, però, ormai scontato che per la nostra analisi asintotica tutto ciò non sia in alcun modo rilevante.

Formalmente possiamo riassumere il ragionamento appena fatto in una equazione di ricorrenza.

$$T_{MS}(A, n) = \begin{cases} \Theta(1) & , \text{ se } n \leq 1 \\ 2 \cdot T_{MS}(A, \frac{n}{2}) + \Theta(n) & , \text{ altrimenti} \end{cases} \quad (7.16)$$

Ricordiamo che anche quando siamo andati a calcolare la complessità della versione ricorsiva dell'algoritmo di insertion sort abbiamo dovuto risolvere una equazione di ricorrenza. Tuttavia, ci sono importanti differenze. Ora abbiamo che la dimensione delle chiamate ricorsive è sempre la metà dell'input originale e inoltre dobbiamo farne due.

Nel caso dell'insertion sort abbiamo visto una tecnica molto semplice che consiste nell'andare a svolgere in modo simbolico la ricorrenza per un numero di volte pari a n fino al raggiungimento del caso base, per poi sommare a destra e a manca; dopodiché effettuiamo le dovute semplificazioni e quello che otteniamo è esattamente il costo dell'algoritmo.

Ora, per risolvere questa equazione di ricorrenza cercheremo di fare lo stesso, ma purtroppo il coefficiente moltiplicativo delle chiamate ricorsive non ci permette di applicare esattamente lo stesso meccanismo perché non possiamo semplicemente sommare a destra e a manca in quanto non avremo alcuna semplificazione.

Vediamo quindi qual è l'idea per risolvere questo tipo di equazioni di ricorrenza.

In prima battuta, assumendo di non essere nel caso base, andiamo a scrivere un'espressione del caso ricorsivo.

$$T_{MS}(A, n) = 2 \cdot T_{MS}\left(A, \frac{n}{2}\right) + n \quad (7.17)$$

In modo simile a come abbiamo fatto nel caso dell'insertion sort continuiamo fino al raggiungimento del caso base, facendo attenzione che questa volta dobbiamo andare a dimezzare man mano la dimensione del problema.

$$\begin{aligned} T_{MS}(A, n) &= 2 \cdot T_{MS}\left(A, \frac{n}{2}\right) + n \\ T_{MS}\left(A, \frac{n}{2}\right) &= 2 \cdot T_{MS}\left(A, \frac{n}{4}\right) + \frac{n}{2} \\ T_{MS}\left(A, \frac{n}{4}\right) &= 2 \cdot T_{MS}\left(A, \frac{n}{8}\right) + \frac{n}{4} \\ &\vdots \\ &\vdots \\ T_{MS}(A, 2) &= 2 \cdot T_{MS}(A, 1) + 2 \\ T_{MS}(A, 1) &= 1 \end{aligned}$$

L'idea che abbiamo applicato nel caso dell'equazione di ricorrenza dell'algoritmo dell'insertion sort ricorsivo era di sommare a destra e a sinistra per poi andare ad effettuare le necessarie semplificazioni.

Ora, però, a causa del fattore moltiplicativo che troviamo nei vari termini dell'equazione tale approccio non può funzionare.

Quindi cosa possiamo fare?

Se andassimo a moltiplicare ciascuna equazione per un termine che ci permette di ricondurci al caso semplice dell'equazione di ricorrenza dell'algoritmo di insertion sort potremmo, però, applicare lo stesso approccio.

$$\begin{aligned}
 1 \cdot T_{MS}(A, n) &= 2 \cdot T_{MS}\left(A, \frac{n}{2}\right) + n \\
 2 \cdot T_{MS}\left(A, \frac{n}{2}\right) &= 4 \cdot T_{MS}\left(A, \frac{n}{4}\right) + n \\
 4 \cdot T_{MS}\left(A, \frac{n}{4}\right) &= 8 \cdot T_{MS}\left(A, \frac{n}{8}\right) + n \\
 &\vdots \\
 &\vdots \\
 T_{MS}(A, 1) &= 1
 \end{aligned}$$

Andando adesso a sommare a destra e a manca.

$$\sum_{i=0}^h 2^i \cdot T_{MS}\left(A, \frac{n}{2^i}\right) = \sum_{i=1}^h 2^i \cdot T_{MS}\left(A, \frac{n}{2^i}\right) + h \cdot n + 1$$

Apportiamo le dovute semplificazioni.

$$T_{MS}(A, n) = \sum_{i=0}^h 2^i \cdot T_{MS}\left(A, \frac{n}{2^i}\right) - \sum_{i=1}^h 2^i \cdot T_{MS}\left(A, \frac{n}{2^i}\right) = 2^0 \cdot T_{MS}\left(A, \frac{n}{2^0}\right) = h \cdot n + 1$$

Dunque $T_{MS}(A, n) = h \cdot n + 1$.

Dobbiamo però trovare il valore del termine h , che indica quante volte dobbiamo effettuare la somma prima di arrivare al caso base.

Ma è semplice ottenere il valore di h . Sappiamo che n verrà diviso per 2, per 4, e così via, fino a quando tale divisione non darà come risultato un valore minore o uguale di 1. In quel momento saremo arrivati al caso base.

Formalmente.

$$\frac{n}{2^h} \leq 1 \iff n \leq 2^h \implies h = \log_2(n)$$

Sostituendo otteniamo finalmente il valore della complessità cercato.

Dunque $T_{MS}(A, n) = \log_2(n) \cdot n + 1 = \Theta(n \cdot \log(n))$

Abbiamo quindi mostrato che la complessità dell'algoritmo di merge sort è proprio $\Theta(n \cdot \log(n))$.

Sottolineiamo che tale algoritmo necessita però anche spazio aggiuntivo per effettuare le varie copie di sottoparti dell'array in input. Pertanto ha anche una complessità spaziale che nel complesso è pari alla dimensione dell'array in input.

7.11 Equazioni di ricorrenza

Andiamo adesso ad approfondire la trattazione delle varie tecniche di calcolo che ci permettono di risolvere le equazioni di ricorrenza.

Vediamo in primo luogo una tecnica, basata sulla stessa idea degli approcci precedenti ma, che ha un impatto grafico più immediato e che quindi potrebbe risultare più semplice; sebbene dietro le quinte farà esattamente la stessa cosa degli approcci precedenti.

In particolare la struttura grafica che andremo a vedere rispetta esattamente quello che viene detto l'albero di ricorsione dell'equazione di ricorrenza.

Tale tecnica può essere applicata nel caso di equazioni di ricorrenza semplici; nel caso di equazioni di ricorrenza particolarmente complesse sono preferibili altri approcci.

Graficamente andiamo a rappresentare l'albero di ricorsione dell'algoritmo.

Avremo un nodo per ogni chiamata ricorsiva in cui andremo ad indicare la dimensione dell'input della rispettiva chiamata.

7.11.1 Esempio 1: equazione di ricorrenza dell'algoritmo merge sort

Come esempio pratico applichiamo tale tecnica per risolvere l'equazione di ricorrenza dell'algoritmo di merge sort.

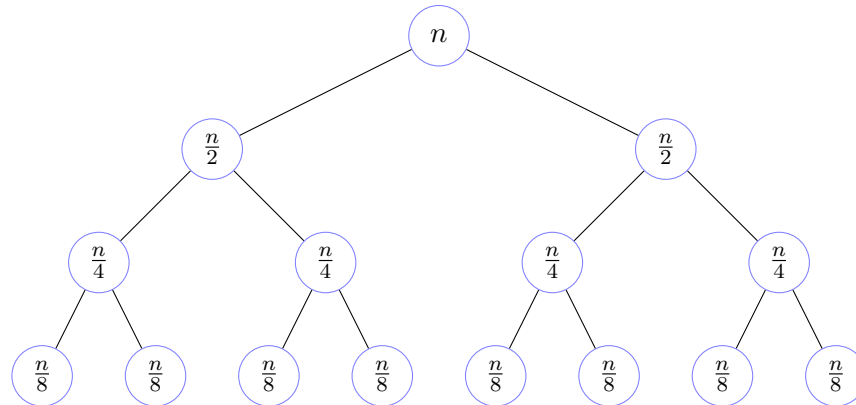


Figura 7.17: Albero di ricorsione del merge sort

Una piccola curiosità.

Nell'espressione utilizzata per risolvere l'equazione di ricorrenza dell'algoritmo di merge sort che abbiamo visto precedentemente con la prima tecnica abbiamo indicato con h il numero di volte che abbiamo effettuato la somma delle varie equazioni. Non si è trattato di una scelta casuale, si tratta infatti proprio dell'altezza dell'albero di ricorsione appena mostrato.

Inoltre notiamo che in ogni livello dell'albero ottenuto ogni nodo è associato alla stessa dimensione del problema e il numero di nodi per livello ci dice esattamente quante volte verrà effettuata una chiamata ricorsiva per una certa istanza con una specifica dimensione.

Costruiamo quindi una tabella per sintetizzare tutte le informazioni che possiamo ottenere dall'albero di ricorsione che abbiamo costruito.

Sottolineiamo che i valori della tabella sono specifici per il caso in questione ovviamente.

Di seguito mostriamo la tabella costruita su la figura 7.17

Livello	Numero nodi	Dimensione in input	Contributo per nodo	Contributo per livello
0	1	n	n	n
1	2	$\frac{n}{2}$	$\frac{n}{2}$	n
2	4	$\frac{n}{4}$	$\frac{n}{4}$	n
3	8	$\frac{n}{8}$	$\frac{n}{8}$	n
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	2^i	$\frac{n}{2^i}$	$\frac{n}{2^i}$	n

È banale adesso ottenere la complessità.

Basta appunto sommare il contributo totale per livelli per il numero di livelli, che sappiamo essere pari all'altezza dell'albero, cioè h .

$$\sum_{i=0}^h n = n \cdot \sum_{i=0}^h 1 = n \cdot h$$

Come prima andiamo a sostituire il valore di h in funzione del numero di nodi dell'albero.

$$h = \log_2 n \implies n \cdot h = n \cdot \log_2(n)$$

Quindi, ancora una volta, abbiamo concluso che l'algoritmo di merge sort ha una complessità di $\Theta(n \cdot \log(n))$.

7.11.2 Esempio 2

Facciamo un altro esempio di tale tecnica, modificando leggermente l'equazione di ricorrenza del merge sort, facendo sì che il contributo per nodo sia quadratico nella dimensione dell'input e non più lineare.

$$T(n) = \begin{cases} 1 & , \text{ se } n \leq 1 \\ 2 \cdot T\left(\frac{n}{2}\right) + n^2 & , \text{ altrimenti} \end{cases} \quad (7.18)$$

Come prima applichiamo i medesimi passi.

Costruiamo l'albero di ricorsione e la tabella associata e calcoliamo infine la soluzione.

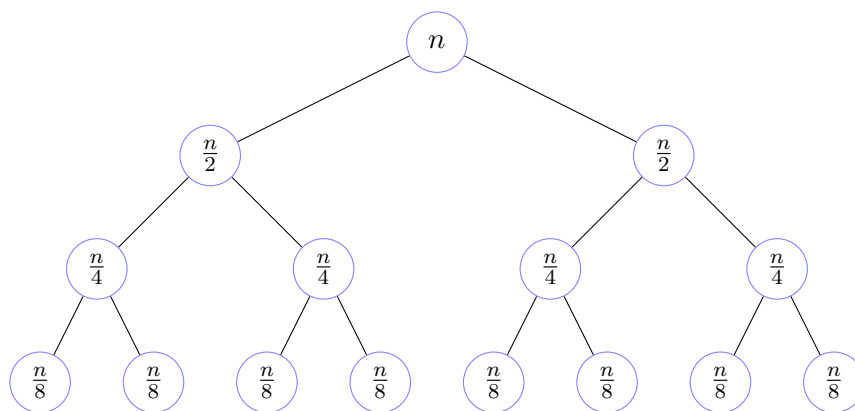


Figura 7.18: Albero di ricorsione dell'esempio 2

Di seguito mostriamo la tabella costruita su la figura 7.18

Livello	Numero nodi	Dimensione in input	Contributo per nodo	Contributo per livello
0	1	n	n^2	n^2
1	2	$\frac{n}{2}$	$\frac{n^2}{4}$	$\frac{n^2}{2}$
2	4	$\frac{n}{4}$	$\frac{n^2}{16}$	$\frac{n^2}{4}$
3	8	$\frac{n}{8}$	$\frac{n^2}{64}$	$\frac{n^2}{8}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	2^i	$\frac{n}{2^i}$	$\frac{n^2}{2^{2i}}$	$\frac{n^2}{2^i}$

Questa volta visto che il contributo totale per livello non è uniforme dobbiamo effettuare una sommatoria.

$$\sum_{i=0}^h \frac{n^2}{2^i} \quad (7.19)$$

Andiamo ad effettuare le opportune semplificazioni.

Osserviamo che utilizziamo la formula generale per la serie geometrica per le somme parziali.

$$\sum_{k=0}^t (r)^k = \frac{r^{t+1} - 1}{r - 1} \quad (7.20)$$

Pertanto

$$\sum_{i=0}^h \frac{n^2}{2^i} = n^2 \cdot \sum_{i=0}^h \left(\frac{1}{2}\right)^i = n^2 \cdot \frac{\left(\frac{1}{2}\right)^{h+1} - 1}{\frac{1}{2} - 1} = n^2 \cdot 2 \cdot \left[1 - \left(\frac{1}{2}\right)^{h+1}\right] = n^2 \cdot 2 \cdot \left(1 - \frac{1}{2^{h+1}}\right)$$

Applichiamo la sostituzione $h = \log_2(n)$.

$$n^2 \cdot 2 \cdot \left(1 - \frac{1}{2^{\log_2(n)+1}}\right) = n^2 \cdot 2 \cdot \left(1 - \frac{1}{n \cdot 2}\right) = n^2 \cdot 2 \cdot \left(\frac{2n-1}{2n}\right) = n \cdot (2n-1) = 2n^2 - n = \Theta(n^2)$$

7.11.3 Esempio 3

Facciamo un ulteriore esempio.

Prendiamo in considerazione la seguente equazione di ricorrenza.

$$T(n) = \begin{cases} 1 & , \text{ se } n \leq 1 \\ 2 \cdot T\left(\frac{n}{5}\right) + 3 \cdot T\left(\frac{n}{25}\right) + 4 & , \text{ altrimenti} \end{cases} \quad (7.21)$$

Applichiamo sempre la stessa tecnica grafica.

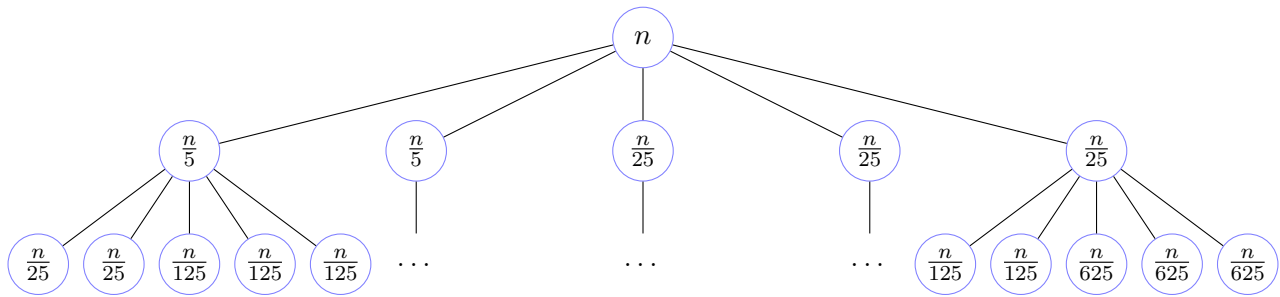


Figura 7.19: Albero di ricorsione dell'esempio 3

Di seguito mostriamo la tabella costruita su la figura 7.19

Livello	Numero nodi	Dimensione in input	Contributo per nodo	Contributo per livello
0	1	n	4	$4 = 4 \cdot 5^0$
1	5	$\frac{n}{5} \mid \frac{n}{25}$	4	$20 = 4 \cdot 5^1$
2	25	$\frac{n}{25} \mid \frac{n}{125} \mid \frac{n}{625}$	4	$100 = 4 \cdot 5^2$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	5^i	$\frac{n}{5^i} \mid \dots \mid \frac{n}{5^{2i}}$	4	$4 \cdot 5^i$

Attenzione però, se rappresentiamo la forma complessiva dell'albero di ricorsione ottenuto da questo esempio possiamo notare che differisce dagli alberi di ricorsione degli esempi precedenti.

L'albero degli esempi precedenti era un albero pieno, perché indipendentemente dalla direzione, dovevamo arrivare sempre alla stessa altezza per raggiungere le foglie.

In questo caso non è la stessa cosa.

Immaginiamo di andare sempre a sinistra, graficamente ovviamente, cioè se andiamo a considerare la prima chiamata ricorsiva riduciamo l'input di 5 volte ogni passo, che significa che all' i -esimo passo l'input si ridurrà alla dimensione di $\frac{n}{5^i}$.

Andiamo invece a guardare il ramo diametralmente opposto.

Se facessimo sempre l'ultima chiamata ricorsiva non andremmo a ridurre l'input di $\frac{n}{5}$ ad ogni passo, ma andremmo a ridurre di $\frac{n}{25}$.

Significa che il caso base lo raggiungiamo prima se andiamo a destra (effettuando sempre l'ultima chiamata ricorsiva) di quando lo raggiungiamo andando a sinistra (effettuando sempre la prima chiamata ricorsiva).

Inoltre, potremmo percorrere una via di mezzo, andando un po' a sinistra, un po' a destra, e così via; quindi effettuando prima una chiamata ricorsiva del primo tipo, poi una del secondo tipo, in modo del tutto casuale. In questo modo avremo una riduzione della dimensione dell'input che è a metà strada tra le due estreme che abbiamo descritto.

Mostriamo la forma generica dell'albero di ricorsione associato all'esempio corrente.

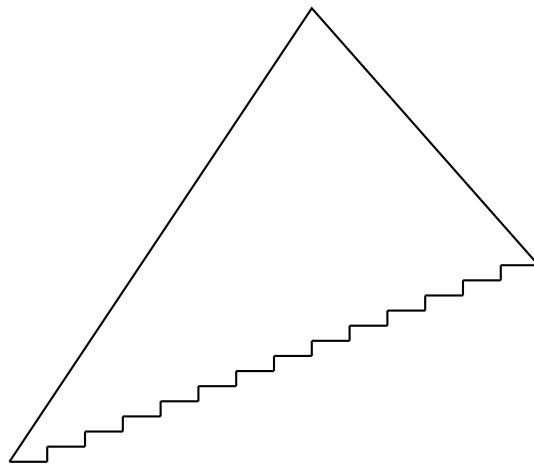


Figura 7.20: Forma generica dell'albero di ricorsione dell'esempio 3

Notiamo che come descritto il ramo tutto a destra, quello che si ottiene dividendo la dimensione dell'input sempre per 25, sarà corto, perché dividendo sempre per 25 più velocemente raggiungiamo 1, che è il caso base.

Il ramo di sinistra, che invece va a dividere la dimensione dell'input sempre per 5, sarà chiaramente più lungo. Inoltre, come si può notare, da sinistra verso destra i rami andranno via via ad accorciarsi.

Quindi, non possiamo, intuitivamente, essere sicuri di come fare a fare il calcolo per la risoluzione dell'equazione di ricorrenza, perché è vero che ogni livello, quando il livello è pieno, ha un contributo costante, ma ad un certo punto i livelli cominciano a scarseggiare, a diventare sempre più piccoli, perché man mano che andiamo in profondità ci saranno sempre più rami che avranno raggiunto il caso base e quindi saranno terminati.

Diventa veramente difficile da calcolare la complessità di un algoritmo ricorsivo con una equazione di ricorrenza simile in questo modo, sebbene, nel caso specifico, ci sia perfino la semplificazione del contributo costante per ogni istanza del problema.

Qual è l'approccio che si deve adottare, quindi?

È un approccio per approssimazione.

Ci sono casi in cui quello che possiamo fare al meglio è trovare un'approssimazione, dire cioè che il tempo si trova tra due stime.

L'esempio corrente è uno di questi casi.

Se poi asintoticamente queste due stime coincidono allora avremo trovato lo stesso andamento asintotico.

Se, invece, le due stime sono diverse dal punto di vista asintotico avremo trovato un limite superiore e un limite inferiore.

Come fare a calcolare in questo caso le due stime limite?

L'idea intuitiva è la seguente.

Possiamo sicuramente affermare che fino a quando il ramo che termina più velocemente non è arrivato al caso base l'albero avrà, per ogni livello i , un numero di nodi che sapremo calcolare facilmente grazie alla tabella; che nel caso specifico è pari a 5^i .

Se, quindi, ci fermassimo a calcolare il costo esattamente al livello in cui il primo ramo termina otterremmo sicuramente una stima inferiore, visto che mancano i contributi dovuti a tutti gli altri rami non ancora terminati. Questo quindi ci permette di avere un limite inferiore.

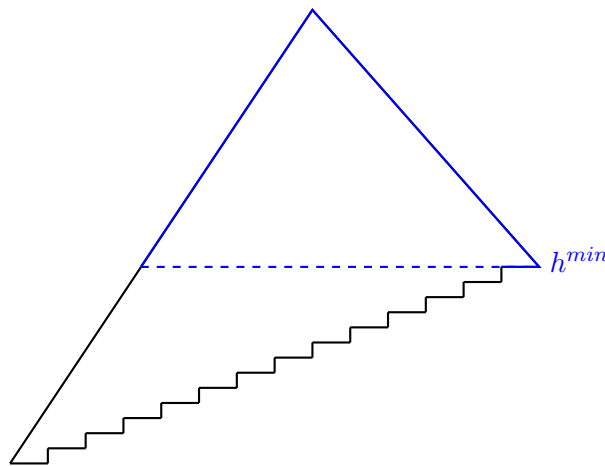


Figura 7.21: Esempio 3 limite inferiore

Immaginiamo, poi, di avere un albero pieno che termina esattamente allo stesso livello in cui termina il ramo più lungo e di calcolare il tempo considerando tale albero.

È chiaro che stiamo calcolando una stima per eccesso, perché non solo stiamo contemplando i valori appartenenti all'albero originale, ma stiamo prendendo anche ulteriori contributi che in realtà non esistono.

Questo quindi ci permette di avere un limite superiore.

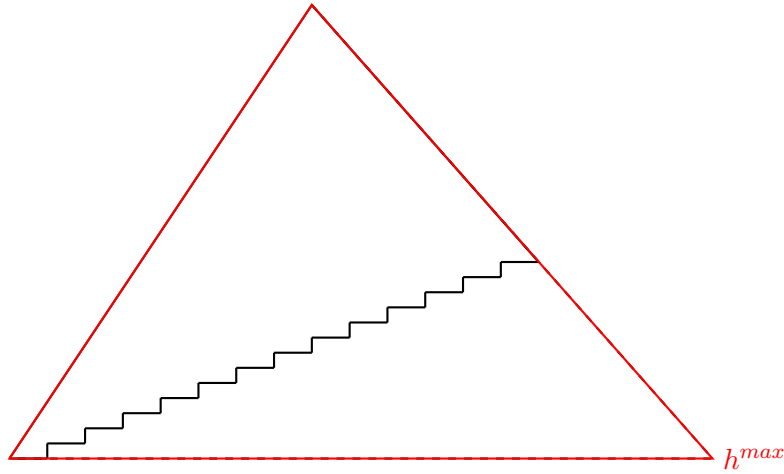


Figura 7.22: Esempio 3 limite superiore

Quindi in questo modo, combinando le cose, possiamo calcolare i due valori limite per difetto e per eccesso.

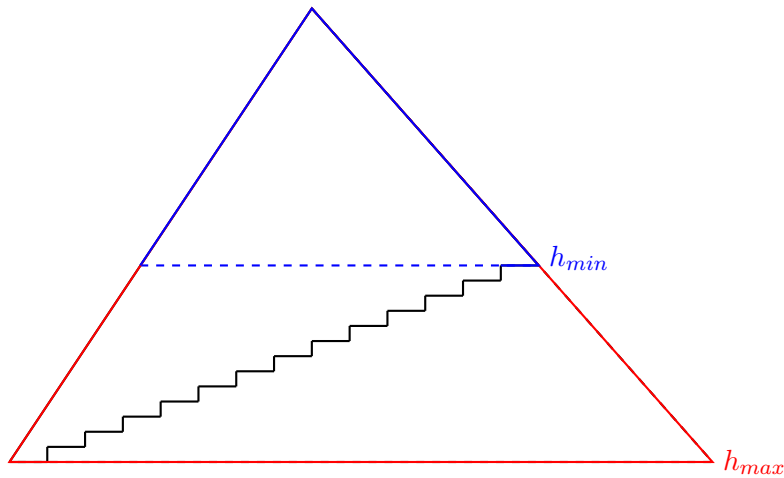


Figura 7.23: Esempio 3 limite inferiore e superiore

Ovviamente, poiché il costo per livello cambia, non possiamo effettuare un semplice prodotto ma dovremo utilizzare una sommatoria.

Tornando al caso specifico possiamo quindi ora calcolare facilmente i valori limite.

Il valore limite inferiore.

$$\frac{n}{25^{h_{min}}} \leq 1 \iff n \leq 25^{h_{min}} \iff h_{min} \geq \log_{25}(n) = \frac{\log_5(n)}{\log_5(25)} \iff h_{min} \geq \frac{1}{2} \cdot \log_5(n) \implies h_{min} = \frac{1}{2} \cdot \log_5(n)$$

Il valore limite superiore.

$$\frac{n}{5^{h_{max}}} \leq 1 \iff n \leq 5^{h_{max}} \iff h_{max} \geq \log_5(n) \implies h_{max} = \log_5(n)$$

Non ci resta altro che esplicitare il valore di h_{min} e h_{max} in funzione della dimensione dell'input.

Ma, per il ragionamento effettuato precedentemente, è chiaro che h_{min} è la profondità del ramo che termina per primo, h_{max} quella del ramo che termina per ultimo.

Utilizziamo per semplicità per entrambi i valori il $\log_5$.

$$\sum_{i=0}^{h_{min}} 4 \cdot 5^i \leq T(n) \leq \sum_{i=0}^{h_{max}} 4 \cdot 5^i$$

Ci basta, quindi, integrare il tutto ed effettuare le opportune sostituzioni.

$$4 \cdot \frac{5^{h_{min}+1} - 1}{5 - 1} \leq T(n) \leq \frac{5^{h_{max}+1} - 1}{5 - 1} \cdot 4 \iff$$

$$5 \cdot 5^{\frac{1}{2} \cdot \log_5(n)} - 1 \leq T(n) \leq 5 \cdot 5^{\log_5(n)} - 1 \iff$$

$$5 \cdot n^{\frac{1}{2}} - 1 \leq T(n) \leq 5 \cdot n - 1 \iff$$

$$\sqrt{n} \leq T(n) \leq n$$

Segue, chiaramente

$$T(n) = \Omega(\sqrt{n}) \wedge T(n) = O(n)$$

7.11.4 Esempio 4

Vediamo un altro esempio.

Consideriamo la seguente equazione di ricorrenza.

$$T(n) = \begin{cases} 1 & , \text{ se } n \leq 2 \\ n \cdot T(\sqrt{n}) + n^2 & , \text{ altrimenti} \end{cases} \quad (7.22)$$

A differenza delle equazioni di ricorrenza che abbiamo visto fino ad ora, in cui il numero di chiamate era indipendente da input, in questo caso il numero di chiamate ricorsive dipende dall'input.

Inoltre l'andamento della dimensione dell'input non deriva da una banale divisione, è infatti radice di n ; infine, il termine è n^2 .

Applichiamo il solito approccio, disegnando l'albero di ricorsione e la tabella associata.

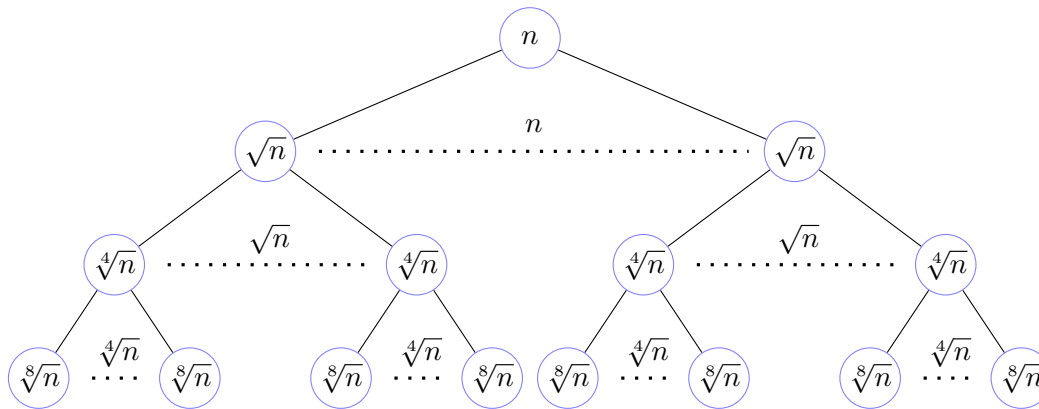


Figura 7.24: Albero di ricorsione dell'esempio 4

Di seguito mostriamo la tabella costruita su la figura 7.24

Livello	Numero nodi	Dimensione in input	Contributo per nodo	Contributo per livello
0	1	n	n^2	n^2
1	n	$n^{\frac{1}{2}}$	$n = \left(n^{\frac{1}{2}}\right)^2$	$n^2 = n \cdot n$
2	$n^{\frac{3}{2}} = n \cdot n^{\frac{1}{2}}$	$n^{\frac{1}{4}}$	$n^{\frac{1}{2}} = \left(n^{\frac{1}{4}}\right)^2$	$n^2 = n^{\frac{3}{2}} \cdot n^{\frac{1}{2}}$
3	$n^{\frac{7}{4}} = n^{\frac{3}{2}} \cdot n^{\frac{1}{4}}$	$n^{\frac{1}{8}}$	$n^{\frac{1}{4}} = \left(n^{\frac{1}{8}}\right)^2$	$n^2 = n^{\frac{7}{4}} \cdot n^{\frac{1}{4}}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	$n^{\frac{2^i-1}{2^{i-1}}}$	$n^{\frac{1}{2^i}}$	$n^{\frac{1}{2^{i-1}}}$	n^2

Visto che il contributo per livello è lo stesso ci basta effettuare il prodotto tra l'altezza, h , dell'albero di ricorsione e il contributo per livello.

Dobbiamo quindi calcolare h in funzione del numero di nodi.

Come sempre, dobbiamo trovare il valore del livello per il quale l'input fa andare la ricorsione nel caso base.

$$n^{\frac{1}{2^h}} \leq 2 \iff \log_2 \left(n^{\frac{1}{2^h}} \right) \leq 1 \iff \frac{1}{2^h} \cdot \log_2(n) \leq 1 \iff \log_2(n) \leq 2^h \iff h \geq \log_2(\log_2(n)) \implies h = \log_2(\log_2(n))$$

Possiamo ora finalmente arrivare alla soluzione.

$$T(n) = h \cdot n^2 = \log_2(\log_2(n)) \cdot n^2 = \Theta(n^2 \cdot \log(\log(n)))$$

7.11.5 Esempio 5

Vediamo un ultimo esempio.

Consideriamo la seguente equazione di ricorrenza.

$$T(n) = \begin{cases} 1 & , \text{ se } n \leq 2 \\ 3 \cdot T(\sqrt{n}) + \log(n) & , \text{ altrimenti} \end{cases} \quad (7.23)$$

Applichiamo il solito approccio, disegnando l'albero di ricorsione e la tabella associata.

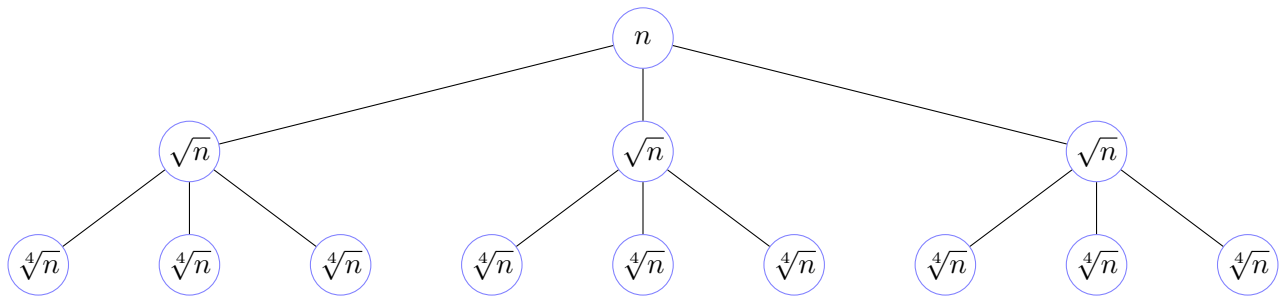


Figura 7.25: Albero di ricorsione dell'esempio 5

Di seguito mostriamo la tabella costruita su la figura 7.25

Livello	Numero nodi	Dimensione in input	Contributo per nodo	Contributo per livello
0	1	n	$\log(n)$	$\log(n)$
1	3	$n^{\frac{1}{2}}$	$\log\left(n^{\frac{1}{2}}\right)$	$3 \cdot \log\left(n^{\frac{1}{2}}\right)$
2	3^2	$n^{\frac{1}{4}}$	$\log\left(n^{\frac{1}{4}}\right)$	$3^2 \cdot \log\left(n^{\frac{1}{4}}\right)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	3^i	$n^{\frac{1}{2^i}}$	$\log\left(n^{\frac{1}{2^i}}\right)$	$3^i \cdot \log\left(n^{\frac{1}{2^i}}\right)$

Visto che il contributo per livello non è lo stesso dobbiamo effettuare una sommatoria.

$$\sum_{i=0}^h 3^i \cdot \log \left(n^{\frac{1}{2^i}} \right) \quad (7.24)$$

Applichiamo le opportune semplificazioni.

$$\begin{aligned} \sum_{i=0}^h 3^i \cdot \log \left(n^{\frac{1}{2^i}} \right) &= \sum_{i=0}^h 3^i \cdot \frac{1}{2^i} \cdot \log(n) = \log(n) \cdot \sum_{i=0}^h \left(\frac{3}{2} \right)^i = \log(n) \cdot \left[\frac{\left(\frac{3}{2} \right)^{h+1} - 1}{\frac{3}{2} - 1} \right] = \\ &= 2 \cdot \log(n) \cdot \left[\left(\frac{3}{2} \right)^{h+1} - 1 \right] \end{aligned}$$

Come sempre calcoliamo il valore dell'altezza dell'albero, h .

$$n^{\frac{1}{2^h}} \leq 2 \iff \log_2 \left(n^{\frac{1}{2^h}} \right) \leq 1 \iff \frac{1}{2^h} \cdot \log_2(n) \leq 1 \iff \log_2(n) \leq 2^h \iff h \geq \log_2(\log_2(n)) \implies h = \log_2(\log_2(n))$$

Possiamo ora, effettuando le opportune sostituzioni, finalmente arrivare alla soluzione.

$$\begin{aligned} 2 \cdot \log(n) \cdot \left[\left(\frac{3}{2} \right)^{h+1} - 1 \right] &= 2 \cdot \log(n) \cdot \left[\frac{3}{2} \cdot \left(\frac{3}{2} \right)^{\log_2(\log_2(n))} - 1 \right] = \\ 2 \cdot \log(n) \cdot \left[\frac{3}{2} \cdot \left(\frac{3^{\log_2(\log_2(n))}}{2^{\log_2(\log_2(n))}} \right) - 1 \right] &= 2 \cdot \log(n) \cdot \left[\frac{3}{2} \cdot \left(\frac{3^{\log_2(\log_2(n))}}{\log_2(n)} \right) - 1 \right] = \\ 2 \cdot \log(n) \cdot \left[\frac{3}{2} \cdot \left(\frac{2^{\log_2(\log_2(n))} \cdot \log_2 3}{\log_2(n)} \right) - 1 \right] &= 2 \cdot \log(n) \cdot \left[\frac{3}{2} \cdot \left(\frac{(\log_2(n))^{\log_2 3}}{\log_2(n)} \right) - 1 \right] = \\ 2 \cdot \log(n) \cdot \left[\frac{3}{2} \cdot \left((\log_2(n))^{\log_2(3)-1} \right) - 1 \right] &\leq 3 \cdot \log(n) \cdot \left((\log_2(n))^{\log_2(3)-1} \right) \approx \\ (\log_2(n))^{\log_2(3)} &= \Theta \left((\log(n))^{\log_2(3)} \right) \end{aligned}$$

7.12 Quick sort

Vediamo adesso l'ultimo algoritmo di ordinamento che tratteremo, il quick sort.

Qual è l'idea base del quick sort?

Per capirla dobbiamo considerare nuovamente l'algoritmo di merge sort.

Abbiamo visto che nel merge sort se andiamo a dividere un array in due parti di dimensione più o meno uguali, applichiamo ricorsivamente la funzione ricorsiva `merge_sort` e poi dopo, al termine delle chiamate, andiamo a fare il merge delle due componenti, quello che otteniamo è necessariamente un array ordinato; e l'abbiamo anche dimostrato per induzione.

La peculiarità del merge sort è che per funzionare ha bisogno dell'operazione finale di merge, che, come abbiamo osservato, non solo richiede spazio accessorio in memoria, ma ha anche, sebbene abbia una complessità lineare nella dimensione dell'input, una costante moltiplicativa particolarmente pesante, tanto da rendere il merge sort tra gli algoritmi che, nonostante asintoticamente sia uno dei migliori, non performa poi così bene in pratica.

E questo è tutto dovuto all'operazione di merge.

Il quick sort, vedremo, utilizza un approccio simile a quello del merge sort, ma leggermente diverso.

Invece di suddividere l'array in due parti uguali, senza alcuna altra informazione sulle proprietà che le sottoparti dell'array hanno, per poi fare del lavoro in post ordine, come nel merge sort, l'idea del quick sort è di fare lavoro in preorder, quindi prima delle chiamate ricorsive, per andare a trovare un punto dell'array che in qualche modo possa separare l'array in due parti, che al momento non sono ordinate ma, che hanno una proprietà fondamentale. Tutto ciò che sarà a sinistra del punto individuato sarà minore o uguale di ciò che sta alla destra del punto.

Quindi, quello che fa il quick sort è di chiedere ad un'opportuna funzione ausiliaria, che chiameremo `partition`, di identificare un punto, che non sarà necessariamente al centro dell'array (vedremo che in casi limite può essere estremamente lontano dal centro), in modo tale che gli elementi che sono a sinistra di questo punto saranno necessariamente tutti quanti minori o uguali degli elementi che sono alla destra di questo punto.

In particolare, l'algoritmo di `partition` per ottenere questa proprietà farà degli spostamenti nell'array. Questi spostamenti, poiché l'ordinamento avverrà successivamente, possono essere fatti liberamente; per altro quello che farà, sarà spostare elementi piccoli verso sinistra e elementi grandi verso destra, andando alla ricerca del punto con la proprietà voluta.

Quindi in un certo senso comincia ad ordinare, ovviamente non è un ordinamento complessivo sull'array, ma è un qualcosa che va nella direzione dell'ordinamento.

Una volta che l'algoritmo di `partition` ha identificato il punto cercato terminerà, l'algoritmo di quick sort prenderà il valore dell'indice restituito dalla funzione `partition` per andare a richiamarsi ricorsivamente sulle partizioni dell'array appena create, mediante una funzione ricorsiva che chiameremo appunto `quick_sort`.

Se assumiamo per ipotesi induttiva che la funzione ricorsiva `quick_sort` ordini l'array sul quale viene chiamata, al ritorno dalle due chiamate sulle due partizioni dell'array non sarà più necessario fare alcun lavoro in post ordine perché globalmente l'array sarà ordinato.

Se non fosse ordinato, vista l'ipotesi di correttezza della funzione ricorsiva `quick_sort`, l'unica possibilità è che dovrebbe esistere un elemento della partizione a sinistra maggiore di un elemento della partizione a destra, ma la funzione `partition` ha assicurato che una tale evenienza non possa accadere.

Conseguentemente dopo aver chiamato la funzione ricorsiva `quick_sort` avremo ordinato l'array, senza alcuna necessità di fare ulteriore lavoro a valle delle chiamate ricorsive.

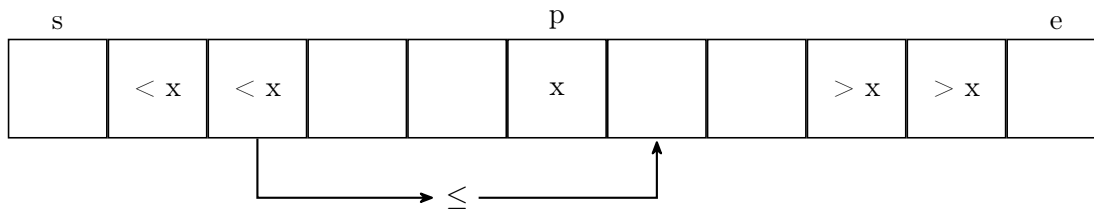


Figura 7.26: Esempio astratto della proprietà richiesta

Osserviamo, però, che il lavoro che la funzione `merge` esegue nell'algoritmo del merge sort, nel quick sort è implicitamente fatto dalla funzione `partition`.

La differenza è che viene effettuato in preorder.

Inoltre, vedremo, che come la funzione `merge`, la funzione `partition` ha una complessità lineare nella dimensione dell'array.

Quindi, in prima battuta, sembra che stiamo soltanto spostando del lavoro.

Questo è vero, ma come abbiamo anticipato, il punto che la funzione `partition` andrà ad identificare non è detto che si trovi al centro dell'array. E, come vedremo, se il punto trovato è posizionato troppo in vicinanza degli estremi dell'array l'algoritmo di quick sort perderà di efficienza e degenererà, nel caso peggiore, in un algoritmo che è quadratico nella dimensione dell'array.

Dimostreremo, tuttavia, che se però la posizione del punto individuato dalla funzione `partition` si posiziona, per quanto lontano dal centro, in una posizione che è proporzionale alla dimensione dell'array, allora l'algoritmo di quick sort si comporterà comunque come un algoritmo di tipo $n \cdot \log(n)$; ed è questo il motivo per cui l'algoritmo di quick sort, nonostante abbia un andamento asintotico nel caso peggiore che è quadratico, è in pratica uno degli algoritmi migliori.

In particolare si può dimostrare, e noi non lo faremo, che il caso medio corrisponde al caso migliore, che è $n \cdot \log(n)$.

Come al solito, prima di passare all'algoritmo vero e proprio facciamo un esempio del suo funzionamento.

Passo 0: L'algoritmo inizia settando i parametri di inizio (s) e fine (e) dell'array, rispettivamente a 0 e a 7.

s							e
2	5	7	1	3	8	6	4

Passo 1: Prendiamo come pivot l'elemento all'indice s (2) e facciamo partire l'indice i dal precedente di s e l'indice j dal successivo di e .

Per ricordarci quale elemento sia il pivot, lo coloreremo di rosso.

i	s						e	j
	2	5	7	1	3	8	6	4

Passo 1.01: Decrementiamo l'indice j e valutiamo se il valore di cui è indice nell'array è maggiore del pivot (2). In questo caso $4 > 2$ quindi si continua.

i	s							e, j
	2	5	7	1	3	8	6	4

Passo 1.02: Decrementiamo l'indice j e valutiamo se il valore di cui è indice nell'array è maggiore del pivot (2). In questo caso $6 > 2$ quindi si continua.

i	s					j	e	
	2	5	7	1	3	8	6	4

Passo 1.03: Decrementiamo l'indice j e valutiamo se il valore di cui è indice nell'array è maggiore del pivot (2). In questo caso $8 > 2$ quindi si continua.

i	s	j					e	
	2	5	7	1	3	8	6	4

Passo 1.04: Decrementiamo l'indice j e valutiamo se il valore di cui è indice nell'array è maggiore del pivot (2). In questo caso $3 > 2$ quindi si continua.

i	s	j					e	
	2	5	7	1	3	8	6	4

Passo 1.05: Decrementiamo l'indice j e valutiamo se il valore di cui è indice nell'array è maggiore del pivot (2). In questo caso $1 < 2$ quindi ci fermiamo.

i	s	j					e	
	2	5	7	1	3	8	6	4

Passo 1.06: Incrementiamo l'indice i e valutiamo se il valore di cui è indice nell'array è minore del pivot (2). In questo caso $2 < 2$ quindi ci fermiamo.

s, i			j				e
2	5	7	1	3	8	6	4

Passo 1.07: Se $i < j$ scambiamo i valori di cui sono indici e continuiamo procedendo nello stesso identico modo dei passi da 1.1 a 1.6; altrimenti il ciclo termina e restituiamo il valore dell'indice j .

Nel nostro caso $i < j$ quindi eseguiamo ciò che è stato scritto appena sopra.

s, i			j				e
1	5	7	2	3	8	6	4

Passo 1.08: Decrementiamo l'indice j e valutiamo se il valore di cui è indice nell'array è maggiore del pivot (2). In questo caso $7 > 2$ quindi si continua.

s, i		j					e
1	5	7	2	3	8	6	4

Passo 1.09: Decrementiamo l'indice j e valutiamo se il valore di cui è indice nell'array è maggiore del pivot (2). In questo caso $5 > 2$ quindi si continua.

s, i	j						e
1	5	7	2	3	8	6	4

Passo 1.10: Decrementiamo l'indice j e valutiamo se il valore di cui è indice nell'array è maggiore del pivot (2). In questo caso $1 < 2$ quindi ci fermiamo.

s, i, j							e
1	5	7	2	3	8	6	4

Passo 1.11: Incrementiamo l'indice i e valutiamo se il valore di cui è indice nell'array è minore del pivot (2). In questo caso $5 > 2$ quindi ci fermiamo.

s, j	i						e
1	5	7	2	3	8	6	4

Passo 1.12: In questo caso $i > j$ quindi il ciclo termina e restituiamo j .

s, j	i						e
1	5	7	2	3	8	6	4

Passo 2: Salviamo in una variabile p il valore restituito. Effettuiamo la prima chiamata ricorsiva sul sottoarray che ha come indice iniziale il valore s che rimane invariato, e come indice finale il valore di e che sarà uguale a p .

s, p							e
1	5	7	2	3	8	6	4

Passo 3: Il sottoarray considerato è di dimensione 1, quindi è banalmente ordinato e dunque non verrà effettuato alcun lavoro.

s, e							
1	5	7	2	3	8	6	4

Passo 4: Ritorniamo al Passo 2 ed effettuiamo la seconda chiamata ricorsiva sul sottoarray che ha come indice iniziale il valore di s uguale a $p + 1$, e come indice finale il valore di e rimane invariato.

s, p								e	
1	5	7	2	3	8	6	4		

Passo 5: Prendiamo come pivot l'elemento all'indice s (5).

i		s					e		j
1	5	7	2	3	8	6	4		

Passo 5.01: Decrementiamo l'indice j e valutiamo se il valore di cui è indice nell'array è maggiore del pivot (5). In questo caso $4 < 5$ quindi ci fermiamo.

i		s					e, j	
1	5	7	2	3	8	6	4	

Passo 5.02: Incrementiamo l'indice i e valutiamo se il valore di cui è indice nell'array è minore del pivot (5). In questo caso $5 = 5$ quindi ci fermiamo.

s, i							e, j	
1	5	7	2	3	8	6	4	

Passo 5.03: Poiché vale $i < j$ scambiamo i valori di cui sono indici.

s, i							e, j	
1	4	7	2	3	8	6	5	

Passo 5.04: Decrementiamo l'indice j e valutiamo se il valore di cui è indice nell'array è maggiore del pivot (5). In questo caso $6 > 5$ quindi continuiamo.

s, i						j	e
1	4	7	2	3	8	6	5

Passo 5.05: Decrementiamo l'indice j e valutiamo se il valore di cui è indice nell'array è maggiore del pivot (5). In questo caso $8 > 5$ quindi continuiamo.

s, i						j	e
1	4	7	2	3	8	6	5

Passo 5.06: Decrementiamo l'indice j e valutiamo se il valore di cui è indice nell'array è maggiore del pivot (5). In questo caso $3 < 5$ quindi ci fermiamo.

s, i						j	e
1	4	7	2	3	8	6	5

Passo 5.07: Incrementiamo l'indice i e valutiamo se il valore di cui è indice nell'array è minore del pivot (5). In questo caso $7 > 5$ quindi ci fermiamo.

s		i				j	e	
1	4	7	2	3	8	6	5	

Passo 5.08: Poiché vale $i < j$ scambiamo i valori di cui sono indici.

	s	i		j			e
1	4	3	2	7	8	6	5

Passo 5.09: Decrementiamo l'indice j e valutiamo se il valore di cui è indice nell'array è maggiore del pivot (5). In questo caso $2 < 5$ quindi ci fermiamo.

	s	i	j				e
1	4	3	2	7	8	6	5

Passo 5.10: Incrementiamo l'indice i e valutiamo se il valore di cui è indice nell'array è minore del pivot (5). In questo caso $2 < 5$ quindi continuiamo.

	s		i, j				e
1	4	3	2	7	8	6	5

Passo 5.11: Incrementiamo l'indice i e valutiamo se il valore di cui è indice nell'array è minore del pivot (5). In questo caso $7 > 5$ quindi ci fermiamo.

	s		j	i			e
1	4	3	2	7	8	6	5

Passo 6: In questo caso $i > j$ quindi il ciclo termina e restituiamo j .

	s		j	i			e
1	4	3	2	7	8	6	5

Passo 7: Salviamo in una variabile p il valore restituito e poi come fatto precedentemente effettuiamo le due chiamate ricorsive sul sottoarray compreso tra gli indici s e p e sul sottoarray compreso tra gli indici $p + 1$ e e . E così via.

	s		p				e
1	4	3	2	7	8	6	5

Figura 7.27: Esempio di esecuzione del quick sort

7.12.1 Algoritmo: quick sort

Vediamo adesso l'algoritmo.

Essendo un algoritmo ricorsivo avremo una funzione interfaccia e una funzione ricorsiva, oltre che, come anticipato, la funzione **partition**.

Algoritmo 114: Algoritmo di Quick sort: funzione di interfaccia

Signature `quick_sort`: $Array(D) \times \mathbb{N} \rightarrow \emptyset$

function `quick_sort`(A, n)

```
    // chiamo la funzione ricorsiva con input
    // l'array in input
    // come posizione iniziale la prima posizione dell'array
    // come posizione finale l'ultima posizione dell'array
```

```
1  quick_sort(A, 0, n-1)
```

La funzione ricorsiva, come nel caso del merge sort, prende come input l'array e due indici che identificano il primo e l'ultimo indice del sottoarray da prendere in considerazione.

Algoritmo 115: Algoritmo di Quick sort: funzione ricorsiva

Signature `quick_sort`: $\text{Array}(D) \times \mathbb{N} \times \mathbb{N} \cup \{-1\} \rightarrow \emptyset$

function `quick_sort`(`A`, `s`, `e`)

```
    // se l'indice di inizio dell' sottoarray considerato
    // è minore dell'indice finale
    // significa che il sottoarray considerato è formato
    // da almeno 2 elementi e pertanto potrebbe essere necessario
    // ordinarlo
1  if (s < e) then
    // mediante una funzione accessoria
    // setto l'indice dell'array
    // con la proprietà che tutti i valori del sottoarray considerato
    // alla sinistra di tale indice siano minori o uguali ai valori
    // alla destra di tale indice
2  p  $\leftarrow$  partition(A, s, e)
    // richiamo la funzione ricorsivamente
    // sulla partizione dell' sottoarray considerato di sinistra
3  quick_sort(A, s, p)
    // richiamo la funzione ricorsivamente
    // sulla partizione dell' sottoarray considerato di destro
4  quick_sort(A, p+1, e)
```

Vediamo adesso la funzione accessoria `partition`

Algoritmo 116: Algoritmo di Quick sort: funzione accessoria **partition****Signature partition:** $Array(D) \times \mathbb{N} \times \mathbb{N}^* \rightarrow \emptyset$ **function partition**(A, s, e)

// x: elemento pivot che servirà per separare ciò che è minore o uguale a sinistra e
 // ciò che è maggiore a destra

// salvo in una variabile accessoria il primo valore del sottoarray considerato

1 **x** $\leftarrow$ A[s]

// setto l'indice i al precedente del primo indice del sottoarray considerato e
 // l'indice j al successivo dell'ultimo indice del sottoarray considerato

2 **(i, j)** $\leftarrow$ (s-1, e+1)

// i due indici i e j andranno l'uno verso l'altro

// man mano che l'indice i avanza alla sua sinistra

// ci saranno tutti gli elementi che già sono minori del pivot

// man mano che l'indice j retrocede alla sua destra

// ci saranno tutti gli elementi che già sono maggiori del pivot

// tra i due indici ci saranno gli elementi che devono essere

// scambiati per trovare la giusta collocazione

// quando i due indici si incontreranno

// o l'indice i avrà sorpassato l'indice j l'algoritmo potrà terminare

// fino a quando l'indice i è minore dell'indice j

3 **repeat**

 // fino a quando il valore del pivot

 // è minore dell'elemento del sottoarray di indice j

4 **repeat**

 // decrementa j spostandolo verso sinistra

5 | **j** $\leftarrow$ j-1**6** **until** (A[j] <= x)

 // in questo modo in j ci sarà l'ultimo indice del sottoarray minore o uguale

 // al pivot e alla destra di j ci saranno tutti elementi maggiori del pivot

 // fino a quando il valore del pivot

 // è maggiore dell'elemento del sottoarray di indice i

7 **repeat**

 // incrementa i spostandolo verso destra

8 | **i** $\leftarrow$ i+1**9** **until** (A[i] >= x)

 // in questo modo trovo il primo indice del sottoarray maggiore o uguale

 // al pivot e alla sinistra di i ci saranno tutti elementi minori del pivot

```
10      // se i due indici non si sono incontrati od incrociati
      if (i<j) then
11          // scambio gli elementi del sottoarray di posizione i e j
          // in modo tale da posizionare correttamente gli elementi secondo il pivot
          swap(A, i, j)
      // se ci sono ancora elementi tra i due indici i e j l'algoritmo dovrà continuare
12 until (j <= i)
      // restituisco il valore dell'indice j, che è proprio il punto cercato
13 return j
```

Sottolineiamo che la scelta del pivot nell'algoritmo di partition è decisivo nella suddivisione del sottoarray in partizioni.

Per semplicità abbiamo scelto di prendere il primo elemento del sottoarray considerato ma non è la scelta ottimale, in quanto in prima posizione potrebbe esserci il valore minimo o massimo, o comunque un valore vicino ad essi. Infatti, una tecnica per rendere ancora migliore il quick sort è quella di scegliere un valore a caso nel sottoarray considerato e scambiarlo con l'elemento in prima posizione, per poi prenderlo come pivot.

Questo piccolo accorgimento rende l'algoritmo di quick sort mediamente superiore, in quanto randomizza e rende equiprobabili le possibilità di presenza di un buon valore o di un pessimo valore come pivot.

7.12.2 Proprietà della funzione `partition`

Concentriamoci sull'indice di partizione, p , che viene restituito dalla funzione `partition`.

Chiaramente, assumiamo che sia un valore all'interno del sottoarray considerato, cioè che vada dall'indice iniziale, s , all'indice finale, e .

Ma è lecito chiedersi se tale indice di partizione possa essere un qualunque valore o se ci sono dei valori problematici.

Supponiamo, ad esempio, che l'indice di partizione sia proprio uguale all'ultimo indice del sottoarray considerato. Quello che si avrebbe è che la seconda chiamata della funzione ricorsiva di `quick_sort` sarebbe una chiamata che non farebbe nulla in quanto nel nuovo sottoarray da considerare l'indice iniziale sarebbe addirittura maggiore dell'indice finale.

Andiamo a guardare la prima chiamata ricorsiva.

In questo caso si avrebbe una chiamata ricorsiva con lo stesso input precedente. Andremmo in ricorsione infinita.

Cioè, l'algoritmo di quick sort non riuscirebbe mai a raggiungere il caso base.

Risulta, quindi, ovvio, che l'indice di partizione non potrà mai assumere un valore uguale a quello finale del sottoarray considerato.

Il medesimo ragionamento, semplicemente dualizzato, si applicherebbe anche alla situazione in cui l'indice di partizione sia uguale al precedente dell'indice iniziale del sottoarray considerato.

Significa che una proprietà aggiuntiva che dobbiamo assicurare è che il valore dell'indice di partizione deve necessariamente andare dal valore dell'indice iniziale del sottoarray considerato al precedente dell'indice finale del sottoarray considerato.

Dunque le proprietà che vogliamo che valgano sono:

1. $\forall i, j \in [s, e] (i \leq p \wedge p < j \implies A[i] \leq A[j]);$
2. $s \leq p < e.$

Cerchiamo di capire perché questa particolare implementazione dell'algoritmo di `partition` soddisfi due proprietà richieste.

Teorema 7.12.1 (Dimostrazione proprietà di `partition`).

Dopo una chiamata di `partition(A, s, e)`, si ha che:

1. $\forall i, j \in [s, e] (i \leq p \wedge p < j \implies A[i] \leq A[j]);$
2. $s \leq p < e.$

Dimostrazione.

Partiamo verificando la proprietà numero 2.

La prima cosa che vogliamo dimostrare è che al termine della chiamata `partition(A, s, e)`, $p < e$.

Osservando l'algoritmo della funzione `partition` notiamo che il valore dell'indice j (che è quello che alla fine dell'algoritmo viene restituito, e quindi sarà il valore di p alla fine) all'inizio dell'algoritmo viene settato a $e + 1$.

Notiamo, però, che nell'algoritmo troviamo un ciclo `repeat...until` esterno e due ulteriori cicli `repeat...until` interni. Pertanto siamo sicuri che j verrà decrementato necessariamente almeno di una unità; di conseguenza al primo ingresso nel ciclo `repeat...until` esterno l'algoritmo entrerà anche nel primo ciclo `repeat...until` interno e avremo sicuramente che $j \leq e$.

Il problema è che non è detto che l'algoritmo vada a decrementare j più di una volta; pertanto ancora non possiamo essere certi che $j < e$.

Tuttavia, l'algoritmo, una volta uscito dal primo ciclo `repeat...until` interno, dovrà eseguire il corpo del secondo ciclo `repeat...until` interno. E la prima volta che entriamo nel ciclo `repeat...until` esterno, il corpo del secondo ciclo `repeat...until` interno verrà per forza eseguito una sola volta.

Vediamo perché.

L'algoritmo parte con $i = s - 1$, dopo il primo incremento di i , avremo, ovviamente $i = s$; ma notiamo che come pivot abbiamo scelto proprio $A[s]$, e pertanto necessariamente usciremo dal secondo ciclo `repeat...until` interno dopo un solo incremento di i ; questo la prima volta che entriamo nel ciclo `repeat...until` esterno.

Di conseguenza alla fine del secondo ciclo `repeat...until` interno avremo

- $i = s$
- $j \leq e$

Ma visto che, a causa del costrutto `if...then` della funzione `quick_sort`, chiamiamo la funzione `partition` se e solo se $s < e$, possiamo essere sicuri che nel caso peggiore alla fine del secondo ciclo `repeat...until` interno, cioè se $j = e$, avremo $i < j$.

Pertanto, non solo effettueremo uno `swap`, ma soprattutto continueremo ad iterare nel ciclo `repeat...until` esterno.

Ora possiamo essere certi che quindi alla fine dell'algoritmo della funzione `partition` avremo $j < e$. O decremteremo più di una volta j la prima volta che entriamo nel primo ciclo `repeat...until` interno, oppure entreremo necessariamente almeno due volte nel ciclo `repeat...until` esterno (e quindi anche nel primo ciclo `repeat...until` interno); in ogni caso avremo sempre $j < e$.

Per completare la verifica della proprietà numero 2 dimostriamo che al termine della chiamata `partition(A, s, e)`, avremo $s \leq p$.

Osserviamo che la prima volta che l'algoritmo entra nel primo ciclo `repeat...until` interno, grazie alla condizione di uscita del ciclo, non è possibile che j venga decrementato un numero di volte tale da raggiungere un valore strettamente minore di s . Infatti, se anche dovesse raggiungere un valore pari ad s , vista la scelta del pivot, la condizione di uscita del ciclo comporterebbe l'interruzione del primo ciclo `repeat...until` interno.

Quindi la prima esecuzione del ciclo **repeat...until** esterno non può portare j a sfiorare. Avremo necessariamente $s \leq j$.

Abbiamo, inoltre, già osservato che se l'algoritmo rientra nel ciclo **repeat...until** esterno allora viene anche eseguita l'istruzione di **swap**. Pertanto avremo portato in posizione i , un valore minore o uguale al pivot; di conseguenza, se anche j , mediante decrementi, dovesse arrivare ad un valore pari ad i si fermerà, vista la condizione di uscita del primo ciclo **repeat...until** interno; e sappiamo che alla fine del primo ciclo **repeat...until** interno avremo $i = s$. Infine, dopo il primo ingresso nel ciclo **repeat...until** esterno i può assumere valori strettamente maggiori di s (in generale avremo $i \geq s$), e visto che abbiamo osservato che in un certo senso i funge da barriera inferiore rispetto ai possibili valori di j segue necessariamente che alla fine dell'algoritmo avremo $s \leq j$.

Per la verifica di tale proprietà risulta fondamentale la condizione di arresto del primo ciclo **repeat...until** interno; se controllasse solo l'uguaglianza con il pivot non andrebbe ad arrestare i decrementi di j per valori strettamente minori del pivot e l'algoritmo non potrebbe mantenere la proprietà desiderata.

Verifichiamo, adesso, la proprietà numero 1.

Quello che andremo a dimostrare è che la funzione **partition** verifica un invariante; cioè che tutto ciò che sta strettamente alla sinistra di i deve essere minore o uguale del valore del pivot, e che tutto ciò che sta strettamente alla destra di j deve essere maggiore o uguale del valore del pivot.

Se dimostriamo che tale invariante è vero all'inizio della funzione **partition** e ad ogni iterazione del ciclo **repeat...until** esterno continua ad essere vero, visto che necessariamente i e j si incontreranno avremo che quando ci fermeremo alla sinistra di j avremo tutto ciò che è minore o uguale del pivot e alla destra di j avremo tutto ciò che è maggiore o uguale del pivot. Pertanto proprio la proprietà numero 1 che vogliamo verificare.

Formalizziamo l'invariante appena descritto; usiamo k come indice delle iterazioni del ciclo **repeat...until** esterno.

Indicheremo, inoltre, con i_k il valore di i alla k -esima iterazione del ciclo **repeat...until** esterno; e con j_k il valore di j alla k -esima iterazione del ciclo **repeat...until** esterno.

Quindi formalmente

$$\forall k (\forall i (s \leq i \leq i_k) \wedge \forall j (j_k \leq j \leq e) \implies A[i] \leq pivot \leq A[j])$$

Se riusciamo a dimostrare tale invariante per ogni k , visto che alla fine avremo che gli indici i e j andranno a sovrapporsi, avremo anche verificato la proprietà numero 1 della funzione **partition**, come già osservato.

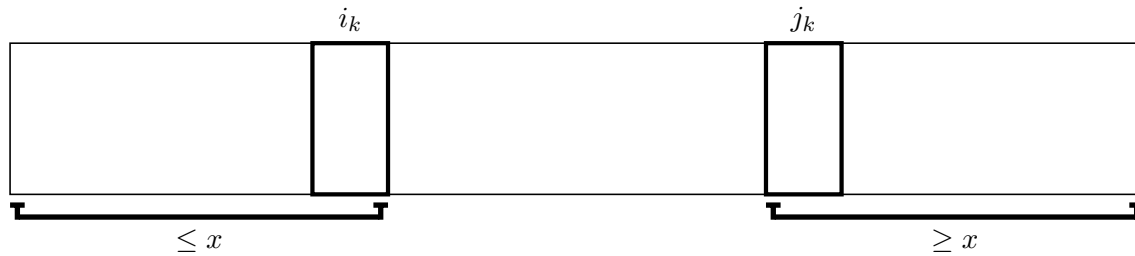
Dimostriamo l'invariante descritto per induzione su k .

Il caso base è banale. Risulta immediato che per $k = 0$, cioè prima di aver eseguito alcun ciclo **repeat...until** esterno, abbiamo che $i = s - 1$ e $j = e + 1$, e quindi l'invariante risulta vacuamente verificato.

Passimo al caso induttivo, supponiamo l'invariante vero per un generico ma fissato k , e consideriamo il caso $k + 1$.

Andiamo, pertanto, a considerare la $k + 1$ -esima iterazione del ciclo **repeat...until** esterno.

Prima della $k + 1$ -esima iterazione la situazione è la seguente.

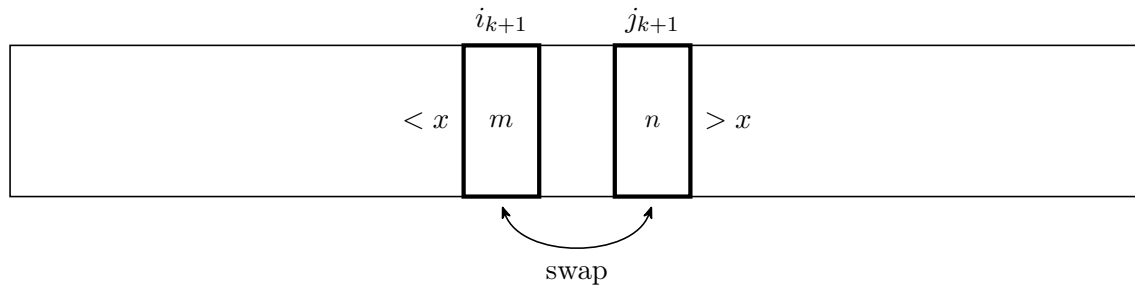


L'algoritmo, chiaramente, mediante il primo ciclo **repeat...until** interno, andrà a decrementare j fino al valore di j_{k+1} . Ovviamente avremo che $A[j_{k+1}] \leq pivot$ e che $\forall z (j_{k+1} < z \leq e \implies A[z] > pivot)$.

Dopodiché, l'algoritmo passa al secondo ciclo **repeat...until** interno che, specularmente, andrà ad incrementare i fino al valore di i_{k+1} . Ovviamente avremo che $A[i_{k+1}] \geq pivot$ e che $\forall z (s \leq z < i_{k+1} \implies A[z] < pivot)$.

Se, alla fine del secondo ciclo **repeat...until** interno i due indici i e j non si sono ancora incontrati, l'algoritmo esegue uno scambio tra l'elemento dell'array in posizione i_{k+1} e quello in posizione j_{k+1} .

Alla fine della $k + 1$ -esima iterazione la situazione è la seguente (supponendo il caso peggiore, cioè che i e j non si siano incontrati). Situazione successiva all'istruzione $\text{swap}(A, i, j)$.



Visto che, come abbiamo detto $A[i_{k+1}] \geq pivot$ e $A[j_{k+1}] \leq pivot$, a seguito dello scambio avremo $A[i_{k+1}] \leq pivot$ e $A[j_{k+1}] \geq pivot$.

Quindi, dall'ipotesi induttiva segue la dimostrazione dell'invariante, e di conseguenza della proprietà 1 della funzione **partition**.

□

7.12.3 Proprietà dell'algoritmo quick sort

Solo dopo aver assicurato le proprietà della funzione `partition` possiamo dimostrare, prima di tutto, che l'algoritmo di quick sort termini, e in secondo luogo, dimostrarne anche la correttezza.

Dimostriamo che l'algoritmo di quick sort termini.

Teorema 7.12.2 (Dimostrazione terminazione del quick sort).

Dimostrazione.

La dimostrazione che l'algoritmo di quick sort termini si basa sulla seconda proprietà che caratterizza la funzione `partition`; cioè che l'indice restituito dalla funzione `partition`, p , chiamata sull'intervallo $[s, e]$, sia, come detto, limitato in uno specifico intervallo. Abbiamo infatti che necessariamente $s \leq p < e$.

Grazie a questa proprietà siamo sicuri che le successive chiamate ricorsive della funzione `quick_sort` verranno sempre effettuate su intervalli via via strettamente minori, in termini di ampiezza.

Ricordiamo che preso un intervallo $[s, e]$, la sua dimensione è data da $e - s + 1$.

Segue, quindi

- $\forall p \in [s, e) (p - s + 1 < e - s + 1 \iff p < e)$
- $\forall p \in [s, e) (e - (p + 1) + 1 = e - p < e - s + 1 \iff p > s - 1)$

Pertanto, ovviamente, siamo certi che prima o poi l'algoritmo raggiungerà il caso base, e quindi la tesi.

Osserviamo che per tale dimostrazione risulta fondamentale la seconda proprietà caratterizzante della funzione `partition`.

□

Vediamo ora la dimostrazione di correttezza dell'algoritmo di quick sort.

Teorema 7.12.3 (Dimostrazione correttezza del quick sort).

Dimostrazione.

Dimostriamo la correttezza dell'algoritmo di quick sort per induzione sulla dimensione dell'array in input, n ; in modo analogo a quanto fatto per la dimostrazione di correttezza dell'algoritmo di merge sort.

I casi base, $n = 0 \vee n = 1 \iff s = e - 1 \vee s = e$, sono banalmente verificati. L'algoritmo non fa nulla e la correttezza risulta vera vacuamente.

Passiamo al caso induttivo, considerando l'algoritmo corretto fino ad un certo generico ma fissato n , e consideriamo il caso per $n + 1$.

Applichiamo l'algoritmo. Poniamoci nel caso in cui $s < e \iff n \geq 2$, altrimenti risulta banalmente verificato.

Come prima istruzione avviene una chiamata alla funzione `partition`, che abbiamo dimostrato essere corretta e che sappiamo restituire un indice (e potenzialmente effettuare scambi) con la proprietà che tutti gli elementi alla sinistra di tale indice risultano minori o uguali degli elementi posti alla destra di tale indice.

Successivamente verranno effettuate due chiamate ricorsive della funzione `quick_sort`, e, come abbiamo dimostrato nella prova di terminazione dell'algoritmo, tali chiamate vengono effettuate su intervalli strettamente minori di quello di partenza, sfruttando l'indice restituito dalla funzione `partition`.

Pertanto per ipotesi induttiva la tesi.

Osserviamo che per tale dimostrazione sono risultate fondamentali entrambe le proprietà caratterizzanti della funzione `partition`.

□

7.12.4 Complessità dell'algoritmo quick sort

Passiamo adesso a calcolare la complessità dell'algoritmo di quick sort.

Vedremo che se l'indice di partizione raggiunge proprio uno dei valori estremi del suo intervallo di possibilità, cioè l'indice iniziale del sottoarray considerato o il precedente dell'indice finale del sottoarray considerato, allora l'algoritmo avrà costo quadratico.

Ovviamente la funzione di interfaccia ha una complessità costante, cioè $\Theta(1)$.

Abbiamo anticipato che la funzione `partition` è lineare nella dimensione dell'array in input. Andiamo a dimostrarlo.

Osservando la funzione `partition` abbiamo un ciclo di cicli e le cose potrebbero sembrare complesse.

Tuttavia, quello che possiamo osservare è che, per come è definito l'algoritmo, andremo a eseguire soltanto dei decrementi dell'indice j , e soltanto degli incrementi dell'indice i .

Tali indici non potranno mai sfiorare la loro posizione relativa perché nel momento in cui facciamo il controllo di terminazione del ciclo più esterno e troviamo che sono uguali o addirittura l'indice i ha superato l'indice j , ci fermeremo.

Dobbiamo capire, quindi, quanti incrementi dell'indice i e quanti decrementi dell'indice j siano possibili.

Abbiamo dimostrato che l'indice j non può scendere al di sotto del primo indice del sottoarray considerato, e visto che parte dal successivo dell'ultimo indice del sottoarray considerato, complessivamente, nelle varie iterazioni del ciclo esterno non potrà essere decrementato più di n volte, dove n è la dimensione dell'intervallo dell'array su cui incide l'algoritmo.

In maniera simile, anche gli incrementi dell'indice i sono limitati in quanto non appena l'indice i raggiunge un valore uguale a quello dell'indice j , o al massimo lo supera di una posizione, non potrà essere ulteriormente incrementato.

In un certo senso l'indice j funge da barriera superiore (debole, in quanto l'indice i può raggiungere al massimo il valore successivo a quello dell'indice j) nei confronti dell'indice i .

Quindi complessivamente i due cicli interni faranno al massimo $n+1$ iterazioni globalmente, dove n è la dimensione del sottoarray considerato.

Conseguentemente le operazioni complessive che effettua l'algoritmo di `partition` globalmente sono lineari in n , indipendentemente da quante volte verrà eseguito il ciclo `repeat...until` esterno.

Inoltre, anche il ciclo `repeat...until` esterno non può essere eseguito globalmente più di n volte perché ad ogni iterazione del ciclo esterno gli indici i e j vengono incrementati e decrementati, rispettivamente, almeno di una unità.

Significa che, nel caso peggiore, effettuiamo un incremento dell'indice i e un decremento dell'indice j , accorciando man mano la parte di sottoarray da considerare.

Quindi, se ad ogni iterazione del ciclo esterno, nel caso peggiore, accorciamo il sottoarray considerato di 2 posizioni dopo $\frac{n}{2}$ cicli ci dovremo necessariamente fermare per la condizione di terminazione del ciclo `repeat...until` esterno.

Ovviamente, se i cicli interni effettuano più di un incremento o più di un decremento per ogni iterazione del ciclo esterno si avrà l'effetto di raggiungere la condizione di terminazione del ciclo esterno più velocemente.

Quindi il ciclo **repeat...until** esterno verrà eseguito al più $n/2$ volte.

Significa che complessivamente l'intero ciclo esterno, compreso dei due cicli interni, che abbiamo già visto essere lineari in n , e dell'istruzione di **swap**, chiaramente costante, non potrà che costare lineare nella dimensione del sottoarray considerato nel caso peggiore, quindi $O(n)$.

Passiamo adesso alla complessità della funzione ricorsiva **quick_sort**.

Osservando l'algoritmo della funzione ricorsiva **quick_sort** si nota facilmente che è formato da un controllo iniziale a tempo costante, e se non si entra nel caso base avremo una chiamata a **partition**, che abbiamo visto può avere complessità lineare, e due chiamate ricorsive che però hanno dimensioni diverse.

Formalizziamo tale osservazione nell'equazione di ricorrenza associata alla funzione **quick_sort**.

Indicheremo con k la dimensione della partizione di sinistra, cioè la dimensione del sottoarray della prima chiamata ricorsiva.

$$T_{QS}(A, n) = \begin{cases} 1 & , \text{ se } n \leq 1 \\ T_{QS}(A, k) + T_{QS}(A, n - k) + n & , \text{ altrimenti} \end{cases} \quad (7.25)$$

Il valore di k , cioè la dimensione del sottoarray della prima chiamata ricorsiva, in principio non possiamo stabilirlo in quanto dipende dall'indice di partizione p , come abbiamo ripetuto più volte, tale indice può appartenere ad uno specifico intervallo.

Vedremo che il caso peggiore si avrà quando il valore di k è costante, indipendentemente da n .

Se invece la funzione **partition** divide l'array in parti proporzionali ad n , nonostante a livello di costante moltiplicativa possa variare notevolmente (con valori migliori quando l'array viene diviso esattamente a metà), avrà in ogni caso un comportamento che asintoticamente corrisponderà esattamente a $n \cdot \log(n)$.

Quindi quello che faremo è studiare questa equazione di ricorrenza in base a k .

Vedremo il caso in cui k sarà costante, ed il caso in cui k sarà proporzionale ad n , sarà un alfa di n , per alfa si intende un valore da 0 ad 1.

Si può dimostrare, come già detto, che l'algoritmo di quick sort nel caso medio si comporta come nel caso migliore.

Non lo dimostreremo formalmente, ma come nel caso dell'algoritmo di insertion sort daremo il ragionamento intuitivo.

Discuteremo, ovviamente, anche i casi che portano l'algoritmo di quick sort nel caso peggiore; come ad esempio gli array ordinati.

Andiamo adesso a risolvere l'equazione di ricorrenza derivata dalla funzione ricorsiva **quick_sort**.

Vediamo prima il caso peggiore, cioè quando, come anticipato, il valore di k è costante.

Per semplicità di calcolo vedremo il caso in cui $k = 1$.

Chiaramente i ragionamenti che vedremo si estendono facilmente ad un qualunque k costante.

Pertanto l'equazione di ricorrenza considerata è la seguente.

$$T_{QS}(A, n) = \begin{cases} 1 & , \text{ se } n \leq 1 \\ T_{QS}(A, 1) + T_{QS}(A, n-1) + n & , \text{ altrimenti} \end{cases} \quad (7.26)$$

Applichiamo il metodo grafico di risoluzione, che abbiamo ampiamente visto in precedenza.

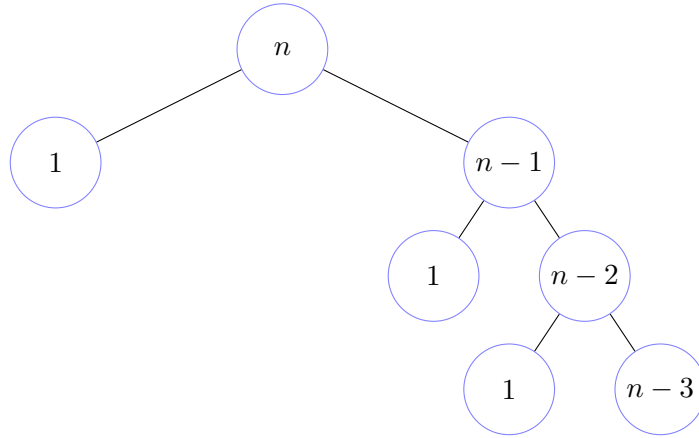


Figura 7.28: Albero di ricorsione del quick sort

Di seguito mostriamo la tabella costruita su la figura 7.28

Livello	Numero nodi	Dimensione in input	Contributo per nodo	Contributo per livello
0	1	n	n	n
1	2	$1 \mid n-1$	$1 \mid n-1$	n
2	2	$1 \mid n-2$	$1 \mid n-2$	$n-1$
3	2	$1 \mid n-3$	$1 \mid n-3$	$n-2$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$n-1$	2	1	1	2

Andiamo a considerare il costo complessivo dell'algoritmo. Visto che il contributo per livello non è costante, come al solito, effettueremo una sommatoria.

$$n + \sum_{i=2}^n i = n + \frac{n \cdot (n+1)}{2} - 1 = \Theta(n^2)$$

Quindi, nel caso di $k = 1$, l'algoritmo di quick sort ha una complessità quadratica nella dimensione dell'array in input; cioè $\Theta(n^2)$.

Risolviamo la stessa equazione di ricorrenza per $k = 2$.

$$T_{QS}(A, n) = \begin{cases} 1 & , \text{ se } n \leq 1 \\ T_{QS}(A, 2) + T_{QS}(A, n-2) + n & , \text{ altrimenti} \end{cases} \quad (7.27)$$

Applichiamo il metodo grafico di risoluzione, che abbiamo ampiamente visto in precedenza.

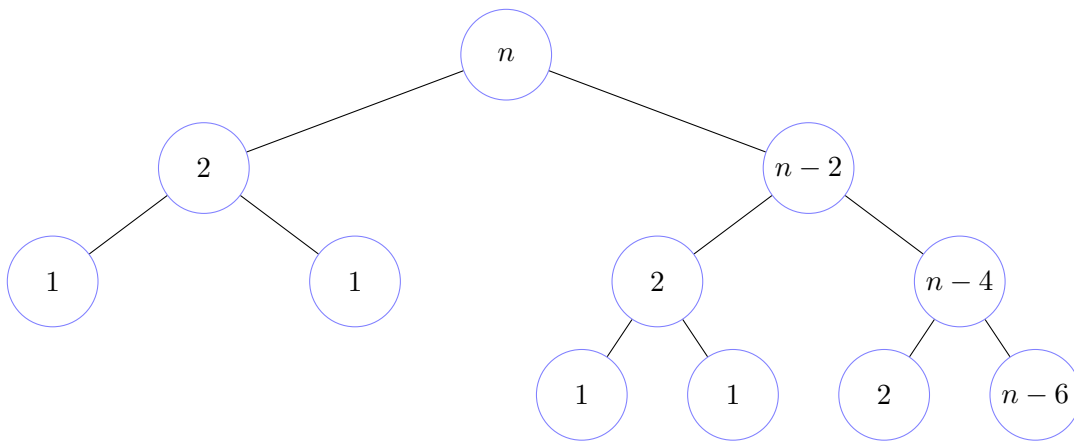


Figura 7.29: Albero di ricorsione del quick sort

Di seguito mostriamo la tabella costruita su la figura 7.29

Livello	Numero nodi	Dimensione in input	Contributo per nodo	Contributo per livello
0	1	n	n	n
1	2	$2 \mid n - 2$	$2 \mid n - 2$	n
2	4	$1 \mid 2 \mid n - 4$	$1 \mid 2 \mid n - 4$	n
3	4	$1 \mid 2 \mid n - 6$	$1 \mid 2 \mid n - 6$	$n - 2$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\lceil \frac{n}{2} \rceil$	$2 \mid 4$	1	1	$2 \mid 4$

Andiamo a considerare il costo complessivo dell'algoritmo.

Visto che il contributo per livello non è costante, come al solito, effettueremo una sommatoria.

$$2n + \sum_{i=4}^n i = 2n + \frac{n \cdot (n+1)}{2} - 6 = \Theta(n^2)$$

Quindi, anche nel caso di $k = 2$, l'algoritmo di quick sort ha una complessità quadratica nella dimensione dell'array in input; cioè $\Theta(n^2)$.

Notiamo che, in generale, per k costante, il ramo lungo dell'albero di derivazione sarà sempre e comunque lineare in n .

Inoltre, sebbene il contributo per livello possa cambiare, il numero di livelli non diminuisce a sufficienza per far sì che con k costante ci sia una complessità diversa da quella quadratica, rimane comunque lineare in n .

È immediato che se sommiamo dei termini lineari in n un numero lineare in n di volte otterremo qualcosa di quadratico in n .

Ecco l'idea generale del perché con k costante l'algoritmo di quick sort avrà sempre e necessariamente complessità quadratica nella dimensione dell'array in input; quindi $\Theta(n^2)$.

Fortunatamente k è costante raramente.

Un caso in cui k è costante, è quando l'array in input è ordinato, o quando l'array è ordinato in senso inverso.

Di solito, però, ovviamente, un algoritmo di ordinamento viene applicato su sequenze non ordinate.

Di conseguenza, si può dimostrare, e non lo faremo, con un'analisi del caso medio che la funzione **partition** quasi sempre partiziona in modo proporzionale alla dimensione l'array in input.

Cioè il valore di k è proporzionale ad n .

$$\exists \alpha \in (0, 1) (k = \alpha \cdot n) \quad (7.28)$$

Andiamo, quindi, a risolvere l'equazione di ricorrenza derivata dalla funzione ricorsiva `quick_sort` nel caso in cui k è proporzionale ad n .

Vedremo che, indipendentemente dal valore di α , l'algoritmo di quick sort avrà un andamento che è $n \cdot \log(n)$.

$$T_{QS}(A, n) = \begin{cases} 1 & , \text{ se } n \leq 1; \\ T_{QS}(A, \alpha \cdot n) + T_{QS}(A, (1 - \alpha) \cdot n) + n & , \text{ altrimenti.} \end{cases} \quad (7.29)$$

Senza ledere alcuna generalità, ma solo per semplificare l'analisi e soprattutto l'approccio grafico, supponiamo che α sia minore o uguale a $\frac{1}{2}$.

$$\alpha \leq \frac{1}{2}$$

Utilizziamo la solita tecnica grafica di risoluzione.

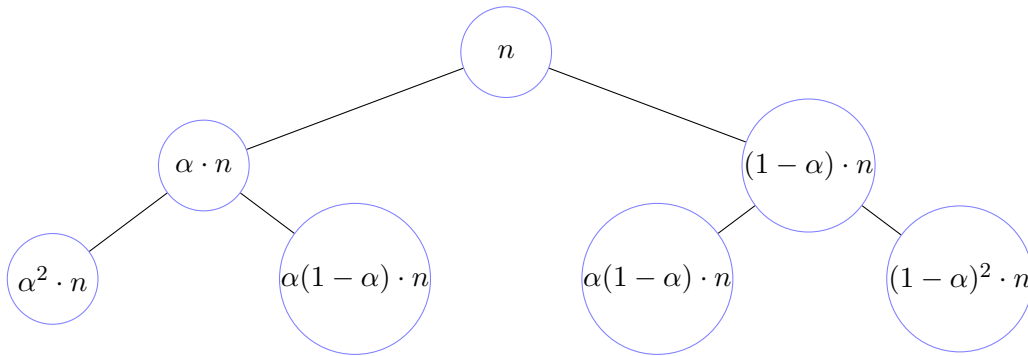


Figura 7.30: Albero di ricorsione del quick sort

Di seguito mostriamo la tabella costruita su la figura 7.30

Livello	Numero nodi	Dimensione in input	Contributo per nodo	Contributo per livello
0	1	n	n	n
1	2	$\alpha \cdot n \mid (1 - \alpha) \cdot n$	$\alpha \cdot n \mid (1 - \alpha) \cdot n$	n
2	4	$\alpha^2 n \mid \alpha(1 - \alpha)n \mid (1 - \alpha)^2 n$	$\alpha^2 n \mid \alpha(1 - \alpha)n \mid (1 - \alpha)^2 n$	n
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	2^i	$\alpha^i n \mid \dots \mid (1 - \alpha)^i n$	$\alpha^i n \mid \dots \mid (1 - \alpha)^i n$	n

Vista la restrizione su α l'albero sarà più profondo a destra che a sinistra.

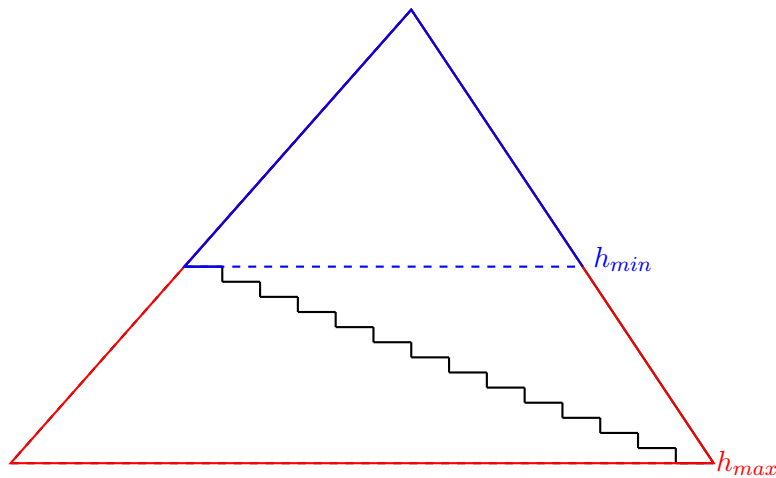


Figura 7.31: Forma generica dell'albero di ricorsione del quick sort con k proporzionale a n

Pertanto applichiamo la tecnica mostrata in precedenza per la stima del limite inferiore e del limite superiore.

Calcoliamo il limite inferiore.

$$\alpha^{h_{min}} n \leq 1 \iff n \leq \left(\frac{1}{\alpha}\right)^{h_{min}} \iff h_{min} \geq \log_{\frac{1}{\alpha}}(n) \implies h_{min} = \log_{\frac{1}{\alpha}}(n)$$

Calcoliamo il limite superiore.

$$(1 - \alpha)^{h_{max}} n \leq 1 \iff n \leq \left(\frac{1}{1 - \alpha} \right)^{h_{max}} \iff h_{max} \geq \log_{\frac{1}{1-\alpha}}(n) \implies h_{max} = \log_{\frac{1}{1-\alpha}}(n)$$

Visto che il contributo per livello è costante, possiamo effettuare il prodotto.

$$n \cdot \log_{\frac{1}{\alpha}}(n) \leq T_{QS}(A, n) \leq n \cdot \log_{\frac{1}{1-\alpha}}(n) \implies T_{QS}(A, n) = \Theta(n \cdot \log(n)) \quad (7.30)$$

Abbiamo mostrato che quindi quando k è proporzionale ad n , indipendentemente dal fattore di proporzione α , la complessità dell'algoritmo di quick sort è $n \cdot \log(n)$; cioè $\Theta(n \cdot \log(n))$.

Chiaramente al variare del fattore di proporzione cambierà la costante moltiplicativa, che sappiamo essere trascurabile a livello asintotico.

Nel complesso, quindi, abbiamo dimostrato che l'algoritmo di quick sort nel caso migliore ha complessità $n \cdot \log(n)$ e nel caso peggiore risulta quadratico; cioè ha complessità $\Omega(n \cdot \log(n))$ e $O(n^2)$, dove n è la dimensione dell'array in input.

Ricordiamo, che come più volte ripetuto, sebbene non lo dimostreremo, nel caso medio l'algoritmo di quick sort si comporta come nel caso migliore, che è $n \cdot \log(n)$.

Sottolineiamo che la differenza tra le complessità è proprio dovuta all'altezza dell'albero di ricorsione che si ottiene nei due casi.

Quando k è costante abbiamo visto che l'altezza dell'albero è lineare in n ; quando invece k è proporzionale ad n l'altezza dell'albero risulta essere logaritmica in n .

7.13 Teorema sul limite asintotico dell'ordinamento basato su confronti

Abbiamo illustrato una serie di algoritmi che, facendo solo uso di confronti, risolvono il problema dell'ordinamento.

Abbiamo trattato algoritmi con diversa complessità e abbiamo visto che sicuramente c'è modo di risolvere il problema con andamento asintotico migliore, peggiore, e medio pari a $n \cdot \log(n)$; gli algoritmi migliori che abbiamo analizzato risolvono il problema con questa complessità.

Ovviamente è lecito chiedersi se sia il meglio che si può fare.

In generale aver dato un algoritmo di soluzione non significa che quella sia poi la tecnica migliore. Questa è una domanda, in generale, che bisognerebbe sempre porsi quando si progetta un algoritmo.

Precisiamo che esistono algoritmi di ordinamento, in effetti, che ordinano un vettore con complessità asintotiche migliori, tipo bucket sort, radix sort, ce ne sono diversi, che hanno una complessità asintotica lineare.

Già abbiamo osservato tempo fa che meno che lineare non si può avere.

Sicuramente ogni algoritmo di ordinamento deve necessariamente eseguire almeno un numero di istruzioni che è pari alla dimensione dell'array, in quanto per forza di cose bisogna controllare tutti i valori dell'array.

Quindi sembrerebbe lecito pensare che il minimo numero di istruzioni necessarie per risolvere il problema dell'ordinamento è lineare nella dimensione dell'array in input.

Se andassimo a sfruttare ulteriori informazioni su dati, come negli algoritmi di radix sort, bucket sort, eccetera, effettivamente si può ottenere un algoritmo con complessità lineare; ma stiamo sfruttando un'informazione aggiuntiva sui dati, cioè la possibilità di utilizzare i dati come chiave di accesso all'array.

Ma se non abbiamo questa caratteristica, cioè se abbiamo dei dati che non possono essere di per sé utilizzati come puntatori, come indici in un array, e l'unica cosa che abbiamo è una relazione di confronto tra dati, si può dimostrare che meno $n \cdot \log(n)$ operazioni a livello asintotico non si possono fare.

Ovviamente questo nel caso peggiore, e lo vedremo anche nel caso medio.

Infatti, un algoritmo di ordinamento che utilizza solo i confronti nel caso migliore potrà eseguire un numero di operazioni lineare (l'abbiamo visto con l'algoritmo di insertion sort).

Ma vedremo che sia nel caso medio, che nel caso peggiore, non è possibile; si devono fare comunque $n \cdot \log(n)$ operazioni.

Il problema è che per provare queste nostre affermazioni si richiede una dimostrazione che mostri che qualunque algoritmo che risolva correttamente il problema dell'ordinamento, utilizzando solo confronti, deve avere, eccezion fatta per il caso migliore, asintoticamente una complessità superiore o uguale a $n \cdot \log(n)$. Il problema sta nel fatto che in generale gli algoritmi possibili sono infiniti.

Come facciamo, quindi, a fare una dimostrazione di impossibilità se lo spettro di analisi è infinito?

In prima battuta sembrerebbe qualcosa di irrealizzabile.

Tuttavia, se gli algoritmi che ci interessano sono quelli appunto basati soltanto sul confronto, possiamo creare un'astrazione del concetto di un algoritmo che è sufficientemente precisa per far sì che l'analisi di impossibilità possa essere portata a termine.

Vediamo qual è l'idea.

Guardiamo un algoritmo a caso di quelli che abbiamo studiato. Cosa fanno? Confrontano due posizioni nell'array, molto spesso posizioni contigue nell'array, e a seconda che sia vera o meno la relazione d'ordine, quello che possono fare è effettuare degli scambi. In generale gli algoritmi che abbiamo visto funzionano tutti così.

Poi è scegliere cosa confrontare, quando confrontare, quando scambiare, eccetera, che fa davvero la differenza tra i diversi algoritmi. Ma tutti quanti gli algoritmi che possono utilizzare soltanto il confronto, devono confrontare delle celle dell'array e potenzialmente scambiare i valori.

L'idea è di prendere queste operazioni atomiche, che qualunque algoritmo di ordinamento basato su confronti deve poter fare, cioè appunto l'operazione di confronto e l'operazione di scambio, e creare uno schema generale sufficientemente preciso per analizzare un qualunque algoritmo di ordinamento basato solo su confronti; e pertanto, nonostante gli algoritmi in generale siano infiniti, quello che costruiremo è un'astrazione finita.

Andremo a linearizzare l'insieme di confronti e di scambi per capire ciò che l'algoritmo fa o meno.

Di conseguenza, quello che vedremo è una struttura chiamata albero di decisione.

Si tratta di un concetto molto elementare ma che ha applicazioni a diversi livelli molto importanti che è, appunto, un albero in cui ogni nodo fa un controllo di una qualche informazione e decide se andare a destra o a manca a seconda di quella informazione.

Dopodiché, successivamente, a seconda se è andato a destra o a manca continuerà a fare quello che deve fare.

Vediamo un esempio prima di dare una formalizzazione generale.

Prendiamo un array di dimensione 3 e vediamo un potenziale approccio basato su quest'idea.

0	1	2
k_0	k_1	k_2

Figura 7.32: Esempio di array per l'albero di decisione

Vogliamo capire grazie a confronti qual è la sequenza ordinata finale.

All'interno dei nodi dell'albero di decisione che costruiremo andremo a inserire confronti tra gli indici dell'array ma dal punto di vista semantico intendiamo il confronto tra i valori dell'array con quegli indici.

Per semplicità non considereremo il caso di dati duplicati.

Si può estendere l'analisi al caso dei dati duplicati, ma non aggiunge né toglie nulla al ragionamento.

Quindi consideriamo per semplicità il caso di array i cui dati sono per forza diversi.

Quindi, la condizione che viene valutata in un nodo potrà, ovviamente, essere vera o falsa; in base al valore di verità andremo a valutare altre condizioni.

I nodi foglia saranno nodi diversi che stanno ad indicare direttamente il risultato.

In particolare, indicheremo la sequenza di indici, dei valori originali, che dovranno essere riportati nell'array finale per renderlo ordinato.

In questo schema, gli scambi è come se ne andassimo a farli tutti alla fine.

È inutile farli nel mezzo per questioni di semplicità di analisi. A livello asintotico cambia nulla.

Vediamo adesso l'esempio di albero di decisione sull'array in figura 7.32.

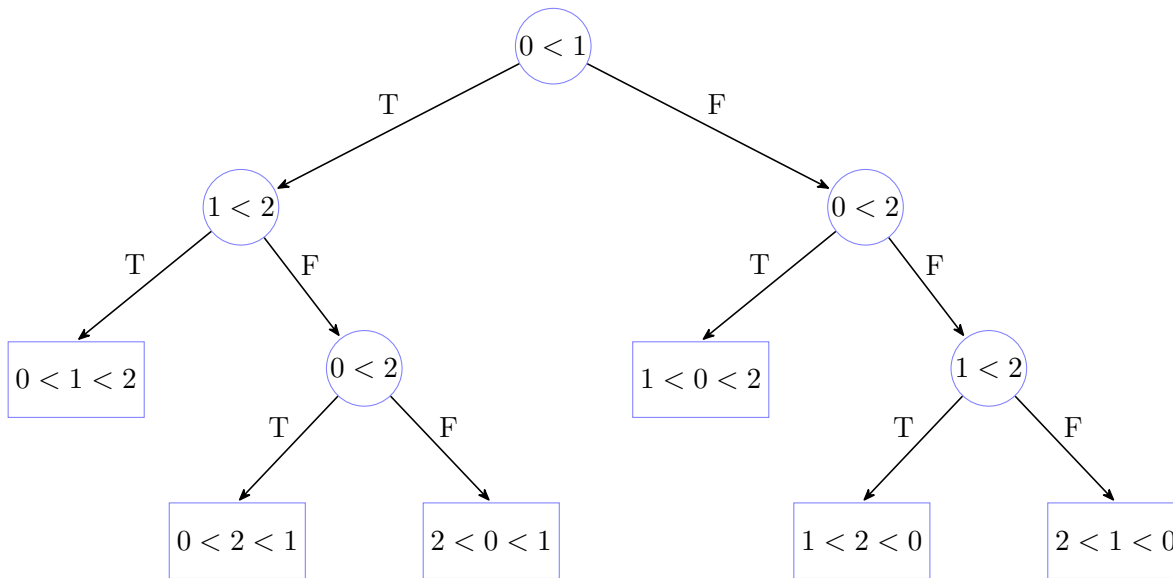


Figura 7.33: Esempio di albero di decisione su un array di dimensione 3

Si può osservare che il funzionamento dell'albero di decisione è esattamente quello che fa l'algoritmo di insertion sort.

L'unica differenza è che in questo caso effettuiamo gli scambi soltanto alla fine; è come se andassimo a decidere prima come e cosa scambiare, e poi solo dopo effettivamente scambiare le cose.

Ma a parte questo dettaglio il funzionamento è esattamente identico.

Infatti, quando abbiamo analizzato l'algoritmo di insertion sort abbiamo osservato che l'algoritmo, per poter ordinare un qualunque array, dovrà per forza seguire una serie di step.

Tali step equivalgono, nell'insieme, proprio ad uno dei percorsi dell'albero di decisione appena mostrato.

Facciamo un'importante osservazione.

Il numero di foglie dell'albero di decisione mostrate sono esattamente tutte le 6 permutazioni di 3 valori.

In generale, infatti, avremo che il numero di foglie dell'albero di decisione di un algoritmo di ordinamento basato solo su scambi è proprio pari al numero di permutazioni degli elementi della sequenza da ordinare.

In generale, ad ogni algoritmo di ordinamento è possibile associare un albero di decisione.

Basta semplicemente seguire quelli che sono i confronti che l'algoritmo effettua e man mano costruire l'albero in base ai valori di verità dei confronti stessi.

Vedremo che c'è un modo per formalizzare questa idea che ad ogni algoritmo di ordinamento basato su confronti possa essere associato un albero di decisione. E viceversa. Un albero di decisione corrisponde ad un algoritmo di ordinamento basato su confronti. In particolare corrisponde ad un algoritmo su una dimensione fissata.

Nell'esempio proposto, l'albero di decisione è un algoritmo di ordinamento su un array di dimensione 3. Lo possiamo vedere come un algoritmo a sé stante.

Cioè, se scrivessimo il codice che implementa l'albero dell'esempio, che ovviamente saranno tre `if` in cascata al più, otterremmo un algoritmo che ordina un array di 3 elementi.

Quindi un albero di decisione di per sé è un algoritmo di ordinamento, l'unica cosa è che ha dimensione fissata, non è parametrico, cioè non è generico sulla dimensione.

Quindi, in generale, ogni algoritmo di ordinamento può essere tradotto in un albero di decisione e ogni albero di decisione è un algoritmo di ordinamento.

Sottolineiamo, inoltre, che non può esistere un algoritmo di ordinamento il cui albero di decisione ha meno di $n!$ foglie, dove n è la dimensione dell'array.

Se fosse possibile, per assurdo, potremmo prendere la permutazione che manca nell'albero di decisione e dare in input a quell'albero di decisione un array che ha gli elementi disposti esattamente come la permutazione mancante. Quella permutazione non potrà essere ordinata da quell'albero di decisione.

Modifichiamo l'esempio precedente 7.33 per mostrare quanto affermato.

Eliminiamo dall'albero di decisione il nodo $0 < 2$ e al suo posto sostituiamo la foglia $0 < 2 < 1$.

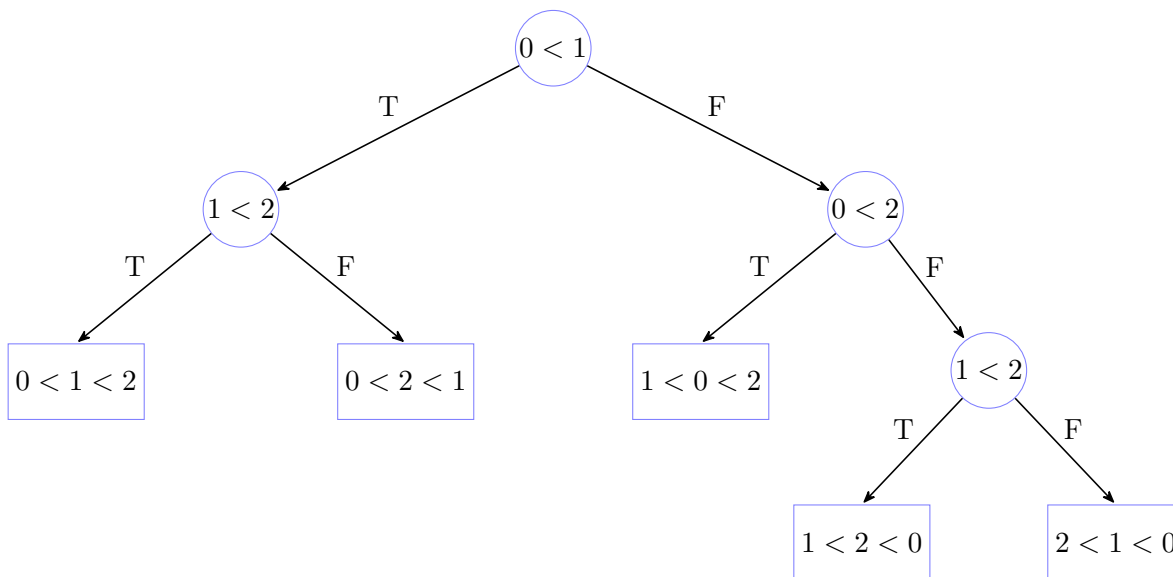
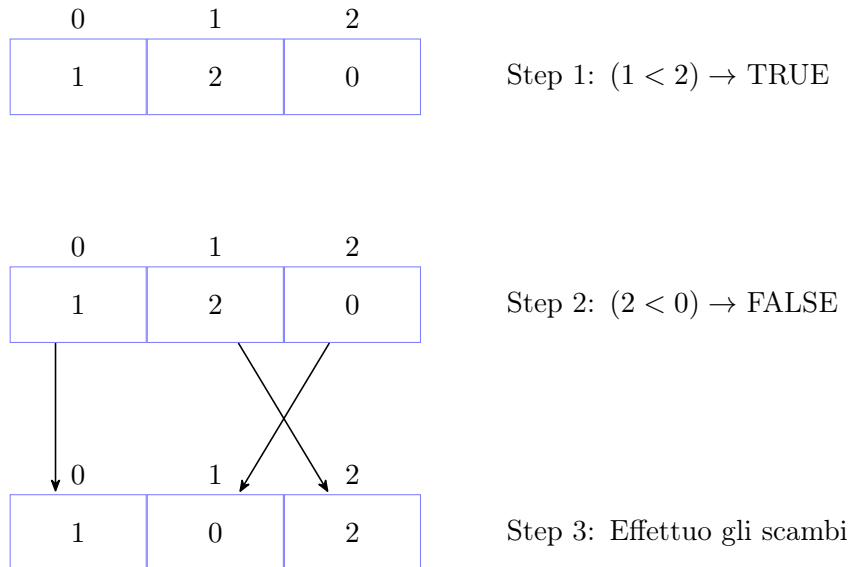


Figura 7.34: Esempio di albero di decisione con una foglia rimossa

Consideriamo il seguente array.

0	1	2
1	2	0

Proviamo ad ordinarlo seguendo l'albero di decisione della figura 7.33.



Si può notare facilmente come l'array risultante non sia ordinato.

Quindi, un albero di decisione è un albero binario, perché a ogni domanda avremo due possibili risposte (ripetiamo che stiamo assumendo assenza di duplicati per semplicità, altrimenti l'albero di decisione sarebbe un albero ternario), e con un numero di foglie pari ad $n!$, dove n è la dimensione dell'array da ordinare.

Ovviamente l'idea è che l'albero di decisione, oltre alla parte strutturale, quindi ad essere un albero binario con un numero pari a $n!$ di foglie, all'interno di ogni nodo vada a considerare un confronto tra dati.

Conseguentemente, se, come abbiamo detto, sebbene non andremo a formalizzarlo, ogni algoritmo di ordinamento che fa uso soltanto di confronti e scambi può essere tradotto in un albero di decisione, cioè in un albero binario con un numero di foglie pari a $n!$, allora dire che non è possibile avere un algoritmo che globalmente abbia una complessità migliore di $n \cdot \log(n)$, corrisponde a riportare questa proprietà in una corrispondente proprietà dell'albero di decisione.

In particolare, un algoritmo di ordinamento, visto come il corrispondente albero di decisione, analizza i confronti, ma sempre lungo un percorso.

L'albero di decisione è come se fosse l'intero programma, ma all'interno di un programma come noto possono essere percorsi dei branch e non altri.

Significa che a seconda dell'input il programma analizzerà solo una parte dei possibili percorsi che corrispondono esattamente ai percorsi dell'albero di decisione associato all'algoritmo.

Quindi, l'albero di decisione in qualche modo ci dà anche una stima sul numero di confronti e quindi asintoticamente sul numero di potenziali operazioni da fare basandoci sulla lunghezza del percorso che verrà eseguito.

Possiamo notare, guardando l'albero di decisione dell'esempio, che la lunghezza dei percorsi è in corrispondenza con l'analisi di complessità fatta per l'algoritmo di insertion sort.

Se infatti andassimo a guardare la famiglia di alberi di decisione dell'algoritmo di insertion sort, ipotizzando di mettere i rami corti a sinistra e i rami lunghi a destra, essa avrà una forma caratteristica con un ramo corto lineare in n ed un ramo lungo che è quadratico in n .

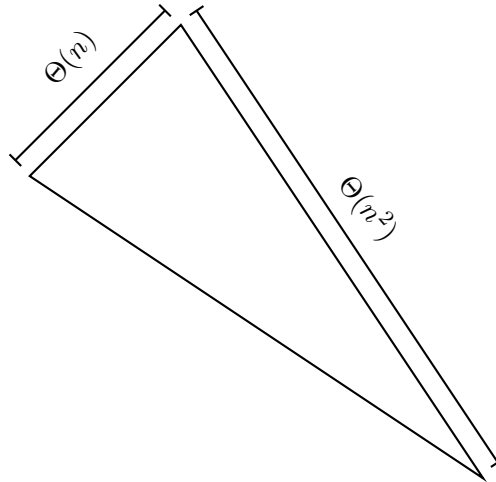


Figura 7.35: Esempio di forma generale di un albero di decisione per l'insertion sort

Quindi il numero di confronti che un algoritmo di ordinamento fa corrisponde esattamente alla lunghezza dei percorsi del suo albero di decisione, dato un certo input chiaramente.

A seconda dell'input verrà seguito un percorso diverso, ma verrà eseguito solo quel percorso.

Quindi, se nell'albero di decisione c'è un ramo corto, quello più corto sarà esattamente il caso migliore. Se c'è un ramo lungo, il più lungo è il caso peggiore.

Se tutti i percorsi dovessero essere più o meno della stessa lunghezza, significherebbe che l'albero sarebbe (più o meno) un albero pieno, e quindi, come sappiamo, si tratta di rami che sono asintoticamente tutti della stessa lunghezza. Quello è il caso degli algoritmi più efficienti possibile.

Conseguentemente, se ora dimostriamo che il migliore algoritmo di ordinamento, e quindi per trasposizione il migliore albero di decisione, non può avere un'altezza minore di $n \cdot \log(n)$, allora saremo sicuri che il caso peggiore di quell'algoritmo sarà almeno di $n \cdot \log(n)$.

Questo perché abbiamo detto che la complessità dell'algoritmo corrisponde alla lunghezza del percorso attraversato nel suo albero di decisione.

Se, quindi, riusciamo a dare un limite inferiore alla lunghezza massima dei percorsi, mostrando che la lunghezza del percorso massimo non può essere inferiore ad $n \cdot \log(n)$, allora avremo dimostrato che il migliore algoritmo possibile, anche quello che ancora non è stato ideato, non potrà fare meno che $n \cdot \log(n)$ confronti, e quindi non potrà avere asintoticamente una complessità minore di $\Theta(n \cdot \log(n))$, per ordinare un array nel caso peggiore.

Se, poi, fossimo in grado anche di stimare mediamente il percorso potremmo anche avere un limite inferiore alla complessità del caso medio. È proprio quello che faremo.

Vedremo che mediamente l'albero migliore possibile ha i percorsi che sono tutti più o meno della stessa lunghezza, ma saranno comunque di lunghezza $n \cdot \log(n)$.

Facciamo una prima osservazione.

Un ramo di un albero di decisione non può essere più corto di n , dove n è la dimensione dell'input; se così non fosse significherebbe che staremmo evitando di fare un confronto tra qualche coppia, e quindi non potrebbe essere un algoritmo di ordinamento corretto.

Quindi un ramo non può essere più corto di n e purtroppo quelli di lunghezza n sono davvero pochi. Nel migliore dei casi, mediamente i percorsi hanno lunghezza che è $n \cdot \log(n)$.

Facciamo un esempio grafico del miglior albero di decisione possibile.

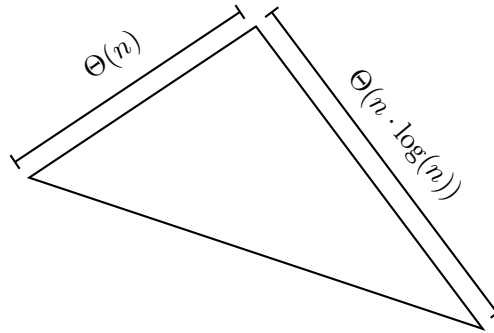


Figura 7.36: Esempio della forma del migliore albero di decisione

Vediamo come dimostrare la complessità del caso peggiore nel migliore albero di decisione possibile.

Ricordiamo che, fissato un certo numero di nodi, l'albero pieno è l'albero binario più basso possibile.

Sappiamo però che un albero pieno non ha possibilità di rappresentare insiemi di dati con cardinalità arbitraria; può rappresentare soltanto insiemi di cardinalità pari a $2^k - 1$, per un certo k .

Ovviamente il migliore albero di decisione possibile, come in generale un qualunque albero, non è detto che sia pieno.

Però, se prendiamo l'albero pieno più piccolo possibile che include l'albero di decisione migliore riusciamo ad ottenere una buona stima dell'altezza.

Infatti prendendo il più piccolo albero pieno che contiene un secondo albero, sebbene sicuramente nell'albero non pieno possono esistere rami più corti di quelli dell'albero pieno, possiamo essere certi che esisterà almeno un ramo dell'albero non pieno avente lunghezza pari ai rami dell'albero pieno.

Quindi, se stimiamo qual è il più piccolo albero pieno che include un albero di decisione possiamo ottenere la stima corretta dell'altezza dell'albero di decisione stesso, in quanto sarà identica all'altezza dell'albero pieno stimato.

Sappiamo, dalla definizione di albero di decisione, che l'albero di decisione è un albero binario con un numero di foglie pari ad $n!$.

Un albero pieno ha, invece, un numero di foglie pari 2^h , dove h è l'altezza dell'albero.

Quindi, sicuramente, non possiamo prendere un albero pieno con un numero di foglie minore al numero di foglie dell'albero di decisione.

$$n! \leq 2^h \quad (7.31)$$

Né tantomeno ci serve prendere qualcosa di troppo più grande.

Dobbiamo, quindi trovare, il valore minimo dell'altezza, h , dell'albero pieno che verifichi la disuguaglianza precedente.

$$n! \leq 2^h \iff \log_2(n!) \leq h$$

Andiamo a svolgere il logaritmo, ricordandoci che il logaritmo di un prodotto è uguale alla somma dei logaritmi.

$$\log_2(n!) = \log_2 \left(\prod_{i=1}^n i \right) = \sum_{i=1}^n \log_2(i)$$

Calcolare il valore preciso di una sommatoria di logaritmi non è semplice. A livello asintotico però ci basta usare un'approssimazione sufficientemente precisa per i nostri intenti.

In particolare notiamo.

$$\sum_{i=\frac{n}{2}}^n \log_2(i) \leq \sum_{i=1}^n \log_2(i) \leq \sum_{i=1}^n \log_2(n)$$

Noi vogliamo il minimo h per cui valga $h \geq \sum_{i=1}^n \log_2(i)$, quindi prendiamo $h = \sum_{i=1}^n \log_2(i)$.

Osserviamo che h dovrebbe essere un intero perché l'altezza è intera, ma se riusciamo ad ottenere un'approssimazione sufficientemente precisa di $\sum_{i=1}^n \log_2(i)$ prenderemo poi l'intero superiore poiché a livello asintotico non cambia nulla.

Quindi quello che vogliamo è ottenere un h tale che

$$\sum_{i=1}^n \log_2(i) \leq h \leq \sum_{i=1}^n \log_2(i)$$

Adesso, come abbiamo visto precedentemente, approssimeremo per eccesso e per difetto $\sum_{i=1}^n \log_2(i)$.

Vediamo l'approssimazione per accesso

$$\sum_{i=1}^n \log_2(i) \leq \sum_{i=1}^n \log_2(n) = \log_2(n) \cdot \sum_{i=1}^n 1 = n \cdot \log_2(n)$$

Vediamo l'approssimazione per difetto

$$\sum_{i=1}^n \log_2(i) = \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor - 1} \log_2(i) + \sum_{i=\lfloor \frac{n}{2} \rfloor}^n \log_2(i) \implies \sum_{i=\lfloor \frac{n}{2} \rfloor}^n \log_2(i) \leq \sum_{i=1}^n \log_2(i)$$

In particolare notiamo

$$\sum_{i=\lfloor \frac{n}{2} \rfloor}^n \log_2 \left(\left\lfloor \frac{n}{2} \right\rfloor \right) \leq \sum_{i=\lfloor \frac{n}{2} \rfloor}^n \log_2(i)$$

Ciò vale perché la sommatoria con $\log_2(i)$ sommerà logaritmi con $i \geq \lfloor \frac{n}{2} \rfloor$ e quindi necessariamente deve essere maggiore di una sommatoria che somma ad ogni i il termine costante $\log_2 \left(\left\lfloor \frac{n}{2} \right\rfloor \right)$

Applichiamo le dovute semplificazioni, eliminando il floor perché ricordiamo che asintoticamente non dà nessun contributo

$$\sum_{i=\frac{n}{2}}^n \log_2 \left(\frac{n}{2} \right) = \log_2 \left(\frac{n}{2} \right) \cdot \sum_{i=\frac{n}{2}}^n 1 = (\log_2(n) - \log_2(2)) \cdot \frac{n}{2} = \frac{n}{2} \log_2(n) - \frac{n}{2}$$

Per il teorema della complessità asintotica precedentemente dimostrato, troveremo la complessità di h

$$\lim_{n \rightarrow \infty} \frac{\frac{n}{2} \cdot (\log_2(n) - \log_2(2))}{n \cdot \log_2(n)} = \frac{1}{2} \implies h = \Theta(n \cdot \log(n))$$

Quindi, dato il calcolo delle approssimazioni per eccesso e per difetto, segue chiaramente

$$h = \Theta(n \cdot \log(n))$$

Abbiamo dimostrato, quindi, che l'altezza del minimo albero pieno contenente un albero di decisione è $\Theta(n \cdot \log(n))$.

Di conseguenza, possiamo dire che l'altezza del migliore albero di decisione è effettivamente $n \cdot \log(n)$ e quindi questo ci dimostra che nel caso peggiore il miglior algoritmo di ordinamento non può avere una complessità inferiore a $\Theta(n \cdot \log(n))$ perché questa è esattamente la lunghezza del percorso massimo dell'albero di decisione migliore, e quindi $\Theta(n \cdot \log(n))$ è la migliore complessità nel caso peggiore.

Invece, quello che dobbiamo fare per andare a lavorare sul caso medio è cercare di fare una media della lunghezza dei percorsi, cioè prendere la lunghezza di tutti i percorsi che stanno nell'albero, sommarli, e dividere per quante foglie ci sono.

Se questa stima, nel caso del miglior albero di decisione, sarà $n \cdot \log(n)$, avremo dimostrato che il miglior algoritmo di ordinamento, nel caso medio, non potrà che essere almeno $\Theta(n \cdot \log(n))$.

Introduciamo il concetto di percorso esterno di un albero.

Il percorso esterno di un albero è la somma di tutti i percorsi dell'albero.

$$PE(T, n) \tag{7.32}$$

Chiaramente, come percorso intendiamo un percorso che dalla radice arriva in foglia.

Il valore del percorso esterno di un albero T con un numero di nodi n dipende, ovviamente, dalla struttura dell'albero.

Più l'albero è sbilanciato più sarà lungo il percorso esterno; più l'albero è bilanciato più il percorso esterno sarà basso.

Andiamo a calcolare il tempo medio facendo, come detto, la media della lunghezza dei percorsi dell'albero.

$$T_{AVG}(T, n) = \frac{PE(T, n)}{n!}$$

Per quanto detto sul percorso esterno di un albero risulta immediato che, a parità di nodi, più un albero sarà sbilanciato più alto sarà il valore di questa stima, più un albero sarà bilanciato più basso sarà il valore di questa stima.

Vediamo, quindi, quali sono gli alberi, fissato un numero di foglie, con il minimo valore del percorso esterno. Anticipiamo che l'albero migliore da questo punto di vista è l'albero completo.

Ricordiamo che un albero completo è un albero binario che deve avere tutti i nodi interni, tranne al più 1, di grado 2 e le foglie sono tutte quante poste ad altezza h , che è l'altezza dell'albero, o ad altezza $h - 1$.

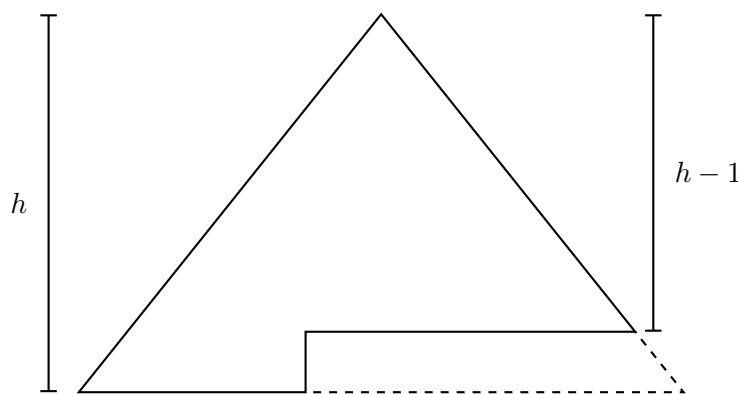
Affinché, però, un albero completo sia un albero di decisione non può esistere nessun nodo interno di grado 1, cioè ogni nodo interno deve avere necessariamente 2 figli.

Significa che gli alberi completi che possono essere alberi di decisione sono una particolare sottoclasse in cui tutti i nodi interni hanno due figli.

Andiamo, adesso, a prendere in considerazione un generico albero completo con un certo valore del percorso esterno e vediamo se spostando qualche nodo, cioè rendendolo non completo, se riusciamo in qualche modo a migliorare o peggiorare la situazione.

Se dimostriamo che comunque andiamo a modificare le cose possiamo peggiorare la situazione, cioè il valore del percorso esterno andrà ad aumentare, significherà che l'albero completo è quello con il minimo valore del percorso esterno.

Prendiamo, quindi, in considerazione un generico albero completo.



Modifichiamo l'albero completo.

Siamo costretti a spostare due nodi alla volta per fare in modo di mantenere la proprietà di essere albero di decisione.

Spostiamo, quindi, due foglie che si trovano ad altezza $h - 1$ e poniamole ad altezza $h + 1$. È l'unico modo nel quale possiamo effettivamente modificare l'albero.

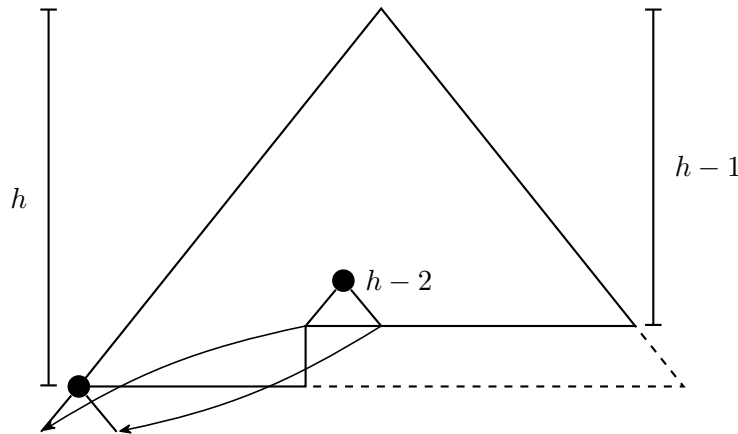


Figura 7.37: Esempio di spostamento di nodi foglie nell'albero completo

Vediamo cosa accade al valore del percorso esterno.

Dobbiamo eliminare il vecchio valore dei percorsi che fanno capo alle due foglie spostate (lunghezza $h - 1$), aggiungere il percorso che fa capo alla nuova foglia generata, cioè il vecchio padre delle foglie spostate (lunghezza $h - 2$), eliminare il valore del percorso del nodo foglia cui abbiamo collegato i due nodi spostati (lunghezza h), e aggiungere il valore dei nuovi percorsi delle due foglie spostate (lunghezza $h + 1$).

$$PE(T, n) - 2(h - 1) + (h - 2) - h + 2(h + 1) = PE(T, n) + 2$$

Quindi il percorso esterno è peggiorato.

Significa che un algoritmo il cui albero di decisione si allontana dall'albero completo è mediamente peggiore.

Quindi, nell'analisi del caso medio del migliore algoritmo di ordinamento, andremo a prendere come classe di riferimento gli alberi completi, e nello specifico, ovviamente, la sottoclasse di alberi completi che è anche albero di decisione.

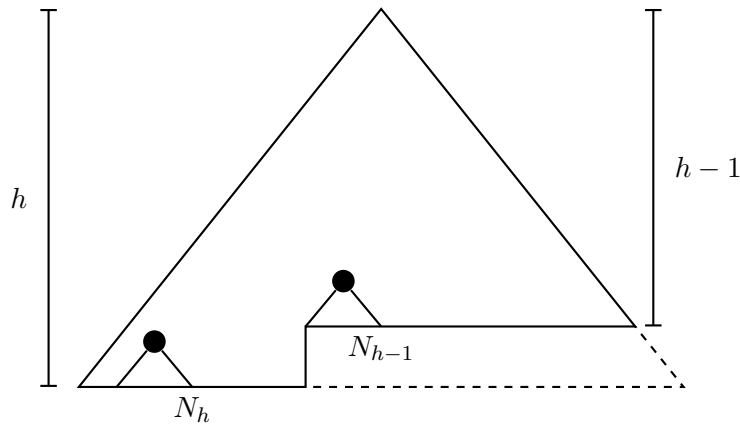


Figura 7.38: Miglior albero di decisione per il caso medio

Fortunatamente calcolare il percorso esterno per un albero completo è semplice perché possiamo sapere facilmente quante sono le foglie ad altezza h e quante sono le foglie ad altezza $h - 1$; dove h è l'altezza dell'albero.

Vediamo come calcolare quante foglie di altezza h e quante di altezza $h - 1$ ci sono in un albero completo.

Un albero completo ha evidentemente un albero pieno che include ed un albero pieno che lo include. E di questi alberi pieni sappiamo calcolare il numero di foglie data l'altezza (2^h).

Se a ciascuna foglia ad altezza $h - 1$ andiamo ad aggiungere 2 foglie otteniamo proprio il più piccolo albero pieno che contiene l'albero completo.

Sappiamo, inoltre, che il numero di foglie dell'albero completo di decisione deve essere $n!$.

Abbiamo quindi due vincoli in due incognite. Possiamo costruire un semplice sistema la cui soluzione ci darà le informazioni cercate.

$$\begin{cases} N_h + 2N_{h-1} &= 2^h \\ N_h + N_{h-1} &= n! \end{cases} \quad (7.33)$$

Svolgendo il sistema.

Effettuiamo una sottrazione membro a membro tra le due equazioni.

$$\begin{aligned} N_{h-1} &= 2^h - n! \\ N_h &= n! - N_{h-1} = n! - 2^h + n! = 2n! - 2^h \end{aligned}$$

A questo punto il calcolo del percorso esterno segue direttamente.

$$PE(T, n) = (h - 1)(2^h - n!) + h(2n! - 2^h) = h \cdot 2^h - h \cdot n! - 2^h + n! + 2h \cdot n! - h \cdot 2^h = (h + 1)n! - 2^h$$

A questo punto, effettuando le dovute sostituzioni, possiamo calcolare la complessità del caso medio.

$$T_{AVG}(T, n) = \frac{PE(T, n)}{n!} = \frac{(h+1)n! - 2^h}{n!} = (h+1) - \frac{2^h}{n!} \leq h+1 \approx h = \Theta(n \cdot \log(n))$$

Abbiamo quindi dimostrato che il miglior algoritmo di ordinamento che si basa solo su confronti e scambi non può avere, nel caso medio, una complessità migliore di $\Theta(n \cdot \log(n))$.

"I know kung-fu"

– *Neo (The Matrix)*